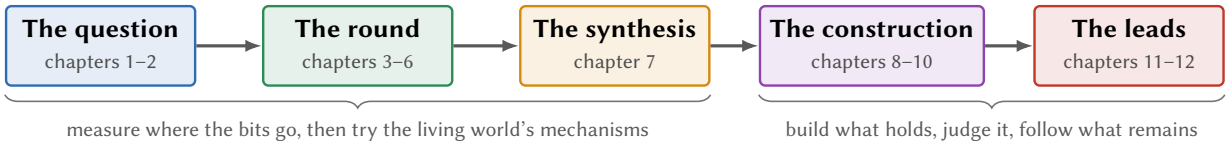
troncons

Volume III: the genome of the weights

Where the noise is and where the structure is; the strand, the seed and the rotation; gene duplication, predictive coding, mimicry; the synonymous codon in the format; the tropical geometry of the router and the spectra of the matrices

Research thesis — the search, its round and its construction



Stone

Toronto, 29 October 2026

Follows Volume I, *the measured foundations*, and Volume II, *the system*. Every figure is regenerated from the measurement data committed in the `troncons` repository; every number cites the file it comes from.

Abstract

Volume I of *troncons* showed that, on an ordinary laptop, serving a model much larger than its memory comes down to the bits read from disk per weight; Volume II built the program that does it, exactly. One frontier remained: 3.84 bits per expert weight lossless, 2.64 bits for the lossy base of the fast mode. Stone proposed to learn from life, which does not store an organism but a genome from which the organism is built. This volume asks the question in a form that can be judged: is there a mechanism inspired by life that reads fewer bits per weight than this frontier, at a fidelity judged by the KL on text against the exact reference, and that transcribes faster than the disk reads what it replaces? The genome is an analogy that guides the search here, never an argument.

The volume gives the research round, the construction and the leads. The round: the question in the terms of the machine (the MXFP4 block to the bit, a rate–distortion bound in the domain of the weights, twelve "genomic" families from the literature); the scout, which found where the noise and the structure are; the recipe read through codons and its seed; gene duplication; predictive coding and pruning; six leads of mimicry, from a two-scale cache to annealing; and the synthesis, which places every mechanism on a common frontier and shows that the error of an expert does not predict the KL. The construction: the zero-synonym codon in an exact `.trc` format v1.2, the synonymous genome served by a fast mode of the engine, and a judge that did neither. The leads: the tropical geometry of the router, and the spectra of the matrices. Each chapter sets out its derivations in full and ends with what could be wrong.

The first answer is harsh: once the scale of their block is known, the weights of gpt-oss behave like noise, and the seed (3.06 bits at the error of the base), the shared patterns (2.90), the lossless recipe (4.05 to 4.37 bits against 3.84) and the kinship between experts (0.0034 bit, of which 0.0016 by chance) all fall. What wins, three times, is the synonym, which changes the code without changing what the computation reads: the sign of a zero (entropy 0.9999999967 bit, never read by the computation) is folded losslessly, 0.12791 and 0.12562 bit per expert weight less, 1.80 GB less to read on the 120b, bit-identical logits; the column context gives −0.142 and −0.123 bit at layer 0; the synonymous mutation, which puts the error where the inputs do not look, serves the 768 experts of the 20b at 2.67 bits instead of 3.86, for a KL of 0.1318 against 0.2640 for the fixed base. On this disk, the format v1.2 gives the J1 milestone of the 120b a ×1.44 for 3.3 % fewer bytes: the gain comes from the pattern of its read requests, specific to this disk. Several numbers published along the way are struck out where they stood: "it is the state of the files, not the format"; two gains of the fast mode, ×1.24 and ×2.00, which compared a fresh file with an old one and become ×0.94 and ×1.28 against v1.2; "S^* has 33 % more KL", equal to the base on ten windows (+0.0004 ± 0.0019). The tropical guard falls (57.1 % of the flips for 10 % of the cells, where 80 % would cost 26.6 %), the power law of the spectra holds for no matrix out of 48 under the main criterion, and the six leads of mimicry remain open (the prefetch with calibrated confidence rather on the side of falling), or borderline for the cache-conditioned routing.

The judge of the volume is the Kullback–Leibler divergence of the next-token law against the exact double-precision reference, averaged over the 2550 positions of the ten evaluation windows, each comparison paired position by position. An independent judge of the construction, with its own decoder written from the description of the format alone, recovered 96 drawn experts to the bit, up to the sign of the zeros, bit-identical logits on ten windows nobody had seen (0 differences in 5120

positions, for each of the two models), and the synonymous mutation better than the frontier outside the tuning windows (-0.1565 ± 0.0334, 4.7 standard deviations); the retained variant, $\lambda = 0$, wins by 2.09 standard deviations on the evaluation windows and by 3.6 outside them. Every verdict has its threshold declared beforehand, its control that can fail, and a judge that did not do the work.

What remains: making v1.2 the default format of the machine's files (Stone's decision) and isolating the variable of the read pattern that makes the speed of this disk; the tail of the KL, made of the flips of the router, which nothing targets yet; the synonymous mutation of the 120b, which waits for its f64 reference; and a corpus other than wiki.test. The answer to Stone's intuition is twofold, and measured: DNA as a recipe does not work on these weights; DNA as a redundant code, where a synonym changes the sequence without changing the phenotype, is exactly what wins. Every figure is drawn from committed data by a committed script.

Résumé

Le Tome I de *troncons* avait démontré que, sur un portable ordinaire, servir un modèle bien plus gros que sa mémoire se joue en bits lus sur le disque par poids ; le Tome II avait construit le programme qui le fait, exactement. Il restait une frontière : 3,84 bits par poids d'expert sans perte, 2,64 bits pour la base avec perte du mode rapide. Stone a proposé de s'inspirer du vivant, qui ne stocke pas un organisme mais un génome d'où l'organisme se construit. Ce tome pose la question jugeable : existe-t-il un mécanisme inspiré du vivant qui fasse lire moins de bits par poids que cette frontière, à une fidélité jugée par la KL sur du texte contre la référence exacte, et qui se transcrive plus vite que le disque ne lit ce qu'il remplace ? Le génome y est une analogie qui oriente la recherche, jamais un argument.

Le tome donne la ronde de recherche, la construction et les pistes. La ronde : la question dans les termes de la machine (le bloc MXFP4 au bit près, une borne distorsion-débit dans le domaine des poids, douze familles « génomiques » de la littérature) ; l'éclaireur, qui a trouvé où sont le bruit et la structure ; la recette lue par codons et sa graine ; la duplication de gènes ; le codage prédictif et l'élagage ; six pistes de mimétisme, du cache à deux échelles au recuit ; et la synthèse, qui place tous les mécanismes sur une frontière commune et montre que l'erreur d'un expert ne prédit pas la KL. La construction : le codon synonyme du zéro dans un format `.trc` v1.2 exact, le génome synonyme servi par un mode rapide du moteur, et un juge qui n'a fait ni l'un ni l'autre. Les pistes : la géométrie tropicale du routeur et les spectres des matrices. Chaque chapitre pose ses dérivations en entier et finit par ce qui pourrait être faux.

La première réponse est sévère : une fois l'échelle de leur bloc connue, les poids de gpt-oss se comportent comme du bruit, et la graine (3,06 bits à l'erreur de la base), les motifs partagés (2,90), la recette sans perte (4,05 à 4,37 bits contre 3,84) et la parenté entre experts (0,003 4 bit, dont 0,001 6 au hasard) tombent tous. Ce qui gagne, trois fois, est le synonyme, qui change le code sans changer ce que le calcul lit : le signe d'un zéro (entropie 0,999 999 996 7 bit, jamais lu par le calcul) se fond sans perte, 0,127 91 et 0,125 62 bit par poids d'expert de moins, 1,80 Go de moins à lire sur le 120b, des logits identiques au bit ; le contexte de la colonne rend −0,142 et −0,123 bit à la couche 0 ; la mutation synonyme, qui place l'erreur là où les entrées ne regardent pas, sert les 768 experts du 20b à 2,67 bits au lieu de 3,86, pour une KL de 0,131 8 contre 0,264 0 à la base fixe. Sur ce disque, le format v1.2 fait gagner ×1,44 au jalon J1 du 120b pour 3,3 % d'octets de moins : le gain vient du motif de ses requêtes de lecture, propre à ce disque. Plusieurs chiffres publiés en route sont barrés à leur place : « c'est l'état des fichiers, pas le format » ; deux gains du mode rapide, ×1,24 et ×2,00, qui comparaient un fichier neuf à un ancien et deviennent ×0,94 et ×1,28 contre le v1.2 ; « S^* a 33 % de KL de plus », égal à la base sur dix fenêtres (+0,000 4 ± 0,001 9). La garde tropicale tombe (57,1 % des bascules pour 10 % des cellules, quand 80 % en coûteraient 26,6 %), la loi de puissance des spectres ne s'écrit pour aucune matrice sur 48 au critère principal, et les six pistes du mimétisme restent ouvertes (le préchargement à confiance calibrée plutôt du côté de tombe), ou à la limite pour le routage conditionné par le cache.

Le juge du tome est la divergence de Kullback-Leibler de la loi du jeton suivant contre la référence exacte en double précision, moyennée sur les 2 550 positions des dix fenêtres d'évaluation, chaque comparaison appariée position par position. Un juge indépendant de la construction, avec son propre décodeur écrit depuis la seule description du format, a retrouvé au bit, au signe des zéros près, 96 experts tirés, des logits identiques au bit sur dix fenêtres que personne n'avait vues (0 différence sur

5 120 positions, pour chacun des deux modèles), et la mutation synonyme meilleure que la frontière hors des fenêtres de mise au point ($-0{,}156\,5 \pm 0{,}033\,4$, 4,7 écarts-types) ; la variante retenue, $\lambda = 0$, l'est à 2,09 écarts-types sur les fenêtres d'évaluation et 3,6 hors d'elles. Chaque verdict a son seuil déclaré avant, son témoin qui sait tomber, et un juge qui n'a pas fait le travail.

Restent : faire du v1.2 le format par défaut des fichiers de la machine (décision de Stone) et isoler la variable du motif de lecture qui fait la vitesse de ce disque ; la queue de la KL, faite des bascules du routeur, que rien ne vise encore ; la mutation synonyme du 120b, qui attend sa référence f64 ; et un autre corpus que wiki.test. La réponse à l'intuition de Stone est double, et mesurée : l'ADN comme recette ne marche pas sur ces poids ; l'ADN comme code redondant, où un synonyme change la séquence sans changer le phénotype, est exactement ce qui gagne. Chaque figure se tire de données commitées par un script commité.

Contents

Abstract iii

Résumé v

List of Figures xv

List of Tables xvii

Listings xxi

Notation and conventions xxiii

1 Introduction 1
- 1.1 The question, asked by Stone and by the machine . . . 1
- 1.2 What "genome" means here . . . 2
- 1.3 The path of the volume . . . 3
- 1.4 What this volume contributes . . . 4
- 1.5 How to read this volume . . . 4

I The question and the round 7

2 The question of the genome: the roof, the bits read per weight, the frontier 9
- 2.1 The roof: what a token must read . . . 10
- 2.2 The MXFP4 block to the bit . . . 10
- 2.3 The frontier, and the rate–distortion bound . . . 12
  - 2.3.1 What a weight costs today . . . 12
  - 2.3.2 The bound of a memoryless coder, in the domain of the weights . . . 12
- 2.4 The state of the art, in twelve families . . . 15
  - 2.4.1 Two filters . . . 15
  - 2.4.2 The families . . . 16
- 2.5 What has never been tried on a model born in 4 bits . . . 16
- 2.6 What the reading predicts . . . 18

3 The scout: where the noise is, where the structure is 21
- 3.1 The instruments and their controls . . . 22
- 3.2 Where the noise is, where the structure is . . . 23
  - 3.2.1 The sign of the zeros: pure noise . . . 23
  - 3.2.2 The energy is concentrated . . . 23
  - 3.2.3 The bound of a memoryless coder . . . 23
- 3.3 Mechanism 1: the zero-synonym codon . . . 25
  - 3.3.1 The mechanism . . . 25

3.3.2 Why the products do not change . . . 25
3.3.3 The measurements . . . 25
3.3.4 The same codon in the base . . . 26
3.4 Mechanism 2: the synonymous mutation . . . 27
3.4.1 Life, and what is transposed from it . . . 27
3.4.2 The derivation . . . 27
3.4.3 The protocol . . . 28
3.4.4 The first failure, and its reason: overfitting . . . 28
3.4.5 The results . . . 29
3.5 The end-to-end judgement: the KL against the f64 reference . . . 31
3.6 The verdict . . . 33

4 The genome of the weights: the recipe, the seed, S^* 35
4.1 The question and its judge . . . 36
4.2 The seed: the literal recipe, and why it cannot suffice . . . 36
4.2.1 The mechanism . . . 36
4.2.2 What theory allows, before any measurement . . . 36
4.2.3 The measurement . . . 37
4.3 The seed that grows: the strand read through codons . . . 38
4.3.1 The idea . . . 38
4.3.2 The best strand: Viterbi, and why it is exact . . . 38
4.3.3 The rotation: spreading the information of each weight . . . 39
4.3.4 Distributing the bits between rows . . . 41
4.4 The accounting, to the bit . . . 42
4.5 The results on the real weights . . . 42
4.6 Where the noise is, where the structure is . . . 44
4.7 The recipe and the base + complement split . . . 45
4.8 The output of the experts: fidelity, and S^* . . . 46
4.9 The KL against the reference: S^* is equal to the base . . . 47
4.10 The transcription . . . 50
4.11 What remains . . . 50

5 Gene duplication 53
5.1 The question, posed in bits . . . 54
5.2 The symmetry that makes kinship invisible . . . 54
5.3 Raw kinship: screen, matching, family . . . 54
5.3.1 Two lemmas for reading a cosine . . . 54
5.3.2 The screen: an upper bound, without bijection . . . 55
5.3.3 One-to-one matching: the true kinship from expert to expert . . . 55
5.3.4 The best parent of each child . . . 55
5.3.5 The family: a common subspace . . . 57
5.4 The exact cost of an expert coded as a mutation . . . 57
5.4.1 The statements . . . 57
5.4.2 The measurements . . . 58
5.5 The truncated mutation and the output of the expert . . . 60
5.6 The most economical tree . . . 61
5.7 Paralogues: serving an absent expert by another . . . 63

5.8 Why the idea fails, and where to look . . . 64

6 Predictive coding and synaptic pruning 67
6.1 The synonymous codons of E2M1 . . . 68
6.2 Lossless predictive coding: what the neighbours say about a code . . . 69
6.3 Pruning and bounded predictive quantisation: the rate–distortion frame . . . 70
6.4 Activity-dependent pruning . . . 72
6.5 The real inputs, and the error at the output of the experts . . . 72
6.5.1 Hebbian pruning: what holds, what overfits . . . 73
6.6 The KL against the f64 reference . . . 74
6.7 What remains . . . 75

7 Mimicry: six leads from the world to the pipeline 77
7.1 The sorting: forty-five candidates, six leads . . . 78
7.2 P1: the two-scale cache memory . . . 79
7.2.1 The exponentially decaying score, a rate estimator . . . 80
7.2.2 The protocol and the mechanical verdict . . . 81
7.2.3 What the generation traces really were . . . 82
7.2.4 The cold windows: frequency, not half-life . . . 83
7.3 P2: the exclusive cache from VRAM to RAM . . . 84
7.3.1 The LRU stack and the theorem . . . 84
7.3.2 The cost of demotion . . . 87
7.3.3 The measurement, the verdict and what the judge corrects . . . 88
7.4 P3: cache-conditioned routing, bounded by the margin . . . 90
7.4.1 The geometry: a shifted wall, a band . . . 90
7.4.2 The decision criterion in one quantity . . . 92
7.4.3 Stage A: the open loop . . . 92
7.4.4 Stage B: the excess KL measured . . . 93
7.4.5 The online biased router, in the engine . . . 94
7.5 P4: prefetching with calibrated confidence . . . 96
7.5.1 The guarantee, set out . . . 96
7.5.2 The simulator and the gain . . . 96
7.5.3 The measurement and the verdict . . . 97
7.6 P5: the choice of the synonym symbols as a spin glass . . . 98
7.6.1 The problem set out . . . 98
7.6.2 The controls: the glass is real at small dimension . . . 100
7.6.3 Step 1: the gain can be measured and does not transfer . . . 100
7.6.4 The announced mechanism, and what the judge says of it . . . 103
7.7 P6: fidelity decided by call . . . 104
7.7.1 The model and its quantities . . . 104
7.7.2 The measurement, the verdict and what the judge corrects . . . 105
7.8 The map from the world to the pipeline . . . 107
7.9 What could be wrong . . . 107

8 The synthesis of the round: a common frontier, two parts of the KL 111
8.1 A single judge, and why ratios . . . 112
8.2 The common frontier . . . 113

8.3 Three measured properties of the weights . . . 116
8.3.1 Once the block scale is known, the weights are noise . . . 116
8.3.2 A few giant columns carry the structure . . . 116
8.3.3 The output error of an expert does not predict the KL . . . 117
8.4 Two parts of the KL . . . 120
8.4.1 What the quantiles say, and the engine . . . 120
8.4.2 The smooth part: the KL at second order . . . 121
8.4.3 The flip part . . . 122
8.4.4 The router as tropical geometry . . . 122
8.5 Mechanism by mechanism . . . 126
8.5.1 Why S^* does not win . . . 126
8.5.2 The lesson, in one sentence per level . . . 126
8.6 The context of a column symbol . . . 126
8.7 The construction specification . . . 128
8.7.1 Lot A: what it carries, and the proof that it changes no bit . . . 129
8.7.2 Lot B: the synonymous genome, and the measurement that was missing at first 131
8.7.3 The leads that the measurements already point to . . . 132
8.8 What could be wrong . . . 133

II The construction 135

9 A synonymous codon in the format (lot A, v1.2) 137
9.1 Sixteen codes for fifteen values . . . 138
9.2 The exact gain of the fold . . . 138
9.3 Who reads code 8, and why the bits do not change . . . 141
9.4 The `.trc` v1.2 format to the bit . . . 144
9.4.1 Why the column stays out . . . 146
9.5 What was written, and its controls . . . 148
9.6 The two recoded files . . . 149
9.6.1 The codon through the judge's decoder . . . 151
9.7 Milestone J1 redone on the 120b . . . 152
9.7.1 Where the gain comes from: the pattern of read requests . . . 153
9.8 What remains . . . 156
9.9 What could be wrong . . . 156

10 The synonymous genome, fast mode (lot B) 159
10.1 What is built . . . 160
10.2 The mechanism . . . 160
10.2.1 The output error and the input matrix . . . 160
10.2.2 The choice of the symbols . . . 161
10.2.3 The transformation in the eigenbasis of H . . . 162
10.2.4 The error in the directions of the inputs . . . 163
10.3 The fast mode does not touch the exact mode . . . 167
10.4 The companion `<nom>.rapide.trc`, to the bit . . . 167
10.5 The quality measurements . . . 168
10.5.1 The protocol and its controls . . . 168
10.5.2 The benchmark of the reference tool . . . 169

10.5.3 The end-to-end KL, the 768 experts substituted . . . 169
10.5.4 Criterion B2, and the settling of the two λ . . . 170
10.5.5 The tail of the KL . . . 172
10.5.6 Perplexity does not judge fidelity . . . 174
10.5.7 The rate of the symbols . . . 174
10.5.8 The frontier of the variants . . . 174
10.6 The fast mode in the engine . . . 176
10.7 Speed, and what the judge struck through . . . 176
10.7.1 The lot's sessions . . . 176
10.7.2 An identity, and what it separates . . . 177
10.7.3 What the judge measured . . . 178
10.8 The 120b . . . 179
10.9 What remains . . . 179
10.10 What could be wrong . . . 180

11 The judge of the construction **181**
11.1 What the judge redid, and by which path . . . 182
11.2 The `.trc` v1.2 through the judge's decoder . . . 183
11.2.1 Vectorised rANS decoding is normative decoding . . . 183
11.2.2 J-T0: the four whole files . . . 183
11.2.3 J-T1: 96 drawn experts, decoded by the judge . . . 184
11.2.4 What the v1.0 → v1.2 control proves . . . 184
11.3 The new windows, and the experts they call . . . 185
11.4 J-C1: the logits to the bit on the new windows . . . 185
11.5 J-K: the KL of the synonymous genome outside R1 . . . 187
11.5.1 The judge's KL, and its control . . . 187
11.5.2 The numbers . . . 187
11.5.3 The uncertainty of a mean KL: by positions or by windows . . . 188
11.5.4 Redoing a result at two standard deviations . . . 190
11.6 J-V: speed, and what it separates . . . 190
11.6.1 The trap . . . 190
11.6.2 The three-file session . . . 190
11.6.3 The 120b . . . 191
11.6.4 The 20b . . . 192
11.6.5 What makes the disk stage: reserve R1 . . . 192
11.7 The reading: what is false, too good, or wrongly attributed . . . 193
11.8 What holds according to the judge, and what is struck through . . . 193
11.9 What the judge's method teaches . . . 194
11.10 What remains . . . 195
11.11 What could be wrong . . . 195

III The leads and the assessment **197**

12 The tropical geometry of the router **199**
12.1 The question and the declared threshold . . . 199
12.2 The router as a tropical polynomial . . . 200
12.3 The data and the controls . . . 201

12.4 The verdict: the curve of the guard . . . 202
12.5 Why it falls . . . 203
12.5.1 The law of the margin: almost exponential . . . 203
12.5.2 The flip law . . . 206
12.5.3 Propagation: one flip calls for others . . . 206
12.5.4 The guard number: a closed form . . . 207
12.5.5 The extrapolation to the fast mode (model) . . . 209
12.6 After the verdict: two explorations . . . 210
12.6.1 The guard of the roots . . . 210
12.6.2 The memory of conflict: a multi-cue guard . . . 210
12.7 What could be wrong . . . 212

13 The spectra of the matrices and the fractal 213
13.1 The question and the quantity . . . 214
13.2 The control of chance: Marchenko-Pastur . . . 214
13.3 The instrument: Clauset, Shalizi and Newman, step by step . . . 214
13.3.1 The declared threshold, and why it was amended before the reading . . . 215
13.3.2 The declared criterion can never say yes: the judge's theorem . . . 216
13.3.3 The controls, the break, the second path . . . 216
13.4 The verdict: not a power law . . . 217
13.4.1 The exponents, by role and by layer . . . 219
13.5 The lever and the ceiling: what is not judged . . . 220
13.6 What could be wrong . . . 222

14 Discussion 223
14.1 What the volume establishes, together . . . 223
14.2 The struck-through figures . . . 224
14.3 The lesson of the quest . . . 224
14.4 What could be wrong, at the scale of the volume . . . 225
14.5 Future work . . . 226

A Protocols: declared thresholds and controls 229
A.1 What is inherited . . . 229
A.2 The declared thresholds, step by step . . . 230
A.3 The controls and their breaks . . . 232
A.4 The amendments . . . 233
A.5 The protocols of the judges . . . 234
A.6 What could be wrong . . . 234

B Tools 237
B.1 The chain of Volume III . . . 237
B.2 The committed tools . . . 238
B.3 Each figure and the files it reads . . . 255
B.4 The licences . . . 258

C Logs 261
C.1 The log folders . . . 261
C.2 The sessions that carry a speed figure . . . 263

C.3 The passes cut, discarded, or inconclusive 263

Bibliography **265**

List of Figures

2.1 The MXFP4 block, bit by bit . . . 11
2.2 The frontier base against the distortion–rate bound . . . 14

3.1 The sign of the zeros, tensor by tensor . . . 24

4.1 The strand read by codons, on a true Gaussian . . . 40
4.2 Rate–distortion of the strand, tensor by tensor . . . 43

5.1 The kinship score of every candidate parent . . . 56
5.2 The most economical tree of the experts of layer 23 . . . 62

7.1 P1: hit rate per block of the two generations of the 120b . . . 82
7.2 P1: cold windows, hit rate of the LRU against the policy . . . 84
7.3 P1: the latent space of the blocks of expert usages . . . 85
7.4 P2: the stack distances and the band $(V, V + R]$ . . . 87
7.5 P2: disk share against the size of the RAM . . . 89
7.6 P3: the wall of the router shifted by β . . . 91
7.7 P3, stage A: misses avoided against cells changed . . . 93
7.8 P4: precision, coverage and simulated gain . . . 98
7.9 P5: slice of a landscape with 5^{10} states . . . 101
7.10 P5: gains per instance and damping . . . 102
7.11 P5: where the error is moved . . . 103
7.12 P6: the bytes–energy plane and r as a function of θ . . . 106

8.1 The common frontier of the round . . . 114
8.2 A giant column draws the strand's error . . . 118
8.3 The expert's output error does not predict the KL . . . 119
8.4 Two parts of the KL, position by position . . . 123
8.5 The latent space of the 6 132 states of layer 23 of gpt-oss-20b . . . 125

9.1 The wheel of the sixteen E2M1 codons . . . 139
9.2 The gain theorem, tensor by tensor . . . 140
9.3 The 256 bytes before and after the fold . . . 141
9.4 Why the GEMV returns the same bits . . . 142
9.5 The .trc v1.2 format, to the bit . . . 145
9.6 Why the column context stays out of v1.2 . . . 147
9.7 The codon by the judge's decoder . . . 151
9.8 What the J1 milestone measures . . . 153
9.9 Where the J1 gain comes from . . . 155

10.1 The synonymous mutation in the eigenbasis of H . . . 164
10.2 A row of 32 weights before and after the synonymous mutation . . . 165

10.3 The output error of an expert in the directions of its inputs, frontier and synonym . . 166
10.4 Criterion B2, window by window 171
10.5 The tail of the KL 173
10.6 The frontier of the variants, 768 experts 175

11.1 The experts that were seen 186
11.2 The difficulty of the windows 189

12.1 The real polyhedra of the router of layer 12 201
12.2 The curve of the tropical guard 203
12.3 The guard by layer 204
12.4 The law of the margin 205
12.5 The flip law 206
12.6 The theorem of the guard number 208
12.7 The extrapolation of the guard (model) 209

13.1 The tail of the spectra of layer 12 218
13.2 The exponents by matrix 220

List of Tables

1.1 The genome analogy, and what it designates here . . . 3

2.1 The law of the fifteen MXFP4 values . . . 12
2.2 Four points of the rate–distortion bound . . . 15
2.3 The twelve families of the state of the art . . . 17
2.4 The empty box . . . 18
2.5 The prognosis of the reading . . . 19

3.1 The instruments of the scout and their controls . . . 22
3.2 The sign of the zeros . . . 23
3.3 The energy of the weights by block exponent . . . 24
3.4 The zero-synonym codon, measured . . . 26
3.5 The diagnosis of overfitting . . . 29
3.6 The synonymous mutation without entropy constraint . . . 30
3.7 The synonymous mutation under entropy constraint . . . 30
3.8 The end-to-end KL, base against synonymous mutation . . . 32
3.9 The verdict of the scout . . . 33

4.1 The seed predicted by lemma 1 . . . 37
4.2 The seed measured against theory . . . 38
4.3 The accounting of the recipe . . . 42
4.4 Verdict W, tensor by tensor . . . 44
4.5 The structure of the matrices: real against permuted . . . 45
4.6 The output fidelity of the strand . . . 47
4.7 S^* at the output of the experts . . . 48
4.8 S^* against the base, the KL on ten windows . . . 49
4.9 The transcription cost of the strand . . . 49

5.1 The Hungarian matching, layer 12 . . . 56
5.2 The exact cost of an expert coded as a mutation . . . 59
5.3 The verdicts of duplication . . . 59
5.4 The output of the children, on real inputs . . . 60
5.5 Paralogues, by depth . . . 63

6.1 The zero-synonym codon with the real codec, on the two models . . . 68
6.2 What the neighbours say about a code . . . 69
6.3 The rate–distortion frame, in weight error . . . 71
6.4 Activity-dependent pruning, predicted error . . . 72
6.5 The error at the output of the experts . . . 73
6.6 The KL of the quantisation and pruning plans . . . 74
6.7 The paired comparisons of the plans . . . 75

7.1 The removal of duplicate candidates . . . 78
7.2 P1: the verdict at the declared threshold . . . 81
7.3 P1: the real answer against the tokens after the end . . . 83
7.4 P1: the cold windows of 256 tokens . . . 83
7.5 P2: the verdict . . . 88
7.6 P2: service time of the misses per token . . . 88
7.7 P3, stage A: misses avoided and cells changed . . . 92
7.8 P3, stage B at $N = 146$: the disagreements of first choice . . . 94
7.9 P3: the online biased router . . . 95
7.10 P3 online: the drift per layer . . . 96
7.11 P4: the lead and its judge . . . 97
7.12 P5: the controls . . . 100
7.13 P5: gains of cyclic descent, lead and judge . . . 101
7.14 P6: the decoding of the 120b . . . 106
7.15 The map from the world to the pipeline . . . 108
7.16 The six leads before their judges . . . 109

8.1 The KL protocols of the round . . . 112
8.2 The common frontier, points judged at the KL . . . 115
8.3 Property (a): noise given the scale . . . 116
8.4 Property (b): the giant columns . . . 117
8.5 Property (c): expert error against KL . . . 120
8.6 The quantiles of the per-position KL . . . 120
8.7 Each mechanism and the property that decided it . . . 127
8.8 What lot A was to return . . . 129

9.1 The readers of code 8 . . . 143
9.2 What changes from v1.0 to v1.2 . . . 144
9.3 The files of lot A . . . 148
9.4 What the fold returns on the disk . . . 150
9.5 A-C1: the logits to the bit . . . 150
9.6 J1 of the 120b, session s2 . . . 152
9.7 The crossed pattern . . . 154

10.1 The pieces of lot B and their controls . . . 161
10.2 The fast companion, field by field . . . 168
10.3 The end-to-end KL, 768 experts substituted . . . 169
10.4 The paired differences, window by window . . . 172
10.5 The fast mode served by the engine . . . 176

11.1 What the judge redid, and by which path . . . 182
11.2 J-T0: the four files, through the judge's reader . . . 184
11.3 J-C1: the logits to the bit on the new windows . . . 187
11.4 J-K: the KL on the new windows . . . 187
11.5 J-V: the three-file session of the 120b . . . 191
11.6 J-V: the three-file session of the 20b . . . 192
11.7 What holds according to the judge of the construction . . . 194

12.1 The tropical guard: the controls . . . 202
12.2 The curve of the guard . . . 202
12.3 The guard by layer . . . 204
12.4 The propagation of flips . . . 207
12.5 The extrapolation of the guard . . . 209
12.6 The multi-cue guard . . . 211

13.1 The power of the two criteria . . . 215
13.2 The controls of the instrument . . . 217
13.3 The verdict by family . . . 217
13.4 The exponents by role and by layer . . . 219

14.1 The verdicts of Volume III . . . 223
14.2 The figures and sentences struck through in Volume III . . . 225

A.1 The criteria that decide, and their verdicts . . . 230
A.2 The declared thresholds of Volume III, dated by git . . . 231
A.3 The controls and their breaks . . . 232

B.1 The committed tools of Volume III . . . 238
B.2 Each figure and the files it reads . . . 256
B.3 The licences of the dependencies of Volume III . . . 258

C.1 The log folders of Volume III . . . 261
C.2 The speed sessions of Volume III . . . 263

Listings

9.1 The $-0 \to +0$ fold in the code 148

Notation and conventions

The conventions are those of Volumes I and II. Units are decimal: 1 GB = 10^9 bytes; binary multiples are written KiB, MiB, GiB. Every number carries its provenance: [meas.] measured here on Stone's machine with a committed tool and log, [calc.] computed from named numbers, [lit.] literature (publisher's number, not reproduced). A sentence that a later measurement contradicted is struck out where it stood (~~like this~~), with what replaced it. A file of the repository cited as the source of a number is set as `mesures/genome/`; its path is relative to the root of the `troncons` repository. Figures are numbered III-$x.y$ in the inventory of the figures (`memoire/tome3/INVENTAIRE-figures.md`); a figure drawn outside the common style says so in its caption.

Names of the work

Name	Meaning
troncons	the program and the project; Volume II describes the program
control, break	the control of a measurement is a case whose answer is known in advance; its break forces the control's condition to false, and the instrument must then fail; every new control is broken once before it is trusted (Section 3.1)
20b, 120b	gpt-oss-20b (24 layers of 32 experts: 768 experts) and gpt-oss-120b (36 layers of 128 experts: 4 608 experts), the two models of the volume, their expert weights in MXFP4
MXFP4, E2M1, E8M0	the block format of the expert weights: 32 codes E2M1 (4 bits: sign, two exponent bits, one mantissa bit; sixteen codes for fifteen values, ±0 being equal) and one shared scale E8M0 (one byte, an exponent alone): 17 bytes per block (Chapter 2)
KL	the judge of the volume: Kullback–Leibler divergence of the next-token law of the modified model against the f64 reference, $\mathrm{KL}_t = \sum_i p_{t,i} \log(p_{t,i}/q_{t,i})$ in nats at position t (only $\log p_{t,i} > -16$ is summed), averaged over the scored positions (Section 3.5, Definition 8.1)
f64 reference	`trc-reference`, the exact computation of gpt-oss in double precision (Volume II, lot R); its logits are the truth of every KL
fork	a modified copy of the reference for a need of the round (capturing expert inputs, substituting a base)
R1	the ten evaluation windows of wiki.test of Volume II (windows 0 to 9, 512 tokens each); the KL scores 2 550 positions there, 255 per window. Not to be confused with *the reserve R1*
calibration	windows 10 to 21 of wiki.test (6 132 positions), where the mechanisms were tuned; disjoint from R1

Name	Meaning
fresh windows	the ten windows of wiki.test, touched neither by the calibration nor by R1, on which the judge of the construction re-did the results (Section 11.3)
P0, P1, P2, P2-10, P3	the KL protocols of the round (Table 8.1): P0 lossless (bit-identical logits); P1 the scout, 32 experts out of 768 on R1; P2 the genome, 8 experts on 3 windows; P2-10 the same 8 on the 10 windows of R1 (pass K10); P3 predictive coding and pruning, 128 experts on windows 5 to 9. A KL is compared across protocols only as a ratio to the base of its own protocol
K10	the pass that re-did S^* against the base on the 10 windows of R1 (protocol P2-10), declared before; it replaces the 3-window ratio (reserve R4)
lots R, A, B	R: the reference and the judge of Volume II; A: the zero-synonym codon in the format v1.2 (Chapter 9); B: the synonymous genome and the fast mode (Chapter 10)
arbiter, F1	the arbiter is the role of Volume II that settled the format; F1 is its measurement of the frontier (rANS codes per lane, scales as offsets from the row median), reported in Volume II
frontier	the two points the round had to beat: lossless, 3.8431 bits per weight (A_ctx) or 3.8434 (A_flux); lossy, the *base* of the fast mode (sign and four magnitude classes), 2.6441 bits read at 4.29 % relative weight error (Section 2.3)
A_ctx, A_flux	two accountings of the same lossless code: A_ctx counts the codes coded by rANS per lane and per scale context, the scales, tables and headers; A_flux counts the stream of the codec of study 03, as the arbiter accounts it. They differ by $3 \cdot 10^{-4}$ bit per weight
strand, codon	in the genome of chapter 4, the strand is the bit string of a row, read through a sliding window of l bits (the codon) that a mixer and a table turn into a weight; elsewhere, *codon* also names a 4-bit E2M1 code. Analogies, not arguments
zero-synonym codon, fold	codes 0 (+0) and 8 (−0) give the same value; the fold $-0 \to +0$ rewrites every 8 as 0 before coding, without changing a single value (Section 3.3, Chapter 9)
synonymous mutation	the choice, among the symbols of the base (same decoder), of those that minimise the output error weighted by H rather than the weight error (Section 3.4)
H, e_k, λ_k	H the (damped) second-moment matrix of the inputs x of an expert matrix, weighted by the router weight g ($\sum g^2 x x^\top / \sum g^2$), e_k and λ_k its eigenvectors and eigenvalues
λ	the weight of the entropy constraint of the synonymous mutation ($\lambda = 0$: none; $\lambda = 1$); not to be confused with λ_k
δ, δ^*	the damping added to H; $\delta^* = 1$ the value chosen without looking at the evaluation; in Chapter 8, δ also writes the scale offset that sets κ
κ	two meanings: the scale class of a block, three classes set by its E8M0 against the row median (chapters 3 to 10); in Chapter 12, the guard number $\sigma/\mathbb{E}[m]$ of the router
S^*	the system of the genome: strand, rotation, weighted Viterbi and three exact exceptions per row (Chapter 4)

Name	Meaning
v1.0, v1.2	versions of the `.trc` format: v1.0 the original files; v1.2 adds the fold of the zero (Section 9.4); v1.1 (two levels) is reserved and written by no coder
fast companion	the file `<name>.rapide.trc` that the fast mode of the engine reads, a plain `.trc` holding the synonym base (Section 10.4)
J1	the milestone of Volume II, the exact service of gpt-oss-120b in a bounded window; here, its generation speed re-measured (Section 9.7)
ABBA, $r(B/A)$	a timed series alternating the two files (A B B A A B B A), each pass a fresh process; $r(B/A)$ the ratio of their median speeds, judged against 2σ of the series
reserves R1 to R6	the conditions of the inventory of the figures that keep a figure or a sentence true (Section 1.5); R1: the ×1.44 of the J1 comes from the format v1.2 through the pattern of its read requests, on this disk
III-$x.y$	the number of a figure in the inventory (`memoire/tome3/INVENTAIRE-figures.md`); the printed number follows the chapters of the PDF, where the introduction is chapter 1

Chapter 1

Introduction

Where we start, what this chapter establishes, what remains

The starting point is the first two volumes. Volume I measured Stone's machine and showed that, on it, the disk is the slowest stage: what counts is the number of bits a token must read. Volume II built troncons, which runs gpt-oss-120b (63.4 GB) with 384 MB of memory, exactly (bit-identical logits between the coded file and the raw file), at 2.22 times the best measured baseline; its expert weights were read at 3.84 bits per weight losslessly, and the base of the "base + complement" split at 2.64 bits, lossy. **This volume establishes** what Stone's biomimetic idea, the genome of the weights, has yielded: a round of four teams that put it to the test and said, numbers in hand, why almost everything falls; three mechanisms that win, all three "synonyms" in the sense of the genetic code; their construction in the format and the engine, and the judge that redid it; six leads borrowed from other disciplines; two mathematical leads, the tropical geometry of the router and the spectra of the matrices. **What remains** is gathered by the discussion (Chapter 14): the tail of the KL, which nothing targets yet; the 120b served by its synonym base; another corpus; the read pattern that makes the speed of this disk.

In brief

An AI model is a very long text of numbers. Stone proposed to learn from life, which does not store an organism but a genome, a recipe from which the organism is built. This volume tells what that idea became once it was judged by measurement. The first answer is harsh: once the scale of their block is known, the weights of gpt-oss behave like noise, and noise cannot be summarised; the seed, the shared patterns, the kinship between experts and the lossless recipe all fall for that reason. But life has a move other than the recipe: the redundancy of the genetic code, where several codons designate the same amino acid. The three mechanisms that win are of that kind. The first erases a piece of information that the computation never reads (the sign of a zero), losslessly, and makes the 120b read 1.80 GB less. The second uses what the coder already knows (the energy class of each input column). The third, the synonymous mutation, chooses an approximation of the weights whose error falls where the real inputs do not look: served alone, the base of the 768 experts of the 20b halves the KL of the fixed base of Volume II. The volume also says what does not work, as precisely as what does: the fast mode does not bring the announced speed, the tropical guard does not see the cascading flips, the spectra do not follow a power law, and none of the six borrowed leads clearly holds under its judge.

1.1 The question, asked by Stone and by the machine

The mission. Stone wrote it on 15 October 2026, and Volume I quotes it in full: to run "un modele ultra lourd quelcoque d'une ai [...] sans ram ni rien" (any ultra-heavy AI model, with no RAM or

anything), in a word “compresser à l’extreme puis décompresser en un temps record au besoin à l’instant exacte du noeud” (compress to the extreme, then decompress in record time when needed, at the exact instant of the node). The first two volumes took the second half of the sentence literally: the `.trc` format and the engine decompress each expert at the exact moment the computation asks for it, then release it. This volume takes the first half: compress further.

The idea. The same day, Stone added a lead of his own:

> *J’ai penser à du biomimetisme informatique par exemple simjler une forme de molecule d’adn virtuel qui peut stocker une grande qua tutoriel d’informations en quelque sorte creer une nouvelle convention differentes du bit une transcription et implémentation*

(I thought of computer biomimicry, for example simulating a form of virtual DNA molecule that can store a great quantity of information, in a way creating a new convention different from the bit, a transcription and implementation), then, the next day, his instruction for the study: “le but est de concevoir un systeme fonctionnel et si il ne voye rien de convaincaint vers l’ADN au terme ils recherche le moyen biomimétique le plus prométeur jusqu’à trouver” (the goal is to design a working system, and if they see nothing convincing towards DNA in the end, they search for the most promising biomimetic means until they find it) (`idees/biomimetisme-genome-des-poids.md`). Stone’s conviction (“j’ai la certitude que ce qu’on cherche se trouve dans du biomimetisme”, I am certain that what we are looking for lies in biomimicry) is the working hypothesis of the volume; measurement judges it, and the volume reports both.

What the machine makes of it. A new convention of symbols creates no room: DNA writes with four letters, that is two bits per letter, and no alphabet goes below the entropy of the message [1, 2]. The strength of life is not its alphabet but its mechanisms. And the machine says which ones count: the disk is the slowest stage (2.82 GB/s measured in Volume I), and the card waits while it reads, with, according to the computation of the idea, on the order of 214 operations available per compressed byte. A genome of the weights therefore spends computation to save bytes, the very trade this machine asks for. The question of the volume, in its judgeable form, is this:

> *Is there a mechanism, inspired by life, that makes the disk read fewer bits per expert weight than the frontier of Volume II (*3.84 *bits losslessly;* 2.64 *bits for the base, lossy), at a fidelity judged by the KL on text against the exact reference, and that transcribes on the card faster than the disk reads what it replaces?*

The question is judged at every step by a threshold declared before the measurement (appendix A), and by a judge that did not do the work.

1.2 What “genome” means here

The word is an analogy, and the volume says so each time it uses it. In living things, a sequence of bases is read in triplets, the *codons*, and the genetic code translates them into amino acids [3]. This code is degenerate: 64 codons for some twenty amino acids and a stop signal, so that several *synonymous* codons designate the same one. A mutation that changes a codon into a synonym has no effect on the protein; and the natural code is arranged so that the most probable errors fall on similar amino acids [4, 5]. Table 1.1 gives what the volume transposes, and where.

Table 1.1: The genome analogy, and what it designates in this volume. An analogy chooses a direction of research; it proves nothing.

In living things	In troncons	Chapter
the genome, a recipe rather than a result	a seed and coefficients, or a strand of bits read through a table, from which the weights are rebuilt	Chapter 4
the ribosome reading the RNA codon after codon	a 12-bit window sliding along the strand and returning weights through a table	Chapter 4
gene duplication	an expert coded as a mutation of a parent expert	Chapter 5
predictive coding, synaptic pruning	coding only the deviation from a prediction; removing the useless weights	Chapter 6
the synonymous codon	the sign of an MXFP4 zero, which the computation never reads	Chapter 9
the synonymous mutation	a base of the weights whose error falls where the real inputs do not look	Chapter 10
expression on demand	the expert decoded at the moment of the computation, then released (already built in Volume II)	—

Two warnings hold for the whole volume. Life tolerates errors that a language model perhaps does not; and a beautiful analogy is not an argument: each mechanism is judged by measurement, not by the beauty of the image (`idees/biomimetisme-genome-des-poids.md`, § 6). The volume also borrows from disciplines other than biology (climate, the physics of spin glasses, statistics, the cortex); these borrowings are stated as such in Chapter 7.

1.3 The path of the volume

The volume has three parts, like Volume II.

The question and the round. Chapter 2 asks the question in the terms of the machine: the MXFP4 block to the bit, the frontier, the rate–distortion bound in the domain of the weights, and the twelve "genomic" families of the literature. Chapter 3 gives the scout, which found where the noise and the structure are, and designed the first two synonym mechanisms. Chapters 4 to 6 give the other three teams of the round: the recipe (the strand read through codons, the seed), gene duplication, predictive coding and pruning. Chapter 7 gives mimicry, six leads coming from other disciplines towards the path of the bytes. Chapter 8 gathers the round on a single axis, the KL on text, and says why each mechanism wins or falls.

The construction. Chapter 9 gives lot A: the zero-synonym codon in the `.trc` format, version 1.2, lossless. Chapter 10 gives lot B: the synonymous genome on the 768 experts and the fast mode of the engine. Chapter 11 gives the judge that redid both lots by its own paths.

The leads and the assessment. Chapter 12 gives the tropical geometry of the router and its guard; Chapter 13 the spectra of the matrices; Chapter 14 the discussion, the verdicts gathered, what could be wrong at the scale of the volume, and future work. The appendices give the protocols (declared thresholds and controls), the tools and the logs.

What went to Volume II. The construction met a question that is not the genome's: why two files of the same model read at different speeds on this disk. The disk-placement stream and its judge answered

it (the pattern of read requests that the format imposes, `genome/JUGEMENT-placement.md`); by the decision of that judge, their work goes to Volume II, in a chapter "The disk stage". This volume keeps only its consequence, reserve R1, which fixes how to read the speed gain of the format v1.2 (Section 11.6.5).

1.4 What this volume contributes

1. **The reason for the failures, measured.** Once the scale of the block is known, the weights are noise (the neighbouring code brings only 0.005 bit; the rotated rows are Gaussian to within 1.5 %; two matched experts have the cosine of two experts drawn at random); a few giant columns carry the structure; and the output error of an expert does not predict the KL, which has a smooth part and a tail of routing flips (Chapter 8).
2. **Three synonym mechanisms that win**: the zero codon, lossless (3.843 4 → 3.706 4 bits per weight on the reference tensors, in the A_flux accounting, that of the codec of study 03; 3.843 1 → 3.706 1 in that of the codes, A_ctx, which Chapters 2 and 3 give; 1.80 GB less to read on the 120b); the column context, lossless (3.684 4 bits); the synonymous mutation, lossy (Chapters 3 and 8).
3. **An exact format v1.2**, built, of which a decoder written by the judge from the description alone recovers each drawn expert to the bit, up to the sign of the zeros, and whose logits are bit-identical on ten windows nobody had seen; on this disk, the J1 of the 120b gains ×1.44 with it, through the pattern of its read requests (Chapters 9 and 11).
4. **A fast mode served by the engine**: the 768 experts of the 20b served by their synonym base ($\lambda = 0$), 2.67 bits per weight, KL 0.131 8 against 0.264 0 for the fixed base, recovered outside R1 by the judge, and served by the card as faithfully as by the f64 reference (Chapter 10).
5. **Ideas killed by the numbers**, each with the property that decided it: the seed, the patterns, duplication, the lossless recipe, the Hebbian rule, pruning, the tropical guard, the power law of the spectra; and two speed gains struck out as gains of the fast mode (Chapter 14).

1.5 How to read this volume

The judge: the KL. Almost all the verdicts of the volume are given by a single number, the Kullback–Leibler divergence (the KL) between the next-token law given by the modified model and that of the exact f64 reference of Volume II, averaged over positions of text. It is set out in five sentences in Section 3.5, before its first use, and taken up again with its two parts in Section 8.1. The other names of the work (the R1 windows, the lots, the protocols P0 to P3, A_ctx and A_flux…) are in the table of notation, at the start of the volume.

The provenance of the numbers. Every number carries its provenance: [meas.] measured here, with a committed tool and log; [calc.] computed from named numbers; [lit.] literature, not reproduced; and, when needed, [hyp.] for a stated hypothesis. Every number comes from a committed file, cited by its path (`likethis`); none is copied from memory.

The struck-out sentences. When a measurement has contradicted a published number or sentence, even our own, the sentence is struck out ~~like this~~ where it stood, and we say what replaced it. The volume carries many of them: they are the trace that the judges did their work.

The figures. The figures in the style of the volume have a number in the inventory of the figures (III-$x.y$, the chapter of the inventory and the rank, `memoire/tome3/INVENTAIRE-figures.md`);

the text uses it only for a figure planned and not drawn, and calls the others by their printed number, which follows the chapters of the PDF (the introduction is chapter 1 there). Each is drawn from the committed data by a committed script, and its caption, judged, opens on the idea of the figure in one sentence. The figures of mimicry and the slice of the router are included as their leads drew them, outside the common style, and their caption says so; those of the spectra come from a separate document of their lead.

The reserves. Six reserves of the inventory of the figures run through the volume; so as not to confuse them with the ten evaluation windows that Volume II calls R1, the volume always writes "reserve R1". Reserve R1: the ×1.44 gain of the J1 comes from the format v1.2 through the pattern of its read requests, specific to this disk, never from "the state of the file". R3: the measurements made with 32 substituted experts and those made with 768 are joined by no line. R4: the KL ratio of S^{*} comes from the ten-window pass; the old three-window number is struck out. R5: the figures outside the common style are stated as such. R6: the second draw of P3, in progress in another session, gives no number to the volume. (R2 concerns Volume II.)

What could be wrong. Each chapter closes on its own section; that of the whole volume is in Section 14.4.

Part I

The question and the round

Chapter 2

The question of the genome: the roof, the bits read per weight, the frontier

Where we start, what this chapter establishes, what remains

The starting point is two volumes and an idea. Volume I set the roof: when the disk limits, the speed of a token is the throughput of the disk divided by the bytes it must read, and only the number of bits per weight has a slope of -1 on that roof with nothing in return. Volume II built the engine and the `.trc` format, which reads the experts of gpt-oss at **3.8431 bits per weight losslessly** (MXFP4 codes in rANS with one table per lane, scales as deviations from the row median; the arbiter's measurement F1) and provides for a base of **2.6441 bits** (sign and four magnitude classes) at **4.29 %** relative squared error on the weights. Study 03 had measured that, given the scale of their block, the codes are almost independent: the neighbouring code brings only 0.005 bit. Stone, finally, had set out an idea (`idees/biomimetisme-genome-des-poids.md`) and an instruction, on 2026-10-16: "le but est de concevoir un systeme fonctionnel" (the goal is to design a working system). **This chapter establishes** the question in bits; the MXFP4 block to the bit, with the law of its fifteen values; the frontier to beat and what a rate–distortion bound computed in the domain of the weights says about it, set out as an *upper bound* on $R(D)$ and proved to be one; the state of the art in twelve families, each passed through two filters (which dependency does it exploit? which error does it measure?); the box that nobody had filled on a model born in 4 bits; and what the reading predicts, which the next four chapters measure. **What remains** is the measurements themselves: the scout (Chapter 3), the recipe (Chapter 4), duplication (Chapter 5), predictive coding and pruning (Chapter 6), then the synthesis of the round (Chapter 8). Sources: `genome/00-etat-de-l-art.md`; `genome/etat-de-l-art-pour-le-memoire.md`; `genome/01-eclaireur.md` § 3; `mesures/genome/etat-de-l-art/journaux/borne-rd.json`, `borne-rd-passe1.json`, `zero-canonique.json`, `zero-temoin-sans-drapeau.json`; `mesures/genome/etat-de-l-art/sources.csv`, `recherches.csv`; Volume I, chapter 3 (the roof).

In brief

A language model is a very long text of numbers: 19 billion expert weights for gpt-oss-20b, 115 billion for the 120b. Stone's idea fits in an image: life does not store an organism, it stores a genome, a few billion letters from which the organism is built. Can we, likewise, store a *recipe* of the weights rather than the weights? The image is an analogy, not a proof, and this chapter translates it into a question the machine can judge: how many bits must be read from disk per weight, and at what error? Information theory's answer is harsh: if the weights, seen by themselves, behave like independent draws, no recipe describes them more briefly than their entropy coding. The reading of the literature therefore says first where each "genomic" idea

> bets on a dependency, then where the only remaining margin lies: not in the weights, but in the error one chooses to measure.

2.1 The roof: what a token must read

Volume I proved it (the roof theorem, chapter 3): under the sole assumption that a stage of throughput W passes at most $W\tau$ bytes in a time τ, a token that must read O bytes on that stage lasts at least O/W. For weights read once each, $O = Nb/8$ (N weights read per token, b bits per weight), hence

$$\text{tokens/s} \;\le\; \frac{8\,W}{N\,b}, \qquad \frac{\partial \ln(\text{tokens/s})}{\partial \ln b} = -1. \tag{2.1}$$

On Stone's machine, the slowest stage is the disk: 2.82 GB/s measured [meas.] (`fondations/journaux/01-machine/debits-mesures.json`, cited by `genome/00-etat-de-l-art.md` § 1). Halving the bits read per weight doubles the roof; removing 3.56 % of the expert bytes read raises it by $1/(1-0.0356) - 1 = 3.7\,\%$ [calc.]. The whole quest of the genome is therefore judged in **bits read per weight**, at a given error.

Computation is not the bottleneck. The card has on the order of 214 free operations per compressed byte read (fondations/01), 926 at the real throughput of the engine's disk according to the correction of Volume II (m12) [calc.]. A mechanism whose transcription costs less than this budget is free in time: it is judged only by its bits and its error. The chapters of the round count this cost whenever it is not zero (Section 4.10).

2.2 The MXFP4 block to the bit

The experts of gpt-oss are published in MXFP4 [6], the small floating-point format of the Microscaling family [7, 8]. A block carries 32 consecutive weights of a row in 17 bytes (Figure 2.1): one E8M0 scale byte, an exponent alone, and sixteen bytes of codes, two 4-bit codes per byte.

The E2M1 code. A 4-bit code c reads $c = (s, E_1E_0, M)$: one sign bit, two exponent bits, one mantissa bit. Its value is

$$\text{E2M1}(c) = (-1)^s \times \begin{cases} M/2 & \text{if } E = 0 \quad \text{(subnormal)}, \\ 2^{E-1}\,(1 + M/2) & \text{if } E \in \{1, 2, 3\}, \end{cases} \tag{2.2}$$

which gives the eight magnitudes $\{0,\ 0.5,\ 1,\ 1.5,\ 2,\ 3,\ 4,\ 6\}$ [calc.]. Sixteen patterns for fifteen values: code $0 = (0, 00, 0)$ and code $8 = (1, 00, 0)$ are both worth zero, one $+0$, the other -0. The figure says so in its caption: it is the first synonymous codon of the volume, and the next chapter measures its price.

The value of a weight. Weight i of block b is $w_{32b+i} = 2^{\,e_b - 127}\,\text{E2M1}(c_i)$, e_b being the E8M0 byte of the block. The following chapters often write the equivalent form of the kernels of llama.cpp and of `trc-reference`, with a doubled table:

$$w = \pm K[m] \cdot 2^{\,e-128}, \qquad K = \{0, 1, 2, 3, 4, 6, 8, 12\}, \tag{2.3}$$

where m is the 3-bit magnitude; the two writings give the same products since $K[m] = 2\,|\text{E2M1}(c)|$ and $2^{e-128} = 2^{e-127}/2$ [calc.]. A block costs $8 + 32 \times 4 = 136$ bits, that is 4.25 bits per weight before any coding [calc.]. A row of 2 880 weights has 90 blocks; an expert matrix of gpt-oss, $2\,880 \times 2\,880 = 8\,294\,400$ weights.

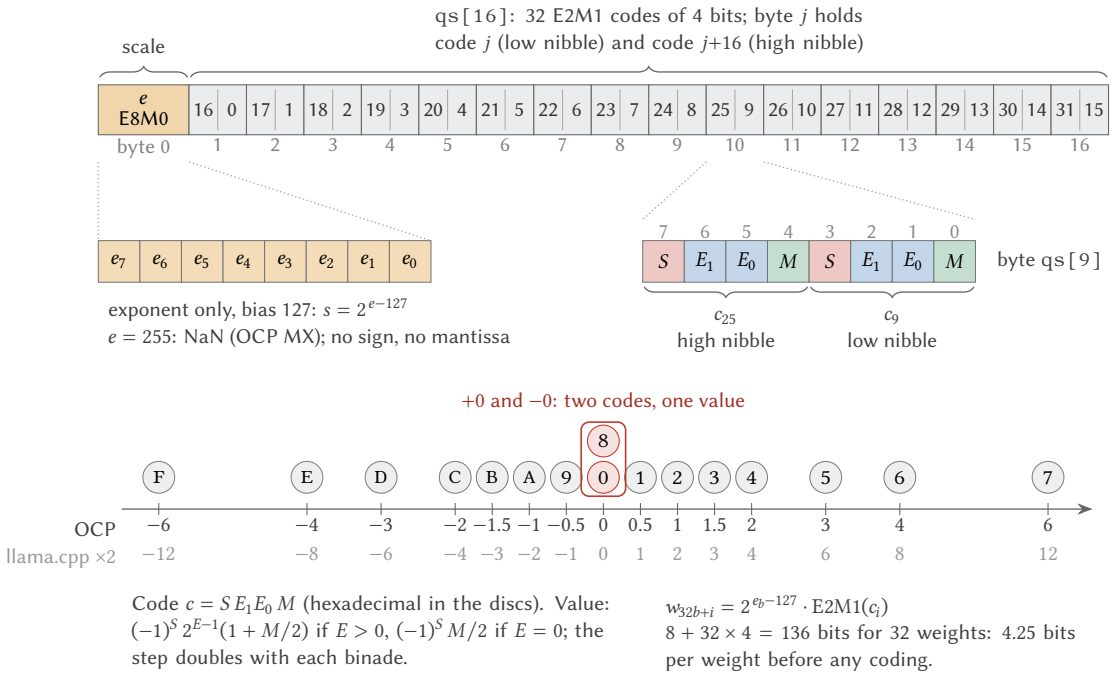


Figure 2.1: Thirty-two weights share one byte of scale; sixteen code patterns give fifteen values, because +0 and −0 are the same number. [schem.] The scale e is an E8M0 byte (an exponent alone); sixteen bytes carry the codes, code i in the low nibble of byte i when $i < 16$, in the high nibble of byte $i - 16$ otherwise. Each code is an E2M1 float: a sign, two exponent bits, one mantissa bit. Above the axis, the values of the OCP specification; below, in grey, those of the doubled table that llama.cpp uses ($\{0, 1, 2, 3, 4, 6, 8, 12\}$ with the scale halved, the same products), which the wheel of codons in Chapter 9 prints. The pair +0 (code 0), −0 (code 8) is the first synonym codon of Chapter 9. What the figure does not show: the order of the blocks in a tensor (row after row, 90 blocks per row of 2 880 weights).

Data: OCP Microscaling Formats (MX) v1.0 specification; llama.cpp, `ggml-common.h` (`block_mxfp4`).

$\lvert K\rvert$ (doubled units)	0	1	2	3	4	6	8	12
$\lvert$E2M1$\rvert$	0	0.5	1	1.5	2	3	4	6
$P(V = +\lvert K\rvert)$ (%)	13.694	8.601	10.152	5.461	7.883	5.595	3.998	1.462
$P(V = -\lvert K\rvert)$ (%)		8.602	10.153	5.460	7.882	5.595	3.998	1.462

Table 2.1: The law of the fifteen MXFP4 values on the 12 reference tensors of gpt-oss-20b, ±0 merged [meas.]. Data: `mesures/genome/etat-de-l-art/journaux/borne-rd-passe1.json` (`loi_v`, in doubled units K); $E[V^2] = 17.847$ in these units. The law is symmetric: only the deviation of the scale from the row median (the context κ of the format) and the magnitude class carry information beyond this table.

The law of the values. On the 12 reference tensors of the frontier (layers 0, 8, 16 and 23 of gpt-oss-20b, gate, up and down matrices, 3 185 049 600 weights), the fifteen values have the law of Table 2.1 [meas.] (`mesures/genome/etat-de-l-art/journaux/borne-rd-passe1.json`, field `loi_v`). It is symmetric to within $1.3 \cdot 10^{-5}$, and zero is the most frequent value in it: 13.69 % of the weights. Its entropy is $H(V) = 3.7228$ bits, against $H(C) = 3.8598$ for the sixteen codes of the same tensors (`genome/01-eclaireur.md` § 2.1): the gap, 0.137 bit, is the price of the sign of the zeros.

2.3 The frontier, and the rate–distortion bound

2.3.1 What a weight costs today

Two points form the frontier that the round had to beat (`genome/00-etat-de-l-art.md`, In brief, point 1; the arbiter's measurement F1, reported in Volume II, chapter 2):

- **lossless**: 3.8431 bits per weight (A_ctx: codes in rANS with one table per lane and per scale context, E8M0 scales as deviations from the row median, tables and headers counted), each stream decoded exactly [meas.];
- **lossy**, the *base* of the fast mode: the sign and four contiguous magnitude classes, {0, 0.5}, {1, 1.5, 2}, {3, 4}, {6} in E2M1 units, each rebuilt by its centroid weighted by s^2 (0.237 166; 1.424 298; 3.418 577; 6; `mesures/genome/genome/frontiere.json`), at **2.6441 bits** read for a relative squared error on the weights $D = \sum s^2(v - \hat{v})^2 / \sum s^2 v^2 = \mathbf{4.2920\ \%}$ [meas.].

The base is a *function of the code*: the exact code is recovered behind it by a lossless complement, which the format reserves for its two-level version. That is its strength (it nests) and its constraint (it is scalar and fixed in advance); the bound that follows says how much this constraint costs.

2.3.2 The bound of a memoryless coder, in the domain of the weights

The model. We write $w = s\,v$, with v drawn i.i.d. from the law of Table 2.1 (±0 merged), and $s = 2^{e-128}$ known to the decoder. The distortion is the pooled relative error $D = \sum s^2(v - \hat{y})^2 / \sum s^2 v^2$. A single quantiser for all blocks (a single slope, called *uniform* allocation) is exactly the constraint of the frontier base. The rate–distortion function of the discrete source V under $d(v, \hat{y}) = (v - \hat{y})^2$ is

$$R(D) = \min_{Q(\hat{y}|v)\ :\ \mathbb{E}\, d(V,\hat{Y}) \le D} I(V;\hat{Y}), \tag{2.4}$$

and the source coding theorem with a fidelity criterion [9] says that every rate above $R(D)$ is achievable by long enough codes, none below.

The Blahut–Arimoto algorithm. For a slope $\beta > 0$ (bits per unit of distortion), one iterates to the fixed point [10]

$$Q(\hat{y} \mid v) = \frac{q(\hat{y})\, 2^{-\beta(v-\hat{y})^2}}{\sum_{y'} q(y')\, 2^{-\beta(v-y')^2}}, \qquad q(\hat{y}) = \sum_{v} p(v)\, Q(\hat{y} \mid v), \tag{2.5}$$

then reads $R = \sum_{v,\hat{y}} p(v) Q(\hat{y} \mid v) \log_2 \frac{Q(\hat{y}|v)}{q(\hat{y})}$ and $D = \sum p(v) Q(\hat{y} \mid v)(v - \hat{y})^2$. At low slopes, the number of iterations needed grows like $1/\beta$; that is where the tool struggled.

The history of the passes, as it is. Pass 1 of `borne_rd.py` made its own control B1 fail at the low end of the curve (at $\beta = 10^{-6}$, the point had not converged), while B2 held (a Gaussian follows 2^{-2R} to within 10^{-5} bit) and its break failed. Pass 2, with the correction declared after the failure (a point of distortion above $\mathbb{E}[v^2]$ is replaced by the zero-rate point), made B1bis and B2 hold, but only 243 points out of 1 619 converged, and the curve with optimal allocation between exponents was read outside its domain: it is not published. Pass 3 computed only the uniform allocation. In the zone from 1.8 to 3.2 bits, Blahut–Arimoto did not reach the declared tolerance (10^{-11}) in 200 000 iterations; by the rule declared before pass 3, no *converged* number is therefore published at 2.64 bits nor at 4.29 %. What remains true without convergence, and which was read *after* pass 3 (the reading is stated as such, `genome/01-eclaireur.md` § 3), is the following lemma.

Lemma 2.1 (Every iterate is an achievable point)**.** *For every test channel $Q(\hat{y} \mid v)$ and every law q on the reproductions,*

$$\sum_{v,\hat{y}} p(v)\, Q(\hat{y} \mid v) \log_2 \frac{Q(\hat{y} \mid v)}{q(\hat{y})} \;\geq\; I(V;\hat{Y}) \;\geq\; R\big(\mathbb{E}\, d(V,\hat{Y})\big), \tag{2.6}$$

with equality on the left if and only if q is the marginal $m(\hat{y}) = \sum_v p(v) Q(\hat{y} \mid v)$. The point (D, R) read at any iteration of Blahut–Arimoto is therefore above the curve: $R(D) \leq R$.

Proof. Write $\log_2 \frac{Q}{q} = \log_2 \frac{Q}{m} + \log_2 \frac{m}{q}$ under the sum. The first term gives $\sum p(v) Q(\hat{y} \mid v) \log_2 \frac{Q(\hat{y}|v)}{m(\hat{y})} = I(V;\hat{Y})$, by definition of the mutual information of the pair $(V, \hat{Y})$ with law $p(v)Q(\hat{y} \mid v)$. The second depends only on $\hat{y}$: $\sum_{\hat{y}} \big(\sum_v p(v) Q(\hat{y} \mid v)\big) \log_2 \frac{m(\hat{y})}{q(\hat{y})} = \sum_{\hat{y}} m(\hat{y}) \log_2 \frac{m(\hat{y})}{q(\hat{y})} = D(m \,\|\, q)$, the Kullback–Leibler divergence [11], non-negative and zero if and only if $q = m$ (Gibbs' inequality, [2], § 2.6). Hence the first inequality. The second is definition (2.4): Q is an admissible channel for the distortion $D = \mathbb{E}\, d(V,\hat{Y})$ it achieves, so $R(D)$, the minimum over these channels, is at most $I(V;\hat{Y})$. ∎

Two consequences make the figure readable. First, the curve traced by the iterates, converged or not, is an **upper bound** on $R(D)$, never an estimate from below. Second, $R(D)$ is convex; every segment between two achievable points is achievable by time sharing (coding a fraction θ of the blocks with the first code and the rest with the second gives $(\theta D_1 + (1-\theta)D_2,\ \theta R_1 + (1-\theta)R_2)$), and the lower convex hull of the iterates remains an upper bound. It is the hull that is read at the error and at the rate of the frontier.

What the bound says. The 40 iterates of the uniform allocation, 25 of them converged, are plotted in Figure 2.2 [meas.] [calc.] (`memoire/figures/tome3/donnees/eclaireur-nombres.csv`, drawn from `borne-rd.json`). Table 2.2 keeps the four points that the scout published. At the error of the frontier, the chord between two iterates gives $R(4.29\,\%) \leq 2.168$ bits, **0.476 bit below the frontier**; at its rate (2.6441 bits), the distortion is at most 1.84 %, **0.43 times its error** [calc.]. The scout wrote

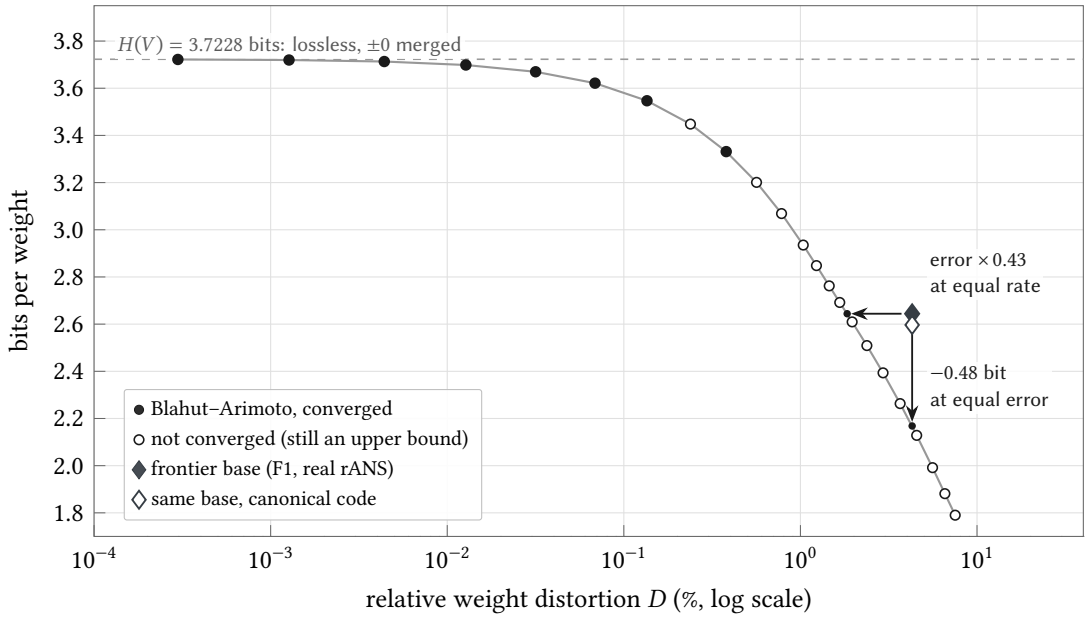


Figure 2.2: The gap between the frontier base and the distortion–rate bound of a memoryless coder lies in the shape of the quantiser, not in a structure of the weights. [calc.] The source is the 15-valued code v (with ±0 merged) scaled by its block's E8M0, one quantiser for every block (the frontier's own constraint). Each Blahut–Arimoto iterate is an achievable point, converged or not, because $\sum p\,Q \log(Q/q) = I(V;\hat{Y}) + D(\text{marginal} \,\|\, q) \geq R(D)$; the curve is therefore an upper bound on $R(D)$. This reading was set after the third pass; the rule declared before it published converged points only. Between 1.8 and 3.4 bits, 15 of the 40 iterates did not reach the declared tolerance: hollow marks. The chords between two points are achievable by time sharing. At the frontier's error (4.29%), the bound is at most 2.17 bits, 0.476 bit below the frontier (2.6441 bits, real rANS); at its rate, the distortion is at most 1.84%. The bound is that of codes *knowing the scale*: the scales are not counted in it, while the frontier's bits count them (with its context κ). On the experts of the $\lambda = 1$ protocol (16 experts), the scales cost 0.0375 bit per weight and the symbols alone $H_0 = 2.6415$ bits; against that H_0, the gap is 0.473 bit instead of 0.476, and the conclusion does not change. What the figure does not show: an allocation that gives more rate to the heavy blocks (computed outside its domain, not published), and any output or KL error: this metric is not a judge (Chapter 8).

Data: `mesures/genome/etat-de-l-art/journaux/borne-rd.json` (`uniforme_precis`), `zero-temoin-sans-drapeau.json`, `zero-canonique.json`, `mesures/genome/etat-de-l-art/passes/frontiere-lam0-damp0.01-e0_8_16_24.json`; script `memoire/outils/tome3_eclaireur.py`.

Rate (bits per weight)	Relative distortion D	Reading
2.1284	4.55 %	roughly the error of the frontier, 0.51 bit less
2.2631	3.67 %	below the frontier in rate *and* in error
2.5091	2.38 %	
2.6095	1.96 %	at the rate of the frontier, less than half its error

Table 2.2: Four points of pass 3 (uniform allocation, a single quantiser for all blocks), each an upper bound on $R(D)$ by lemma 2.1 [meas.]. Data: `mesures/genome/etat-de-l-art/journaux/borne-rd.json` (`uniforme_precis`); `genome/01-eclaireur.md` § 3.

"about 0.47 bit less" and "less than half its error" (`genome/01-eclaireur.md` § 3); the judge of the figures redid the chord through the convex hull of the 40 iterates and found the same numbers (`memoire/tome3/JUGEMENT-figures2-ronde.md`, M12).

An accounting precaution. The bound is that of codes *that know the scale*: the scales are not counted in it, whereas the 2.6441 bits of the frontier count them (with its context κ). On the 16 experts of the scout's protocol where the frontier was redone with the entropy estimator, the scales cost 0.0375 bit per weight and the symbols alone $H_0 = 2.6415$ bits; against this H_0, the gap is 0.473 bit instead of 0.476 [calc.] (`eclaireur-nombres.csv`, after `passes/frontiere-lam0-damp0.01-e0_8_16_24.json`). The conclusion does not change.

Where the gap is. It is not in a structure of the weights: the model of the bound assumes them independent given the scale, which study 03 measured to be true to within 0.005 bit. It is in **the shape of the quantiser**. The frontier is scalar and constrained to be a function of the code; the bound is reached by long, free codes. The gap is shared between two known gains. The first is that of an entropy-constrained scalar quantiser, which chooses its cells for their cost in bits and not for their number. The second is the space-filling gain of a vector or trellis code: at high rate, a spherical quantisation cell in high dimension has, at equal volume, a moment of inertia smaller than the cube of a uniform scalar quantiser by a factor $\pi e/6$, that is at most

$$10 \log_{10} \frac{\pi e}{6} = 1.53 \text{ dB}, \qquad \frac{1}{2} \log_2 \frac{\pi e}{6} = 0.2546 \text{ bit per weight} \tag{2.7}$$

at equal error [calc.] [12, 13]. A generator of weights (seed, automaton, dictionary) has no other structure to exploit than this one: it can only aim at the same bound. The chapter on the recipe (Chapter 4) finds exactly this split, measured.

2.4 The state of the art, in twelve families

2.4.1 Two filters

First filter: which dependency? An i.i.d. source has no structure other than its law. Its shortest description, on average, is its entropy coding [1]; the theorem holds for every description, generating programs included, in the sense of Kolmogorov complexity, whose expectation per symbol tends to the entropy for an i.i.d. source [2, chapter 14]. A "genomic" idea can therefore win losslessly only the dependency it exploits, and that dependency must exist in our weights. On the 72 expert tensors of gpt-oss-20b, study 03 and measurement F1 had given [meas.] (`genome/00-etat-de-l-art.md` § 1): an order-0 H(code) of 3.860 bits out of 4; a contribution of the previous code of 0.005 bit; a

contribution of the scale context of about 0.035 bit (2.642 → 2.607 for the base); the E8M0 scales given the row median at 1.01 bits for 32 weights, that is 0.032 bit per weight.

Second filter: which error? With loss, what counts is not $\|\Delta W\|$ but what the error changes at the output. For a matrix W and its inputs x, with second-moment matrix $H = \mathbb{E}[xx^\top]$,

$$\mathbb{E}\,\|\Delta W x\|^2 = \mathbb{E}\big[x^\top \Delta W^\top \Delta W x\big] = \mathbb{E}\,\mathrm{tr}\big(\Delta W x x^\top \Delta W^\top\big) = \mathrm{tr}\big(\Delta W\, H\, \Delta W^\top\big), \tag{2.8}$$

by the cyclicity of the trace and the linearity of expectation [calc.]. If H were proportional to the identity, the output error would be the error on the weights; it is not: the activations of language models have dominant channels, and protecting the hundredth of the weights that read them reduces the error a great deal [14] [lit.]. The whole margin of a lossy mechanism lies in (2.8).

2.4.2 The families

Table 2.3 arranges the twelve families read by the scout, at the primary source (the abstract of each paper, through the arXiv or Crossref API, on 2026-10-16; the published formulation for the works whose mechanism counts: GPTQ, OBC, QMoE, MatGPTQ, SeedLM, Tied Trit-Planes; `mesures/genome/etat-de-l-art/sources.csv`). The authors' numbers are publisher's numbers, not reproduced here [lit.]. For each: what it proposes, and what we lack to use it on weights born in MXFP4.

Two computations that judge the seed before any measurement. SeedLM approximates a block of C weights by k columns of a pseudo-random matrix. If V is a uniform random subspace of dimension k and w a fixed vector, $\mathbb{E}\,\|P_V w\|^2 = \frac{k}{C}\|w\|^2$: by rotation invariance, P_V has the same law as $O^\top P_0 O$ for O uniform and P_0 the projector onto the first k coordinates, and $\mathbb{E}\,[O^\top P_0 O] = \frac{\mathrm{tr}\,P_0}{C} I = \frac{k}{C} I$ [calc.]. A block of almost i.i.d. weights therefore leaves a residual of relative energy close to $1 - k/C$. SeedLM buys it back by choosing the best seed among 2^{16}; but one cannot gain more information than the index one stores: 16 bits per block at most [calc.]. Section 4.2 derives the exact law (the best of N subspaces follows a Beta law) and confronts it with the real weights.

2.5 What has never been tried on a model born in 4 bits

The scout queried the arXiv API on 2026-10-16, queries logged with their number of answers (`mesures/genome/etat-de-l-art/recherches.csv`): "MXFP4 AND gpt-oss", "lossless AND MXFP4", "gpt-oss AND compression", "Matryoshka AND quantization", "any-precision AND LLM", "DNA AND neural network AND weight compression" (0 answers), "cellular automata AND weights AND compress" (0), "dictionary learning AND large language model AND weight AND compression", "genetic code AND error AND quantization" (0). Table 2.4 crosses the moves with their application to a model born in 4 bits.

The box taken. Rows 4 and 5 together, within the frame of rows 2 and 3: *choosing the symbols of the nested base by error feedback on the real inputs, under an entropy constraint, without touching the decoder or the exact code.* The reason for the choice is a number: it is the only move of the list that does not bet on a dependency between weights, which 03 measured to be absent (0.005 bit); it bets on a dependency between *inputs*, which the engine can measure. Life has the same move, and the scout took it as a guide, as an analogy: the genetic code is arranged so that the probable point errors fall on similar amino acids, to the point that it is "one in a million" among the possible codes [5]; the

Family	What it proposes (literature, not reproduced)	What we lack
1. Seeded recipe	SeedLM [15]: a block approximated by a few pseudo-random columns of an LFSR, seed and coefficients stored; PRANC [16], NOLA [17], intrinsic dimension [18]	PRANC, NOLA and the intrinsic dimension train *in* the subspace; SeedLM works after the fact but lossily, on BF16; a random subspace of dimension k keeps on average k/C of the energy of a block
2. Generating networks	HyperNetworks [19], HyperNEAT [20], neural cellular automata [21], HyperNCA [22]	all generate weights *trained to be generable*; none describes after the fact a model trained freely
3. Kinship and merging of experts	MC-SMoE [23], Merging Experts into One [24], HC-SMoE [25], Sub-MoE [26], D^2-MoE [27], MoBE [28], OMP-MoE [29]	all start from BF16 and measure a task accuracy; none measures kinship in bits on 4-bit codes
4. Differential coding	BitDelta [30] (the delta of a fine-tuning in 1 bit), Delta-CoMe [31]	a parent: a fine-tuning has one, an expert does not, unless it was born by copy
5. Predictive and lossless entropy coding	predictive cortex [32], DeepCABAC [33], ZipNN [34], DFloat11 [35], Huff-LLM [36], TRACE [37]	all win on the BF16 exponent, highly redundant; an MXFP4 code no longer has an exponent per weight, there remains 0.14 bit of redundancy per code ($4 - 3.86$) and 0.14 on the scales, already taken
6. Pruning	synaptic density [38], OBD [39], OBS [40], Deep Compression [41], removed experts [42, 43]	sensitivity is read in the output, not in the weights; OBS is the first "error feedback"
7. Dictionaries and vector codes	sparse coding [44], AQLM [45], QuIP# [46], QTIP [47], HyperQuant [48], MASA [49]	they *replace* the grid and start from BF16; on an i.i.d. source, they win only the space-filling gain (2.7)
8. Error feedback	OBC [50], GPTQ [51], QMoE [52], AWQ [14], MoEQuant [53], EAQuant [54], sigma-delta neuron [55]	a starting point born in 4 bits, an imposed and nested grid, the cost in rate; and the constraint that a lossless complement return the exact code behind the base
9. Nested formats, models born in 4 bits	Matryoshka [56], Any-Precision [57], MatGPTQ [58], MX [7], MXFP4 training [59], gpt-oss [6], Scale-QLoRA [60], Tied Trit-Planes [61], BitNet [62]	MatGPTQ, the closest, starts from BF16 and truncates the high bits of an integer; Tied Trit-Planes requantises MXFP4 experts read from SSD, but at 4.06 bits and without nesting
10. DNA in the proper sense	computation [63], storage [64–66], error-correcting genetic code [5], codon usage bias [67]	the density of DNA is physical, not informational; what transposes is the arrangement of a code so that the error falls where the phenotype does not see it
11. The evolution of codes	EvoPress [68], HyperNEAT	an optimiser, not a representation: it creates no information
12. Distributed memories and indexes	holographic representations [69], hippocampal index [70]	an index recalls only a memory already stored: it is a cache, already built in Volume II

Table 2.3: The twelve "genomic" families of the state of the art, read at the primary source [lit.]. Synthesis of `genome/00-etat-de-l-art.md` § 2; addresses, publication and consultation dates: `mesures/genome/etat-de-l-art/sources.csv` and `memoire/sources/genome-etat-de-l-art.bib`.

Move	Done elsewhere	On a model born in 4 bits?
lossless on MX codes and scales	ZipNN, DFloat11, TRACE: on BF16	troncons (03, F1: 3.84 bits); nothing else found
base nested in the native code, exact complement	Matryoshka, Any-Precision, MatGPTQ: on integers, from BF16	troncons (07, F1: base a function of the code)
requantising the published MXFP4 experts	Tied Trit-Planes (4.06 bits, SSD), ESTS (after pruning and re-fine-tuning)	never *below* 4 bits with an exact complement
error feedback on the real inputs	OBC, GPTQ, QMoE, MatGPTQ: from BF16	**never** to choose a base nested in an MXFP4 code
entropy constraint in the feedback	QMoE codes the entropy after the fact; HyperQuant codes with Rice	**never** in the choice of the symbols of a native base
seed, hypernetwork, automaton, dictionary after the fact	SeedLM, MoBE, Sub-MoE, D^2-MoE (BF16)	not found; Chapters 4 and 5

Table 2.4: What has been done elsewhere and what has not been done on a model born in 4 bits [lit.]. "Never" means "not found by the logged queries", not "non-existent". Source: `genome/00-etat-de-l-art.md` § 3.

neuron codes a continuous signal in spikes by carrying its error over to what follows, in the manner of a sigma-delta modulator [55]. Chapter 3 builds this mechanism, the "synonymous mutation", and measures it.

2.6 What the reading predicts

Table 2.5 is the prognosis written on 2026-10-16, before any measurement of the round (`genome/00-etat-de-l-art.md` § 4). It says, for each family, the dependency it exploits and the measurement of troncons that judges it. Chapters 3 to 6 confront it with the numbers; Chapter 8 draws up the balance. This chapter announces no result: it fixes what could have contradicted them.

The last row calls for a clarification, which the rest of the volume makes necessary: "going beyond the bound of the weights" does not mean going below the $R(D)$ of Figure 2.2, which is impossible in its metric. It means *changing the metric*: a base whose error on the weights *rises* can make less error at the output (2.8), and that is the only way in which a lossy mechanism gains something the bound of the weights does not forbid it. The following chapters will also show that one must go further still, to the Kullback–Leibler divergence on text, because the output error of an expert misleads in its turn.

What could be wrong

- **Only abstracts were read**, plus the published formulations of the six key methods: a number from an abstract is a publisher's number, never reproduced. A paper may contain an experiment on an MXFP4 model that its abstract does not mention: "never tried" means "not found by the logged queries".
- **The searches covered only arXiv and Crossref**: conference proceedings not deposited on arXiv may be missing.
- **The near-independence of the codes** comes from 03 (order 1, neighbour in the row, scale context). A higher-order, long-range or between-experts dependency is not measured there; Chapters 5 and 6 measure it.

Family	Dependency exploited	Measurement that judges it	Prognosis
seed, hypernetwork, automaton	regularity of the weights	0.005 bit for the neighbour (03)	no lossless gain; lossy, at best the bound $R(D)$ of the weights
expert kinship, delta	correlation between experts	mutual information between experts (Chapter 5)	a gain if and only if the mutual information is large
lossless predictive	sequential dependency	0.005 bit (03)	at most 0.005 bit
pruning	useless weights	measured sensitivity (Chapter 6)	depends on the output, not on the weights
vector code, dictionary	shape of the law	space-filling gain, at most 0.2546 bit	at most 0.25 bit, new decoder
synonymous mutation	**correlation of the inputs**	$H = \mathbb{E}[g^2 xx^\top]$ on the real inputs (Chapter 3)	**the only one that can go beyond the bound of the weights**

Table 2.5: The prognosis of the reading, declared before the measurements of the round [calc.] [lit.]. Source: `genome/00-etat-de-l-art.md` § 4.

- **The Kolmogorov argument is on average over the source**: a particular tensor may have a short description (the seed that initialised it, for example); after training, nothing indicates it, and that seed is not published.
- **The bound of Figure 2.2** assumes the codes independent given the scale; an unmeasured dependency would lower it further. It is an upper bound by lemma 2.1; reading the non-converged iterates as bounds was set out *after* pass 3, and the rule declared beforehand published only converged points. It does not count the scales (0.0375 bit).
- **The space-filling gain** (0.2546 bit) is an asymptotic high-rate bound; at 2 or 3 bits it is smaller, which strengthens the prognosis.
- **The metric of the bound** is the pooled error on the weights, dominated by a small share of the blocks (Section 3.2): it locates the frontier, it judges no mechanism.

Chapter 3

The scout: where the noise is, where the structure is

Where we start, what this chapter establishes, what remains

The starting point is the frontier of Chapter 2: 3.8431 bits per weight losslessly, the base of 2.6441 bits at 4.29 % error on the weights, and codes almost independent given their scale (0.005 bit for the neighbour). The reading had pointed to an empty box: choosing the symbols of a nested base by error feedback on the real inputs, under an entropy constraint, without touching the decoder. **This chapter establishes**, before designing anything, where the noise and the structure of the weights are: the sign of a zero is a perfect coin, and the energy is concentrated in a few blocks, so that the metric of the weights is not a judge. Then two mechanisms, each with its proof and its measurement. The first, the *zero-synonym codon*, beats the frontier with no loss of value: 3.7061 bits instead of 3.8431 on the 12 reference tensors, logits of the f64 reference bit-identical. The second, the *synonymous mutation*, first fails through overfitting, then, regularised and under an entropy constraint, beats the lossy frontier under the declared criterion: 2.563 bits instead of 2.642, and an expert output error divided by three to four on held-out inputs. End to end, its variant without entropy constraint lowers the KL against the reference by 27 %. **What remains** is a variant that is below the frontier both in bits and in KL (none is: the KL was measured only on the variant $\lambda = 0$, dearer by 0.011 bit); the porting of the zero codon into the format, which Chapter 9 does; the synonymous genome as a second file, which Chapter 10 does. Sources: `genome/01-eclaireur.md`; `genome/etat-de-l-art-pour-le-memoire.md` § 2; `genome/05-synthese.md` §§ 0 and 1.3; `mesures/genome/etat-de-l-art/journaux/` (`zero-compte-*`, `zero-canonique`, `temoin-*`, `diagnostic-L0-E0`, `choix-delta`, `kl-*`) and `passes/`; `mesures/genome/construction/synthese/apparie-eclaireur.json`; tools under `outils/genome/eclaireur/`.

In brief

A synonymous mutation changes a codon without changing the protein (Section 1.2). The scout looked, in the weights of gpt-oss, for the equivalent of this redundancy, keeping the analogy for what it is: an image that guides, not an argument. It found two. The first is in the format itself: MXFP4 has sixteen codes for fifteen values, because +0 and −0 are the same number; the sign of a zero is a piece of information that the computation never reads, like a part of the genome that codes for nothing, and removing it saves 3.6 % of the bytes without changing a single bit of the logits. The second is in the way of being wrong: a weight error in a direction that the real inputs of the expert do not excite is invisible at its output. Choosing the symbols of the base so as to store the error there costs the decoder nothing and divides the output error by three to four.

Tool	Role	Control (declared beforehand)	Result [meas.]
`capture/` (Rust, fork of `trc-reference`)	captures the real inputs of the experts and the routing, substitutes bases, rewrites a code	logits of the fork equal to those of `trc-reference` to the bit (24 tokens); break: YaRN without scaling	0 logits different; break: 4 826 112 (`temoin-fourche*.json`)
`synonyme.py`	the frontier base and the synonymous mutation; weight and output errors	T0: error of the frontier on the 12 tensors $= 0.042\,919\,745$ to within 10^{-6}; T1: $H = I$, $\lambda = 0$ reproduces the frontier; T2: $H_0 = 2.642$ to within 0.001	T0 0.042 919 745 124 98 (break 0.4246); T1 0 symbols different (with the real H: 1 789 132); T2 2.6415 (`temoins*.json`)
`zero-synonyme/` (Rust, fork of measurement F1)	real rANS coder, exact decoding, with or without $-0 \to +0$	without the flag, reproduces F1 to the bit	all quantities of F1 equal (A_ctx 3.843 137; `zero-temoin-sans-drapeau.json`)
`zero_compte.py`	exact counts of the ± 0	H(code) of the 12 tensors $= 3.860$ to within 0.001; break (code 1 counted as zero): the gap moves by more than 0.05	3.8598; break: 0.1369 → 0.3514
`borne_rd.py`	the bound $R(D)$ of Section 2.3	B1: the ends of the curve; B2: a Gaussian to within 0.05 bit	pass 1: B1 fails; B2 holds (10^{-5} bit); break fails; correction declared afterwards (Section 2.3)

Table 3.1: The instruments of the scout, their controls and their breaks [meas.]. Source: `genome/01-eclaireur.md` § 1; logs under `mesures/genome/etat-de-l-art/journaux/`.

3.1 The instruments and their controls

Each instrument has its control, declared before its first measurement, whose answer is known in advance, and its break, which forces the control to fail (Table 3.1). The logs are under `mesures/genome/etat-de-l-art/journaux/`, the tools under `outils/genome/eclaireur/`. Three names in the table come from Volume II: `trc-reference` is the exact reference, the program that computes gpt-oss in double precision (f64) to judge all the others; a *fork* is a copy of it modified for a need of the round (here, capturing the inputs of the experts, or substituting a base for them); YaRN [71] is the scaling of the position frequencies by which gpt-oss extends its context, which the break of the first control removes, to check that the bitwise comparison can fail.

The real inputs. The fork of the f64 reference captured, at each layer, the normalised state that enters the experts (in f32) and the routing, on wiki.test tokenised by `trc-jetons`; its first ten windows equal those of the `.kld` logits file of the R1 judgement of Volume II (checked by the tool, `journaux/capture-*.json`). The **calibration** is made of windows 10 to 21 (6 132 positions), the **held-out evaluation** of windows 0 to 9 (5 110 positions, those of R1). Each position sends its input to 4 experts out of 32 per layer: on average 766 calibration inputs and 639 evaluation inputs per expert [meas.]. This number, small compared with the dimension 2 880, decides the first failure of Section 3.4.

	+0 (code 0)	−0 (code 8)	H(sign | zero)	H(code)	H(value)	gap
gpt-oss-20b, 72 tensors	6.3920 %	6.3929 %	0.999 999 996 7	3.8638	3.7360	**0.1278**
12 reference tensors	6.8464 %	6.8475 %	0.999 999 99	3.8598	3.7228	0.1369
gpt-oss-120b, 108 tensors	6.2779 %	6.2789 %	0.999 999 995 1	3.8667	3.7411	**0.1256**

Table 3.2: The sign of the zeros in the MXFP4 codes of the experts [meas.]. Entropies in bits per weight (the sign: in bits per zero). 20b: 19 110 297 600 weights; 120b: 114 661 785 600. The gap H(code) − H(value) is what a lossless coder pays for a sign that the computation does not see. Data: `mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`, `zero-compte-120b.json`, `zero-compte-20b-12.json`.

3.2 Where the noise is, where the structure is

3.2.1 The sign of the zeros: pure noise

The exact counts of the +0 (code 0) and of the −0 (code 8) over all the expert tensors of the two models give Table 3.2 and Figure 3.1. On gpt-oss-20b, 12.8 % of the codes are zeros, half +0, half −0; the entropy of the sign of a zero is 0.999 999 996 7 bit, within $3 \cdot 10^{-9}$ of a perfect coin [meas.].

Figure 3.1 shows the gap tensor by tensor: it ranges from 0.117 to 0.196 bit per weight on the 20b, from 0.115 to 0.239 on the 120b, highest at layer 0, and nowhere goes below 0.11 bit. The "whole model" line is drawn under the bars: a first version placed it on top with a white halo that hid the top of 37 bars, and the judge of the third pass of the figures made it fail (GJ1, `memoire/tome3/JUGEMENT-figures3.md` § 6) before it was redrawn (commit `bb01679`). No block has all its 32 codes zero (0 out of 597 196 800 for the 20b, 0 out of 3 583 180 800 for the 120b): the scale of a block is never itself a synonym.

Why the sign of a zero is random [calc.]. An MXFP4 quantiser rounds to nearest. An original weight of small magnitude becomes ±0 according to its own sign, and the signs of the small weights are equiprobable. The format keeps this sign because E2M1 codes the sign separately [8]; the computation never sees it, since $K[8] = -0 = K[0]$. It is information inherited from the quantisation, the sign of a BF16 weight that no longer exists, without any effect on the value.

3.2.2 The energy is concentrated

The metric of the frontier is the *pooled* relative error on the weights. Yet the energy of the 12 reference tensors is concentrated in a few block exponents (Table 3.3): the blocks of exponent 129 carry 54.8 % of the energy for 5.3 % of the weights; 129 and 130 together, 74.2 % for 5.8 %; the eight heaviest exponents, 96.9 % for 14.6 % [meas.] [calc.].

The consequence is a warning that the whole round verified at its own expense. A rate allocation that served only the heavy blocks would "beat" the frontier in the pooled metric while destroying 85 % of the weights. **The metric of the weights is not a judge.** Hence the judge of Section 3.4 (the output error of the experts on held-out inputs), then that of Section 3.5 (the end-to-end KL).

3.2.3 The bound of a memoryless coder

The third look is the rate–distortion bound of Section 2.3: at the error of the frontier, an ideal coder that looks only at the weights would read at most 2.168 bits, that is 0.476 bit less; this gain is in the shape of the quantiser, not in a structure of the weights. The three looks say the same thing: seen by

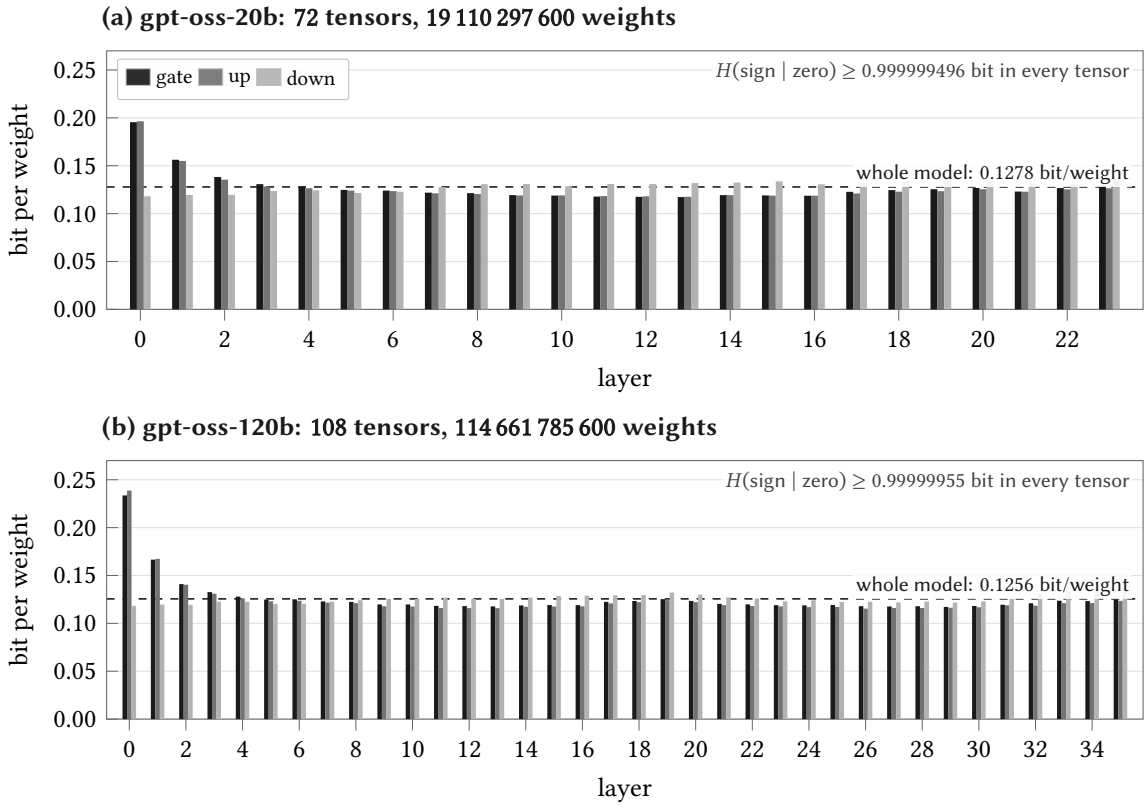


Figure 3.1: The sign of the zeros is a fair coin that costs an eighth of a bit. [meas.] For each expert tensor, the gap $H(\text{code}) - H(\text{value})$, from exact counts of +0 (code 0) and −0 (code 8): what a lossless coder pays to keep a sign that the computation never sees ($K[8] = -0 = K[0]$). Over gpt-oss-20b the zeros are 6.3920% +0 and 6.3929% −0 of the weights, and $H(\text{sign} \mid \text{zero}) = 0.9999999967$ bit; per tensor the gap goes from 0.117 to 0.196 bit (20b) and from 0.115 to 0.239 (120b), highest in layer 0. No block has its 32 codes at zero (0 of 597 196 800; 0 of 3 583 180 800): the scale is never a synonym. What the figure does not show: the attention and router tensors (not MXFP4), and the coded rate once −0 is merged (Chapter 9).

Data: `mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`, `zero-compte-120b.json` (`par_tenseur`); script `memoire/outils/tome3_eclaireur.py`.

rank	exponent e	share of weights	share of energy	cumulative energy
1	129	5.29 %	54.80 %	54.8 %
2	130	0.47 %	19.37 %	74.2 %
3	128	3.44 %	8.90 %	83.1 %
4	127	5.37 %	3.48 %	86.6 %
5 to 8	133, 134, 131, 136	0.01 %	10.37 %	96.9 %

Table 3.3: The energy of the weights of the 12 reference tensors, by E8M0 block exponent, ranked by decreasing energy [meas.] [calc.]. Energy of an exponent: number of weights $\times\, 4^{e-128}$, relative to the total (the law of the codes is the same from one exponent to another to first order). Data: `mesures/genome/etat-de-l-art/journaux/borne-rd-passe1.json`, field `exposants` (3 185 049 600 weights); the proposed cumulative figure (III-2.2) is not drawn.

itself, an MXFP4 weight is noise around its scale. The two mechanisms that follow therefore add no structure to the weights: the first removes what the computation does not read, the second moves the error to where the inputs do not look.

3.3 Mechanism 1: the zero-synonym codon

3.3.1 The mechanism

Before entropy coding, each nibble 8 (−0) becomes 0 (+0). The decoder does not change: it returns the canonical code, and the table of the stream no longer has a frequency for 8 (in the byte alphabet of A_ctx, the 256 symbols become 225 possible ones, 15×15). The cost of transcription is zero. The analogy is that of non-coding DNA: a part of the file, 0.128 bit per weight, carries a sign that nobody reads.

3.3.2 Why the products do not change

Proposition 3.1 (Bit exactness of a dot product after $-0 \to +0$)**.** *Let $y = \mathrm{fl}(\cdots \mathrm{fl}(\mathrm{fl}(t_1 + t_2) + t_3) \cdots + t_n)$ be an IEEE 754 sum with round-to-nearest [72], in a fixed order, of terms $t_j = \mathrm{fl}(w_j x_j)$ with finite x_j, and w' equal to w except $w'_j = +0$ where $w_j = -0$. If at least one term t_j is non-zero, then $y' = y$ to the bit. If all are zero, $y, y' \in \{+0, -0\}$ and $y = y'$ in value.*

Proof. We have $t'_j = t_j$ if $w_j \neq -0$, and $t_j, t'_j \in \{+0, -0\}$ otherwise, since $(\pm 0) \cdot x_j = \pm 0$ for finite x_j. Let S_k and S'_k be the partial sums. Induction hypothesis: $S_k = S'_k$ to the bit, or else $S_k, S'_k \in \{\pm 0\}$. It holds for $k = 1$. At step $k + 1$, three cases. (i) If $t_{k+1} = t'_{k+1}$ is non-zero: when $S_k = S'_k$, the operands are equal and so is the sum; when $S_k, S'_k \in \{\pm 0\}$, $\mathrm{fl}(\pm 0 + t) = t$ exactly for $t \neq 0$, so $S_{k+1} = S'_{k+1} = t_{k+1}$. (ii) If $t_{k+1}, t'_{k+1} \in \{\pm 0\}$ and $S_k = S'_k \neq 0$: $\mathrm{fl}(S \pm 0) = S$ on both sides. (iii) If both partial sums and both terms are zero, both sums stay in $\{\pm 0\}$: only the sign of a zero can differ, since $(+0) + (-0) = +0$ but $(-0) + (-0) = -0$. At the first non-zero term, (i) joins the two sequences again, and (ii) keeps them equal afterwards. If no term is non-zero, only (iii) applies. ∎

In an integer kernel (E2M1 values doubled into signed integers), −0 and +0 are already the same integer. The case of a product of 2 880 terms all zero does not occur in gpt-oss, where no block is zero; if it did, only the sign of an output zero would change, then disappear at the non-zero bias of the expert. For the f64 reference, the same induction applies to each of its eight accumulators, then to their tree reduction.

The proposition leaves open the sign of a zero partial sum. *Proposition A1* of Chapter 9 closes it for the GEMV kernel of the engine: an accumulator that starts from +0 and advances by FMA with round-to-nearest is never −0 (an exact zero sum of two terms that are not both −0 rounds to +0), so that a step on a zero code is the same in both computations, and bit equality holds at each step, even for an entirely zero sum. The proposition of this chapter suffices for the reference; A1 is what the engine needs.

3.3.3 The measurements

The threshold was declared before the pass, at the head of `outils/genome/eclaireur/zero-synonyme/src/main.rs`: the codon beats the frontier if A_ctx falls by at least 0.05 bit per weight. Table 3.4 gives the measurements, all by the real rANS coder of F1, each stream decoded exactly.

Measurement (12 reference tensors)	without the codon (F1)	with	gap
A_ctx, everything included (bits per weight)	3.843 137	**3.706 142**	−0.136 995 (−3.56 %)
A_ctx codes alone	3.805 616	3.668 621	−0.136 995
A_flux (format of study 03)	3.843 442	3.706 406	−0.137 036
mean chunk (20b, aligned on 4 KiB)	11 990 368 B	11 564 160 B	−3.55 %
base alone (level 0 of B)	2.606 647	2.559 045	−0.047 602
nibbles rewritten	—	218 097 443 out of 3 185 049 600	6.85 %

Table 3.4: The zero-synonym codon, measured by the real rANS coder [meas.]. Data: `mesures/genome/etat-de-l-art/journaux/zero-canonique.json` and `zero-temoin-sans-drapeau.json`; `genome/01-eclaireur.md` § 4.3. The coder recovers the entropy gap to within 0.000 04 bit (0.136 995 against 0.1369).

Measured: 0.137 bit, for a threshold of 0.05. The zero-synonym codon beats the frontier losslessly. On the 72 expert tensors of the 20b, the entropy gap is 0.1278 bit; the next chapter of the round also measures it with the real codec on the two whole models (Section 6.1).

The proof to the bit. On 24 tokens of the f64 reference, with the 759 416 633 −0 nibbles rewritten as +0 in all the experts read, **0 logits out of 4 826 112** differ to the bit, maximal absolute gap 0 (`journaux/temoin-zero.json`); the break makes all 4 826 112 differ (`temoin-zero-casse.json`). End to end on the 2 550 positions of the ten windows of R1, the logits file of the canonical model is **identical byte for byte** to that of the reference: same SHA-256 fingerprint, `d97fc421…5362a38` (`journaux/kl-zero-canonique.sha256`) [meas.].

On gpt-oss-120b [calc.]. At 0.1256 bit per weight, the 114 661 785 600 expert weights make

$$114\,661\,785\,600 \times 0.1256/8 = 1.800 \cdot 10^{9} \text{ bytes} :$$

1.80 GB that no computation reads, out of the 60.9 GB of the lossless file (03), that is −3.0 %. The real coder was run by the scout only on the 20b; Chapter 6 ran it on the whole 120b.

The roof [calc.]. The expert bytes read per token fall by 3.56 %; when the disk limits, the roof (2.1) rises by $1/(1-0.0356)-1 = 3.7$ %, without changing anything else. Chapter 9 measures what the engine makes of it on Stone's disk, and why the measured number is not that one.

3.3.4 The same codon in the base

In the base (sign and class), a zero keeps its sign: the class {0, 0.5} is rebuilt as $\pm 0.237\,s$. On the canonical code, this sign is no longer stored (−0.0476 bit, Table 3.4); the decoder can *draw it from a hash of the position* (`splitmix64` of $rC + c$) without reading a bit. The error on the weights is the same to the weight, since $|0 - (\pm 0.237\,s)|$ does not depend on the sign. A Weyl sequence as hash fails (neighbouring rows almost equal); `splitmix64` holds (correlation 0.01, at the noise level). The *output* error of this variant has not been measured: its pass was stopped after 4 experts (`passes/frontiere0-ARRETEE-4-experts.log`). It remains "equal in weight error at −0.048 bit, not judged on the output".

3.4 Mechanism 2: the synonymous mutation

3.4.1 Life, and what is transposed from it

The genetic code is not an arbitrary function from codons to amino acids: among a million possible codes, the natural code is one of the best at making the probable errors fall on similar amino acids [5]. The neuron, for its part, codes a continuous signal in spikes by carrying its error over to what follows, in the manner of a sigma-delta modulator [55]. Transposed, as an analogy: the "phenotype" of an expert matrix is its output Wx on the inputs the model really gives it; a weight error in a direction that these inputs do not excite is a synonymous mutation. The mechanism chooses the symbols of the base so that the error goes there.

3.4.2 The derivation

The output error. For a matrix W of R rows and C columns, its inputs x and the router weight g of the expert, the output error of a reconstruction $\hat{W}$ is, by (2.8),

$$\mathbb{E}\big[g^2\|(W-\hat{W})x\|^2\big] = \mathbb{E}[g^2]\,\operatorname{tr}\big((W-\hat{W})\,H\,(W-\hat{W})^\top\big), \qquad H = \frac{\mathbb{E}[g^2 xx^\top]}{\mathbb{E}[g^2]}. \tag{3.1}$$

The trace is a sum over the rows: the rows of W are treated independently. For a row w, we seek $\min_q (w-q)^\top H (w-q)$, q ranging over the reconstructions allowed column by column, $q_j = \pm c_k\, s_{b(j)}$, c_k one of the four centres of the base and $s_{b(j)}$ the E8M0 scale of the block of column j.

Lemma 3.2 (Optimal error feedback)**.** *Let w be the current row (error already carried over included), j a column and F the set of columns still free, $j \in F$, H_F the submatrix of H on F. If column j is fixed to q_j and the other columns of F are left to adjust, the minimal increase of the squared error $\delta^\top H_F \delta$ is*

$$\frac{(w_j-q_j)^2}{[H_F^{-1}]_{jj}}, \qquad \text{reached by } \delta = -\frac{w_j-q_j}{[H_F^{-1}]_{jj}}\, H_F^{-1} e_j, \tag{3.2}$$

where δ is the modification of w on F; on the free columns other than j, the correction is $\delta_{F\setminus j} = -(w_j-q_j)\,[H_F^{-1}]_{F\setminus j,\,j}/[H_F^{-1}]_{jj}$.

Proof. We minimise $\delta^\top H_F \delta$ under the constraint $e_j^\top(w+\delta) = q_j$, that is $e_j^\top \delta = q_j - w_j$. The Lagrangian is $L(\delta,\mu) = \delta^\top H_F \delta + \mu\,(e_j^\top\delta - (q_j-w_j))$. Its gradient in δ, $2H_F\delta + \mu e_j$, vanishes at $\delta = -\frac{\mu}{2} H_F^{-1} e_j$. The constraint gives $-\frac{\mu}{2}\,[H_F^{-1}]_{jj} = q_j - w_j$, that is $\frac{\mu}{2} = -\frac{q_j-w_j}{[H_F^{-1}]_{jj}}$, hence $\delta = \frac{q_j-w_j}{[H_F^{-1}]_{jj}} H_F^{-1} e_j$, which is the stated expression. The increase is $\delta^\top H_F \delta = \frac{(q_j-w_j)^2}{[H_F^{-1}]_{jj}^2}\, e_j^\top H_F^{-1} H_F H_F^{-1} e_j = \frac{(q_j-w_j)^2}{[H_F^{-1}]_{jj}}$. H_F being positive definite, it is a minimum. This is the Optimal Brain Surgeon lemma [40], applied to quantisation by OBC and GPTQ [50, 51]. ∎

The sequential rule, through a single factor. In the order of the columns $1,\dots,C$, the free set at step j is $F_j = \{j,\dots,C\}$. The quantities $[H_{F_j}^{-1}]_{jj}$ and $[H_{F_j}^{-1}]_{F_j j}$ are read in the upper Cholesky factor U of H^{-1} ($U^\top U = H^{-1}$): $[H_{F_j}^{-1}]_{jj} = U_{jj}^2$, and the correction of the following columns is $-\frac{w_j-q_j}{U_{jj}}\,U_{j,F_j}$ (GPTQ). The factor is computed by a single factorisation.

Lemma 3.3 (The Cholesky factor of H^{-1} by reversal)**.** *Let P be the permutation that reverses the order of the columns. If $PHP = LL^\top$, L lower triangular with positive diagonal, then $U = PL^{-1}P$ is upper triangular with positive diagonal and $U^\top U = H^{-1}$: it is the upper Cholesky factor of H^{-1}.*

Proof. $P = P^\top = P^{-1}$. From $PHP = LL^\top$ comes $H = (PLP)(PLP)^\top = VV^\top$ with $V = PLP$, *upper* triangular (reversing the order of the rows and columns of a lower triangular matrix makes it upper triangular). Then $H^{-1} = V^{-\top}V^{-1} = U^\top U$ with $U = V^{-1} = PL^{-1}P$, upper triangular as the inverse of an upper triangular matrix, with diagonal $1/L_{C+1-i,C+1-i} > 0$. The Cholesky factor with positive diagonal is unique: it is U. ■

The cost is $n^3/3$ (Cholesky) plus $n^3/3$ (triangular inverse), instead of n^3 (inverse) plus $n^3/3$ [calc.]; the tool checked it against the direct computation, relative gap $7 \cdot 10^{-14}$ (`outils/genome/eclaireur/synonyme.py`, `facteur_inverse`) [meas.].

The entropy constraint. The symbol k of column j minimises

$$\left(|w_j| - c_k\, s_{b(j)}\right)^2 + \lambda\, \bar{c}\, U_{jj}^2\, \ell(k), \qquad \ell(k) = -\log_2 p(k), \tag{3.3}$$

p being the law of the symbols of the frontier base of the tensor, $\bar{c}$ the mean error per weight of the frontier in the units of H. Divided by U_{jj}^2, this is, by (3.2), "increase of the output error + $\lambda\,\bar{c}$ bits": λ is the price of a bit in units of frontier error. With $\lambda = 0$, it is feedback alone.

The decoder does not change. Same eight symbols, same four centres, same scales: the mechanism lives entirely in the coder, and its cost of transcription is zero.

Proposition 3.4 (The control that can fail: $H = I$ reproduces the frontier)**.** *If $H = I$ and $\lambda = 0$, the synonymous mutation chooses, for each weight, the symbol of the frontier base.*

Proof. With $H = I$, $H^{-1} = I$ and $U = I$: no correction is carried over, and each weight chooses the centre nearest to $|w_j|/s_{b(j)}$, with the sign of w_j. The centres are the centroids of a *contiguous* partition of the magnitudes, {0, 0.5}, {1, 1.5, 2}, {3, 4}, {6}, with centres 0.237; 1.424; 3.419; 6. The Voronoi boundaries, midpoints of two neighbouring centres, are 0.831; 2.421; 4.709 [calc.]. Each magnitude falls into the cell of its class: $0.5 < 0.831 < 1$, $2 < 2.421 < 3$, $4 < 4.709 < 6$. The symbol chosen is therefore that of the frontier function. ■

Measured (control T1): 0 symbols different out of 8 294 400; with the real H, 1 789 132 [meas.].

3.4.3 The protocol

Experts: layers 0, 8, 16 and 23, experts 0, 4, ..., 28 (96 matrices, 796 million weights); later passes on subsets declared before them. Calibration: windows 10 to 21; held-out evaluation: windows 0 to 9; output errors weighted by g^2. The *expert* output error replaces the three matrices at once, with the bounded SwiGLU of gpt-oss between them. **Criterion, declared beforehand** (head of `synonyme.py`): rate at most that of the frontier, with the same estimator, *and* expert output error lower on each of the two halves of the evaluation and in each of the four layers.

3.4.4 The first failure, and its reason: overfitting

The declared pass ($\lambda = 0$, $\delta = 0.01$, the damping of GPTQ: $H + \delta\,\overline{\operatorname{diag} H}\, I$) **does not beat** the frontier: rate 2.6789 bits (H_0) against 2.6381, and, on the evaluation, layer 0 is worse (0.094 and 0.067 expert output error, halves 1 and 2, against 0.059 and 0.052; `passes/synonyme-lam0-damp0.01.json`) [meas.].

The diagnosis, with hypothesis and predictions declared beforehand (`journaux/diagnostic-L0-E0.json`), explains it by a number. Each expert receives only about 766 calibration inputs in

Gate matrix, layer 0, expert 0	calibration inputs	output error, calibration	output error, evaluation
frontier base	—	3.40 %	3.49 %
synonym, $\delta = 0.01$	762	0.21 %	4.38 %
synonym, $\delta = 0.1$	762	0.28 %	3.27 %
synonym, $\delta = 1$	762	0.72 %	2.74 %
synonym, $\delta = 10$	762	1.70 %	2.82 %
synonym, $\delta = 0.01$, half calibration	381	1.61 %	4.01 %
synonym, $\delta = 1$, half calibration	381	1.39 %	2.82 %

Table 3.5: The diagnosis of overfitting, gate matrix of expert 0, layer 0 (499 evaluation positions) [meas.]. Data: `mesures/genome/etat-de-l-art/journaux/diagnostic-L0-E0.json` (the proposed figure III-2.3 is not drawn).

dimension 2 880: the estimated covariance has rank at most 766, and the feedback pushes the error into the 2 100 or so directions that the sample does not see, but that the evaluation excites. Table 3.5 shows it on the gate matrix of expert 0, layer 0.

Predictions (1) and (2) hold: the gap between calibration and evaluation exists, and it narrows as δ grows. Prediction (3) **fails**: with half the inputs, the evaluation does not get worse (4.01 % against 4.38 % at $\delta = 0.01$; 2.82 % against 2.74 % at $\delta = 1$). The defect is therefore not first the number of inputs, it is the lack of regularisation of H: at $\delta = 0.01$, the feedback trusts badly estimated directions, whatever the size of the sample in this range. Over the 96 matrices, the median ratio of synonym to frontier is 0.044 in calibration and 0.84 in evaluation. *The synonym of a sample is not the synonym of the law.*

Choosing δ without looking at the evaluation. Procedure declared beforehand (`journaux/choix-delta.json`): expert 2 of the four layers, outside the measured experts; the calibration cut in two (windows 10 to 15 to fit, 16 to 21 to judge); $\delta \in \{0.1,\ 0.3,\ 1,\ 3,\ 10\}$. Mean ratio of output error, synonym to frontier: 0.623; 0.563; **0.538**; 0.553; 0.589 [meas.]. Hence $\delta^* = 1$: $H + \overline{\operatorname{diag} H}\, I$, the measured covariance pulled towards isotropy with equal weight.

3.4.5 The results

$\lambda = 0$. The output error falls everywhere (Table 3.6), but the rate rises by 0.011 bit: under the strict declared criterion (rate at most that of the frontier), $\lambda = 0$ does not yet win. Hence the entropy constraint.

$\lambda = 1$. Under the criterion declared beforehand (Table 3.7), **8 cells out of 8** (two halves, four layers), at 0.078 bit less: **the synonymous mutation beats the lossy frontier**. The thinnest margin is layer 0 (×0.97 on half 1).

What this says about the structure. The error on the weights *rises* (from 4.3 to 4.9 %) while the output error is divided by three to four. The exploitable structure is not in the weights, it is in the *inputs*; the weights are noise seen by themselves, and a base whose error is stored in the silent directions of the inputs costs less. This is the conclusion of Section 2.6, measured.

	Frontier	Synonym $\lambda = 0$	Ratio
rate H_0 of the symbols (bits)	2.6381	2.6488	+0.011
rate $H(\text{symbol} \mid \kappa)$	2.5953	2.6079	+0.013
relative error on the weights (gate, up, down)	4.40 / 4.25 / 4.40 %	4.71 / 4.55 / 4.76 %	×1.07
output error of the matrices, evaluation	3.16 / 3.60 / 2.89 %	1.72 / 1.99 / 2.21 %	median ×0.59
expert output error, halves 1 / 2	3.03 / 2.53 %	**0.84 / 0.65 %**	×0.28 / ×0.26
layer 0	5.85 / 5.19 %	5.10 / 4.12 %	
layer 8	13.57 / 15.19 %	5.96 / 6.62 %	
layer 16	7.87 / 8.47 %	3.65 / 3.69 %	
layer 23	3.01 / 2.51 %	0.83 / 0.64 %	
experts improved	—	32 out of 32	median ×0.67

Table 3.6: The synonymous mutation, $\lambda = 0$, $\delta^* = 1$, 8 experts per layer [meas.]. Data: `mesures/genome/etat-de-l-art/passes/synonyme-lam0-damp1.json` and `.csv`.

	Frontier (same experts)	**Synonym** $\lambda = 1$	Gap
rate H_0 of the symbols (bits)	2.6415	**2.5634**	**−0.078**
rate $H(\text{symbol} \mid \kappa)$	2.5989	2.5232	−0.076
rate read, H_0 + scales (0.0375)	2.679	**2.601**	−0.078
relative error on the weights	4.26 / 4.20 / 4.44 %	4.83 / 4.59 / 5.20 %	higher
output error of the matrices, evaluation	3.18 / 3.48 / 2.72 %	1.78 / 1.94 / 2.09 %	median ×0.60 (43 out of 48)
expert output error, halves 1 / 2	3.94 / 3.14 %	**1.18 / 0.74 %**	×0.30 / ×0.24
layer 0	5.50 / 4.87 %	5.32 / 4.29 %	×0.97 / ×0.88
layer 8	26.80 / 25.11 %	10.70 / 10.44 %	×0.40 / ×0.42
layer 16	4.92 / 5.14 %	2.98 / 3.11 %	×0.61 / ×0.61
layer 23	3.92 / 3.12 %	1.17 / 0.73 %	×0.30 / ×0.23
experts improved	—	14 out of 16	median ×0.73, worst ×1.10
exact complement $H(\text{code} \mid \text{symbol})$	1.219	1.689	+0.47

Table 3.7: The synonymous mutation, $\lambda = 1$, $\delta^* = 1$, experts 0, 8, 16 and 24 of each layer (subset declared before the pass), against the frontier redone on the same experts [meas.]. Data: `passes/synonyme-lam1-damp1-e0_8_16_24.json`, `passes/frontiere-lam0-damp0.01-e0_8_16_24.json`.

The price, in one number. The exact complement behind the base costs 1.689 bits instead of 1.219: the base is no longer a function of the code ($H(\text{base} \mid \text{code}) > 0$). Read in two levels, the exact expert would cost 2.564 + 1.689 = 4.25 bits instead of 3.86 [calc.]: *the synonymous mutation is not a level 0 of the base + complement split.* Its use is that of a **second genome**: an exact file (the canonical code, 3.706 bits) and a separate synonym base (2.60 bits read), the disk paying for the room of both (about 6.3 bits per weight stored), the engine reading only one of them per expert miss. This is the "fast mode and exact mode" of Stone's idea; Chapter 10 builds it.

The aberrant expert. Layer 8, expert 24: modest matrix errors (frontier from 2.4 to 3.9 %), but an expert output error of **67.6 %**; synonym $\lambda = 0$: 18.9 % [meas.]. The output of this expert is small compared with its intermediate terms (compensation between the SwiGLU and down): a small relative error upstream becomes large downstream. It weighs heavily in layer 8 and is the first candidate to remain exact.

ECSQ without feedback, and the 1-bit lead. An entropy-constrained scalar quantiser without feedback ($\lambda = 0.5$) slides along the curve: 2.615 bits (H_0), expert output error 3.18 / 2.72 % against 3.03 / 2.53; it does not win (`passes/ecsq-lam0.5-damp0.01.json`). The synonymous mutation with one magnitude bit (4 symbols, about 1.96 bits) was launched then *stopped after 2 experts* to leave the machine to the KL: 20.5 and 19.5 % expert output error (layer 0, experts 0 and 8) against 25.7 and 25.1 % for the 1-bit frontier on the same experts (`passes/synonyme1-ARRETEE-2-experts.log`). Two experts do not make a measurement: it must be redone in full.

3.5 The end-to-end judgement: the KL against the f64 reference

The judge, in five sentences. At each position t of a text, a language model returns a probability law over the next token; let p_t be that of the f64 reference of lot R of Volume II (the exact model, computed in double precision) and q_t that of the model in which experts have been replaced. The **KL of the position** is the Kullback–Leibler divergence [11]

$$\mathrm{KL}_t = \sum_i p_{t,i} \log \frac{p_{t,i}}{q_{t,i}} \quad \text{(in nats)}, \tag{3.4}$$

zero when the two laws are equal, and otherwise equal, up to the cut term, to the mean extra cost, in nats per token, of predicting with q_t a text drawn from p_t. The instrument `juge kl` sums only the tokens i such that $\log p_{t,i} > -16$: those to which the reference gives less than $e^{-16} \approx 1.1 \cdot 10^{-7}$ [calc.] are left aside, everywhere and for all models; without renormalisation, the sum so cut can be very slightly negative. The **KL of a pass** is the mean of KL_t over the scored positions; here, over the ten evaluation windows of wiki.test that Volume II calls R1, that is 2 550 positions (255 per window). It is the judge of the whole volume: an error on the weights or at the output of an expert is only a clue to it, and Definition 8.1 takes it up again with its two parts, the smooth part and the tail of routing flips (Section 8.1).

The protocol (P1 of the round). The `kl` subcommand of the fork, then `juge kl` of lot R of Volume II: the 10 windows of R1 (2 550 scored positions), the f64 reference, experts 0, 4, …, 28 of layers 0, 8, 16 and 23 replaced by their base, that is **32 experts out of 768**, each served at each position that routes it, against `ref-openai-20b.kld`. The divergence [11] is read position by position, in nats;

2 550 positions, 32 experts out of 768 replaced	Frontier base	**Synonym** $\lambda = 0$	Gap
mean KL (± uncertainty of the judge)	0.0360 ± 0.0029	**0.0262 ± 0.0026**	−27 %
median KL	0.0088	**0.0038**	−56 %
KL, 90th percentile	0.061	0.037	−40 %
KL, 99th percentile	0.495	0.410	−17 %
KL, 99.9th percentile	1.59	1.70	+7 %
identical first choice (Wilson 95 %)	87.1 % [85.8, 88.4]	**90.9 % [89.7, 91.9]**	disjoint
perplexity (reference 460.2)	485.9 (+5.4 %)	**468.3** (+1.8 %)	÷3

Table 3.8: The end-to-end KL against the f64 reference (protocol P1) [meas.]. Data: `mesures/genome/eta t-de-l-art/journaux/kl-frontiere-juge.json`, `kl-synonyme-d1-juge.json`; position by position: `kl-*-positions.csv`.

the identical first choice is given with its Wilson interval [73],

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad z = 1.96; \tag{3.5}$$

for $\hat{p} = 0.8714$ and $n = 2\,550$, it gives $[0.8578,\ 0.8838]$ [calc.], the interval that the log carries. Threshold, declared beforehand: the synonymous mutation is better end to end if its mean KL is below that of the frontier *and* its rate of identical first choice is above.

Which variant was judged. The pass was written before the result of $\lambda = 1$: it judges the variant $\lambda = 0$, $\delta^* = 1$, at rate +0.011 bit, whose output errors are of the same order. The KL was measured on the variant $\lambda = 0$, at 2.649 bits; $\lambda = 1$ (2.563 bits) was judged only by the expert output error. The synthesis of the round had to correct an attribution of this KL to $\lambda = 1$ (Section 8.2). **No variant of the synonymous mutation is therefore measured both below the frontier in bits and below it in KL.**

The declared threshold holds (Table 3.8): lower mean KL, higher first choice, disjoint Wilson intervals. The scout had compared the two means without pairing them (gap of 0.0098, about 3.6 times the uncertainty of a mean). The synthesis paired them position by position, from the committed CSVs (`mesures/genome/construction/synthese/apparie-eclaireur.json`) [calc.] on [meas.]. Its control, declared beforehand, recomputes each mean KL to within 10^{-12} (0.036 013 763 948 on both sides, gap $1.8 \cdot 10^{-13}$) and its break (a line removed) fails. The paired difference, synonym minus base, is −0.009 845; naive standard deviation 0.001 90 (5.2 σ), standard deviation by blocks of windows, which respects the correlation of the positions of a same window (they share their past, hence their flips), **0.003 34, that is 2.9** σ. 2 060 positions out of 2 550 (80.8 %) have less KL with the synonymous mutation; the identical first choice rises by 3.73 ± 0.59 points. The derivation of the two uncertainties is in Chapter 11.

What the tail says. The 99.9th percentile does not fall (1.59 → 1.70). The positions most affected are those where the routing flips (the R1 judgement of Volume II: routing flips are the first amplifier of a perturbation), and a better base does not remove them. Chapter 8 makes them the second part of the KL, and Chapter 12 measures it.

Mechanism	Bits per weight read	Error	Verdict at the declared threshold
lossless frontier (F1)	3.8431	0	reference
zero-synonym codon	**3.7061** (real rANS, 12 tensors); −0.128 on the whole 20b, −0.126 on the whole 120b	**0**: bit-identical logits; `.k1d` equal byte for byte	**beats the lossless frontier** (threshold 0.05)
frontier base (07, F1)	2.6441 (rANS); 2.679 (H_0 + scales)	weights 4.29 %; expert output 3.9 / 3.1 % (16 experts)	reference
frontier base on canonical code, hashed sign	2.5965 (real rANS)	weights 4.29 %; output not measured	equal in weights at −0.048 bit; not judged
ECSQ $\lambda = 0.5$, without feedback	2.615 (H_0)	expert output 3.18 / 2.72 % against 3.03 / 2.53	does not win
synonym $\lambda = 0$, $\delta = 0.01$ (declared)	2.679 (H_0)	expert output 1.03 / 0.79 %; layer 0 worse	**does not win**
synonym $\lambda = 0$, $\delta^* = 1$	2.649 (H_0)	expert output 0.84 / 0.65 %; KL 0.0262 against 0.0360	wins in error and in KL; not under the strict criterion (+0.011 bit)
synonym $\lambda = 1$, $\delta^* = 1$	**2.563 (H_0),** −0.078	**expert output 1.18 / 0.74 % against 3.94 / 3.14 %**, 8 cells out of 8; KL not measured	**beats the lossy frontier**

Table 3.9: The verdict of the scout [meas.]. Source: `genome/01-eclaireur.md` § 7, with the correction of `genome/05-synthese.md` § 0 (the KL belongs to $\lambda = 0$).

What the KL says about the frontier itself. Replacing 32 experts out of 768 (4 % of the experts, in 4 layers out of 24) by the 2.64-bit base already costs a KL of 0.036, six times the threshold of architecture B (0.0055, fondations/07 § 4.2); the synonymous mutation brings it down to 0.026, still four times above. The base, even a synonym one, is a **fast mode** or a **draft**, not a silent replacement for the exact mode.

3.6 The verdict

Seen from the weights, gpt-oss is noise: codes independent to within 0.005 bit, sign of the zeros at 1.0000 bit of entropy, and a rate–distortion bound that only a better quantiser or a vector code approaches, with nothing to "generate". The two mechanisms that win (Table 3.9) add no structure to the weights: the first **removes information that the computation never reads**, the second **moves the error to where the real inputs do not see it**. Both are, in the sense of the analogy, synonyms: differences of code without difference of phenotype. The rest of the round took up both: the first enters the format (Chapter 9), the second becomes a second file (Chapter 10).

What could be wrong

- **The zero codon and the fingerprint of the experts.** The `.trc` format checks the fingerprint of each decoded expert against the bytes of the GGUF; a canonical expert has other bytes.

A fingerprint of the canonical code is needed: a decision of format, which Chapter 9 takes.

- **Bit exactness** is proved for an IEEE dot product in fixed order and measured on the f64 reference (24 tokens at the logit, 2 550 positions at the fingerprint of the `.k1d`); for the kernels of the engine, it is proposition A1 of Chapter 9 and its measurement.
- **The subset.** 4 layers out of 24, 8 then 4 experts out of 32 per layer; the gain of the synonymous mutation varies a great deal from one layer to another (×0.97 to ×0.23); a layer where it would reverse is not excluded. The KL substitutes only 32 experts out of 768.
- **The rate estimator** of the synonymous mutation is the entropy (H_0 or $H \mid \kappa$), not a real coder. For the frontier, the real coder gives 2.6441 against 2.6066 + scales; nothing indicates that the gap is different for the synonym symbols, but it is not measured.
- δ^* was chosen on expert 2 of the four layers: other experts than those measured, but of the same layers.
- **The calibration and the evaluation** come from the same corpus (wiki.test), disjoint windows; an evaluation on another domain (code, dialogue) could reduce the gain.
- **The two comparisons** are made on different subsets ($\lambda = 1$ on 4 experts per layer, $\lambda = 0$ on 8), each against its own frontier.
- **The KL judges $\lambda = 0$, not $\lambda = 1$:** the 0.078-bit gain of $\lambda = 1$ is judged only by the expert output error, and the round showed that this error does not predict the KL (Chapter 8).
- **The bound of Section 2.3** assumes the codes independent given the scale; an unmeasured dependency would lower it.

Chapter 4

The genome of the weights: the recipe, the seed, S^*

Where we start, what this chapter establishes, what remains

The starting point is the frontier (3.8431 bits lossless; a base of 2.6441 bits at 4.2920 % error on the weights, on the 12 tensors of F1), codes almost independent given their scale, and Stone's instruction for this subject, on 2026-10-16: "le but est de concevoir un systeme fonctionnel et si il ne voye rien de convaincaint vers l'ADN au terme ils recherche le moyen biomimétique le plus prométeur jusqu'à trouver" (the goal is to design a working system, and if they see nothing convincing towards DNA in the end, they search for the most promising biomimetic means until they find it). The reading (Chapter 2) predicted that a recipe would gain nothing losslessly, and that with loss it would at best aim at the bound of the weights. **This chapter establishes** that the literal recipe (a seed and coefficients, in the manner of SeedLM) does not beat the frontier, and says so before the measurement through a lemma on the best of N random subspaces; that the same idea, brought to the dimension of a whole row and made searchable by the Viterbi algorithm (the *strand read through codons*), beats it on the 12 tensors in weight error, by 0.20 to 0.60 bit; that at the same point it loses in output error, and why (a table that clips the massive columns); that the corrected system, S^*, wins in expert output 8 times out of 8, but that its end-to-end KL is **equal** to that of the base. It also proves that no recipe followed by an exact complement goes below the entropy of the codes. **What remains** is the kernel on the card, the format of a "STRAND" stream kind, error feedback between columns combined with the strand, and the KL with all the experts coded. Sources: `genome/genome.md`; `genome/genome-pour-le-memoire.md`; `mesures/genome/genome/00-seuils-declares.md`, `csv/`, `kl/`, the logs `*.json`; `genome/05-synthese.md` §§ 1.2, 2.2, 2.4; tools under `outils/genome/recette/` and `outils/genome/recette-kl/`.

In brief

Messenger RNA is read three bases at a time, from a fixed starting point, without commas, and each codon is translated by a fixed and degenerate table (Section 1.2). The chapter takes this image seriously, as an image: a *strand* of bits stored for each row of a matrix, a 12-bit window that slides along the strand and returns a weight at each step through a table of 256 values, learnt on the tensor, as the genetic code is fixed for a species. The best strand is found exactly by dynamic programming. This codon reader wins where a short seed loses, because its dimension is unlimited. But it wins in weight error and loses at the output: its table clips the few giant columns that the real inputs read most. Once these columns are kept apart, it wins in expert output, and the divergence on text does not move. That is the lesson of the chapter, and of the round: every intermediate judge can be wrong, and only the text decides.

4.1 The question and its judge

A recipe "works" if it reads fewer bits per weight from the disk at equal error; the gain is read in the roof (2.1). The thresholds are in `mesures/genome/genome/00-seuils-declares.md`, committed before any measurement of a recipe (commit `e777d6d`, then the clarification of the sample, `2698ccd`, before the first pass).

Criterion W. On the sample (experts 0, 8, 16 and 24 of each of the 12 tensors; one row in four, 720 whole rows per expert, 34 560 rows, 99.5 million weights), at the relative error of the base *measured on the same rows*, the recipe reads fewer bits than the frontier F1 of the same tensor *on each of the 12 tensors* and globally. A recipe without effect would pass all 12 with probability $2^{-12} \approx 2.4 \cdot 10^{-4}$ (sign test) [calc.]. The error is $e = \sum(w - \hat{w})^2 / \sum w^2$ in the real domain of the weights, scale included, accumulated over the rows of the tensor: the measure of the frontier. Criteria O (output of a layer of experts), F (fidelity as a function of rate), K (KL), C (transcription) and L (exact complement) are declared in the same file; each is judged where it is measured.

4.2 The seed: the literal recipe, and why it cannot suffice

4.2.1 The mechanism

A block of C weights $w \in \mathbb{R}^C$ is approximated by $\hat{w} = U(g)\,t$, where $U(g)$ is a $C \times P$ matrix drawn from a pseudo-random generator by a seed g of K bits, and $t \in \mathbb{R}^P$ are quantised coefficients (4-bit two's-complement integers, a shared 4-bit exponent). The encoder tries the 2^K seeds, projects w onto each subspace (pseudo-inverse precomputed by Cholesky), quantises, keeps the best. Rate: $(K + 4 + 4P)/C$ bits per weight. This is the form of SeedLM's problem [15]; the code is written from scratch (`outils/genome/recette/`). Two generators: a 16-bit Galois LFSR ($x^{16} + x^{14} + x^{13} + x^{11} + 1$, period 65 535 checked by the test `periode_maximale`) and a counter-based bit mixer. Control T3: a block generated exactly by a seed and quantisable coefficients is recovered at zero error (`bloc_engendre_retrouve_exactement`).

4.2.2 What theory allows, before any measurement

Lemma 4.1 (The best of N random subspaces)**.** *Let w be a uniform direction on the sphere of $\mathbb{R}^C$ and V a uniform random subspace of dimension P, independent of w. The share of energy outside V, $\rho = \|P_{V^\perp} w\|^2 / \|w\|^2$, follows the law* Beta$(a, b)$ *with $a = (C - P)/2$ and $b = P/2$.*

Proof. By rotation invariance, one can fix $V = \mathrm{span}(e_1, \dots, e_P)$ and take $w = z/\|z\|$ with z standard Gaussian in $\mathbb{R}^C$. Then $\rho = \sum_{i>P} z_i^2 / \sum_i z_i^2 = X/(X+Y)$ with $X = \sum_{i>P} z_i^2 \sim \chi^2_{C-P}$ and $Y = \sum_{i \le P} z_i^2 \sim \chi^2_P$, independent. The ratio $X/(X+Y)$ of two independent chi-squares with $2a$ and $2b$ degrees of freedom has, by definition, the law Beta(a, b). ∎

Corollary 4.2 (The expected minimal residual)**.** *For $N = 2^K$ independent subspaces, with residuals $\rho_1, \dots, \rho_N$, and F the distribution function of* Beta(a, b)*,*

$$\mathbb{E}\big[\min_i \rho_i\big] = \int_0^1 \big(1 - F(\varepsilon)\big)^N \,\mathrm{d}\varepsilon \;\approx\; \Gamma\Big(1 + \frac{1}{a}\Big)\Big(\frac{a\,B(a,b)}{N}\Big)^{1/a}. \tag{4.1}$$

Proof. For a non-negative variable, $\mathbb{E}[M] = \int_0^\infty \mathbb{P}(M > \varepsilon)\,\mathrm{d}\varepsilon$; for $M = \min_i \rho_i$ and independent identically distributed ρ_i, $\mathbb{P}(M > \varepsilon) = (1 - F(\varepsilon))^N$, and $\rho \le 1$. For the asymptote, the density of

(C, P, K)	rate (bits per weight)	expected minimal residual	Shannon bound at the same rate
(8, 3, 16)	4.00	0.79 %	0.39 %
(12, 4, 16)	3.00	3.82 %	1.56 %
(12, 3, 16)	2.67	**6.32 %**	2.48 %
(14, 4, 16)	2.57	7.07 %	2.83 %
(16, 5, 16)	2.50	7.84 %	3.13 %
(16, 4, 16)	2.25	10.74 %	4.42 %
(8, 1, 16)	3.00	5.35 %	1.56 %

Table 4.1: The expected minimal residual of the seed on an isotropic source, by lemma 4.1 and its corollary, before quantisation of the coefficients [calc.]. The Shannon bound of a Gaussian, 2^{-2R}, is given at the same rate. Data: `mesures/genome/genome/graine-theorie.json`; `genome/genome.md` § 2.2.

Beta(a, b) is $\varepsilon^{a-1}(1-\varepsilon)^{b-1}/B(a,b) \approx \varepsilon^{a-1}/B(a,b)$ near 0, hence $F(\varepsilon) \approx \varepsilon^a/(a\,B(a,b))$. For large N, only the region where F is small counts, and $(1-F)^N \approx \exp(-NF) = \exp(-c\,\varepsilon^a)$ with $c = N/(aB(a,b))$. The change of variable $u = c\,\varepsilon^a$ gives $\int_0^\infty e^{-c\varepsilon^a}\mathrm{d}\varepsilon = c^{-1/a}\,\frac{1}{a}\int_0^\infty u^{1/a-1}e^{-u}\,\mathrm{d}u = c^{-1/a}\,\Gamma(1+1/a)$, which is (4.1). ∎

The exact computation by quadrature (`outils/genome/recette/graine_theorie.py`, `graine-theorie.json`), before any quantisation of the coefficients (which can only add error), gives Table 4.1 [calc.].

What this says, before the measurement. Around 2.64 bits, the seed leaves *at least* 6.3 % error on an isotropic source, where the base leaves 4.29 %; to reach 4.29 %, it needs nearly 3 bits. The reason fits in one line: the K bits of the seed buy a random codebook of 2^K subspaces in a dimension $C \leq 16$. Shannon proves that the random codebook reaches the rate–distortion function [9], but *as the dimension tends to infinity*; at $C = 8$ or 12, the gap to the bound is a factor 2.5 in error. And the $4P + 4$ bits of the coefficients describe a fine position in a subspace chosen at random: they pay for a precision that the coarse choice of the subspace has already lost.

4.2.3 The measurement

`genome graines` (`graines.jsonl`, `graines.log`): expert 0 of each of the 12 tensors, 32 rows spread over the whole height, whole rows; the error of the base, measured on the same weights, is 4.559 % on this sample (Table 4.2).

The lemma holds on the real weights: the measurement follows the prediction, plus the quantisation of the coefficients (0.6 to 2 points). Real blocks of weights behave, for the seed, like uniform directions: no concentration that the best of 2^{16} subspaces could have seized. At the error of the base, the seed needs **3.06 bits per weight** (interpolation in the log of the error between (12, 4) and (10, 3); linearly, 3.07 [calc.]), against 2.64: **it does not beat the frontier**, by 0.42 bit. It does not even beat lossless coding (1.4 % error for 4 bits, when 3.84 bits restore the weights exactly). The generator does not matter: LFSR and mixer give 1.58 and 1.41 %, the gap coming from the correlation of the successive states of an LFSR, which shift the same bits by one place.

(C, P, K), generator	rate	measured error	prediction of lemma 4.1
(8, 3, 16), mixer	4.00	1.41 %	0.79 %
(8, 3, 16), LFSR	4.00	1.58 %	0.79 %
(10, 3, 16)	3.20	3.89 %	2.98 %
(12, 4, 16)	3.00	4.89 %	3.82 %
(8, 1, 16), shape-gain	3.00	7.34 %	5.35 %
(12, 3, 16)	2.67	7.51 %	6.32 %
(14, 4, 16)	2.57	8.24 %	7.07 %
(16, 5, 16)	2.50	9.10 %	7.84 %
(12, 2, 16)	2.33	11.76 %	9.99 %
(16, 4, 16)	2.25	12.15 %	10.74 %

Table 4.2: The seed measured on the real weights, against lemma 4.1 [meas.] [calc.]. Data: `mesures/genome/genome/graines.jsonl`, `csv/graine-mesure-theorie.csv`.

4.3 The seed that grows: the strand read through codons

4.3.1 The idea

If the defect of the seed is its dimension, one needs a seed *as long as the row*, and a way to search for it. Messenger RNA is read by the ribosome three bases at a time, from a fixed starting point, without commas [3], and each codon is translated by a fixed and degenerate table [4]. Transposed, as an analogy of construction:

- the **strand**: a sequence of bits stored on the disk, one per row of a matrix;
- the **reading window**: l bits; it advances by k_b bits at each step and returns v weights. Two neighbouring steps share $l - k_b$ bits: the codons overlap, and that is where the strand differs from RNA;
- the **genetic code**: the window $s \in \{0, \dots, 2^l - 1\}$ is translated into v values by $\texttt{table}[\texttt{hache}(s, j) \gg (32 - q)]$, $j < v$: a bit mixer chooses a cell of a table of 2^q values; 2^l windows for 2^q meanings, a degenerate code like the real one;
- the **adaptation of the organism**: the table is learnt on the weights of the tensor (Lloyd: each cell takes the weighted mean of the weights it serves).

Rate: k_b/v bits per weight, plus the first window (l bits per row). It is a trellis code, the "bit-shift" trellis of Mao and Gray, taken up with computed codes by QTIP [47, 74]; trellis-coded quantisation goes back to Marcellin and Fischer [13, 75]. *They pose the problem thus*: an i.i.d. Gaussian source, after a random rotation. *What we lacked*, and what is derived here: its use on a model born in 4 bits against a base already nested in the format; the distribution of the bits between rows by their energy, measured and decomposed; the complete accounting against the frontier F1 of the `.trc` format. The code is written from scratch (`outils/genome/recette/src/codons.rs`).

4.3.2 The best strand: Viterbi, and why it is exact

The state at step p is the window s_p (l bits). The transition $s \to s'$ is allowed if and only if the $l - k_b$ high bits of s' are the $l - k_b$ low bits of s, that is $s' = ((s \ll k_b) \mid b) \bmod 2^l$ for some $b < 2^{k_b}$.

Lemma 4.3 (The predecessors in the bit-shift trellis)**.** *The predecessors of s' are exactly the $s = (t \ll (l - k_b)) \mid u$, $t < 2^{k_b}$, with $u = s' \gg k_b$. They depend on s' only through u.*

Proof. The transition writes $s' = ((s \ll k_b) \mid b) \bmod 2^l$: the $l - k_b$ high bits of s' are the $l - k_b$ low bits of s, and its k_b low bits are b. So $s' \gg k_b = s \bmod 2^{l-k_b}$: u fixes the $l - k_b$ low bits of s, and its k_b high bits, t, are free. Conversely, every s of this form leads to s' with $b = s' \bmod 2^{k_b}$. ■

Proposition 4.4 (Exactness of the Viterbi). *Let $C_p(s)$ be the minimal cost $\sum_{i \le p} \sum_j \omega_{i,j} (x_{i,j} - \hat{x}_{i,j})^2$ of a strand whose window at step p is s. Then $C_0(s) = d_0(s)$ and*

$$C_p(s') = M_p(s' \gg k_b) + d_p(s'), \qquad M_p(u) = \min_{t < 2^{k_b}} C_{p-1}\big((t \ll (l - k_b)) \mid u\big), \tag{4.2}$$

with $d_p(s') = \sum_j \omega_{p,j} (x_{p,j} - \texttt{table}[\texttt{hache}(s', j)])^2$. The strand of minimal cost is obtained by keeping the argmin t of each $M_p(u)$ and tracing back from $\arg\min_s C_{P-1}(s)$.

Proof. The cost is additive by step, and the term of step p depends only on the window s_p: this is Bellman's principle of optimality, on which the Viterbi algorithm rests [76]. An optimal path ending in s' at step p is made of an optimal path ending in a predecessor of s' at step $p - 1$, then of $d_p(s')$; lemma 4.3 gives the exact set of these predecessors, which depends only on $u = s' \gg k_b$. Recurrence (4.2) follows, and tracing back the argmins rebuilds the path. ■

The cost. Per step, 2^l comparisons (the minimum over t, by contiguous slices of 2^{l-k_b} thanks to the lemma) and $2^l \cdot v$ evaluations; traceback memory: 2^{l-k_b} bytes per step. For $l = 12$: 4 096 states, about $2.3 \cdot 10^5$ weights per second and per shape on the 16 threads of the i7-13620H [meas.].

The controls. T2 (accounting of the bits): the strand written (`ecrire_brin`) then read back by the decoder alone (`lire_brin`, which knows only the strand and the table) reproduces exactly the reconstruction whose error is counted (tests `brin_ecrit_puis_lu_*`). T1 (known source): on 128 i.i.d. Gaussian rows, $l = 12$, $k_b = 2$, $v = 1$, $q = 8$, a Gaussian table without learning, the error is **0.073 05** (`gauss-T1.json`), within the declared interval [0.0625, 0.080]. The bottom of the interval is the Shannon bound $2^{-2 \cdot 2}$: beating it would expose an accounting fault; the top is the midpoint between the published trellis at $l = 16$ (0.069, QTIP) and the best published vector quantiser of dimension 8 (0.089). Break (zero table): 0.996, the control fails (`gauss-T1-casse.json`) [meas.].

The strand on a true Gaussian. Figure 4.1 gives the curve of the strand alone on 64 Gaussian rows (32 at $l = 16$, except the shape (2, 1), on 64). The gap to the Shannon bound stays between 0.46 and 1.66 dB at $l = 12$, between 0.29 and 1.08 dB at $l = 16$ [meas.] [calc.] (`memoire/figures/tome3/donnees/genome-nombres.csv`). At 2 bits, the window length buys most of the gap: 0.0880; 0.0785; 0.0728; 0.0694; 0.0675 for $l = 8$, 10, 12, 14, 16, against the bound 0.0625. The cheap mixer (4 operations) gives 0.0721 against 0.0728 for the strong one; the size of the table matters little ($q = 6$: 0.0732; $q = 10$: 0.0728). These two facts fix the setting of the rest of the chapter: $l = 12$, $q = 8$.

4.3.3 The rotation: spreading the information of each weight

A row w of 2 880 weights is rotated in pieces of 64: $y = \frac{1}{8} H_{64} D\, w$, $D = \mathrm{diag}(\pm 1)$ drawn from a seed (no bit stored), H_{64} the Hadamard matrix of order 64.

Proposition 4.5 (The rotation preserves the product and the error). *Let R be the block-diagonal matrix of the $\frac{1}{8} H_{64} D_m$. Then R is orthogonal; $Wx = (WR^\top)(Rx)$; and for every reconstruction $\hat{W}'$ of the rotated rows $W' = WR^\top$, the error is preserved: $\|W - \hat{W}' R\|_F = \|W' - \hat{W}'\|_F$.*

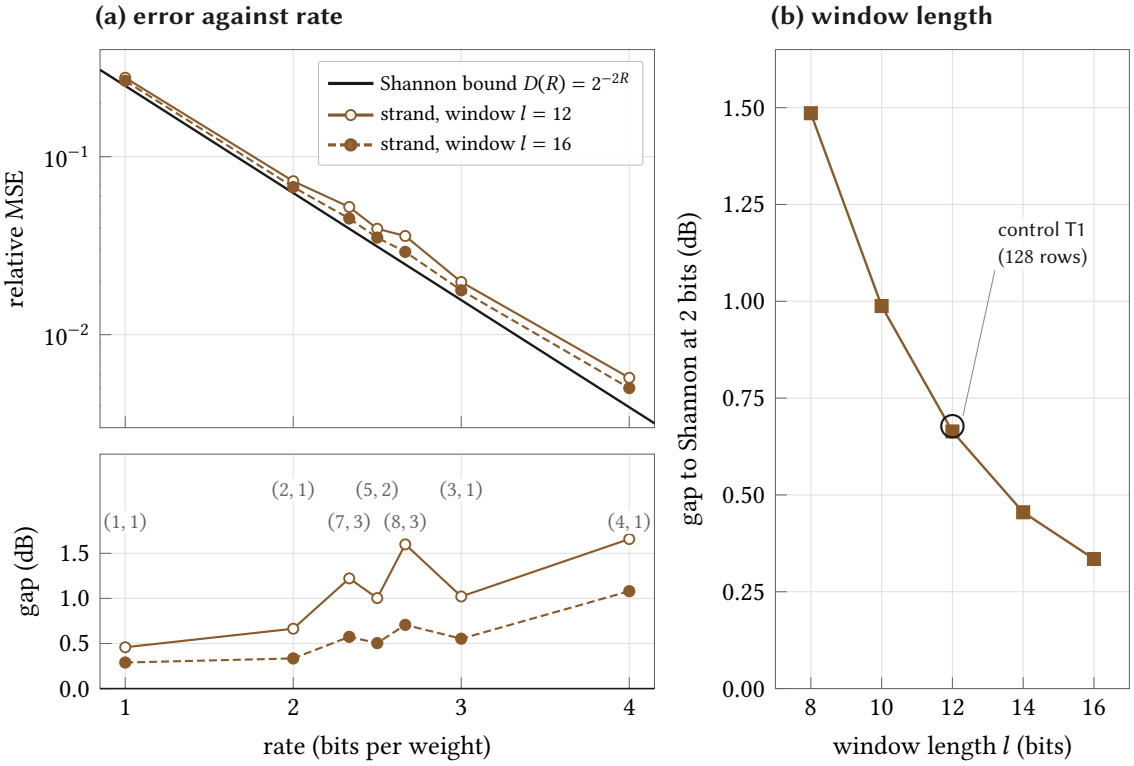


Figure 4.1: On a true Gaussian, the strand read by codons stays less than two decibels from Shannon's bound, and the window length buys most of the gap. [meas.] A strand of bits is read through a sliding window of l bits; a hashed mixer turns each window into an index in a table of 2^q values (here $q = 8$, Gaussian quantiles, no Lloyd pass); Viterbi chooses the bits. The source is i.i.d. Gaussian, rows of 2 880 values: 64 rows at $l = 12$; at $l = 16$, 32 rows (92 160 values), except the shape $(2, 1)$, measured on 64. The reference is Shannon's distortion–rate function $D(R) = 2^{-2R}$, which no code of finite length reaches. (a) The seven shapes (bits k_b advanced per step, values v emitted, written at the top of the lower panel), from 1 to 4 bits; the gap stays between 0.46 and 1.66 dB at $l = 12$ (hollow, solid line) and between 0.29 and 1.08 dB at $l = 16$ (filled, dashed). (b) At 2 bits, the window length buys most of the gap ($l = 8$: 0.0880; $l = 16$: 0.0675, against the bound 0.0625). At $l = 12$ the cheap mixer gives 0.0721 against 0.0728 for the strong one, three Lloyd passes 0.0722, $q = 6$ and $q = 10$ give 0.0732 and 0.0728 (not drawn: they sit on the point $l = 12$); the control T1, same setting on 128 rows, gives 0.0731 (ink ring). What the figure does not show: real weights (next figures), where the rotation must first make the rows Gaussian; nor the decoding cost.

Data: `mesures/genome/genome/gauss-balayage.jsonl`, `gauss-T1.json`; script `memoire/outils/tome3_genome.py`.

Proof. $H_{64}H_{64}^\top = 64\,I$ and $D_m D_m^\top = I$, so $(\frac{1}{8}H_{64}D_m)(\frac{1}{8}H_{64}D_m)^\top = \frac{1}{64}\,H_{64}D_m D_m^\top H_{64}^\top = I$: each block is orthogonal, and so is R. Then $WR^\top Rx = Wx$. Finally $W - \hat{W}'R = (W' - \hat{W}')R$, and the Frobenius norm is invariant under right multiplication by an orthogonal matrix: $\|AR\|_F^2 = \mathrm{tr}(ARR^\top A^\top) = \|A\|_F^2$. ■

One therefore stores the rotated rows, and rotates the activation *once per token*: 64×6 additions per piece (the fast Hadamard transform), that is 6 additions per input element, shared by all the rows of all the matrices that read this input. The tool nevertheless measures the error in the original domain, after the inverse rotation (`detourner`): that is the second path of this identity. Why rotate: an outlier weight (an E8M0 scale far above the median of its row) can be reached by the strand only by sacrificing its neighbours, which share its window. After rotation, it is spread over 64 positions, and the source becomes Gaussian, which is what the trellis is made for. It is, as an image, the holographic memory of Stone's list: the information of each element spread over all. Each rotated row is divided by its standard deviation σ (kept on 16 bits): the coded source is $x = y/\sigma$, of variance 1.

4.3.4 Distributing the bits between rows

A row r has, for each shape k, an error $D_r(k)$ and a cost $R_r(k)$ (bits of the strand, first window, 3 bits of shape index, 16 bits of σ). We minimise $\sum_r D_r(k_r)$ under $\sum_r R_r(k_r) \le R$.

Proposition 4.6 (The Lagrangian choice is optimal for its budget)**.** *For every $\lambda \ge 0$, the choice $k_r(\lambda) = \arg\min_k D_r(k) + \lambda R_r(k)$, made row by row, is optimal for the budget $R(\lambda) = \sum_r R_r(k_r(\lambda))$ it spends.*

Proof. If another choice k' spent at most $R(\lambda)$ with a smaller error, then $\sum_r D_r(k'_r) + \lambda \sum_r R_r(k'_r) < \sum_r D_r(k_r) + \lambda \sum_r R_r(k_r)$, which contradicts the minimality of each term of the sum. (The result bears Everett's name, 1963; the source is not reread here and does not enter the bibliography.) ■

The rate–distortion curve of the recipe is the set of points $(R(\lambda), D(\lambda))$ for 400 values of λ; the bits at equal error are read by linear interpolation in log(error) between two neighbouring points.

Proposition 4.7 (The gain of the distribution, at high rate)**.** *For Gaussian rows of variances σ_r^2, with distortions $D_r = \sigma_r^2\, 2^{-2R_r}$ (high rate, $D_r < \sigma_r^2$), the optimal distribution of the bits at mean rate $\bar{R}$ gains, at equal mean error,*

$$\Delta R = \frac{1}{2}\log_2 \frac{\mathrm{AM}(\sigma_r^2)}{\mathrm{GM}(\sigma_r^2)} \quad \textit{bits per weight} \tag{4.3}$$

over the uniform distribution, AM and GM being the arithmetic and geometric means of the σ_r^2.

Proof. With n rows, we minimise

$$\frac{1}{n}\sum_r \sigma_r^2 2^{-2R_r} \quad \text{under} \quad \frac{1}{n}\sum_r R_r = \bar{R}.$$

The Lagrangian gives $2\ln 2\,\sigma_r^2 2^{-2R_r} = \mu$ for every r: all the rows end at the same distortion D (the reverse water-filling of [2, § 10.3.3]). Then $R_r = \frac{1}{2}\log_2(\sigma_r^2/D)$, and the mean gives $\bar{R} = \frac{1}{2}\log_2 \mathrm{GM} - \frac{1}{2}\log_2 D$, that is $D_{\mathrm{opt}} = \mathrm{GM}\cdot 2^{-2\bar{R}}$. The uniform distribution ($R_r = \bar{R}$) gives $D_{\mathrm{unif}} = \mathrm{AM}\cdot 2^{-2\bar{R}}$. At equal error, the uniform one therefore needs $\frac{1}{2}\log_2(\mathrm{AM}/\mathrm{GM})$ more bits. By the arithmetic–geometric mean inequality, the gain is non-negative, zero if all the variances are equal. ■

The base of B takes none of it: it puts 2.64 bits everywhere. Section 4.6 computes this gain on the real tensors: from 0.009 to 1.27 bits.

Item	bits	per weight
strand of a row of n weights	$l + (n/v - 1)\,k_b$	$\approx k_b/v$
shape index of the row	3	0.0010
σ of the row (rotated source)	16	0.0056
tables: 5 shapes × 256 values × 16 bits, for 32 experts	20 480	0.000 08
E8M0 scales (raw source only): δ of F1 + 8/2 880 of medians		0.036 to 0.043
alignment of each expert matrix on 4 096 bytes, at worst	32 768	0.0040
exceptions of S^*: count (8 bits) + 3× (column 12 + code 4 + scale 8) per row	80	0.028

Table 4.3: Everything the recipe reads from the disk, item by item [calc.]. Per weight: divided by 2 880 for an item per row (for example $80/2\,880 = 0.028$), by 8 294 400 for an item per matrix ($32\,768/8\,294\,400 = 0.0040$). Source: `genome/genome.md` § 4.

4.4 The accounting, to the bit

Everything the recipe reads from the disk is counted (Table 4.3), by a second path in Python (`outils/genome/recette/analyse.py`) which gives the verdict. The frontier F1 of each tensor is read in `f1-par-tenseur.csv`: base $+\delta + 8/2\,880 + 0.000\,07$.

An under-accounting is stated: the Rust log (`codons-*-l12.json`) counts only *one* table per tensor instead of five (0.000 06 bit per weight less); the second path counts all five, and it is the one that judges. The two differ by less than 0.0002 bit per tensor.

4.5 The results on the real weights

Control S0: the frontier redone. The command `genome frontiere` recounts the 12 whole tensors (3 185 049 600 weights), recomputes the class centres (0.237 166; 1.424 298; 3.418 577; 6.0 in E2M1 units) and recovers the published error 0.042 919 745 124 980 to within $5.8 \cdot 10^{-15}$ (`frontiere.json`). Break (each class reads the centre of the next): 1.2128, the control fails (`frontiere-casse.json`) [meas.].

Verdict W. Reading through codons, $l = 12$, $q = 8$, 2 Lloyd iterations on the first 48 sampled rows of each expert, five shapes $(k_b, v) = (2,1), (7,3), (5,2), (8,3), (3,1)$, that is 2; 2.33; 2.5; 2.67; 3 bits per weight. Second path: `verdict-*-l12-second-chemin.json`, equal to the Rust log to within 10^{-4} bit. Figure 4.2 plots the curves, Table 4.4 the gains.

12 tensors out of 12: criterion W holds for the rotated source. Globally (a single λ for the 34 560 rows, cumulative error 4.257 %), 2.049 bits against 2.644; this number is dominated by the high-energy rows, and the mean per tensor is the one to retain. On the **raw source** (E8M0 scales kept, without rotation), 11 tensors out of 12 win, by 0.15 to 0.57 bit, but **blk.8.ffn_down fails**: even at 3 bits everywhere, the recipe leaves 6.43 % error there, where the base leaves 4.63 % at 2.64 bits. Its rows carry blocks whose scale is far above the median; the strand can reach them only by sacrificing their neighbours. The rotation settles exactly this case (0.231 bit gained on this tensor): that is why it was retained.

Where the gain comes from: space filling against distribution. The fixed shapes (all rows at the same shape) give, by interpolation between neighbouring shapes, the gain of *space filling alone* (`codons-tourne-l12.json`, field `fixes`): 0.14 to 0.18 bit on 11 tensors (0.135 on average over the 12), and −0.14 bit on blk.8.ffn_down, where the distribution is indispensable. The distribution

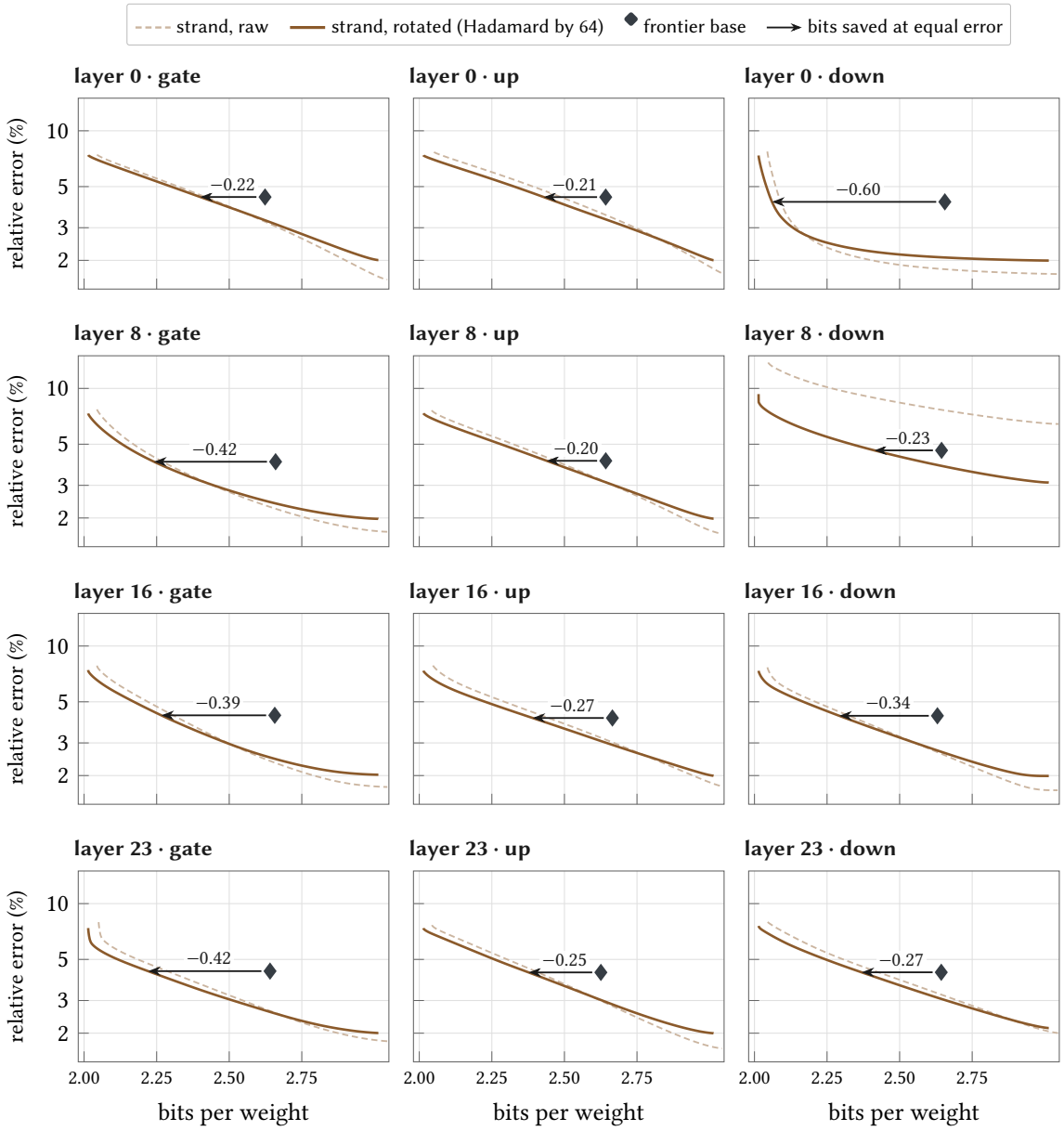


Figure 4.2: The rotated strand beats the base in weight error, tensor by tensor. [meas.] Twelve expert tensors of gpt-oss-20b (layers 0, 8, 16, 23; gate, up, down), window $l = 12$, each measured on the sample of the W criterion: experts 0, 8, 16 and 24, 2 880 rows per tensor, not the whole tensor. Each curve sweeps the bit allocation between rows (Lagrangian, 5 shapes) and gives the relative quadratic weight error $\sum(w - \hat{w})^2 / \sum w^2$, the same quantity as in the previous figure, against the full rate of the strand (strand, window, shape index, σ, tables, scales, alignment: everything counted). Dashed light brown: the strand on raw rows; solid brown: after the Hadamard rotation by blocks of 64, which makes the rows Gaussian. The grey diamond is the frontier base (real rANS rate, its error on the same sample); the arrow, the bits the rotated strand saves at the same error: 0.20 to 0.60 bit, 0.319 on average over the 12 tensors. In layer 8, down, the raw strand never reaches the base's error (1 missing point). What the figure does not show: the output error of the experts and the KL, where this gain does not hold (weighting, exceptions and S^*: Chapter 8); the weight error is not a judge.

Data: `mesures/genome/genome/csv/courbes-tourne-l12.csv`, `courbes-brut-l12.csv`, `verdict-par-tenseur.csv`, `mesures/genome/genome/codons-tourne-l12.json`; script `memoire/outils/tome3_genome.py`.

Tensor	error of the base	frontier (bits)	rotated strand at equal error	gain (bits)	gain
blk.0.ffn_gate	4.394 %	2.6234	2.4005	0.223	8.5 %
blk.0.ffn_up	4.389 %	2.6422	2.4291	0.213	8.1 %
blk.0.ffn_down	4.137 %	2.6558	2.0604	0.595	22.4 %
blk.8.ffn_gate	4.022 %	2.6597	2.2410	0.419	15.7 %
blk.8.ffn_up	4.068 %	2.6424	2.4391	0.203	7.7 %
blk.8.ffn_down	4.631 %	2.6445	2.4130	0.231	8.8 %
blk.16.ffn_gate	4.227 %	2.6569	2.2660	0.391	14.7 %
blk.16.ffn_up	4.089 %	2.6653	2.3905	0.275	10.3 %
blk.16.ffn_down	4.203 %	2.6302	2.2920	0.338	12.9 %
blk.23.ffn_gate	4.323 %	2.6406	2.2203	0.420	15.9 %
blk.23.ffn_up	4.254 %	2.6256	2.3768	0.249	9.5 %
blk.23.ffn_down	4.260 %	2.6434	2.3691	0.274	10.4 %
mean of the 12		**2.6442**	**2.3248**	**0.319**	**12.1 %**

Table 4.4: Verdict W, rotated source: bits of the strand at the error of the base measured on the same rows [meas.]. Data: `mesures/genome/genome/csv/verdict-par-tenseur.csv`; `verdict-tourne-l12-second-chemin.json`.

between rows brings the rest: from 0.03 bit (blk.0.ffn_up) to 0.43 bit (blk.0.ffn_down) [meas.]. The errors of the fixed shapes on the rotated weights of blk.0.ffn_gate (0.0739; 0.0529; 0.0402; 0.0364; 0.0201) are those of the strand on a true Gaussian (0.0728; 0.0522; 0.0394; 0.0358; 0.0198) to within 1.5 %: **after rotation, the weights are, for the strand, indistinguishable from Gaussian noise**. This is the split announced in Section 2.3: the gain is the space filling of a trellis code, which the scalar base does not have, and the spread of the energies, which it ignores.

4.6 Where the noise is, where the structure is

`outils/genome/recette/structure.py` computes the singular values of expert 0 of each tensor (2 880 × 2 880 matrix, in exact f64), against a control: the same matrix with its entries randomly permuted, which destroys any rank structure (the spectrum then follows the Marchenko–Pastur law; Table 4.5).

The noise. Inside a row, once rotated and normalised, there is nothing to take beyond the variance: the strand does there exactly what it does on a true Gaussian, the neighbouring codes share only 0.005 bit, and the seed of dimension 8 to 16 finds what it would find in noise. **No short recipe generates these weights**: that is the answer, negative, to the question “are the weights recipe-able?”.

The structure is at second order, and it is large. The energy concentrates on a few directions (5 % of the directions carry 23 to 68 % of the energy, against 17 to 19 % for the same matrix permuted; a single direction carries up to 8.5 %, 22 times chance), hence on certain rows (kurtosis up to 348, computed distribution gain up to 1.27 bits). It is this structure that the distribution of the bits between rows cashes in, and that the base of B, which puts the same number of bits everywhere, leaves on the table.

What remains to be taken. The Gaussian bound with ideal distribution between rows (`borne-shannon-par-tenseur.json`) is 1.07 to 2.25 bits at the error of the base; the measured recipe is

Tensor	energy of the first 144 directions real / permuted	1st direction real / permuted	kurtosis of the rows	distribution gain (high rate, [calc.])
blk.0.ffn_gate	27.4 / 17.2 %	0.81 / 0.14 %	4.0	0.019
blk.0.ffn_up	23.2 / 17.2 %	0.29 / 0.14 %	3.8	0.009
blk.0.ffn_down	67.7 / 18.9 %	8.48 / 0.38 %	347.7	1.272
blk.8.ffn_gate	45.2 / 17.3 %	1.62 / 0.14 %	12.9	0.572
blk.8.ffn_up	30.0 / 17.2 %	0.41 / 0.14 %	4.9	0.111
blk.8.ffn_down	43.1 / 18.5 %	7.00 / 0.44 %	343.2	0.250
blk.16.ffn_gate	54.2 / 18.1 %	4.60 / 0.39 %	182.9	0.645
blk.16.ffn_up	30.4 / 17.2 %	0.72 / 0.14 %	6.3	0.160
blk.16.ffn_down	43.3 / 17.4 %	1.54 / 0.21 %	42.0	0.343
blk.23.ffn_gate	42.0 / 17.3 %	2.27 / 0.14 %	17.2	0.426
blk.23.ffn_up	33.5 / 17.2 %	1.01 / 0.14 %	7.9	0.130
blk.23.ffn_down	33.3 / 17.7 %	3.45 / 0.19 %	79.7	0.089

Table 4.5: The structure of the expert matrices (expert 0 of each tensor): share of energy of the first 5 % of the singular directions and of the first one, against the same matrix permuted; kurtosis of the rows; distribution gain by Proposition 4.7 [meas.] [calc.]. Data: `mesures/genome/genome/structure-spectre.json`, `csv/spectre.csv`.

0.19 to 0.99 bit above. The gap comes from the strand at $l = 12$, 0.3 to 0.8 dB from the bound, and from the smallest shape, 2 bits: ideal water-filling would give 0 or 1 bit to the weakest rows. A "zero" shape (row not stored), costed on the same CSVs, would bring the global rate down to 0.36 bit *at equal cumulative error*, which mainly shows that the cumulative error on the weights lets itself be fooled by the distribution (a handful of very high-energy rows dominate it); it is not retained. The rank structure (a few dominant directions) is not exploited: storing r directions (u, v in f16) costs $r \cdot 5\,760 \cdot 16/8\,294\,400 = 0.011\,r$ bit per weight; for blk.0.ffn_down, the first direction (8.5 % of the energy) would save $\frac{1}{2}\log_2(1/0.915) = 0.064$ bit at equal error for 0.011 bit of cost; for the up tensors (0.3 to 1 %), nothing [calc.].

Shared patterns, combined. `genome motifs` (`motifs-*.json`): a block of d weights is the sum of M patterns, each drawn from a dictionary of 2^b patterns shared by the 32 experts of the tensor, learnt by weighted k-means stage by stage (residual quantisation). Control: the 4-level scalar quantiser of a Gaussian gives 0.1175 [77]. At the error of the base (4.257 %), the patterns need **2.90 bits** at best (between (8, 12, 1) at 1.54 bits for 18.2 % and (8, 12, 2) at 3.05 bits for 3.65 %): **they do not beat the frontier**. A dictionary learnt in dimension 4 to 8 is a vector quantiser of small dimension: even the best lattice of dimension 8 (0.089 at 2 bits on a Gaussian, QTIP) remains far from the strand (0.073). Sharing the dictionary between experts costs nothing (0.002 bit) and saves nothing more: the experts have no patterns of their own that the common dictionary would miss.

4.7 The recipe and the base + complement split

Lemma 4.8 (The exact complement gains nothing for free)**.** *Let X be the MXFP4 codes of a tensor and Y the bits of the recipe, a deterministic function of X. For every coding of the complement that conditions each code X_i on a function Z_i of Y,*

$$R_Y + \sum_i H(X_i \mid Z_i) \;\geq\; H(Y) + H(X \mid Y) \;=\; H(X). \tag{4.4}$$

Proof. $H(X,Y) = H(X)+H(Y \mid X) = H(X)$ since Y is a function of X; and $H(X,Y) = H(Y)+H(X \mid Y)$. Hence the equality on the right. Then $H(X \mid Y) \le \sum_i H(X_i \mid Y)$ (subadditivity), $H(X_i \mid Y) \le H(X_i \mid Z_i)$ (conditioning on less, Z_i being a function of Y, does not reduce entropy), and $R_Y \ge H(Y)$ (no lossless code goes below the entropy). ■

What the combination loses is $(R_Y - H(Y)) + (\sum_i H(X_i \mid Z_i) - H(X \mid Y))$: the information on X that the bits of the strand carry *together* and that a weight-by-weight context does not see. **Measured** (`codons-*-l12.json`, field `H_code_sachant_recette`: Z_i the reconstruction of the recipe brought back to the scale of the block, in 65 bins of 0.375): recipe + complement + scales = **4.05 to 4.37 bits** for the rotated source (4.09 to 4.90 for the raw one), against 3.80 to 3.83 lossless for the same tensors. The loss, 0.21 to 0.53 bit, is the second term. Criterion L fails, as the lemma announced.

What the combination gives all the same. In architecture B, level 0 is read at each cache miss and the complement only for the admitted experts. Replacing the base by the recipe at level 0 divides the bytes of a miss by $2.644/2.325 = 1.14$ at equal weight error, or, at the rate of the frontier of each tensor, gives an error of 2.07 to 3.82 % instead of 4.02 to 4.63 % (interpolated on `courbes-tourne-l12.csv`); the complement that follows then costs 0.21 to 0.53 bit more than with the base [calc.]. These numbers are in weight error; the next section says what they are worth.

4.8 The output of the experts: fidelity, and S^*

The inputs and the measurement. The normalised states and the routing of the f64 reference, captured by the scout: "calib" (windows 10 to 21, tuning only) and "eval" (windows 0 to 9, those of the `.kld` of R1, measurement only). For expert e of layer ℓ, the first 512 positions routed to it (fewer if the expert is rare). Output of the expert: $y = W_{\text{down}}\,\text{SwiGLU}_{\text{oai}}(W_{\text{gate}}h + b, W_{\text{up}}h + b) + b$ in f64, exact weights, base or recipe; relative error $\sum \|y - \hat{y}\|^2 / \sum \|y\|^2$.

At point W, the recipe loses at the output. Layer 0, expert 0 (499 positions): weight error 4.25 % (base 4.27 %), but output error **7.63 %** against **6.77 %** for the base, the recipe winning on only 146 positions out of 499 (`sortie-tourne-point-W-interrompu.log`) [meas.]. Criterion O, declared at this point, **fails** on this first expert; the pass was stopped there to measure what really decides, fidelity as a function of rate (protocol F, declared before its pass in `00-seuils-declares.md`). *The squared error on the weights does not measure fidelity*: at equal weight error, the error of the base does less damage at the output than that of the strand.

The weighting. By Proposition 4.5, the output of a row is $\sum_c \Delta w_c x_c = \sum_{c'} \Delta w'_{c'} x'_{c'}$ with $x' = Rx$. Its expected square is $\sum_{c',c''} \mathbb{E}[\Delta w'_{c'} \Delta w'_{c''}]\, \Sigma'_{c'c''}$, $\Sigma' = R\Sigma R^\top$, $\Sigma = \mathbb{E}[xx^\top]$ of the real inputs. For a *white* error in the rotated domain ($\mathbb{E}[\Delta w'_{c'} \Delta w'_{c''}] = 0$ for $c' \neq c''$), there remains

$$\mathbb{E}\,\|\Delta w^\top x\|^2 = \sum_{c'} h_{c'}\, \mathbb{E}\big[\Delta w'^2_{c'}\big], \qquad h = \operatorname{diag}\big(R\,\Sigma\,R^\top\big). \tag{4.5}$$

Weighting the Viterbi by h (the weights ω of Proposition 4.4) therefore minimises the expected output error under this hypothesis (`outils/genome/recette/src/fidelite.rs`, header) [calc.]; it is the diagonal of the Hessian of the quadratic layer loss, the move of GPTQ reduced to what the trellis can do.

Layer, expert (positions)	base	uniform at 2.64 b	weighted at 2.64 b	bits at equal output (unif. / weight.)
0, 0 (499)	6.77 %	5.25 %	5.18 %	2.41 / 2.38
0, 16 (512)	5.13 %	3.88 %	3.82 %	2.38 / 2.39
8, 0 (512)	8.30 %	8.80 %	7.91 %	2.68 / 2.60
8, 16 (131)	7.45 %	8.93 %	9.41 %	2.87 / > 2.9
16, 0 (127)	5.72 %	6.02 %	5.40 %	2.68 / 2.58
16, 16 (150)	4.81 %	11.04 %	—	> 2.9 / —

Table 4.6: The output fidelity of the strand, uniform and weighted by (4.5), pass F stopped after five and a half experts, the verdict being settled [meas.]. Data: `mesures/genome/genome/fidelite.log`, `verdict-fidelite.json`, `csv/fidelite.csv`.

Verdict F fails (Table 4.6): the weighting wins on 4 experts out of 5, the uniform one on 2 out of 6; two experts lose in both variants. The diagnosis (`erreur_colonnes.py`, `erreur-colonnes-ponderee-264.json`): the error of the strand is not spread over the columns like that of a white noise, and the hypothesis of (4.5) is false where it counts. In blk.8.ffn_down (expert 16), column 1011 carries **144 times** the mean energy of a column and receives **334 times** the mean error; the columns of rank 0 modulo 64 receive 7.6 times their share [meas.]. The mechanism: a massive column is spread, by the rotation, into 64 large values in each row; the table (256 values, quantiles of the source) clips them; the clipping errors, of the same sign in all the rows, come back through the inverse rotation *onto the massive column*. Now a massive column of down reads a unit of the intermediate activation, and a massive column of gate or up an input channel: where the error does most harm. The base, for its part, makes the largest weights of each block almost exact (its class {6} is exact). Of the 15 matrices analysed, 5 have a column whose error exceeds 10 times the mean, and in 4 of them it is the most energetic column of the matrix; the fifth (gate, layer 16, expert 0, column 2 744, ×29.9) is the most energetic column of the up matrix of the same expert, which reads the same input.

The remedy, and the system S^*. Keep *apart, exact*, the 3 weights of largest magnitude of each row (0.1 %; $8 + 3 \times 24$ bits per row, 0.028 bit per weight, counted): they are, row after row, the entries of the massive columns. Layer 8, expert 16, uniform: the output error goes from 8.93 % to 6.20 % (base 7.45 %), the weight error from 4.63 % to 3.16 % (`fidelite-exceptions.log`). The strand, the rotation, the weighting and these exceptions form S^*, declared before its pass.

Verdict F(S^*) holds (Table 4.7): at 2.64 bits, S^* makes less output error than the base on **8 experts out of 8** (sign test, $2^{-8} \approx 0.004$), and on each expert the majority of the positions confirms it (from 333 out of 512 to 106 out of 106; the binomial law gives $p < 10^{-5}$ for each). At equal output error, S^* reads 2.09 to 2.58 bits, 2.36 on average, against 2.64: 0.28 bit less per weight read (10.8 %). That is less than the 0.32 bit of criterion W: the weight error exaggerated the gain, and the output judges it. The deliverable of the subject drew from it "×1.12 tokens per second at the roof of the disk for the experts read at level 0"; that is a computation at equal output error, and the next section says that the KL does not confirm it.

4.9 The KL against the reference: S^* is equal to the base

The protocol (P2 of the round). The scout's fork, extended with substitution in F32 (`outils/genome/recette-kl/`); the 8 experts of Table 4.7 replaced (layers 0, 8, 16, 23 × experts 0 and 16:

Layer, expert (positions)	base	S^* at 2.64 b	positions where S^* wins	S^* at 2.4 b	bits of S^* at equal output
0, 0 (499)	6.77 %	**5.09 %**	440 / 499	6.56 %	2.37
0, 16 (512)	5.13 %	**3.65 %**	432 / 512	4.74 %	2.34
8, 0 (512)	8.30 %	**7.79 %**	333 / 512	10.05 %	2.58
8, 16 (131)	7.45 %	**6.72 %**	93 / 131	8.50 %	2.54
16, 0 (127)	5.72 %	**4.57 %**	122 / 127	5.92 %	2.43
16, 16 (150)	4.81 %	**3.26 %**	149 / 150	4.30 %	2.30
23, 0 (150)	5.68 %	**3.15 %**	148 / 150	4.10 %	2.09
23, 16 (106)	3.97 %	**2.56 %**	106 / 106	3.36 %	2.23
mean		**ratio 0.747**			**2.36**

Table 4.7: S^* (rotation, weighting, exceptions 0.1 %) at the output of the experts, held-out inputs [meas.]. Experts (16, 16), (23, 0) and (23, 16) had served in no choice. Data: `mesures/genome/genome/fidelite-sstar.log`, `fidelite-sstar.json`, `verdict-fidelite-sstar.json`, `csv/fidelite-sstar.csv`.

8 out of 768), base against S^* at 2.64 bits, and `juge comparer` of lot R, which pairs the two KLs position by position against the same truth (`ref-openai-20b.kld`). Rule of the judge: equal if $|\bar{d}| \leq 2\sigma(\bar{d})$; better if $\bar{d} + 2\sigma(\bar{d}) < 0$. Control of the fork: with no expert substituted, one window, KL at most 10^{-9} (`kl/temoin-vide*.json`).

What was published, and what replaced it. The first pass covered 3 windows (765 positions; `mesures/genome/genome/kl/comparer-base-contre-recette.json`): KL 0.0226 for the base, 0.0301 for S^*, ratio ~~1.335~~ (a three-window ratio, which the K10 pass brings down to 1.018, 0.020 93/0.020 57 [calc.], Table 4.8), paired difference +0.0076 ± 0.0047 (1.6 σ), first choice 91.4 against 90.3 % [meas.]. The synthesis of the round had first concluded from it "S^* has 25 % less expert error and ~~33 % more KL~~" and "~~fails~~", then struck it out (`genome/05-synthese.md` §§ 0, 1.2, 2.4): the K10 pass, declared beforehand, redid the comparison on the **10 windows** of R1 (2 550 positions; `kl/comparer10-base-contre-recette.json`, commit `39e9aa4`). It is the one that counts (reserve R4 of the volume).

S^* is equal to the base (Table 4.8): +0.000 36 ± 0.001 86, KL ratio 1.018 [calc.]. What changed between the two passes is the number of positions, multiplied by more than three: the gap of the first 3 windows was within the noise, and it dissolved into it. The shape of the two distributions says why the mean does not move. S^* reduces the smooth part of the KL (median ÷1.21, 99th percentile from 0.411 to 0.338) and thickens the extreme of its tail (99.9th percentile from 1.37 to 1.59, maximum from 3.01 to 3.22): the two compensate [meas.]. **25 % less expert error, 0 % less KL.** Chapter 8 makes it property (c) of the round (the output error of an expert does not predict the KL) and puts forward a hypothesis, stated as such: the weighting of S^* is *diagonal* (4.5), it does not see the correlation, in the rotated domain, between the 64 positions that carry the same massive channel, whereas the base leaves the contribution of the massive channels almost intact, and with it the margins of the router of the following layers.

P2, 10 windows, 2 550 positions, 8 experts out of 768	base	S^*
mean KL (± uncertainty)	0.020 57 ± 0.002 12	0.020 93 ± 0.002 35
median KL	0.001 84	0.001 52
KL, 90th percentile	0.0311	0.0275
KL, 99th percentile	0.411	0.338
KL, 99.9th percentile	1.367	1.588
maximal KL	3.008	3.216
identical first choice	91.88 %	91.92 %
perplexity (reference 460.24)	461.62	468.71
paired KL difference, S^* minus base	+0.000 36 ± 0.001 86: **equal**	
paired difference of first choice	+0.04 ± 0.56 point: equal	

Table 4.8: S^* against the base, the end-to-end KL on 10 windows (K10 pass) [meas.]. Verdicts of the judge: statistical, equal in KL and in first choice; literal, the KL of S^* is a little above. Data: `mesures/genome/genome/kl/comparer10-base-contre-recette.json` (and `.csv`, `.txt`).

Step of the decoder of a weight (shape $v = 1$)	operations
window: 64-bit register shifted by k_b, new bits inserted, mask of l bits	3
strong mixer (xor, then three times multiplication and shift-xor)	9 (the short one: 3)
table index (shift)	1
table read (256 values in shared memory)	1
multiply-add with the rotated activation	1
total per weight	**15** (the short one: **9**)

Table 4.9: The transcription cost of the strand, in operations per weight [calc.]. For $v = 2$ or 3, the window is shared between the v weights of the step: $(3 + 12\,v)/v$ operations per weight with the strong mixer. Source: `genome/genome.md` § 8.4.

4.10 The transcription

The budget [calc.]. At 2.32 bits per weight (the mean of verdict W), a weight costs 0.29 byte read, hence $214 \times 0.29 = 62$ operations of budget (269 at the throughput of the disk according to the correction of Volume II): the decoder takes 15 of them (Table 4.9), **a quarter** (6 % at the throughput of the disk). Per row, a multiplication by σ is added; per token, the rotation of the activation, 6 additions and a sign change per input element, shared by the 2 880 rows that read it, that is 0.002 operation per weight. The rANS decoder of study 03 costs about 12 operations per 4-bit code: the strand is of the same order as the lossless decoder already measured.

Measured on the CPU (`genome decoder`, `decodeur-*.json`). A whole expert (8 294 400 weights, blk.16.ffn_up, expert 0) read back from its 2 880 strands by the decoder alone, in ABBA series of 8 against the MXFP4 dequantisation of the same expert by the same path, with the machine loaded by the three other teams: shape (5, 2) with the strong mixer, $9.6 \cdot 10^7$ weights per second against $10.2 \cdot 10^7$ (ratio 0.94); with the short one, $7.5 \cdot 10^7$ against $6.3 \cdot 10^7$ (1.20); shape (2, 1) strong, $6.9 \cdot 10^7$ against $7.1 \cdot 10^7$ (0.97) [meas.]. The strand goes as fast as the dequantisation of the native format on the same path; the spread of the passes under load (0.05 to 0.17 s) is of the order of the gap, and the absolute throughputs are worth nothing under this load: only the ratio counts. On the card, nothing is measured.

4.11 What remains

1. **The kernel on the card**: a GEMV that reads the strand (sliding window in a 64-bit register, mixer, table in shared memory), rotates the activation in pieces of 64 and accumulates; to be measured against the 2.82 GB/s of the disk and against the MXFP4 kernel of the engine.
2. **The format**: a "STRAND" stream kind in `.trc` (per matrix, the shape index and σ of each row, then the strands end to end; the resident tables; the seed of the rotation, a constant of the format).
3. **Error feedback between columns** combined with the strand (the LDLQ of QuIP and QTIP, or the synonymous mutation of Chapter 3): the strand won the space filling and the distribution, the exceptions the massive columns; feedback would win the correlation of the inputs, and it is the prediction that would decide the hypothesis of Section 4.9. It is not measured and does not enter the construction lots.
4. **The window** $l = 16$ and finer shapes: 0.03 to 0.08 bit more on a Gaussian, at the price of a coding 16 times slower (offline, once).
5. **The rank structure**: 1 to 8 dominant directions stored apart, up to 0.05 bit net on the down matrices.
6. **gpt-oss-120b**: redo W and F; and **the KL with all the experts coded**: 19.1 billion weights at $2.3 \cdot 10^5$ weights per second and per shape, about 23 processor hours for one shape per row, 115 for the five [calc.]; or a coder on the card.

What could be wrong

- **The error on the weights is not the quality**, and the output error is not either. Criterion W is in cumulative error on the weights, which the distribution can fool; criterion F is in expert output error, which S^* passed 8 times out of 8 without moving the KL. Only the KL judges, and it substitutes only 8 experts out of 768.
- **The sample**: 4 experts out of 32, one row in 4, 12 tensors out of 72, gpt-oss-20b alone; the

120b is not measured (the entropies of the codes of the two models differ by 0.001 bit, which makes a large gap unlikely, without proving it).

- **The tables are learnt on the sample** (48 rows per expert) and evaluated also on these rows; their cost is counted, but slight overfitting is possible. The output applies them to whole experts and to inputs they have never seen.
- S^* **was chosen after seeing 5 experts out of 8** (the two remedies come from the measured failures); the other 3 had served for nothing and win too, but the risk of having tuned S^* on its own judges remains. The KL, which is not fooled, makes it equal to the base.
- **Few positions** for some experts (106 to 150 in layers 16 and 23, 131 for (8, 16)): each verdict per expert remains significant by the binomial law, but the size of the effect is less precise. The weighting is estimated on at most 1 024 routed positions: for a rare expert, h is noisy.
- **The interpolation in** log(**error**) between two points of the Lagrangian curve assumes the curve locally exponential; with 400 values of λ, the interpolation error is below 10^{-3} bit.
- **The frontier F1** is measured by a real coder; the recipe is counted bit by bit, with no tile header other than the worst-case alignment. A format that imposed finer tiles on the strand would add their headers.
- **The transcription is not measured on the card**; the rotation of the activation is computed to be negligible, not measured in the engine (for down, the input differs per expert and is rotated for each active expert).
- **The hypothesis that explains the tail of** S^* (diagonal weighting against massive channels) is not measured.
- **SeedLM and QTIP** are read at the source; their numbers on Llama are not reproduced and serve only as a landmark (QTIP publishes 0.069 at 2 bits on a Gaussian, found here at 0.0675 for $l = 16$ and 0.0731 for $l = 12$).

Chapter 5

Gene duplication

Where we start, what this chapter establishes, what remains

The starting point is Stone's hypothesis (`idees/biomimetisme-genome-des-poids.md` § 5 bis): "les experts d'une couche (ou de couches voisines) sont parents : stocker un expert parent et, pour chaque descendant, une petite mutation coûte moins de bits que l'expert entier" (the experts of a layer, or of neighbouring layers, are related: storing a parent expert and, for each descendant, a small mutation costs fewer bits than the whole expert). The frontier: 3.8431 bits lossless, 2.6441 bits for the base at 4.2920 % error. The state of the art (Table 2.3, family 3): mergers of experts on BF16, judged by task accuracy, never in bits on 4-bit codes; and the model card does not say whether the experts of gpt-oss were born by copy [6]. **This chapter establishes** the kinship of the experts of gpt-oss-20b, measured on the MXFP4 codes and on the weights, up to a permutation of the neurons (the only symmetry of an expert, proved), in the same layer and in the neighbouring layer; its exact cost in bits (a conditional mutual information, added costs counted); the most economical coding tree; the output error of a truncated mutation and of a paralogue on real inputs. **Duplication does not beat the frontier**: it exists, it can be measured, and it is worth a thousandth of a bit. A sentence of the sources is corrected here, on their own data. **What remains** is the KL of the truncated mutation (not measured, its output error being enough to rule it out), the exact cost and the paralogues on gpt-oss-120b, and the screen over the 24 layers. Sources: `genome/duplication.md`; `genome/duplication-derivations.md`; `genome/duplication-pour-le-memoire.md`; `mesures/genome/duplication/00-seuils-declares.md`, `csv/`, logs `*.json`; tools under `outils/genome/duplication/`.

In brief

In living things, a gene sometimes duplicates; the two copies then drift, one keeping the function, the other taking on a new one, and the gene family born from it long keeps the trace of its common copy [78]. The image suggests a compression: if the 32 experts of a layer were born from a copy, storing a parent and mutations would cost less than 32 experts. The image is an analogy, and the question is posed in bits: how much does one expert teach about another, once their neurons are put back in the same order? The measured answer is: almost nothing. After the best matching, two matched experts resemble each other as much as two experts drawn at random; what they share belongs to the whole layer, to its input columns, not to a lineage. The experts of gpt-oss were trained to be *different*: the router sends each one only the tokens where it is useful.

5.1 The question, posed in bits

A child c coded given an already decoded parent p costs $H(c \mid p)$ instead of $H(c)$; the gain is the mutual information $I(c;p)$, minus what must be stored to link the child to the parent. For a network, a single freedom makes two experts comparable: permuting the hidden neurons, which does not change the function (Proposition 5.1). Stone's duplication becomes

$$\max_{p,\,\Pi} I(c;\ \Pi p) \quad \text{against} \quad 3.84 \text{ bits per weight.} \tag{5.1}$$

The notation is that of the model: an expert of gpt-oss-20b at layer ℓ, index e; $d = f = 2\,880$; $W_g, W_u \in \mathbb{R}^{f\times d}$ (gate, up), $W_d \in \mathbb{R}^{d\times f}$ (down), biases $b_g, b_u \in \mathbb{R}^f$, $b_d \in \mathbb{R}^d$. The thresholds are declared in `mesures/genome/duplication/00-seuils-declares.md`, with an addendum and struck-out corrections, all prior to the reading of the sample.

5.2 The symmetry that makes kinship invisible

Proposition 5.1 (Invariance under permutation of the neurons)**.** *Let $\Pi \in \{0,1\}^{f\times f}$ be a permutation matrix. The expert with weights*

$$(\Pi W_g,\ \Pi W_u,\ W_d\Pi^\top,\ \Pi b_g,\ \Pi b_u,\ b_d)$$

computes the same function as $(W_g, W_u, W_d, b_g, b_u, b_d)$.

Proof. The expert computes $y = W_d\,\sigma(W_g x + b_g,\ W_u x + b_u) + b_d$, where σ acts component by component: $\sigma(g,u)_k = s(g_k, u_k)$ with $s(g,u) = \min(g,7)\cdot\mathrm{sigmoid}(1.702\cdot\min(g,7))\cdot(\mathrm{clamp}(u,-7,7)+1)$ (the bounded SwiGLU of gpt-oss, `ops::swiglu_oai` of `trc-reference`). A function applied component by component commutes with every permutation: $\sigma(\Pi a, \Pi b) = \Pi\,\sigma(a,b)$. So $W_d\Pi^\top\sigma(\Pi W_g x + \Pi b_g, \Pi W_u x + \Pi b_u) + b_d = W_d\Pi^\top\Pi\,\sigma(W_g x + b_g, W_u x + b_u) + b_d = y$, since $\Pi^\top\Pi = I$. ∎

Consequences. Two experts born from the same copy and then trained can have their neurons in any order: comparing W_g^A and W_g^B row i against row i sees nothing. Kinship is judged after the best matching of the neurons, as one matches those of two networks to connect or merge them [79, 80]. There is no other continuous symmetry: s is not homogeneous (the threshold 7, the sigmoid, the "+1" on u), so no rescaling of a neuron can be compensated by W_d. A matching by permutation is the only freedom.

What the permutation does to the MXFP4 blocks. The blocks of 32 run along the input of each matrix. For W_g and W_u (input d), Π permutes whole rows: each block stays whole. For W_d (input f), $W_d\Pi^\top$ permutes the columns: the blocks are crossed. A position-by-position conditional code stays exact in both cases (Section 5.4).

5.3 Raw kinship: screen, matching, family

5.3.1 Two lemmas for reading a cosine

Lemma 5.2 (The Gaussian gain of a parent)**.** *Let a child row x and a parent p be given, the second known to the decoder, with $x = \alpha p + z$, $z \perp p$, and $\rho = \cos(x,p)$. For a Gaussian source and a squared-error distortion $D < \sigma_x^2(1-\rho^2)$, coding the child given the parent saves*

$$g(\rho) = -\frac{1}{2}\log_2(1-\rho^2) \quad \textit{bit per weight,} \tag{5.2}$$

independently of D.

Proof. By orthogonal projection, $\|x\|^2 = \|\alpha p\|^2 + \|z\|^2$ and $\|\alpha p\|^2 = \rho^2\|x\|^2$, so $\sigma_z^2 = \sigma_x^2(1-\rho^2)$. The rate–distortion function of a Gaussian of variance σ^2 is $R(D) = \frac{1}{2}\log_2(\sigma^2/D)$ for $D < \sigma^2$ [9]. The child alone costs $R_1 = \frac{1}{2}\log_2(\sigma_x^2/D)$; given the parent, z remains to be coded, $R_2 = \frac{1}{2}\log_2(\sigma_x^2(1-\rho^2)/D)$, valid as long as $D < \sigma_z^2$. The difference is (5.2). ∎

Values [calc.]: $g(0.1) = 0.0072$; $g(0.2) = 0.0294$; $g(0.3) = 0.0680$; $g(0.5) = 0.2075$; $g(0.9) = 1.198$ bits. A cosine of 0.1 is worth only a hundredth of a bit.

Lemma 5.3 (The floor of chance)**.** *For a fixed target and N independent vectors uniform on the sphere of $\mathbb{R}^n$, n large, the maximum of the N cosines in absolute value is roughly $\sqrt{2\ln(2N)/n}$.*

Sketch. For u uniform on the sphere and e fixed, $\sqrt{n}\,\cos(u,e)$ tends in law to a standard normal. The maximum of N values $|Z|$, that is of $2N$ one-sided Gaussian tails, sits at first order where the expected number of exceedances is 1: $2N\,\mathbb{P}(Z > t) \approx 1$, and $\mathbb{P}(Z > t) \approx e^{-t^2/2}$ at first logarithmic order, hence $t \approx \sqrt{2\ln(2N)}$. Divide by $\sqrt{n}$. (The next-order corrections lower this level slightly.) ∎

For the dictionary of the screen ($N = 31 \times 2\,880 = 89\,280$ rows, $n = 2\,880$): $\sqrt{2\ln 178\,560/2\,880} = 0.0916$ [calc.]. The chance control of the screen gives 0.084 [meas.], of the same order, a little below because the rows of the dictionary are not independent. Any ρ found at the level of 0.08 to 0.09 is only a maximum of chance correlations.

5.3.2 The screen: an upper bound, without bijection

`ecran_parente.py` seeks, for each target row (a "gene"), the best $|\cos|$ among all the rows of the same role of the 31 other experts of the layer (89 280 candidates), without requiring a one-to-one matching: it is the upper bound of a gene-by-gene kinship. Layers 0 to 4 (log interrupted to free the memory of the machine, `ecran-20b-interrompu-couches-0-4.log`) [meas.]: for gate, $\bar{\rho}$ from 0.12 to 0.21 against a chance control of 0.084 (Gaussian gain of 0.010 to 0.039 bit); for down, $\bar{\rho}$ from 0.32 to 0.44 against 0.076 to 0.108 (0.09 to 0.18 bit). The planted control ($\rho_0 = 0.9$, `ecran-temoin-planter-0.9.json`) is recovered at 0.900 and 1.198 bits, and the chance dictionary does not see it (0.083).

5.3.3 One-to-one matching: the true kinship from expert to expert

`paires_arbre.py`: cost $\sum_{\text{roles}} \cos^2$ between neurons, assignment by the Hungarian algorithm [81] (the scipy implementation [82]), a single permutation joint over the three matrices (the function requires it), score = mean of $-\frac{1}{2}\log_2(1-\cos^2)$ over the matched neurons. Layer 12, four experts (Table 5.1).

Matched gate and up are at the level of chance; only down carries a little structure. The planted control is recovered to the thousandth of a bit: the instrument reads g.

5.3.4 The best parent of each child

The declared sample: children 0, 8, 16 and 24 of layers 0, 8, 16 and 23 (16 children, 48 matrices). For each, `mutation.py` seeks the best parent among 63 candidates (31 of its layer, 32 of the previous layer; `csv/scores-parente.csv`). Figure 5.1 shows the 880 scores.

Pair	cos gate	cos up	cos down	score (bit per weight)
(0, 2)	0.044	0.034	0.108	0.0053
(0, 3)	0.039	0.034	0.106	0.0050
(2, 3)	0.049	0.034	0.107	0.0049
chance	0.040	0.041	0.039	0.0016
planted, $\rho_0 = 0.5$	0.500	0.500	0.500	0.2077 (expected 0.2075)

Table 5.1: The Hungarian matching of four experts of layer 12, against a chance parent and a planted copy [meas.]. Data: `mesures/genome/duplication/paires-temoin-planter-0.5.json`. The planted permutation is recovered at 100 %.

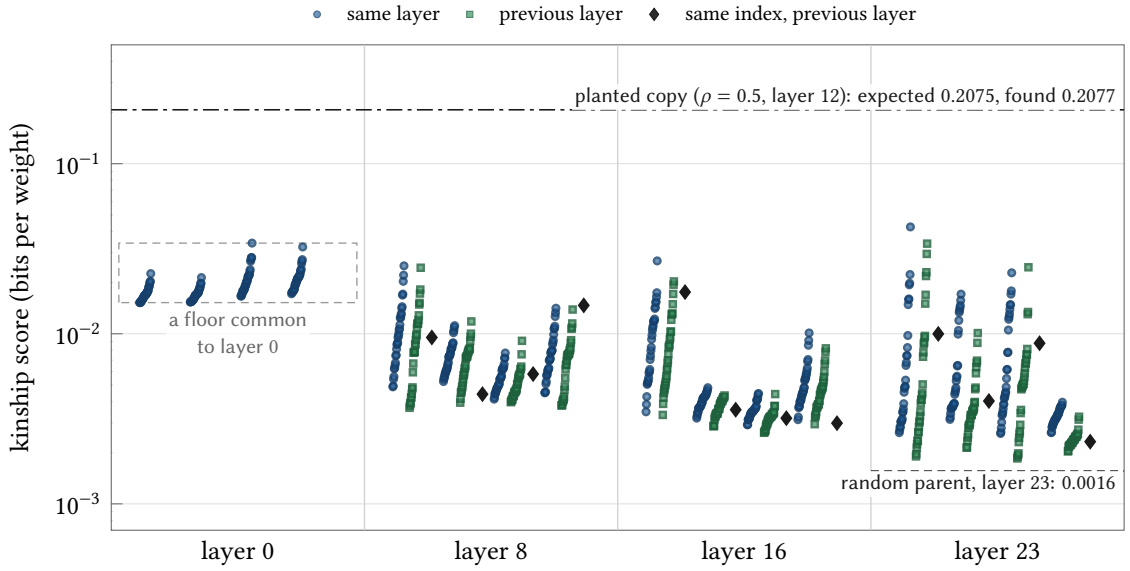


Figure 5.1: No expert is the copy of another: every candidate parent stays far below a planted copy. [meas.] For 4 child experts in each of 4 layers of gpt-oss-20b, the kinship score of every candidate parent: the bits per weight that the best permutation of its neurons (Hungarian matching on gate, up and down together) saves on the child, $-\frac{1}{2}\log_2(1-\rho^2)$ averaged over the three matrices. Each column of points is one child, its candidates ranked by score: the 31 other experts of the same layer (blue discs), 31 experts of the previous layer (green squares), the expert of the same index in the previous layer (ink diamonds). All 880 scores lie between 0.0018 and 0.0425 bit. The highest (child 0, parent 22, layer 23) is ×27 the random-parent control of its own layer (0.0016, 8 pairs; the control of layer 12, 2 pairs, gives 0.0016), and 4.9 times below a copy planted at $\rho = 0.5$, which the same instrument finds (dash-dot line: a control of the instrument, measured in layer 12). In layer 0, the 124 same-layer candidates all score from 0.0153 to 0.0341 (dashed frame), with a cosine of the down matrices from 0.23 to 0.31 (median 0.11 in the other layers): a floor common to the whole layer, a shared statistic of its columns, not a kinship; no random-parent control was measured in that layer. What the figure does not show: the cost of storing the permutation and the parent index (added in the lossless account D1, which falls), nor the 120b, measured on a single layer.

Data: `mesures/genome/duplication/csv/scores-parente.csv`, `paires-temoin-planter-0.5.json`, `arbre-c23.json`; script `memoire/outils/tome3_duplication.py`.

What the figure shows. All the scores lie between 0.0018 and 0.0425 bit per weight [meas.]. The highest, child (23, 0) and its parent (23, 22), is 27 times the chance control of its layer (0.0016, mean of 8 pairs; `arbre-c23.json`) and remains 4.9 times below a copy planted at $\rho = 0.5$, which the same instrument recovers. The best parent barely stands out from the lot: its score exceeds the median of the candidates by 0.0008 to 0.038 bit (`genome/duplication.md` § 2.3). Twelve parents out of sixteen are in the same layer, four in the previous one, of which only one at the same index: (7, 24) for (8, 24), at 0.0147 bit, the only trace of a filiation by index, and it is tiny. The same index one layer down is not favoured: its score is, depending on the child, below or above the median of the 32 candidates of the previous layer.

Layer 0, a common floor. In layer 0, the four children choose the same parent, (0, 9): a *hub*, not a lineage. The 124 same-layer candidates there all have a score of 0.0153 to 0.0341, with a cosine of the down matrices of 0.23 to 0.31 (median 0.11 in the other layers) [meas.]: a statistic shared by all the columns of the layer, not a kinship. The caption says so, and says too that no chance control was measured at this layer; the exact cost of Section 5.4 measures it by another path.

5.3.5 The family: a common subspace

`spectre_famille.py` computes, for layer 12, the Shannon rate of a Gaussian of the same spectrum at $\varepsilon = 4.29\,\%$ by reverse water-filling (`spectre-c12.json`) [meas.]: a common basis of the layer (the Karhunen–Loève transform of the family) would save 0.06 to 0.07 bit more than a diagonal basis, net of the control (gate 0.072, up 0.056, down 0.070). But it costs $n^2 = 8.3 \cdot 10^6$ numbers per layer and per role, that is 0.5 bit per weight in f16 spread over 32 experts [calc.]: seven times its gain. The diagonal gain of down (0.187 bit) is already taken by the E8M0 scales, which follow the norm of each row of down.

5.4 The exact cost of an expert coded as a mutation

5.4.1 The statements

Proposition 5.4 (The lossless gain is a conditional mutual information)**.** *Let C be the E2M1 code (16 symbols) of a weight of the child, κ its scale class (the class of the format: sign of e minus the upper median of the row, 3 classes) and P the code of the parent at the matched position. An entropy coder with one table per context κ spends $H(C \mid \kappa)$ bits per weight; with one table per (κ, P), $H(C \mid \kappa, P)$. The gain is*

$$H(C \mid \kappa) - H(C \mid \kappa, P) = I(C; P \mid \kappa) \;\geq\; 0, \tag{5.3}$$

zero if and only if C and P are independent given κ.

Proof. This is the definition of the conditional mutual information, $I(C; P \mid \kappa) = H(C \mid \kappa) - H(C \mid \kappa, P)$; its non-negativity and the case of equality are those of [2, th. 2.6.5 and 2.5.2]: it is a mean of Kullback–Leibler divergences between $p(c, p' \mid \kappa)$ and $p(c \mid \kappa)p(p' \mid \kappa)$, zero if and only if these laws coincide. ∎

The bias of the estimator. H is estimated by exact counts ("plug-in"). Its downward bias is, at first order, $(K-1)/(2N \ln 2)$ for K non-empty cells and N samples [83]. For $H(C \mid \kappa, P)$: $K \leq 3{\times}16{\times}16 = 768$ and $N = 8\,294\,400$, hence a bias of at most $767/(2{\times}8\,294\,400{\times}\ln 2) = 6.7 \cdot 10^{-5}$ bit [calc.]. The estimated mutual information is inflated by at most this amount: any gain below 10^{-4} bit is indistinguishable from zero.

The added information, counted [calc.].

- *The permutation*: $\log_2(f!)$ bits per expert, shared by its three matrices.

$$\log_2(2\,880!) = \frac{\text{lgamma}(2\,881)}{\ln 2} = 28\,948.6 \text{ bits;} \qquad \text{per weight, } \frac{28\,948.6}{3 \times 2\,880^2} = 1.163 \cdot 10^{-3}.$$

  Second path, by Stirling:

$$f \log_2 f - f \log_2 e + \tfrac{1}{2} \log_2(2\pi f) = 33\,096.5 - 4\,155.0 + 7.1 = 28\,948.6.$$

- *The tables*: 768 counts of 16 bits per matrix for D1, that is $768 \times 16/8\,294\,400 = 1.48 \cdot 10^{-3}$ bit per weight; 192 counts for D2b ($\kappa \times 8$ base symbols ×8), $3.70 \cdot 10^{-4}$.
- A parent is therefore useful only if $I(C; P \mid \kappa)$ exceeds $2.64 \cdot 10^{-3}$ bit per weight for D1, $1.53 \cdot 10^{-3}$ for D2b.

The chance parent is not a zero. Matching the child to its parent by a permutation drawn at random leaves the columns in place (for gate and up, input dimension j stays j): any structure common to *all* the experts along the column survives. The chance control measures this common part; the part proper to kinship is the difference $I(C; P \mid \kappa) - I(C; P_{\text{chance}} \mid \kappa)$.

Proposition 5.5 (The gain at identical reconstruction)**.** *If the reconstruction of the child is fixed (the base of B, or an ECSQ alone), with indices Q, and a parent context P' (the base symbol of the parent, or its quantised prediction) enters the coder, the error on the weights and at the output does not change by one bit, and the rate goes from $H(Q \mid \text{ctx})$ to $H(Q \mid \text{ctx}, P')$: the gain is $I(Q; P' \mid \text{ctx})$, with the same biases and the same costs as in Proposition 5.4.*

Proof. The reconstruction is a function of Q alone; adding a context to the entropy coder changes only the law under which Q is coded, hence only the rate, and Proposition 5.4 applies to Q instead of C. ■

This is the only way to credit kinship without confusing it with a change of quantiser.

Remark 5.6 (Why the subtracted prediction loses on a discrete grid). The weights of the child live on a discrete grid (16 values per block). A uniform quantiser whose step Δ falls on the grid (Δ a multiple of 2^{e-128}) gives a *zero* error on part of the weights. The residual $x - \alpha p$ is no longer on the grid: its quantisation error is about $\Delta^2/12$ everywhere. At equal rate, the ECSQ of the residual loses what the grid gave for free, and what the parent saves, $g(\rho)$ of lemma 5.2, is smaller as soon as ρ is small.

5.4.2 The measurements

Tables per matrix: `csv/d1-sans-perte.csv`, `csv/d2-avec-perte.csv`, `csv/d2b-base.csv`; verdicts: `verdicts.json` (tool `synthese.py`, whose control `--casser` makes D1 "work" everywhere and holds; `verdicts-casse.json`). Table 5.2 gives their means per layer and per role.

Reading (Table 5.3). Losslessly, the parent saves on average 0.0034 bit per weight on the codes, of which 0.0016 are recovered with a chance parent: the part proper to kinship is 0.0019 bit, below the $2.64 \cdot 10^{-3}$ of added costs. With loss, at identical reconstruction, the frontier of 2.6441 bits would at best come down to 2.6429. The subtracted mutation *loses* 0.49 bit, as remark 5.6 announced. The largest gain of a single matrix is 0.0170 bit (gate of (0, 16)), of which 0.0145 are recovered with the chance parent; the largest net gain is 0.0109 bit (gate of (23, 0), parent (23, 22)), then 0.0098 (gate of (23, 16), parent (22, 13)). Nothing comes near a tenth of a bit.

layer	role	$H(C \mid \kappa)$	D1 parent	D1 chance	D2b parent	D2b chance	ECSQ alone	subtracted	ECSQ + parent
0	gate	3.7686	0.0082	0.0072	0.0064	0.0056	1.564	1.882	1.562
0	up	3.7855	0.0064	0.0060	0.0050	0.0047	1.559	1.929	1.558
0	down	3.8185	0.0002	0.0001	0.0002	0.0000	1.049	1.283	1.043
8	gate	3.8208	0.0045	0.0004	0.0040	0.0003	1.538	2.049	1.533
8	up	3.8005	0.0008	0.0004	0.0006	0.0003	1.655	2.119	1.653
8	down	3.7984	0.0017	0.0001	0.0013	0.0000	1.626	2.168	1.619
16	gate	3.8099	0.0033	0.0004	0.0029	0.0002	1.490	1.999	1.485
16	up	3.8098	0.0007	0.0003	0.0005	0.0002	1.564	2.107	1.562
16	down	3.7988	0.0011	0.0001	0.0009	0.0000	1.568	2.177	1.564
23	gate	3.7978	0.0082	0.0019	0.0069	0.0013	1.449	2.091	1.439
23	up	3.7921	0.0041	0.0018	0.0031	0.0013	1.607	2.123	1.601
23	down	3.7892	0.0020	0.0001	0.0015	0.0000	1.584	2.121	1.579

Table 5.2: The exact cost of an expert coded as a mutation, means of 4 children per layer and per role, in bits per weight [meas.]. D1: lossless gain on the codes; D2b: gain at reconstruction identical to the base; the last three columns: rate at 4.29 % weight error of an ECSQ alone, of the subtracted mutation, and of the ECSQ with the parent as context (without the scales, 0.034 bit, nor α, 0.0028). Data: `mesures/genome/duplication/csv/d1-sans-perte.csv`, `d2-avec-perte.csv`, `d2b-base.csv`.

Measurement	What it requires (declared)	Result [meas.]	Verdict
D1, lossless	gain $> 2.64 \cdot 10^{-3}$ on the 48 matrices	mean gain 0.0034, chance 0.0016, net 0.0019; 21 out of 48	**does not work**
D2, subtracted mutation	below 2.6441 and below the ECSQ alone by more than one standard deviation of chance ($3.0 \cdot 10^{-4}$)	−0.486 bit on average: worse than without parent	**does not work**
D2c, parent context, same reconstruction as the ECSQ	same	+0.0016 bit, α included; 43 out of 48 beat chance by more than one standard deviation	a real gain, of no consequence
D2b, base + parent, same reconstruction as the base	gain $> 1.53 \cdot 10^{-3}$ and above chance on the 48	gain 0.0028, chance 0.0012; 24 out of 48; frontier 2.6441 → 2.6429	**does not work**
O, output	D2 at most the error of the base, 4 layers out of 4	base 6.4 %, mutation 8.6 %; 4 children out of 16 below the base	**does not work**

Table 5.3: The verdicts of duplication against the declared thresholds [meas.]. Data: `mesures/genome/duplication/verdicts.json`; `genome/duplication.md` § 3.

child	parent	tokens	base of B weights / output	mutation at $\varepsilon \approx 4.29\,\%$	ECSQ alone at $\varepsilon \approx 4.29\,\%$	truncated mutation weights / output	raw parent output
(0, 0)	(0, 9)	762	4.3 / 6.6	4.7 / 7.1	4.8 / 7.0	94.3 / 97.4	217.0
(0, 8)	(0, 9)	698	4.3 / 6.3	4.6 / 6.5	4.7 / 6.7	95.3 / 100.3	222.1
(0, 16)	(0, 9)	1 334	4.6 / 4.5	5.4 / 5.5	4.6 / 5.1	93.4 / 103.6	95.9
(0, 24)	(0, 9)	914	4.5 / 4.8	4.5 / 5.2	4.5 / 4.3	91.3 / 94.0	101.0
(8, 0)	(8, 31)	1 329	4.7 / 8.3	3.9 / 18.5	5.1 / 29.3	92.5 / 120.3	168.3
(8, 8)	(7, 30)	1 761	4.5 / 7.2	4.1 / 11.4	4.1 / 11.3	93.7 / 106.1	121.0
(8, 16)	(7, 11)	300	4.2 / 7.0	3.9 / 22.9	3.9 / 31.5	94.9 / 105.6	144.4
(8, 24)	(7, 24)	928	5.4 / 20.3	4.5 / 12.5	4.2 / 12.8	93.2 / 121.5	255.4
(16, 0)	(16, 17)	330	4.5 / 6.1	4.0 / 11.1	3.9 / 11.4	95.0 / 77.8	113.0
(16, 8)	(16, 24)	240	4.0 / 3.1	3.8 / 2.9	3.8 / 3.1	98.3 / 64.8	113.0
(16, 16)	(16, 6)	147	4.1 / 5.2	4.1 / 8.2	3.8 / 11.1	98.5 / 83.3	135.0
(16, 24)	(16, 11)	1 237	4.2 / 5.5	4.6 / 7.0	4.2 / 7.1	97.6 / 82.9	207.5
(23, 0)	(23, 22)	303	4.7 / 6.2	4.8 / 8.3	3.6 / 6.5	95.5 / 90.0	54.9
(23, 8)	(23, 16)	56	4.5 / 3.3	3.8 / 2.3	3.8 / 2.7	96.1 / 80.0	56.2
(23, 16)	(22, 13)	239	4.7 / 4.4	3.9 / 5.5	4.5 / 5.9	96.2 / 107.1	176.2
(23, 24)	(23, 11)	2 048	3.9 / 3.6	3.8 / 3.5	3.7 / 4.6	99.0 / 24.2	28.6
mean			— / **6.4**	— / **8.6**	— / **10.0**	**95.3 / 91.2**	

Table 5.4: Relative errors in %, weights then output of the expert, on the real inputs routed to each child [meas.]. Data: `mesures/genome/duplication/csv/t-o-sortie.csv`; `genome/duplication.md` § 4.

5.5 The truncated mutation and the output of the expert

Inputs: the normalised states really routed to each child (capture of the scout, 12 windows of 511 tokens, SHA-256 fingerprints in each `mutation-cL.json`), at most 2 048 tokens per child; output of the expert in f64, biases included; control: the exact weights give a zero error (`csv/t-o-sortie.csv`, column `temoin_exact`) [meas.].

Reading (Table 5.4).

- **The truncated mutation** (the child served by $\alpha\odot$ parent, residual discarded) keeps 1 to 9 % of the energy of the weights (4.7 % on average) and gives 24 to 121 % error on the output. The values below 100 % of layers 16 and 23 come from the biases: they stay exact and carry a large share of the output of the deep experts (for (23, 24), the raw parent itself makes only 28.6 %). This is the error one would have by serving only the biases: the truncated mutation brings nothing.
- **Same weight error, very different output error.** The ECSQ alone and the mutation fall to 3.6 to 5.4 % weight error, like the base; but the output error of the ECSQ alone ranges from 0.63 to 4.5 times that of the base: the common step erases the blocks of small scale, which the base keeps. Chapter 6 sees it from its side, by another path.
- (8, 24) is the exception where the base itself gives 20.3 %: its output error is not uniform either. It is the aberrant expert of the scout (Section 3.4), seen here on other inputs.
- The quantisation step for O is that of the sweep closest to 4.29 % (no interpolation possible on a reconstruction): hence the weight errors of 3.6 to 5.4 %.

5.6 The most economical tree

Proposition 5.7 (The optimal coding order is a maximum-weight spanning tree)**.** *If each expert is coded alone (B bits) or as a mutation of an already decoded parent (B − s bits, s ≥ 0 the gain of the edge, symmetric), the minimal cost of a valid decoding order of E experts is* $BE - \max_T \sum_{(i,j)\in T} s_{ij}$*, the maximum ranging over the spanning trees T of the complete graph of the experts, and Kruskal's algorithm [84] finds it.*

Proof. A valid decoding order gives every expert but one (the root) a parent decoded before it: the "child, parent" edges form a spanning tree oriented from the root, and the cost is $BE - \sum_T s$. Conversely, every spanning tree, rooted at any expert and traversed breadth-first, gives a valid order of the same cost, since s is symmetric. Minimising the cost therefore amounts to maximising $\sum_T s$. Kruskal's greedy choice (take the heaviest edge that does not close a cycle) is optimal by the cut property: for every cut of the graph, an edge of maximal weight crossing it belongs to a maximum-weight spanning tree. (For non-symmetric gains, one would need the maximum-weight arborescence of Chu–Liu and Edmonds; our scores are symmetric by construction, $\cos^2$ and matching.) ■

The gain per weight of the layer is $\sum_T s/E$, and it does not exceed $\frac{E-1}{E} \max s$. `paires_arbre.py--couche23` matches one to one the 496 pairs of the most "related" layer of the sample, then draws the tree (`arbre-c23.json`) [meas.]: scores of the pairs, 10 % / median / 90 % / 99 %: 0.0031 / 0.0038 / 0.0138 / 0.0689 bit; chance control (8 pairs): 0.0016; edges of the tree, min / median / max: 0.0037 / 0.0101 / 0.0892. **Gain of the layer coded by its tree: 0.0212 bit per weight**, as a Gaussian bound.

What the figure shows. Figure 5.2 draws the tree by its own structure, not by a projection: a star around expert 31, which carries the four heaviest edges, (17, 31) at 0.089, (5, 31) at 0.089, (15, 31) at 0.077, (25, 31) at 0.069, then thin branches; 16 experts out of 32 are leaves. Expert 31 is a hub, the only trace of a family in all the measurements. The inset is the shadow of the same experts by a classical MDS on $d_{ij} = \log_2(s_{\max}/s_{ij})$; its fidelity is stated in the caption and must be stated in the text: the plane keeps 30.9 % of the variance of the distances, and the rank correlation between distances in the plane and true distances is 0.61, against 0.11 for permuted distances (`phylogenie-c23.json`) [meas.]. Two neighbours on this shadow are often, not always, neighbours in the data. The judge of the figures noted that in Figure 5.2 four edge values out of ten are closer to another edge than to their own (`memoire/tome3/JUGEMENT-figures3.md`, M4, a reserve on a figure that holds): the value of an edge is read in the CSV, not by eye. And above all, the figure is not a genealogy: no edge is a copy, the tree is the cheapest coding order, not a history.

The heaviest edge, in real bits. `mutation.py--couche23--enfants17--parent23,31` (`mutation-arete-max-c23-e17-p31.json`) [meas.]: D1 parent 0.0222 / 0.0136 / 0.0149 bit (gate, up, down), D1 chance 0.0003 / 0.0004 / 0.0001, D2b 0.0199 / 0.0108 / 0.0117; weight error of the truncated mutation 83.9 / 91.4 / 91.4 %. Output of expert 17: base 10.4 %, mutation at $\varepsilon \approx 4.29$ % 15.3 %, truncated mutation 109.9 %. This is the best case found in the model: 0.017 bit on average, 0.014 net of the added costs, that is 3.84 → 3.83. The ratio between the real gain on the codes and the Gaussian score is here $0.017/0.089 = 0.19$, and $0.0076/0.0425 = 0.18$ for (23, 0); applied to the tree, it gives about **0.004 bit per weight for the whole of layer 23** [calc.], before the added costs ($2.6 \cdot 10^{-3}$): a net gain of the order of a thousandth.

On gpt-oss-120b. `paires_arbre.py--modele120b--couche18--experts8` (28 pairs, `paires-120b-c18-8experts.json`) [meas.]: scores from 0.0030 to 0.0054 bit (chance 0.0016);

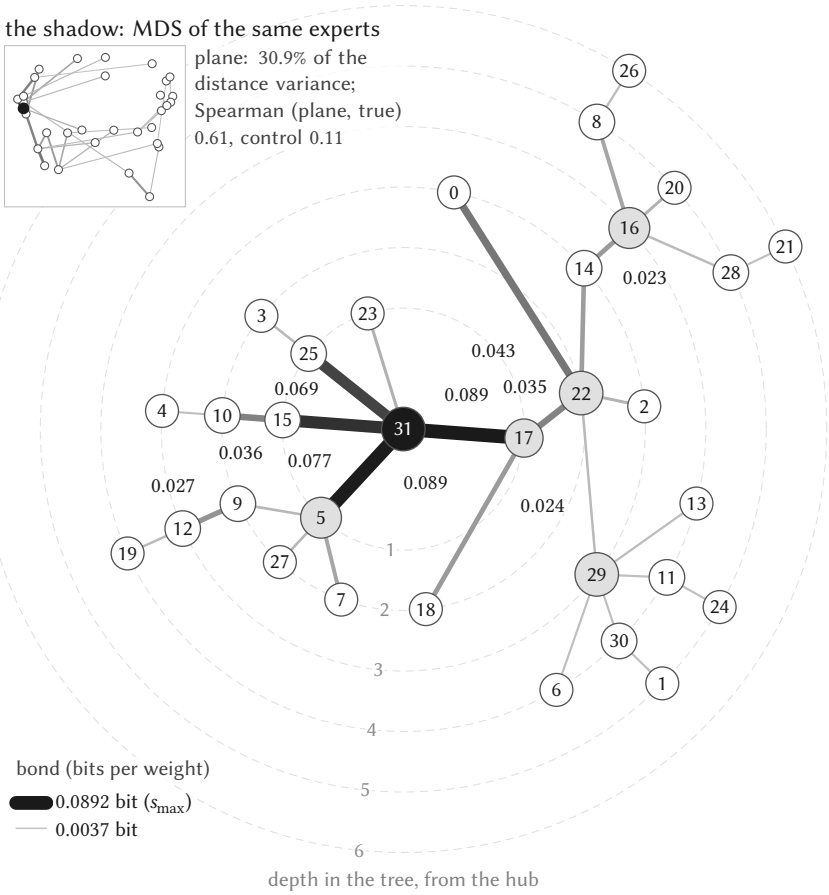


Figure 5.2: The cheapest coding order of the 32 experts of layer 23 is a star around expert 31, then thin branches. [meas.] [calc.] Every pair of experts (496 pairs) gets a kinship score s_{ij}; the maximum spanning tree (Kruskal) keeps the 31 edges that would cost the fewest bits if each expert were coded knowing its parent. The tree is drawn by its own structure, not by a projection: the hub at the centre, each expert on the circle of its depth, each subtree in a sector proportional to its leaves plus one, so no two edges cross (a drawing computation, not a measurement). Edge thickness and depth of grey follow $s_{ij}/s_{\max}$, and the bonds above a quarter of $s_{\max}$ carry their value, set beside the edge where its box touches no other line, disc or value (a drawing computation). Disc size follows the degree; tinted discs have at least three neighbours. 3 experts have the highest degree, 5 (22, 29, 31); expert 31 is the hub by weight (ink disc): its edges sum to 0.335 bit, against 0.116 for expert 22, and the 4 heaviest edges of the tree all touch it; 16 experts are leaves. Scale: even the strongest bond is worth 0.089 bit per weight, against 0.0016 for a random parent of the same layer. Inset: the shadow of the same experts by classical MDS on $d_{ij} = \log_2(s_{\max}/s_{ij})$, the drawing of the first pass; the plane keeps 30.9% of the variance of the distances, and the rank correlation between plane and true distances is 0.61 (control with permuted distances: 0.11): neighbours on that page are often, not always, neighbours in the data. What the figure does not show: a genealogy. No edge is a copy; the tree is the cheapest coding order, not a history.

Data: `mesures/genome/duplication/csv/phylogenie-c23-noeuds.csv`, `phylogenie-c23-aretes.csv`, `phylogenie-c23.json` (`outils/genome/duplication/figure_phylogenie.py`), `arbre-c23.json`; script `memoire/outils/tome3_duplication.py`.

layer	ε best paralogue	ε, $\beta = 1$	ε chance	ε, 5th choice	layer: substituted	layer: removed
0	0.792	1.401	0.945	1.215	0.167	0.133
8	0.842	1.439	0.936	1.239	0.139	0.092
16	0.658	0.749	0.721	0.924	0.071	0.076
23	0.335	0.435	0.467	0.564	0.037	0.037

Table 5.5: The relative error of an expert served by another, mean over the experts of each layer (32, 32, 31, 31), at most 96 routed tokens per expert [meas.]. Routing recomputed from the F32 router of the GGUF, equal to the captured routing at 100 % (control). The last two columns: relative error of the output of the *layer* when the expert is substituted by its best paralogue, or removed (routing renormalised over the other three). Data: `mesures/genome/duplication/csv/paralogues.csv`, `paralogues-c*.json`.

matched cosines 0.038 (gate), 0.032 (up), 0.094 (down) against 0.040 at chance. The same picture as on the 20b. The deliverable of the subject says so by striking out its own sentence: "~~gpt-oss-120b is not measured~~"; it is, on one layer and 8 experts out of 128, and the exact cost (D1, D2b) and the paralogues are not redone there.

5.7 Paralogues: serving an absent expert by another

A cache miss could be served without reading a byte, by an already resident expert: a paralogue, in the sense of the analogy. Life draws a robustness from its gene families: a removed gene is often compensated by a paralogue [85]. Cache-aware routing [86] favours the reuse of experts already in memory and is judged by task accuracy and speed; here we measure the functional bound of any substitution.

Lemma 5.8 (The error of a paralogue)**.** *Let y_c be the output of the missing expert and y_s that of a resident expert, on the inputs routed to c. With a scalar gain β chosen by least squares,*

$$\varepsilon(c,s) = \min_{\beta} \frac{\sum \|y_c - \beta y_s\|^2}{\sum \|y_c\|^2} = 1 - \rho_f^2, \qquad \rho_f = \frac{\langle y_c, y_s\rangle}{\|y_c\|\,\|y_s\|}, \quad \beta^* = \frac{\langle y_c, y_s\rangle}{\|y_s\|^2}, \tag{5.4}$$

the inner product ranging over all the tokens and all the components.

Proof. $\|y_c - \beta y_s\|^2 = \|y_c\|^2 - 2\beta\langle y_c, y_s\rangle + \beta^2\|y_s\|^2$ is a quadratic in β, minimal at β^*, where it equals $\|y_c\|^2 - \langle y_c, y_s\rangle^2/\|y_s\|^2 = \|y_c\|^2(1-\rho_f^2)$. ∎

For the output of the layer, expert c enters with its routing weight $g_c \in (0,1)$: the relative error of the layer is about $\sum g_c^2\|y_c - y_s\|^2 / \sum \|\sum_j g_j y_j\|^2$, smaller than ε by a factor that the measurement gives. The paralogue reads no byte: it beats any compression in rate. It is worth something only if its error is at the level of that of the base.

Reading (Table 5.5). The outputs of the experts resemble each other more in depth, but through a component *common to all*: the chance control falls with them (0.47 at layer 23), and what is proper to the best paralogue is only 0.13 above chance. A cache miss is much better served by the base of B (expert error of 5 to 8 %, Table 5.4) than by the best neighbour (33 to 84 %).

A sentence of the sources, corrected. The deliverable of the subject and the synthesis of the round write: "in the output of the layer, ~~substituting is never better than removing the expert~~ (16.7 against 13.3 %; 13.9 against 9.2; 7.1 against 7.6; 3.7 against 3.7 %)" (`genome/duplication.md` § 0, point 6; `genome/05-synthese.md` § 3.2). The numbers in parentheses contradict it: at layer 16, the mean of the substituted error (0.071) is *below* that of the removed error (0.076). Redone here from `csv/paralogues.csv` [calc.] on [meas.]: substituting is better than removing for 4 experts out of 32 at layer 0, 5 out of 32 at layer 8, **25 out of 31 at layer 16** and **21 out of 31 at layer 23**, where the two means are equal to within 10^{-3}. The correct sentence is therefore: *removing the expert is better than substituting it at layers 0 and 8; at layers 16 and 23, the best paralogue does a little better than removal for the majority of the experts, but by at most half a point of layer error on average.* The conclusion of the chapter does not change: the bar for a paralogue is not the absent expert but the removal of the expert, and the best paralogue departs from it by only a few thousandths, far from what the base achieves.

5.8 Why the idea fails, and where to look

The reason, in numbers. A duplicated gene keeps its resemblance as long as selection does not take it away. The experts of gpt-oss, for their part, were trained to be *different*: the router sends an expert only the tokens where it is useful, and two experts that resembled each other would share the same tokens. The measurement confirms it: after the best permutation, the mutual information between a child and its best parent is of the order of 10^{-3} bit per weight out of 3.84, and what remains of it is for half or more a property of the whole layer (the input columns, the output space of down), not of the pair. Truncating the mutation means losing the expert: 96 % of its energy is its own. *They pose the problem thus* (Git Re-Basin, Entezari et al., MC-SMoE [23]): match the neurons of two networks or of two experts in order to merge them, judged by task accuracy. *The term we lacked*: the rate, in bits per weight read, of an expert coded given another. *The one they do not have*: the measurement on 4-bit codes, where the mutual information is counted exactly, and the separation between proper kinship and the structure common to the layer, by the control of the chance parent.

Where the noise is, where the structure is. (a) Part of the structure between experts is **common to the layer**, not proper to a pair: the chance parent (same input columns, neurons in any order) takes back 46 % of the gain on average, and 88 to 94 % in layer 0, where the hidden state carries outlier channels. These are statistics per input column shared by the 32 experts; Chapter 6 takes them as a column context, and it is the only context that passes its threshold. (b) Proper kinship grows with depth (layer 23: D1 gate 0.0082 against 0.0019 at chance; a hub, expert 31) but stays below a hundredth of a bit. (c) The rest is noise for any duplication: the best parent explains 1 to 9 % of the energy of the weights of a child (at most 16 % for a single matrix).

What the measurement saw in passing. The ECSQ alone (common step, exponent context) falls to 0.89 to 1.78 bits (1.52 on average, without the 0.034 bit of scales) at 4.29 % *weight* error, far below the frontier; but its *output* error varies from 0.63 to 4.5 times that of the base. The lever is not kinship, a property of the weights among themselves, but *usage*, a property of the weights before their inputs: a weight is worth what its real inputs make it weigh. The subject drew its direction for what followed from this (answer by delegation of 2026-10-17, reversible: it is the only mechanism measured here whose gain counts in tenths of a bit and not in thousandths); Chapter 6 measures this allocation by usage, and Chapter 8 will say what the KL left of it.

What could be wrong

- **The parent is chosen by a Gaussian score on the weights**, not by the gain on the codes. Another parent might gain a little more in D1. The best score of the whole list (0.043) bounds the Gaussian gain of any parent; the real D1 is always below the score.
- **A single permutation per pair, joint over the three roles.** Permutations per role would be freer but would no longer describe the same expert (the function requires the same Π). The screen without bijection bounds what any further freedom could gain.
- **The sample**: 16 children out of 768, four depths. Layer 23 shows a stronger kinship than the others; layers 17 to 22 are not measured. The screen was interrupted at layer 4.
- **The real inputs** come from 12 windows of a single corpus; at most 2 048 tokens per child, 78 to 96 for the paralogues.
- **The entropy estimator** is biased downwards by at most $6.7 \cdot 10^{-5}$ bit: negligible compared with the reported gains, but of the same order as the smallest.
- **Lemma 5.3** is an asymptote; the measured control (0.084) is a little below it (0.092).
- **The sentence corrected in Section 5.7** is corrected on means of 31 and 32 experts at 96 tokens at most; the gap between substituting and removing at layers 16 and 23 is a few thousandths, with no published uncertainty.
- **The machine was shared and loaded** (free memory from 0.7 to 2.6 GB): the durations in the logs measure nothing, and no throughput number is reported.

Chapter 6

Predictive coding and synaptic pruning

Where we start, what this chapter establishes, what remains

The starting point is that, on the 12 expert tensors of the frontier (gpt-oss-20b, layers 0, 8, 16 and 23, gate, up and down; 3 185 049 600 weights), lossless coding reads 3.8431 bits per weight (A_ctx), and the base of B 2.6441 bits for 4.2920 % relative squared error; the scale context κ (below, equal to, above the median of the row) is already in the format. The reading predicted that lossless predictive coding would gain only what the neighbour brings (0.005 bit), and that pruning would depend on the output, not on the weights (Table 2.5). **This chapter establishes** the exact gain of the synonymous codons of E2M1, proved and measured with the real codec on the two whole models; that lossless predictive coding, in the strict sense, removes almost nothing, except one predictor: the energy class of the *input column*; a system lossless in value, folded zero and column, at **3.6844 bits** against 3.8434, decoded exactly; the rate–distortion frame in which synaptic pruning is only the zero-rate regime of a reverse water-filling; that the metric of the frontier is wrong for judging a loss; and that the output of the experts, then the KL against the reference, make everything that fills across the absolute scales fail, even at equal expert error, leaving only one lossy point: partitions relative to the row, tensor by tensor, at **0.911 times the bits** of the base and a **statistically equal KL**. **What remains** is the card kernels for the tables per context, error feedback on the alphabet of the cells, a sensitivity criterion judged per token or against the KL, and the measurement on the 24 layers, then on the 120b. Sources: `genome/predictif-elagage-pour-le-memoire.md` (the deliverable of the subject); `genome/REPRISE-predictif-elagage.md`; `mesures/genome/predictif-elagage/` (`resume-12.csv`, `taux-reel-*.json`, `contextes-*.json`, `debit-distorsion-12.*`, `plans/`, `sortie-experts-*`, `kl/`); `genome/05-synthese.md` § 1.1; tools under `outils/genome/predictif-elagage/`.

In brief

Two moves of the nervous system served as guides, as analogies. The retina does not transmit the image, it transmits what distinguishes it from what its neighbours predicted: that is predictive coding [32], and a neuron of the fly's eye even adjusts its response to the law of its inputs to draw the most information from them [87]. The developing brain, for its part, makes too many synapses and then removes those that activity does not call upon: that is synaptic pruning [38]. Transposed to the weights, the first move asks what the neighbours of a code say about it, and the answer is: almost nothing, except the column through which it reads its input. The second move is formulated exactly in rate theory: set a price for the bit, and let the synapses that do not pay it disappear. It gains a great deal on weight metrics, and almost all of that gain vanishes when judged on text. What holds is modest and solid: a zero sign that one stops paying for, a map of columns, and a base that saves 9 % of the bits without degrading the divergence beyond

Tensors	lossless to the bit	zero folded	gain	log
12 of the frontier (20b)	3.8434	**3.7064**	0.1370	`taux-reel-12-gptoss20b.json`
72, all layers (20b)	3.8509	**3.7230**	0.1280	`taux-reel-toutes-gptoss20b.json`
108, all layers (120b, 60.9 GB)	3.8549	**3.7292**	0.1257	`taux-reel-toutes-gptoss120b.json`

Table 6.1: The zero-synonym codon with the real codec (rANS of study 03, accounting of the arbiter, A_flux), in bits per weight [meas.]. The "lossless to the bit" row recovers exactly the arbiter's A_flux (3.8434): a second path of the same codec. Logs under `mesures/genome/predictif-elagage/`.

> the noise of the measurement.

6.1 The synonymous codons of E2M1

The fact [meas.]. On the 12 tensors, $P(+0) = 0.068\,46$ and $P(-0) = 0.068\,48$; more broadly, $P(+c) = P(-c)$ to within 10^{-4} for each magnitude. Per tensor, $P(+0)$ and $P(-0)$ differ by less than 10^{-4}, from 0.059 to 0.098 each (`histo-12-gptoss20b.npz`). The quantiser that produced the MXFP4 of gpt-oss keeps the sign of the original value even when its magnitude rounds to zero. Yet E2M1 has two codes for zero, `0b0000` and `0b1000`; ggml reads both as the integer 0, the f64 reference as $+0$ and -0, which change no non-zero sum (Proposition 3.1).

The biological parallel, as an image. The synonymous codons of the genetic code (Section 1.2) often differ by their third base, Crick's "wobble" position [88], and a mutation that goes from one to the other is silent. The two codes ± 0 are two synonymous codons; the information that distinguishes them has no phenotype. Canonicalising every -0 into $+0$ before entropy coding amounts to reducing codon usage to a single codon per meaning.

Proposition 6.1 (The exact gain of folding)**.** *Let C be the 4-bit code of a weight and $V = v(C)$ its value, $v(c) = (-1)^{c_3} K[c \,\&\, 7]$, injective except on $\{0, 8\}$. Let S be the sign bit. Then*

$$H(C) = H(V) + P(V = 0)\, H(S \mid V = 0). \tag{6.1}$$

The same statement holds conditionally on any context the decoder knows (scale class κ, neighbouring byte).

Proof. V is a function of C, so $H(V \mid C) = 0$ and $H(C, V) = H(C)$. By the chain rule, $H(C, V) = H(V) + H(C \mid V)$. Given $V = v \neq 0$, C is determined (v injective outside 0): $H(C \mid V = v) = 0$. Given $V = 0$, $C \in \{0, 8\}$ is determined by S: $H(C \mid V = 0) = H(S \mid V = 0)$. Hence $H(C \mid V) = \sum_v P(V = v)\, H(C \mid V = v) = P(V = 0)\, H(S \mid V = 0)$. Replacing $H(\cdot)$ by $H(\cdot \mid \kappa)$ everywhere changes no step. ∎

Folding 8 into 0 replaces C by C' of the same value, and $H(C') = H(V)$: the gain is $P(V = 0)\, H(S \mid V = 0)$, that is roughly $P(V = 0)$ bit when the sign is a fair coin. **Computation set out** [calc.]: $P(V = 0) = 0.068\,46 + 0.068\,48 = 0.136\,94$; $H(S \mid V = 0) = h(0.068\,46/0.136\,94) = h(0.499\,93) = 1.0000$ bit, h being the binary entropy; gain = 0.136 94 bit per weight. Measured by the conditional entropy (context κ, tables per tensor): 0.136 939 [meas.]; by the real codec: 0.1370 [meas.]. The form of the gain per byte, which serves the format, is taken up in Chapter 9.

Predictor, in addition to κ	H (bits per weight)	gain	verdict
none (κ alone)	3.7923	—	reference
exact δ (± 2)	3.7915	0.0007	ruled out
whole byte (pair of nibbles of the block)	3.7885	0.0037	already in A_ctx
left code	3.7846	0.0076	ruled out
code above (previous row)	3.7831	0.0091	ruled out
parent expert (same place, expert $e-1$)	3.7879	0.0043	ruled out
left × above (867 contexts)	3.7922	0.0000	ruled out (tables)
constraint of the block maximum	3.7921	0.0001	ruled out
class of the input column (quartile)	**3.7608**	**0.0314**	**retained**

Table 6.2: Lossless predictive coding: what each context removes from the entropy of a code, 12 tensors, tables included [meas.]. Declared threshold: 0.015 bit. Data: `mesures/genome/predictif-elagage/contextes-gptoss20b.json`.

The controls. Each stream is decoded and compared byte by byte, no lane outside its final state; each decoded nibble returns the original value ($d = o$, or $o = 8$ and $d = 0$). Break (8 sent to 1): the value control fails (`taux-reel-12-gptoss20b-casse.json`). End to end, the f64 reference with rebuilt weights (hook `alterer_expert`) gives, for the identity plan, `.kld` entries identical byte for byte to the reference (510 entries) and `juge kl` = 0 exactly (`kl/identite-2f-octets.json`); the folded-zero plan gives identical values, hence the same output. These measurements join those of the scout by another path (Table 3.4) and extend them to the whole 120b.

6.2 Lossless predictive coding: what the neighbours say about a code

The frame. For a discrete alphabet, "predict, then code the deviation" is written exactly: code the code with a law conditioned on a context X that the decoder knows. The gain is the mutual information

$$I(C; X \mid \kappa) = H(C \mid \kappa) - H(C \mid \kappa, X). \tag{6.2}$$

The frequency estimator is biased downwards; by Miller's expansion [83], with tables per tensor, the bias on the gain is $(K_X - K_\kappa)(16-1)/(2N_t \ln 2)$ bit per symbol, K being the number of contexts and $N_t = 265\,420\,800$ weights per tensor. **Threshold declared beforehand**: 0.015 bit per weight, the header cost of one more stream (0.40 % of 3.84 bits, study 03).

Reading (Table 6.2).

- **The neighbours predict almost nothing.** Given the scale of its block, an MXFP4 code is an almost independent draw; what the literature calls the "spatial dependency" of the weights is here below 0.01 bit.
- **The parent expert predicts nothing** (0.004): the experts of a layer are not mutated copies of one another at the level of the codes, which Chapter 5 puts in numbers on the values, with permutation.
- **Each block contains at least one large code** ($|\text{E2M1}| \geq 2$): 0 blocks out of 99 532 800 lack one; the OCP rule of the scale ($2^{\lfloor \log_2 \max \rfloor - 2}$) is indeed followed, but the constraint is worth only 10^{-4} bit: a block almost always has several large codes.
- **The input column carries a structure.** Some columns have, on average over all the rows and all the experts of a tensor, larger weights relative to the scale of their block. The ratio between

the most energetic column and the median ranges from 1.11 (blk.0 down) to 2.93 (blk.23 gate), in root energy. The inputs of gate and up are the hidden state, some of whose channels carry outlier activations; training answered with weights of different size. It is a fact of the input, shared by the 32 experts: a map of 2 bits per column and per tensor (720 bytes) suffices. It is the same statistic that the chance parent of Chapter 5 took back at 88 to 94 % in layer 0.

The controls, and a struck-out correction. Context equal to the code itself: $H = 0$ (holds). "Left" context randomly permuted: gain of $6.33 \cdot 10^{-5}$ for an expected bias of $6.26 \cdot 10^{-5}$ (holds, `contextes-temoin-corrige.json`). The deliverable carries a first formula, struck out: "~~Expected bias $(51-1) \cdot 15/(2N \ln 2) = 5.4 \cdot 10^{-6}$ with pooled N, control failed.~~" Corrected on 2026-10-16: the tables are per tensor, the bias is computed with N_t and $51 - 3$ added parameters; with the old formula (the break of the corrected control), the control does fail (`contextes-temoin-corrige-casse.json`).

The real codec, $\kappa\times$ column. Streams of nibbles separated by context, alphabet of 16, zero folded, map and tables counted (`taux-reel-colonne-12-gptoss20b.json`) [meas.]: **3.6844 bits per weight**, against 3.7103 for the same split in κ alone (real gain of the column: 0.0259) and 3.7064 for the byte alphabet in κ. Per tensor, the column saves nothing on blk.23.ffn_down (3.6744 → 3.6741: the input of down is the output of a SwiGLU, with no outlier channel) and most of it on gate and up. The system lossless in value, zero folded and column, therefore reads **3.6844 bits against 3.8434**, that is −4.1 % [calc.], and Chapter 8 shows where this gain lives (almost all of it in layer 0).

6.3 Pruning and bounded predictive quantisation: the rate–distortion frame

The problem. Replace each code c, in a block of scale e, by a cell s of a partition P_k of the 16 codes ordered on the real line ($-12, \dots, -1, -0, +0, +1, \dots, +12$ in doubled E2M1 magnitudes K), chosen according to a context k that the decoder knows, rebuilt as $\hat{w} = r_{k,s}\, 2^{e-128}$. Rate: $H(s \mid k)$, one table per tensor and per context. Distortion: the sum of the $(w - \hat{w})^2$. It is an entropy-constrained quantiser on a discrete source [89, 90].

Lemma 6.2 (The optimal centre of a cell)**.** *For a fixed cell, the weighted centroid $r^* = B/A$ is the reconstruction that minimises $\sum_i 4^{e_i - 128}(K_i - r)^2$, and the remaining error is $C - B^2/A$, with $A = \sum_i 4^{e_i}$, $B = \sum_i 4^{e_i} K_i$, $C = \sum_i 4^{e_i} K_i^2$ (up to a factor 4^{-128}).*

Proof. The function $r \mapsto C - 2rB + r^2 A$ is a quadratic in r, with derivative $2(Ar - B)$, zero at B/A; $A > 0$ makes it a minimum, where the value is $C - 2B^2/A + B^2/A = C - B^2/A$. (The vector case is that of Chou, Lookabaugh and Gray [89].) ∎

Lemma 6.3 (Exact dynamic programming)**.** *With a multiplier μ, the cost of a partition into intervals of the 16 ordered positions is a sum of terms each depending only on its cell, $c(i, j) = D_{[i,j]}/E + \mu\, n_{[i,j]}\big(-\log_2(n_{[i,j]}/n_k)\big)$, E being the energy of the context and n the counts. The best partition is obtained exactly by the recurrence*

$$J(0) = 0, \qquad J(j) = \min_{0 \le i < j} J(i) + c(i, j-1), \tag{6.3}$$

in $16 \times 17/2 = 136$ cell evaluations, instead of the 2^{15} partitions.

Family, at mean error per tensor equal to the base (4.337 %)	bits per weight	ratio to the base	share pruned
base (frontier, same instrument)	2.632	1	—
pruning by magnitude (the rest exact, zero folded)	2.665	1.012	—
free partitions, context κ	2.359	0.896	—
free partitions, row δ bounded to ±3	2.353	0.894	—
dead zone alone, context e (Lagrangian pruning)	2.318	0.881	—
free partitions, context e (reverse water-filling)	**2.019**	**0.767**	31 %

Table 6.3: The families of the rate–distortion frame, in weight error [meas.]. Ideal entropies, plus the scales (0.0375) and the tables; the base with the same instrument is 2.6321, against 2.6441 with the real codec (gap of 0.46 %). Data: `mesures/genome/predictif-elagage/debit-distorsion-12.json` and `.csv`.

Proof. A partition into intervals of $\{0,\dots,j-1\}$ ends with a last interval $[i, j-1]$, preceded by a partition of $\{0,\dots,i-1\}$. The cost being additive by cell, the minimal cost decomposes as (6.3) (Bellman's principle). There is one possible cell per pair $i \le j$ of the 16 positions, that is 136, each evaluated once by lemma 6.2 on cumulative sums. This is the histogram segmentation of Muresan and Effros [91], and of Wu and Zhang [92]. The cells are intervals because, for the squared distortion, an optimal cell is convex on the line. ■

Proposition 6.4 (Reverse water-filling, and pruning as its zero-rate regime)**.** *A μ common to all the contexts achieves the minimum of the total distortion under a total rate constraint: at the points where each context is on its convex curve $D_k(R_k)$, the Karush–Kuhn–Tucker condition is $-\partial D_k/\partial R_k = \mu$ for every active context, and a context whose slope at zero rate is weaker than μ stays at zero rate. The low-energy contexts (small scale e, little-active column) reach their zero rate first: a single cell, the whole context rebuilt by its centre, almost zero.*

Proof. We minimise $\sum_k D_k(R_k)$ under $\sum_k R_k \le R$, $R_k \ge 0$, with convex decreasing D_k. The Lagrangian $\sum_k D_k(R_k) + \mu(\sum_k R_k - R) - \sum_k \nu_k R_k$ gives, at the optimum, $D_k'(R_k) + \mu - \nu_k = 0$, $\nu_k \ge 0$, $\nu_k R_k = 0$. For an active context ($R_k > 0$), $\nu_k = 0$ and $-D_k'(R_k) = \mu$; for an inactive context, $-D_k'(0) = \mu - \nu_k \le \mu$. Convexity makes these conditions sufficient. This is the reverse water-filling of [2, th. 10.3.3], transposed to a discrete source by contexts. A low-energy context has a slope $-D_k'(0)$ proportional to its energy: it becomes inactive first. ■

Synaptic pruning thus appears as the zero-rate regime of reverse water-filling: one does not choose to prune, one chooses a price for the bit, and the synapses that do not pay it disappear. This is the move of Optimal Brain Damage [39] and of Deep Compression [41], written on our cells.

The metric, and why that of the frontier is wrong here. The first pass used the global error of the frontier, $\sum(w-\hat{w})^2 / \sum w^2$ over the 12 tensors. Yet the share of the energy per tensor is extremely unequal [meas.]: blk.23.ffn_down 91.5 %, blk.16.ffn_down 8.0 %, blk.8.ffn_down 0.40 %, the other nine together 0.015 %. A common μ under this metric empties the nine small tensors: 0.32 bit "at 4.29 %" (`ecsq_e_metrique_globale`, for the record), absurd. **Metric retained**: the mean over the tensors of their own relative error, each matrix counting as much; the base is 4.3367 % in it (from 4.05 to 4.80 % per tensor).

Why pruning by magnitude loses (Table 6.3). Pruning means folding the small magnitudes into the zero symbol; the rate falls only by the entropy of these symbols, while all the other codes keep

Family	predicted output error	bits per weight	ratio to the base (2.632)
base	4.489 %	2.632	1
partitions by e (weight metric)	4.494 %	2.022	0.768
partitions by e, masses weighted by activity	4.461 %	2.005	0.762
partitions by (e, q)	4.487 %	1.695	0.644

Table 6.4: Activity-dependent pruning, at output error *predicted* by (6.4) [meas.] [calc.]. At this point, 40.9 % of the weights are folded into the cell that contains zero. Data: `mesures/genome/predictif-elagage/plans/debit-sensibilite-12.json`.

their 3 bits and more. At equal error, it is better to enlarge *all* the cells a little than to empty a few. The small magnitudes carry little energy, but also few bits: the 50 % smallest weights of a tensor carry only 4.3 % of its energy, and removing them removes only about half the bits that would be needed. Reverse water-filling by e, for its part, gains 23 % of the bits on the weight metric; Section 6.5 says what is left of it.

6.4 Activity-dependent pruning

The frame. For a matrix W and inputs x of covariance Σ, by (2.8),

$$\mathbb{E}\,\|\Delta W\,x\|^2 = \operatorname{tr}(\Delta W\,\Sigma\,\Delta W^\top) = \sum_{i,j,j'} \Delta W_{ij}\Delta W_{ij'}\Sigma_{jj'} \;\approx\; \sum_{i,j} \Delta W_{ij}^2\,\Sigma_{jj}, \tag{6.4}$$

a diagonal approximation, exact on average if the errors of different columns are uncorrelated. The sensitivity of a weight is $s_j = \mathbb{E}[x_j^2]$, the activity of its column. This is the criterion of Wanda ($|W_{ij}|\cdot\|X_j\|$) [93], close to that of SparseGPT [94], and the diagonal term of OBD. The biological analogy is explicit: in the developing brain, synaptic pruning depends on activity, a little-used synapse is removed first.

The system. The masses of lemma 6.2 become $A = \sum 4^e s_j$; the context becomes (e, q), q being the activity quartile of the column in the expert (a map of 2 bits per column and per expert matrix, 0.0007 bit per weight, counted). The s_j come from windows 0 to 4 of the corpus of lot R, captured by the f64 reference (inputs of gate and up: the normalised state routed to the expert; input of down: the SwiGLU computed with the exact weights).

A prediction, struck out. The deliverable announced among its results: "~~Activity-dependent pruning: 1.695 bits (×0.644) at equal error.~~" It struck it out on 2026-10-17 (`genome/predictif-elagage-pour-le-memoire.md` § 0, point 7): it is an error *predicted* by the diagonal approximation (Table 6.4); measured, this plan breaks the expert and the KL (0.271, Section 6.6).

6.5 The real inputs, and the error at the output of the experts

The inputs [meas.]. The f64 reference (`trc-reference`, batched path) ran on the 10 windows of 512 tokens of the corpus of lot R with a capture hook (`sonde capture`, `capture-entrees-20b.json`, 2 828 s): for layers 0, 8, 16 and 23, the normalised state after the attention of each position routed to each expert, 20 440 vectors of 2 880 per layer (235.5 MB in f32, outside the repository). Calibration of the plans that read the activations: windows 0 to 4; evaluation: windows 5 to 9.

Plan	real bits	gate	up	down alone	**whole expert**	ratio
base (frontier)	2.6444	1.16 %	2.52 %	2.72 %	**5.57 %**	1
partitions by κ	2.3743	1.12 %	2.70 %	2.37 %	**5.48 %**	0.98
dead zone by e	2.3283	1.44 %	2.15 %	2.51 %	34.8 %	6.2
partitions by e (weights)	2.0178	1.03 %	2.65 %	2.38 %	22.0 %	4.0
partitions by row δ (± 3)	2.353 (ideal)	1.14 %	2.67 %	2.42 %	5.48 %	0.98
partitions by e, Hebbian masses	**2.1815**	0.92 %	2.38 %	2.45 %	**5.70 %**	1.02
partitions by (e, row, column), Hebbian	1.7556	2.36 %	4.30 %	6.41 %	9.23 %	1.66
partitions by (e, column activity)	1.6942	—	—	—	(KL 0.271)	—

Table 6.5: The relative error at the output of the experts, real inputs (windows 5 to 9), mean of the four layers [meas.]. Real bits: `taux_plan`, exact decoding (the base gives 2.6444 there against 2.6441 published by the arbiter: second path). Data: `mesures/genome/predictif-elagage/sortie-experts-12.json`, `sortie-experts-12-hebb.json`, synthesis `resume-12.csv`.

The measurement (`sortie_experts.py`): for each expert, exact output $y = W_d\,\mathrm{swiglu}(W_g x + b_g, W_u x + b_u) + b_d$ and rebuilt output $\hat{y}$ (the three matrices of the plan), relative errors summed over the experts (weighting by the tokens) then averaged over the four layers. Controls: identity and folded-zero plans, zero error exactly (holds).

Per layer, the expert error (0 / 8 / 16 / 23) is [meas.]: base 5.1 / 7.6 / 6.7 / 2.9 %; κ 6.1 / 7.6 / 6.5 / 1.8 %; Hebbian by e 5.1 / 8.3 / 7.1 / 2.3 %; Hebbian (e, row, column) 6.4 / 17.8 / 9.5 / 3.2 %.

What the measurement revealed (Table 6.5). The plans that fill across the absolute scales keep each matrix below 3 % relative error, and the whole expert becomes wrong at 22 to 35 %. The cause: the quantity that the relative error of a matrix measures is not the one that counts. A row i of gate or of up acts on the output through column i of down, and training distributes the scale freely between the two: the up path has a rescaling symmetry, $u_i \to c\,u_i$ and $W_d[:,i] \to W_d[:,i]/c$ change almost nothing (almost, because the bound on u in the SwiGLU is not homogeneous, Proposition 5.1). A row of small scale in gate can therefore weigh heavily at the output, and reverse water-filling by matrix empties precisely these rows.

Hebbian sensitivity. It answers at first order (`histo_hebb.py`). Linearising the output around the exact weights,

$$\mathbb{E}\,\|\Delta y\|^2 \approx \sum_{i,j} \Delta W_{ij}^2\, r_i\, s_j, \qquad s_j = \mathbb{E}[x_j^2], \quad r_i = \|W_d[:,i]\|^2\, \mathbb{E}\Big[\Big(\frac{\partial a_i}{\partial g_i}\Big)^2\Big] \tag{6.5}$$

(or $\partial a_i/\partial u_i$ for the up matrix), a being the intermediate activation. The first factor, s_j, is the activity of the input (presynaptic); the second, r_i, the influence of the output (postsynaptic); the weight of a synapse is their product, Hebb's product. The factorisation $\mathbb{E}[(\partial a/\partial g)^2 x^2] \approx \mathbb{E}[(\partial a/\partial g)^2]\,\mathbb{E}[x^2]$ neglects their correlation, and the diagonal approximation (6.4) that of the columns: two hypotheses that the measurement judges.

6.5.1 Hebbian pruning: what holds, what overfits

- **Hebbian masses, context e alone** (no map sent: r_i and s_j serve only the coder, to choose the partitions and the centres): **2.1815 real bits, expert error 5.70 %** against 5.57 % for the base at 2.6444, that is 0.825 times the bits for the same error to within 2 % [meas.]. The same reverse

Plan	real bits	mean KL	±	median	99th percentile	same first choice
base (frontier)	2.6444	0.0636	0.0050	0.0227	0.777	81.6 %
Hebbian by e	2.1815	0.195	0.009	0.075	1.54	60.8 %
Hebbian (e, row, column)	1.7556	0.192	0.008	—	—	59.4 %
Hebbian (δ, row, column)	1.9619	0.091	0.005	—	—	73.4 %
partitions by e (weights)	2.0178	0.309	0.011	0.209	1.83	58.5 %
partitions by (e, column activity)	1.6942	0.271	0.009	0.188	1.51	57.1 %
partitions by κ (equal mean error)	2.3743	0.0804	0.0063	0.0263	1.10	78.4 %
partitions by κ, each tensor at the error of the base	**2.4079**	**0.0702**	0.0059	0.0235	0.911	79.5 %

Table 6.6: The KL against the f64 reference (protocol P3: 128 experts out of 768, 1 275 positions) [meas.]. Data: `mesures/genome/predictif-elagage/kl/kl-12-*-f5-9.json`, synthesis `resume-12.csv`.

water-filling without Hebbian sensitivity broke the expert (22 %): it was indeed the gate/up ↔ down scale symmetry that broke it, and the postsynaptic influence that repairs it.

- **With maps of rows and columns** (16 classes, 9 273 cells): predicted 4.33 %, measured **9.23 %**, layer 8 at 17.8 %. The prediction does not hold: the quartiles of r_i and s_j are estimated on about 320 inputs per expert (windows 0 to 4), and the factorisation of (6.5) neglects their correlation; a plan that gives zero bits to the classes judged inactive pays dearly for any estimation error. Overfitting measured, not supposed.
- **Variant relative to the row** ($\delta\times$ row $\times$ column): 1.9619 real bits, KL 0.091, paired difference $+0.027 \pm 0.004$ against the base (`kl/comparer-base-hebb_d_lc-f5-9.json`): loses.

6.6 The KL against the f64 reference

The protocol (P3 of the round). Windows 5 to 9 (1 275 scored positions), only the 12 matrices of the frontier altered, that is **128 experts out of 768** (4 whole layers out of 24; `genome/05-synthese.md` § 1.1), probe `alterer` then `juge kl` of lot R against the slice of the reference (`kl_tranche.py`, fingerprint in `kl/tranche-ref-f5-9.json`). Control of the chain: identity plan, `.kld` entries identical byte for byte to the reference, KL = 0 exactly. The absolute KL of this protocol is not comparable with that of the scout (P1, 32 experts): four times more experts are replaced in it. What is comparable is the ratio to the base of the same protocol and, better, the paired difference position by position, with the rule of lot R (equal if $\bar{d} \le 2\sigma(\bar{d})$ and $\bar{e} \ge -2\sigma(\bar{e})$ for the first choice).

What the KL decides (Tables 6.6 and 6.7).

- **Every plan that fills across the absolute scales loses**, by 3 to 5 times the KL of the base: partitions by e (0.309), pruning by column activity (0.271), Hebbian by e (0.195). The plans that break the expert break the model.
- **Even at equal expert error.** Hebbian by e makes 5.70 % expert error against 5.57 % for the base, and three times its KL (paired $+0.131 \pm 0.007$, first choice -20.8 points). The mean relative error at the output of an expert, weighted by $\|y\|^2$, is not enough to judge: it is held by the tokens and the dimensions of large output; reverse water-filling across the scales concentrates the error on what is

Plan, against the base	KL, paired difference	first choice (points)	verdict of the judge
partitions by κ, equal mean error	$+0.0167 \pm 0.0049$ ($3.4\,\sigma$)	-3.2 ± 1.0	worse
partitions by κ, tensor by tensor	$+\mathbf{0.0066} \pm \mathbf{0.0053}$ ($1.2\,\sigma$)	-2.1 ± 1.1	**equal**
Hebbian by e	$+0.131 \pm 0.007$	-20.8 ± 1.4	worse
Hebbian (e, row, column)	$+0.128 \pm 0.007$	-22.2 ± 1.4	worse
Hebbian (δ, row, column)	$+0.027 \pm 0.004$	-8.2 ± 1.2	worse

Table 6.7: The paired comparisons of the plans against the base, same 1 275 positions, same truth (`juge comparer`) [meas.]. Data: `mesures/genome/predictif-elagage/kl/comparer-base-*-f5-9.json`.

small (blocks and rows of small scale, 23 % of the weights folded into zero), which the RMS norm of the following layers brings back to scale and which the KL sees. The lesson of the subject: at the output, one should judge the relative error *per token*, or the KL directly, not the total energy.

- **At equal mean error, the κ plan is not worth the base in KL** ($+0.0167 \pm 0.0049$, $3.4\,\sigma$): it moves the error towards layer 0 (6.1 % expert error against 5.1 %), and an early error in the network weighs more in the KL. Hence the plan "each tensor at the error of the base" (`plans.py --par-tenseur`, one μ per tensor: domination tensor by tensor), at **2.4079 real bits, 0.911 times the base**. Paired against the base: $+0.0066 \pm 0.0053$ ($1.2\,\sigma$), first choice -2.1 ± 1.1 points. **Statistically equal to the base** by the rule of lot R; literally a little above.

This is the lossy point that the subject retains: 9 % fewer bits than the base of the frontier, at a KL equal within the noise of the measurement. The cautious reading is that of the deliverable itself: the central paired difference remains positive, and with more positions it could become significant. Chapter 8 places this point on the common frontier of the round, under the name "partitions by κ, equal by the rule".

6.7 What remains

- **The card kernels**: a decoder with a table per (κ, column) or per (e, q), and the reconstruction by a table of centres (4 to 16 registers); their connection into the coder of the format. The lossless part (zero folded and column) has entered the format v1.2 (Chapter 9).
- **Error feedback** (GPTQ, "closed-loop predictive coding") on the alphabet of the cells, with the full Hessian of the real inputs; and a sensitivity criterion judged per token, or learnt against the KL, since the mean energy misled.
- **The dominated κ plan** + column + zero folded on the 24 layers, and its KL; then gpt-oss-120b.

What could be wrong

- **The lossy rates are ideal entropies**, not a codec, except where the "real bits" column says so; the gap of the real codec measured on the base is 0.46 %, and it may be larger for tables with many contexts (2 077 cells in e, 3 808 in (e, q)) and rare cells.
- **The diagonal approximation** of the sensitivity (6.4) neglects the correlations between input columns, and the Hebbian factorisation (6.5) those between input and derivative; the exact measurement at the output judges them, and made a prediction fail (struck out).
- **The inputs**: 10 windows of 512 tokens from a single corpus; about 640 inputs per expert and per layer, about 320 for calibration: the activity quartiles of a little-routed expert are noisy.

- **The KL alters only 4 layers out of 24**; the effect on the 24 is not measured (the KL would grow, for the base as for the plan). It covers 1 275 positions of a single corpus, and the 9 % advantage of the dominated κ plan is at a statistically *equal* KL, not a better one (central difference +0.0066).
- **The gain of the synonymous codons** assumes that every engine reads ±0 as 0: true of ggml (integer table) and of the reference; for the kernels of troncons, it is proposition A1 of Chapter 9 and its measurement.
- **No figure** carries this chapter: the two proposed (III-5.1, III-5.2) are not drawn, and the tables stand in for them.

Chapter 7

Mimicry: six leads from the world to the pipeline

Where we start, what this chapter establishes, what remains

The starting point is that the round of the genome had kept three gains (the zero codon, the column context, the synonymous mutation) and closed the global tropical guard, the seed, the duplicate, the Hebbian rule and the spectra (`genome/05-synthese.md` § 2 and § 3). The engine had only a shared LRU cache (`trc-cache` exports only `Lru` and `Belady`). Six scouts, each in charge of a domain of the world (life, physics, mathematics, engineering, human systems, the Earth and the cosmos), had returned 45 candidates, a good share of them already judged by a measurement. **This chapter establishes** the sorting of these candidates into six leads and what each gave under its judge. P1, the two-scale cache memory: the mechanical "fails" is decided by 147 tokens produced after the end of the message; on the real answer the LFU(32) gains +0.0185, over three blocks: *open*. P2, the exclusive cache: a theorem (the two-stage exclusive cache is worth an LRU of capacity $V+R$) and a reduction of the disk share by 28 and 51 %; the real bargain is "3.2 GB of RAM or an overlapped descent": *open*. P3, cache-conditioned routing: −10 % of misses for a KL of 0.0314 in the engine, but no condition holds at its unfavourable bound: *borderline*. P4, calibrated prefetching: the guaranteed precision holds (0.92 for about 1 % more bytes), the gain does not (3.3 to 3.5 %): *open, rather on the side of fails*. P5, the spin glass: the optimiser lowers the calibration error by 52 % and the held-out error by only 5 %: *open*, and the announced mechanism does not hold at the threshold. P6, fidelity decided by call: *open* at decoding, "holds" at prefill, but a "holds" that could hardly fail. Each lead brings its derivations, set out in full here. **What remains** is healthy generations of the 120b (P1, P4), the ABBA passes of the engine (P2, P4), the second draw of P3 (in progress in another session, none of its numbers here), the real weights of P5 (step 2), the end-to-end KL of P6 (step 1). Sources: `genome/mimetisme/tri.md`; for each lead k, `genome/mimetisme/Pk/resultat.md`, `pour-le-memoire.md`, `JUGEMENT.md` (P3: also `JUGEMENT-etage-B.md`, `JUGEMENT-N238.md`, `JUGEMENT-routeur-ligne.md`); thresholds and logs under `mesures/mimetisme/`.

In brief

Mimicry asks other disciplines how they solved a problem that resembles ours: keeping in memory what will serve again, letting the rest go, deciding fast under uncertainty. The image is that of a borrowing: a climatologist knows that a slow reservoir integrates the forcing of a fast reservoir; a physicist knows that a spin glass has valleys where a greedy descent stops; a statistician knows how to guarantee a precision without knowing the law of the data. None of these resemblances is a proof: each proposes a measurement, and it is the measurement that decides. Out of six leads, none clearly "holds" under its judge; four remain open for a precise

Table 7.1: The removal of duplicates: the same mechanisms under the names of different domains (`genome/mimetisme/tri.md` § 1). The number follows the name of the scout.

Common mechanism	Scouts and original names	Signal
LFU with a long half-life	life #1 (ACT-R), Earth #1 (Hasselmann, two reservoirs), human #1 and #8 (Anderson and Schooler), engineering #2, physics #4 (Omori–Utsu, Kohlrausch), Earth #7 and #4	six domains, one lever: a half-life longer than 8 tokens
Routing that favours the residents	mathematics #1 (cache-conditional), Earth #6 (sticky fault), human #5 (item substitution, two choices)	three domains
Predicting the experts to come	mathematics #2 (conformal prediction), human #2 (case law), life #2 (olfactory circuit of the fly), life #4, Earth #5, engineering #2	six domains
Prefetching subordinate to demand	engineering #4 (TCP Nice, LEDBAT), human #3 (BOLA), mathematics #2	three
Choosing fidelity at read time	physics #3 (Fermi occupation), life #5 and #7a (foveation), human #6 (triage), engineering #6, human #3, life #3	six
Exclusive cache	engineering #1 and #5	one, but measured
Ceiling of batched verification	mathematics #5, life #6	two, closed
Curve without a cliff	mathematics #8, engineering #7	two, closed
Allocation of slots between layers	life #7b (Charnov), Earth #8, human #7	three, closed

> reason, put in numbers (traces too short or degenerate, an unmeasured overlap, an unvalidated simulator, synthetic weights), and two say something clear: an inclusive RAM stage smaller than the VRAM is of no use, and a router that favours the present experts removes 10 % of the misses for a KL of the order of 0.03, at the edge of what is admitted.

7.1 The sorting: forty-five candidates, six leads

Where the candidates come from. Each scout read its domain and proposed mechanisms transposable to the path of the weights, with, when it was possible without the card, a first measurement on the traces of the engine. The sorting (`genome/mimetisme/tri.md`, 2026-10-27) removed their duplicates, ranked them, and wrote for each of the six leads retained a complete protocol: data, quantity, threshold, controls, second path, cost. The thresholds of the sorting are committed before the document (`mesures/mimetisme/tri/00-seuils-declares.md`, commit `ec124f5`); a sorting measures nothing. Ranking criterion: expected gain × novelty × feasibility on existing data, without a graphics card, with priority to the real bottleneck (bytes read per word, flips of the routing, prefetching). Common constraints: no card, two threads, normal priority, RAM aimed below 1.5 GB; as long as the lock on the card exists, no heavy reading of a model file.

The same mechanisms under different names. Removing the duplicates is the first result of the sorting: six domains reach the same lever under six names (Table 7.1).

The strongest signal is the long-memory LFU: it comes out of four scouts positively (life, Earth and cosmos, human, engineering) and out of two negatively (physics: the shape of the kernel does not

matter; mathematics: a learnt predictor adds only +0.012). But all its measurements had been made on *reading* traces: the decaying LFU had never been measured on a generation, which is the real load. And the "LFU −0.075 in generation" of engineering was not the counter-example it seemed: its "lfu" has no decay (the counter rises, only the "adm" policy halves it, read in `outils/mimetisme/ingenierie/cache2.py`). The question remained open, hence P1.

Two families. Three of the six leads touch the bottleneck without touching quality (P1, P2, P4: fewer bytes read for the same output to the bit); three trade quality for bytes (P3, P5, P6) and are judged by the KL. The latter share a bar: the slope of the fast mode, 0.13 of KL for −34 % of bytes, linearised as 0.038 for 10 points; P6 was to provide the curve $\mathrm{KL}(f)$ that this slope lacked.

What is ruled out, and why. The sorting rules out some thirty candidates, each by a number or a written reason (`tri.md` § 4). The most telling: the physical memory kernels (Omori–Utsu, Kohlrausch), closed by measurement E1 (+0.0051, +0.0028, +0.0044 against a threshold of +0.015: the shape does not matter, the half-life does, and it goes into P1); the Fermi cut-off (removing the expert of small weight), closed by E2 (0.09 % of the misses below 0.05; the gateway to the fast mode becomes P6); batched verification, killed by its ceiling ($G(4) = 1.44$, $G(8) = 1.84$ for a free draft); the learnt eviction predictor (+0.0124 against the best tuned LFU, below +0.05); segments with ghost lists (the SLRU with a 20 % probation collapses, 0.257 against 0.738); the cache in coded form, dominated by P2 (−8 % of bytes for 120 ms of card per token); and, narrowly, reverse water-filling by usage (0.094 bit achieved against a threshold of 0.126), the best compression lead not retained.

The shared instruments. Three rules hold for all the leads. (i) The *group sequential stopping* of O'Brien and Fleming [95], calibrated by Monte Carlo by the human scout (constant $c = 2.840$): 6.8 windows out of 10 on average over five decisive series, with a floor of 6 windows, because one series crossed the boundary at 4 windows ($t = -5.1$) and was no longer decisive at 10 ($t = -2.1$). (ii) The *pairing rule* of lot R: a mode is better if $\bar{d} + 2\sigma(\bar{d}) < 0$, σ by blocks of windows. (iii) *Separate prefill and generation in any cache judgement*: admission policies win at reading (+0.081) and lose at generation (−0.094) (engineering, H3); it is the common trap, and P1 and P4 fell into it otherwise than expected (Section 7.2.3).

The common form of the six protocols. Each lead committed its threshold before any reading of data, and each judge checked the order of the commits by `git log`; each lead has a control whose answer is known in advance and a break that must make it fail; each main number has a second path; and each lead was redone by an independent judge, written without the code of the lead. The verdict of this chapter is always that of the judge.

7.2 P1: the two-scale cache memory

The idea. A cache of experts decides whom to keep. The LRU keeps the most recent; the LFU, the most frequent. Human memory forgets as a power law [96], and the climate has two reservoirs, a fast one that follows the forcing and a slow one that integrates it [97]; the "cache" version of this idea is the LRFU [98], a score that decays with a half-life. If such a score wins in generation, it is the cheapest change of the project: one counter per expert, 4 608 floats for the 120b. The four records are reread through Crossref on 2026-10-29; the papers are not, and nothing in the verdict depends on them.

7.2.1 The exponentially decaying score, a rate estimator

Definition 7.1 (Half-life score)**.** Let an expert be accessed at tokens $s_1 < s_2 < \dots$ For a half-life h (in tokens), its score at token t is

$$c_h(t) = \sum_{s_i \le t} 2^{-(t-s_i)/h}, \qquad \text{updated by } c \leftarrow c \cdot 2^{-\Delta t/h} + 1. \tag{7.1}$$

The cache evicts the resident with the smallest score. We set $\tau = h/\ln 2$, the time constant.

Lemma 7.2 (Expectation and variance)**.** *If the accesses form a Poisson process of rate λ per token, $\mathbb{E}[c_h] = \lambda h/\ln 2$ and $\mathrm{Var}[c_h] = \lambda h/(2\ln 2)$. So $\hat{r}_h = c_h \ln 2/h$ is an unbiased estimator of λ, with variance $\lambda \ln 2/(2h) = \lambda/(2\tau)$.*

Proof. The score is a Poisson process filtered by the kernel $k(u) = 2^{-u/h} = e^{-u/\tau}$. By Campbell's formula, $\mathbb{E}[c_h] = \lambda \int_0^\infty e^{-u/\tau}\,\mathrm{d}u = \lambda\tau$ and $\mathrm{Var}[c_h] = \lambda \int_0^\infty e^{-2u/\tau}\,\mathrm{d}u = \lambda\tau/2$. Multiplying by $\ln 2/h = 1/\tau$: $\mathbb{E}[\hat{r}_h] = \lambda$ and $\mathrm{Var}[\hat{r}_h] = \lambda/(2\tau)$. ∎

The relative deviation, $\sqrt{\ln 2/(2h\lambda)}$, is the price of a short memory: at $\lambda = 0.1$ access per token and $h = 8$, it is $\sqrt{0.6931/1.6} = 0.66$ [calc.].

Lemma 7.3 (Lag bias)**.** *If the rate drifts linearly, $\lambda(t) = \lambda_0 + \lambda' t$, then $\mathbb{E}[\hat{r}_h(t)] = \lambda(t) - \lambda'\tau$, and the mean squared error is $\mathrm{MSE}(\tau) = \lambda/(2\tau) + \lambda'^2\tau^2$.*

Proof. $\mathbb{E}[\hat{r}_h(t)] = \int_0^\infty \lambda(t-u)\, e^{-u/\tau}\,\mathrm{d}u/\tau = \lambda(t) - \lambda' \int_0^\infty u\, e^{-u/\tau}\,\mathrm{d}u/\tau = \lambda(t) - \lambda'\tau$. The MSE is the variance of lemma 7.2 plus the square of the bias. ∎

Proposition 7.4 (Optimal horizon)**.** *For $0 < \lambda' < \infty$, the MSE is minimal at $\tau^* = \big(\lambda/(4\lambda'^2)\big)^{1/3}$, which is neither 0 (the LRU) nor ∞ (the LFU).*

Proof. $\mathrm{d}\,\mathrm{MSE}/\mathrm{d}\tau = -\lambda/(2\tau^2) + 2\lambda'^2\tau$ vanishes at $\tau^3 = \lambda/(4\lambda'^2)$; the second derivative $\lambda/\tau^3 + 2\lambda'^2$ is positive. ∎

The horizon grows when the drift weakens and when the rate is low. The measurement places the best h, chosen on the readings of the other genres, at $h^* = 32$ ($\tau = 46$ tokens) at 326 slots, and at 32 or 64 at 922 [meas.] (`mesures/mimetisme/P1/juge-120b-*.json`, field `cv`).

Proposition 7.5 (The LRU–LFU continuum)**.** *As $h \to 0$, the order of the scores tends to the order of the last accesses (LRU); as $h \to \infty$, to the order of the counts (LFU).*

Proof. $c_h(t) = 2^{-(t-s_{\text{last}})/h}\big(1 + \sum_{i \ne \text{last}} 2^{-(s_{\text{last}}-s_i)/h}\big)$. As $h \to 0$, the second factor tends to 1 and the first orders by s_{last}; as $h \to \infty$, each term of the sum (7.1) tends to 1 and the score to the count. ∎

This is the LRFU family [98]. The controls check it to the bit: at $h = 0.001$ the hit rate equals that of the LRU to within 0 on eight traces; at $h = \infty$ it equals the "lfu" of `cache2` to within 0 [meas.] (`mesures/mimetisme/P1/temoins.json`).

Two reservoirs. The mixture $M(w) = (1-w)\,\hat{r}_8 + w\,\hat{r}_{128}$ is the output of a linear filter with impulse response $k(u) = (1-w)\frac{\ln 2}{8}2^{-u/8} + w\frac{\ln 2}{128}2^{-u/128}$, a sum of two relaxations, with transfer function

$$K(s) = \frac{1-w}{1 + 8s/\ln 2} + \frac{w}{1 + 128s/\ln 2}, \tag{7.2}$$

two real poles: it is Hasselmann's two-reservoir structure, stated here as an analogy of form. $M(0) =$ LFU(8) and $M(1) =$ LFU(128) to within 0 (control W3). The mixture is not better than the LFU(32) alone: -0.0031, $+0.1042$, -0.0657, $+0.0892$ against -0.0176, $+0.1136$, -0.0449, $+0.0981$ on the four cases of the verdict [meas.].

Table 7.2: P1: the verdict at the declared threshold, 120b, 326 slots, $h^* = 32$ by cross-validation [meas.]. σ by blocks of 32 tokens. The judge redoes each number to within 10^{-5} by its own path.

Case	LRU	LFU(32)	d	σ	z	oracle h (not retained)
generation prose-en	0.7379	0.7204	**−0.0176**	0.0213	−0.8	$h = 8$: +0.0055
generation wiki	0.3944	0.5080	+0.1136	0.0416	2.7	$h = 32$
chain prose-en	0.7402	0.6953	−0.0449	0.0224	−2.0	$h = 8$: +0.0021
chain wiki	0.3957	0.4938	+0.0981	0.0493	2.0	$h = 32$

Data: `genome/mimetisme/P1/resultat.md` § 2; `mesures/mimetisme/P1/juge-120b-326.json`; judge: `genome/mimetisme/P1/JUGEMENT.md` § 2.

From hit rate to speed. If the disk bounds everything, the time of a token is proportional to the misses, $v \propto 1/(1-s)$, and a cache of hit rate s_1 against s_0 speeds up by a factor

$$\rho = \frac{1 - s_0}{1 - s_1}. \tag{7.3}$$

The declared threshold (+0.02 of hit rate on 0.2621 of misses) is worth $\rho = 0.2621/0.2421 = 1.083$; the wiki generation (0.3944 → 0.5080) would be worth $0.6056/0.4920 = 1.23$, but on a loop; a cold window of the 120b (LRU 0.45, $\bar{d} = +0.094$) would be worth $0.55/0.456 = 1.21$ [calc.]. (The note of the lead writes $\rho \approx 1.19$ for this last case; the computation set out gives 1.21.)

7.2.2 The protocol and the mechanical verdict

Declared protocol (`genome/mimetisme/P1/protocole.md`, commit `c861e8f`, after the thresholds `ec124f5`). Data: seven traces of the 120b (five readings: code, dialogue, prose-en, prose-fr, wiki; two generations of 256 tokens: prose-en and wiki), and those of the 20b. Quantity: the global hit rate, under the group rule of `trc-cache` (the 4 experts of a layer are pinned during their group), 326 and 922 slots. h^* chosen by cross-validation on the readings of the *other* genres. Threshold: $\mathrm{LFU}(h^*) - \mathrm{LRU} \geq +0.02$ and $z > 2$ on the two generations and the chain (cache warmed by reading the same genre, then generation); dead below +0.005 on one of them.

Controls [meas.] (`mesures/mimetisme/P1/temoins.json`). W0: the LRU of the lead equals that of `cache2` (gap 0 on 8 traces), and the landmarks of the threshold come back (0.7379 and 0.45279). W1: $\mathrm{LFU}(h = 0.001) = \mathrm{LRU}$ to within 0. W2: $\mathrm{LFU}(\infty)$ = "lfu" of `cache2` (0.66268 and 0.42632). W3: the extremes of the mixture, to within 0. W4, the break: the inverted decay ($h = 8$) falls below the LRU by more than 0.02 on each trace tested, narrowly on the wiki generation (0.0269). W5: the verdict function returns "holds", "fails", "open", "open" on four fabricated gaps. W6: Belady [99] bounds every policy from above (margins 0.094 and 0.165). Second path: a Rust kernel against the Python one, hit-rate gap at most 9.8×10^{-6} over 108 series.

At the declared threshold, P1 **fails**: the prose-en generation is below +0.005 (Table 7.2). Even the oracle h of this trace gives only +0.0055. At 922 slots, nothing either (+0.0058, +0.0099, −0.0442, −0.0126). What the chain shows: warming the cache by reading the same genre does not help the generation (LRU 0.7379 → 0.7402), and harms the LFU without decay (prose-en: 0.6627 in generation alone, 0.1411 in the chain); decay rescues the LFU at the reading–generation boundary (LFU(8) in the chain: +0.0021 and +0.0856). The counts of the reading become pollution when the generation moves to another region of experts.

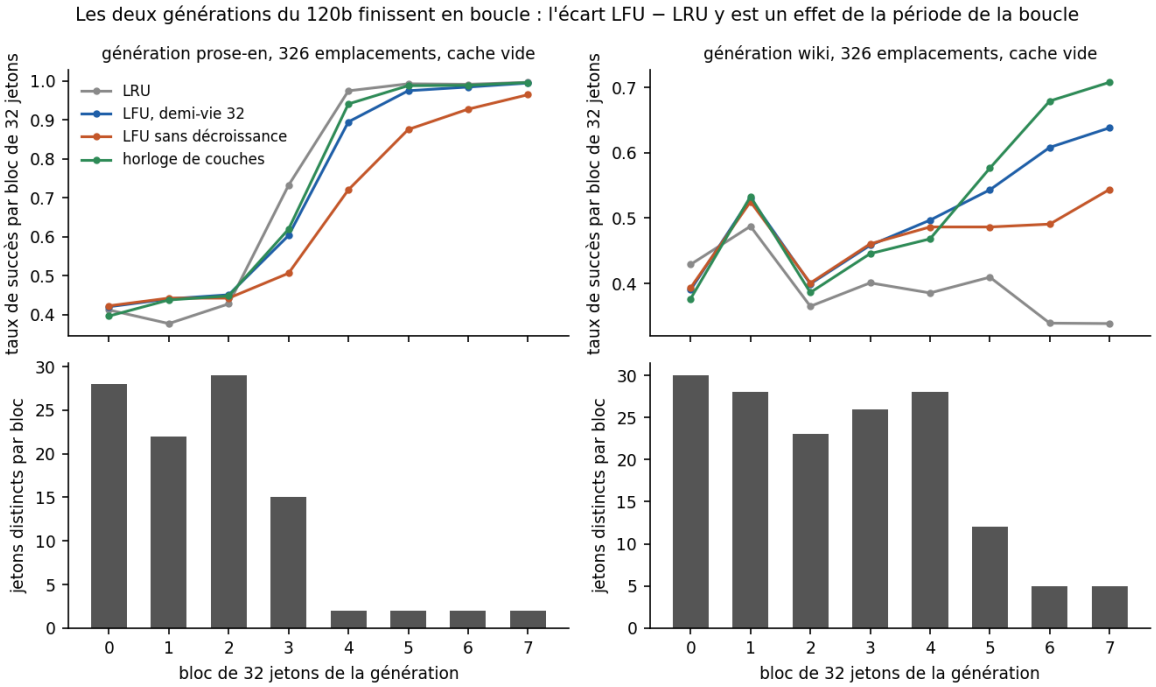


Figure 7.1: The whole LFU − LRU gap of the generations of the 120b is decided in their second half, where the distinct tokens collapse. [meas.] Top: hit rate per block of 32 tokens, 326 slots, empty cache (LRU, LFU with half-life 32, LFU without decay, layer clock). Bottom: distinct tokens per block. The original title, in French, says "finissent en boucle" (end in a loop); the judge showed that the second half of prose-en is made of tokens produced after the end of the message (<|return|> at token 109), and that wiki is a raw completion without a chat template. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P1/figure-blocs.csv`; script `outils/mimetisme/P1/figures_p1.py`.

7.2.3 What the generation traces really were

The lead saw, before its judge, that the two generations end badly: prose-en falls to 2 distinct tokens per block of 32 from token 128 on, wiki to 4 or 5 over its last 100 tokens (Figure 7.1, bottom panels). It wrote this down as "loop of greedy decoding". The judge read the tokens themselves and corrected the reading [meas.] (`genome/mimetisme/P1/JUGEMENT.md` §4.1): in `t120b.prose-en.generation.jetons`, the harmony sequence is complete (<|end|>, <|start|>, <|channel|>, <|message|> at tokens 47 to 52 and 64 to 69), then **<|return|> at token 109**. The model finished its answer cleanly; it is the tracer that forced 147 more tokens, which no chat engine produces. The sentence of the lead therefore reads thus: ~~"loop of greedy decoding"~~: tokens produced after <|return|>.

On the real answer, every h gives a positive gain, from +0.012 to +0.022 (Table 7.3); the negative comes entirely from the tail, where two tokens fit in the LRU. The head makes only three whole blocks of 32: no usable z, and it is not a result, only the proof that the "fails" measures the tracer. The wiki trace is not a conversation either: its prompt is raw WikiText cut in the middle of a word, the model does completion on it and repeats "Here is a summary"; on its head (tokens 0 to 159), LFU(32) − LRU = +0.041, on its tail +0.234 [meas.]: the wiki gain of the verdict is also mostly a loop effect.

Table 7.3: P1, prose-en generation of the 120b, 326 slots, cut at the end of the message (probe S1 of the judge, declared before computation) [meas.].

	head 0–108 (real answer)	tail 109–255 (after the end)
LRU	0.415	0.977
LFU(8) − LRU	+0.0215	−0.0063
LFU(32) − LRU	**+0.0185**	−0.0443
LFU(∞) − LRU	+0.0134	−0.1409

Data: `genome/mimetisme/P1/JUGEMENT.md` § 4.1; `mesures/mimetisme/P1/juge/sondes.json`.

Table 7.4: P1: the 42 cold windows of 256 tokens of the readings [meas.]. The rows of the 120b at 326 are redone by the judge (to within 10^{-4}); the others are not.

Model, capacity	LRU per genre	$\bar{d}$	σ	z	LFU wins	genres $\geq +0.02$
120b, 326	0.35 to 0.51	+0.0939	0.0029	32.8	42/42	5/5
120b, 922	0.69 to 0.81	+0.0314	0.0030	10.4	38/42	4/5
20b, 217	0.67 to 0.78	+0.0528	0.0020	27.0	42/42	5/5
20b, 307	0.82 to 0.90	+0.0248	0.0014	18.2	41/42	5/5

Data: `mesures/mimetisme/P1/fenetres-120b.json`, `fenetres-20b.json`; `genome/mimetisme/P1/resultat.md` § 4.

A hole in the specification. Does the count of an expert survive its eviction? The protocol did not say. The judge, losing the count at eviction, obtained LFU(∞) = 0.2705 instead of 0.66268; the landmark W2 is recovered, to within 10^{-6}, only if the count of each of the 4 608 experts is *kept for life* [meas.] (`mesures/mimetisme/P1/juge/juge-oubli.json`). That is the convention measured by P1; in the engine, it costs a table of 4 608 counters. The verdict is robust to it: with an "in-cache" LFU, the prose-en generation gives −0.0111 and fails too.

7.2.4 The cold windows: frequency, not half-life

To know whether the gain exists on a text that moves forward, the lead declared before computation a post hoc analysis (`genome/mimetisme/P1/posthoc-declare.md`): the five readings cut into 42 disjoint windows of 256 tokens, empty cache at the start of each, same choice of h^*, same threshold, and the requirement of more than one genre ($\geq +0.02$ for 3 genres out of 5).

Under the declared rule, the analysis would say "holds" (Table 7.4). But *its break fails*: the inverted decay does not fall below the LRU in a 256-token window (it exceeds it by +0.0174 on the 120b). What the window sees is not the half-life: the LFU *without* decay already gives +0.0849, decay adds only +0.009 ($\sigma = 0.0018$). Over 256 tokens, the counts have no time to age; what pays is frequency, a few hot experts (Figure 7.2). And this gain does not transfer to the healthy generations of the 20b: at 217 slots, code −0.0015, dialogue +0.0015, prose-fr +0.0524, wiki −0.0297 [meas.]. The imposed text of a reading and the greedy text of the model do not have the same structure of expert usage.

The space of usages. Why does reading prepare generation so badly? Figure 7.3 represents each block of 32 tokens by the vector of its expert usages (4 608 dimensions, norm 1). The generation forms a separate block in the matrix of cosine similarities (0.4 to 0.5 with the reading, 0.3 for the repeated tail); in the principal plane (41 and 15 % of the variance for wiki, 30 and 13 % for prose-en), the reading

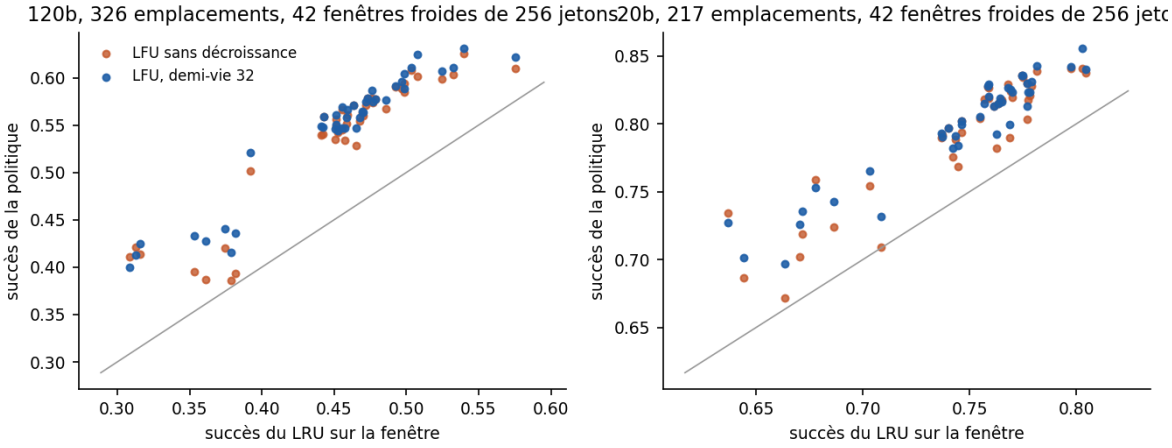


Figure 7.2: On a cold window of 256 tokens, the LFU beats the LRU 42 times out of 42, with or without decay: it is frequency that pays. [meas.] Each point is a window; the diagonal is the LRU. Left: 120b, 326 slots; right: 20b, 217. The title of the left panel runs over into the right one in the original. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P1/fenetres-120b.csv`, `fenetres-20b.csv`; script `outils/mimetisme/P1/figures_p1.py`.

occupies a compact cloud and the generation goes off on a distinct trajectory. A cache warmed by the reading contains the wrong region, and the old counts pollute.

Verdict of the judge: open. The number is reproduced to the bit by an independent path, and the mechanical "fails" is exact at the declared threshold. But it does not judge the LFU in generation: it is decided by 147 tokens after the end. On the real answer, the LFU(32) gains +0.0185, below the threshold of +0.02 and over three blocks: neither held nor failed. To close P1: at least four generations of the 120b stopped at <|`return`|>, judged on the tokens before the end alone, and the convention of the count written down (kept for life).

7.3 P2: the exclusive cache from VRAM to RAM

The idea. In a storage hierarchy, when the upper level evicts a block, it can *demote* it into the lower level instead of duplicating it: this is the exclusive hierarchy of Wong and Wilkes [100]. The cache of the engine does the opposite: in `Cache::demander`, the RAM is filled at each disk miss and refreshed at each VRAM hit (read by the sorting and by the judge in `moteur/crates/trc-cache/src/lib.rs`, lines 153 to 241). It therefore contains the most recent experts, already in VRAM. The simulation of the scout said so without appeal: the inclusive cache with $R \leq V$ adds 0.0000 of hit rate.

7.3.1 The LRU stack and the theorem

Definition 7.6 (Stack distance)**.** The LRU stack is the list of the experts already seen, from the most recent (rank 1) to the oldest. The stack distance of an access to x is the rank of x in the stack just before the access (infinite at the first access). We write $F(C)$ for the share of accesses with stack distance at most C.

Lemma 7.7 (Inclusion, Mattson et al.)**.** *An LRU cache of capacity C contains, at every instant, the first C elements of the LRU stack. An access is a hit in it if and only if its stack distance is at most C; the hit*

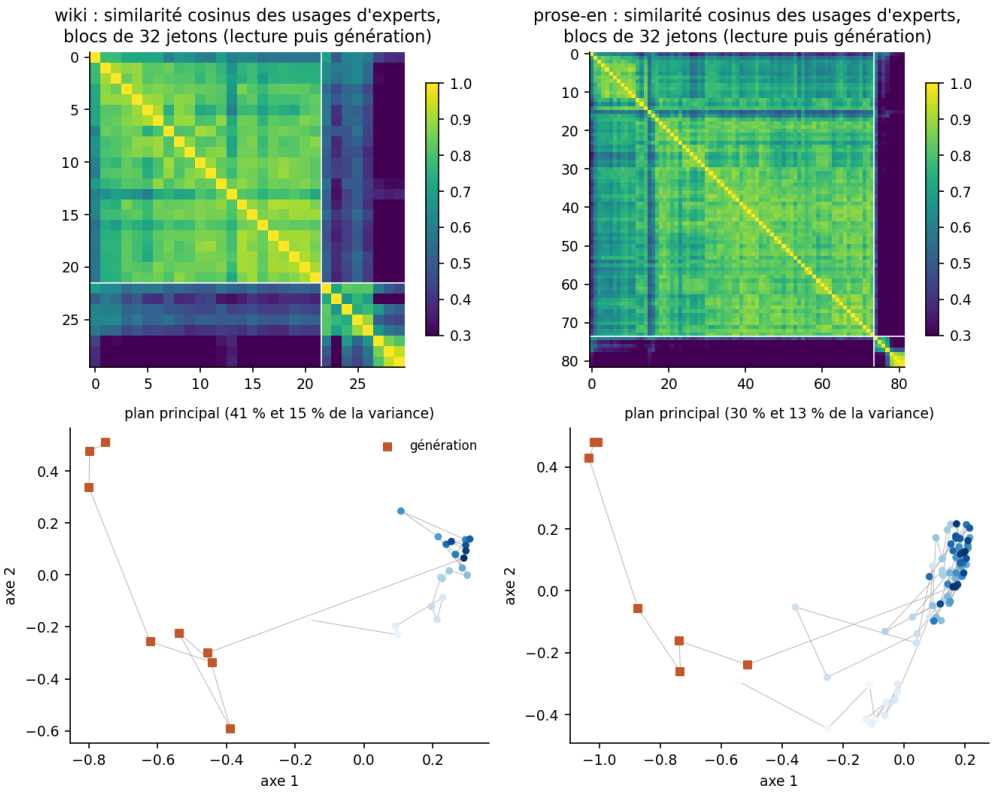


Figure 7.3: The generation occupies another region of the space of expert usages than the reading that precedes it. [meas.] Top: cosine similarity between blocks of 32 tokens (reading then generation, separated by the white line). Bottom: projection on the first two principal axes of the centred vectors; squares: generation. The tail of the generation is made of tokens after the end of the message (judge, Section 7.2.3). (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P1/figure-topologie.csv`; script `outils/mimetisme/P1/figures_p1.py`.

rate of an LRU of capacity C is therefore $F(C)$.

Proof. By induction on the accesses. An access of distance $d \le C$ brings x to the top; ranks $1, \dots, d-1$ slide down one place, the set of the first C ranks is unchanged: a hit. An access of distance $d > C$ brings x to the top and pushes rank C to rank $C+1$, which is exactly the least recent of the cache, the one the LRU evicts: a miss. ■

This is the property of the stack algorithms of Mattson, Gecsei, Slutz and Traiger [101] (record reread through Crossref by lot O on 2026-10-15; the paper is not reread here, and the proof above does not rely on it).

Theorem 7.8 (The two-stage exclusive cache is worth an LRU of capacity $V+R$)**.** *Let a VRAM stage of capacity V and a RAM stage of capacity R be given, both LRU. On a VRAM miss, the VRAM victim is demoted to the top of the RAM; if the requested expert is in RAM, it is promoted and leaves the RAM; if the RAM overflows, its oldest disappears. Then, at every instant, the VRAM contains ranks $1, \dots, V$ of the stack, the RAM ranks $V+1, \dots, V+R$, and the total hit rate is $F(V+R)$.*

Proof. By induction on the accesses; true for the empty stack. Consider an access to x of rank d. *If* $d \le V$, x goes to the top, ranks $1, \dots, d-1$ slide down one place; the set of ranks $1, \dots, V$ is unchanged, and so is the RAM: a VRAM hit. *If* $V < d \le V+R$, x is in RAM; it is promoted and leaves it. The old rank V becomes rank $V+1$: it is the VRAM victim (its oldest), which enters at the top of the RAM, in its place of rank $V+1$. The old ranks $V+1, \dots, d-1$ slide to ranks $V+2, \dots, d$, the ranks beyond d do not move: the RAM is exactly ranks $V+1, \dots, V+R$ of the new stack. A RAM hit. *If* $d > V+R$ (or at the first access), x goes to the top, the VRAM victim (rank V) becomes rank $V+1$ and enters at the top of the RAM; the RAM overflows by one element, its oldest, which is rank $V+R+1$ of the new stack, and it leaves. A disk miss. ■

Corollary 7.9 (The stack band)**.** *The share served by the RAM is $F(V+R) - F(V)$, the disk share $1 - F(V+R)$. The gain of the exclusive cache is the mass of the stack distances lying in the band $(V, V+R]$.*

With the pinning of the groups, the victim is the oldest *outside the group*, and the equality is no longer exact. Measured (control T2, 7 traces, $R = 80$ and 336): the gap to the LRU of capacity $V+R$ does not exceed 6.6×10^{-4}; on the 20b without pinning, it is exactly 0 [meas.] (`mesures/mimetisme/P2/holdout20b-t2-sans-epinglage.json`). Figure 7.4 shows the theorem on the two generations: the gain is the area under F between V and $V+R$, 0.08 on prose-en where the curve is almost flat after V, 0.32 on wiki whose working set straddles the boundary.

The inclusive cache, seen through the same stack. The judge measured that an inclusive cache whose RAM has $R' = V + R = 662$ entries has *exactly* the disk share of the exclusive cache at $R = 336$ (0.184489 and 0.288493 on the two generations, pinning included), without any demotion [meas.] (`mesures/mimetisme/P2/juge/budget-egal.json`). The same stack argument explains it; the following proposition is set out by the writing, under a hypothesis on the rule of the engine that the judge read in the code but that is not measured in itself.

Proposition 7.10 (The inclusive cache with RAM $R' \ge V$)**.** *Suppose, without pinning, that the inclusive RAM receives each expert read from the disk, that the date of each of its entries is refreshed at each access (served by the VRAM or by it), and that it evicts its oldest entry. If $R' \ge V$, the RAM contains at every instant ranks $1, \dots, R'$ of the stack, and the total hit rate is $F(R')$.*

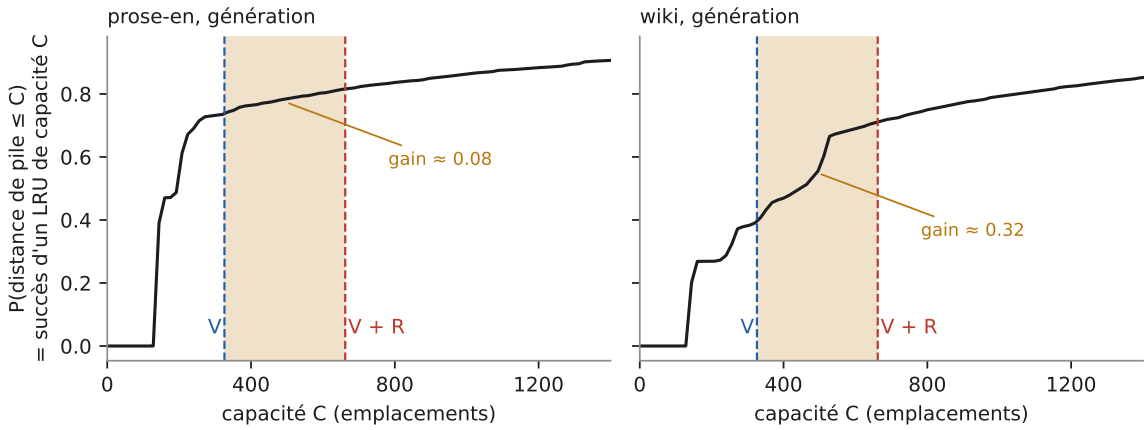


Figure 7.4: The gain of an exclusive stage is the mass of the stack distances in the band $(V, V+R]$. [meas.] [calc.] $F(C)$, hit rate of an LRU of capacity C, on the two generations of the 120b; hatched band for $V = 326$, $R = 336$. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P2/explore-distances.csv`; script `outils/mimetisme/P2/figures.py`.

Proof. By induction; true at the start, as long as the RAM is not full (each expert seen entered it at its first access). Suppose the RAM equal to the first R' ranks; since $V \le R'$, the VRAM (the first V ranks, lemma 7.7) is included in it. An access of rank $d \le R'$ is served (by the VRAM if $d \le V$, by the RAM otherwise) and refreshes the entry of x in the RAM: the RAM remains the first R' ranks of the new stack. An access of rank $d > R'$ is a disk miss; x enters the RAM and its oldest entry, the old rank R', now rank $R'+1$, leaves. ∎

Hence, at equal capacities in entries, $1 - F(662)$ for the inclusive cache at 662 as for the exclusive $(326, 336)$. For $R < V$ the argument does not go through: an entry of the RAM can age while the VRAM serves experts absent from the RAM. The lead gives only a sketch, and the measurement stands in for it: 0 RAM hits at $R = 80$ and 160 on the 7 traces (`mesures/mimetisme/P2/grille.csv`, inclusive mode), at most 7.65×10^{-4} at $R = V = 326$ [meas.] (`mesures/mimetisme/P2/t1bord.csv`).

7.3.2 The cost of demotion

A disk miss costs $a = 4.1$ ms (11.6 MB at 2.82 GB/s); a miss served by the decoded RAM $b = 1.09$ ms (13.25 MB at 12.2 GB/s); a demotion from VRAM to RAM costs b too [calc.]. Let φ be the number of VRAM misses per token and ρ the share of them served by the RAM; each VRAM miss causes a demotion. Per token:

$$\begin{aligned} T_{\text{LRU}} &= \varphi\, a, \\ T_{\text{serial}} &= \varphi\,\big[(1-\rho)\,a + \rho\,b + b\big], \\ T_{\text{overlapped}} &= \varphi\,\big[(1-\rho)\,a + \rho\,b\big]. \end{aligned} \tag{7.4}$$

The serial gain is $\varphi\,[\rho(a-b) - b]$, positive if and only if

$$\rho > \frac{b}{a-b} = \frac{1.09}{3.01} = 0.362. \tag{7.5}$$

With a fraction ω of the demotion overlapped by the computation, profitability requires $\rho(a-b) \ge (1-\omega)\,b$, that is $\omega \ge 1 - \rho(a-b)/b$. The measured times say it directly: on prose-en, the LRU alone

Table 7.5: P2, $V = 326$, $R = 336$: the lead and the judge [meas.]. Gap below 10^{-5} on the disk shares, below 0.1 % on the demotions.

Generation		inclusive disk	exclusive disk	reduction	demotions/token	demotion/345 ms
prose-en	lead	0.2573	0.1845	28.3 %	36.5	11.5 %
	judge	0.257270	0.184489	28.29 %	36.469	11.52 %
wiki	lead	0.5926	0.2885	51.3 %	85.9	**27.2 %**
	judge	0.592556	0.288493	51.31 %	85.938	27.15 %

Data: `genome/mimetisme/P2/resultat.md` § 1; `genome/mimetisme/P2/JUGEMENT.md` § 1; `mesures/mimetisme/P2/verdict.json`, `juge/verdict-juge.json`.

Table 7.6: P2: service time of the misses per token, $V = 326$, $R = 336$ [calc.] on [meas.]. In parentheses, the gain over the LRU alone.

Trace	LRU alone	serial exclusive	overlapped exclusive
prose-en, generation	154.7 ms	160.9 (−6.1)	121.1 (+33.6)
wiki, generation	357.6 ms	313.8 (+43.8)	220.1 (+137.5)

Data: `mesures/mimetisme/P2/net-selectif-120b.csv` (rows $s = 0$); redone by the judge to within 1 ms (J4c).

costs 154.7 ms per token, the overlapped exclusive 121.1 ms and the demotion 39.75 ms, so one needs $\omega \geq 1 - 33.6/39.75 = 0.154$ [calc.]; $\rho \approx 0.30$ is below threshold (7.5) there, and $\rho \approx 0.52$ on wiki above it.

7.3.3 The measurement, the verdict and what the judge corrects

Declared threshold (`genome/mimetisme/P2/seuil-declare.md`, committed before any reading of a trace): relative reduction of the disk share, exclusive against inclusive, $V = 326$, $R = 336$, of at least 15 % on the two generations, *and* demotion traffic at most 25 % of a 345 ms token; dead below 5 % on one of them.

The first condition holds on the two generations (28.3 and 51.3 % against 15 %), the second fails on wiki (27.2 % against 25 %): **open** (Table 7.5). Relative to the disk time proper to wiki (357.6 ms with the LRU alone), the demotion is still worth 26.2 %: the verdict does not move. Controls, all held and able to fail: $R = 0$ reproduces the LRU alone to the integer (T0); the inclusive cache at $R \leq V$ adds 0 (T1, tolerance relaxed to 10^{-3} at the edge $R = V$, said beforehand); the exclusive cache equals the LRU of $V + R$ to within 6.63×10^{-4} (T2); demotion removed reproduces the LRU alone, filling at the miss reproduces the inclusive cache (T3); the scout's simulator is recovered to within 4.8×10^{-4} (T4). Second path: a timestamp-based Rust implementation, equal to the Python one on 154 cells. Figure 7.5 shows the inclusive cache stuck to the LRU alone as long as $R \leq V$, then pulling away; the exclusive one takes off from $R = 80$.

What the judge corrects (`genome/mimetisme/P2/JUGEMENT.md` § 3). (1) The reference of the threshold, "committed in 6f3d737", designates a commit that is on no branch (amended version of a commit that carried a trailer); the commit on `main` is `6d325d8`, same second, identical content. (2) The reference "inclusive at $R = 336$" is degenerate by construction: "28 and 51 % against the inclusive cache" reads "against nothing". The real alternative is the inclusive cache with $R' = 662$, which has the same disk share (Proposition 7.10) without any demotion; the real bargain of the exclusive cache

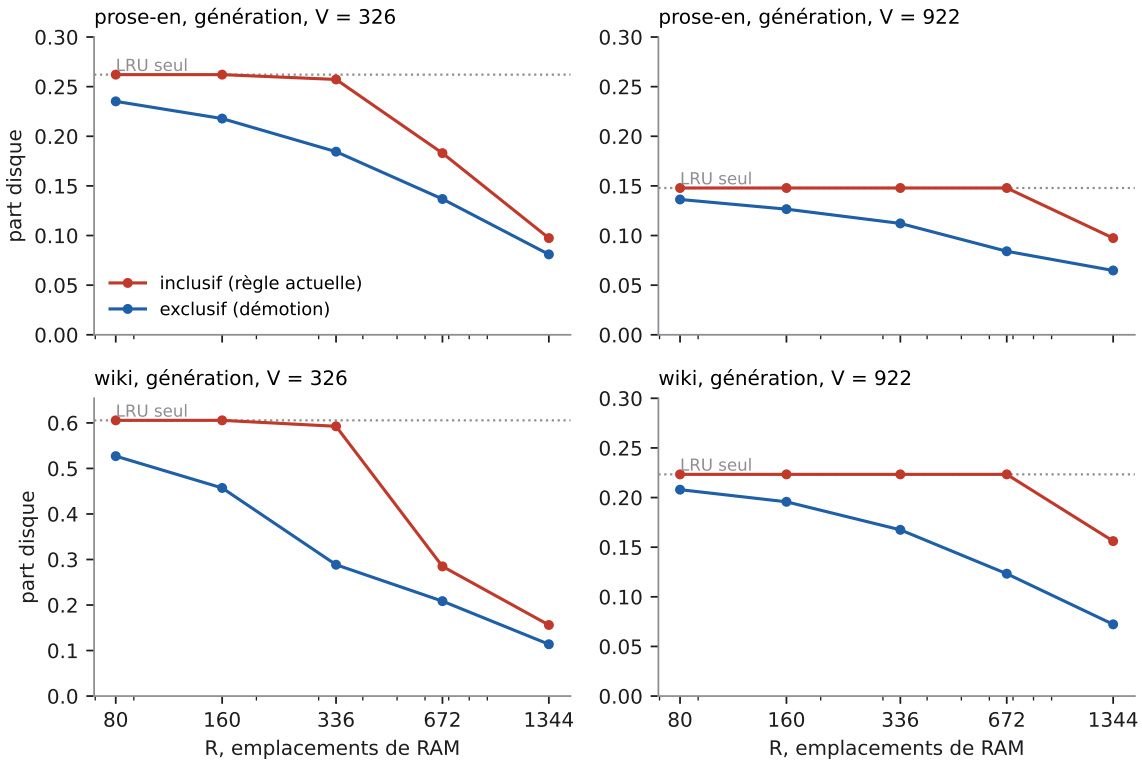


Figure 7.5: The inclusive RAM stage is of no use as long as $R \leq V$; the exclusive one serves from the first entry. [meas.] Share of the accesses served by the disk against R, two generations of the 120b, $V = 326$ and 922; dotted, the LRU alone. The judge recalls that $V = 922$ decoded entries make 12.2 GB of VRAM, more than the card (8 188 MiB): it is not an option on this machine. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P2/grille.csv`; script `outils/mimetisme/P2/figures.py`.

is therefore to save V entries of RAM (about 3.2 GB: 7.68 GB encoded against 4.45 GB decoded) against the demotion traffic. Serially, this inclusive cache beats the exclusive one (+25.7 against −6.1 ms on prose-en, +105.0 against +43.8 on wiki); overlapped, the exclusive one wins. (3) $V = 922$ is not an option on the card of the machine. (4) Selective demotion (demoting only the victims reused at least s times in VRAM), chosen after the fact and stated as such, is only a setting on the same curve: its advantage exists only serially, and overlapped it loses on wiki (253.2 ms against 220.1). (5) The theorem is not new: it is Mattson's inclusion and the argument of Wong and Wilkes; it is correct, and the judge redoes it exactly by a third path (stack distances by a Fenwick tree: 6 801 = 6 801 and 10 635 = 10 635 misses).

Verdict of the judge: open, reproduced. What P2 really brings: an inclusive RAM stage of size $R \leq V$ is of no use (the current `--ram-tiede`, in simulation); the exclusive cache reaches the disk share of an inclusive one of $V + R$ entries with V fewer entries of RAM, at the price of 36 to 86 demotions per token. Stone's decision is not "exclusive or inclusive at equal R", it is "3.2 GB more RAM, or an overlapped demotion". The crux remains the overlap of the demotion, to be measured in the engine in ABBA, against the inclusive cache at $V + R$, which requires no code.

7.4 P3: cache-conditioned routing, bounded by the margin

The idea. Skliar et al. add a bias to the logits of the experts already in cache before the top-k of the router [102]. Three scouts reached it by other paths: the seismic fault that sticks (rate-and-state friction), the substitution of an out-of-stock item, the power of two choices of queues; these are analogies, and the measurement alone decides. P3 changes the *stream of requests*, not the policy: Belady is optimal for a fixed stream, and the policy no longer moves (the best predictor exceeds the tuned LFU by +0.011 to +0.013, mathematics M1). It is also the riskiest of the six: a wanted flip costs like a suffered flip, and the flips carry 96.5 % of the KL on the positions that flip (`genome/05-synthese.md` § 2.2).

This chapter cites only what is judged. Stage A (open loop), stage B at $N = 146$, stage B at $N = 238$ and the online biased router in the engine each have their committed judgement. A second draw is in progress in another session (`genome/mimetisme/P3/REPRISE-second-tirage.md`): none of its numbers is cited. A control at $\beta = 0.3$ outside the engine (`resultat-temoin-b0.3.md`) has no judgement of its own: it is not cited either.

7.4.1 The geometry: a shifted wall, a band

A router of gpt-oss-20b returns, per cell (position t, layer ℓ), 32 logits $z \in \mathbb{R}^{32}$; the chosen set $S(z)$ is that of the four largest. In $\mathbb{R}^{32}$, the regions $\{z : S(z) = S_0\}$ are polyhedral cones; their walls are hyperplanes $\{z_i = z_j\}$ of the braid arrangement, and the wall that counts for S is the tropical locus $\{z : z_{(4)} = z_{(5)}\}$, where the *margin* $m(z) = z_{(4)} - z_{(5)}$ vanishes. The margin is, up to $\sqrt{2}$, the distance from z to the nearest wall that changes S (Chapter 12 makes it its subject).

The bias of Skliar et al. is a *translation* $z' = z + \beta\,\mathbf{1}_R$, where $\mathbf{1}_R$ indicates the resident experts. It does not change the geometry of the regions; it moves the point. For a pair (a absent, b resident), this amounts to shifting the wall $z_a = z_b$ to $z_a = z_b + \beta$ (Figure 7.6). Every cell whose point falls in the band between the two walls changes its choice: a was taken, b now is. *The share of cells changed is the probability mass of the band.*

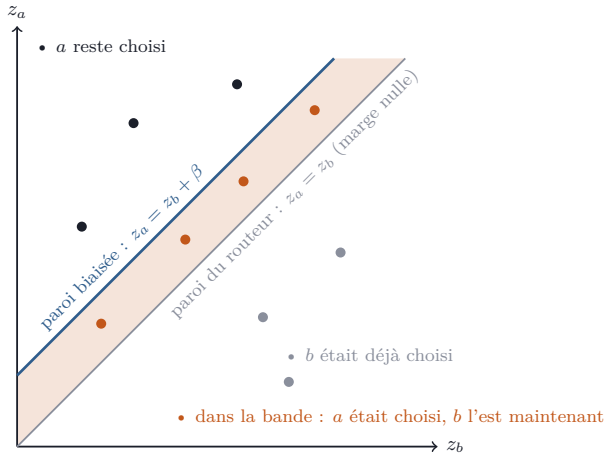


Figure 7.6: The bias towards the residents does not deform the regions of the router: it shifts a wall, and the band swept is the share of the cells that change. [schem.] Plane (z_b, z_a) of an absent a and a resident b: wall of the router $z_a = z_b$ (zero margin) and biased wall $z_a = z_b + \beta$. A diagram, not a measurement. (figure outside the common style of the thesis, included as its lead drew it)

Data: `genome/mimetisme/P3/figures/fig-paroi.tex`.

The mass of the band, measured. The distribution function of the margin over the 122 640 cells (24 layers × 5 110 positions) is known to within 3×10^{-6}, recomputed independently of the logs [meas.] (`mesures/mimetisme/P3/etage-A.json`):

β (logit)	0.01	0.03	0.1	0.3
$P(m < \beta)$	0.0818	0.2240	0.5632	0.9035
mean density $P(m < \beta)/\beta$	8.2	7.5	5.6	3.0

The mass grows almost linearly up to $\beta \approx 0.03$ (density at zero $g(0) \approx 8$ per unit of logit, consistent with the almost exponential law of the margin of Chapter 12), then saturates. The mean range of the logits is 4.136: $\beta = 0.1$ corresponds to $\lambda = 0.024$ in the normalisation of the paper.

The residence factor. Variant V2 (swapping only the 4th absent for the 5th resident) changes a cell exactly when $m < \beta$, the 4th is absent and the 5th resident. If the margin were independent of residence,

$$\text{share changed}(\beta) = P(m < \beta) \cdot P(a \text{ absent}) \cdot P(b \text{ resident} \mid a \text{ absent}). \tag{7.6}$$

The measured ratio "share changed / $P(m < \beta)$" is 0.237, 0.232, 0.225, 0.210 at the four β at $N = 146$, and 0.222 to 0.188 at $N = 238$ [meas.]: almost constant, it is indeed a residence factor. With $P(a \text{ absent})$ read from the rank-4 misses at $\beta = 0$ (0.544 and 0.301), one draws, at the smallest β, $P(b \text{ resident} \mid a \text{ absent}) = 0.437$ and 0.738 [calc.].

A swap avoids more than one miss. At small β, each swapped expert avoids 1.44 to 1.57 misses at $N = 146$ and 1.18 to 1.25 at $N = 238$ [meas.]: the miss of the moment, plus the absent one that does not enter the cache and does not evict a useful one from it, plus the recency restored to the resident. The exchange rate at the −10 % point is $0.1 \times 216\,397/(0.1198 \times 122\,640) = 1.47$ misses per cell at $N = 146$, and 1.21 at $N = 238$ [calc.].

Table 7.7: P3, stage A: misses avoided and share of cells changed, by β [meas.]. Each number is redone by the judge, miss for miss, with another GGUF reader, another matrix product and another cache.

	$N = 146$ (19 %)	$N = 238$ (31 %)
misses at $\beta = 0$	216 397 (hit rate 0.5589)	108 472 (hit rate 0.7789)
V1 (all residents), $\beta = 0.03$	−5.1 %; 6.0 % changed	−7.5 %; 5.4 %
V1, $\beta = 0.06$	−9.9 %; 11.9 %	−14.3 %; 10.5 %
V1, $\beta = 0.1$	**−16.1** %; 19.3 %	**−22.6** %; 16.6 %
V1, $\beta = 0.3$	−40.9 %; 48.2 %	−53.0 %; 37.7 %
V2 ($4^{\text{th}}/5^{\text{th}}$ only), $\beta = 0.1$	−10.4 %; 12.7 %	−15.3 %; 11.4 %
crossing of −10 % (V1)	$\beta = 0.060$; 12.0 %	$\beta = 0.041$; 7.3 %
a priori KL at this crossing (0.309× share)	0.037	0.023
$\beta = -0.1$ (favouring the absent ones)	+17.4 %	+28.2 %
random mask, same density, $\beta = 0.1$	+0.7 %	+1.9 % (judge: +1.48 %)

Data: `genome/mimetisme/P3/resultat.md` § 2; `mesures/mimetisme/P3/etage-A.csv`, `affinage.csv`; judge: `genome/mimetisme/P3/JUGEMENT.md` § 2.

7.4.2 The decision criterion in one quantity

Proposition 7.11 (The threshold on the slope κ)**.** *Let $f^*(N)$ be the share of cells to change to remove 10 % of the misses at budget N, and suppose the excess KL linear in the share changed,* $\mathrm{KL} = \kappa f$ *(small-β regime). The success threshold ($\mathrm{KL} \le \varepsilon = 0.038$ at −10 %) is equivalent to*

$$\kappa \le \kappa_{\max}(N) = \frac{\varepsilon}{f^*(N)}, \tag{7.7}$$

and the death threshold ($\mathrm{KL} > 0.07$) to $\kappa > 0.07/f^*(N)$.

Proof. At the point $f = f^*$, $\mathrm{KL} = \kappa f^* \le \varepsilon \iff \kappa \le \varepsilon/f^*$; likewise for 0.07. ■

With $f^*(146) = 11.98$ % and $f^*(238) = 7.31$ % (interpolated crossings): $\kappa_{\max} = 0.038/0.1198 = 0.317$ and $0.038/0.0731 = 0.520$; death is at $\kappa = 0.584$ and 0.958 [calc.]. The only κ known before stage B was that of the spontaneous flips between two almost exact engines:

$$\kappa_0 = \frac{0.0145 - 0.00168}{5\,087/122\,640} = \frac{0.01282}{0.04148} = 0.309, \tag{7.8}$$

2.5 % below $\kappa_{\max}$ at $N = 146$: a tiny margin. Form (7.7) is the one to retain: a threshold on a slope, ε being the linearised slope of the fast mode times ten points.

7.4.3 Stage A: the open loop

Thresholds (`mesures/mimetisme/P3/00-seuils-declares.md`, commit `1606f9f`, the only file of the commit, before any contact with the data). Model: gpt-oss-20b, 5 110 positions (10 windows of 511), 122 640 cells; global LRU cache with the group rule, $N = 146$ (19 %) and 238 (31 %); causal flips (residents = content of the cache just before the group), unbiased mixing weights. The sets are computed on the trajectory of the exact model: this is the *open loop*.

The lever has amplitude, and it comes from residence: the random mask of the same density gains nothing, the inverted sign loses (Table 7.7). At small β, a single expert changes per changed cell (1.00 to 1.02), and the expert of highest logit stays chosen in 99.98 to 100 % of the cells: the lever touches only the last ranks. The V1 and V2 curves overlap in the plane (share changed, misses avoided)

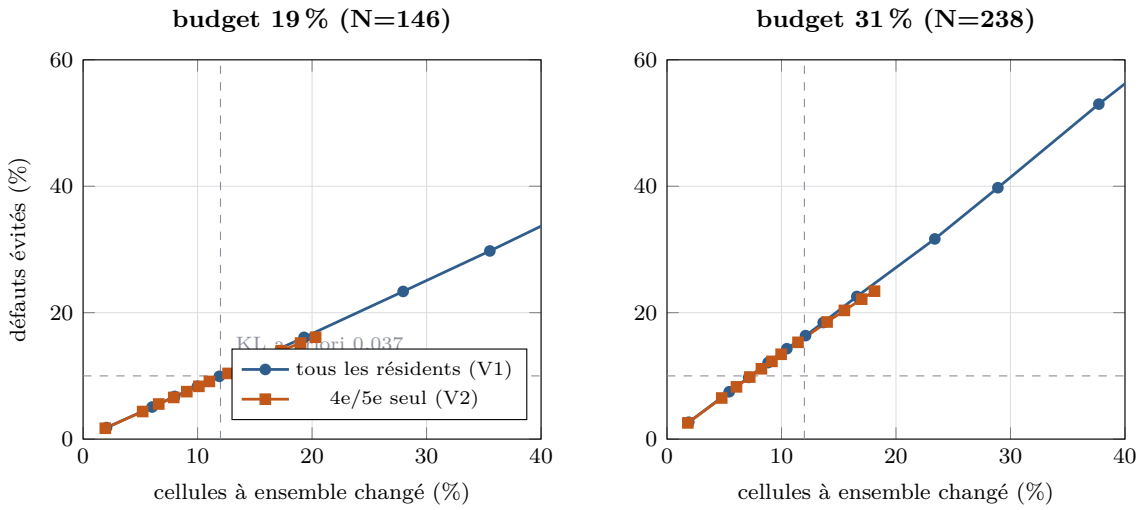


Figure 7.7: The currency of the lever is the number of cells changed: the two variants of the bias fall on the same curve. [meas.] Misses avoided against share of cells with a changed set, V1 and V2, at the two budgets; markers at −10 % of misses and at the crossing. The label "KL a priori 0.037" is the constant of the spontaneous flips; the judge shows that it is probably too low for an online router (0.090 per root of a cascade). In the original, this label overlaps the legend. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P3/series/*.csv`; `genome/mimetisme/P3/figures/fig-pareto.tex`, script `outils/mimetisme/P3/p3_series.py`.

(Figure 7.7): the currency of the lever is the number of cells changed, not the shape of the bias. If speed followed 1/misses, −10, −16 and −23 % of misses would give ×1.11, ×1.19 and ×1.29 [calc.], which is not measured.

Controls [meas.]: W1, the logits recomputed from h and the F32 router recover the set of the capture in 122 640 cells out of 122 640 (break: without the bias of the router, 0.6544); W2, $\beta = 0$ reproduces the misses of the replay; W3, the signs; W4, the bound E7 respected (ratios 0.53 to 0.78; tolerance of 1.5 not derived, stated); W5, a second, independent LRU reproduces the exact count on the 16 rows; W6, causality by truncation. Declared verdict of stage A: **open, stage B justified** (−10 % reached at both budgets, a priori KL below 0.07).

What the judge of stage A adds (`genome/mimetisme/P3/JUGEMENT.md` § 4). The numbers are exact. Two readings are too optimistic. (1) Stage B, as written, imposes on the f64 reference the sets of the open loop at *all* the cells: it forbids the secondary flips that the wanted flip would trigger in the following layers. Yet 59 % of the flips of lot R are secondary (3 006 out of 5 087), and $P(\text{flip at } \ell+1 \mid \text{flip at } \ell) = 26.6\,\%$ against 3.4 % (Chapter 12). Stage B therefore measures the *direct* cost of the flips, a plausible lower bound: a stage-B KL above 0.07 kills P3, a KL below 0.038 is not enough for "holds". (2) The direction of the error of the a priori KL is probably known: relative to the roots of the cascades alone, the constant is $0.0128/(2\,081/122\,640) = 0.755$ instead of 0.309 [calc.], hence an a priori KL of 0.090 at $N = 146$ (above death) and 0.055 at $N = 238$.

7.4.4 Stage B: the excess KL measured

Stage B imposes, in the f64 reference of lot R, the biased sets, and measures the excess KL and the first output choice against the reference. Thresholds (declared with stage A, recalled in `mesures/mime`

Table 7.8: P3, stage B at $N = 146$: where the disagreements of first choice fall, according to the margin of the reference (log-probability of the 1st minus that of the 2nd) [meas.] by the judge.

	break T1 (51)	$\beta = 0.06$ (295)	$\beta = 0.07$ (317)	all positions
disagreements at margin < 0.1	84.3 %	53.6 %	53.3 %	14.4 % of the positions
disagreements at margin < 0.3	94.1 %	86.1 %	85.8 %	37.2 % of the positions
median margin of the disagreements	0.015	0.087	0.088	(agreements: 0.51)
first choice if margin ≥ 0.1	99.6 %	93.7 %	93.2 %	
first choice if margin ≥ 0.3	99.8 %	97.4 %	97.2 %	

Data: `genome/mimetisme/P3/JUGEMENT-etage-B.md` §4; `mesures/mimetisme/P3/juge/etage-B/`.

`tisme/P3/02-etage-B-declare.md`, commit `c4a037a`, before any pass): "holds" if, at the same β, -10 % of misses, KL ≤ 0.038 and identical first choice ≥ 95 %; "fails" if KL > 0.07 at the β of -10 %.

At $N = 146$. At $\beta = 0.06$: misses -9.93 % (194 904), KL 0.0209, first choice 88.4 %; at $\beta = 0.07$: -11.53 %, KL 0.0239, first choice 87.6 % [meas.]. The judge reread them from the `.kld` files with a second reader, to the last digit shown (KL 0.0208902 and 0.0239339), and the control $\beta = 0$ falls to zero to the bit (same SHA-256 fingerprint as the reference). The KL holds, the first choice fails: **inconclusive at the literal threshold**. The judge asked where the disagreements of first choice come from (Table 7.8): largely from near-ties of the reference (half below 0.1 logit of margin, 86 % below 0.3, against 14 and 37 % of the positions), but not only: outside the positions with margin < 0.1, the first choice stays at 93.7 and 93.2 %, below 95 %. This split is post hoc and does not change the verdict.

At $N = 238$, with a weighted criterion fixed after reading. The following criterion was written *after* reading $N = 146$, and says so in so many words (`mesures/mimetisme/P3/03-critere-marge-declare.md`, commit `e8bf954`, before any pass at $N = 238$): the "weighted" reading requires the first choice ≥ 95 % on only the positions where the margin of the reference is at least 0.3 logit. Its reason, independent of the number read: taking the 2nd token instead of the 1st costs at most the margin in log-probability; a flip at margin < 0.3 is cheap. A "holds" at $N = 238$ alone is called *partial*. At the first step that crosses -10 %, $\beta_1 = 0.041$: misses 97 604, that is -10.0192 % (20.8 misses below the threshold of 97 624.8); KL 0.0117202; literal first choice 91.96 %; weighted 98.75 % (20 disagreements out of 1 601 positions) [meas.], reproduced by the judge to the last digit. Declared verdict: **weighted "holds, partial"; literal "inconclusive"**.

The judge of $N = 238$ raises a serious point of scope (`genome/mimetisme/P3/JUGEMENT-N238.md` § 5, G1): on the real data, the weighted condition *never bites before the KL*. Eight real files of KL ≤ 0.029 all pass it at 97.1 % or more, llama.cpp included (92.55 % literal and 98.06 % weighted against the same reference); the only real file that misses it has a KL of 0.243. The lever at β_1 is closer to the reference than llama.cpp in the weighted reading, and 0.6 point further in the literal one. The weighted "holds" therefore says, to date, "-10 % of misses and KL ≤ 0.038", not a third thing.

7.4.5 The online biased router, in the engine

Only the online trial lifts the open loop: the bias applies to the logits recomputed at each layer on the real trajectory, with the residents drawn from the cache of the engine at the same instant. Mechanism, quantities, thresholds, control and break are declared before the least line of engine

Table 7.9: P3, online biased router in the engine, 238 slots, against the same engine without bias [meas.]. Redone by the judge to the last digit shown; the weighted reading is counted on the 1 592 positions of P0 with margin ≥ 0.3.

β	misses (change)	KL	literal	weighted
0 (P0, and P0b)	108 679	0	100 %	100 %
0.041	97 780 (−10.0286 %)	**0.031388**	88.549 %	**96.357 %**
0.05	95 610 (−12.0253 %)	0.037692	87.569 %	96.043 %
0.3 (break)	50 911 (−53.1547 %)	0.089069	76.980 %	88.631 %

Data: `genome/mimetisme/P3/JUGEMENT-routeur-ligne.md` § 3; `mesures/mimetisme/P3/routeur-ligne/`, `mesures/mimetisme/P3/juge/routeur-ligne/`.

code (`mesures/mimetisme/P3/06-routeur-ligne-declare.md`, commit `0f38124`, then the code in `eb78770`): a kernel `routeur_topk_biais` chooses the four experts by `logit + b`, computes the weights on the unbiased logits, behind a flag absent by default; baseline for comparison: the same engine without bias (P0), 238 slots, same windows.

At the point estimate, the weighted reading **holds** and the literal one is **inconclusive**; the break $\beta = 0.3$ fails, and the control $\beta = 0$ is equal to the bit (trace and 5 120 fingerprints) to the engine before the option (Table 7.9). The judge read the kernel line by line: the bias enters only the selection, not the weights nor the order; its emulation on 20 000 draws and the real data of layer 0 show no leak.

Serious point G1: nothing holds at its unfavourable bound. By resampling the ten windows, the upper bound of the KL is above 0.038 for 20 seeds out of 20 (0.03831 to 0.03849); the lower bound of the weighted reading is below 95 % for 14 seeds out of 20 (94.96 to 95.02 %); the margin of 31 misses at the threshold falls to −86 if the cache restarts empty at each window, and the −10 % is not reached at three neighbouring budgets out of four (230: −9.45 %; 234: −9.75 %; 246: −9.84 %) [meas.]. "Holds" is an estimate at the edge; "borderline" is the right word.

The drift, layer by layer. Online, the share of cells changed rises with depth, whereas it is flat in open loop (Table 7.10). The judge separates what cannot come from the bias alone: a cell is "inconsistent" if an incoming expert was absent from the real cache, or an outgoing one present, which is impossible if its logits are those of P0. At least 39.3 % of the cells changed are certain drift (6 207 out of 15 799), 0 in layer 0, 59 % in layer 23; 87.5 % of them have an upstream layer changed at the same step: the drift passes mostly through the residual of the step, the routing cascade, and five times less through the KV history.

Verdict of the judges: borderline, not a building block. The lever exists, it comes from the cache, and it is measured in the engine: −10 % of misses for a KL of 0.031. But none of the three conditions holds at its unfavourable bound, the weighted gate does not bite before the KL, and a single budget, a single reading text, a single pass carry it. The lever cannot serve as a building block without a second draw (another text, another budget) that confirms it; that draw is in progress elsewhere, and this chapter does not cite it.

Table 7.10: P3 online, $\beta_1 = 0.041$: share of the cells of the layer (in %) [meas.] by the judge.

layer	0	3	6	9	12	15	18	21	23
changed online	7.38	8.65	11.12	10.02	12.64	17.63	15.19	12.41	20.12
changed in open loop	7.30	6.87	8.00	5.91	6.32	9.80	8.51	5.13	7.50
natural flips (P0 ≠ ref.)	0.04	0.27	0.57	0.55	0.88	1.57	1.60	1.94	2.95
certain drift	0	1.70	3.01	3.72	5.23	7.20	6.01	6.30	11.94
inconsistent share of the changed	0	19.6	27.1	37.0	41.3	40.8	39.5	50.7	59.3

Data: `genome/mimetisme/P3/JUGEMENT-routeur-ligne.md` § 5; `mesures/mimetisme/P3/juge/routeur-ligne/cellules.json`.

7.5 P4: prefetching with calibrated confidence

The idea. The prefetching of the project died of its *precision*: $p = 0.692$, +34 % of bytes, −11 % of speed (lot O, § 9.2), and lot O wrote that it would come back only with $p > 0.9$. Two borrowings: conformal prediction, which guarantees the precision of a subset of predictions without any distributional assumption [103, 104]; and background traffic that occupies only the free resource and backs off on demand [105]. The predictor is case-based reasoning, like case law: the 20 earlier tokens of the same text whose path of experts most resembles vote for the experts of layer $\ell_0 + d$.

7.5.1 The guarantee, set out

A token t crosses L layers; at layer ℓ it asks for k experts. The cache (global LRU) leaves a set of misses. At the end of the routing of ℓ_0, the predictor proposes for $\ell = \ell_0 + d$ candidates e of confidence $s(e) \in [0, 1]$ (weighted share of the 20 neighbours that carry e). A *load* is a candidate absent from the cache with $s \geq \theta$; its precision is $p(\theta) = P(e \in S \mid e \text{ loaded}, s \geq \theta)$, its coverage $u(\theta)$ the share of the misses preceded by a correct load.

Proposition 7.12 (Split conformal calibration, fixed sequence)**.** *On the calibration half, let $n(\theta)$ be the loads of confidence $\geq \theta$, of which $x(\theta)$ are correct, and $\mathrm{LB}(\theta) = B^{-1}(\delta;\, x, n - x + 1)$ the one-sided Clopper–Pearson bound [106] of level $1 - \delta$. One tests $H_\theta : p(\theta) < 0.9$ going down the grid from the highest threshold that retains at least 30 loads, and stops at the first failure; θ^* is the last threshold rejected. If the labels are exchangeable, $P\big(p(\theta^*) < 0.9\big) \leq \delta$, without multiplicity correction.*

Sketch. For each fixed θ, $P(\mathrm{LB}(\theta) \geq 0.9 \mid p(\theta) < 0.9) \leq \delta$ by construction of the Clopper–Pearson bound. In a fixed sequence stopped at the first failure, a family-wise error requires that the *first* true hypothesis met be wrongly rejected, which happens with probability at most δ; this is the principle of "Learn then Test" [104]. ■

Declared setting: $\delta = 0.1$ (one-sided bound at 90 %), grid with a step of 0.02 (`mesures/mimetisme/P4/00-seuils-declares.md`). The exchangeability hypothesis is false for ordered tokens (the history of neighbours grows); the gap goes in the right direction in the four sessions (test precision 0.919, 0.905, 0.947, 0.922 against 0.911, 0.904, 0.918, 0.914 in calibration), without proving it.

7.5.2 The simulator and the gain

The time of a layer without prediction is $t_\ell = a + n_d\tau + \mathbb{1}[n_d > 0]\, q + c + o$, with n_d misses, $\tau = 4.1$ ms per read, $q = 1.0 + 0.85$ ms (copy, decoding), $a + c = 2.22$ ms of computation and o a calibration offset (the base takes 460 ms per token). A background read starts only in a computation window but finishes

Table 7.11: P4, 120b, 326 slots (7 %), $d = 2$, "read ahead" arm, test half: the lead / the judge [meas.]. Gains from the simulator, not validated.

Session	θ^*	precision	coverage	extra bytes	simulated gain
code	0.86 / 0.86	0.919 / 0.914	10.9 / 10.9 %	+0.84 / +0.91 %	+3.58 / +3.46 %
dialogue	0.80 / 0.82	0.905 / 0.918	16.9 / 15.1 %	+1.47 / +1.14 %	+4.96 / +4.39 %
prose-en	0.90 / 0.90	0.947 / 0.947	7.9 / 7.9 %	+0.41 / +0.42 %	+2.60 / +2.56 %
prose-fr	0.86 / 0.86	0.922 / 0.920	9.5 / 9.4 %	+0.72 / +0.74 %	+2.89 / +2.82 %

Data: `genome/mimetisme/P4/resultat.md` § 3; `genome/mimetisme/P4/JUGEMENT.md` § 1; `mesures/mimetisme/P4/juge/juge-120b.json`.

its course: it delays the next demand by $\delta_r = \tau - (a + c) = 1.9$ ms. A correct read saves τ; a wrong one costs δ_r. With U correct loads and C loads on the test half:

$$\text{saving} = U\tau - C\,\delta_r, \qquad \text{gain} = \frac{\text{saving}}{T_{\text{base}} - \text{saving}}. \tag{7.9}$$

For code: $U = 12\,770$, $C = 13\,892$, $T_{\text{base}} = 516.7$ ms $\times$ 1 271 tokens; saving $= 52\,357 - 26\,395 = 25\,962$ ms, gain $= 25\,962/(656\,726 - 25\,962) = 4.12$ % [calc.], against 3.58 % simulated. When $\delta_r \to 0$ one recovers the formula of the sorting, which overestimates by 2 to 2.4 times. The formula of lot O, $f - (1 - p)/p \times$ disk share with $p = 0.918$, $f = 80/460 = 0.174$ and a disk share of 0.69, gives +11 %: it assumes that every correct read is hidden in full, which is false when the computation window (2.2 ms) is shorter than the read (4.1 ms). Ceiling (perfect predictor): ×1.21 by the computation of lot O.

7.5.3 The measurement and the verdict

Declared threshold (commits `47339c2`, amendment `4732d35`, before the first session measurement): at least +5 % of simulated speed for at most 5 % more bytes, on 3 reading genres out of 4 and in generation, with an achieved precision of at least 0.85; dead below +2 % or below 0.8 precision. Rule 3 of the amendment: as long as the simulator is not validated, the verdict cannot go beyond "open".

Controls. T0 (two equal LRUs) holds. T2, perfect predictor: ×1.2247 (v1) then ×1.1786 (v2), which misses the ×1.21 ± 2 % by 0.6 point. **T3 misses**: a synthetic predictor of precision 0.692 was to reproduce the engine's −11 %; the simulator gives +5.2 then +4.9 %. It is the only measured point of the engine, and the simulator does not recover it: all its gains are orders of magnitude, with an unknown sign of error. The break by shuffled labels holds: no calibration finds a threshold, 7 sessions out of 7.

What holds: the precision guarantee, 0.905 to 0.947 (0.918 aggregated; 0.922 for the judge) for +0.4 to +1.5 % of bytes, where the killed prefetching had 0.692 for +34 %. What does not hold: the gain, +3.51 % on average (+3.31 % for the judge), 0 genres out of 4 at +5 % (Table 7.11). The "pin without reading anything" arm is dead (+0.09 %). Figure 7.8 shows the bottleneck: holding 0.9 precision costs the coverage, 8 to 17 % of the misses, with the same slope for the four genres. On the 20b at 154 slots, the four genres pass +5 % (+5.05 to +9.10 %), not redone by the judge.

What the judge adds (`genome/mimetisme/P4/JUGEMENT.md` § 3). (1) The "generation" leg rests on the two degenerate traces of P1 (Section 7.2.3): the "98.9 % hit rate, nothing to prefetch" of prose-en is that of a loop of blanks; on wiki, the calibration finds no threshold because its calibration

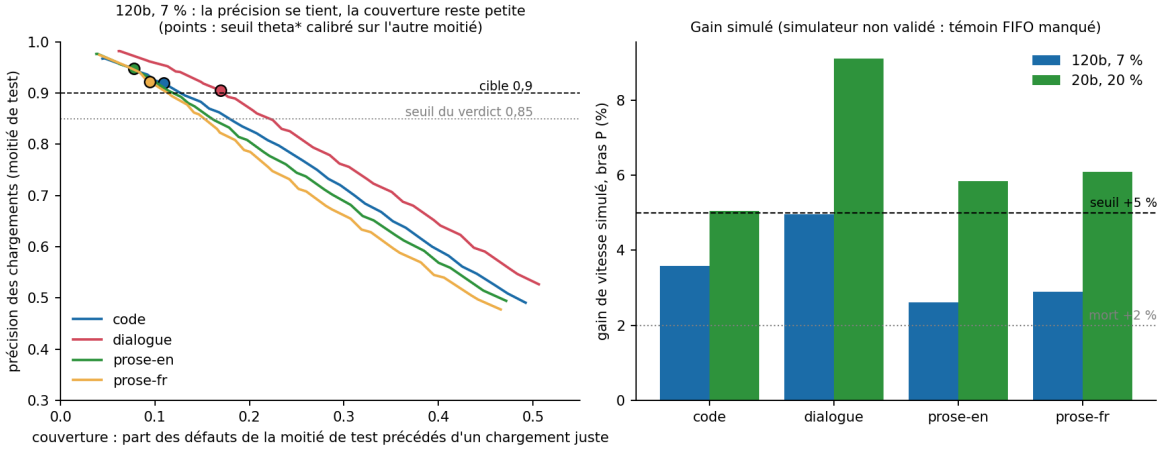


Figure 7.8: Calibration holds the precision at 0.9; it is the coverage that stays small, and the gain with it. [meas.] Left: precision–coverage curves of the test half, per session of the 120b at 7 %; points: θ^* calibrated on the other half. Right: simulated speed gain, "read ahead" arm, against the threshold of +5 % and death at +2 %; simulator not validated (control T3 missed). (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P4/figures/p4-precision-couverture.csv`, `p4-gains-par-session.csv`; script `outils/mimetisme/P4/p4_figures.py`.

half (before the loop) has a precision of 0.73 at $\theta = 0.9$, whereas its test half (in the loop) reaches 0.979. Neither a failure nor a success of the mechanism: unrepresentative data. (2) The gain depends on the anchoring at 460 ms: code makes 92 misses per token in the simulated LRU, which already exceeds 460 ms and makes the calibration offset negative; anchored per session, the gain of code falls to +0.42 %; the reading mean goes from +3.08 to +3.61 % depending on the anchoring, never more than one genre out of 4 above +5 %. (3) A sentence of the code is inaccurate (the offset o is fitted on the same four sessions), with no effect on the gain.

Verdict of the judge: open, rather on the side of fails. Conformal calibration settles the precision problem that had killed prefetching; the bottleneck is now coverage. The only valid arbiter: an ABBA pass of the engine with the gate θ^*, which would also validate the simulator.

7.6 P5: the choice of the synonym symbols as a spin glass

The idea. The synonymous mutation of lot B (Chapter 10) puts each expert weight into one of eight levels by minimising a weighted error in a single greedy pass; it is the only lossy gain that has passed the KL (-0.0098 ± 0.0033). The physicist recognises in it the energy of a spin glass, and knows that the greedy pass stops there in a valley; simulated annealing [107] and replica exchange [108] cross the barriers. It is the domain furthest from computation of all the candidates retained, and the scout found no annealing nor replica exchange applied to the quantisation of a large language model.

7.6.1 The problem set out

A row of an expert matrix has $n = 2\,880$ weights w_j, in blocks of 32 sharing a scale $s_b = 2^{e_b - 128}$ (MXFP4). The synonymous mutation puts each weight into one of $k = 8$ levels $\pm c_m s_j$ ($m = 0, \ldots, 3$, $c = (0.4743,\ 2.8486,\ 6.8372,\ 12)$ in doubled magnitudes). Let q be the vector of the chosen levels and

$r = w - q$ the residual. The output error, averaged over the inputs x_i of the expert weighted by the square g_i^2 of the router weight, and the code cost are

$$E(q) = r^\top H r + \lambda' \sum_j \ell(q_j), \qquad H = \frac{\sum_i g_i^2\, x_i x_i^\top}{\sum_i g_i^2} + \delta\, \bar{m}\, I, \tag{7.10}$$

with $\ell(m) = -\log_2 p_m$ the code length of level m, $\bar{m}$ the mean of the diagonal of H without damping and δ the damping of GPTQ (deployed value $\delta = 1$).

Lemma 7.13 (Change of a single weight)**.** *If the level of coordinate j is replaced, the residual changes by $\delta r = q_j - q_j'$ at position j only, and*

$$E(q') - E(q) = 2\,\delta r\,(Hr)_j + \delta r^2 H_{jj} + \lambda'\big(\ell(q_j') - \ell(q_j)\big).$$

Proof. $r' = r + \delta r\, e_j$, so $r'^\top H r' - r^\top H r = 2\,\delta r\, e_j^\top H r + \delta r^2\, e_j^\top H e_j$. ∎

Keeping the vector Hr up to date costs n operations per accepted change ($Hr \mathrel{+}= \delta r\, H_{:,j}$); evaluating the $k-1$ candidates of a coordinate costs $O(k)$. That is what makes the sweeps practicable; the energy kept incrementally equals the energy recomputed from scratch to within 1.3×10^{-15} relative [meas.].

Three readings of the same object. (i) *The closest vector.* With $H = VV^\top$, V upper triangular, $r^\top H r = \sum_i \big(\sum_{j \le i} V_{ji} r_j\big)^2$: term i depends only on $r_1, \dots, r_i$. Choosing q_1 that minimises the first term, then q_2 given q_1, etc., is Babai's nearest-plane algorithm [109], and GPTQ run from back to front is identical to it [51, 110]. Each term is minimised exactly given the past, never looking at the future. (ii) *A spin glass.* Writing the level of j by a Potts state $\sigma_{jm} \in \{0,1\}$, $\sum_m \sigma_{jm} = 1$, $r_j = w_j - s_j \sum_m c_m \sigma_{jm}$,

$$E_{\mathrm{err}} = \mathrm{const} - 2 \sum_{j,m} h_{jm}\, \sigma_{jm} + \sum_{j,j',m,m'} J_{jj'}^{mm'}\, \sigma_{jm}\sigma_{j'm'}, \quad h_{jm} = s_j c_m (Hw)_j,\ J_{jj'}^{mm'} = H_{jj'} s_j s_{j'} c_m c_{m'}, \tag{7.11}$$

an eight-state Potts model on the complete graph of 2 880 sites, whose couplings are not independent: H is a sum of N rank-one terms (147 to 1 334 calibration inputs per expert, according to the logs reread by the judge) plus the damping. One seeks a *discrete* vector almost orthogonal to N patterns x_i: this is the frame of the symmetric binary perceptron, for which Aubin, Perkins and Zdeborová conjecture a "frozen" landscape where local algorithms get stuck [111]. This connection is a frame, not a result: their patterns are independent Gaussians and their weights ±1. (iii) *A Gibbs measure*, $\pi_T(q) \propto e^{-E(q)/T}$.

Proposition 7.14 (The two dynamics preserve π_T)**.** *(a) The heat bath on one coordinate, which redraws q_j with probability $p_m \propto e^{-\Delta E_m/T}$ (ΔE_m of lemma 7.13), satisfies detailed balance for π_T; at $T \to 0$, it is cyclic coordinate descent. (b) In replica exchange at temperatures $T_1 < \cdots < T_{n_t}$, accepting the exchange of the states of two copies a, b with probability $\min\big(1, e^{(\beta_a - \beta_b)(E_a - E_b)}\big)$, $\beta = 1/T$, preserves the product law $\prod_a \pi_{T_a}$.*

Proof. (a) The drawing law is π_T conditioned on the other coordinates: $P(q \to q') = \pi_T(q')/Z_j$, where $Z_j = \sum_m \pi_T(q \text{ with } j \leftarrow m)$ does not depend on the current level of j; so $\pi_T(q)\,P(q \to q') = \pi_T(q)\pi_T(q')/Z_j$, symmetric in q, q'. (b) The exchange multiplies the weight of the product law by $\dfrac{\pi_{T_a}(q_b)\,\pi_{T_b}(q_a)}{\pi_{T_a}(q_a)\,\pi_{T_b}(q_b)} = e^{-\beta_a E_b - \beta_b E_a + \beta_a E_a + \beta_b E_b} = e^{(\beta_a - \beta_b)(E_a - E_b)}$; the Metropolis rule on this ratio restores detailed balance. ∎

Simulated annealing is the same idea without exchange: a single copy whose T decreases. Setting of step 1: 8 temperatures from 0.02 to 1.0 times the mean Babai error per weight, exchange at each sweep, 40 sweeps, then quenching at $T = 0$ of the best state visited; all copies start from Babai.

Table 7.12: P5: the controls, all declared before being run [meas.].

Control	Set-up	Result
W1 brute force	10 weights, 5 levels ($5^{10} = 9\,765\,625$ states), 1 000 correlated H	Babai misses the optimum on **847 / 1 000** (median excess 106 %, up to ×33): the instrument can fail
W1, replica exchange	~~200 sweeps, 8 temperatures~~; 3 000 sweeps, 12 temperatures	~~at the optimum everywhere~~: fails on 43 / 1 000; final setting: **1 000 / 1 000**
W2 breaks	0 sweeps; frozen temperatures; permuted H	Babai exactly; optimum on only 107 / 847; 2 / 847
W3 sphere decoding	Fincke–Pohst [112], $n = 16$, real block of H	exchange = sphere on 10 / 10; Babai not optimal on 6 / 10 (8 to 44 % above)
W4	generic Babai against `synonyme.py::choisir`	0 symbols different out of 2 048

Data: `genome/mimetisme/P5/resultat.md` § 2; `mesures/mimetisme/P5/temoins-*.json`, `W3-sphere-n16.json`.

7.6.2 The controls: the glass is real at small dimension

Thresholds of step 1 committed before any reading of a capture (`mesures/mimetisme/P5/00-seuil-declare.md`, commit `ab92f4e`); three addenda, each committed one second before its first log (checked by the judge).

At small dimension and strong conditioning, the glass is real (Table 7.12): Babai loses up to ×33 on the toy, 44 % on a real block of 16 columns, and replica exchange solves it. Figure 7.9 shows a slice of it: a basin that the greedy pass does not reach. The question of the lead is whether this glass still exists at $n = 2\,880$ and $\delta = 1$, where the damping softens it.

7.6.3 Step 1: the gain can be measured and does not transfer

Step 1, without a model file (the lock on the card existed): the real H of layers 0, 8, 16, 23, built from the captures of inputs, with *synthetic* Gaussian weights; 8 instances (layers × experts 0 and 16), 64 rows each, $\delta = 1$, rate equal to within ±0.005 bit. Threshold: a fall of the held-out evaluation error of at least 10 % on at least 6 instances out of 8; dead below 3 % in median.

At the better of the two λ (at equal rate), the median rises to 5.46 % for cyclic descent; the lead publishes 6.8 % for replica exchange. On the calibration, the fall is 51.5 % (descent) and 54.6 % (exchange); on the optimised objective, 6.8 and 7.8 % [meas.]. **Open**: below the success threshold, outside the death zone (Table 7.13 and Figure 7.10). The optimiser is good on what it is given, and what it learns hardly transfers: the ratios of objective to evaluation are 8 to 10.

Why: the error moved into the kernel of the calibration. Decompose the error in the eigenbasis (λ_k, u_k) of H_{train}: $E = \sum_k e_k\,(u_k^\top H u_k)$, $e_k = \|\Delta W u_k\|^2$. An optimiser that minimises $\sum_k \lambda_k e_k$ at a roughly fixed budget of weight error *moves* the error towards the directions of small λ_k. Yet the calibration H has rank at most N, from 147 to 1 334 for $n = 2\,880$; in its kernel it is zero and only the damping sees the error, whereas the evaluation places 40 % (Babai) to 47 % (descent) of its own there. On L8-E0 (32 rows), the evaluation error falls by 38 % in the first three groups of directions (0.00602 → 0.00373) and rises by 6 % in the other eleven (0.01225 → 0.01303) [meas.] (Figure 7.11). This is the mechanism of a model learnt on N examples in dimension $n > N$: the empirical energy is flat where the true energy is not.

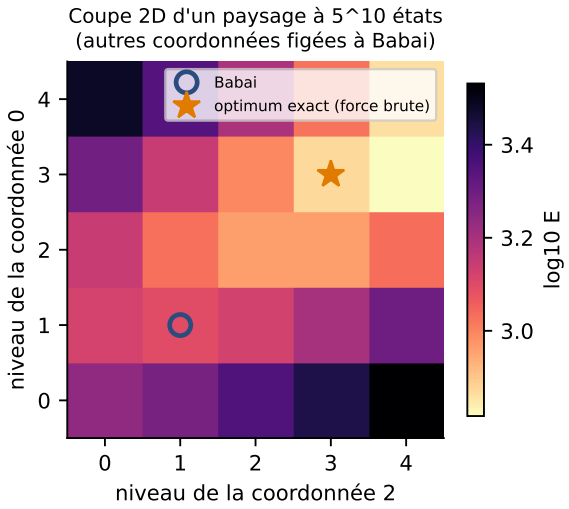


Figure 7.9: On a ten-weight toy, the greedy pass stops in a valley that the exact optimum leaves. [meas.] Slice of an energy landscape with 5^{10} states (instance 464) along two coordinates, the other eight frozen at the Babai solution; circle: Babai; star: levels that the exact optimum gives to these two coordinates. The energy of a cell of the slice is not that of the optimum (1 248 for Babai, 37.9 for the optimum, all coordinates free). (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P5/figures/paysage-toy.csv`; script `outils/mimetisme/P5/figures.py`.

Table 7.13: P5: held-out evaluation gain of cyclic descent ($\lambda = 0$) against Babai, in %, $\delta = 1$, 64 rows [meas.]. The judge redoes each instance to within 4×10^{-14} point, then on new weights (seed 2026).

Instance	N calib.	lead	judge, same weights	judge, new weights	ΔH_0 (bit)
L0-E0	762	4.74	4.74	0.40	+0.0032
L0-E16	1 334	8.87	8.87	6.87	+0.0045
L8-E0	1 329	7.88	7.88	8.06	+0.0040
L8-E16	300	3.83	3.83	−0.27	+0.0033
L16-E0	330	3.23	3.23	6.87	+0.0040
L16-E16	147	1.43	1.43	3.90	+0.0025
L23-E0	303	7.55	7.55	12.07	+0.0035
L23-E16	239	5.43	5.43	4.07	+0.0035
median		5.09	5.09	5.47	
≥ 10 %		0 / 8	0 / 8	1 / 8	

Data: `genome/mimetisme/P5/JUGEMENT.md` § 2; `mesures/mimetisme/P5/etape1/gains-d1.csv`, `juge/`.

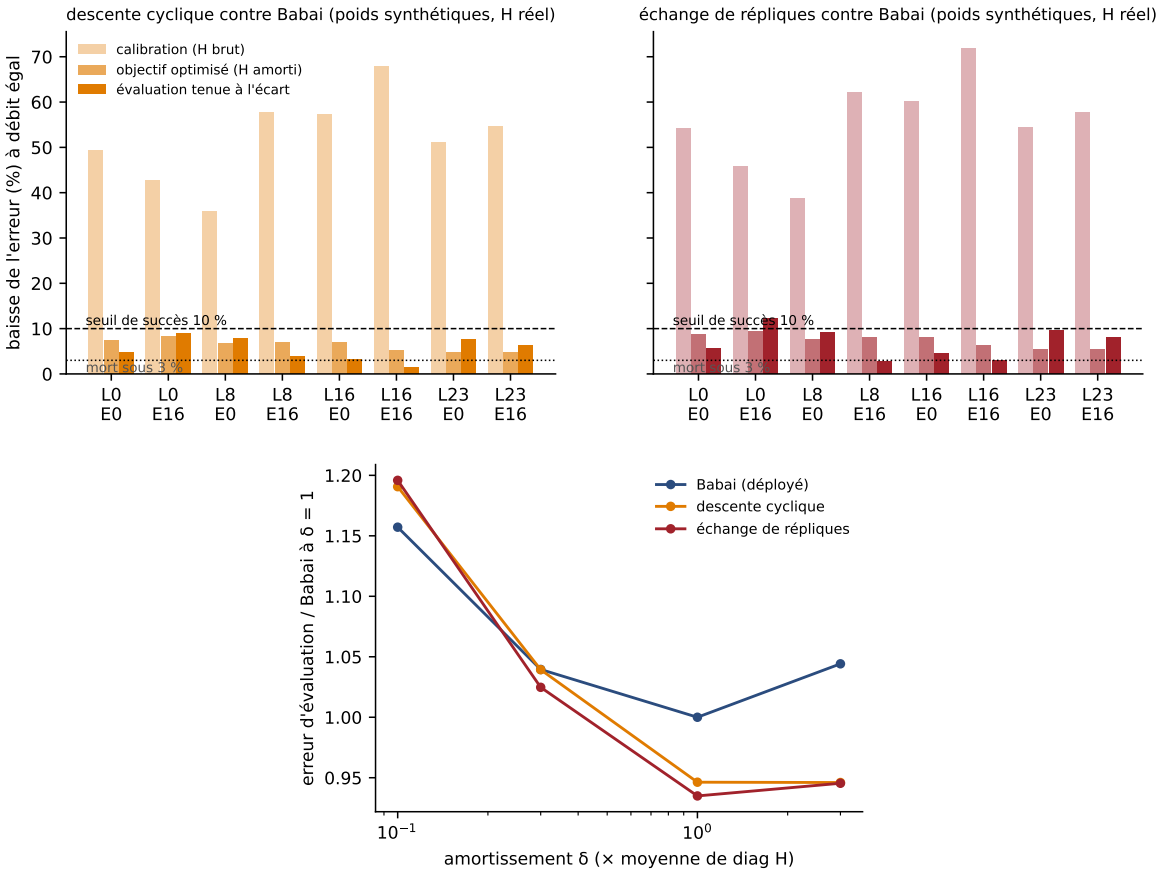


Figure 7.10: The optimiser removes half of the calibration error and twenty times less of the held-out error. [meas.] Top: fall of the error against Babai, per instance, on the calibration, on the optimised objective and on the held-out evaluation; cyclic descent (left) and replica exchange (right), success (10 %) and death (3 %) thresholds. The bars of replica exchange are those of the lead; the judge reproduces them only in class (median 4.8 % against 6.8 %). Bottom: mean evaluation error against the damping δ, relative to deployed Babai ($\delta = 1$). Synthetic weights (step 1). (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P5/etape1/gains-d1.csv`, `etape1/table-delta.csv`; script `outils/mimetisme/P5/figures.py`.

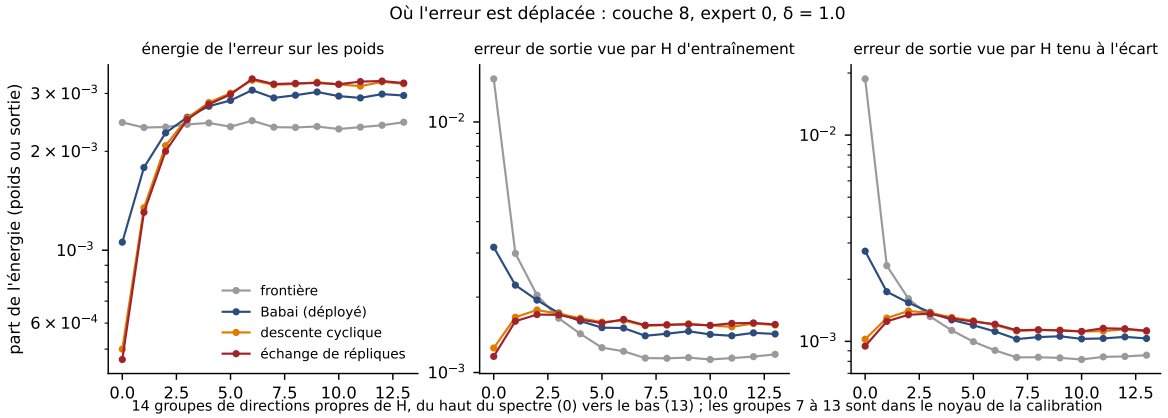


Figure 7.11: The optimiser pushes the error towards the directions that the calibration does not see, and the evaluation finds it there. [meas.] Layer 8, expert 0, $\delta = 1$: share of the energy per group of eigendirections of H, from the top of the spectrum (0) towards the bottom (13), on the weights (left), seen by the training H (middle), seen by the held-out H (right); frontier, Babai, cyclic descent, replica exchange. Groups 7 to 13 are in the kernel of the calibration. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P5/figures/spectre-erreur.csv`; script `outils/mimetisme/P5/figures.py`.

Damping and heat. Mean evaluation error relative to deployed Babai: Babai 1.157, 1.040, 1.000, 1.044 at $\delta = 0.1, 0.3, 1, 3$; descent 1.191, 1.039, 0.946, 0.946; exchange 1.196, 1.025, 0.935, 0.945 [meas.]. $\delta = 1$ is near the optimum of Babai; the optimiser does not move the best δ, and overfits below it. A table that compared with Babai *at the same* δ gave "11 %" at $\delta = 3$: an artefact, struck out by the lead itself. The Gibbs estimator (a state drawn at finite T would generalise better than the ground state) was tried on L8-E0: −8.1, −9.0, −8.9, −7.9, −4.3, −0.3 % at $T/e_b = 0$ to 1; a plateau within the noise, then degradation; no gain in generalisation from heat.

In bits. At high resolution, $D \propto 2^{-2R}$, an error ratio ρ is worth $\Delta R = -\frac{1}{2}\log_2 \rho$: $\rho = 0.935 \to 0.049$ bit per weight, $0.946 \to 0.040$, $0.866 \to 0.104$, against $0.80 \to 0.161$ aimed at [calc.], checked by the judge. The cost: the descent takes about 0.1 s per row against 6 ms for Babai (×15 to ×30), the exchange about fifteen times more.

7.6.4 The announced mechanism, and what the judge says of it

The lead looked for the cause of the non-transfer in the finiteness of the calibration sample and declared a test (addendum 3, commit `d958a46`): train on the calibration plus windows 0 to 4 of the evaluation, judge on windows 5 to 9; "supported" if the median gain of the descent on the test rises by at least 2 points and rises on at least 6 instances out of 8. It wrote: "+5.9 to +7.3 % (… +2.6 points; ~~supported threshold held~~)". The judge (`genome/mimetisme/P5/JUGEMENT.md` § 4) rereads the declaration: the two medians published side by side make +1.41 points, which falls into "undecided"; the +2.6 is the median of the rises per instance, another quantity that the declaration did not fix. Then he isolates N by a test declared beforehand (J5): *remove* half of the calibration inputs, same source, same test. Dividing N by 2.3 in median lowers the median gain by only 0.43 point, below the threshold of 0.5: **not supported**. What addendum 3 measured mixes "more N" and "inputs taken from the evaluation file, neighbours of the test". What is clear: N matters for Babai itself (dividing N by 2 costs it 6 % of evaluation error in median).

The judge's replica exchange reproduces the fall of the lead's objective to within 0.1 point on each instance, but not its evaluation gain: median 4.8 %, below its own descent (5.1 %), 0 / 8 above 10 % (the 12.3 % of L0-E16 becomes 6.3 %). At equal quality of objective, two exchanges land on states whose held-out error differs by 2 to 6 points: the "+1.4 points above the descent" is optimiser noise.

Verdict of the judge: open; the mechanism does not hold at the threshold. The main number holds for cyclic descent, identically and on new weights; that of the exchange is reproduced only in class. Step 2 (real weights) is not authorised by the rule and has not run. The lever "number of calibration inputs" remains a hypothesis, to be measured with a fixed source and distant windows.

7.7 P6: fidelity decided by call

The idea. The mixing weights of the router are a softmax over four scores, like the occupation numbers of four levels of a physical system; a low occupation weighs little in the output. The physical image (a Fermi surface) is an analogy: what it proposes is to serve in *fast* mode (the compressed companion of lot B) the expert called with a small weight, instead of removing it. The idea of degrading according to importance is not new [42, 113], nor that of refining in mixed precision [114]; what P6 was to provide the project is the curve that the whole of lot B lacks, the KL as a function of the share f of the misses served fast, of which only the ends were measured (0.0044 and 0.145).

7.7.1 The model and its quantities

An access $a = (t, \ell, s)$ calls expert x_a with weight w_a ($\sum_s w = 1$). A *stay* σ of an expert in the cache runs from its entry (a miss) to its eviction; $A(\sigma)$ is the set of its accesses. The gate fixes the level of a stay at its entry: $\varphi(\sigma) = F$ (fast) if and only if $w_{a_0(\sigma)} < \theta$. A stay costs $\rho = 0.718$ exact expert if it is fast, 1 otherwise. The perturbed energy of the "accept" rule and the total energy are

$$\mathscr{E} = \sum_{\sigma} \mathbb{1}[\varphi(\sigma) = F] \sum_{a \in A(\sigma)} w_a^2, \qquad \mathscr{E}_{\text{all}} = \sum_{a} w_a^2. \tag{7.12}$$

Assumption 7.15 (Small perturbation)**.** The output deviation at access a is $w_a\, \varepsilon_x$, with ε_x an expert deviation of common variance; the total deviation variance is then proportional to the energy. This is not the KL.

The bytes saved are $S = (1 - \rho)\, f$, f being the share of fast stays; all-fast gives $S_{\text{all}} = 0.282$ and $\mathscr{E} = \mathscr{E}_{\text{all}}$. The quantity of the lead is the energy per byte saved, relative to that of all-fast:

$$r(\theta) = \frac{\mathscr{E}/\mathscr{E}_{\text{all}}}{S/S_{\text{all}}} = \frac{\mathscr{E}/\mathscr{E}_{\text{all}}}{f}. \tag{7.13}$$

Proposition 7.16 (The break by chance)**.** *If φ is drawn independently of the weights with $P(F) = f$, then $\mathbb{E}[\mathscr{E}] = f\, \mathscr{E}_{all}$ and $\mathbb{E}[r] = 1$.*

Proof. $\mathscr{E} = \sum_\sigma \varphi(\sigma)\, e(\sigma)$, with $e(\sigma) = \sum_{A(\sigma)} w^2$ independent of $\varphi(\sigma)$; so $\mathbb{E}[\mathscr{E}] = f \sum_\sigma e(\sigma) = f\, \mathscr{E}_{\text{all}}$. ∎

Measured: 1.0001 ± 0.023 over 50 draws and 7 traces [meas.].

Proposition 7.17 (The gate alone)**.** *Let $p(\theta) = P(w < \theta)$ and $\hat{\mathscr{E}}(\theta) = \mathbb{E}[w^2;\, w < \theta]/\mathbb{E}[w^2]$. For a count without carry-over (each access carries its own label) and a selection independent of the state of the cache, $r_0(\theta) = \hat{\mathscr{E}}(\theta)/p(\theta) = \mathbb{E}[w^2 \mid w < \theta]/\mathbb{E}[w^2] < 1$ as soon as the law of w is not degenerate.*

Proof. $\mathbb{E}[w^2;\, w<\theta] = p(\theta)\,\mathbb{E}[w^2 \mid w<\theta]$; since w^2 increases on $[0,1]$, $\mathbb{E}[w^2 \mid w<\theta] < \mathbb{E}[w^2]$ if $P(w\geq\theta)>0$ and $P(w<\theta)>0$. ■

Measured: $r_0(0.20) = 0.414$, $r_0(0.25) = 0.614$ [meas.]. The lead concluded from it: ~~"the misses are not correlated with the small weights; the gain of the gate comes from the law of w, not from the cache"~~. That is false, says the judge (`genome/mimetisme/P6/JUGEMENT.md` § 4.2): the misses are distributed as 19.4, 23.1, 26.8 and 30.6 % over ranks 0 to 3, and 29.6 % of the misses are below $\theta = 0.20$, against 24.9 % when the rank within the group is destroyed; the state of the cache gives the gate about 19 % more bytes saved at equal θ (29.6/24.9 = 1.19). The test of the lead (shuffling the vectors by layer) kept the rank: the correlation goes into S, not into r, which sees nothing of it.

Carry-over. A "fast" label fixed at entry holds for the whole stay: $\mathscr{E} = \mathscr{E}_{\text{gate}} + \mathscr{E}_{\text{rest}}$. At $\theta = 0.20$ on the generations of the 120b, $\mathscr{E}_{\text{gate}}/\mathscr{E}_{\text{all}} = 0.0476$ and $\mathscr{E}_{\text{rest}}/\mathscr{E}_{\text{all}} = 0.1609$: the rest carries 77 % of the energy [meas.]. With $f = 0.0855/0.282 = 0.303$, $r = 0.2085/0.303 = 0.688$ [calc.]. The entry weight predicts the rest badly (correlation 0.34 and 0.25). The *clairvoyant bound*, which would know the largest weight the expert will receive during its stay ($\varphi^* = F \iff \max_{A(\sigma)} w < \theta$), reaches $r^* = 0.185$: carry-over costs a factor 3.7. The rule "promote at θ'" rereads the whole exact expert at the first hit of weight $\geq \theta'$, because the fast file is not nested in the exact one.

Variant B, prefill. In a batch of n_β tokens, each key $u = (\text{batch}, \ell, x)$ is read once. One reads in fast mode the f keys of smallest score, with three scores: the maximum $m_u = \max_j w_j$ (rule A), the mass $M_u = \sum_j w_j$ (rule B), the energy $e_u = \sum_j w_j^2$ (rule C).

Proposition 7.18 (The energy oracle)**.** *At fixed f, rule C minimises $\mathscr{E}$ among all subsets of $f\,|U|$ keys; it therefore bounds A and B from below.*

Proof. $\mathscr{E} = \sum_{u\in F} e_u$ is additive; the subset of $f|U|$ keys of smallest sum is that of the $f|U|$ smallest terms. ■

The maximum is a poor substitute for e_u because it ignores the number of calls ($m_u^2 \leq e_u \leq n_u m_u^2$); the mass $M_u = n_u \bar{w}_u$ counts them.

7.7.2 The measurement, the verdict and what the judge corrects

Thresholds (`mesures/mimetisme/P6/00-seuils-declares.md`, commit `4d2b915`, amendment `c206831`, before any reading of a trace): decoding, "holds" if on *each* of the two generations of the 120b $r \leq 0.70$ with at least 8 % of bytes saved at $\theta = 0.20$; variant B, "holds" if $r \leq 0.70$ at 8 % on 3 genres out of 4 and $r_B \leq 0.9\, r_A$.

Decoding: open. No rule passes 8 % on wiki (7.78 ± 0.38 % for "accept"), none has $r > 0.90$ (Table 7.14). Cumulated over the two traces, "accept" and "promote at 0.40" would pass, but the declared rule required each trace: verdict **open**, and the lead did not move its threshold. The judge adds that "open" comes from the choice of a fixed θ rather than from the mechanism: the weights pile up around 0.2, and S goes from 2.2 % ($\theta = 0.15$) to 8.5 % (0.20) then 16 % (0.25); at exactly $S = 8$ %, trace by trace, r is 0.669 and 0.573, both below 0.70 (exploratory, outside the threshold).

Table 7.14: P6, decoding of the 120b, $\theta = 0.20$, 323 slots, LFU with half-life 8, $\rho = 0.718$: bytes saved S and energy per byte r [meas.]. Reproduced by the judge to within 10^{-4} in S and 10^{-3} in r.

Rule	prose-en S / r	wiki S / r	cumulative S / r	holds?
accept	10.10 % / 0.725	**7.78 %** / 0.565	8.55 % / 0.688	no
promote at 0.30	8.22 % / 0.461	6.95 % / 0.391	7.37 % / 0.442	no
promote at 0.40	9.29 % / 0.573	7.51 % / 0.451	8.10 % / 0.539	no
promote at 0.50	9.77 % / 0.613	7.74 % / 0.553	8.41 % / 0.612	no
clairvoyant (bound)	8.85 % / 0.140	7.19 % / 0.227	7.74 % / 0.185	outside the verdict

Data: `genome/mimetisme/P6/resultat.md` § 2; `genome/mimetisme/P6/JUGEMENT.md` § 1; `mesures/mimetisme/P6/juge/juge.json`.

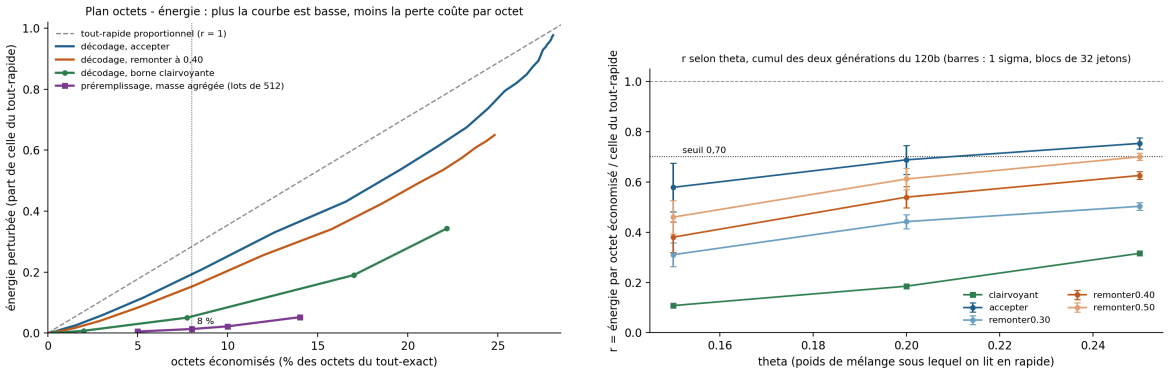


Figure 7.12: A gate on the call weight costs less energy per byte than all-fast, and the aggregated mass of prefill almost nothing; but energy is not the KL. [meas.] Left: perturbed energy against bytes saved (decoding of the 120b: accept, promote at 0.40, clairvoyant bound; prefill: mass aggregated by batches of 512; dashes: proportional all-fast, $r = 1$). Right: r as a function of θ, cumulated over the two generations, bars at 1σ (blocks of 32 tokens), threshold 0.70. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/mimetisme/P6/courbe-plan-decodage.csv`, `courbe-plan-prefill-B.csv`; script `outils/mimetisme/P6/figures.py`.

Variant B: holds, but could hardly fail. At 8 % of the prefill bytes, $r_B = 0.039$ to 0.060 (threshold 0.70) and $r_B/r_A = 0.61$ to 0.66 on the four genres of the 120b, and r_B from 0.030 to 0.074 on the 20b; random drawing gives $r = 1.00$ [meas.]. The judge measures the oracle of Proposition 7.18: r_B is within 4 % of it on the four genres (and within 1.5 to 4.3 % on eight genres, 120b and 20b). The threshold "fails if $r_B \geq r_A$" could hardly fail, since the maximum is a worse substitute for the energy than the sum whatever the law of the weights. At step 0, the mass says only that the energy of a batch concentrates in the often-called experts; it says nothing about the KL.

The curve KL(f), set out as a hypothesis. Step 0, that of the bytes–energy plane of Figure 7.12, can only eliminate. Under the linear hypothesis $\mathrm{KL}(f) - \mathrm{KL}(0) = (\mathrm{KL}(1) - \mathrm{KL}(0))\,\mathscr{E}(f)/\mathscr{E}_{\mathrm{all}}$, with the ends measured on the 20b (0.0044 and 0.1448), 8 % of the prefill bytes would cost 0.002 of KL and 14 % from 0.006 to 0.009. This is not a result: 96.5 % of the KL of the fast mode comes from routing flips, which are non-linear, and the convexity could go either way. Step 1, which would measure the points (f, KL) on the 20b, has not been launched (lock on the card, 8.31 GB to read); the judge asks that it keep the maximum, chance *and* the energy oracle as comparisons at equal bytes: if the KL

follows the energy, mass and oracle give the same KL.

Verdict of the judge. Decoding of the 120b: **open**, reproduced. Variant B: "holds" at step 0, reproduced, but almost the oracle of its own measurement: nothing is said about quality. The gate saves 7.8 to 10.1 % of the bytes read in generation for 16 to 26 % of the energy of all-fast; only step 1 will say what the energy is worth.

7.8 The map from the world to the pipeline

The sorting arranges each borrowed concept on the stage of the pipeline it touches: **F** the format and entropy coding, **Q** the choice of the symbols, **K** the cache of experts, **R** the routing, **L** reading, scheduling and prefetching, **J** the judge and the method. Table 7.15 gives the essentials, with today's verdict (figure III-6.17, which would draw it, is not drawn). Table 7.16 gathers the six leads before their judges.

7.9 What could be wrong

What could be wrong

- **Two generation traces for the 120b, and they are degenerate.** P1, P2, P4 and P6 judge the generation on the same two traces of 256 tokens; half of prose-en is made of tokens produced after the end of the message, and wiki is a raw completion that repeats itself. The judges of P1 and P4 said so; those of P2 and P6 did not raise it, and their generation numbers carry the same reserve.
- **The simulated hit rates exceed those of the engine** (0.74 against 0.59 on prose-en): every simulated cache gain (P1, P2, P4, P6) is an upper bound before the ABBA of the engine. The expert sizes are equal in it, the warm RAM absent.
- **P3 rests only on its judgements**: a single budget carries the "holds", a single reading text, a single online pass; the second draw, in progress elsewhere, may confirm it as well as contradict it, and this chapter does not know it.
- **The simulator of P4 is not validated** (T3 missed): its gains are orders of magnitude with an unknown sign of error.
- **P5 is measured on synthetic** Gaussian weights, on 64 rows and the sole input H of gate and up; the real weights have giant columns correlated with H, and the Babai–optimum gap may be larger or smaller there. No claim about the quality of the text.
- **P6 is measured in energy**, not in KL; and its "holds" at prefill could hardly fail.
- **The proposition on the inclusive cache** (Proposition 7.10) is set out by the writing under a hypothesis on the rule of the engine (refreshing of the RAM at each access) read in the code by the judge, not executed; it explains the equality measured by the judge, it does not replace it.
- **The sources of the leads**: the sorting reopened no external source; the writing reread on 2026-10-29 the records of those this chapter cites (Crossref, arXiv, USENIX, RFC), not the papers. The scouts cited by number ("life #1") are known here only through their reports.
- **The ranking of the sorting is not measured**: "gain × novelty × feasibility" is a judgement of order; two leads ruled out (reverse water-filling by usage, BOLA) might be worth more than P4 or P5 once put in numbers.

Table 7.15: The map from the world to the pipeline: the borrowed concepts, their stage, their verdict (`geno me/mimetisme/tri.md` § 6, verdicts of the leads updated by their judges).

Concept from the world	Domain	Stage	Verdict
Slow and fast reservoir (Hasselmann); power-law forgetting (ACT-R)	climate; psychology	K	P1: open
Seismic aftershocks (Omori–Utsu), Hawkes processes	seismology	K	closed (E1)
Tides, periodic phase of the layer	geophysics	K	small gain, comparator of P1
Exclusive hierarchy (demotion)	storage	K	P2: open
Compressed cache	architecture	K	dominated by P2
Nucleation, nucleus barrier	thermodynamics	K	closed (E3)
Allocation by marginal utility (Charnov, hydraulics, METRIC)	ecology, hydrology, logistics	K	closed
Admission by frequency (TinyLFU); segments with ghost lists	systems	K	closed in generation; not measured
Sticky fault; item substitution, two choices	seismology; retail, queues	R	P3: borderline
Tropical hypersurface, margin of the top-k	mathematics	R	guard closed (Chapter 12); band in P3
Maslov smoothing	mathematics	R	closed
Conformal prediction; case law	statistics; law	L	P4: open, rather fails
Olfactory circuit of the fly	neuroscience	L	variant of P4
Background traffic that yields (TCP Nice, LEDBAT)	networks	L	in P4, not measured alone
Parareal, COLA, P wave	simulation, earthquakes	L	closed (no byte)
Batched verification; erasure codes	information	L	closed
Buffer-based control (BOLA)	video	L	not measured, after P6
Fermi occupation, gate; foveation, triage, Pareto tail	physics; vision, medicine, economics	R, L	P6: open (decoding), step 0 (B)
Iterative refinement in mixed precision	scientific computing	Q, L	in P6
Spin glass, annealing, replica exchange	physics of disordered systems	Q	P5: open
Reverse water-filling by usage	rate–distortion	Q	not retained, to be grafted
Tensor networks, holography, Landauer	condensed matter	F, Q	unfavourable, closed
Seed, genome, duplication, Hebbian, spectra	biology	F, Q	closed (Chapters 4 to 6, Chapter 13)
Group sequential stopping	clinical trials	J	instrument

Table 7.16: The six leads before their judges.

Lead	Main number (judge)	Verdict of the judge	What is missing to decide
P1	LFU(32) − LRU: −0.0176 at the threshold; +0.0185 on the real answer	open (mechanical "fails")	4 generations of the 120b stopped at < | `return` | >
P2	disk share −28 and −51 %; demotion 11.5 and 27.2 % of the token	open	overlap of the demotion (engine ABBA), against the inclusive cache at $V + R$
P3	−10.03 % of misses, KL 0.0314 online	borderline	second draw (another text, another budget), in progress elsewhere
P4	precision 0.92; simulated gain +3.3 %	open, rather fails	engine ABBA with the gate θ^*
P5	descent −5.1 % of held-out error	open; mechanism not held	test of N with a fixed source; real weights (step 2)
P6	decoding $r = 0.69$; prefill $r_B = 0.04$ to 0.06	open; B holds at step 0	end-to-end KL on the 20b (step 1)

- **The figures are outside the common style**: drawn by each lead, with their own titles (among them that of Figure 7.1, which the judge corrects), their own number formatting and a panel title that runs over; they are included as they are, and the numbers read on them have been checked against the judgements, not redrawn.

Chapter 8

The synthesis of the round: a common frontier, two parts of the KL

Where we start, what this chapter establishes, what remains

At the start, four teams and four deliverables, each judged by its own measure: the scout (Chapter 3) had its frontier at 2.644 1 bits per weight and its synonymous mutation, the genome team (Chapter 4) its recipe S^*, duplication (Chapter 5) its kinship of experts, the predictive coding and pruning team (Chapter 6) its partitions, its Hebbian and its zero-synonym codon. The `.trc` v1.0 format of Volume II was bit-exact, at 3.843 4 bits per expert weight (A_flux); the engine returned the logits of the raw coding to the bit. Three KL passes existed, under three different protocols. **This chapter establishes** a common frontier of the four teams under a single judge, the Kullback-Leibler divergence on the text, as a ratio to the base of the same protocol and as a paired difference; the reason for each failure, tied to one of three measured properties of the weights; a decomposition of the KL into a smooth part, quadratic and posed here in full, and a tail of router flips; the tropical frame that makes the router's regions polyhedra; and the specification of the two construction lots, with its three statements (proposition A1, lemmas A2 and B2) proved. **What remains** is the construction itself (lot A in Chapter 9, lot B in Chapter 10, their judge in Chapter 11), the tropical guard (Chapter 12) and the spectra (Chapter 13), which this chapter names without measuring them. Sources: `genome/05-synthese.md`, `genome/construction-synthese-pour-le-memoire.md`, the files of `mesures/genome/construction/synthese/k10/` written by `outils/genome/synthese/frontiere.py --k10`, the logs of the scout's judge (`mesures/genome/etat-de-l-art/journaux/kl-*-juge.json`) and of the K10 pass (`mesures/genome/genome/kl/comparer10-base-contre-recette.json`).

In brief

Four teams tried to compress the weights of gpt-oss by looking for a structure in them: a seed, motifs, a kinship, a prediction. Placed on a single quality axis, the KL of the whole model on real text, almost all of them fall, and they fall for one of three measured reasons: once the scale of its block is known, a weight is noise; a few giant columns carry what is left of structure; and the output error of an expert does not say what that error will cost the text. The three mechanisms that win share a family likeness: they are *synonyms*. The image comes from the genetic code, where several codons give the same amino acid; it is an analogy, not a proof. Here, the sign of a zero is a codon the computation does not read, and it is folded; the class of the input column is information the decoder already has; and the synonymous mutation changes codes where the expert's inputs do not look. Why the expert error misleads: the KL has two parts. A smooth part, small and quadratic, which the error follows; and a tail, made of the positions where a small error flips the choice of experts at a later layer, and which carries most of the mean. The

> router cuts the space of states into polyhedra; the tail lives on their faces. The chapter ends with the construction specification: two lots, their criteria declared beforehand, and the three proofs on which they rest.

8.1 A single judge, and why ratios

The judge. Each team of the round had begun with its own measure: the relative error on the weights, then the output error of an expert on real inputs. Each designated winners that the next measure brought down (Section 8.3). The judge that remains is that of lot R of Volume II: the Kullback-Leibler divergence [11] of the next-token distribution given by the modified model, against the f64 reference, position by position, on the windows of wiki.test.

Definition 8.1 (The KL of a position, its mean)**.** At a position t, let p_t be the next-token distribution returned by the f64 reference and q_t that of the model under judgement. The KL of the position is $\mathrm{KL}_t = \sum_i p_{t,i} \log(p_{t,i}/q_{t,i})$ (in nats; the `juge kl` instrument only sums the i such that $\log p_{t,i} > -16$). The KL of a pass is the mean $\overline{\mathrm{KL}} = \frac{1}{n}\sum_t \mathrm{KL}_t$ over the n scored positions.

Three protocols. The three passes of the round neither replace the same experts nor score the same positions (Table 8.1). An absolute KL of P3, which replaces four times as many experts, does not compare with a KL of P1: the base ranges from 0.0206 to 0.0636 depending on the protocol.

Table 8.1: The KL protocols of the round [meas.]. The KL of the base is that of the scout's frontier substituted for the same experts; ± one naive standard deviation, that of the instrument.

Prot.	Team	Experts replaced	Positions scored	KL of the base
P0	(lossless)	—	logits identical to the bit	exactly 0
P1	scout	32 of 768 (layers 0, 8, 16, 23; experts 0, 4, …, 28)	2 550 (10 windows)	0.0360 ± 0.0029
P2	genome	8 of 768 (layers 0, 8, 16, 23; experts 0, 16)	765 (3 windows)	0.0226 ± 0.0031
P2-10	genome (K10)	the same 8	2 550 (10 windows)	0.0206 ± 0.0021
P3	predictive-pruning	128 of 768 (the 12 whole tensors)	1 275 (windows 5 to 9)	0.0636 ± 0.0050

Data: `mesures/genome/construction/synthese/k10/frontiere.csv` (columns `kl_base_protocole`, `kl_incertitude`); `genome/05-synthese.md` § 1.1.

What compares from one protocol to another is the *ratio* to the base of the same protocol; what compares within a protocol, better still, is the *paired difference*.

Definition 8.2 (Paired difference, rule of lot R)**.** For two models A (the base) and B judged at the same positions against the same truth, let $d_t = \mathrm{KL}_t^B - \mathrm{KL}_t^A$ and $\bar{d}$ its mean. With $\sigma(\bar{d})$ the uncertainty of $\bar{d}$, B is *better* if $\bar{d} + 2\sigma(\bar{d}) < 0$, *equal* if $|\bar{d}| \le 2\sigma(\bar{d})$, *worse* otherwise.

Two uncertainties for $\bar{d}$. The naive estimator treats the n positions as independent: $\sigma_{\text{naive}} = s_d/\sqrt{n}$, with s_d the empirical standard deviation of the d_t. But the positions of one window share their past, hence their routing flips (Section 8.4): their d_t are correlated. The second estimator, which

`frontiere.py` computes, takes the window as the unit. With W windows of equal size and $\bar{d}_w$ the mean of window w,

$$\bar{d} = \frac{1}{W}\sum_{w=1}^{W} \bar{d}_w, \qquad \sigma_{\text{win}}(\bar{d}) = \frac{1}{\sqrt{W}}\sqrt{\frac{1}{W-1}\sum_{w=1}^{W}(\bar{d}_w - \bar{d})^2}. \tag{8.1}$$

The first equality holds because the windows all have 255 scored positions. If the positions of a window were independent, the two estimators would have the same expectation; when they are positively correlated, σ_{win} is larger, and it is the one needed. Its limit: $W = 10$ blocks, that is 9 degrees of freedom (Section 8.8).

The scout's paired difference, computed here. The scout had compared two means without pairing them (Chapter 3). Its per-position CSVs are committed; `frontiere.py` pairs them [calc.] on [meas.].

- *Control*, declared before execution: the mean KL recomputed from each CSV equals that of `juge kl` to within 10^{-12}. It is 0.036 013 763 948 on both sides for the base and 0.026 168 395 170 for the synonymous mutation; the control holds. Break: removing the first line gives 0.036 021 222 instead of 0.036 013 764, and the control falls (`frontiere-casse.log`).
- *Difference* (synonym minus base): $\bar{d} = -0.009845$; $\sigma_{\text{naive}} = 0.00190$ ($5.2\,\sigma$); $\sigma_{\text{win}} = 0.00334$ ($2.9\,\sigma$). Significantly better by both estimators.
- 2 060 positions out of 2 550 (80.8 %) have less KL with the synonymous mutation; the identical first choice gains $+3.73 \pm 0.59$ points.

The ratio of the two standard deviations, $0.00334/0.00190 = 1.76$ [calc.], measures the correlation within a window: it is 1 for independent positions. It says why, in everything that follows, the window uncertainty is the one that judges.

The rates do not all have the same estimator. Real codec decoded (P0, P3, the frontier of P2), entropy of the symbols without scales H_0 (P1), full accounting of the strand (the genome): the `estimateur_bits` column of the source says it for each point. Within a protocol, base and mechanism have the same one. For the frontier, the real codec returns 2.644 1 bits; H_0 plus the scales give 2.679, because they ignore the context κ that the codec exploits; H_0 alone gives 2.641 5, because it ignores the scales (0.037 5 bit). The two omissions almost cancel [calc.] (`genome/05-synthese.md` § 1.1).

8.2 The common frontier

Table 8.2 gives the points judged at the KL; Figure 8.1 places them. The file `k10/frontiere.csv` counts 25: 12 judged at the KL under P1, P2-10 and P3; the 2 of the P2 pass at 3 windows, which K10 replaces; 7 judged only at an intermediate metric (weight error or expert output error); and the 4 lossless points, whose KL is zero by construction.

What the figure shows. A single mechanism of the round passes significantly below its base: the synonymous mutation $\lambda = 0$, of the scout, at ×0.727 of the KL of its base for 0.011 bit more. Everything else is equal or above. The dashed arrows of the figure start from the *promise* of each loser (the ratio 1 that its intermediate metric announced) and go to the KL it returned: they measure the distance between what the weight or expert error promised and what the text returned. For the Hebbian by e, tuned to have exactly the expert error of the base, the arrow goes from 1 to 3.06.

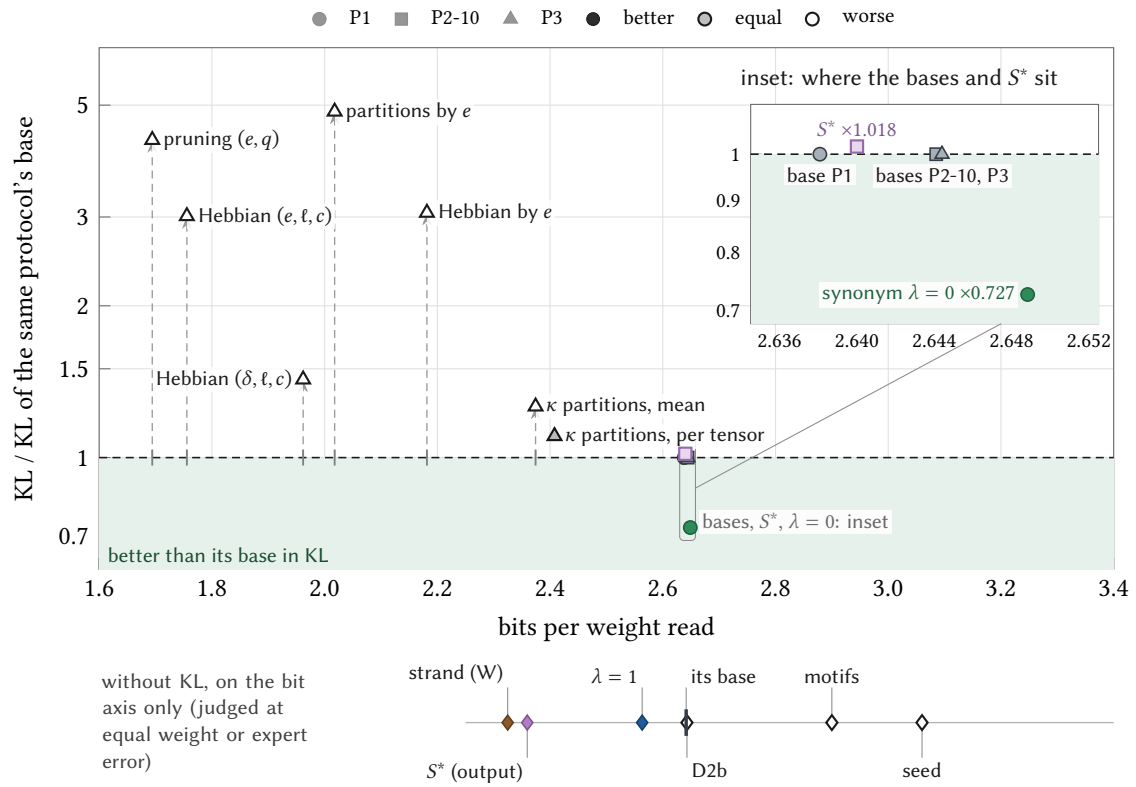


Figure 8.1: Judged by the KL on text, a single mechanism of the round passes under its base: the synonymous mutation $\lambda = 0$. [meas.] [calc.] Each mechanism is placed by its bits per weight and by its KL as a ratio to the base of its own protocol (the protocols differ in substituted experts and windows; an absolute KL from one does not compare with another). Shape gives the protocol (P1: scout, 32 experts; P2-10: genome, 8 experts, 10 windows; P3: predictive-pruning, 12 tensors), colour the variant (green: synonym $\lambda = 0$; violet: S^*; grey: the bases; ink: predictive-pruning), filling the verdict of the declared rule, applied by the script to the paired difference $\bar{d} \pm \sigma$ against the base: better if $\bar{d} + 2\sigma < 0$, equal if $|\bar{d}| \leq 2\sigma$, worse otherwise. The synonymous mutation $\lambda = 0$ is better (×0.73; $\bar{d} = -0.0098 \pm 0.0033$, $-2.95\,\sigma$), at +0.011 bit. The κ partitions per tensor give a KL equal by the rule (×1.103; $+0.0066 \pm 0.0053$, $+1.24\,\sigma$) for 0.911× the bits: the `verdict` column of the source calls them a win, the rule calls them equal, and the figure follows the rule. Two losers have no paired difference (partitions by e, pruning (e, q)): their verdict is the source's, from the ratio (×4.85 and ×4.26). The dashed arrows go from each loser's *promise* (the ratio 1 that its intermediate metric announced, measured or only predicted) to the KL it actually got. S^* is drawn from the 10-window pass K10 (×1.018, paired $+0.0004 \pm 0.0019$), not from the 3-window pass (reserve R4); it hides under the bases, which the inset pulls apart. Below, diamonds: the 7 points without KL, on the bit axis only, filled if they won at their own metric (weight or expert output error), hollow if they fell there (the grey bar: the base of $\lambda = 1$); $\lambda = 1$ is judged there under protocol P1-outputs (16 of the 32 experts of P1, its own base at 2.6415 bits); its KL measured later on 768 experts is another protocol, not drawn here (reserve R3). What the figure does not show: the lossless axis (KL = 0: frontier 3.8434, zero codon 3.7064, column context 3.6844, duplication D1 3.8442, which falls), drawn tensor by tensor in Chapter 9; nor the bit estimators, which differ between protocols (real codec, H_0 of symbols, full strand accounting: column `estimateur_bits` of the source).

Data: `mesures/genome/construction/synthese/k10/frontiere.csv` (`outils/genome/synthese/frontiere.py --k10`), `mesures/genome/etat-de-l-art/passes/synonyme-lam1-damp1-e0_8_16_24.json`; script `memoire/outils/tome3_synthese.py`.

Table 8.2: The common frontier: the points judged at the KL, each against the base of its protocol [meas.] [calc.]. Bits per weight: the protocol's estimator; ratio = $\overline{\mathrm{KL}}/\overline{\mathrm{KL}}_{\mathrm{base}}$; paired: $\bar{d} \pm \sigma$ (Definition 8.2); verdict: the rule, applied by the script.

Mechanism	Prot.	Bits	KL	Ratio	Paired	Same 1st	Verdict
frontier base	P1	2.6381	0.0360	1	—	87.1 %	reference
synonymous mutation $\lambda = 0$	P1	2.6488	**0.0262**	**0.727**	**−0.0098 ± 0.0033**	90.9 %	**better**
frontier base	P2	2.6441	0.0226	1	—	91.4 %	reference
S^*, 3 windows	P2	2.64	0.0301	~~1.335~~	+0.0076 ± 0.0047	90.3 %	equal; replaced by P2-10
frontier base	P2-10	2.6441	0.0206	1	—	91.9 %	reference
S^*, 10 windows	P2-10	2.64	0.0209	1.018	+0.0004 ± 0.0019	91.9 %	equal
frontier base	P3	2.6444	0.0636	1	—	81.6 %	reference
partitions by κ, tensor by tensor	P3	2.4079	0.0702	1.103	+0.0066 ± 0.0053	79.5 %	equal
partitions by κ, equal mean error	P3	2.3743	0.0804	1.263	+0.0167 ± 0.0049	78.4 %	worse
Hebbian (δ, ℓ, c)	P3	1.9619	0.0909	1.429	+0.0273 ± 0.0039	73.4 %	worse
Hebbian by e	P3	2.1815	0.1947	3.059	+0.1310 ± 0.0072	60.8 %	worse
Hebbian (e, ℓ, c)	P3	1.7556	0.1917	3.012	+0.1281 ± 0.0067	59.4 %	worse
pruning by activity (e, q)	P3	1.6942	0.2714	4.265	—	57.1 %	falls (ratio)
partitions by e	P3	2.0178	0.3089	4.854	—	58.5 %	falls (ratio)

Data: `mesures/genome/construction/synthese/k10/frontiere.csv`; P1: `mesures/genome/etat-de-l-art/journaux/kl-*-juge.json`; P2-10: `mesures/genome/genome/kl/comparer10-base-contre-recette.json`; P3: `mesures/genome/predictif-elagage/resume-12.csv` and `kl/comparer-*.json`.

Two corrections, stated. The first concerns the partitions by κ tensor by tensor. The `verdict` column of the source calls them "winners: 0.911 × the bits, KL equal by the rule"; the rule of Definition 8.2 calls them *equal* in KL (+0.0066 ± 0.0053, 1.2 σ), at 0.911 times the bits of the base. The figure follows the rule, applied by its script; the first judge of the figures had flagged it as serious (G1, `memoire/tome3/JUGEMENT-figures-ronde.md`), and the second declared it lifted. "Equal in KL for 9 % fewer bits" is a real rate advance at equal quality; it is not a quality gain. The second concerns the variant of the synonymous mutation that carries the KL. The brief of the synthesis attributed the KL 0.0262 to the variant $\lambda = 1$: ~~$\lambda = 1$: KL 0.0262 against 0.0360~~. The log (`kl-synonyme-d1-juge.json`: "d1" for $\delta^* = 1$, $\lambda = 0$) says that the KL was measured on $\lambda = 0$, at 2.649 bits (+0.011 over the frontier); $\lambda = 1$ (2.563 4 bits against 2.641 5 for the frontier on the same experts) had only been judged at the expert output error, 8 cells out of 8. At the end of the round, therefore, *no variant was measured both below the frontier in bits and below it in KL*; this is measurement B0 of the specification (Section 8.7.2).

What the figure does not show. It keeps the protocol at 32 experts out of 768: the KL of $\lambda = 1$ measured later on the 768 experts (Chapter 10) is another protocol, and no line links Figure 8.1 to the points at 768 experts of Chapter 10 (reserve R3). S^* is placed in it by the K10 pass at 10 windows, not by the pass at 3 windows (reserve R4); it hides beneath the bases, and the inset separates them. The lossless axis (zero KL) is no longer in this figure: it is drawn tensor by tensor in Chapter 9 (Figure 9.6). Its three points, means over the 12 reference tensors, are 3.843 4 bits (lossless frontier), 3.706 4 (zero-synonym codon) and 3.684 4 (with the column context); duplication D1, at 3.844 2, falls [meas.] (`k10/frontiere.csv`).

The points without KL. Judged at a weight or expert output error equal to the base, they can only be placed on the bits axis: the seed and its coefficients (3.06 bits), the shared motifs (2.90), duplication D2b (2.642 9), the strand rotated at criterion W (2.324 8), S^* at equal expert output error (2.359 5), and the synonymous mutation $\lambda = 1$ (2.563 4). The strand and S^* are filled in the figure: they won at their metric; the KL brought them down or made them equal. That is the subject of the next section.

8.3 Three measured properties of the weights

Everything that falls in the round falls for one of three properties, each measured by at least two teams and by two paths.

8.3.1 Once the block scale is known, the weights are noise

Table 8.3: Property (a): once the block scale is known, almost nothing is left to predict [meas.].

Measurement	Value	Chapter
information brought by the neighbouring code, given the scale	0.005 bit (left 0.0076, above 0.0091)	Chapter 6
information brought by the neighbouring expert, same place	0.0043 bit	Chapter 6
cosine of the rows of two paired experts (Hungarian algorithm)	0.04 (gate), 0.03 (up), 0.07 to 0.11 (down); chance 0.04	Chapter 5
error of the strand on the rotated rows, against a true Gaussian	0.0739 against 0.0731 (within 1.5 %); kurtosis 3.0	Chapter 4
the seed against the lemma of the Beta law	above the prediction by 0.6 to 2 points	Chapter 4
sign of the zeros	0.9999999967 bit of entropy, no effect on the value	Chapter 3, Chapter 9

Data: `genome/05-synthese.md` § 2.1 (a), which refers to the deliverables of each team.

The seed, the shared motifs, duplication and the recipe with exact complement all sought a dependence between pieces of the tensor. Table 8.3 says there is none at block scale: given the scale, a code says almost nothing about its neighbour or about the neighbouring expert, and a rotated row follows a Gaussian to within 1.5 %. A single "structure" of this kind can be exploited, and it is the opposite of a structure: the sign of a zero, which the computation does not read, is a fair coin. Merging it removes exactly one bit per zero of minority sign (Chapter 9, Section 9.2).

8.3.2 A few giant columns carry the structure

The last two lines say the same thing by two independent paths. What the experts of a layer share is not a kinship from expert to expert: it is *a statistic per input column*, common to the 32 experts because it comes from their common input, the hidden state and its outlier channels at layer 0. Duplication saw it as a "random kinship" (a parent drawn at random returns 88 to 94 % of the gain of a true parent at layer 0); predictive coding took it as a context, and it is the only context that passed its threshold.

A precision on the column figure. The file gives, for blk.0 gate, 3.607 307 bits with the zero folded and the byte context κ, 3.611 090 with the nibble alphabet and κ, and 3.465 690 with the column. Against the real stream of the format (the byte), the column gain is $3.607307 - 3.465690 = 0.1416$

Table 8.4: Property (b): the structure that remains is concentrated in a few columns and a few directions [meas.].

Measurement	Value	Chapter
energy of the blocks of exponent 129 and 130	74.2 % of the energy for 5.8 % of the weights	Chapter 3
energy of the first 5 % singular directions	23 to 68 % (17 to 19 % for the permuted matrix); kurtosis of the rows up to 348	Chapter 4
most massive column (blk.8 down, expert 16, column 1011)	×144 the mean energy of a column; receives ×334 the mean error of the strand	Chapter 4
share of the pooled error of the frontier held by a single tensor	91.5 % (blk.23 down)	Chapter 6
gain of the column context, per tensor	−0.1416 and −0.1231 bit (layer 0, gate and up); below 0.01 elsewhere	Chapter 9
share of a parent's gain returned by a random parent	46 % on average, 88 to 94 % at layer 0	Chapter 5

Data: `genome/05-synthese.md` § 2.1 (b); column gain: `mesures/genome/construction/synthese/k10/colonne-par-tenseur.csv`, columns `zero_octet_kappa` minus `zero_demi_octet_kappa_colonne`.

bit [calc.]; against the nibble stream, 0.1454. The synthesis wrote "−0.142, −0.128" mixing the two references for gate and up; Chapter 9 and this chapter keep the first, 0.141 6 and 0.123 1, which is that of the decision of lot A.

The giant column, seen by three reconstructions. Figure 8.2 takes one matrix, layer 8, down, expert 16 of the 20b, and its 2 880 columns in natural order. Column 1011 carries ×143.9 the mean energy. The weighted strand, which rotates each row by a Hadamard transform of 64 before reading it by codons, sends it ×334 its mean error: the rotation spreads the giant value into 64 large values per row, the codon table clips them, and these clipping errors, of the same sign in all rows, come back onto the column through the inverse rotation. The frontier base, which keeps the largest codes of each block exact, puts only ×14.0 its mean error there; S^*, which sets aside the three largest weights of each row, brings it down to ×1.05. A second spike, column 960, first channel of the rotation chunk of the giant (960 to 1023), of ordinary energy (×0.83), receives ×310 the mean error of the strand; its cause is not measured, and the figure's drawing stops if a script moved it (the judge of the third pass tested this: `memoire/tome3/JUGEMENT-figures3.md` § 4.4) [meas.] (`synthese-nombres.csv`).

This is property (b) in its sharpest form: a small mean weight error can hide an enormous error on a column that the inputs excite strongly. Any mechanism that clips the massive columns, or drowns them in an average, loses at the expert's output.

8.3.3 The output error of an expert does not predict the KL

Three points have an expert error ratio in [0.75; 1.02] and KL ratios ranging from 1.02 to 3.06 (Figure 8.3). S^* removes 25 % of the expert error and nothing of the KL; the only KL winner divides the expert error by nearly four for a KL gain of 27 %. The relation is neither monotone nor proportional. The figure also shows the correction of S^*: its ratio at 3 windows, ~~1.335, "33 % more KL"~~, is drawn hollow and struck through, with an arrow to the ratio of the K10 pass, 1.018, "the same KL as the base over 10 windows (+0.0004 ± 0.0019)". What changed: three times as many positions; the gap of the first three windows was in the noise (Section 8.5.1).

Why the expert error misleads is the next section. The figure to retain from here: at equal expert

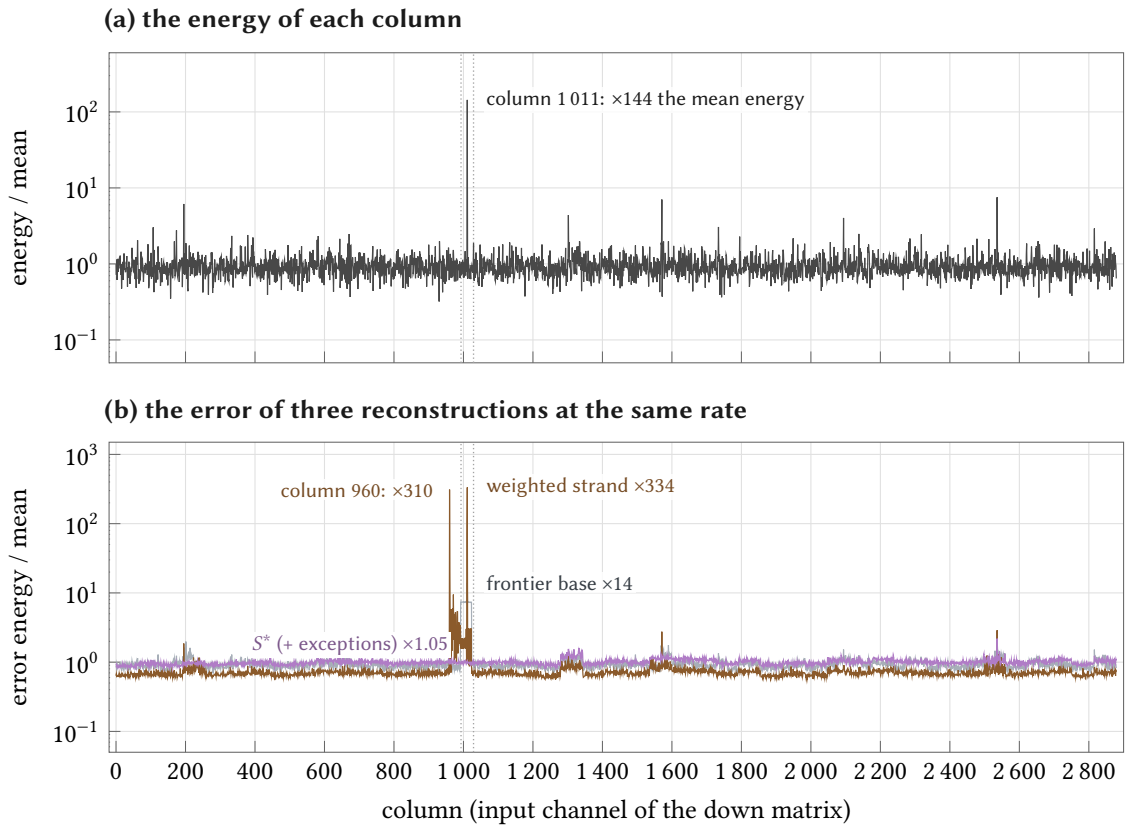


Figure 8.2: A giant column, and the first channel of its rotation block, draw the strand's error; the base contains it, S^* removes it. [meas.] One matrix: layer 8, down, expert 16 of gpt-oss-20b, its 2 880 columns in natural order. (a) The energy $\sum_r w_{rc}^2$ of each column, divided by the mean over the columns. (b) The error energy $\sum_r (\hat{w}_{rc} - w_{rc})^2$ of three reconstructions at the same rate (2.64 bits), each divided by its own mean: the frontier base (grey-blue), the weighted strand (brown), S^* (violet). Column 1 011 (between dotted lines) carries ×144 the mean energy and receives ×334 the strand's mean error: the rotation spreads it into 64 large values per row, the table clips them, and the clipping errors, of the same sign in every row, return onto the column through the inverse rotation. The frontier base keeps the largest weights of each block nearly exact (×14); S^* keeps the three largest of each row aside, exact, and brings the column back to ×1.05. The second peak is column 960, the first channel of the giant's rotation block (960 to 1 023), of ordinary energy (×0.83): it receives ×310 the strand's mean error, against ×0.87 for the base and ×1.04 for S^*; its cause is not measured. The 62 other columns of the block receive ×4.0 the mean, and the 44 other block edges (columns $\equiv 0 \pmod{64}$) ×0.70 (at most ×0.87): the "edges ×7.6" of the source are a mean that this one column carries. What the figure does not show: the other tensors (one matrix only is in CSV; the column classes of blk.0 gate remain to be drawn), nor the cost of the exceptions (counted in the rate of S^*).

Data: `mesures/genome/genome/csv/colonnes-energie-erreur.csv` (`outils/genome/recette/figures_transformations.py`); script `memoire/outils/tome3_synthese.py`.

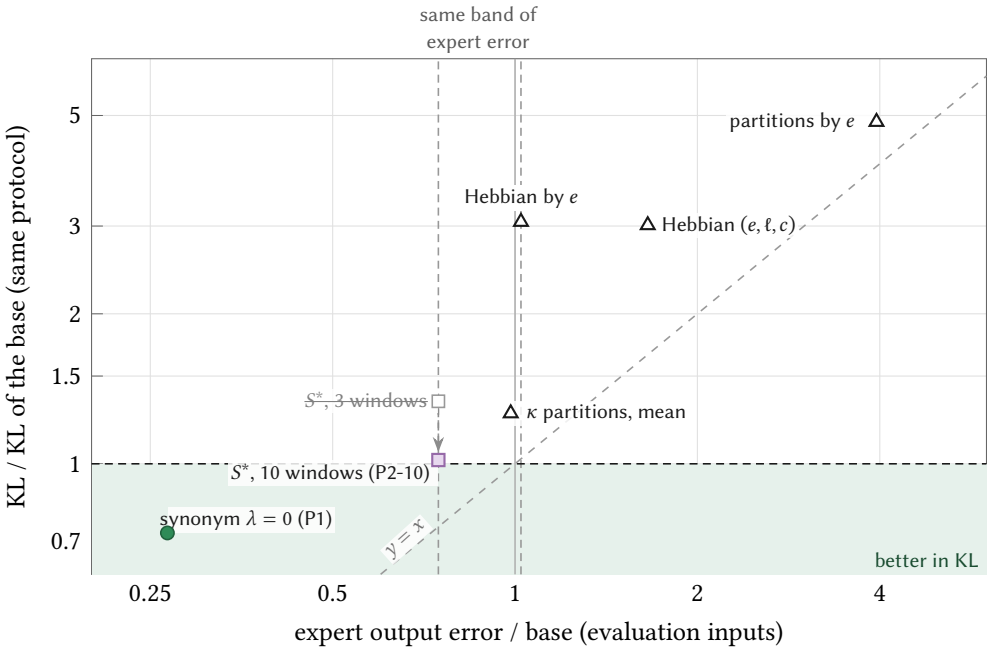


Figure 8.3: The expert's output error does not predict the KL: inside one band of expert error, the KL goes from equal to several times the base. [meas.] Each point is a mechanism judged both ways, as ratios to the base of its own protocol: horizontally, the output error of the substituted experts on held-out inputs; vertically, the KL of the whole model on text. Inside the same band of expert error (0.75 to 1.02), the KL goes from ×1.02 to ×3.06 the base; only the synonymous mutation $\lambda = 0$ passes under 1. S^* is placed by the 10-window pass K10 (×1.018); its 3-window ratio, struck out in the synthesis (05 § 2), is shown hollow and struck, with the arrow of the correction (reserve R4). Shapes, colours and fillings as in the common frontier: the shape gives the protocol (P1 scout, P2-10 genome, P3 predictive-pruning), the filling the verdict of the same rule. What the figure does not show: the uncertainty of each ratio (paired differences in the frontier's source), nor the why. The synthesis reads the KL as a smooth part, which expert error follows, and a tail of routing flips, which it does not see (next figure); this is its working hypothesis, supported by imposing the routing (Volume II, batch R).

Data: `mesures/genome/construction/synthese/k10/erreur-contre-kl.csv` (`frontiere.py --k10`, `mesures/genome/genome/kl/comparer10-base-contre-recette.json`); script `memoire/outils/tome3_synthese.py`.

Table 8.5: Property (c): the ratio of expert output error to the base, against the KL ratio [meas.] [calc.].

Mechanism	Prot.	Expert error	KL	Verdict at the KL
synonymous mutation $\lambda = 0$	P1	0.267	0.727	better
S^*	P2; P2-10	0.747	~~1.335~~; 1.018	equal
partitions by κ, equal mean error	P3	0.984	1.263	worse
Hebbian by e	P3	1.023	3.059	worse
Hebbian (e, ℓ, c)	P3	1.656	3.012	worse
partitions by e	P3	3.951	4.854	falls

Data: `mesures/genome/construction/synthese/k10/erreur-contre-kl.csv`; S^* at 3 windows struck through, replaced by K10 (reserve R4).

error, the KL can triple.

8.4 Two parts of the KL

8.4.1 What the quantiles say, and the engine

Table 8.6: The quantiles of the per-position KL [meas.]. The median tells the smooth part; the 99.9th percentile and the maximum tell the tail.

	base P1	synonym P1	base P2	S^* P2	base P2-10	S^* P2-10
mean	0.0360	0.0262	0.0226	0.0301	0.0206	0.0209
median	0.0088	0.0038 (÷2.3)	0.00172	~~0.00188~~	0.00184	0.00152 (÷1.21)
99th percentile	0.495	0.410	0.437	~~0.550~~	0.411	0.338
99.9th percentile	1.59	1.70	0.899	~~1.43~~	1.37	1.59
maximum	4.17	3.94	1.20	~~2.79~~	3.01	3.22

Data: P1: `mesures/genome/etat-de-l-art/journaux/kl-frontiere-juge.json`, `kl-synonyme-d1-juge.json`; P2: `genome/05-synthese.md` § 2.2 (`comparer-base-contre-recette.json`); P2-10: `mesures/genome/genome/kl/comparer10-base-contre-recette.json`. The columns of S^* at 3 windows are struck through by the synthesis itself: the K10 pass replaces them.

The synonym reduces the median by a factor of 2.3 without touching the tail: the 99.9th percentile even goes from 1.59 to 1.70. The mean only falls by a factor of 1.38 [calc.] (0.0360/0.0262). The same reading holds for S^* over 10 windows: the median falls (÷1.21), the 99th percentile too (0.411 → 0.338), but the extreme of the tail rises (1.37 → 1.59 at the 99.9th percentile, 3.01 → 3.22 at the maximum), and the mean does not move. Over 3 windows, ~~*S^* keeps the median (×1.09) and thickens the tail (99th 0.437 → 0.550, maximum 1.20 → 2.79)*~~: the tail won by sampling chance.

What the engine says, between two exact computations. Lot R of Volume II compared two exact engines with each other (read in `genome/05-synthese.md` § 2.2, which quotes lot R, §§ 2.9 and 2.10) [meas.]: 5 087 routing flips over 122 640 (layer, position) pairs, that is 4.15 %; the 45 % of positions that flip themselves carry 96.5 % of the KL; the router margin, fourth logit minus fifth, has a median of 0.083 over all pairs and 0.0095 over those that flip. And the proof by intervention, declared

before its pass: imposing on one of the two computations the experts chosen by the other removes 88 % of the KL ($0.0145 \to 0.00168$; median $9.8 \cdot 10^{-4} \to 2.2 \cdot 10^{-4}$; first choice $92.5 \to 97.2$ %).

8.4.2 The smooth part: the KL at second order

A perturbation δh of the hidden state, coming from the approximated experts, changes the output logits z by a quantity δz. If no router flips, δz is a smooth function of δh, and the resulting KL is small and quadratic. This is a classical computation of information theory [2]; it is posed here in full, because the whole chapter rests on its form.

Proposition 8.3 (The KL at second order in the logits)**.** *Let $z \in \mathbb{R}^V$ be logits, $p = \mathrm{softmax}(z)$, $\delta \in \mathbb{R}^V$ a perturbation and $q = \mathrm{softmax}(z + \delta)$. Then*

$$\mathrm{KL}(p \,\|\, q) = \tfrac{1}{2}\, \delta^\top F \delta + R, \qquad F = \mathrm{diag}(p) - p\, p^\top, \qquad |R| \le \frac{(\max_i \delta_i - \min_i \delta_i)^3}{24}. \tag{8.2}$$

The quadratic form equals $\delta^\top F \delta = \mathrm{Var}_{I\sim p}(\delta_I)$: the variance of the perturbation under the model's distribution. F is the Fisher information of the categorical distribution parametrised by its logits.

Proof. Step 1: the KL as the gap of a convex function. Write $A(z) = \log \sum_j e^{z_j}$ (the log-partition). Then $\log p_i = z_i - A(z)$ and $\log q_i = z_i + \delta_i - A(z+\delta)$, hence

$$\mathrm{KL}(p\|q) = \sum_i p_i(\log p_i - \log q_i) = \sum_i p_i\big(A(z+\delta) - A(z) - \delta_i\big) = A(z+\delta) - A(z) - p^\top \delta,$$

since $\sum_i p_i = 1$.

Step 2: the derivatives of A. $\partial A/\partial z_i = e^{z_i}/\sum_j e^{z_j} = p_i$, so $\nabla A(z) = p$. Then $\partial p_i/\partial z_j = p_i(\mathbb{1}[i=j] - p_j)$, so $\nabla^2 A(z) = \mathrm{diag}(p) - pp^\top = F$. More generally, along a line $z + \theta\delta$, the successive derivatives of $\theta \mapsto A(z+\theta\delta)$ are the cumulants of the variable $X = \delta_I$, $I \sim p_\theta = \mathrm{softmax}(z+\theta\delta)$: the first is $\mathbb{E}_{p_\theta} X$, the second $\mathrm{Var}_{p_\theta} X$, the third $\mathbb{E}_{p_\theta}(X - \mathbb{E}X)^3$.

Step 3: Taylor with Lagrange remainder. There exists $\theta \in (0,1)$ such that

$$A(z+\delta) = A(z) + p^\top\delta + \tfrac{1}{2}\delta^\top F \delta + \tfrac{1}{6}\,\mathbb{E}_{p_\theta}(X - \mathbb{E}_{p_\theta} X)^3.$$

Carried into step 1, the first-order term cancels exactly: $p^\top\delta$, the expectation of δ_I under p, subtracts itself. What remains is $\mathrm{KL} = \frac{1}{2}\delta^\top F\delta + R$ with $R = \frac{1}{6}\mathbb{E}_{p_\theta}(X - \mathbb{E}X)^3$.

Step 4: the bound on the remainder. X takes its values in $[a,b]$, $a = \min_i \delta_i$, $b = \max_i \delta_i$. Then $|X - \mathbb{E}X| \le b - a$, so $|\mathbb{E}(X - \mathbb{E}X)^3| \le (b-a)\,\mathrm{Var}\, X$, and the variance of a variable bounded in $[a,b]$ is at most $(b-a)^2/4$. Hence $|R| \le \frac{1}{6}(b-a)^3/4 = (b-a)^3/24$.

Step 5: the variance. $\delta^\top F\delta = \sum_i p_i\delta_i^2 - (\sum_i p_i\delta_i)^2 = \mathrm{Var}_p(\delta_I)$. It is invariant if a constant is added to all the δ_i, like the softmax itself. ∎

Remark 8.4 (What the proposition says about the smooth part). If the output logits depend linearly on the final state through the head W_o, $\delta z = W_o \delta h_L$, the smooth part is $\frac{1}{2}\delta h_L^\top W_o^\top F W_o \delta h_L$: quadratic in the perturbation, and weighted by the model's distribution. An expert error twice as small gives, as long as nothing flips, a smooth part four times as small, and only the component of the error that $W_o^\top F W_o$ sees counts. This is why the synonymous mutation, which weights its error by the real inputs of each expert ($H = \mathbb{E}[g^2 x x^\top]$, Chapter 10), reduces the median: it puts the error away in the directions that the inputs do not excite.

8.4.3 The flip part

The proposition only holds if the set of chosen experts does not change. But the router of a layer ℓ keeps the $k = 4$ experts with the largest logits. If δh pushes the gap between the fourth and the fifth beyond its margin $m_\ell = z_{(4)} - z_{(5)}$, the set changes, and the output of the layer jumps by a quantity that no longer depends on $\|\delta h\|$, but on the difference between two experts. And two experts of the same layer do not resemble each other: serving an expert by its best paralogue leaves 33 to 84 % of the energy of its output in error depending on the layer (Chapter 5).

Assumption 8.5 (The form of the flip part)**.** Writing δs_ℓ for the perturbation of the gap between the two scores at stake at layer ℓ,

$$\mathbb{E}[\mathrm{KL}] \;\approx\; \mathbb{E}\Big[\tfrac{1}{2}\,\delta z^\top F\,\delta z\Big] \;+\; \sum_\ell \mathbb{P}\big(m_\ell < |\delta s_\ell|\big)\,\mathbb{E}\big[\mathrm{KL} \mid \text{flip at } \ell\big].$$

The second term depends on $\|\delta h\|$ only through a probability; its conditional value depends on the difference between two experts, not on the size of the error.

This is a working hypothesis, not a proposition: it gives the form, not the terms. It rests on three measured facts: the intervention of lot R (imposing the routing removes 88 % of the KL between two exact engines), the distribution of the KL (96.5 % on the positions that flip), and the quantiles of the round (Table 8.6). It has not been measured *for the mechanisms of the round* by imposed routing: the measurement that would judge it, the KL pass of S^* and of the base with the experts imposed from the reference, has not been done (Section 8.8). It explains what property (c) observes: the expert error drives the first term, and almost nothing of the second, which carries the mean.

Two parts, position by position. Figure 8.4 turns each of the 2 550 positions of protocol P1 into a point: the KL of the base on the abscissa, that of the synonymous mutation on the ordinate, same positions and same truth. The mass passes below the diagonal: the median is divided by 2.3, and 2 060 positions are won. The tail keeps its shape: the 25 largest positions of each (the "1 %") carry 31 % of the mean KL of the base and 38 % of that of the synonym, so that the mean only falls by ×1.38. It does not keep its positions: only 10 of the 25 are common to the two tails. This is what assumption 8.5 predicts: a flip depends on a small margin and on a perturbation of given sign; when the weights change, the perturbations change, and the positions that flip change with them. The tail of the base is spread over 9 of the 10 windows, at most 7 in a single one: flips cluster by window, through their common past, which justifies the window uncertainty of Equation (8.1) [meas.] (`synthese-nombres.csv`). The figure does not redo the intervention; these logs have no routing column, and it does not say which positions flipped.

8.4.4 The router as tropical geometry

Assumption 8.5 gives the flips the leading role; one must then know where they take place. The router of a layer computes $z(h) = W_r h + b_r$ (32 logits for the 20b, 128 for the 120b) and keeps the $k = 4$ largest; h is the normalised state that enters the experts.

Proposition 8.6 (The router's regions are polyhedra)**.** *For every set S of k experts, the region $\mathcal{R}_S = \{h : z_i(h) \geq z_j(h)\ \forall i \in S, j \notin S\}$ is a convex polyhedron. The function $\Phi(h) = \sum_{i \in S(h)} z_i(h)$, the sum of the k largest logits, is convex and piecewise linear; it is a tropical polynomial of the* $(\max, +)$ *semiring, and the faces of its pieces lie on the hyperplanes $z_i = z_j$.*

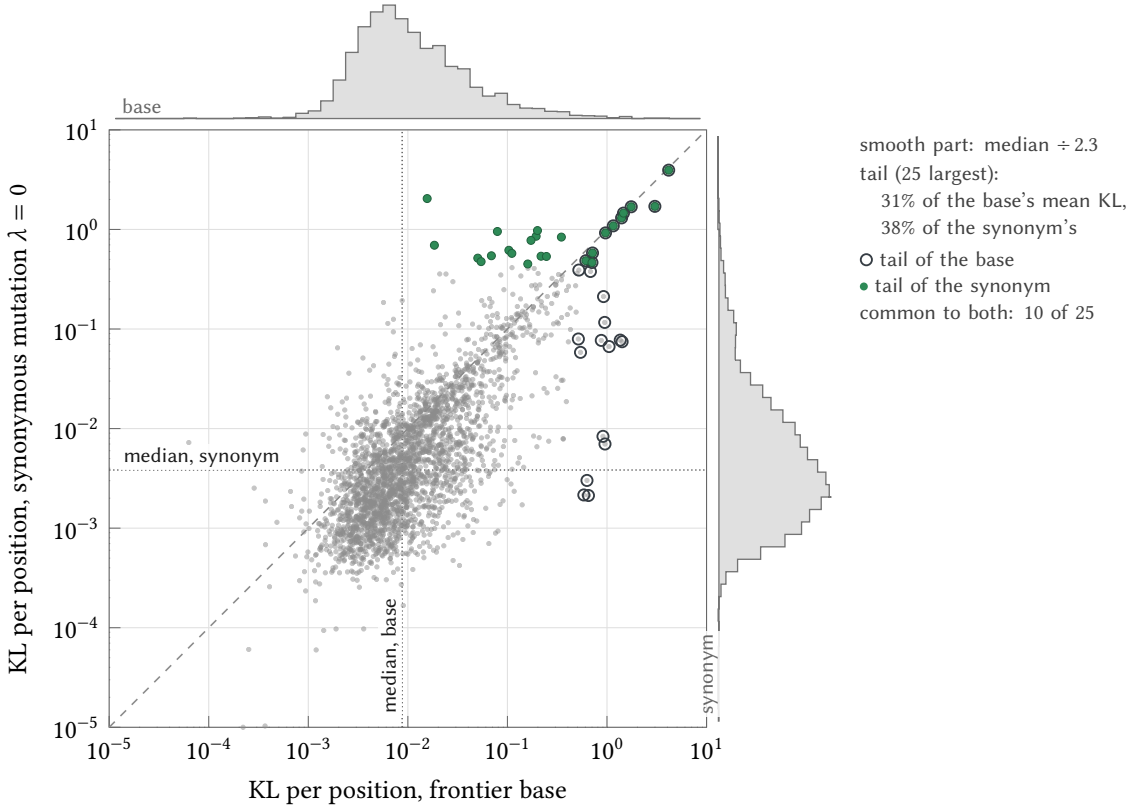


Figure 8.4: The KL has two parts: its bulk passes under the diagonal; its tail keeps its shape, not its positions. [meas.] [calc.] The 2 550 positions of the scout's KL pass (10 windows of wiki.test, 32 experts of 768 substituted: protocol P1), the frontier base horizontally, the synonymous mutation $\lambda = 0$ vertically, same positions and same truth; marginal densities on the margins. The bulk passes under the diagonal: the median is divided by 2.3 (0.0088 to 0.0038), and the synonym has less KL on 2 060 positions. The tail keeps its shape: the 25 largest positions of each (the "1%", $\lfloor 2\,550/100 \rfloor$, that is 0.98%) carry 31% of the base's mean KL and 38% of the synonym's, so the mean only falls by ×1.38 and the 99.9th percentile rises (1.59 to 1.70). It does not keep its positions: only 10 of the 25 are common to both tails (hollow rings: the base's tail; green discs: the synonym's). The base's tail spreads over 9 of the 10 windows, at most 7 in one. The synthesis reads the bulk as the smooth part, which expert error follows, and the tail as routing flips, which move when the weights move; this is its working hypothesis, supported by the intervention that imposes the routing (Volume II, batch R, experiment B), which this figure does not repeat. The only KL below 10^{-5} is negative ($-6.9 \cdot 10^{-6}$, rounding noise) and is drawn at that floor. What the figure does not show: which positions flipped an expert (these logs have no routing column), nor the other 736 experts, left exact.

Data: `mesures/genome/etat-de-l-art/journaux/kl-frontiere-positions.csv`, `kl-synonyme-d1-positions.csv`, `kl-frontiere-juge.json`, `kl-synonyme-d1-juge.json`; script `memoire/outils/tome3_synthese.py`.

Proof. Each condition $z_i(h) - z_j(h) \geq 0$ is written $(w_i - w_j)^\top h + (b_i - b_j) \geq 0$: a closed half-space. $\mathscr{R}_S$ is their finite intersection, hence a convex polyhedron. For Φ: the sum of the k largest numbers of a list is the maximum, over the subsets T of size k, of $\sum_{i\in T} z_i$; so $\Phi(h) = \max_{|T|=k} \big(\sum_{i\in T} w_i\big)^\top h + \sum_{i\in T} b_i$, a finite maximum of affine functions, convex and piecewise linear. In the semiring where addition is max and multiplication is +, it is a tropical polynomial in the coordinates of h, with monomials indexed by T. Two pieces T, T' join where $\sum_T z = \sum_{T'} z$; for two neighbours that differ only by an exchange $i \leftrightarrow j$, this is the hyperplane $z_i = z_j$. ∎

The link between piecewise linear networks and tropical geometry is that of Zhang, Naitzat, and Lim [115]: ReLU networks are tropical rational maps, and their decision boundaries are tropical hypersurfaces [lit.]. The top-k router is not a ReLU; it falls under the same (max, +) algebra.

Proposition 8.7 (The margin and the distance)**.** *Let $h \in \mathscr{R}_S$, and let* (4) *and* (5) *be the indices of the fourth and fifth logits, with margin $m(h) = z_{(4)}(h) - z_{(5)}(h) \geq 0$. The Euclidean distance from h to the hyperplane $z_{(4)} = z_{(5)}$ is $m(h)/\|w_{(4)} - w_{(5)}\|$. The distance from h to the boundary of its region is*

$$d(h, \partial\mathscr{R}_S) = \min_{i\in S,\, j\notin S} \frac{z_i(h) - z_j(h)}{\|w_i - w_j\|} \;\leq\; \frac{m(h)}{\|w_{(4)} - w_{(5)}\|}.$$

Proof. The distance from a point h to the hyperplane $\{u \;:\; a^\top u + c = 0\}$ is $|a^\top h + c|/\|a\|$; with $a = w_{(4)} - w_{(5)}$, $c = b_{(4)} - b_{(5)}$, the numerator is $m(h)$. The boundary of $\mathscr{R}_S$ is contained in the union of the hyperplanes $z_i = z_j$, $i \in S$, $j \notin S$, and to reach the boundary of a convex polyhedron from an interior point, one must cross one of its half-spaces: the distance to the boundary is the minimum of the distances to these hyperplanes (each minimum is reached in the normal direction, which leaves no other half-space first, otherwise that one would be closer). The pair ((4), (5)) appears in this minimum, hence the inequality. ∎

Remark 8.8. The synthesis writes that the Euclidean distance "to the nearest boundary" is $m/\|w_{(4)} - w_{(5)}\|$ (`genome/05-synthese.md` § 3.1). Proposition 8.7 makes it precise: it is exactly the distance to the hyperplane of the pair at stake, and an upper bound on the distance to the boundary of the region, reached when (4), (5) is also the nearest pair in Euclidean distance. The margin in logit units, the one the logs measure, is the quantity that decides the flip; it is the one that Chapter 12 uses.

The regions, seen in the latent space. Figure 8.5 projects the 6 132 calibration states of layer 23 of the 20b (2 880 dimensions) and colours them by their first expert. The states group by first expert: in the original space, 48.1 % of the 10 nearest neighbours of a state have the same first expert, against 12.2 % by chance. The states of small margin, in the sense of $z_{(1)} - z_{(2)}$ (the 614 of the tenth percentile, below 0.0155), stand more often at the edges of the groups: 0.70 of their neighbours have another first expert, against 0.47 beyond the median in the t-SNE plane, and 0.67 against 0.43 in the original space. This is the image, in the real states, of Proposition 8.6: the fragile states are near the faces.

Two precisions, which the judged caption gives and which must not be lost. First, the projection is faithful only to what it says: PCA keeps 21.0 % of the variance, with a trustworthiness $T(10) = 0.792$; the t-SNE, written from scratch and deterministic, rises to $T = 0.947$, guarded by a threshold declared before the pass, but its distances between groups and its areas cannot be read. Next, the margin that marks the edges is that of the *first* expert; the margin of the set of the four served experts, $z_{(4)} - z_{(5)}$, that of Proposition 8.7, does not mark them (0.51 below the tenth percentile as beyond the median): the colour of the figure is the first expert, and a face between two sets of four does not necessarily separate two colours. The figure illustrates the geometry; it does not measure the guard. Chapter 12 takes up this frame on the routing logs and judges it.

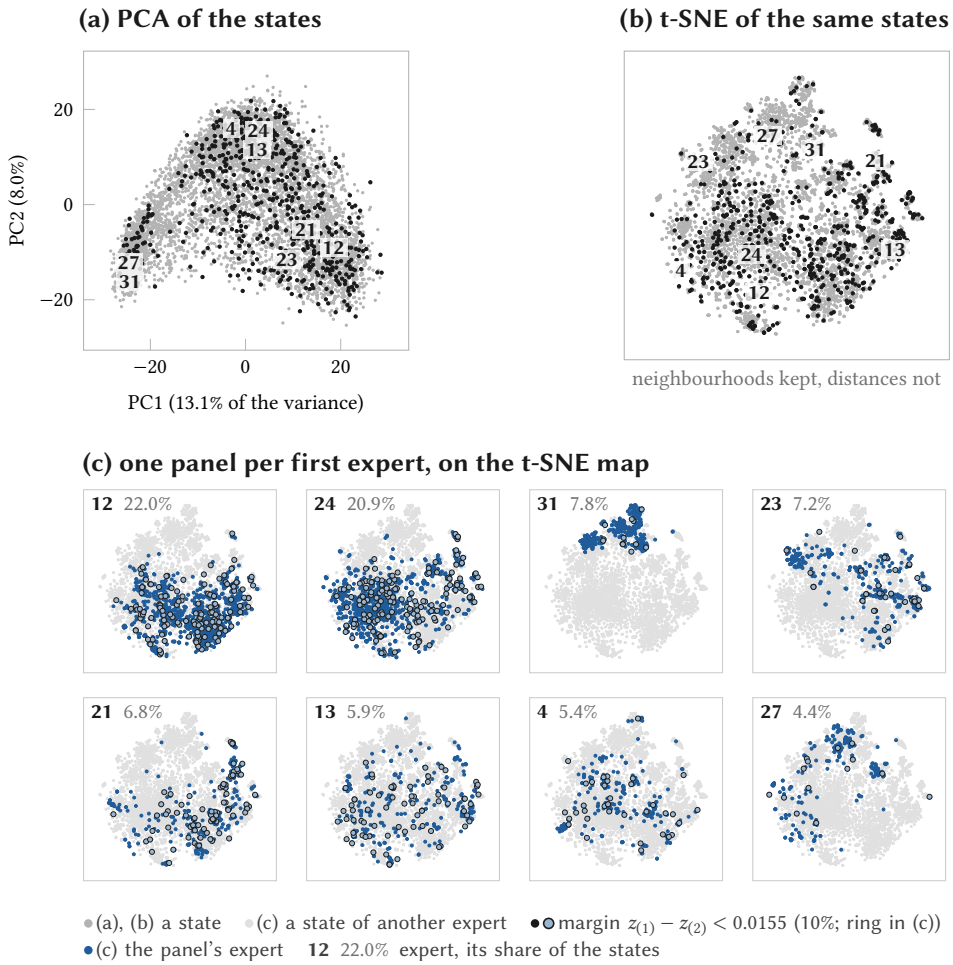


Figure 8.5: The 6 132 states of layer 23 gather by first expert, and the states of small margin stand more often on the edges of the groups (0.70 of their neighbours with another expert, against 0.47). [meas.] Calibration states of gpt-oss-20b (h after the post-attention norm, 2 880 dimensions, windows 10 to 21 of wiki.test). First expert: the largest router logit; margin: $z_{(1)} - z_{(2)}$, the gap between the two largest biased logits. In black (ringed in (c)), the 614 states of smallest margin (10%, below 0.0155). (a) PCA, centred and unscaled: 21.0% of the variance, trustworthiness $T(10) = 0.792$, continuity $C(10) = 0.917$, Spearman correlation of distances 0.59. (b) t-SNE written from scratch, deterministic (perplexity 30): $T = 0.947$, $C = 0.959$, Spearman 0.41; kept by the threshold declared before the run (T and C at least those of the PCA), which its PCA initialisation already meets: it exceeds it by 0.155 and 0.042. Its distances between groups and its areas do not read. Numbers: where the states of the group are densest (density, not purity: in the PCA the group is often a minority there), moved aside a little where two names overlapped. (c) The eight most frequent first experts (80.5% of the states; 19 others). Share of the 10 nearest neighbours with the same first expert: 48.1% in the original space, 28.7% (PCA), 44.2% (t-SNE), 12.2% by chance (sum of the squared shares). Edge of a state, the share of its 10 neighbours with another first expert: 0.70 for the states of small margin against 0.47 beyond the median (t-SNE plane), 0.67 against 0.43 in the original space (judge of the first flow); the margin of the set of four experts served, $z_{(4)} - z_{(5)}$, does not mark them (0.51 below the 10th centile, as 0.51 beyond the median) and is not drawn. Controls: the router re-read reproduces the 6 132 sets of the log (without its bias: 3 482); fidelity 1 for the full PCA, $T = 0.500$ for a permuted plane; edges equal to the judge's (with 9 neighbours, they depart from them).

Data: `mesures/genome/tome3/latents/etats-c23.csv`, `etats-c23.json` (seed 20261027), by `outils/genome/tome3/latents_etats.py`; captures `h-L23.f32`, `routage-L23.bin` (scout); edges in the original space `outils/genome/tome3/juge-latents/juge-etats-marges.json`; `memoire/outils/tome3_latents.py` (controls `latents-temoins.json`).

8.5 Mechanism by mechanism

Table 8.7 ties each verdict to the property that decided it.

8.5.1 Why S^* does not win

S^* is the most instructive case, because it wins everything except the KL: the weight error, the output error (8 experts out of 8), and even the error orthogonal to the gain by the team's second path (Chapter 4). ~~Its paired KL is $+0.0076 \pm 0.0047$ ($1.6\,\sigma$): equal by the rule, unfavourable in estimate, and its tail is thicker.~~ Over 3 windows (765 positions), its paired KL was $+0.0076 \pm 0.0047$; over 10 windows (2 550 positions, K10 pass declared beforehand), it is $+0.00036 \pm 0.00186$: equal to the base, equal first choice (91.92 against 91.88 %) [meas.] (`comparer10-base-contre-recette.json`). Twenty-five per cent less expert error, zero per cent less KL: this is property (c) in its sharpest form.

Why the extreme of the tail rises is a hypothesis, which the synthesis gives and which no pass settles. The Hadamard rotation spreads the error of each row over 64 columns; the output $Wx = (WR^\top)(Rx)$ sees this error through Rx, where a massive activation is itself spread over the 64 positions of its chunk. The weighting of S^* is diagonal ($h = \mathrm{diag}(R\Sigma R^\top)$): it does not see the correlation, in the rotated domain, between the 64 positions that carry the same massive channel. The base, for its part, returns the largest codes of each block almost exact, and leaves the contribution of the massive channels almost intact, which protects the router margins of the following layers, which read the same state. The synonymous mutation, which weights by the full H, does not have this flaw, and it is the one that wins. The prediction that would settle it, the KL of S^* combined with feedback through the full H, is neither measured nor in the lots.

8.5.2 The lesson, in one sentence per level

- In the domain of the weights, there is nothing to generate: the gain is taken on what the computation *does not read* (the sign of a zero) or on what the coder *already knows* (the scale, the column).
- In the domain of the outputs, the mean of the error is not enough: what counts is where it falls. The synonym places it where the inputs do not look, the Hebbian where the energy is low; both have a low expert error, only the first wins in KL.
- In the domain of the text, what costs is the *flip*: an error is worth only its probability of making a later layer cross a face of the router.

8.6 The context of a column symbol

Property (b) gave a lossless winner: the energy class of the input column as the coder's context. The specification described it to the bit, under a CODES_COL stream kind; lot A finally left it out of the format, for a gain too small in view of its price (Section 9.4.1). The specification remains the reference should a model with more numerous outlier channels make it pay; it contains a derivation that must be posed, because a parallel decoder must find the column of each symbol without recomputing anything.

The layout of `block_mxfp4`. An MXFP4 block counts 17 bytes: the exponent e (E8M0), then 16 `qs` bytes that carry 32 codes of 4 bits. Byte t ($0 \le t < 16$) carries weight t of the block in its low nibble and weight $16 + t$ in its high nibble [8, 116]. An $R \times C$ matrix has $B = C/32$ blocks per row; the blocks follow one another in order β (row by row).

Table 8.7: Each mechanism of the round, its verdict, and the measured property that decided it (a: noise given the scale; b: giant columns; c: expert error ≠ KL) [meas.] [calc.].

Mechanism	Verdict	Property	The figure that says it
zero-synonym codon	wins, lossless	(a), inverted: it removes pure noise instead of seeking its structure	0.1370 bit over the 12 tensors; logits identical to the bit
column context	wins, lossless (layer 0)	(b): the only first-order structure left given the scale is per input column	-0.1416 / -0.1231 bit at layer 0; threshold 0.015
synonymous mutation	better in KL; rate to be confirmed ($\lambda = 1$)	(c), in the right direction: weighted by $H = \mathbb{E}[g^2 x x^\top]$, it reduces the smooth part	paired -0.0098 ± 0.0033; median ÷2.3; higher weight error ($\lambda = 0$: 4.7 against 4.4 %)
partitions by κ, per tensor	equal in KL, 0.911× the bits	relative to the row, it moves no bits from one scale to another	paired $+0.0066 \pm 0.0053$ ($1.2\,\sigma$)
seed + coefficients	falls	(a): a random codebook of dimension 8 to 16 on uniform directions	3.06 bits at the base's error
shared motifs	falls	(a): vector quantiser of small dimension	2.90 bits
recipe + exact complement	falls	(a): the entropy of the codes is already reached	4.05 to 4.37 bits against 3.84
duplication (parent)	falls	(a) between experts; the part that looks like kinship is (b)	0.0034 bit of which 0.0016 by chance; D2b 2.6429
strand read by codons, criterion W	wins in weights, falls at expert output	(b): the table clips the massive columns	column 1011: energy ×144, error ×334; output 7.63 against 6.77 %
S^*	wins at expert output, ~~falls~~ equal in KL	(c): the smooth part falls, the extreme of the tail rises	expert error ×0.747; paired $+0.0004 \pm 0.0019$
Hebbian by e	falls at equal expert error	(b) and (c): the error goes to the blocks of small scale, which the RMS norm of the following layers brings back to scale	expert error ×1.02; KL ×3.06
partitions by e, pruning by activity	fall	(b) misused: the per-matrix metric empties the rows of small scale	KL ×4.3 to ×4.9

Data: `genome/05-synthese.md` § 2.3; figures in `k10/frontiere.csv`, `k10/erreur-contre-kl.csv`, `k10/colonne-par-tenseur.csv`.

The chain. The CODES_COL stream codes the $R \cdot B \cdot 32$ canonical nibbles in order β; symbol n is nibble n, the low one of byte $\lfloor n/2 \rfloor$ if n is even, the high one otherwise. The interleaved rANS decoder (Chapter 9; Volume II, chapter on the format) produces, on lane l of macro-step m, the 32-bit word $w = 32m + l$, and its sub-step $k \in \{0, \dots, 7\}$ produces one nibble. Step by step:

$$n = 8w + k \qquad \text{(a word carries 8 nibbles)} \tag{8.3}$$

$$\beta = \lfloor n/32 \rfloor = \lfloor w/4 \rfloor \qquad \text{(32 nibbles per block; } 8(w \bmod 4) + k < 32\text{)} \tag{8.4}$$

$$u = n \bmod 32 = 8\,(w \bmod 4) + k \qquad \text{(rank of the nibble in its block)} \tag{8.5}$$

$$t = \lfloor u/2 \rfloor, \quad \text{high} \iff u \text{ odd} \qquad \text{(the byte, then the nibble)}$$

$$i(u) = \lfloor u/2 \rfloor + 16\,(u \bmod 2) \qquad \text{(the position of the weight in its block)} \tag{8.6}$$

$$\mathrm{col}(n) = (\beta \bmod B) \cdot 32 + i(u) \qquad \text{(the column of the weight in its row)} \tag{8.7}$$

$$\mathrm{ctx}(n) = 4\,\kappa(\beta) + q(\mathrm{col}(n)) \in [0, 12) \qquad \text{(the context)} \tag{8.8}$$

The second equality of (8.4) checks out: $n = 32\lfloor w/4 \rfloor + 8(w \bmod 4) + k$ and $0 \le 8(w \bmod 4) + k \le 8 \cdot 3 + 7 = 31$. Consequence: *the 8 nibbles of a word are in the same block*, so they share the same scale class $\kappa(\beta)$ ($\delta < 128$, $= 128$, > 128, that of v1.0); a single κ per word, as in CODES_CTX, and the decoder keeps its read-ahead of κ. Only $q(\mathrm{col})$ changes from one sub-step to the next, through the column map. Example: $w = 6$, $k = 3$ give $n = 51$, $\beta = 1$, $u = 19$, byte $t = 9$ in its high nibble, $i = 9 + 16 = 25$, and column 25 of the second block of the row, that is $32 + 25 = 57$ if $B > 1$ [calc.].

The column map. The class $q(c) \in \{0, 1, 2, 3\}$ of column c occupies bits $2(c \bmod 4)$ and $2(c \bmod 4)+1$ of byte $\lfloor c/4 \rfloor$: $C/4$ bytes, 720 for $C = 2880$, a multiple of 8 since C is a multiple of 32. The coder computes it for the whole tensor, all experts together:

$$E(c) = \sum_{\text{experts, rows, blocks } b \text{ of column } c} 4^{\delta_b - 128}\, K[\text{code} \,\&\, 7]^2, \qquad K = (0, 1, 2, 3, 4, 6, 8, 12),$$

then $q(c) = \#\{i \,:\, t_i < E(c)\}$, with t_1, t_2, t_3 the 0.25, 0.5 and 0.75 quantiles of the C sorted values. The factor $4^{\delta - 128}$ is the square of the scale $2^{\delta - 128}$: $E(c)$ is the energy of the column in units of the doubled values. The decoder reads the map; it does not recompute it. This is exactly the map that gave 3.684 4 bits [meas.].

The per-matrix choice rule. CODES_COL costs the decoder about two and a half times a CODES_CTX stream (Section 8.7.1). Hence a normative rule for the coder: CODES_COL only if the estimated gain, tables included, reaches 0.015 bit per weight, the threshold that predictive coding had declared for one more stream (0.40 % of 3.84 bits). On the 12 reference tensors, two matrices pass (gate and up of layer 0), for 3.684 3 bits on average, against 3.682 5 when taking the six matrices that gain something: the rule takes 92 % of the column gain (0.0221 bit out of 0.0239) for a more expensive decoder on two matrices instead of six [calc.] (`k10/apparie-eclaireur.json`, field `colonne`).

8.7 The construction specification

Stone's instruction for this step, on 17 October 2026: "vu qu'on a les conclusions je propose d'avancer avec ce qu'on a on a déja pris beaucoup de temps" (*since we have the conclusions I propose to move forward with what we have, we have already taken a lot of time*). We build with the measured winners, in two lots in this order: **A** is lossless and touches the exact file; **B** adds a second file, lossy, and does not touch the first. Each step has its control, its break and its threshold, declared in the specification before any construction.

8.7.1 Lot A: what it carries, and the proof that it changes no bit

Lot A carries three things: the zero-synonym codon in all code streams (the coder rewrites every nibble 8 as 0 before entropy coding); the column context, only where it returns at least 0.015 bit; the fingerprint of the canonical expert, without which the engine's control can no longer compare a decoded expert with its origin.

Table 8.8: What lot A was to return, predicted by the specification [calc.] on [meas.]. Chapter 9 measures it.

	gpt-oss-20b	gpt-oss-120b
expert weights	19 110 297 600	114 661 785 600
lossless v1.0 (A_flux, all tensors)	3.8509 bits	3.8549 bits
zero merged (A_flux, all tensors) [meas.]	3.7230 bits (−0.1280)	3.7292 bits (−0.1257)
expert bytes saved (zero only)	305.6 MB	1 801.2 MB
`.trc` v1.0 file, recorded	11 157 405 696 B	57 730 662 400 B
expected v1.2 file (zero only)	≈ 10.85 GB	≈ 55.93 GB
speed ceiling with the disk limiting	+3.4 %	+3.4 %

Data: `genome/05-synthese.md` § 4.1, after `mesures/genome/predictif-elagage/taux-reel-toutes-gptoss\{20b,120b\}.json`.

The ceiling, posed. With the disk limiting, the time of a token is proportional to the bytes read, and the speed gain is at best the inverse of the ratio of the bytes [calc.]:

$$\frac{1}{1-0.1257/3.8549}-1=\frac{1}{0.96739}-1=3.37\,\%.$$

Aligning each chunk on 4 096 bytes costs at most $4\,608\times4\,096=18.9$ MB on the 120b, that is $18.9\cdot10^6\times8/114\,661\,785\,600=0.0013$ bit per weight [calc.]. Chapter 9 shows that this ceiling only bounded the effect of the bytes, and that the speed followed another variable.

Proposition 8.9 (A1: the engine's GEMV returns the same bits)**.** *The `produit_bloc` kernel (`moteur/crates/trc-noyaux/noyaux/gemv.cuh`, at commit `000cef9`) starts from $s=+0$ and accumulates $s\leftarrow\mathrm{fmaf}(v,x,s)$ in a fixed order, with $v=$ `valeur_e2m1x2`(c), the doubled magnitude of code c: an integer of $\{0,1,2,3,4,6,8,12\}$, with sign given by bit 3, so that $v=-0$ for code 8 and $+0$ for code 0. If the inputs x are finite, replacing code 8 by code 0 changes no bit of s at any step.*

Proof. We follow the two computations in parallel, the original and the canonical, and show by induction on the steps that s is bit-equal in both and is never -0.

(0) A grid. Every finite binary32 is an integer multiple of 2^{-149} (the smallest subnormal). v is an integer; so the exact product $v\cdot x$ is a multiple of 2^{-149}, and so is the exact sum $v\cdot x+s$. Such a sum, if it is not zero, is at least 2^{-149} in absolute value; its round-to-nearest is therefore not zero (below the normal threshold, all multiples of 2^{-149} are representable; above it, round-to-nearest never returns zero).

(1) The accumulator is never -0. At the start, $s=+0$. At a step, fmaf computes $v\cdot x+s$ exactly and rounds it once, $s'=\mathrm{RN}(v\cdot x+s)$. If the exact sum is zero, IEEE 754 returns $+0$ in round-to-nearest, unless both terms are -0 [72]; but $s\neq-0$ by the induction hypothesis, so $s'=+0$. If the exact sum is not zero, $s'\neq0$ by (0). In both cases, $s'\neq-0$.

(2) A step on a zero code is the same in both computations. For $v=\pm0$ and finite x, $v\cdot x=\pm0$ exactly. If $s\neq0$, $s'=\mathrm{RN}(\pm0+s)=s$. If $s=0$, then $s=+0$ by (1), and $s'=+0$ whatever the sign of $v\cdot x$, since $(-0)+(+0)=+0$ and $(+0)+(+0)=+0$. In both computations, $s'=s$.

(3) The other steps have the same operands. A non-zero code is the same in both computations; x and s are bit-equal by induction; fmaf is a function of its operands. So s' is bit-equal.

By induction on the steps, s is bit-equal in both computations at every step. ■

The butterfly reduction between lanes (`somme_warp_f32`) then adds bit-equal partial sums, in the same order; the block scale and the bias are applied afterwards. Consequence: the output of each expert GEMV, hence each logit, is identical to the bit. Argument (0) makes precise that of the synthesis ("$|v| \geq 1$ for a non-zero code, so $|v \cdot x| \geq |x|$ does not round to zero"): what prevents a non-zero sum from rounding to zero is that everything stays on the grid $2^{-149}\mathbb{Z}$, and this holds because the magnitudes are *doubled*, hence integers, and the scale does not enter the accumulation. The scout's proposition (Chapter 3) left open the sign of a zero partial sum; the real kernel's start at +0 settles it. Chapter 9 measures it on the card and on the logits.

The E2M1 value table, an invariant. It is read in a single place of the kernels, the function `valeur_e2m1x2`; the f64 reference reads the GGUF and is not touched. Any new reader of codes (a batched prefill kernel, a GEMM) must read code 8 as zero: this is the invariant that lot A wrote into the architecture (§ 5.6).

The decoder and the compact LUT. If the column context entered the format, a decoder with twelve contexts per symbol would be needed. The normative LUT of a table occupies $M = 4\,096$ words of 32 bits, 16 KiB; CODES_CTX loads three per thread block, 48 KiB, the limit of a block's static shared memory. Twelve contexts would require $12 \times 16 = 192$ KiB, beyond even the 99 KiB of dynamic shared memory of a block on sm_89 (CUDA programming guide, read, not recorded on the card) [lit.]. In geometry 2, the symbol fits in 4 bits; for each context c we store $\mathrm{sym}[c]$, 4 096 symbols two per byte (2 KiB), and $f[c][16]$, $\mathrm{cum}[c][16]$ in u16 (64 B). A decoding step becomes

$$\begin{aligned} \mathrm{slot} &\leftarrow x \;\&\; 4095, \\ s &\leftarrow \big(\mathrm{sym}[c][\mathrm{slot} \gg 1] \gg 4\,(\mathrm{slot} \;\&\; 1)\big) \;\&\; 15, \\ y &\leftarrow f[c][s]\,(x \gg 12) + \mathrm{slot} - \mathrm{cum}[c][s], \end{aligned}$$

then the usual renormalisation ($x \leftarrow (y \ll 16) \mid \mathrm{words}[p]$ if $y < L$, otherwise $x \leftarrow y$). For 12 contexts: $12 \times (2048 + 64) = 25\,344$ bytes, plus the map (720 B), that is ≈ 25.5 KiB, below the static limit [calc.].

Lemma 8.10 (A2: the compact LUT returns the same sequence)**.** *For every slot $i \in [0, 4096)$, the normative LUT gives $(s, f[s], i - c[s])$ with s the unique symbol such that $c[s] \leq i < c[s] + f[s]$; the compact LUT stores the same s at $\mathrm{sym}[c][i]$ and reads $f[s]$ and $c[s]$ in the tables of the same context. The two steps therefore compute the same y and the same renormalisation, and the sequence of states, of words read and of symbols is the same.*

Proof. The two LUTs are built from the same frequencies f and cumulative counts c of the context; the normative one stores at i the triple $(s, f[s], i - c[s])$, the compact one stores s at i and leaves $f[s]$ and $c[s]$ in two small tables. At equal state x, both read the same slot $i = x \bmod 4096$, draw the same s from it, and compute $y = f[s]\lfloor x/4096 \rfloor + (i - c[s])$ from the same values. The renormalisation depends only on y and on the pointer. By induction on the steps, states, pointers and symbols coincide. The LUT is a representation of the table: what is normative is the sequence of decoded symbols. ■

The cost, posed. A compact step makes two more shared-memory reads and one more subtraction than a geometry-1 step; geometry 2 makes 8 steps per word instead of 4. A CODES_COL matrix

therefore costs about 2.5 times a CODES_CTX matrix [calc.]. The engine decodes an expert in 0.92 ms (54.2 ms per token for 59 misses on the 120b, lot O). If gate and up of a single layer switch to CODES_COL, an expert of that layer costs $0.92 \times (1 + 2 \times 2.5)/3 \approx 1.8$ ms; at $59/36 \approx 1.6$ misses per token in that layer, this is $\approx +1.5$ ms out of 457 ms per token, 0.3 % [calc.], and decoding already overlaps with the reads.

The criteria of lot A, declared beforehand. They are in Chapter 9 with their measurements; their form is kept here. A-T1 to A-T4: named refusals of the format, codec and decoder byte for byte, fingerprints at 100 %. A-C1: 0 differing logits between v1.2 and v1.0 over 10 windows, 20b and 120b (Proposition 8.9); the break makes them differ. A-C2: gain on the chunk bytes of at least 0.12 bit per expert weight on each of the two models (predicted 0.1280 and 0.1257; 0.12 leaves 0.0013 bit of alignment and a margin). A-C3: no speed regression in an ABBA series, $r \geq 1 - 2\sigma_{\text{noise}}$, and a gain is announced only if it exceeds $2\sigma_{\text{noise}}$. A-C4: the decoding throughput of CODES_COL at least 10 times that of the disk. If A-C4 fell, the column would be withdrawn and the zero alone would already return 86 % of the lot's gain on the 12 measured tensors (0.1370 out of 0.1591 bit) [calc.].

8.7.2 Lot B: the synonymous genome, and the measurement that was missing at first

Lot B carries the synonymous genome as a second file, `<nom>.rapide.trc`, read instead of the exact file when the engine serves an expert in *fast mode*. It is not nested in the exact file: the exact complement behind the synonym base would cost 1.689 bits instead of 1.219 (Chapter 3). This is the "fast mode and exact mode" of Stone's idea. The companion's format and the coder are in Chapter 10; what decides its construction is kept here.

The missing measurement. The variant that beats the frontier in bits ($\lambda = 1$, 2.563 bits H_0) had no KL; the one that had a better KL ($\lambda = 0$) was not below the frontier in bits (Section 8.2). Lot B therefore begins with **B0**: the scout's KL pass, protocol P1, on the variant to be built, compared paired with the frontier base. The criterion, declared beforehand: $\lambda = 1$ is kept alone if it is significantly better; equal, it still dominates the frontier (0.078 bit less at equal KL), and both variants are built as two points of the fast mode, which **B2** settles on the whole model; worse, $\lambda = 0$ is kept. Chapter 10 and the judge of Chapter 11 say what B0 and B2 decided.

What it would return. With $\lambda = 1$, 2.56 to 2.60 bits per weight read, the companion would weigh $\approx$ 6.14 to 6.24 GB (20b) and $\approx$ 36.9 to 37.4 GB (120b), that is a ratio $f \approx 0.69$ to 0.71 of the bytes of a fast chunk to an exact v1.2 chunk, and a speed ceiling of ×1.41 to ×1.46 with the disk limiting. It fits in RAM on the 20b (6.2 + 1.9 GB of residents under 16 GB), not on the 120b [calc.] on [meas.] (`genome/05-synthese.md` § 4.7).

The two uses. *Cache misses*: an "every miss in fast mode" policy converges to pure fast mode; the form that keeps a meaning has two levels, a cold expert served in fast mode and re-read in exact mode once promoted. On the 120b, the engine's record is 59 misses for 144 calls per token, so a share of calls served in fast mode $\varphi \leq 59/144 = 41$ % at the height of the policy [calc.]. *The speculative draft* [117]: the verification of $k + 1$ positions by the exact model returns the text exact to the bit; the draft only proposes. The specification drew a lemma from it, which says where it pays.

Lemma 8.11 (B2: the draft must be resident)**.** *In the disk-limited regime, let b_e be the exact expert bytes read per token served simply, b_d the bytes the draft reads from the disk per proposed token, α the*

acceptance rate, and $U(2) = 2 - \rho$ *the ratio of the exact bytes read by a verification pass of two positions to those of a single one* ($\rho \in [0, 1]$*: the share of the second position's misses already paid by the first). With one proposed token* ($k = 1$)*, speculative decoding reads fewer bytes per returned token than simple decoding if and only if*

$$\alpha > 1 - \rho + b_d/b_e. \tag{8.9}$$

Proof. A cycle reads b_d bytes for the draft and $(2-\rho)\,b_e$ for the verification; it returns on average $1+\alpha$ tokens (the token of the verification, plus the proposed token if it is accepted, with probability α). Simple decoding reads b_e per token. The cycle wins if and only if $b_d + (2-\rho)\,b_e < (1+\alpha)\,b_e$; dividing by $b_e > 0$, $b_d/b_e + 2 - \rho < 1 + \alpha$, that is $\alpha > 1 - \rho + b_d/b_e$. ■

Corollary 8.12 (What the condition costs)**.** *(i) Since* $\alpha \leq 1$*, one needs* $\rho > b_d/b_e$*: a draft read from the disk must be paid for by the reuse of experts between neighbouring positions.*

(ii) Draft read on miss from `<nom>.rapide.trc`*:* $b_d/b_e \approx f \approx 0.7$ *(same misses, chunks 0.7 times smaller); one needs* $\rho > 0.7 + (1-\alpha)$*, that is* $\rho > 0.8$ *for* $\alpha = 0.9$*. With independent routing, the union of the experts of two tokens of the 120b weighs* $2 - t/E = 2 - 4/128 = 1.969$ *times that of a single one, so* $\rho \approx 0.03$*, far from the condition. The real* ρ*, at the engine's cache, is not measured.*

(iii) Draft resident in RAM ($b_d = 0$)*: the condition becomes* $\alpha > 1 - \rho$*, that of Stone's idea. On the 120b, the fast file (≈ 37 GB) does not fit in 16 GB; the only resident draft would be another model, the 20b in fast mode (same vocabulary), whose agreement with the 120b is not measured.*

(iv) The lemma counts bytes, not time. The engine draws only 1.55 GB/s of useful throughput from the disk out of 1.848 because it has fewer than four reads in flight per layer; verifying two positions doubles the queue. That gain is bounded by $1.848/1.55 = \times 1.19$ *and does not depend on the draft [calc.] on [meas.].*

Proof. (i) If $\rho \leq b_d/b_e$, the right-hand side of (8.9) is at least 1, and $\alpha \leq 1$ cannot exceed it. (ii) Replace b_d/b_e by 0.7 in (8.9): $\rho > 1 - \alpha + 0.7$. With $k = 4$ experts out of $E = 128$ drawn independently for each token, an expert of the second position is already loaded with probability k/E; the union counts on average $k + k(1 - k/E)$ experts, that is $2 - k/E$ times k, and $\rho = k/E = 0.03$. (iii) $b_d = 0$ in (8.9). (iv) The ratio of throughputs is a measured ratio, independent of α. ■

Lemma 8.11 reads in one sentence: on a machine where the disk limits, a draft that reads the disk almost never pays. The figure it deserves, the (ρ, α) plane with the line (8.9) for three values of b_d/b_e (III-7.5 of the inventory), is not drawn; the condition says it. Step B5 of the specification, the draft, *is not built* before ρ is measured on the routing traces and α by the greedy agreement of fast against exact.

The steps of lot B, declared beforehand. B0 (the KL of $\lambda = 1$, paired); B1 (the companion's format, decoded symbols equal to the written symbols); B1c (the coder on the card, equal to the Python coder on 8 drawn matrices); B2 (the KL of the whole 20b model in fast mode, better than the whole frontier, and at the level of llama.cpp's best requantisation at equal or fewer bits); B3 (the two-level policy, replayed on the traces before being built); B4 (the 120b); B5 (the draft, under the condition of lemma 8.11). The benchmark of B2 (ii) is an answer by delegation of 2026-10-17, reversible: the best requantisation that llama.cpp produces from the same file, because it is the reference tool that Stone would use without troncons, free and already on the machine.

8.7.3 The leads that the measurements already point to

None of the following leads is in the lots; each carries what is measured and the measurement that would judge it.

The tropical guard. The only lead the measurements point to: flips dominate the KL between two close computations, and the margin predicts the flip (median 0.083 over all pairs, 0.0095 over those that flip). The construction it suggests: in fast mode, after the fast experts of layer ℓ, look at the margin of the router of layer $\ell + 1$, and redo layer ℓ in exact mode for the positions below a threshold τ. The cost is the share of cells below τ, the gain the share of flips avoided; both can be read on the routing logs of lot R, without a new pass of the model. The decision threshold, declared in the specification before any reading: the guard is worth building if it catches at least 80 % of the flips for at most 10 % of the cells. Chapter 12 judges it.

The bridge expert. Serving a missing expert by its best paralogue leaves 79, 84, 66 and 33 % of the energy of its output in error (layers 0, 8, 16, 23), and ~~substituting is never better than removing the expert~~; corrected in Chapter 5: removing the expert is better than substituting it at layers 0 and 8; at layers 16 and 23, the best paralogue does slightly better than removal for the majority of the experts, by at most half a point of layer error on average (`genome/duplication.md` § 0, point 6, and `genome/05-synthese.md` § 3.2 strike the same sentence). The bar for a bridge is therefore not the absent expert, it is removal. Nothing beyond that is measured.

The fractal. The data of the round gave the share of energy at only seven ranks per tensor: too few for a slope. The literature on heavy-tailed spectra [118] gives the hypothesis to measure [lit.]; Chapter 13 measures it. Compression by iterated function systems has the prognosis of the seed, for the same reason (property (a)). A Hilbert curve to arrange the experts on the disk hardly counts at the granularity of a 12 MB chunk [calc.] on [meas.].

8.8 What could be wrong

What could be wrong

- **Three protocols, a single axis.** The KL ratios bring together passes that have neither the same replaced experts, nor the same windows, nor the same number of positions (765 to 2 550). A ratio is not invariant to the number of replaced experts: the KL grows with it, and not linearly, because of the flips. Figure 8.1 shows them side by side; it does not merge them into one curve, and it keeps 32 experts out of 768 for P1 (reserve R3).
- **The rates do not all have the same estimator** (real codec, H_0 without scales, accounting of the strand). Between protocols, a gap of 0.04 bit can come from the estimator alone.
- **The window uncertainty** rests on 10 blocks, 9 degrees of freedom. The verdict of the synonymous mutation ($2.9\,\sigma$) holds by both estimators; another split could widen it.
- **The "smooth part and tail of flips" reading** is a working hypothesis (assumption 8.5). It rests on direct measurements between two exact engines and on the quantiles of the round; it has not been measured for the mechanisms of the round by imposed routing. Proposition 8.3, for its part, is an exact computation; what is hypothesis is the separation of the observed KL into its two terms.
- S^* **equal in KL** rests on the K10 pass, declared beforehand by the genome team, but which the synthesis first copied by hand from a file not yet committed; `frontiere.py --k10` now draws it from the committed log, and its figures are those of this chapter. The explanation of its tail (diagonal weighting in the rotated domain) is a hypothesis that no pass settles.
- **Proposition 8.7** makes a sentence of the synthesis precise; the margin of the logs is in logit

units, and the Euclidean distance is a faithful image of it only if the norms $\|w_i - w_j\|$ vary little from one pair to another, which is not measured here.

- **Figure 8.5** is a projection: 21.0 % of the variance for PCA; the t-SNE keeps the neighbourhoods ($T = 0.947$), not the distances. It colours by the first expert, and the margin of the four served experts does not mark the edges in it: it illustrates the geometry of the router, it does not prove that the flips of a set of four can be seen in it.
- **The column context** is measured only on 12 tensors of the 20b (4 layers); on the 120b and the other layers, its gain may be larger or zero. The gate figure of layer 0 depends on the reference (0.1416 against the byte stream, 0.1454 against the nibble stream); the synthesis mixed them.
- **Proposition 8.9** covers the kernel as it is at commit `000cef9`; it rests on integer doubled magnitudes and on a scale applied outside the accumulation (the grid of argument (0)). A kernel that multiplied the scale inside the accumulation, or a GEMM on dequantised weights starting from a -0, would leave the proof; the architecture's invariant says so.
- **The shared-memory limit** (99 KiB on opt-in, 48 KiB static) is read in NVIDIA's documentation, not recorded on the card; the compact LUT fits under 48 KiB, which makes the specification independent of this figure.
- **Lemma 8.11** assumes α independent of the position and ρ constant; it counts bytes, not time; its figures for ρ are those of independent routing, not measured at the cache.
- **All the measurements of the round are on wiki.test**, calibration and evaluation on disjoint windows of the same corpus; on code or dialogue, the gain of the synonymous mutation may fall.
- **The figures of lot R** quoted here (flips, 88 %) are read in the synthesis, which quotes them; this chapter has not redone them from the logs of Volume II.

Part II

The construction

Chapter 9

A synonymous codon in the format (lot A, v1.2)

Where we start, what this chapter establishes, what remains

At the start, the engine of Volume II read the `.trc` v1.0 format (one level, CODES_CTX streams with a table per lane); the two files of the machine were in v1.0, 11 157 405 696 bytes for gpt-oss-20b and 57 730 662 400 for gpt-oss-120b. The predictive coding team had measured, with real decoded rANS streams, that folding nibble 8 (−0) into nibble 0 takes the 120b from 3.8549 to 3.7292 bits per expert weight and the 20b from 3.8509 to 3.7230 (Chapter 6). The synthesis had proved that the engine's GEMV returns the same bits on the two codes (Proposition 8.9). **This chapter establishes** the exact gain of the fold, proved then checked tensor by tensor on 180 tensors; that all the code readers of the project read code 8 as 0; the v1.2 format to the bit and the lemma that guarantees its invariant; the two recoded files, exact in value expert by expert, 1 800 486 912 bytes fewer on the 120b; logits identical to the bit; and milestone J1 redone on the 120b, ×1.454 in generation, with what explains it on this disk: the pattern of read requests that the format imposes. **What remains** is to isolate the exact variable of the pattern, which no one has yet found, and to remeasure the gain on any other disk. Sources: `genome/06-codon-v1-2.md`, `genome/construction-codon-pour-le-memoire.md`, `genome/JUGEMENT-construction.md`, `genome/JUGEMENT-placement.md`; measurements under `mesures/genome/construction/codon/`, `mesures/genome/construction/juge/`, `mesures/disque/placement/juge/seance/`.

In brief

In the genetic code, several three-letter codons give the same amino acid: GGU, GGC, GGA and GGG all give glycine. These codons are called synonymous, and the cell reads them alike [4, 88]. The image is only an analogy, but it fits here. A weight of gpt-oss fits in 4 bits: one sign bit and three magnitude bits, sixteen codes for fifteen values, because +0 and −0 are two codes of the same zero. The original quantiser kept the sign of values rounded to zero; this sign is a perfectly fair coin, and no computation reads it. Storing it nevertheless costs almost one bit per zero, and one weight in eight is a zero. Lot A removes it: the coder rewrites every −0 as +0 before entropy coding. The gain is predicted exactly (twice the share of the −0) and checked to within two ten-thousandths of a bit on every tensor. The logits do not change by one bit, and the proof is written on the real kernel. The file of the 120b loses 1.80 GB, and, on this machine, the engine runs 1.45 times faster. This last figure is not the one the bytes predicted (3.4 % at most): it comes from the way the new format splits its reads on this disk, and holds only for it.

9.1 Sixteen codes for fifteen values

The E2M1 code. MXFP4 stores the weights in blocks of 32 that share an E8M0 scale, 2^{e-127}; each weight is a 4-bit code in the E2M1 format: one sign bit, two exponent bits, one mantissa bit [7, 8]. Its eight magnitudes are 0, 0.5, 1, 1.5, 2, 3, 4, 6. llama.cpp and the engine *double* them, $K = (0, 1, 2, 3, 4, 6, 8, 12)$, and move the factor 2 into the scale, 2^{e-128}. For a code $c \in \{0, \dots, 15\}$, with bits $c_3c_2c_1c_0$,

$$v(c) = (-1)^{c_3}\, K[c \,\&\, 7]. \tag{9.1}$$

The map v is injective except on $\{0, 8\}$: $v(0) = +0$ and $v(8) = -0$, two codes for a single value, since $-0 = +0$ in IEEE 754 [72]. Sixteen codes, fifteen values.

The coin of the sign. The quantiser of gpt-oss keeps the sign of a value rounded to zero. Over the 72 expert tensors of gpt-oss-20b (19 110 297 600 weights), code $+0$ has the share 6.3920 % and code -0 the share 6.3929 %: the sign of a zero has an entropy of 0.9999999967 bit [meas.] (`mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`). The entropy of the codes is 3.8638 bits, that of the values 3.7360, and the difference, 0.1278 bit per weight, is what a sign that nothing reads costs. Figure 9.1 shows the sixteen codes on the folded line of values: before, two spokes for zero; after, a single one, which carries both shares.

9.2 The exact gain of the fold

Proposition 9.1 (The entropy of a code with a synonym)**.** *Let C be the 4-bit code of a weight, $V = v(C)$ its value and $S = C_3$ its sign bit. Then*

$$H(C) = H(V) + \mathbb{P}(V = 0)\, H(S \mid V = 0). \tag{9.2}$$

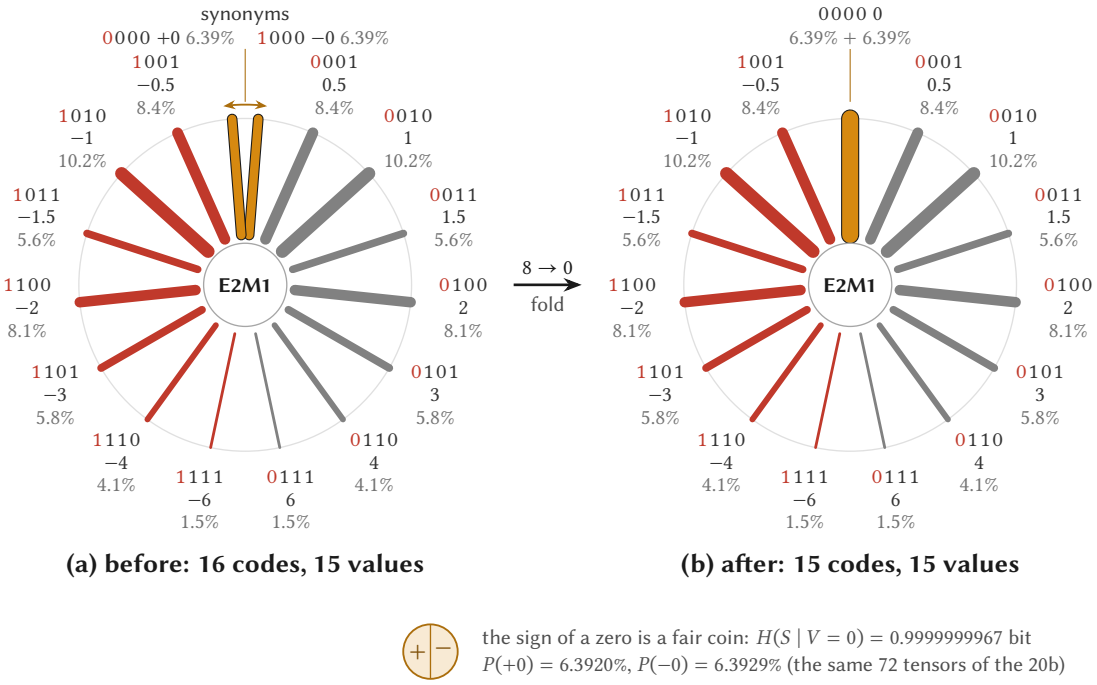
Proof. V is a function of C, so $H(V \mid C) = 0$ and $H(C, V) = H(C)$. By the chain rule [2], $H(C, V) = H(V) + H(C \mid V)$. Given $V = v \neq 0$, C is determined, since v is injective away from 0: $H(C \mid V = v) = 0$. Given $V = 0$, $C \in \{0, 8\}$, and C is determined by S: $H(C \mid V = 0) = H(S \mid V = 0)$. Hence $H(C \mid V) = \sum_v \mathbb{P}(V = v)\, H(C \mid V = v) = \mathbb{P}(V = 0)\, H(S \mid V = 0)$, and $H(C) = H(V) + \mathbb{P}(V = 0)\, H(S \mid V = 0)$. ∎

Corollary 9.2 (The expected gain per weight)**.** *Folding 8 into 0 replaces C by $C' = \gamma(C)$, with $\gamma(8) = 0$ and γ the identity elsewhere; C' is in bijection with V, so $H(C') = H(V)$. The entropy gain is $\mathbb{P}(V = 0)\, H(S \mid V = 0)$ bit per weight. When the sign of a zero is a fair coin, $H(S \mid V = 0) = 1$, the -0 are half of the zeros, and*

$$\textit{gain} = \mathbb{P}(V = 0) = 2 \times \textit{(share of the } -0\textit{)}. \tag{9.3}$$

The result holds conditionally on any context X known to the decoder (scale class κ, neighbouring byte): replace $H(\cdot)$ by $H(\cdot \mid X)$ everywhere, and the proof does not change.

Remark 9.3 (The byte alphabet). The CODES_CTX stream does not code nibbles but bytes, two codes at a time, with one table per scale class. The same proof, applied to the byte $O = (C_{\text{low}}, C_{\text{high}})$ and to its value $(V_{\text{low}}, V_{\text{high}})$, gives a gain of $\mathbb{P}(V_{\text{low}} = 0)\, H(S_{\text{low}} \mid \cdot) + \mathbb{P}(V_{\text{high}} = 0)\, H(S_{\text{high}} \mid \cdot)$ bits per byte if the signs of the zeros are independent of everything else, that is $\mathbb{P}(V = 0)$ bit per weight. The fold removes from the alphabet the bytes one of whose nibbles is 8: 16 bytes with high nibble 8, 16 with low nibble 8, one of them common (0x88), that is $16 + 16 - 1 = 31$ bytes; $256 - 31 = 225$ remain [calc.].



(a) before: 16 codes, 15 values **(b) after: 15 codes, 15 values**

the sign of a zero is a fair coin: $H(S \mid V = 0) = 0.9999999967$ bit
$P(+0) = 6.3920\%$, $P(-0) = 6.3929\%$ (the same 72 tensors of the 20b)

Figure 9.1: The fold 8 → 0 removes one of the two codes of zero: sixteen codes for fifteen values, then fifteen codes. [meas.] The fifteen values of E2M1 are ranked at equal steps (24°) around a circle: 0 at the top, positive values clockwise, ±6 meeting at the bottom. Values are those of the OCP standard; the llama.cpp table doubles them, the factor 2 going into the scale. The red bit is the sign bit. The width of a ray grows with the measured share of its code in the 72 expert tensors of gpt-oss-20b (19 110 297 600 weights); the width is affine in the share (0.4 pt + 44 pt × share), so the zero ray thickens by ×1.88 at the fold, not ×2. Before: two codes, `0000` (+0) and `1000` (−0), for one value; after: fifteen codes, the zero ray carries both shares (outlined in ink). The count of `1000` equals the −0 folded by the coder of lot A, a second tool. The coin is measured on the same tensors: nothing predicts the sign of a zero, so the rANS coder recovers almost a whole bit per zero. Analogy, not proof: the synonymous codons of the genetic code (GGU, GGC, GGA, GGG all give glycine). Neither the context classes κ nor the byte alphabet of the coder are drawn; their effect on the gain is the subject of Figure 9.2.

Data: `mesures/genome/predictif-elagage/histo-toutes-gptoss20b.npz`, `mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`; control `mesures/genome/construction/codon/tailles.json`; script `memoire/outils/tome3_codon.py`.

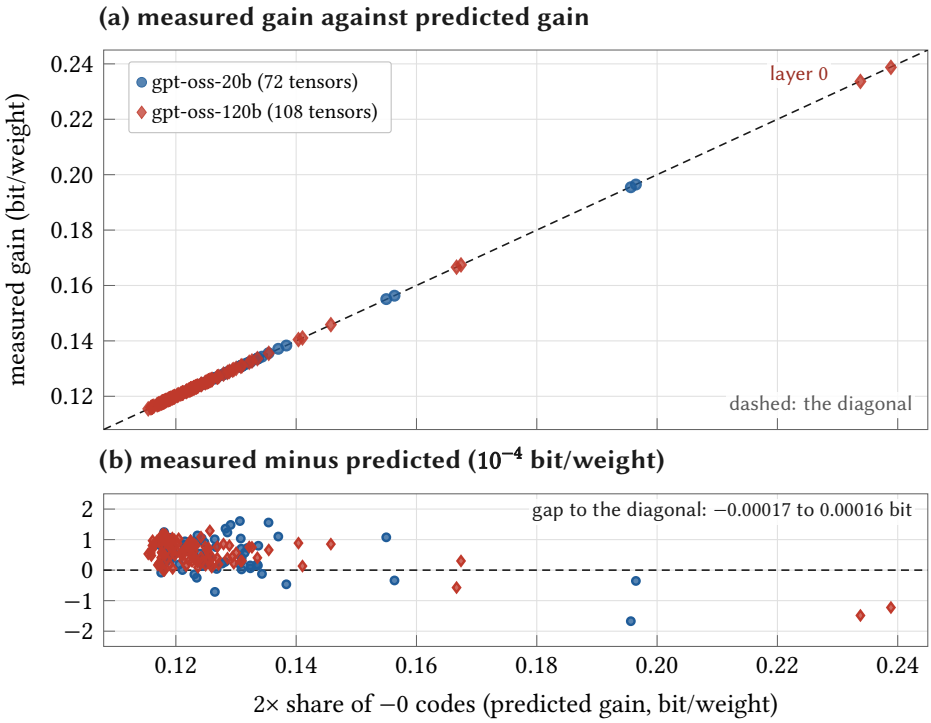


Figure 9.2: The 180 expert tensors of the two models sit on the diagonal that the corollary predicts. [meas.] [calc.] (a) Measured gain of the fold (bits per expert weight, coded chunks) against twice the share of −0 codes. Under a fair coin the −0 are half of the zeros, and the corollary predicts the diagonal (dashed); the two points of layer 0 of the 120b are the richest in zeros. (b) The same points, measured minus predicted, in 10^{-4} bit per weight: the context κ and the byte alphabet of the coder move no tensor further than −0.00017 to 0.00016 bit.

Data: `mesures/genome/construction/codon/tailles-par-tenseur.csv`; script `memoire/outils/tome3_codon.py`.

The expected figure, and the measured figure. The coder of lot A folded 1 221 698 587 nibbles 8 on the 20b (6.393 % of the expert weights) and 7 199 491 948 on the 120b (6.279 %); the verification, which canonicalises the slices of the GGUF by its own path, counts the same numbers to the unit [meas.] (`tailles.json`). The corollary therefore predicts [calc.]

$$2 \times 0.0639288 = 0.12786 \text{ bit (20b)}, \qquad 2 \times 0.0627889 = 0.12558 \text{ bit (120b)},$$

and the gain measured on the coded chunks is 0.12791 and 0.12562 bit (Section 9.6). The gap, four to five hundred-thousandths of a bit, is a crumb of table rounding: 225 symbols instead of 256 to normalise over 4 096 slots.

Tensor by tensor. Figure 9.2 redoes the comparison on each of the 180 expert tensors of the two models (72 of the 20b, 108 of the 120b). The gap between the measured gain and twice the share of the −0 stays between −0.00017 and +0.00016 bit, on average +0.00005 [calc.] on [meas.] (`tailles-par-tenseur.csv`). This is a stronger result than it looks: if the sign of a zero were predicted by the scale class or by the neighbouring byte, which the coder knows, the measured gain would be smaller than $\mathbb{P}(V = 0)$ (the conditional corollary would give $\mathbb{P}(V = 0)\, H(S \mid V = 0, X) < \mathbb{P}(V = 0)$). The 180 points on the diagonal say that, in every tensor, the sign of a zero is a fair coin that nothing predicts, and that the rANS coder [119, 120] recovers its whole bit.

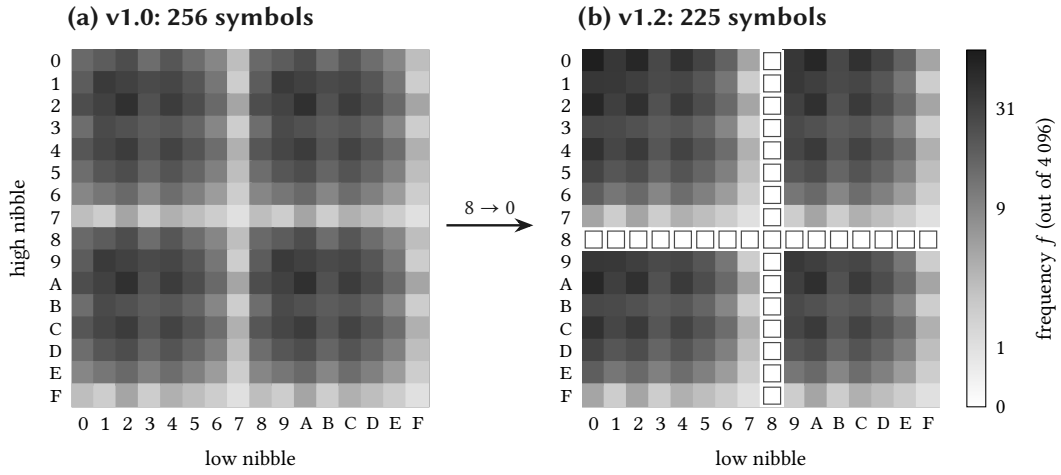


Figure 9.3: After the fold, row and column 8 go out: 225 bytes remain of 256. [meas.] The CODES_CTX table ($\kappa = 1$) of gpt-oss-120b, layer 18, up matrix, as a 16 × 16 grid (high nibble in rows, low nibble in columns); grey level $\log_{10}(1 + f)$, f the frequency out of 4 096 (scale on the right), like a density. White cells framed in ink: the bytes of zero frequency, absent. Their mass moves onto row and column 0. Rows 7 and F are pale in both panels because the codes ±6 (±12 in the llama.cpp table) are rare: pale is rare, framed white is absent. One table of one layer; the others are in the same file.

Data: `mesures/genome/construction/codon/tables-octets.csv`; script `memoire/outils/tome3_codon.py`.

Where the zeros are. The gain ranges from 0.115 bit (120b, layer 26, up) to 0.239 bit (120b, layer 0, up, where 11.9 % of the weights are −0, hence about 24 % of the zeros); layer 0 is everywhere the richest in zeros (20b: 0.196 bit in gate and in up), then layer 1; the means per matrix (gate, up, down) are equal to within 0.002 bit [meas.]. Layer 0 is also the one where the hidden state carries outlier channels and where the column context returned its 0.14 bit (Section 8.3.2). One explanation links them, which is not tested: a block scale pulled upwards by a giant column crushes the other weights of the block towards zero.

What the fold does to the tables. Figure 9.3 shows a CODES_CTX table (120b, layer 18, up, $\kappa = 1$) as a 16 × 16 grid, before and after: row and column 8 go dark, and their mass moves to row and column 0. Over the 540 CODES_CTX tables of layers 0, $L/2$ and $L - 1$ of the two models, a second reader, written from the specification alone (`tables_octets.py`), finds: in v1.0, 540 tables out of 540 gave a slot to the bytes carrying an 8, for a mass of 412 to 1 039 slots out of 4 096 (13 % on average); in v1.2, none [meas.] (`tables-octets.json`). These 13 % of slots returned to the other bytes are, in bits, the gain of the corollary.

9.3 Who reads code 8, and why the bits do not change

The fold is lossless only if *every* code reader gives code 8 the value 0. Lot A listed all the readers of the repository (search for `e2m1`, `kvalues`, `& 8`, `& 7` in `moteur/`).

−0 and +0 have the same value; the only question is that of the *bits* of a sum in which one term goes from −0 to +0. Proposition 8.9 answers it for the GEMV, in three moves: a grid (v an integer, x a binary32, so every exact sum is a multiple of 2^{-149} and does not round to zero if it is not zero); the accumulator, started at +0, is never −0; a step on a zero code therefore leaves the accumulator unchanged in both computations. Figure 9.4 draws its state machine, and its measurement.

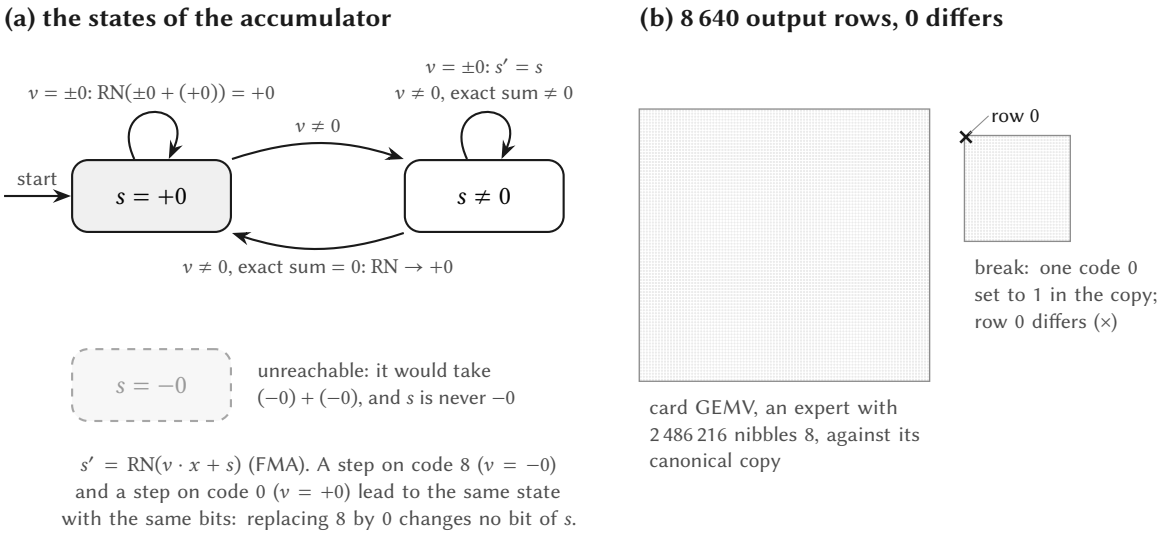


Figure 9.4: Replacing code 8 by code 0 changes no bit of the GEMV, and the instrument that says so can fail. [schem.] [meas.] Proposition A1. (a) Diagram: the states of the accumulator s of the engine's GEMV, which starts at +0 and advances by $s' = \mathrm{RN}(v \cdot x + s)$ in a fixed order; -0 is never reached, since IEEE 754 round-to-nearest returns +0 for an exact zero sum unless both terms are -0. (b) Measurement (test `gemv_lit_le_code_8_comme_0`, ok; counts from 06 § 1): a synthetic expert of gpt-oss size (three $2\,880 \times 2\,880$ matrices), inputs with full mantissas, exponents from 2^{-10} to 2^{10} and zeros of both signs, non-exact sums; one cell per output row. The small grid is the break (one code 0 set to 1 in the copy): row 0 differs, crossed. The proof assumes finite inputs; with $\pm\infty$ or NaN both outputs are NaN.

Data: `trc-noyaux/tests/carte.rs::gemv_lit_le_code_8_comme_0`, `mesures/genome/construction/codon/essais-espace-2026-10-17-a.log`; counts `genome/06-codon-v1-2.md` § 1, `decisions/ARCHITECTURE.md` § 4.16.

Table 9.1: The code readers and the value they give to code 8.

Reader	Where	Code 8	Status
ggml (llama.cpp b11515)	`ggml-common.h`, l. 1124 to 1129: `kvalues_fp4` (int8) 0, 1, 2, 3, 4, 6, 8, 12, 0, −1, … , −12	0 (integer)	read at the source on 2026-10-17, MIT licence [lit.]
engine GEMV	`trc-noyaux/noyaux/gemv.cuh`, `valeur_e2m1x2` (l. 33)	−0 (f32)	sole reader of the values in the kernels
CUDA decoder	`decodeur.cuh`	does not read the values	indifferent
f64 reference	`trc-reference/src/poids.rs`, `K_MXFP4[8]` (l. 24)	−0 (f64)	reads the GGUF, not the `.trc`
benches and controls	`banc_k.cu` l. 1049, `tests/carte.rs` l. 379	−0	controls only

Data: `genome/06-codon-v1-2.md` § 1; ggml: `https://raw.githubusercontent.com/ggml-org/llama.cpp/b11515/ggml/src/ggml-common.h`, SHA-256 0061131b…, accessed on 2026-10-17.

The measurement on the card. `trc-noyaux/tests/carte.rs::gemv_lit_le_code_8_comme_0` takes a synthetic expert with the dimensions of gpt-oss (three 2880×2880 matrices, that is $3 \times 2880 = 8640$ output rows) whose sign is drawn for every magnitude, magnitude 0 included: it carries 2 486 216 nibbles 8. It is compared with its canonicalised copy, with arbitrary x: full mantissas, exponents from 2^{-10} to 2^{10}, one x in 97 at $+0$ and one in 97 at -0 (the case that exercises step (2) of the proof), sums not exact in f32. Result: **8 640 rows, 0 differing at the bit**. Break: a code 0 of the copy, facing a non-zero x, set to 1, and row 0 differs [meas.] (`essais-espace-2026-10-17-a.log`).

The f64 reference. `trc-reference/tests/code_8.rs` takes an MXFP4 matrix of 64×2880 (a third of the magnitudes at 0, sign drawn) and its canonicalised copy: each dequantised weight is equal in value, and the outputs of `Matrice::produit` (eight accumulators started at $+0$, FMA, fixed tree reduction) are bit-equal on 5 vectors, zeros of both signs included; the break (a code 0 changed to 1) changes the output [meas.]. The reference reads the GGUF, it is not touched by the format; this control establishes only that it too gives code 8 the value of ggml.

Corollary 9.4 (The f64 reference). *In the f64 reference, each accumulator starts at* $+0$ *and advances by* $\text{acc} \leftarrow \text{RN}(d \cdot x + \text{acc})$, *with* $d = K[c] \cdot 2^{e-128}$ *the dequantised weight. If no non-zero exact sum* $d \cdot x + \text{acc}$ *rounds to zero, the induction of Proposition 8.9 applies in binary64: the accumulator is never* -0, *a step on a weight* ± 0 *leaves it unchanged, and the final reduction adds bit-equal accumulators.*

Proof. Steps (1), (2) and (3) of the proof of Proposition 8.9 use the grid only to establish that a non-zero exact sum does not round to zero; here, that is the hypothesis. The rest is the same induction. ■

The hypothesis of the corollary is not free, and it is a precision that this chapter adds to the note of lot A [calc.]. In the GEMV, the scale stays outside the accumulation and the magnitudes are integers: the grid $2^{-149}\mathbb{Z}$ always holds. In the reference, the scale is already in d; a product $d \cdot x$ smaller than half the smallest subnormal (2^{-1075}) would round to a signed zero, and the accumulator could be -0. That would require an activation of the order of 2^{-940} facing the smallest scale: nothing of the kind

exists in a model that returns finite logits, and the control measures it on its five vectors. But the proof for the reference holds under this hypothesis, not without it.

The limit of the proposition. It assumes the x finite. If an x is $\pm\infty$ or NaN, $v \cdot x$ is NaN for $v = \pm 0$ in both computations, and the output is NaN on both sides; only the sign bit or the payload of a NaN could differ, which is not a usable logit. The engine produced no NaN over the 10 240 steps evaluated by lot A.

The written invariant. Lot A carried it into the architecture (§ 5.6): every code reader reads code 8 as zero; every new kernel that reads codes (batched prefill, GEMM) keeps it, and its control compares an expert with its canonicalised copy.

Lemma 9.5 (The invariant of the tables is enough)**.** *If the table of a context gives $f[s] = 0$ to a symbol s, no stream, correct or corrupted, makes the normative decoder produce s.*

Proof. The decoder reads slot $i = x \bmod 4096$ of its state and returns the symbol that carries it. By construction of the LUT, slot i carries the symbol s such that $c[s] \le i < c[s] + f[s]$, with c the cumulative frequencies. If $f[s] = 0$, the interval $[c[s], c[s])$ is empty, and no slot carries s. The state x, whatever it is, and therefore whatever the words read, designates only one slot. ■

The compact LUT of the card and the normative LUT of the processor are built from the same frequencies, read in the same TABLES section; the lemma holds for both. It has an engineering consequence: to guarantee that no v1.2 stream produces a nibble 8, it suffices to check the tables at opening, once, and not every decoded symbol. The check costs, for the 120b, 36 layers × 3 matrices × 3 contexts × 31 bytes = 10 044 reads [calc.].

9.4 The `.trc` v1.2 format to the bit

The decision is written in the architecture (§ 4.16), because every modification of the format goes through that document. The brief spoke of "v1.1" for the current files: they are in v1.0, and v1.1 (two levels, the base of B and its complement) is reserved and written by no coder. All the comparisons therefore set v1.2 against v1.0; the reader accepts 1.0, 1.1 (header) and 1.2.

Table 9.2: What changes from v1.0 to v1.2, and nothing else.

Where	Offset	v1.0	v1.2
header, minor version	10 (u16)	0	**2**
header, flags	12 (u32)	bit 0 (aligned); bit 1 (levels); bit 2 (raw)	+ **bit 3 CANONIQUE**
directory, each entry	56 (u64)	reserved, 0	**`empreinte_canonique`**: empreinte64 of the canonical expert
CODES_CTX tables	—	256 frequencies	the same, built on the canonical codes; frequency 0 for the 31 bytes that carry an 8, checked at opening

Data: `decisions/ARCHITECTURE.md` § 4.16; `genome/06-codon-v1-2.md` § 2.

Figure 9.5 shows these four fields at their place in the file, to the offset; the rest of the file does not change.

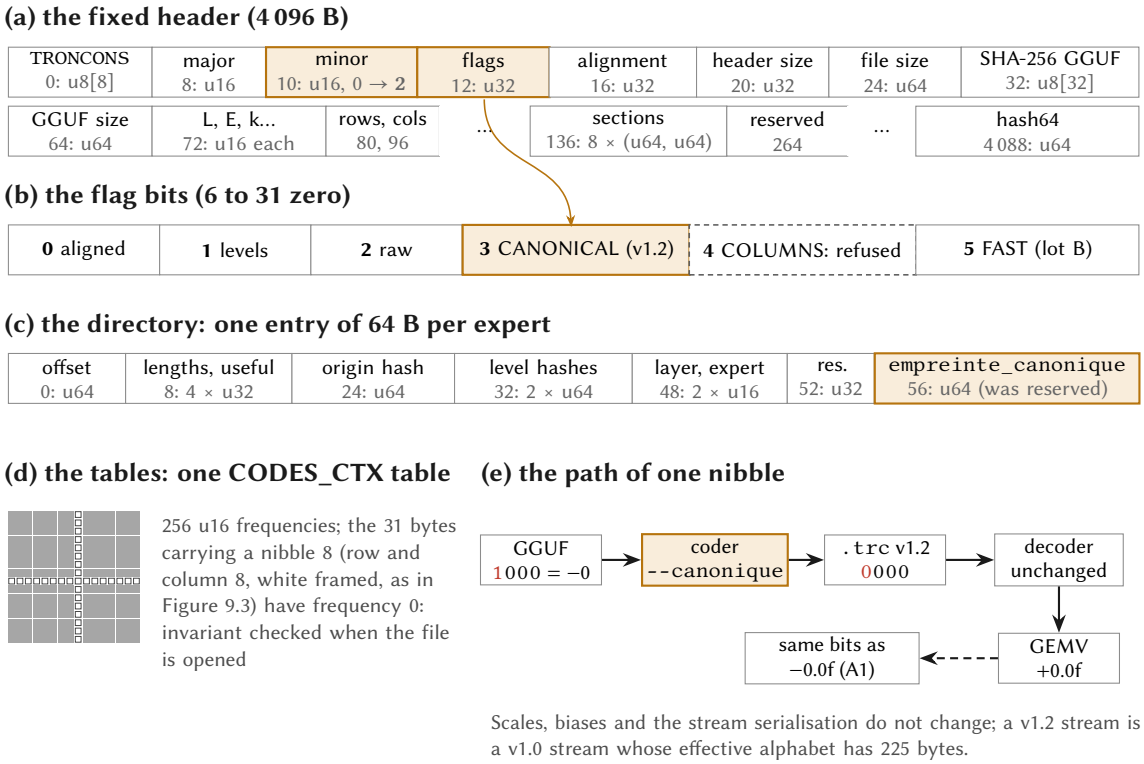


Figure 9.5: Four fields change from v1.0 to v1.2, highlighted in the orange of v1.2; the rest of the file is unchanged. [schem.] Second line of each box: the byte offset, then the type. (a) The minor version (offset 10) set to 2 and (b) bit 3 CANONICAL of the flags word; (c) the reserved field at offset 56 of each directory entry, which becomes the hash of the canonical expert (`empreinte_canonique` is the identifier of the code, kept as is); (d) the CODES_CTX tables, unchanged in form but built on canonical codes. In (a), the three sizes are the alignment (16), the header size (20) and the file size (24); L, E, k, from offset 72, are the layers, the experts per layer and the active experts; the rows and columns of the matrices are at 80 and 96, the table of the eight sections, (offset, length) pairs, at 136. Bit 4 (the column context of the plan) stays out of v1.2 and is refused (Figure 9.6); bit 5 is the fast companion of lot B, exclusive of bit 3. The origin hash (offset 24 of an entry) stays the hash of the GGUF bytes: a v1.2 file keeps the proof of its provenance. Field widths are not to scale.

Data: `decisions/ARCHITECTURE.md` §§ 4.3, 4.7, 4.16; `genome/06-codon-v1-2.md` § 2.

The canonical expert and its two fingerprints. The canonical expert is the byte sequence of the original expert (for $j = 0, 1, 2$, the $R \cdot B$ blocks of 17 bytes $[e, \texttt{qs}]$ in order β, then the biases) in which each nibble 8 of the `qs` is replaced by 0; the scales and the biases do not change. The directory keeps two fingerprints per expert: the original fingerprint (offset 24), that of the bytes of the GGUF, which keeps the proof of provenance; and the canonical fingerprint (offset 56), that of what the decoder must return. A v1.2 file is no longer bit-equal to the GGUF; it is equal *in value*, and the verifier must compare in value, not only in bytes (Section 9.6).

The refusals. `trc-format` refuses, each refusal with its control and its break in `tests/aller_retour.rs`: bit 3 under a minor version below 2; bit 3 on a raw file (the reference path of controls C3 and C12 keeps the bytes of the GGUF as they are) or on a two-level file (v1.2 does not define the canonical form of BASE_CTX and COMPL_CTX); a non-zero canonical fingerprint without bit 3; a CODES_CTX table of a canonical file that gives a slot to a byte carrying an 8; bit 4 (the COLONNES of the specification) and any version 1.3 or above.

What does not change. The serialisation of the streams, the normative decoding, the decoder kernels, the size of the tables (3×512 bytes per matrix), the format of the parts. *A v1.2 stream is a v1.0 stream whose effective alphabet has 225 bytes instead of 256.* This is why the card's decoder and the GEMV have not had a single line changed, and why the lot's risk lies in the coder, the verifier and the engine's control. The synonymous codon is carried into a format through the tables, not through the decoder: this is the engineering lesson of the lot, and it rests on lemma 9.5.

9.4.1 Why the column stays out

The specification proposed adding the input-column context (Section 8.6), "if it fits in the format without risk". Lot A left it out (answer by delegation of 2026-10-17, reversible; reason in one line: the gain does not pay for the risk).

The figure. On the 12 measured tensors of the 20b, only gate and up of layer 0 pass the threshold of 0.015 bit per weight, with −0.1416 and −0.1231 bit; layers 8 and 23 return 0.005 to 0.010 bit, layer 16 nothing (Figure 9.6, panel a). The negative bars of the panel are mostly the cost of moving from the byte alphabet to the nibble alphabet, 0.0029 to 0.0050 bit. Referred to the whole model, where each matrix of a layer counts 265 420 800 weights and the 20b $24 \times 3 \times 265\,420\,800 = 19\,110\,297\,600$ expert weights [calc.]:

$$\frac{(0.1416 + 0.1231) \times 265\,420\,800}{19\,110\,297\,600} = 0.0037 \text{ bit per weight}, \qquad \frac{0.2647 \times 265\,420\,800}{8} \approx 8.78 \text{ MB},$$

that is 0.08 % of the 11.2 GB of the file, against 0.128 bit for the zero (panel b). The price, on the other side: a new stream kind (geometry 2, twelve contexts per symbol), a new decoding kernel with its compact LUT, a new LUT mounting in the scheduler, a second independent decoder to extend, and a decoding 2.5 times more expensive on the matrices concerned (Section 8.7.1). Panel (c) gives again the lossless axis of the common frontier, which Figure 8.1 no longer carries: 3.8434, then 3.7064 with the zero, then 3.6844 with the column, on average over the 12 tensors. These 12 tensors count 3 tensors of layer 0 out of 12, against 3 out of 72 in the model: this is why the column gains 0.0220 bit on average there, against 0.0037 over the whole model. A sentence of the first version of the caption said that the column was "not a coded file": it was false (the gain comes from a real codec, rANS streams coded and decoded exactly); the judge flagged it (GN2) and the judged caption is corrected.

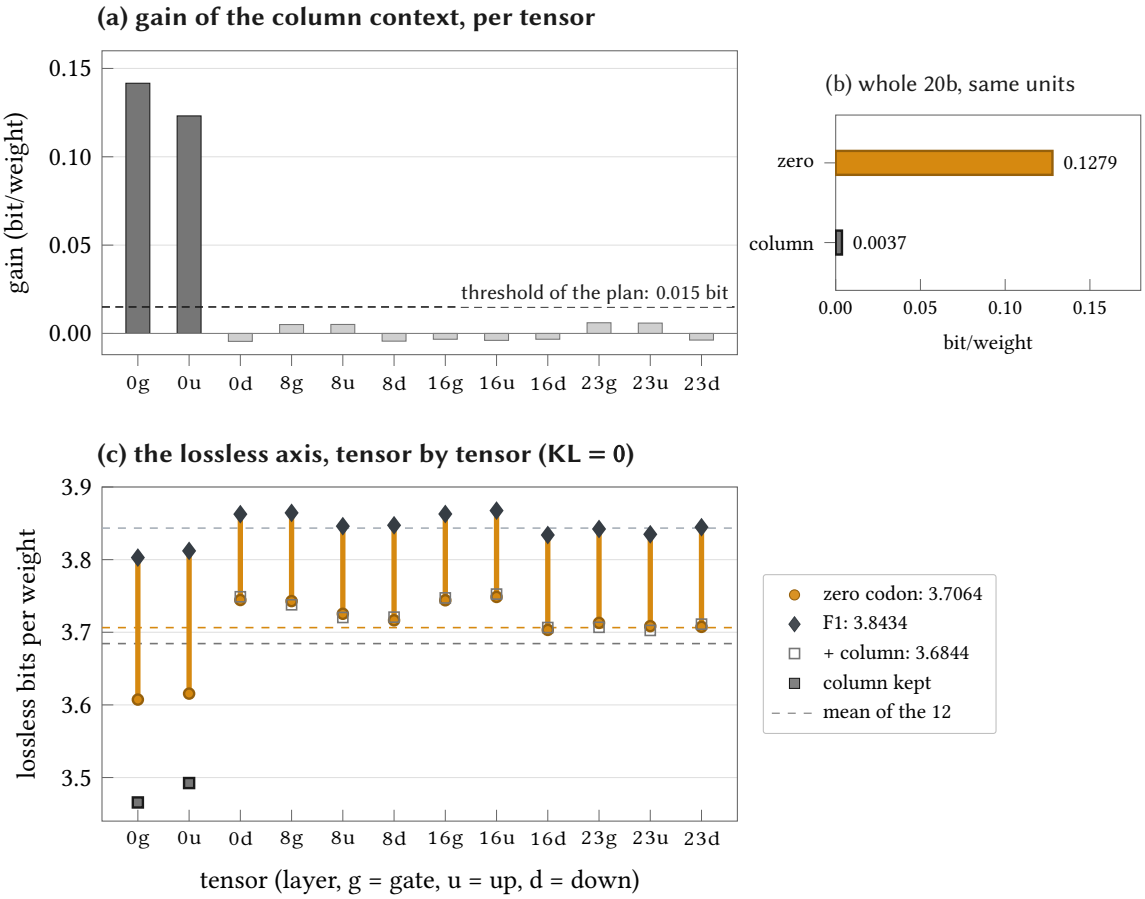


Figure 9.6: The column context gains on two tensors of layer 0; on the whole model it weighs little next to the zero codon. [meas.] [calc.] Twelve tensors of gpt-oss-20b (four depths only: 0, 8, 16, 23). (a) The rate gained by adding the input column class to the byte context κ, in bits per weight (real codec: streams coded by the rANS coder and decoded exactly; no `.trc` file written). This gain includes the cost of going from the byte alphabet to the nibble alphabet, 0.0029 to 0.0050 bit on the twelve tensors: the negative bars are mostly that cost. Only gate and up of layer 0 pass the threshold of 0.015 bit per weight declared by the plan (05 § 4.3); elsewhere the gain is at most a few thousandths, or negative. (b) In the same units, not on the same length: those two tensors brought back to the whole model, 0.0037 bit per weight (8.78 MB), against the measured gain of the zero codon, 0.1279 bit. (c) The lossless axis (KL = 0 exactly, logits equal to the bit), real codec: grey diamonds, the F1 frontier; orange, after the zero codon (the bar is its gain); dark grey squares, with the column context, filled where the rule keeps it; it loses on 6 of 12. Dashed: the means of the 12, the three lossless points of the common frontier of Chapter 8. These twelve include 3 tensors of layer 0 (3 of 72 in the model): on them the column gains 0.0220 bit on average, against 0.0037 for the whole model in (b). Not shown: the price on the other side, a new stream kind, a new decoding kernel and a second decoder to extend (06 § 2).

Data: `mesures/genome/construction/synthese/colonne-par-tenseur.csv`, `mesures/genome/construction/codon/tailles.json`; control `mesures/genome/construction/synthese/k10/frontiere.csv`; script `memoire/outils/tome3_codon.py`.

The three other answers by delegation of the lot, reversible: the CANONIQUE flag is refused on a raw file (raw is the reference path of the controls and must keep the bytes of the GGUF); `--canonique` is not the coder's default option (without it, it writes v1.0 byte for byte, which keeps the identity control); the new files are called `<modèle>-v12.trc`, next to the v1.0 ones, without erasing anything.

9.5 What was written, and its controls

Table 9.3: The files of lot A, and what changes in them. The CUDA decoder and the GEMV do not change.

File	What changes
`decisions/ARCHITECTURE.md`	§ 4.15 (v1.2 in the list), § 4.16 (the decision), § 5.6 (the invariant of code 8)
`trc-format/src/lib.rs`	version 2, `DRAPEAU_CANONIQUE = 8`; canonicalisation of a nibble and of a byte; refusal of bit 3; `empreinte_canonique`; check of the tables at opening
`trc-format/tests/aller_retour.rs`	v1.2 round trip; seven named refusals, two control acceptances
`trc-codec/src/lib.rs`, `tests/rans.rs`	`canoniser_qs` (returns the number of −0 folded); coding and decoding are those of v1.0
`trc-codeur` (`coder`, `verifier`, `main`)	option `--canonique`; two fingerprints per expert; check in value by a second path; break `--casser-8`
`trc-ordo/src/lib.rs`, `banc-chemin`	control mode against the canonical fingerprint (step loop and streamed prefill)
`trc-noyaux/tests/carte.rs`, `trc-reference/tests/code_8.rs`	Proposition 8.9 on the card; the reference and code 8

Data: `genome/06-codon-v1-2.md` § 3.

The fold itself fits in three functions, quoted as they are committed (Listing 9.1): a nibble 8 becomes 0, a byte is canonicalised nibble by nibble, and the coder canonicalises the `qs` of a matrix while counting the −0 it folds, the number that the control then checks. Nothing else of the coding changes. The code and its comments are quoted to the bit, so they stay in French: "le demi-octet canonique" is the canonical nibble, "l'octet canonique" the canonical byte, and the third comment says that each nibble 8 (−0) becomes 0 (+0) and that the function returns the number of nibbles changed.

Listing 9.1: The −0 → +0 fold in the code: `moteur/crates/trc-format/src/lib.rs`, lines 67 to 77, then `moteur/crates/trc-codec/src/lib.rs`, lines 317 to 327 (extracts to the bit, checked by line-by-line comparison).

```rust
/// Le demi-octet canonique : 8 -(0) devient 0 (+0) ; tout autre est inchangé.
#[inline]
pub const fn canoniser_demi(c: u8) -> u8 {
    if c == 8 { 0 } else { c }
}

/// L'octet canonique : ses deux demi-octets canonisés.
#[inline]
pub const fn canoniser_octet(o: u8) -> u8 {
    canoniser_demi(o & 15) | (canoniser_demi(o >> 4) << 4)
```

```rust
}

/// Canonise des `qs` en place : chaque demi-octet 8 -(0) devient 0 (+0). Rend le
    nombre de
/// demi-octets changés.
pub fn canoniser_qs(qs: &mut [u8]) -> u64 {
    let mut n = 0u64;
    for q in qs.iter_mut() {
        let c = canoniser_octet(*q);
        n += ((*q & 15) == 8) as u64 + ((*q >> 4) == 8) as u64;
        *q = c;
    }
    n
}
```

The tests. `cargo test --offline --workspace`, under the card lock: 82 tests, 0 failures [meas.] (`essais-espace-2026-10-17-a.log`). Each new control falls when it is broken (`mesures/genome/construction/codon/tests-casses.txt`): the check of the canonical tables removed makes `cassures_v12_refusees` fail; the check "canonical fingerprint zero without flag" removed, likewise; bit 3 accepted under v1.1 makes two tests fail; `canoniser_qs` made ineffective makes `matrice_canonique_se_reconstruit_et_gagne` fail.

The modified coder still writes v1.0 to the bit. The 20b recoded without `--canonique` by the modified coder has the SHA-256 `e8bd6ae1`… [121], that of `gpt-oss-20b.trc` recorded by lot F on 2026-10-15, by an independent path (`Get-FileHash`) [meas.] (`journaux/recoder.log`).

9.6 The two recoded files

Coder: `trc-codeur coder … --canonique --fils 6`, the SHA-256 of the GGUF recomputed by the coder and equal to the expected one; 361 s for the 20b, 1 395 s for the 120b. The v1.0 files stay in place; the v1.2 files are new, `gpt-oss-20b-v12.trc` (SHA-256 `ef1cf740`…) and `gpt-oss-120b-v12.trc` (`97c292ae`…).

C1 in value. 768 experts out of 768 and 4 608 out of 4 608 are exact, each of the eight checks at zero faults: fingerprint of the part, decoding, bytes against the GGUF canonicalised by a second path, fingerprint of the decoded against the canonical one, fingerprint of the raw slices against the origin, fingerprint of the canonicalised slices against the canonical one, no 8, equality in value [meas.] (`journaux/verifier-\{20b,120b\}-v12.json`; 274 and 781 s). Two breaks on the 20b: `--casser 5 7` (one bit of the first median flipped) makes exactly expert (5, 7) false; `--casser-8 11 3` (a nibble 0 of the decoded changed to 8) makes exactly expert (11, 3) false, with the "bytes", "fingerprint of the decoded" and "no 8" checks fallen, and the "value" check *held*. The instrument distinguishes equality of bytes from equality of values, and that is exactly what v1.2 changes.

The computations, posed. The gain per expert weight is the difference of the chunk bytes, in bits, divided by the number of expert weights [calc.]:

$$\text{20b} : \quad \frac{(9\,226\,493\,952 - 8\,920\,932\,352) \times 8}{19\,110\,297\,600} = \frac{2\,444\,492\,800}{19\,110\,297\,600} = 0.12791 \text{ bit},$$

$$\text{120b} : \quad \frac{(55\,416\,086\,528 - 53\,615\,599\,616) \times 8}{114\,661\,785\,600} = \frac{14\,403\,895\,296}{114\,661\,785\,600} = 0.12562 \text{ bit}.$$

Table 9.4: What the fold returns on the disk [meas.].

	gpt-oss-20b	gpt-oss-120b
expert weights	19 110 297 600	114 661 785 600
bits per weight, stream, v1.0 → v1.2	3.85057 → 3.72266	3.85456 → 3.72892
chunk bytes, v1.0	9 226 493 952	55 416 086 528
chunk bytes, v1.2	8 920 932 352	53 615 599 616
gain per expert weight (A-C2: ≥ 0.12)	**0.12791 bit**	**0.12562 bit**
bytes saved	305 561 600	1 800 486 912
ratio of chunks v1.2 / v1.0	0.96688	0.96751
`.trc` file, v1.0	11 157 405 696	57 730 662 400
`.trc` file, v1.2	10 851 844 096	55 930 175 488
largest chunk, v1.0 → v1.2	12 144 640 → 11 755 520	12 251 136 → 11 878 400

Data: `mesures/genome/construction/codon/tailles.json`, `tailles-par-tenseur.csv` (`outils/genome/construction/codon/analyse_codon.pytailles`; control: the sum per tensor equals the total of the log, and one added byte makes it fall).

A-C2, declared at 0.12 bit, holds on both. The specification expected ≈ 10.85 GB and ≈ 55.93 GB: the files are 10 851 844 096 and 55 930 175 488 bytes. The predictive coding team had measured 0.1280 and 0.1257 bit on streams per whole tensor; cut per expert in the format, one measures 0.12791 and 0.12562, the gap (less than 0.0001 bit) being that of the partial tiles and of the alignment. The speed ceiling that the bytes allow, with the disk limiting, is $1/0.96751 - 1 = 3.36\,\%$ on the 120b and $1/0.96688 - 1 = 3.43\,\%$ on the 20b [calc.]; this is the "3.4 %" of the specification.

The logits to the bit (A-C1). `troncons evaluer --flux`, 10 windows of 512 tokens, the 64-bit fingerprint of the f32 logits of each of the 5 120 steps.

Table 9.5: A-C1: the logits of v1.2 against those of v1.0 [meas.].

	gpt-oss-20b	gpt-oss-120b
control of the program: today's v1.0 against the CSV of 2026-10-15	5 120 / 5 120 equal	— (see the next row)
v1.2 against v1.0	**5 120 / 5 120 equal**	**5 120 / 5 120 equal** to the CSV of 2026-10-15
`.kld`	equal byte for byte	equal byte for byte to that of 2026-10-15
PPL of the scored positions	461.978	422.206
break (falsified median, layer 0, expert 30)	512 rows out of 512 differing	—

Data: `genome/06-codon-v1-2.md` § 7; `outils/genome/construction/codon/pilote_codon.py`, phases `c1-20b`, `c1-120b`.

On the 120b, the comparison is stronger than required: v1.2, evaluated by the streamed prefill with the program of 17 October, returns to the bit the logits that v1.0 returned through the step loop with the program of the 15th. The format, the program and the evaluation path changed; not one bit of the logits. A discovery on the way: the first pass of the break was refused in 8 s, before any logit, by the

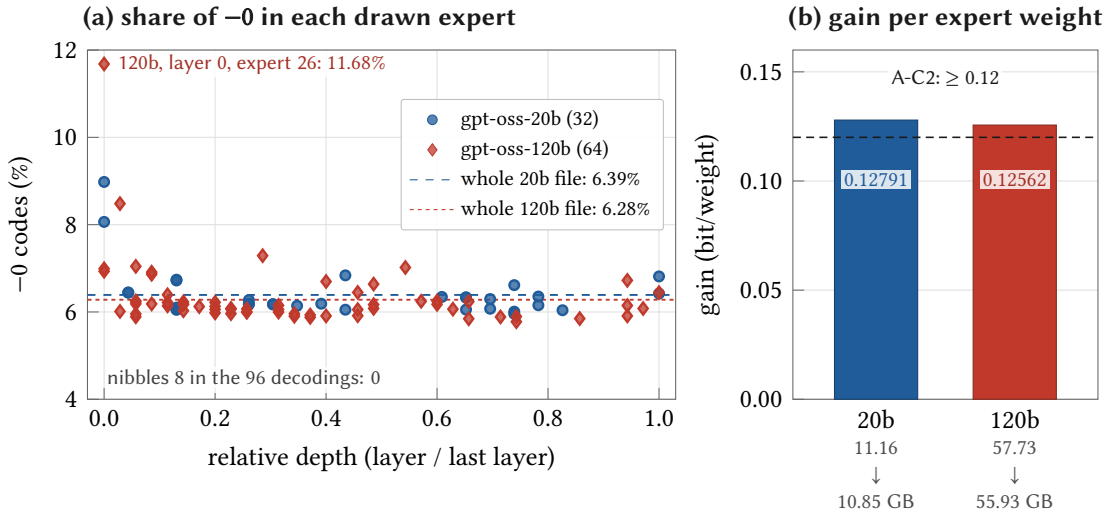


Figure 9.7: The judge's own decoder finds the canonical expert in all 96 drawn experts, and the gain of both files passes the threshold declared before. [meas.] (a) The 96 experts drawn from the two v1.2 files and decoded by the judge: share of −0 codes in the GGUF expert, by relative depth; dashed, the share over the whole file. All 96 decode to the canonical expert, with no nibble 8 left; the 40 v1.0 controls decode to the GGUF bytes (40/40). The set of nibbles changed between v1.0 and v1.2 is exactly the set of −0 (note of the judge § 1.2). The share of −0 varies by expert: the highest, 11.68%, is in layer 0 of the 120b; the file gain does not depend on it. The axis is drawn from the data: no expert lies outside the frame. (b) The gain per expert weight of the four files read whole (aligned chunks), their sizes below, against the threshold A-C2 declared before.

Data: `mesures/genome/construction/juge/J-T1-echantillon.csv`, `J-T0-fichiers.json`; `mesures/genome/construction/codon/tailles.json`; script `memoire/outils/tome3_juge.py`.

engine's production guard ("couche 0, expert 30 : empreinte des 11 234 960 octets utiles lus ... au lieu de ...", *layer 0, expert 30: fingerprint of the* 11 234 960 *useful bytes read ... instead of ...*); it is a control of the guard, kept, and the break of the logits was redone with `--sans-garde`, a declaration added to the pilot before the second pass. A break must get through all the guards upstream of the instrument it tests.

9.6.1 The codon through the judge's decoder

The judge of the construction (Chapter 11) had built nothing. It wrote its own decoder from the description of the format alone, and its own GGUF reader, without opening the lot's code [meas.] (`genome/JUGEMENT-construction.md` § 2).

- **J-T0, the four whole files**: 0 structural faults; the eight sizes of Table 9.4 given again to the byte; 0 CODES_CTX tables out of 540 in v1.2 with a slot for an 8 (540 out of 540 in v1.0).
- **J-T1, 96 drawn experts** (32 of the 20b, 64 of the 120b, draw committed before any decoding): 96 out of 96 hold the eight checks; the v1.0 control, 40 out of 40; the nibbles that change between the v1.0 decoded and the v1.2 decoded are exactly the −0 of the GGUF, not one more nor one less (for example 1 746 601 changed for 1 746 601 −0 in expert 109 of layer 19 of the 120b); three breaks fall. The share of the −0 in the sample, 6.42 ± 0.11 % (20b) and 6.33 ± 0.10 % (120b), is at 0.3 and 0.5 standard deviation from that of the files; the highest, 11.68 %, is at layer 0 of the 120b (Figure 9.7).
- **J-C1, on ten windows that no one had seen**: 0 differing logit fingerprints out of 5 120 on the 20b, 0 out of 5 120 on the 120b; `.kld` equal; break (the sign of the bias of row 0 of gate flipped for

the 32 experts of layer 0): 512 out of 512 differing from position 0. Proposition 8.9 is true of the real engine, on new text.

What the sample does not cover, the judge writes: the probability that a defect touching at least 5 % of the experts escapes 32 draws is $0.95^{32} \approx 19\,\%$, at 64 draws $0.95^{64} \approx 3.7\,\%$ [calc.]. The verifier of lot A covered 100 % of the experts by its two paths; the draw judges this verifier, it does not replace it.

9.7 Milestone J1 redone on the 120b

The protocol. Eight fresh processes, A B B A A B B A, A = `gpt-oss-120b.trc` (original v1.0, written on 15 October), B = `gpt-oss-120b-v12.trc`; `mesurer--flux--emplacements326`, the wiki-3000 prompt, 256 greedy tokens; on mains power, no llama.cpp before each pass, card lock. The number of cache slots is fixed at 326, that of J1: otherwise it is computed at opening from the free VRAM and the size of the arrival buffers (12× the largest chunk), and one more slot would change the cache, hence the bytes read, and would mix two effects. Criteria, declared beforehand (commit `a50c380`): A-C3, no regression if $r \geq 1 - 2\sigma$, gain announced only if $r - 1 > 2\sigma$, σ the relative standard deviation of the four v1.0 passes; equal tokens; ratio of bytes published (predicted 0.967); peak memory $|B - A| \leq 2 \times \text{noise} + 0.5$ MB.

Session s1, under load. On 17 October, from 10:24 to 10:50, while other teams were reading the disk: medians 1.558 and 1.732 tokens/s, $r = 1.112$, but $\sigma = 21.6\,\%$ (passes 5 to 7 are all slow, whatever the file). No regression; gain not announced ($0.11 < 2\sigma = 0.43$). In the first four passes, calm ones, v1.2 ran 1.4 to 1.7 times faster: 3.3 % fewer bytes cannot return 40 %. The lot looked for a control that judges the format alone: a fresh copy of v1.0, then v1.0 recoded by the same coder on the same day as v1.2, with the same content to the bit (SHA-256 equal to that of the original v1.0). Both turned out to be confounded by their own fragmentation, recorded by `fsutil file queryextents` [122]: 289 545 fragments for the copy, 47 745 in the sample for the recoded v1.0, against 65 for the original v1.0 and 233 for v1.2.

Session s2, on a calm machine. Declared before its passes (commit `71905dd`); night of 21 to 22 October; none of the 22 passes under load.

Table 9.6: J1 of the 120b, session s2, in ABBA [meas.]. A = original v1.0, B = v1.2.

pass	1 A	2 B	3 B	4 A	5 A	6 B	7 B	8 A
tokens/s	1.981	2.873	2.892	1.994	2.006	2.897	2.887	1.776
throughput seen by the engine (GB/s)	1.81	2.63	2.66	1.82	1.83	2.67	2.65	1.61

Data: `mesures/genome/construction/codon/j1-s2-passes.csv`, `j1-s2-verdicts.json`.

- **Tokens**: the 256 identifiers equal in the eight passes, text `e4c6dbac`..., that of s1 and of all the passes of the 120b of 15 and 16 October; 59.08 misses per token, cache hit rate 0.5897, 326 slots in all passes. The format changes neither the routing nor the cache.
- **Bytes** of chunks per token, deterministic: $686\,756\,916 / 710\,118\,798 = 0.9671$, for a prediction of 0.967; the ratio of the chunks of the whole file is 0.9675, and the experts that this text calls are slightly richer in zeros than the average.
- **Speed**: medians 1.988 (v1.0) and 2.890 (v1.2) tokens/s, $r = 2.890/1.988 = 1.454$ [calc.], $\sigma = 5.6\,\%$; paired ratios 1.450, 1.451, 1.444 and 1.626. At the declared threshold: no regression, and **gain announced** ($r - 1 = 0.454 > 2\sigma = 0.113$).

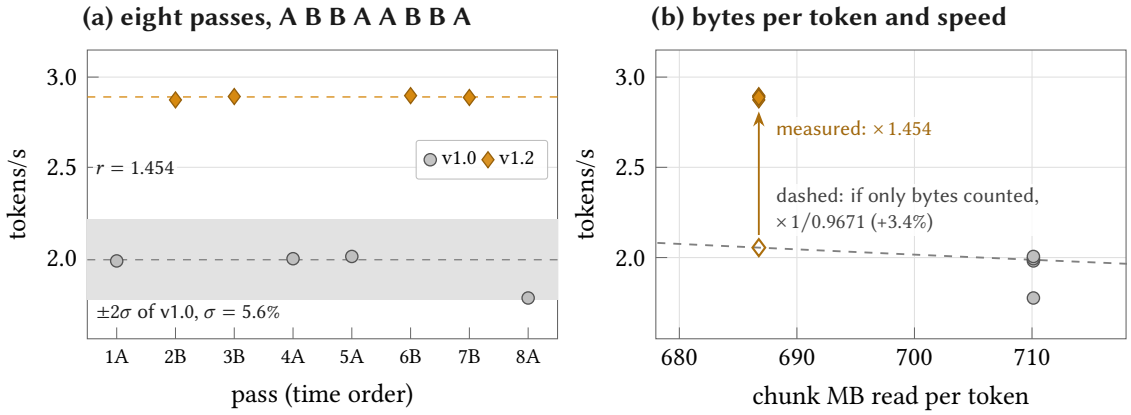


Figure 9.8: On the 120b, v1.2 runs ×1.454 faster than v1.0, and its bytes would give it only +3.4% at most. [meas.] gpt-oss-120b, session s2 of lot A, quiet machine, eight fresh processes, the same 256 tokens in all. (a) The passes in time order: medians 1.988 (v1.0) and 2.890 (v1.2) tokens/s, dashed; grey band, $\pm 2\sigma$ of the v1.0 passes. (b) The bytes read per token are deterministic, ×0.9671; if only their amount counted, v1.2 would sit on the dashed curve, at the hollow diamond (+3.4% at most). Where the rest comes from: Figure 9.9.

Data: `mesures/genome/construction/codon/j1-s2-passes.csv`, `j1-s2-verdicts.json`; script `memoire/outils/tome3_codon.py`.

- **Peak memory**: medians 388.67 and 382.21 MB, −6.45 MB (s1: −6.41). 4 472 832 bytes are explained by the 12 pinned arrival buffers, sized on the largest chunk: $12 \times (12\,251\,136 - 11\,878\,400) = 4\,472\,832$ [calc.]; the rest, ≈ 1.9 MB, is not explained. The threshold holds in the literal sense only because the first pass of the session (468.4 MB) makes a noise of 79.8 MB; without it, the v1.0 passes range from 388.60 to 388.70 MB and the gap is clear.

Figure 9.8 shows it in two panels: the eight passes in time order, and, on the right, the bytes against the speed. If the quantity of bytes alone counted, v1.2 would be at the hollow diamond, at 3.4 % at most. It is at 45 %.

9.7.1 Where the gain comes from: the pattern of read requests

The disk stage alone. In the same session s2, `troncons disque --n 1500`, without computation, A B C C B A, reads A (original v1.0, 65 fragments) at 1.742 and 1.741 GB/s, B (v1.2, 233 fragments) at 2.254 and 2.391, and C (the recoded v1.0, bit-equal to A) at 1.356 and 1.331 [meas.] (`disque-s2.json`). J1 redone with C against B gives $r = 1.595$, $\sigma = 4.2$ %.

The first reading, and its strike-through. Lot A had drawn a cautious conclusion from it, which the judge of the construction had reproduced, and which is false in its attribution:

> ~~This gain is not attributable to the format. Two files with the same content to the bit (A and C) are read at 1.74 and 1.34 GB/s at the disk stage, and return 1.99 and 1.81 tokens/s: the physical layout of a file on this NVMe moves the speed by 30 % without touching one byte of the content. [...] The share of the format, for its part, is bounded by the ratio of the bytes read: at most +3.4 % with the disk limiting.~~

What changed, on 2026-10-27: the judge of the disk placement (`genome/JUGEMENT-placement.md`) read *one and the same file* with the request boundaries of the other format, without rewriting anything. The bound of 3.4 % holds only for the bytes; it does not bound the effect of the pattern. The gap between

A and C, for its part, has another cause: the extreme fragmentation of C. `06-codon-v1-2.md` § 8, the lot's note for the thesis and the judgement of the construction (§§ 0 and 6) carry the same correction; ~~one row of the table of § 8 of that judgement keeps the old attribution without a strike-through~~, and this chapter follows the correction (Section 9.9). Since 2026-10-29, that row is struck through too, with what changed (`genome/JUGEMENT-construction.md` § 8).

Table 9.7: The crossed pattern (gpt-oss-20b, disk stage alone, session J-bis of the placement judge, 26 October): the same file read with the request boundaries of the other format, GB/s [meas.].

Case	Passes	Median
A20, native pattern (v1.0)	1.792; 1.793; 1.813; 1.809	1.801
A20, pattern of B20	2.509; 2.493; 2.483; 2.564	2.501
B20, native pattern (v1.2)	2.511; 2.487; 2.556; 2.545	2.528
B20, pattern of A20	1.811; 1.829; 1.805; 1.774	1.808

Data: `mesures/disque/placement/juge/seance/Jb3-*`; `genome/JUGEMENT-placement.md` § 5.3.

What the crossed pattern shows. At the thresholds declared beforehand ($e_{\text{self}} \leq 3.2\,\%$, $S_j = 11.8\,\%$), A20 read with the pattern of B20 runs ×1.389 faster than read with its own, and B20 read with the pattern of A20 runs ×0.715: *slowness follows the pattern*, not the file. B20, the v1.2 file, read with the boundaries of v1.0, runs like A20 (1.808 against 1.801). Two more experiments make it precise. J9: A20 read with requests shortened by 3.5 % runs ×1.421, B20 with requests lengthened by 3.5 % ×0.713; the step of the destination window does not decide. J10 and J11: cutting each v1.0 chunk into two requests of 6 MB, same bytes, does not make it fast, it slows it down (×0.851 at eight reads in flight, ×0.882 at four): *length alone is not the variable.* What separates the slow patterns from the fast ones, in the judge's eight cases, is descriptive: slow are the requests that end exactly at the end of a v1.0 chunk, or that overrun the end of a v1.2 chunk by 3.5 %; fast are those that stop before. This description is not a mechanism; the exact variable is not isolated.

The cascade, in a three-file session. The judge of the construction put the three files of the 120b into a single balanced plan, A B R R B A A B R R B A, A = original v1.0, B = v1.2, R = the fast frontier companion; none of the 36 passes under load [meas.] (`genome/JUGEMENT-construction.md` § 6.2). B against A: medians 2.889 and 2.004 tokens/s, $r(B/A) = 1.442$, $\sigma = 5.8\,\%$; the J1 of lot A (1.454) is reproduced. The disk stage alone, in the same session, reads B 1.37 times faster than A. Panel (b) of Figure 9.9 splits this ratio into two factors [calc.]:

$$r(B/A) = \underbrace{\frac{1}{0.9671}}_{\text{bytes: } 1.034} \times \underbrace{\frac{1.442}{1.034}}_{\text{remainder: } 1.394} .$$

The first factor is a ceiling: what the bytes saved would return if the disk alone limited. The second is a remainder, hence a lower bound on the effect of the pattern. The fast companion adds ×1.284 against v1.2; that is the business of Chapter 10, and its quality is measured only against the exact engine of the 120b, a proxy.

The recoded v1.0. The inset of panel (a) settles the case of C: A120 and C120 have the same content to the bit and the same directory (same SHA-256, same pattern); C120 is slower because of its 267 473

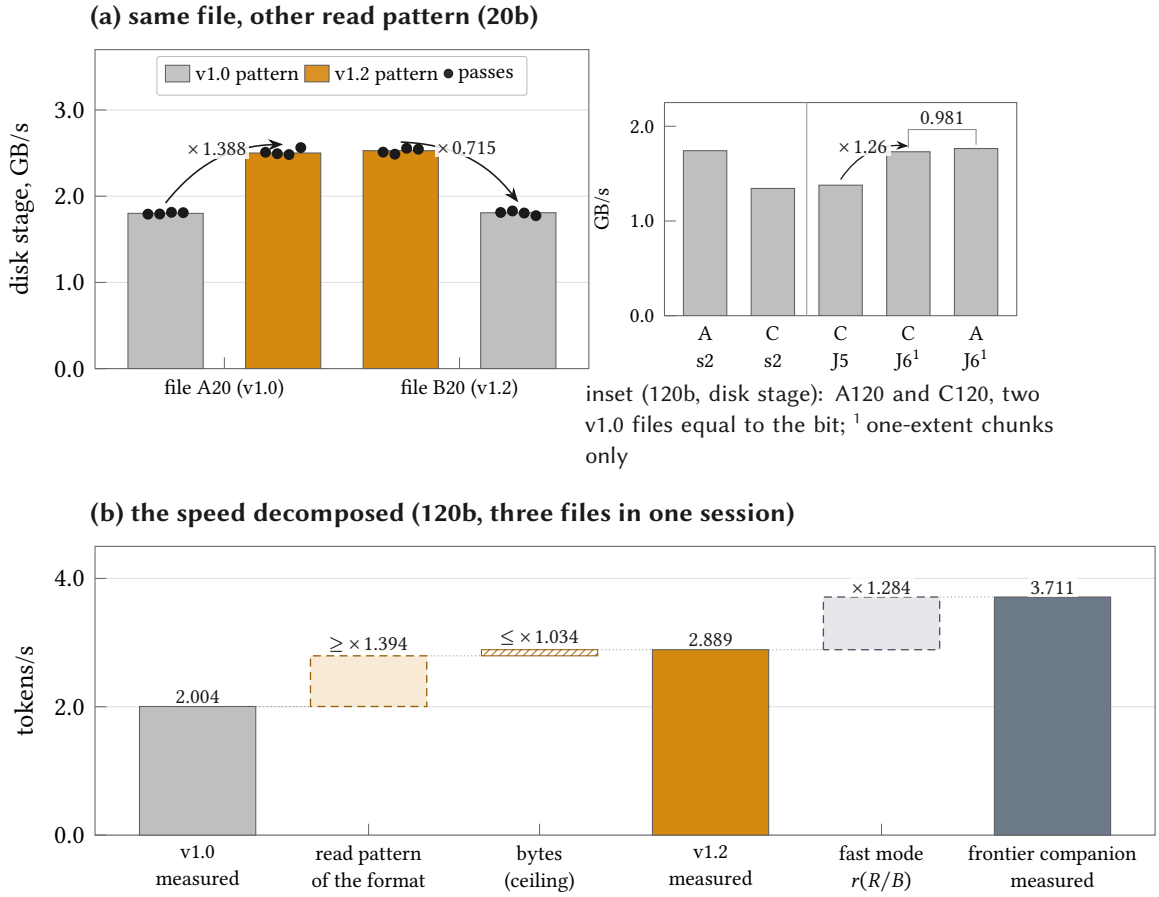


Figure 9.9: The gain comes from the read pattern that the v1.2 format imposes, on this disk; the 20b and the 120b share the figure by panel. [meas.] The sequel of Figure 9.8. (a) gpt-oss-20b, disk stage alone, session J-bis of the judge of the placement: the same file read with the request boundaries of the other format takes the speed of that format, A20 × 1.388, B20 × 0.715 (bars: medians; dots: the four passes of each case, none set aside; colour: the read pattern; abscissa: the file). Inset, gpt-oss-120b: A120 and C120 have the same content to the bit; in s2 C120 is slower because of its fragmentation (267 473 extents against 65). On its one-extent chunks (J6, 800 chunks) it reads × 1.26 faster than on 1 200 of its chunks drawn at random (J5; medians of 2 and 3 passes), and like A120 on the same chunks (ratio 0.981 of the medians; 0.980 of the means, the value the judge of the placement writes). The s2 bars come from `troncons disque`, the J5 and J6 bars from the judge of the placement's Python reader, 2 to 5% slower (`genome/JUGEMENT-placement.md` § 5.2): each ratio written compares two bars of the same reader. (b) The judge of the construction's balanced session with the three files of the 120b: $r(B/A) = 1.442$ splits into the bytes (hatched, a ceiling) and the read pattern of the format, a remainder and therefore a lower bound (the disk stage alone, in the same session, gives $B/A = 1.370$); the fast companion, the lossy frontier companion of the 120b (its KL, 0.140 ± 0.032, 2σ by windows, is measured against the 120b's exact engine, a proxy, not against the f64 reference; Figure 10.6 shows the variants of the 20b), adds × 1.284 against v1.2. The J1 gain is a gain of the v1.2 format on this machine, through the pattern of its read requests, not through its bytes; the exact variable of the pattern is not isolated (J10, J11). It does not carry over to another disk without being measured there again.

Data: `mesures/disque/placement/juge/seance/Jb3-*`, `J5-*`, `J6-*`, `J1-cartes.json` (`genome/JUGEMENT-placement.md` §§ 5.3, 5.9–5.10); `mesures/genome/construction/codon/disque-s2.json`; `mesures/genome/construction/juge/figures/vitesse-passes.csv`, `disque-passes.csv` (`genome/JUGEMENT-construction.md` § 6.2); script `memoire/outils/tome3_codon.py`.

extents, against 65 for A120. On its 800 single-extent chunks, it is read ×1.26 faster than on 1 200 of its chunks drawn at random, and like A120 on the same chunks (0.980 in the ratio of means) [meas.] (`genome/JUGEMENT-placement.md` §§ 5.9 and 5.10). The two causes are therefore separated: fragmentation explains A against C; the pattern explains A against B.

What J1 says, in one sentence. On this laptop, the v1.2 format makes the engine about 45 % faster on the 120b, through the pattern of its read requests and not through its 3.3 % of bytes; this gain does not carry over to another disk without being remeasured there. What the lot had proposed for the amplification from 1.33 (disk stage) to 1.45 (engine) remains a hypothesis: the card, which waits more on the disk, would run at lower clocks; the card times measured by events (copy 91 to 99 ms per token against 66 to 68; decoding 61 to 67 ms against 42 to 44) vary much more than the bytes.

9.8 What remains

- **Isolate the variable of the pattern**, at the disk stage, by patterns declared beforehand: each v1.0 chunk read as one request of 0.965 *l* then a request for the end; shortened from the start; the pattern of B20 shifted by 64 KiB; the same on A120 and C120 (`genome/JUGEMENT-placement.md` § 12). A read lever can be measured end to end only once it is found. The "decisive" control that lot A proposed (two fresh copies written on the same day, about 113 GB free) is no longer needed: the crossed pattern, one and the same file read with both patterns, stands in for it without rewriting anything.
- **Remeasure** the speed gain on any other disk: the figure of this chapter is that of this NVMe, behind this driver, under this encryption.
- **A second independent decoder**: the specification asked for the decoder of trial J1 of Volume II to be extended to v1.2; the judge of the construction wrote its own from the description of the format alone, and it reads v1.2. This chapter takes it as the second path (the writer's decision, reversible); the trial's decoder, for its part, is not extended.
- **Decide** whether the v1.2 files replace the v1.0 ones as the engine's default files: this is a product choice, left to Stone; no file has been erased, and 57.7 + 11.2 GB remain occupied by the v1.0 files.

9.9 What could be wrong

What could be wrong

- **Proposition 8.9 assumes finite** x and the kernel as it is at commit `000cef9`: integer doubled magnitudes, scale outside the accumulation, start at +0. A future kernel that read the codes otherwise (a table where entry 8 were worth something other than ±0, a GEMM starting from a −0 accumulator, a scale inside the accumulation) could break the equality; the control to take up again is `gemv_lit_le_code_8_comme_0`.
- **Corollary 9.4** holds under a no-underflow hypothesis that neither the lot nor the note stated; it is harmless for model activations, but it is not proved for every input, only measured on five vectors.
- **The gain is that of gpt-oss.** It is twice the share of the −0; an MXFP4 model quantised by a tool that always writes +0 has nothing to gain (v1.2 costs it zero bytes); a model where the sign of a zero were predictable would return less. The 180 tensors say that this is not the case

here.

- **The equal slots of J1** isolate the format, not the product: with the default number of slots, the largest chunk shorter by 372 736 bytes frees $12 \times 372\,736 = 4\,472\,832$ bytes of VRAM, less than one slot (13 254 144 bytes); the default therefore probably does not change, but this is not measured.
- **The attribution of the speed gain** rests on the crossed pattern of the placement judge, measured at the disk stage on the 20b; on the 120b, the cascade of the judge of the construction gives only a lower bound on the effect of the pattern (the remainder after the ceiling of the bytes). The exact variable of the pattern is not isolated, and its mechanism (storage stack, controller, fallback outside the SLC) is not observable from here.
- **One row of the "what holds" table of the judgement of the construction** (§ 8) ~~still writes "pour les fichiers, pas pour le format" (*for the files, not for the format*), without a strike-through, whereas its §§ 0 and 6 are corrected~~; corrected on 2026-10-29: it is struck through, with what changed; this chapter follows the correction, after `genome/JUGEMENT-placement.md` § 12 point 3.
- **One session per speed measurement.** The ratios hold for the state of these files on those days; the crossed pattern was measured four days after the judge's main session, after a re-optimisation of the disk, and the B20/A20 ratio did not move (1.398 and 1.362 on the 22nd; 1.404 and 1.406 on the 26th), which reassures without proving.
- **The rest of the memory** ($\approx$ 1.9 MB out of 6.45) is not explained; the hypothesis (other buffers sized on the largest chunk) is not verified.
- **The column context** is left out on a figure measured on 12 tensors of 4 layers of the 20b; on the 120b and the other layers, its gain is not measured.
- **The codon analogy** is an analogy: a synonymous codon of the genetic code is not always neutral (codon usage bias, speed of translation); here, the sign of a zero is exactly neutral, by Proposition 8.9.

Chapter 10

The synonymous genome, fast mode (lot B)

Where we start, what this chapter establishes, what remains

At the start, the synonymous mutation (choosing the base symbol of each weight by an error feedback weighted by the real inputs, under an entropy constraint) was measured only on 32 experts out of 768 of gpt-oss-20b: the variant $\lambda = 0$ in end-to-end KL ($-0.009\,8 \pm 0.003\,3$, paired, against the frontier), the variant $\lambda = 1$ in expert output error only (Chapter 8). No file, no kernel, no engine option served it. **This chapter establishes** the synonymous mutation written for the 768 experts, its end-to-end KL with all experts substituted, the settling of the two variants $\lambda = 0$ and $\lambda = 1$ by a rule declared beforehand, and the *fast mode* of the engine: a companion file `<nom>.rapide.trc` decoded by the decoder of the exact mode, a `gemv_base` kernel, a `--rapide` option, each with its control. It also establishes, through the judge, what the fast mode does not return: two speed gains published by the lot, struck through as gains of the mode. **What remains** is the synonymous mutation of the 120b (neither calibration nor f64 judge), a corpus other than wiki.test, the tail of the KL (the routing flips) and the decision on the mixed miss policy, to be taken up again with the judge's ratios (Section 10.9). Sources: `genome/07-genome-synonyme.md`, `genome/construction-synonyme-pour-le-memoire.md`, `genome/JUGEMENT-construction.md`, `mesures/genome/construction/synonyme/` (thresholds: `00-seuils-declares.md`) and `mesures/genome/construction/juge/`.

In brief

In living things, a mutation is called *synonymous* when it changes one letter of a codon without changing the amino acid that the codon designates: the sequence changes, the protein does not. The image serves here as an analogy, and as nothing more. A matrix of a gpt-oss expert is judged only by what it does with its real inputs; a weight error in a direction that these inputs never excite hardly changes the output. The synonymous mutation therefore chooses, weight after weight, an approximate version of the matrix (its *base*, three symbol bits per weight) whose error goes precisely into these silent directions. It invents no new decoder: same eight symbols, same four levels, same scales as the "frontier" base of Volume II, where the symbol is a simple function of the code. All the work is in the coder. Over the 768 experts of the 20b, all served by their base, the end-to-end KL falls from 0.264 0 (frontier) to 0.131 8 (synonym, $\lambda = 0$), for 2.67 bits per expert weight instead of 3.86 for the exact file; a judge that did not build it reproduces it on ten windows of text that no one had seen. The engine serves this fast mode, on the card, as faithfully as the double-precision reference. What it does not return is speed in generation: with comparable files, ×1.28 on the 120b and ×0.94 on the 20b; the ×2.00 and ×1.24 first published compared a new companion with an old exact file that this disk reads more slowly because of

> the pattern of its requests.

10.1 What is built

The vocabulary. The exact format of Volume II serves each expert weight in MXFP4: a four-bit E2M1 code (one sign, three magnitude bits) under an E8M0 scale common to a block of 32 weights [123]. The *base + complement* split of Volume I sorts the eight E2M1 magnitudes into four contiguous *classes*; the *base* of a weight is its sign and its class $k \in \{0, 1, 2, 3\}$ (eight symbols, three symbol bits), and the *complement* says which magnitude of the class it carries. Serving the base alone means replacing each weight by $\pm c_k \cdot s_b$, where c_k is the level (the centre) of the class and s_b the scale of the block: this is the *fast mode*, lossy, whose interest is to read fewer bytes. Two ways of choosing the base:

- the **frontier**: the symbol is a function of the exact code (the class where its magnitude falls); this is the base of Volume II, without calibration;
- the **synonym**: the symbol is chosen by the coder so that the *output* error is small on the real inputs, even if that means taking a class neighbouring that of the code; the decoder does not see the difference.

Two settings of the synonym are measured: $\lambda = 0$, the error feedback alone, and $\lambda = 1$, the same under an entropy constraint that penalises rare symbols (Section 10.2.2). Both carry a damping $\delta^\star = 1$ of the input matrix (Equation (10.2)).

The pieces. Table 10.1 gives them, each with the control declared before its measurement (`mesures/genome/construction/synonyme/00-seuils-declares.md`, commits of 17 October 2026 between 8:19 and 12:14, then the amendment of the speed session on the 21st at 23:59).

A decision, and its price. The construction specification planned a coder in Rust on the card. The lot coded the 768 experts with the scout's Python code, without changing a line of it, at about 54 s per expert on one process, three to six processes in parallel (*answer by delegation of 2026-10-17, reversible*). The reason: it is the only way that returns *exactly* the variant whose error had been measured, and W1 proves it to the bit. The price is the 120b, where this coder would cost tens of hours (Section 10.8).

10.2 The mechanism

10.2.1 The output error and the input matrix

Let an expert matrix $W \in \mathbb{R}^{R \times C}$ ($R = C = 2\,880$ for the three matrices gate, up and down of gpt-oss, [6]), its real inputs $x \in \mathbb{R}^C$ and the weight g that the router gives the expert for each of them. The error made by a reconstruction $\hat{W}$, at the output and over the input distribution weighted as the model weights the expert, is

$$\mathscr{E}(\hat{W}) = \frac{\mathrm{E}\left[g^2 \,\|(W - \hat{W})\,x\|^2\right]}{\mathrm{E}[g^2]} = \mathrm{tr}\left(\Delta W \, H \, \Delta W^\top\right), \qquad \Delta W = W - \hat{W}, \tag{10.1}$$

with

$$H = \frac{\mathrm{E}[g^2\, x x^\top]}{\mathrm{E}[g^2]} \qquad \text{and, for the coder,} \qquad H_\delta = H + \delta^\star \, \overline{\mathrm{diag}\, H}\; I, \quad \delta^\star = 1. \tag{10.2}$$

Table 10.1: The pieces of lot B, each with the control declared before its measurement and the result [meas.].

Piece	What it does	Control and break	Result
coding of the 768 experts	`coder_768.sh` calls `synonyme.py ecrire-sym`, the scout's code, *unchanged*	W1: redoing L00-E00 ($\lambda = 0$) returns the scout's file byte for byte; break: $\delta = 0.99$	equal SHA-256 (`9079dcf1`…); break: 117 501 differing symbols
second path for the symbols	`verifier_sym.py` rereads the `.sym`, reconstructs, recomputes the published errors	W2: equal to the scout's log to within 10^{-9} relative (16 experts, layers 0, 8, 16, 23); break: one class bit on one row	maximal gap $4.4 \cdot 10^{-16}$; break: $7.2 \cdot 10^{-3}$, falls
fast-mode kernel	`gemv_base` (`gemv.cuh`, value reader as a parameter)	card against f64 reference on a case exact by construction (Proposition 10.4); exact mode before and after	0 differing rows out of 2 880; exact mode identical to the bit
RAPIDE flag	bit 5 of the header, version 1.2	refusal under 1.2, with raw, with canonical	4 cases out of 4
packer and verifier	`trc-rapide coder`, `trc-rapide verifier`	S3-T1 (Section 10.6); break `--casser C X`	the targeted expert, and it alone
engine option	`troncons … --rapide`	a fast file without `--rapide`, an exact one with it	both refused, named error

Data: `mesures/genome/construction/synonyme/codage/W1-reproduction.json`, `codage/W2-*.json`, `rapide/refus-*.log`; test `gemv_base_exacte_au_bit_et_mode_exact_inchange`; test `drapeau_rapide_v12`.

The computation. $\|\Delta W x\|^2 = x^\top \Delta W^\top \Delta W x = \operatorname{tr}(\Delta W\, x x^\top \Delta W^\top)$; trace and expectation are linear, hence $\mathrm{E}[g^2 \|\Delta W x\|^2] = \operatorname{tr}(\Delta W\, \mathrm{E}[g^2 x x^\top]\, \Delta W^\top)$, and dividing by $\mathrm{E}[g^2]$ gives (10.1). The trace splits into rows: $\operatorname{tr}(\Delta W H \Delta W^\top) = \sum_r \delta_r^\top H \delta_r$, where δ_r is row r of ΔW. *The rows are therefore independent problems*, and the coder treats each row w alone: $\min_q (w-q)^\top H (w-q)$, q ranging over the reconstructions allowed column by column.

The calibration inputs are those of the scout: windows 10 to 21 of wiki.test (6 132 positions), disjoint from the ten evaluation windows R1 (windows 0 to 9), and weighted by the square of the router weight. A given expert sees only the positions that the router sends it: 1 329 for expert 0 of layer 8, in dimension 2 880. Its matrix H therefore has rank at most 1 329, and that is what makes the damping necessary (Section 10.2.3).

10.2.2 The choice of the symbols

Lemma 10.1 (Optimal error feedback, OBS)**.** *Let $H \succ 0$ and a row w. If column j is fixed at a value q_j and the following columns F are left free, the minimal increase of $(w-q)^\top H (w-q)$ is $(w_j - q_j)^2 / [H_F^{-1}]_{jj}$, and it is reached by correcting the free columns by $\delta_F = -(w_j - q_j)\, [H_F^{-1}]_{Fj} / [H_F^{-1}]_{jj}$ [40, 50].*

Proof. We seek the smallest perturbation δ (carried by j and F) such that $(w+\delta)_j = q_j$, in the sense of the form $\delta^\top H \delta$ restricted to these columns. The gradient of the Lagrangian $\tfrac{1}{2}\delta^\top H \delta + \mu\,(e_j^\top \delta + w_j - q_j)$ vanishes for $H\delta = -\mu e_j$, that is $\delta = -\mu H^{-1} e_j$; the constraint gives $-\mu [H^{-1}]_{jj} + w_j - q_j = 0$, so $\mu = (w_j - q_j)/[H^{-1}]_{jj}$ and

$$\delta = -\frac{w_j - q_j}{[H^{-1}]_{jj}}\, H^{-1} e_j, \qquad \delta^\top H \delta = \frac{(w_j - q_j)^2}{[H^{-1}]_{jj}^2}\, e_j^\top H^{-1} H H^{-1} e_j = \frac{(w_j - q_j)^2}{[H^{-1}]_{jj}}.$$

All the inverses are taken on the block of columns $\{j\} \cup F$, hence the index F of the statement. ∎

A single factorisation. In the order of columns $1, \dots, C$, the quantities $[H_F^{-1}]_{jj}$ and $[H_F^{-1}]_{Fj}$ of the lemma are the rows of the *upper* Cholesky factor U of H^{-1} ($U^\top U = H^{-1}$): $[H_F^{-1}]_{jj} = U_{jj}^2$ and the correction is $-\big((w_j - q_j)/U_{jj}\big)\,U_{j,F}$ [51]. The scout obtains U without inverting H: if P reverses the order of the columns and $PHP = LL^\top$, then $U = PL^{-1}P$. Indeed $H^{-1} = P(LL^\top)^{-1}P = PL^{-\top}L^{-1}P = (PL^{-1}P)^\top(PL^{-1}P)$ since $P^2 = I$ and $P^\top = P$, and $PL^{-1}P$ is upper triangular because L^{-1} is lower triangular and P reverses rows and columns. The proof is at the head of the function `facteur_inverse`; the factor so obtained differs by $7 \cdot 10^{-14}$ from the direct factor (Chapter 3).

The choice rule. Column j chooses the symbol (sign $\pm$, class k) that minimises

$$k_j = \operatorname*{argmin}_k \big(|\tilde{w}_j| - c_k\, s_b\big)^2 + \lambda\, \bar{c}\, U_{jj}^2\, \ell(\pm, k), \qquad \ell = -\log_2 p, \tag{10.3}$$

where $\tilde{w}_j$ is the weight already corrected by the preceding columns, p the distribution of the symbols of the frontier base of the same tensor, and $\bar{c} = \operatorname{tr}(\Delta W_F H \Delta W_F^\top)/(RC)$ the mean error of a weight of the frontier in the units of H. Divided by U_{jj}^2, the first term is the increase of the output error (lemma 10.1) and the second is $\lambda\,\bar{c}$ times the code length: λ reads as "fractions of the mean error of a weight, per bit". Then the error committed is carried over to the following columns,

$$\varepsilon_j = \pm\,\frac{|\tilde{w}_j| - c_{k_j} s_b}{U_{jj}}, \qquad \tilde{w}_F \leftarrow \tilde{w}_F - \varepsilon_j\, U_{j,F}.$$

$\lambda = 0$ is the feedback alone; $\lambda = 1$ adds the entropy constraint. This is word for word what the scout's code does (`outils/genome/eclaireur/synonyme.py`, functions `hessienne`, `facteur_inverse`, `choisir`), and lot B did not modify it.

The control that can fail. With $H = I$ and $\lambda = 0$, $U = I$: no correction, each weight takes the nearest centre. The centres being the centres of gravity of contiguous classes, the nearest centre of a magnitude is that of its class, and the synonym returns *exactly* the frontier (0 differing symbols; with the real H, 1 789 132 differing symbols on the scout's tensors, Chapter 3).

The decoder does not change. Same eight symbols, same four centres, same scales: the mechanism lives entirely in the coder, and its transcription cost is zero. This is what the analogy says, insofar as it says anything: the "ribosome" (the decoder) reads a synonymous codon without knowing it.

10.2.3 The transformation in the eigenbasis of H

Proposition 10.2 (The output error in the eigenbasis)**.** *Let (u_k, λ_k) be the eigenpairs of the positive symmetric matrix H and $e_k = \|\Delta W u_k\|^2$ the energy of the error in direction u_k. Then*

$$\mathscr{E}(\hat{W}) = \operatorname{tr}(\Delta W H \Delta W^\top) = \sum_k \lambda_k\, e_k, \qquad \|\Delta W\|_F^2 = \sum_k e_k.$$

Proof. $H = \sum_k \lambda_k u_k u_k^\top$, so $\operatorname{tr}(\Delta W H \Delta W^\top) = \sum_k \lambda_k \operatorname{tr}(\Delta W u_k u_k^\top \Delta W^\top) = \sum_k \lambda_k \|\Delta W u_k\|^2$. The u_k forming an orthonormal basis, $\sum_k u_k u_k^\top = I$, and the same computation with $\lambda_k = 1$ gives the Frobenius norm. ∎

Reading. The proposition says what the synonymous mutation does: it lowers $\sum_k \lambda_k e_k$ by *moving* the energy e_k towards the directions where λ_k is small, at the price of a larger total energy $\sum_k e_k$, that is, of a higher error *on the weights*. It is synonymous for the output, not for the weights.

Figure 10.1 shows it on one matrix, before and after: layer 8, expert 0, the gate matrix of the 20b, frontier against synonym $\lambda = 1$ [meas.]. The calibration matrix H has rank 1 329 there: the 1 551 directions beyond, out of 2 880, weigh nothing in calibration, but the evaluation inputs still weigh them (at least 0.119). The damping $\delta^\star = 1$ adds to each direction $\overline{\operatorname{diag} H} = \operatorname{tr} H/2\,880 = 0.41$, more than this evaluation weight of the silent directions; only 433 eigenvalues exceed it. The synonym increases the error on the weights by 20 % ($\sum_k e_k$: 40.07 → 48.01) and moves it towards the silent directions (from 54 % to 61 % of $\sum_k e_k$ beyond the rank). The output error, for its part, falls:

$$\begin{aligned} \textstyle\sum_k \lambda_k e_k &: \ 16.50 \to 6.15 \ (\div 2.68) \text{ in calibration,} \\ \textstyle\sum_k (u_k^\top H_{\text{ev}} u_k)\, e_k &: \ 16.09 \to 10.28 \ (\div 1.57) \text{ seen by the evaluation.} \end{aligned}$$

The gap between ÷2.68 and ÷1.57 is the overfitting that remains at $\delta^\star = 1$: the cost of the frontier seen by the evaluation stays at its calibration cost (× 0.975), that of the synonym rises (× 1.67). The scout had said it in one sentence: the synonym of a sample is not the synonym of the distribution. A reading remark, from the judge of the figures: the shaded area between the two curves of panel (b′) *is not* the measure of this overfitting, since the frontier has one too (0.17, against 0.32 for the synonym); the cumulative share in calibration reaches 1 at the rank of H for any variant.

A row, seen up close. Figure 10.2 goes down to the level of the weights, in the same matrix: a row of 32 weights (row 2 118, block 52), drawn among the blocks whose number of changed symbols equals the median over the matrix, hence typical by construction [meas.]. The mutation changes 4 symbols out of 32 in it and moves 3 of them away from their exact value (the fourth, a zero, changes sign at the same distance); the error on the weights of the block rises from 7.39 to 11.27 (in units of s_b^2). Over the whole matrix, 17.5 % of the symbols change (5.59 per block on average). A row cannot show where the error goes: the eigenbasis (Figure 10.1) is what shows it.

10.2.4 The error in the directions of the inputs

Figure 10.3 looks at the same matrix from its inputs: the 1 329 real calibration inputs of the expert, projected on the first eigen-axes of H ($\lambda_1 = 279.5$, $\lambda_2 = 33.5$, $\lambda_3 = 21.7$, 28.2 % of the trace), and, in each unit direction v of a plane, the output error $\|\Delta W v\|$ for a unit input of direction v [meas.] [calc.]. In a plane $B = [u_1\ u_k]$, this curve is the support function of the ellipse $M^{1/2}$, $M = B^\top \Delta W^\top \Delta W B$: for $v = Ba$, $\|a\| = 1$, $\|\Delta W v\|^2 = a^\top M a$. The frontier is almost a circle: it spreads its error flat. The synonym crushes it along u_1, where the inputs project: 90 % of the projections are within 30° of u_1 in the plane (u_1, u_2) and within 24° in the plane (u_1, u_3), and for a unit input in these cones the error of the synonym is at most 0.133 and 0.135 of that of the frontier. Along the axes themselves, the ratio is 0.082 (u_1), 0.217 (u_2) and 0.269 (u_3).

What the bound of 0.135 does not say. It bears on the *projections* of the inputs in these planes, not on the whole inputs: 72 % of the squared norm of an input lies outside the first three axes, and an input makes a median angle of 59° with the plane (u_1, u_2). Over the 1 329 real inputs in their 2 880 dimensions, the ratio $\|\Delta W_s x\|/\|\Delta W_f x\|$ is 0.61 in median, from 0.49 to 0.81, and none is below 0.135. The first version of the figure suggested the opposite; the second judge of the latent spaces flagged it, and the judged caption now gives both numbers. In the 2 877 other directions, weighted by the inputs, the output error also falls, from 11.96 to 6.08, and these directions carry 57 % of the output gain: the gain is not only where the figure looks.

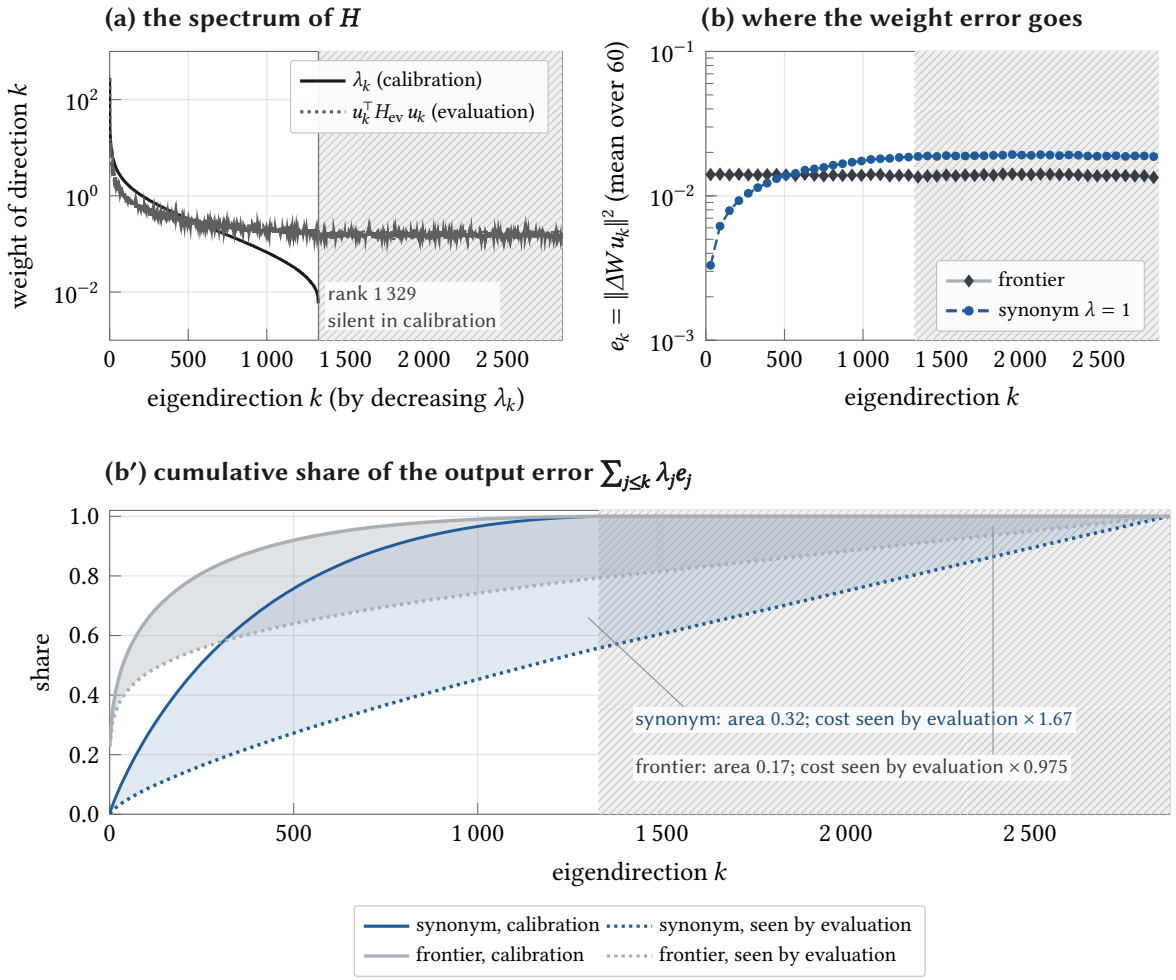


Figure 10.1: The synonym lowers the calibration cost by pushing the error towards the directions calibration does not see; evaluation still sees them. [meas.] [calc.] Before (frontier) and after (synonym $\lambda = 1$), for one matrix: layer 8, expert 0, gate of gpt-oss-20b; $H = \mathrm{E}[g^2 x x^\top]/\mathrm{E}[g^2]$ from the calibration inputs; the output cost is $\mathrm{tr}(\Delta W H \Delta W^\top) = \sum_k \lambda_k e_k$. (a) The calibration H has rank 1 329 (the number of calibration inputs): the 1 551 directions beyond (hatched), out of 2 880, weigh nothing in calibration, but the evaluation inputs still weigh them (at least 0.119). The coder itself minimised the cost with $H + \delta^* \overline{\mathrm{diag}\, H}\, I$, $\delta^* = 1$ (07 § 2): 0.41 added to every direction, more than the evaluation weight of the silent directions; only 433 eigenvalues exceed it. (b) The synonym raises the weight error, $\sum e_k$ 40.07 → 48.01 (+20%), and moves it towards the silent directions (54% → 61% of $\sum e_k$ beyond the rank). (b′) The cumulative share of the output error, in calibration (solid) and seen by the evaluation inputs (dotted). The output cost falls by × 1/2.68 in calibration (16.50 → 6.15) but only by × 1/1.57 seen by the evaluation (16.09 → 10.28): the frontier's cost seen by the evaluation stays at its calibration cost (× 0.975), the synonym's rises by × 1.67; that is the residual over-fitting. The shaded area between the two curves is not its measure: the frontier has one too (0.17, against 0.32 for the synonym), because the calibration share reaches 1 at the rank of H, for any variant. One matrix of one expert; the other two spectra in the same folder are not drawn.

Data: `mesures/genome/construction/synonyme/figures/spectre-L08-E00-gate.csv`; script `memoire/outils/tome3_synonyme.py`.

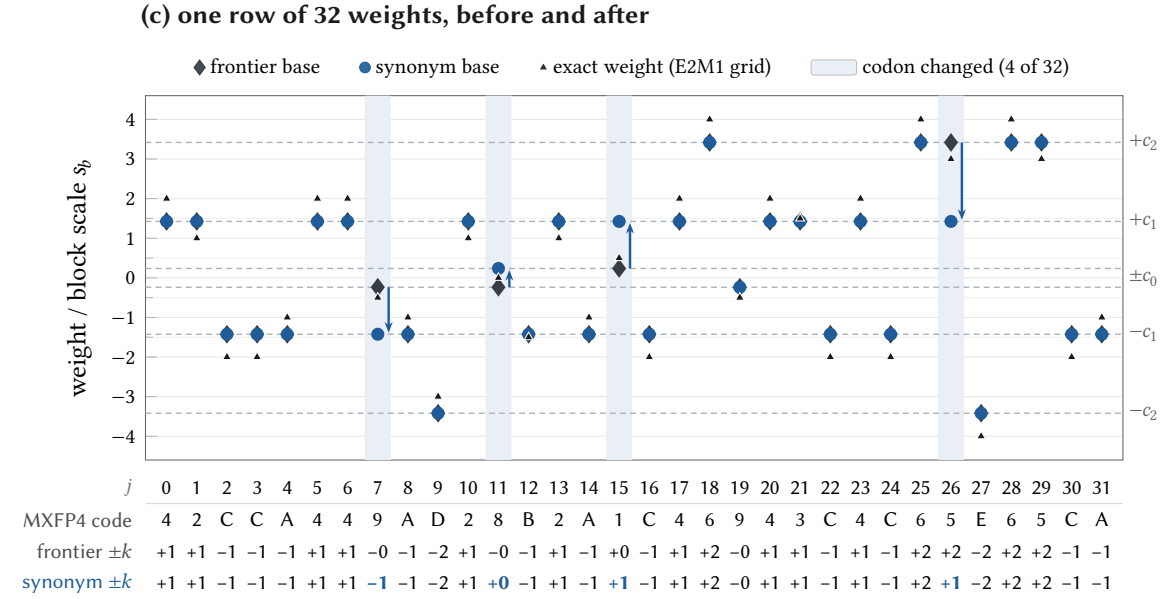


Figure 10.2: The synonymous mutation changes 4 of the 32 symbols of this row and moves 3 of these weights away from their exact value (the other one, a zero, changes sign at the same distance): it is synonymous for the output, not for the weights. [meas.] Panel (c) of Figure 10.1: a row of 32 weights of the same matrix as panels (a), (b) and (b′) (gpt-oss-20b, layer 8, expert 0, gate): row 2 118, block 52 (columns 1 664 to 1 695), in units of the block scale $s_b = 2^{-9}$. Black triangles: the exact weight, on the E2M1 grid; grey diamonds: the frontier base (its symbol is a function of the code); blue circles: the synonym base ($\lambda = 1$, $\delta^* = 1$), as served by the lot B `.sym` files; an arrow, beside the marks, goes from the frontier value to the synonym value where the symbol changes. Dashed lines: the levels $\pm c_k$ of the base ($c_0 = 0.237$, $c_1 = 1.424$, $c_2 = 3.419$; $c_3 = 6.000$ is out of the frame), the same for both bases. In the band, j is the column in the block, then the MXFP4 code (hexadecimal), then each base's symbol: "$+k$" and "$-k$" stand for the level $\pm c_k$ ($k = 0$ to 3), not for a signed zero. The block was drawn (seed) among the 26 155 blocks whose number of changed symbols equals the median over the matrix (4 of 32; mean 5.59, 17.5% of the symbols of the matrix): typical by construction. In this block the synonym raises the weight error (11.27 against 7.39, in s_b^2); over the whole matrix the error energy rises by 20% (40.07 to 48.01) while the output error falls 2.7-fold on calibration (16.50 to 6.15) and 1.57-fold seen by the evaluation. Not shown: the output, which only (a), (b) and (b′) show; one row cannot show where the error goes (to the mute directions of H, panel (b)). Control: the symbols read reproduce $\|W - \hat{W}\|_F^2 = \sum_k e_k$ of the lot B file to 1.8×10^{-12} relative (another expert's files: a relative gap of 41).

Data: `mesures/genome/tome3/latents/rangee-L08-E00-gate.csv`, `rangee-L08-E00-gate.json` (seed 20261027), by `outils/genome/tome3/latents_rangee.py`; `mesures/genome/construction/synonyme/figures/spectre-L08-E00-gate.csv`; `memoire/outils/tome3_latents.py`.

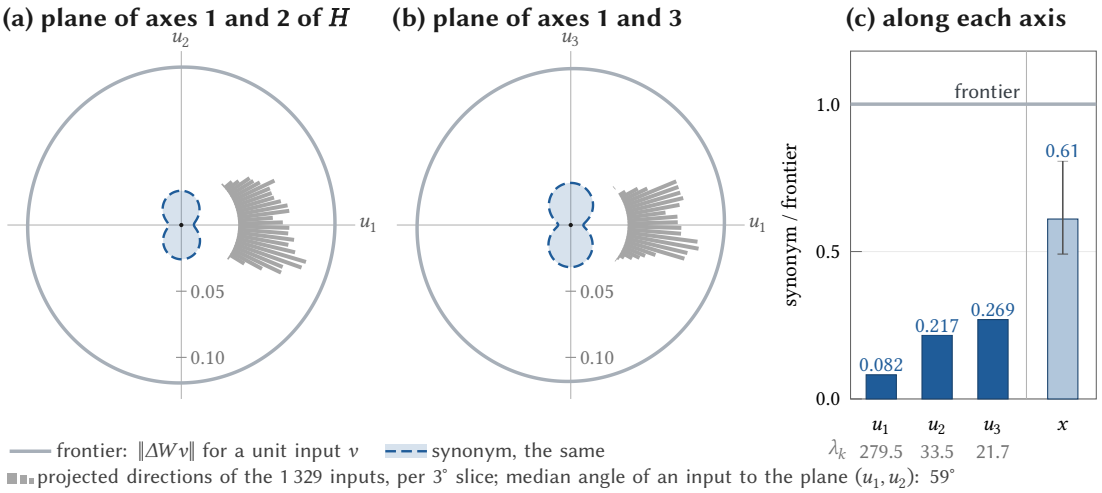


Figure 10.3: In the planes of the first axes of H, where the expert's inputs project, the synonym brings the output error down to at most 0.135 of the frontier's; on the whole inputs, 72% of whose squared norm lies outside these three axes, to 0.61 (median; 0.49 to 0.81). [meas.] [calc.] Expert 0 of layer 8 of gpt-oss-20b, gate matrix, and its 1 329 real calibration inputs x (the positions the router sends it, windows 10 to 21 of wiki.test), on the first eigen-axes u_k of $H = \sum g^2 x x^\top / \sum g^2$ (not damped, the H of Figure 10.1; $\lambda_1 = 279.5$, $\lambda_2 = 33.5$, $\lambda_3 = 21.7$, 28.2% of the trace). (a), (b) In each unit direction v of the plane, each closed curve reaches $\|\Delta W v\|$, the output error for a unit input of direction v, with $\Delta W = W - \hat{W}$ for that base: the support function of the ellipse $M^{1/2}$, $M = B^\top \Delta W^\top \Delta W B$, $B = [u_1\ u_k]$; the radial scale, in the units of the output, is marked on the lower half-axis. The frontier is almost a circle: it spreads its error evenly. In the ring between the two curves, the rose counts the projected directions of the inputs, their projections onto the plane (bar length proportional to the count; its radial place carries no unit): 90% of the projections lie within ±30° of u_1 in (a) and ±24° in (b); for a unit input in that cone, the synonym's error is at most 0.133 (a) and 0.135 (b) of the frontier's. H is not centred: u_1 carries the mean of the inputs, all near $u_1^\top x = 16.5$ (spread 1.84, against 5.98 along u_2), and λ_1 owes most to that mean ($16.5^2 \approx 272$). (c) The ratio synonym / frontier of $\|\Delta W u_k\|$ along each axis: 0.082 along u_1, the axis where the inputs weigh most (their mean), 0.217 along u_2, 0.269 along u_3 (frontier 0.116, 0.119, 0.118). Right of the rule, the same ratio for the 1 329 real inputs in their 2 880 dimensions, $\|\Delta W_s x\| / \|\Delta W_f x\|$: median 0.61, from 0.49 to 0.81 (whisker), 0% of them below 0.135; an input makes a median angle of 59° with the plane (u_1, u_2). These ratios come from the journal of the second-flow judge, recomputed from the GGUF, the `.sym` files and the captures; a control ties them to the committed files: their g^2-weighted quadratic ratio equals $\sqrt{\sum_k \lambda_k e_k^{\text{syn}} / \sum_k \lambda_k e_k^{\text{fr}}}$ of the lot B file to 10^{-9} (with λ_1 off by 1%, it departs). Not shown: the 2 877 other directions, where the synonym puts the weight error it removes here (panel (b) of Figure 10.1); weighted by the inputs, the output error falls there too, from 11.96 to 6.08, and these directions carry 57% of the output gain; the held-out evaluation inputs, on which the gain shrinks (1.57-fold against 2.7-fold). The curves are computed from the ellipses committed by the lot: M is recovered from their 181 points by least squares and recovers e_k and the half-axes of the lot's file to 2.2×10^{-7} relative (with a scale factor off by 1%, it departs from them). Control of the lot: λ_1 to λ_3 and e_1 to e_3 equal those of the lot B file to 3.4×10^{-10} relative (from the evaluation inputs, λ_1 moves by 5.5%).

Data: `mesures/genome/tome3/latents/nuage-L08-E00-gate-points.csv`, `nuage-L08-E00-gate-ellipses.csv`, `nuage-L08-E00-gate.json`, by `outils/genome/tome3/latents_nuage.py`; real inputs `outils/genome/tome3/juge2-latents/journaux/juge2-nuage-rangee.json` (`entrees_reelles`); `memoire/outils/tome3_latents.py` (controls `latents-temoins.json`).

10.3 The fast mode does not touch the exact mode

The fast mode adds a path in an engine whose bit exactness is proved and measured (Volume II). Two propositions say what the addition preserves, and what the new kernel guarantees.

Proposition 10.3 (B1: the exact mode is unchanged)**.** *The `gemv_4_lignes` kernel is parametrised by the value reader V (`moteur/crates/trc-noyaux/noyaux/gemv.cuh`). For V = `ValeurE2M1`, the outputs of the exact mode are identical to the bit to those before the addition; and a fast file is decoded by the same rANS decoder, the same tables and the same grid as the exact mode.*

Proof. For V = `ValeurE2M1`, the sequence of `fmaf` has the same operands in the same order as before the parameter: same value `valeur_e2m1x2(c)` for each nibble, same traversal (word t, byte k, low nibble then high), same butterfly reduction. A sequence of floating-point operations identical in operands and in order returns results identical to the bit; the parameter is resolved at compile time and adds no operation. The rANS decoder, for its part, never reads the value of a symbol (Volume II, specification § 4.9): it returns nibbles; it is the RAPIDE flag of the header that says they are base symbols, and it is the GEMV that reads them differently. Card control: the exact mode before and after a pass in fast mode, on the same bytes, returns the same outputs to the bit (test `gemv_base_exacte_au_bit_et_mode_exact_inchange`) [meas.]. ■

Proposition 10.4 (B2: `gemv_base` is deterministic, and exact on a control)**.** *`gemv_base` computes $y = \hat{W}x$ with $\hat{W}_{rc} = \pm c_k \cdot 2^{e-128}$ in the same tiling and the same order as `gemv_mxfp4`. If the centres are multiples of $\frac{1}{2}$ at most equal to 12, the scales in $[2^{-11}, 2^{-5}]$ and the inputs in $\{-2, \dots, 2\}$, the output of the card equals the double-precision reference to the bit.*

Proof. Determinism. The value $v(c) = \pm c_{c\,\&\,3}$ is chosen by selections, without indexed reads or memory-access divergence, and the order of operations is fixed by the tiling, as for `gemv_mxfp4` (Volume II, chapter on the kernels). *Exactness on the control.* A term is $c \cdot s \cdot x$ with $c = a/2$ (a an integer, $|a| \le 24$), $s = 2^{-m}$ ($5 \le m \le 11$) and x an integer, $|x| \le 2$: it is a multiple of $2^{-(m+1)}$, hence of 2^{-12}, and $|csx| \le 12 \cdot 2^{-5} \cdot 2 = 0.75$. A row of 2 880 terms stays below $2\,880 \times 0.75 = 2\,160 < 2^{12}$, and so does every partial sum, in any order. A multiple of 2^{-12} with absolute value below 2^{12} is written on $12 + 12 = 24$ significant bits: it is exactly representable in f32 (24 bits of mantissa). Each product and each addition therefore lands on an exact f32 float, no operation rounds, and the result depends neither on the order nor on the precision: it equals the exact sum, which the f64 reference also computes exactly. Measured: 0 differing rows out of 2 880 [meas.]. ■

The control of B2 is chosen so that it can fail: the break (the exact mode run on the same bytes, which reads codes and not symbols) returns differing rows, and falls [meas.].

10.4 The companion `<nom>.rapide.trc`, to the bit

The companion is an ordinary `.trc` file (Volume II, chapter on the format), with what Table 10.2 gives; everything else is the exact format. The diagram of the planned figure (III-9.5) has not been drawn: the table stands in for it.

The partition and the centres. The `.partition` key sorts the eight E2M1 magnitudes, in doubled magnitudes $\{0, 1, 2, 3, 4, 6, 8, 12\}$, into the classes 0 0 1 1 1 2 2 3: $\{0, 1\}$, $\{2, 3, 4\}$, $\{6, 8\}$, $\{12\}$. Each centre lies in its class, and the last is magnitude 12 itself; the factor 2^{e-128} instead of 2^{e-127} compensates for the doubling. It is these contiguous classes that make the $H = I$ control true (Section 10.2.2).

Table 10.2: The companion `<nom>.rapide.trc`, field by field: what differs from the exact file, and why.

Element	Value	Reason
version	1.2	the RAPIDE flag is defined in v1.2
flags	bit 0 ALIGNÉ + **bit 5 RAPIDE**; bits 1 (levels), 2 (raw), 3 (canonical) refused with it	a 1.0 or 1.1 reader refuses the file: the exact mode cannot read symbols by mistake
MÉTA	the GGUF pairs, copied from the exact file, plus `troncons.rapide.codeur`, `.variante` (synonyme or frontière), `.exact` (empreinte64 of the header of the exact file, in hexadecimal), `.partition` (`0 0 1 1 1 2 2 3`), `.centres`, `.lambda`, `.amortissement`, `.calibration`	provenance; the verification checks the `.exact` link
RÉSIDENTS	copied byte for byte from the exact file	the companion opens on its own
TABLES	per (layer, matrix): DELTA (context 0), CODES_CTX (contexts 0, 1, 2: counts of the base nibbles over the 32 experts), **CENTRES** (context 0: 4 f32, doubled magnitudes)	same decoder, same tables; the GEMV reads the centres
chunk	MEDIANES, BIAIS, DELTA, CODES_CTX for the 3 matrices (12 streams, as in v1.0)	scales and biases of the exact expert
nibble	bit 3: the sign; bits 1-0: the class k; bit 2: zero. Byte t of a block carries column $32b + t$ (low nibble) and $32b + 16 + t$ (high)	layout of `block_mxfp4`
served value	$\pm c_k \cdot 2^{e-128}$, $c = (0.474\,331, 2.848\,596, 6.837\,154, 12.0)$ in doubled magnitudes	`gemv_base`
original fingerprint	empreinte64 of the *fast expert* (blocks [e, base nibbles], then biases)	the engine's control mode checks it without change

Data: `genome/07-genome-synonyme.md`, § 3; `genome/construction-synonyme-pour-le-memoire.md`, § 2.5.

Three departures from the specification , each so as to touch no path of the exact mode (*answers by delegation of 2026-10-17, reversible*): (1) the CODES_CTX stream kind instead of a new BASE_CTX kind, so that the card's decoder, its tables and its grid remain those of the exact mode, word for word, the flag carrying the meaning; (2) the residents copied instead of an empty section, so that the engine opens the companion like any file (cost: 1.9 GB of disk); (3) the original fingerprint taken on the fast expert, the provenance of the exact expert remaining in the header (SHA-256 of the GGUF) and in `.exact`.

10.5 The quality measurements

10.5.1 The protocol and its controls

Declared beforehand: the f64 reference of lot R with substitution (`genome-capture kl --sub`), the 768 experts served by their base, on the ten windows of R1 (2 550 scored positions, kept away from the calibration), judged by `juge kl` against the exact reference; the paired difference position by position, and its standard deviation *by windows* (the ten window means, $\sigma = s/\sqrt{10}$), as throughout the quest. The KL is that of Kullback and Leibler [11], in the sense and with the cut of Volume II; the first choice carries its Wilson [73] interval.

Two controls were declared wrongly, and corrected in place, struck through, with the time:

- ~~the log of each pass carries 768 substituted experts~~: corrected on 17 October at 10:50, after the frontier pass, which carries 732. The counter counts the experts *loaded*, not the files present, and 36

experts are routed by none of the 2 550 positions. The right control is "substituted = read": frontier 732 out of 732, synonym 725 out of 725, $\lambda = 0$ 729 out of 729; the sets differ because the routing depends on the experts served. Known break: the scout's pass, 32 substituted out of 731 read [meas.].

- ~~the KL recomputed from the CSV equals that of the judge to within 10^{-12}~~: corrected at 11:35, after an unexplained fall on the synonym pass. The CSV carries ten significant digits, that is at most $5 \cdot 10^{-10}$ relative error per value; the absolute threshold came from the scout's KLs of 0.03. The right threshold is 10^{-9} relative: measured gaps $3.5 \cdot 10^{-12}$, $8 \cdot 10^{-12}$ and $5 \cdot 10^{-12}$; the break (one line removed) falls [meas.].

The lesson is that of the lot: a counter control is declared on what the counter counts.

10.5.2 The benchmark of the reference tool

llama.cpp b11515 [124] can requantise the experts of gpt-oss below 2.6 bits only in Q2_0 (2.25 bits per weight, groups of 64): its k-quants and its IQ have super-blocks of 256 weights, which 2 880 does not divide. Experts alone requantised (the rest of the GGUF unchanged, 6 979.93 MiB against 11 536.18), ten windows of R1 on the processor, options of the "processor" base: KL 1.597 ± 0.019, median 1.314, first choice 13.6 %, perplexity 881.2 [meas.]. It is a weak benchmark, which must be said as such: a round-to-nearest at two bits, without an importance matrix, almost destroys the model.

10.5.3 The end-to-end KL, the 768 experts substituted

Table 10.3: The end-to-end KL on R1, the 768 experts of gpt-oss-20b served by their base (substituted f64 reference, 2 550 positions) [meas.]. The naive uncertainty (by positions) is the one that 07 publishes; the one by windows is the right one (Section 11.5.3).

Variant	KL	$\pm\sigma$ naive	$\pm\sigma$ windows	median	99.9th pc.	first choice	perplexity
frontier	0.264 0	0.006 6	0.037 6	0.163	4.07	47.8 %	274.1
synonym $\lambda = 1$	0.152 2	0.006 2	0.020 3	0.061 5	3.93	67.5 %	466.5
synonym $\lambda = 0$	0.131 8	0.005 5	0.018 2	0.055 9	3.37	69.9 %	411.0
llama.cpp Q2_0 (benchmark)	1.597	0.019	0.173	1.314	6.78	13.6 %	881.2

Data: `mesures/genome/construction/synonyme/kl/kl-\{frontiere,synonyme,synonyme-l0\}-768-juge.json`, `repere/kl-q2_0-juge.json`; σ by windows recomputed from the per-position files by `memoire/outils/tome3_synonyme.py`; perplexity of the exact reference: 460.2.

What these numbers say (Table 10.3):

- Serving *all* the experts by the frontier base gives a KL of 0.264, seven times that of 32 substituted experts (0.036, the scout's protocol): the KL grows more slowly than the number of substituted experts (×24 in experts, ×7 in KL), which is expected of errors most of which pass through routing flips, which saturate [calc.]. The two numbers come from two protocols; they are compared here by their ratio, they are joined by no line (reserve R3).
- The synonym keeps its relative advantage when going from 32 to 768 experts. The lot wrote that "the gain does not melt; it grows", comparing $\lambda = 0$ at 32 experts (ratio to the frontier 0.727) with $\lambda = 1$ at 768 (0.576): two variants. At equal variant, $\lambda = 0$ goes from 0.727 to 0.499, and the sentence stays true (finding M5 of the judge).
- The absolute uncertainties published by 07 are naive: by positions, they ignore that the 255 positions of a window share their past. By windows, they are 3.2 to 8.9 times wider depending on the series

(finding M1 of the judge; derivation in Section 11.5.3). No verdict depends on them: all rest on paired differences taken with σ by windows.

The quantiles (frontier → synonym $\lambda = 1$): 90th percentile 0.565 → 0.328; 99th 1.457 → 1.295; maximum 6.49 → 6.94 [meas.]. The smooth part falls a lot, the tail little: median ÷ 2.65, 90th percentile ÷ 1.72, 99th ÷ 1.12, 99.9th ÷ 1.04 [calc.]. The further into the tail, the less the synonym does there: these are the routing flips, which weighting by the inputs of a single expert does not see (Chapter 8).

10.5.4 Criterion B2, and the settling of the two λ

B2 (i), synonym $\lambda = 1$ against the frontier. The rule, declared beforehand: $d = \mathrm{KL}_{\text{synonym}} - \mathrm{KL}_{\text{frontier}}$ position by position; better if $\bar{d} + 2\sigma < 0$, equal if $|\bar{d}| \leq 2\sigma$, worse otherwise. Measured: $\bar{d} = -0.111\,8$, σ by windows 0.029 6 (naive 0.004 5), $\bar{d} + 2\sigma = -0.052 < 0$: **better** ($3.8\,\sigma$), negative in all ten windows, from −0.036 (window 4) to −0.345 (window 7); paired first choice +0.197 ± 0.011 [meas.]. The gate to the next step (the fast mode in the engine), declared "if (i) holds", is open.

B0, the measurement that was missing at first. Made *after* the pass at 768 experts, on the scout's protocol (32 experts, 2 550 positions; the $\lambda = 1$ symbols of the 32 experts are a subset of the 768) [meas.]:

32 experts (scout)	KL	against the frontier	against $\lambda = 0$	first choice
frontier	0.036 0	—		87.1 %
$\lambda = 0$ (scout)	0.026 2	−0.009 8 ± 0.003 3, better	—	90.9 %
$\lambda = 1$	0.035 0	−0.001 0 ± 0.002 2, equal	+0.008 8 ± 0.003 2, worse	87.7 %

At 32 experts, the entropy constraint gives up almost all the KL gain of $\lambda = 0$ in exchange for its 0.078 bit; at 768, $\lambda = 1$ beats the frontier by 42 %. The two facts do not contradict each other (different experts, a different regime of flips), but they say that the decisive comparison is $\lambda = 0$ against $\lambda = 1$ *at 768 experts*. Its rule was declared on 17 October at 12:15, after B0 and before the $\lambda = 0$ coding (commit `ff36671`): $d = \mathrm{KL}_{\lambda=0} - \mathrm{KL}_{\lambda=1}$; if $\bar{d} + 2\sigma < 0$, $\lambda = 0$ becomes the variant of the fast mode; otherwise $\lambda = 1$ stays, which has 0.07 bit less.

B2 between the two λ. Coding $\lambda = 0$, $\delta^\star = 1$ of the 768 experts (768 files of 12 441 636 bytes, that is $4 + 32 + 3 \times 2\,880 \times 1\,440$), then the same KL pass (on 21 October, 1 554 s). Measured: $\bar{d} = -0.020\,4$, σ by windows 0.009 7 (naive 0.003 8), $\bar{d} + 2\sigma = -0.000\,9 < 0$: $\lambda = 0$ **better, narrowly** ($2.09\,\sigma$). Nine windows out of ten go this way; window 7 goes the other (+0.061: 0.113 → 0.174). $\lambda = 0$ has less KL on 1 706 positions out of 2 550; paired first choice +2.4 ± 0.8 points; against the frontier, −0.132 2 ± 0.023 4 ($5.6\,\sigma$) and +22.1 ± 1.1 points of first choice [meas.]. By the rule, $\lambda = 0$ becomes the variant of the fast mode.

The judge, on ten new windows. The judge of the construction drew ten windows of wiki.test that neither the calibration nor R1 touch (Section 11.3) and redid B2 through the engine: $\bar{d} = -0.030\,3 \pm 0.008\,4$, $3.6\,\sigma$, nine windows out of ten (window 0 against, +0.016); and $\lambda = 0$ against the frontier, through the substituted reference: $-0.156\,5 \pm 0.033\,4$ ($4.7\,\sigma$), ten windows out of ten [meas.]. Its declared prediction, "equal", is refuted: the choice of $\lambda = 0$ was not a stroke of luck of the R1 windows. Table 10.4 gives each window; Figure 10.4 draws them.

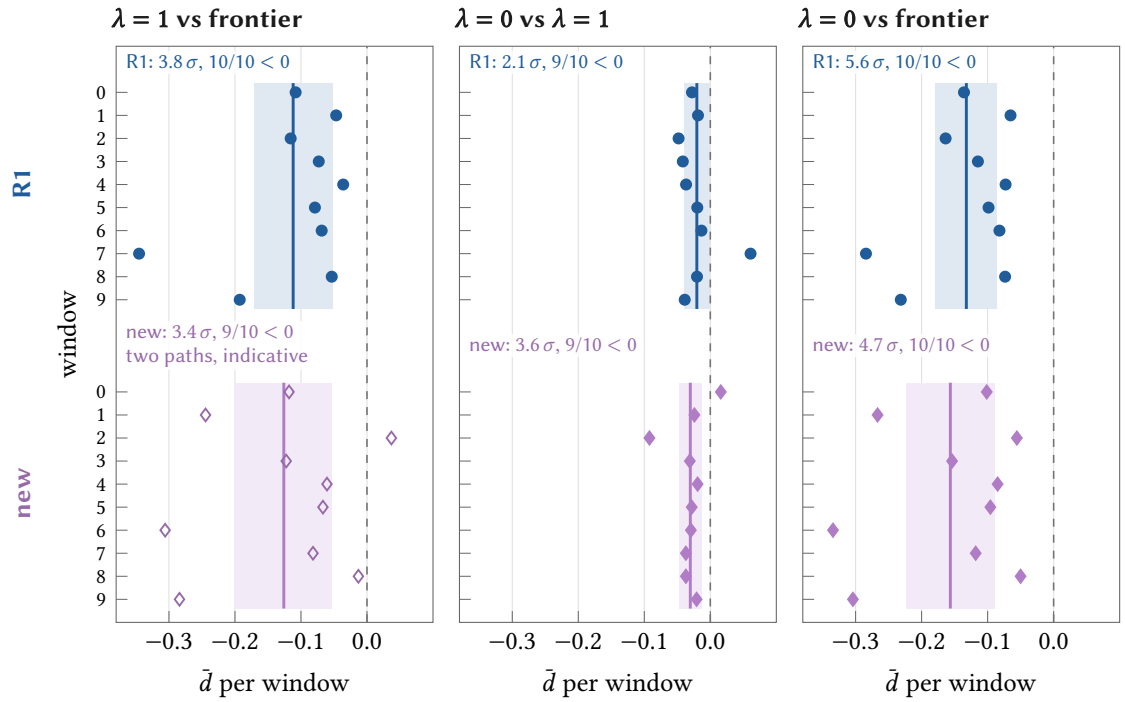


Figure 10.4: Both synonyms beat the frontier on both sets of windows, each in at least 9 of 10 windows; between them the gap is much smaller, and $\lambda = 0$ passes on R1 by a hair. [meas.] Paired differences of the mean KL per window, $\bar{d} = \overline{\mathrm{KL}}_b - \overline{\mathrm{KL}}_a$, negative when the second variant wins; one axis for the three panels, no point outside the frame. Top (circles), the ten windows of R1 by lot B through the substituted f64 reference (for $\lambda = 0$ against the frontier, the judge's recomputation on the same path). Bottom (diamonds), ten new windows of the judge: panel 1 sets $\lambda = 1$ through the engine against the frontier through the substituted reference, two paths, published as indicative only (JUGEMENT-construction § 9; hollow diamonds; on R1, where both paths exist, this mix moves $\bar{d}$ by 0.0014); panel 2 through the engine; panel 3 through the substituted reference. Vertical line and band: mean and $\pm 2\sigma$, σ the standard deviation of the ten windows divided by $\sqrt{10}$. $\lambda = 0$ against $\lambda = 1$ passes on R1 by a hair (2.1 σ, window 7 against) and with more margin on the new windows (3.6 σ). The new windows are drawn from wiki.test, the corpus of R1: they test other text, not another kind of text.

Data: `mesures/genome/construction/synonyme/kl/apparie-*-par-fenetre.csv`, `mesures/genome/construction/juge/figures/apparie-par-fenetre.csv`; two-path check `mesures/genome/construction/synonyme/moteur/kl-moteur-*-positions.csv`; script `memoire/outils/tome3_synonyme.py`.

Table 10.4: The paired differences $\bar{d}$ of each window (negative when the second variant wins) [meas.]. R1: lot B, substituted f64 reference. New windows: the judge; $\lambda = 0$ against $\lambda = 1$ through the engine, $\lambda = 0$ against the frontier through the substituted reference.

Window	0	1	2	3	4	5	6	7	8	9	z
R1, $\lambda = 1$ – frontier	−0.108	−0.047	−0.116	−0.073	−0.036	−0.079	−0.069	−0.345	−0.053	−0.193	3.8
R1, $\lambda = 0$ – $\lambda = 1$	−0.028	−0.019	−0.048	−0.042	−0.037	−0.020	−0.013	+0.061	−0.020	−0.039	2.1
new, $\lambda = 0$ – $\lambda = 1$	+0.016	−0.024	−0.092	−0.031	−0.019	−0.028	−0.029	−0.037	−0.037	−0.021	3.6
new, $\lambda = 0$ – frontier	−0.101	−0.267	−0.056	−0.154	−0.085	−0.096	−0.334	−0.118	−0.050	−0.304	4.7

Data: `mesures/genome/construction/synonyme/kl/apparie-768-par-fenetre.csv`, `apparie-l1-contre-l0-768-par-fenetre.csv`; `mesures/genome/construction/juge/kl/apparie-neuf-moteur-l0-contre-l1-par-fenetre.csv`, `apparie-neuf-sub-l0-contre-frontiere-par-fenetre.csv`. The new windows are not the R1 windows of the same number.

Reading the forest. Figure 10.4 puts the three comparisons on a single axis. Against the frontier, both synonyms win with a margin that no one disputes, on both sets of windows. Between them, the gap is much smaller, and the $\pm 2\sigma$ band of R1 grazes zero: this is the lot's "narrowly", and it is the same rule that gives both verdicts. A remark on paths: the first panel of the new windows sets $\lambda = 1$ through the engine against the frontier through the substituted reference, two paths that the judge publishes "for information only" ($-0.126\,0 \pm 0.036\,6$); on R1, where both paths exist, this mix moves $\bar{d}$ by 0.001 4. The new windows are from wiki.test, the corpus of R1 [125]: they test another text, not another kind of text.

10.5.5 The tail of the KL

Figure 10.5 plots the survival function $P(\mathrm{KL} > t)$ of the 2 550 positions for the three variants and the benchmark. It shows at a glance the two parts of the KL that the synthesis had separated (Chapter 8): the smooth part, where the synonym divides the median by 2.66 ($\lambda = 1$) and 2.92 ($\lambda = 0$), and the tail of routing flips, where the curves rejoin.

The tail made the lot write two sentences that the measurement took back, and which remain struck through in place:

- ~~The tail does not move~~ (07, § 0): struck through by the lot itself on 22 October, because true for $\lambda = 1$ only.
- "With $\lambda = 0$, the tail falls too." Against the *frontier*, it holds: 99th percentile 1.457 → 1.081 (−26 %) on R1, 1.877 → 1.397 (−26 %) on the new windows through the substituted reference; 99.9th −17 % and −18 % [meas.]. Against $\lambda = 1$, ~~the 99th percentile falls by 17 % and the 99.9th by 14 %~~: on the new windows, the 99th falls by only 5 % and the 99.9th *rises* by 6 % (3.61 against 3.40, engine); struck through by the judge (finding M2).

The reason is arithmetic: a 99.9th percentile of 2 550 positions leaves only three positions above it [calc.]. It is not a quantity to set one variant against another at this sample size. The caption of Figure 10.5 says so, and gives the fall at the 99.9th percentile against the frontier by set and by path: $\lambda = 1$ 3.4 % on R1 but 20.8 % on the new windows, $\lambda = 0$ 17.2 % and 15.9 % (synonyms through the engine on the new windows; hence 15.9 % here and 18 % through the substituted reference).

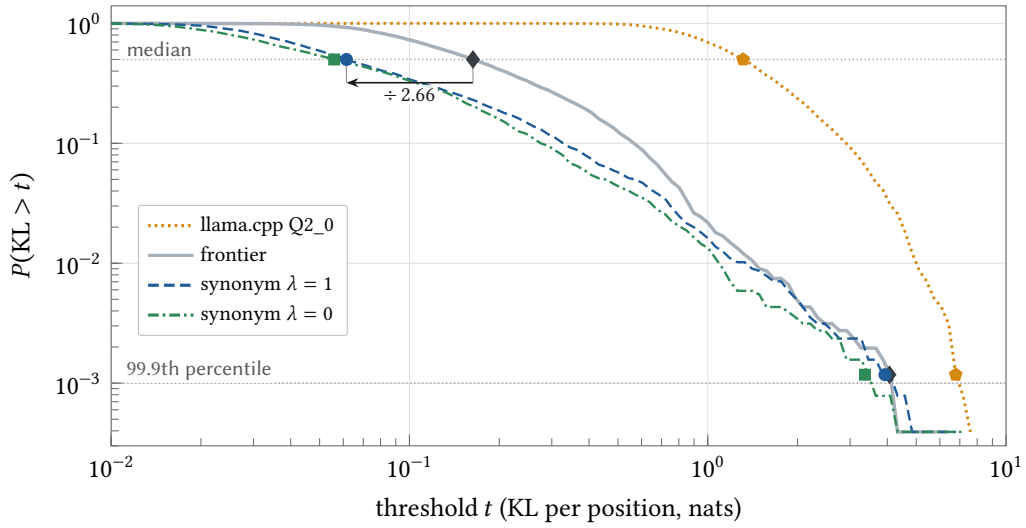


Figure 10.5: The synonym divides the median KL by nearly three; the tail rests on too few positions to rank the variants. [meas.] Survival function $P(\mathrm{KL} > t)$ over the 2 550 scored positions of R1 (10 windows), all 768 experts of gpt-oss-20b replaced in the f64 reference; on the dotted lines, the medians and the 99.9th percentiles, each with the mark of its series. The median of the frontier (0.163) is divided by 2.66 with $\lambda = 1$ (0.061) and by 2.92 with $\lambda = 0$ (0.056). Each 99.9th percentile leaves only 3 positions above it: against the frontier, $\lambda = 1$ lowers it by 3.4% on R1 but by 20.8% on the judge's ten new windows, $\lambda = 0$ by 17.2% and 15.9% (new windows: frontier through the substituted reference, synonyms through the engine). Between $\lambda = 0$ and $\lambda = 1$ the tail is not comparable at this size (barred, JUGEMENT-construction M2). Left of 10^{-2}: at most 0.7% of the positions of a series. Q2_0 is computed by llama.cpp on the processor, another path (a landmark). Positions of one window share their past and are not independent (Figure 10.4).

Data: `mesures/genome/construction/synonyme/kl/kl-*-768-positions.csv`, `repere/kl-q2_0-positions.csv`, `kl/kl-*-768-juge.json`; `mesures/genome/construction/juge/kl/kl-neuf-*-positions.csv`; script `memoire/outils/tome3_synonyme.py`.

10.5.6 Perplexity does not judge fidelity

The perplexity of the frontier *falls* (274.1 against 460.2 for the exact model) while its KL is the worst: on raw text, outside the conversation format for which the model is tuned, a distorted distribution can predict the next token better than the exact model. That of the synonym $\lambda = 1$ stays at +1.4 % of the exact one (logarithm of the ratio 0.013 ± 0.016); that of $\lambda = 0$ goes down (411.0; -0.113 ± 0.015) [meas.]. The lot had drawn a sentence from it: ~~$\lambda = 0$ distorts the distribution in a privileged direction that $\lambda = 1$ does not take~~. On the new windows, the direction reverses: $\lambda = 0$ is at +2.4 % of the exact one (402.1 against 392.5) and $\lambda = 1$ at +15.6 % [meas.]; struck through by the judge (finding M3). The lesson of the synthesis is confirmed once more: only the KL judges fidelity.

10.5.7 The rate of the symbols

The entropies of the symbols over the 72 whole tensors of the 768 experts, by the scout's estimator (Chapter 3) [meas.]:

72 tensors, 768 experts	H_0 of the symbols	$H(\text{symbol} \mid \kappa)$
frontier	2.641 3	2.599 2
synonym $\lambda = 1$	2.568 3 (−0.073 0)	2.527 5 (−0.071 7)

The scout's prediction, on 16 experts, was −0.078 bit. By role (means of H_0, synonym against frontier): gate 2.556 against 2.645 (−0.089), up 2.630 against 2.643 (−0.013), down 2.518 against 2.635 (−0.117), with −0.62, −0.46 and −0.35 bit on the down matrices of layers 0, 1 and 2. The up matrix, which sees the same inputs as gate, hardly yields any bits to the entropy constraint; why is not measured [hyp.: the frontier's error for up, in the units of H, is small compared with its code-length gaps, and $\bar{c}$ normalises by it]. The real rANS coder of the companion [119] returns 2.586 1 bits per weight against 2.659 3 (−0.073 3): the same difference as the entropy, f32 biases, scales and medians included in both.

For $\lambda = 0$, the real companion costs 2.671 7 bits per expert weight: +0.012 4 over the frontier and +0.085 6 over $\lambda = 1$ [meas.]. Without an entropy constraint, the feedback chooses symbols slightly less predictable than the function of the code: $\lambda = 0$ pays for its KL in bits, where $\lambda = 1$ won both at once against the frontier.

10.5.8 The frontier of the variants

Figure 10.6 places the variants in the plane (bits per weight of the real companion, end-to-end KL): $\lambda = 1$ dominates the frontier on both axes; $\lambda = 0$ buys 0.020 4 of KL with 0.085 6 bit. The exact engine is outside the frame, at 3.86 useful bits per expert weight and a KL of 0.003 7 (07, § 4.3, writes 3.85 bits; the figure takes the useful bits of the v1.0 file measured by the judge, `mesures/genome/construction/juge/J-T0-fichiers.json`, the definition taken for the companions), and Q2_0 too, at 2.25 bits and 1.597. The bars are $\pm 2\sigma$ *by windows*.

Reserve R3. These points are measured with the 768 experts substituted; those of the round (Chapter 8, the common frontier) are measured with 32 experts, and no line links the ones to the others. A KL at 32 substituted experts and a KL at 768 do not measure the same thing: the first is the share of a subset, the second the whole model served by its base.

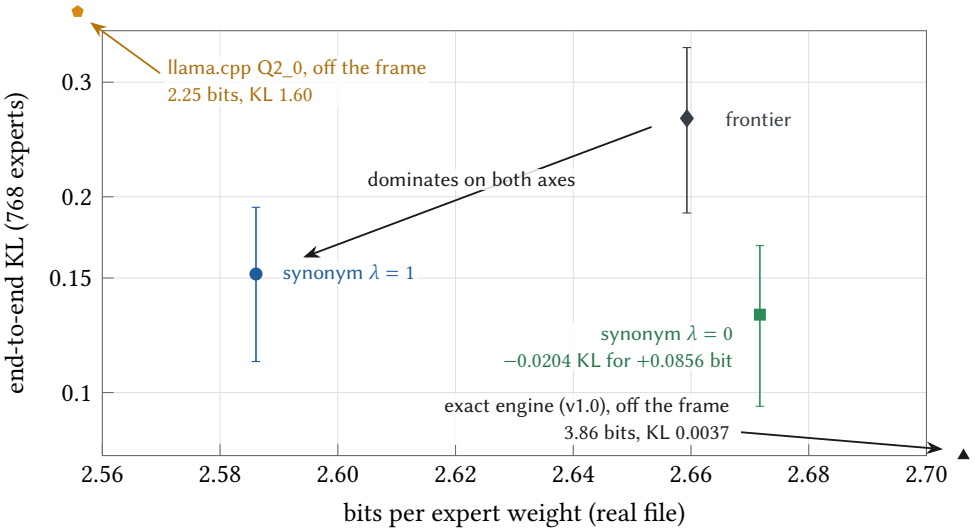


Figure 10.6: Synonym $\lambda = 1$ dominates the frontier on both axes; $\lambda = 0$ buys KL with bits. [meas.] All 768 experts of gpt-oss-20b replaced: bits per expert weight of the real companion file (useful bits) against the end-to-end KL on the ten windows of R1, logarithmic; bars: $\pm 2\sigma$ by windows, recomputed from the per-position files (for the frontier $\sigma = 0.0376$, against 0.0066 by positions: the per-position σ published in 07 is 3.2 to 8.9 times too small). $\lambda = 0$ buys 0.0204 of KL with 0.0856 bit. Margin landmarks, outside the frame (arrows): llama.cpp Q2_0 (its KL computed by llama.cpp on the processor, another path than the substituted reference) and the exact engine against the f64 reference (useful bits of the v1.0 file, the definition used for the companions). Reservation R3: these points are measured with 768 experts replaced; the points of the round (Chapter 8) are measured with 32 experts, and nothing joins them to these.

Data: `mesures/genome/construction/synonyme/figures/variantes-768.csv`, `kl/kl-*-768-positions.csv`, `repere/kl-q2_0-positions.csv`; `mesures/moteur/O/journaux/o3/c4-20b-comparer.json`; `mesures/genome/construction/juge/J-T0-fichiers.json`; script `memoire/outils/tome3_synonyme.py`.

10.6 The fast mode in the engine

S3-T1, the file. `trc-rapide verifier` rereads each companion: for the 768 experts, the decoded nibbles equal those of the `.sym`, the scales and biases those of the exact expert, the fingerprints those of the directory, and the MÉTA link to the exact header holds. Breaks (`--casser C X`): L7 E13 for the frontier, L19 E2 for $\lambda = 1$, L11 E27 for $\lambda = 0$; each time one false expert, the targeted one [meas.]. The frontier companion is 8 285 093 888 bytes (2.659 3 useful bits per weight, 2.660 0 aligned) against 11 157 405 696 for the exact v1.0 file; that of $\lambda = 0$, 8 314 617 856 bytes, written in 301 s.

S3-T2, the engine. `troncons generer --rapide --temoin` on the frontier companion, prompt of the O4 series, 32 tokens: 5 856 experts decoded on the card, brought back down and checked, 0 errors; the text is coherent [meas.].

S3-K, the two paths of the fast mode. Do the engine (f32, card) and the f64 reference return the same fast model? Declared thresholds: KL(fast f64 reference ‖ fast engine) ≤ 0.011 and identical first choice $\geq 95\,\%$. Their source: the gap measured between the exact engine and the exact reference on the 20b, $0.003\,68 \pm 0.000\,46$ and 97.18 % (criterion C4 of Volume II), multiplied by three for the amplification by the flips of a model further from the exact one ($3 \times 0.003\,68 = 0.011$) [calc.].

Table 10.5: The fast mode served by the engine (gpt-oss-20b, 768 experts, R1): bits, size, KL against the exact reference and the two paths (S3-K) [meas.].

Mode served	bits per weight	file	KL against the reference	first choice	S3-K
frontier (control)	2.659 3	8.29 GB	0.262 6	47.6 %	0.002 07; 98.5 %
synonym $\lambda = 1$	2.586 1	8.11 GB	0.152 2	67.4 %	0.001 71; 98.2 %
synonym $\lambda = 0$ (kept by B2)	2.671 7	8.31 GB	0.131 3	69.7 %	0.003 8; 97.8 %
exact (v1.0)	3.86	11.16 GB	0.003 7	97.2 %	—

Data: `mesures/genome/construction/synonyme/rapide/coder-*.json`, `moteur/kl-moteur-*-juge.json`, `moteur/S3K-*.json`; exact: criterion C4 of Volume II and `mesures/genome/construction/juge/J-T0-fichiers.json`.

S3-K holds for the three variants, and each break (the *frontier* f64 reference against the synonym engine) falls: 0.190 and 50.9 % of first choice for $\lambda = 1$, 0.149 and 55.0 % for $\lambda = 0$ [meas.]. The engine serves the fast mode as faithfully as it serves the exact mode, and its KL against the exact reference equals that of the substituted reference to within the gap between the two paths (0.262 6 against 0.264 0; 0.152 2 and 0.152 2; 0.131 3 against 0.131 8). On the new windows, the judge gives S3-K again for $\lambda = 0$: 0.003 7 and 97.8 % (Section 11.5).

10.7 Speed, and what the judge struck through

10.7.1 The lot's sessions

The declared rule: `troncons mesurer`, prompt and options of the O4 series (708 prompt tokens, 256 greedy tokens, rest of 10 s), session A B B A A B B A, A the exact v1.0 file, B the companion; mains power checked before each pass, card lock held; reference noise σ, the relative spread of the A passes among themselves; gain announced if the median ratio B/A exceeds $1 + 2\sigma$. Predicted at the disk ceiling: ×1.41 to ×1.45 (20b), ×1.43 to ×1.46 (120b).

Session s1 (20b): published, inconclusive. It ran during the KL pass B0 (six threads), the computation of the rate and the writing of the companion of the 120b, which reads 57 GB on the same disk. A passes: 2.64, 2.47, 3.18, 3.61 tokens/s (median 2.91); B passes: 3.43, 3.49, 3.36, 3.39 (3.41). Ratio 1.17, $\sigma_A = 17.8\,\%$, threshold 1.36: no gain announced [meas.]. The exact engine ran there at 2.5–3.6 tokens/s against 10.9 in the O4 series on a free machine.

Session s2 (20b), calm machine (on 22 October, from 1:11 to 1:25; two empty readings of the other streams at the start, no pass under load): A the exact v1.0, B the $\lambda = 0$ companion [meas.].

	A passes (exact v1.0)	B passes (fast $\lambda = 0$)
generation, tokens/s	9.39 · 9.69 · 9.83 · 9.39	5.46 · 11.83 · 12.22 · 11.85
median	9.54	11.84
prefill (708 tokens), tokens/s	10.8–11.4	16.9–17.7
bytes read per generation step	0.116 4 GB	0.115 2 GB
VRAM hits; misses over 255 steps	0.899; 2 472	0.856; 3 531
throughput per byte in generation	1.12 GB/s	1.39 GB/s

$\sigma_A = 2.3\,\%$, ratio 1.241, threshold 1.046: gain announced by the rule, ~~×1.24 in generation~~, and ×1.57 in prefill (published, not declared as a criterion). The two files do not generate the same text (the fast mode is lossy; text B falls into a loop), so the routed experts differ; per step, the bytes read are almost equal, and the gain comes from a higher throughput per byte. The lot proposed a cause for it: ~~the system's file cache, which keeps a larger share of a file of 8.3 GB than of a file of 11.2 GB~~ [hyp.].

120b-V, calm machine (on 22 October, from 2:34 to 3:38): A the exact v1.0 file (57.73 GB), B the frontier companion (40.45 GB) [meas.].

	A passes (exact v1.0)	B passes (fast frontier)
generation, tokens/s	2.08 · 1.87 · 2.12 · 1.88	3.95 · 3.97 · 3.98 · 3.97
median	1.98	3.97
prefill, tokens/s	1.43–1.56	2.99–3.01
bytes read per generation step	0.707 GB	0.464 GB
throughput per byte in generation	1.32 GB/s–1.50 GB/s	1.83 GB/s–1.85 GB/s
VRAM hits	0.591	0.610

$\sigma_A = 6.6\,\%$, ratio 2.00, threshold 1.13: gain announced by the rule, ~~×2.00~~, beyond what was predicted at the disk ceiling.

10.7.2 An identity, and what it separates

Proposition 10.5 (B3: the decomposition of a speed ratio)**.** *For a pass X, let v_X be its generation tokens per second, o_X the bytes read per step and $d_X = o_X/t_X$ its throughput per byte, $t_X = 1/v_X$ the time of a step. Then, exactly,*

$$\frac{v_B}{v_A} = \frac{o_A}{o_B} \cdot \frac{d_B}{d_A}.$$

Proof. $d_X = o_X v_X$, so $v_X = d_X/o_X$, and the quotient of the two gives the identity. ∎

The first factor is what a smaller format can return at equal throughput: its *ceiling of the bytes*. The second says whether the same byte was read faster. On the 120b, $1.524 \times 1.31 = 2.00$; on the 20b,

$1.010 \times 1.24 = 1.25$ (the medians give 1.24) [calc.]. The lot left the ×1.31 of the 120b "not measured" and the ×1.24 of the 20b as "[hyp.] the file cache". The identity does not say where the second factor comes from; it says only that it does not come from the bytes, hence not from the mechanism that one believed one was measuring.

10.7.3 What the judge measured

The judge put *three* files of each model in one and the same balanced session (A B R R B A A B R R B A): A the original v1.0, B the exact v1.2, R the fast companion; and it measured the disk stage alone of each file in the same session. Then $r(R/A) = r(R/B) \cdot r(B/A)$, and $r(R/B)$ says what the fast mode is worth against an exact file that the disk stage reads at the same speed (B and R: 0.956 and 1.009 times, below the separation threshold) [meas.]. The plan, its proof and its thresholds are in Section 11.6; the result fits in four numbers:

	$r(R/A)$	$r(R/B)$	ceiling of the bytes o_B/o_R	disk stage R/A
120b (frontier companion)	1.851	1.284	1.47	1.31
20b ($\lambda = 0$ companion)	1.220	0.942	0.98	1.39

- **The lot's measurements are reproduced**: $r(R/A) = 1.220$ on the 20b against 1.241; 1.851 on the 120b, with the settings of J1 (`--flux`, 326 slots) instead of those of the lot, which is enough for the gap with 2.00.
- **Their attribution to the fast mode falls** (serious findings G1 and G2 of the judge). On the 120b, the fast frontier mode returns ×1.28 in generation for a ceiling of the bytes of ×1.47, and ×1.12 in prefill; on the 20b, it generates 6 % *more slowly* than the exact v1.2, because it reads as many bytes per step (115.1 MB against 112.5): its routing, lossy, misses the VRAM more often (0.856 against 0.899). It wins only in prefill (×1.23), where the text is imposed and where the bytes follow the size of the experts.
- **The proposed cause is impossible.** The engine reads each chunk without going through the system cache (unbuffered reads of Windows), and the disk reads 1.000 0–1.005 7 times the bytes requested; no file cache is on the path.
- **The second factor of B3 is measured**: it is the disk stage, which reads the companion 1.31 (120b) and 1.39 (20b) times faster than the original v1.0 at equal bytes per read.

What makes the disk stage (reserve R1). On this disk, the gap follows *the pattern of read requests that the format imposes* (boundaries and lengths of the chunks; reserve R1, Section 11.6.5, where the overturned reading and its measurements are given). For this chapter, this changes the word, not the verdicts: B and R both have a fast pattern (about 11.6 and 8.3 MB per request), A the slow pattern of v1.0 (about 12.0 MB), and $r(R/B)$ compares two fast patterns. These ratios hold for this disk, on that day; they do not carry over to another machine without being remeasured there. The gain of v1.2 over v1.0 itself is in Chapter 9.

What else the judge flagged. The lot's speed rule looked only at the noise of the A passes ($\sigma_A = 2.3\,\%$ on the 20b); the B passes had $\sigma_B = 31\,\%$ because of the first one (5.46 tokens/s, just after the companion was written). With $\sigma = \max(\sigma_A, \sigma_B)$, the gain would not have been announced (finding M4). The declared rule was followed; it was too narrow to see that file B changed behaviour between its first and its second reading. The judge's session takes $\sigma = \max(\sigma_A, \sigma_B, \sigma_R)$.

10.8 The 120b

Why the synonymous mutation is not written for it. Three gaps, each costed [calc.]:

1. *No calibration input*: the scout's captures are those of the 20b; the capture pass of the f64 reference on the 120b would be needed (36 layers, 128 experts; that of the 20b took 1 966 s for 12 windows and 9.95 GB of experts read).
2. *The coding*: 4 608 experts at about 54 s each (the matrices of the 120b have the same dimensions) make $4\,608 \times 54 = 248\,832$ seconds, about 69 h on one process, about 23 h with the parallelism gain seen on the 20b ($\approx \times 3$). This is the case for which the specification plans the coder on the card.
3. *No judge*: the f64 reference of the 120b on the ten windows does not exist (Volume II, what the trial lacks); without it, no end-to-end KL.

What can be measured nonetheless (thresholds declared on 17 October at 12:10, before these measurements): the *frontier* companion of the 120b, without calibration, which has the size of the fast mode to within 0.07 bit, hence its speed.

- **120b-T1**: in the pass with a break, 4 607 correct experts out of 4 608, the only false one is the targeted one (L30 E77); MÉTA link held [meas.].
- **120b-T2**: `troncons generer --rapide --temoin`, 32 tokens: 56 671 experts decoded on the card, brought back down and checked, 0 errors, coherent text; 471 GB read in 2 010 s (the control mode brings each expert back down: this is not a speed) [meas.].
- **A quality proxy, validated first where the true judge exists.** On the 20b, the exact engine against the f64 reference gives C4 again to the last digit ($0.003\,68 \pm 0.000\,46$, 97.18 %), and KL(exact engine ‖ fast engine) equals the KL against the f64 reference to within 0.002: frontier 0.264 5 against 0.262 6, $\lambda = 1$ 0.153 3 against 0.152 2, $\lambda = 0$ 0.133 1 against 0.131 3 [meas.]. The gap of the exact engine to the reference (0.003 7) is small compared with the KLs of the fast mode: the proxy measures them to within 1.5 %.
- **On the 120b** (ten windows of R1, `--flux` like trial J1): KL(exact engine ‖ fast frontier engine) $= 0.140\,4 \pm 0.004\,2$ (median 0.084; 99th percentile 0.98; 99.9th 2.22), first choice 67.5 %, perplexity 401.4 against 422.2 for the exact engine. Controls: the exact engine of the 120b returns the CSV fingerprint of trial J1 (`a80f5f04…9498`) and its perplexity (422.21); exact against exact: 0 [meas.]. The ± is naive, like those of 07.

The same frontier base costs the 120b half as much KL as the 20b (0.140 against 0.265): more experts (128 against 32, four routed each time), and a smaller share of each output passes through a given expert [hyp., not measured]. If the synonym-over-frontier ratio of the 20b (0.50 for $\lambda = 0$) carried over, the synonym of the 120b would be around 0.07; this is an extrapolation, not a measurement.

10.9 What remains

1. **The synonymous mutation of the 120b**: the capture pass of the f64 reference on the 120b, the coder on the card, then the same end-to-end KL with the f64 reference of the 120b on the ten windows.
2. **The mixed miss policy** (serving the base of an absent expert while the exact one arrives) is to be taken up with $r(R/B)$ and not $r(R/A)$: on the 120b, ×1.28 in generation for a KL of 0.14 (proxy); on the 20b, nothing in generation, ×1.23 in prefill. The specification asked for it to be simulated on the traces before being built; this is all the more necessary.

3. **Another corpus** (code, dialogue): the judge's new windows are from wiki.test, like R1 and the calibration.
4. **The tail of the KL** (99.9th percentile 3.37 with $\lambda = 0$): these are the routing flips. The tropical guard was the piece that would target it; it falls at the declared threshold (Chapter 12), which leaves this tail without a mechanism.
5. **The fast mode as a draft** for speculative decoding verified by the exact mode, whose text would be that of the exact model to the bit: an idea of Stone's ("un genre de puissance de raisonnement différent", *a different kind of reasoning power*), not measured here.

10.10 What could be wrong

What could be wrong

- **B2 between the two λ was decided narrowly on R1** ($\bar{d} + 2\sigma = -0.000\,9$, one window in ten against). With ten windows, σ by windows is itself uncertain by about 24 % ($1/\sqrt{2 \cdot 9}$) [calc.]. The judge gave it again at $3.6\,\sigma$ on ten new windows, but still from wiki.test and by a single path for $\lambda = 1$ (the engine; the $\lambda = 1$ substituted reference was not run there). Another kind of text could return "equal", and the rule would then keep $\lambda = 1$. Both companions remain written, verified and served.
- **The unrouted experts**: 36 to 39 experts are routed by none of the 2 550 positions of R1 under substitution, and 11 by none of the twenty windows under the exact reference (Figure 11.1). Their files are verified (S3-T1); no KL judges their base.
- **The spectrum and the cloud are one matrix of one expert** (L8, E0, gate): the shape of the transformation is seen there, not established for the 2 304 matrices. The two other spectra of the same folder are not drawn.
- **The absolute uncertainties of 07** are naive; those of Table 10.3 by windows are right for independent windows, which ten windows of a single article may not quite be.
- **The llama.cpp benchmark is weak** (Q2_0 without an importance matrix); a better requantisation at 2.6 bits or less does not exist in b11515 for rows of 2 880. Beating this benchmark says nothing about the place of the synonym in the state of the art.
- **Speed**: one session per model and per judge, on a single disk, a single morning; the ratios $r(R/B)$ rest on the fact, measured in the session, that B and R are read at the same speed at the disk stage. If this were no longer true on another day, the session would have to be redone, not the figure reused. And on a machine where the v1.0 pattern were not slow, $r(R/A)$ itself would change.
- **The 120b** has only the frontier (without calibration); its quality proxy is a KL between two outputs of the engine, without the f64 reference, validated on another model.
- **The analogy** of the synonymous codon describes the mechanism; it does not prove it. What proves it is the KL, on text kept apart.

Chapter 11

The judge of the construction

Where we start, what this chapter establishes, what remains

At the start, two lots that call themselves finished, and figures produced by their authors alone, on the windows where their mechanisms were tuned. Lot A publishes a lossless `.trc` v1.2 format (the sign of a zero folded, 1.80 GB fewer on the 120b), logits identical to the bit on R1, and a J1 of the 120b 1.45 times faster in v1.2, which it did not itself attribute to the format (Chapter 9); the placement judge has since attributed it to the v1.2 format, through the pattern of its read requests, specific to this disk (reserve R1, Section 9.7.1 and Section 11.6.5). Lot B publishes the synonymous genome on the 768 experts of the 20b, the rule B2 that keeps $\lambda = 0$ narrowly, a fast mode in the engine and two speed gains, ×1.24 and ×2.00 (Chapter 10). The SOCLE requires the work to be reread by a judge that did not do it. **This chapter establishes**, by a judge's own paths (a decoder of the format written from its description alone, a GGUF reader of its own, a KL computed by its code, ten windows of text that the lots never saw, a speed session with three files), what holds, what falls, and what is true but wrongly attributed; and it poses in full the five derivations on which its verdicts rest. **What remains** is the decision on the mixed miss policy, another corpus, and the exact variable of the read pattern that makes the speed of this disk (Section 11.10). Sources: `genome/JUGEMENT-construction.md`, `genome/construction-juge-pour-le-memoire.md`, `genome/JUGEMENT-placement.md`, `mesures/genome/construction/juge/` (thresholds: `00-seuils-declares.md`) and `outils/genome/construction/juge/`.

In brief

A judge redoes. It does not reread the others' computations: it makes other ones, by paths that share with those of the lots only what they must share, the description of the format and the files judged. The judge of the construction wrote neither the decision on the v1.2 format, nor the coder, nor the fast companion, nor the engine. It declared its thresholds on 22 October 2026 around 4 a.m., when none of its tools had yet read one byte of a model. Its verdict fits in two sentences. The construction delivers what it promised *in bytes and in quality*: the zero-synonym codon is lossless and exact to the bit on two models and on new text; the synonymous genome $\lambda = 0$ beats the frontier and $\lambda = 1$ on this new text, more clearly than on the text where it had been chosen. It does not deliver the speed gain announced for the fast mode: the two published ratios are reproduced, but they mostly measured the way this disk reads the old format. This chapter shows how the judge separates it, and why it can be believed.

11.1 What the judge redid, and by which path

Its tools. Everything is under `outils/genome/construction/juge/`; the engine, `juge` and `genome-capture` are built by the judge from `main` (commit `f136e27`) in its own target. Its decoder of the format (`decodeur_juge.py`, Python and numpy) reads the header, MÉTA, RÉSIDENTS, TABLES, RÉPERTOIRE and the chunk parts from the specification alone (`decisions/ARCHITECTURE.md`, §§ 4.2 to 4.12 and 4.16), and the GGUF through its own reader (v3: pairs, tensor information, alignment). It opened neither the crates of the format, nor the codec, nor the decoder of trial J1, nor the tools of lot A before writing it. A single error at the first run, in its MÉTA reader: `self.p += self.u64()` loses the 8 bytes of the length, because Python reads `self.p` before calling `u64`. Corrected and stated in commit `f136e27`, it touched only the check of MÉTA, not the decoding.

Its verdicts , figure by figure (Table 11.1).

Table 11.1: What the judge of the construction redid, by which path, and its verdict [meas.]. The attributions struck through are struck through in the judgement itself.

Published figure	Judge's path	Result	Verdict
sizes of the `.trc` v1.2, gain 0.127 91 and 0.125 62 bit	its reader of the format, whole files	given again to the byte	holds
C1 in value, 768 + 4 608 experts	its decoder, 96 drawn experts and 40 v1.0 controls	96/96, 40/40; three breaks fall	holds
no v1.2 table with a slot for an 8	its reader of the tables	0 out of 540 (540 out of 540 in v1.0)	holds
logits to the bit, 20b and 120b	ten *new* windows, its script, a new break	0/5 120 and 0/5 120; break 512/512	holds
J1: 2.89 against 1.99 tokens/s, $r = 1.454$	three-file session	2.889 against 2.004, $r = 1.442$	holds; attribution corrected (R1)
KL of R1; B2 $-0.020\,4 \pm 0.009\,7$	its KL on the files of lot B	given again	holds
B2: $\lambda = 0$ better ($2.09\,\sigma$)	new windows, engine	$-0.030\,3 \pm 0.008\,4$, $3.6\,\sigma$	holds, strengthened
$\lambda = 0$ better than the frontier	new windows, substituted reference	$-0.156\,5 \pm 0.033\,4$, $4.7\,\sigma$	holds
S3-K: fast engine = substituted reference	new windows	0.003 7; 97.8 %	holds
"the tail falls too", $\lambda = 0$ against $\lambda = 1$	new windows	99th −5 %; 99.9th +6 %	struck through (M2)
"the tail falls too", against the frontier	new windows, substituted reference	99th −26 %, 99.9th −18 %	holds
20b ×1.24, gain of the fast mode	three-file session	$r(R/A) = 1.220$; $r(R/B) = 0.942$	struck through (G2)
120b ×2.00, gain of the fast mode	three-file session	$r(R/A) = 1.851$; $r(R/B) = 1.284$	struck through (G1)
cause of the ×1.24: the file cache	reading of `trc-es`, disk stage	unbuffered reads; disk R/A 1.39	struck through (G2)

Data: `genome/JUGEMENT-construction.md`, §§ 1 and 8.

11.2 The .trc v1.2 through the judge's decoder

11.2.1 Vectorised rANS decoding is normative decoding

An rANS stream of the format is cut into independent tiles of 32 lanes [119, 120]. Normative decoding (specification § 4.9.2) goes through the macro-steps m of a tile, then its sub-steps k, then its lanes $l = 0, \dots, 31$ *in order*. In Python, a code matrix of 2 880 × 2 880 weights would require 4 147 200 sequential steps; the judge does them in 1 024 iterations (256 macro-steps × 4 sub-steps), on an array of $T \times 32$ states at once. It must then be proved that this form is normative decoding.

Notation. A tile has 32 lanes of states $x_l \in [L, 2^{32})$ and a pointer p into the shared array `mots`. A step of lane l computes, with the table c of the context of its word: slot $= x_l \bmod M$; $s = \mathrm{sym}_c(\mathrm{slot})$; $y_l = f_c(s)\,\lfloor x_l/M \rfloor + \mathrm{slot} - \mathrm{cum}_c(s)$; if $y_l < L$, $x_l \leftarrow 2^{16} y_l + \mathtt{mots}[p]$ and $p \leftarrow p+1$, otherwise $x_l \leftarrow y_l$. The format fixes $M = 2^{12}$ and $L = 2^{16}$ (Volume II, chapter on the format).

Lemma 11.1 (J1: the lanes are independent within a sub-step)**.** *At sub-step (m, k), y_l depends only on x_l and on the table of the context of word $32m + l$.*

Proof. The only inputs of the computation of y_l are x_l, the slot (a function of x_l) and the table; no other lane enters it, and the pointer p intervenes only afterwards, in the renormalisation. ∎

Proposition 11.2 (J2: the assignment of the words is a prefix sum)**.** *Let $r_l = [y_l < L]$ and $R_l = \sum_{l' \le l} r_{l'}$. Sequential processing $l = 0, \dots, 31$ gives every lane l such that $r_l = 1$ the word $\mathtt{mots}[p_0 + R_l - 1]$, where p_0 is the pointer at the start of the sub-step, and leaves the pointer at $p_0 + R_{31}$.*

Proof. By induction on l. Before lane l, the pointer is $p_0 + R_{l-1}$ (with $R_{-1} = 0$): each of the preceding lanes that renormalised advanced it by one. If $r_l = 1$, the lane reads $\mathtt{mots}[p_0 + R_{l-1}] = \mathtt{mots}[p_0 + R_l - 1]$ and brings the pointer to $p_0 + R_l$; otherwise it stays at $p_0 + R_{l-1} = p_0 + R_l$. At the end, it is $p_0 + R_{31}$. ∎

Remark 11.3 (A single word per lane and per sub-step). The proposition assumes that a lane that renormalises reads only one word. This is true: before the step, $x_l \ge L = 2^{16}$, so $y_l \ge \lfloor x_l / 2^{12} \rfloor \ge 2^4$ since $f_c(s) \ge 1$ and $\mathrm{slot} - \mathrm{cum}_c(s) \ge 0$; if $y_l < L$, the new state $2^{16} y_l + w \ge 2^{20} \ge L$. Volume II poses the same argument in the lemma of the vectorised decoder of trial J1.

By lemma J1, all the y_l of a sub-step are computed before any write; by proposition J2, the new states too, by the prefix sum `cumsum(renouvelle)` along the lane axis. The tiles being independent (each starts from its beginning and its 32 states), the decoder processes the $T \times 32$ lanes of all the tiles of a stream at each sub-step. The end of tile (every lane back on L) is checked at the end. A code matrix is thus decoded in about 0.5 s, a whole expert in 1.8 s–2.1 s, checks included [meas.].

11.2.2 J-T0: the four whole files

The gain on the aligned parts is 0.127 91 bit per expert weight on the 20b and 0.125 62 on the 120b; threshold A-C2 of lot A (≥ 0.12, taken as is) holds on both [meas.]. The computation, for the 20b: 8×(9 226 493 952−8 920 932 352) = 8×305 561 600 bits, divided by the $24 \times 32 \times 3 \times 2\,880^2 = 19\,110\,297\,600$ expert weights, make 0.127 91 [calc.]. The eight sizes of lot A are given again to the byte, and its 540 tables too (540 out of 540 in v1.0, 0 in v1.2). The fingerprint of the RÉSIDENTS table lies at the exact end of the data area, without padding: the description ("after the data area") reads that way, and the file obeys it.

Table 11.2: J-T0: the four `.trc` files, read in full by the judge's reader [meas.]. Structural faults: header, fingerprints of the header, of MÉTA, of the RÉSIDENTS table, of TABLES and of the RÉPERTOIRE, entries, declared size equal to the size on disk.

	20b v1.0	20b v1.2	120b v1.0	120b v1.2
structural faults	0	0	0	0
file (bytes)	11 157 405 696	10 851 844 096	57 730 662 400	55 930 175 488
chunk parts, aligned	9 226 493 952	8 920 932 352	55 416 086 528	53 615 599 616
useful bytes of the chunks	9 224 936 032	8 919 387 648	55 406 705 152	53 606 030 768
largest chunk	12 144 640	11 755 520	12 251 136	11 878 400
CODES_CTX tables with a slot for an 8	216 out of 216	**0 out of 216**	324 out of 324	**0 out of 324**

Data: `mesures/genome/construction/juge/J-T0-fichiers.json`.

11.2.3 J-T1: 96 drawn experts, decoded by the judge

The draw is written in the file `tirage-experts.json`, committed before any decoding (commit `93a88a5`). It takes 32 experts of the 20b then 64 of the 120b, without replacement, by `Generator(PCG64(20261022))`. For each, in v1.2, eight checks: (a) each lane of each tile ends on L; (b) the reconstructed blocks $[e, qs]$ equal byte for byte to the blocks of the GGUF canonicalised by the judge's code (nibble 8 → 0); (c) the biases equal to the bit; (d) each weight equal *in value* to the weight of the non-canonicalised GGUF; (e) the fingerprint of the reconstructed canonical expert equal to `empreinte_canonique`; (f) the fingerprint of the GGUF blocks as they are equal to `empreinte_origine`; (g) the fingerprint of the part equal to that of the directory; (h) no nibble 8.

- **96 experts out of 96 hold the eight checks** [meas.].
- **The control** (answer known in advance): the 32 experts of the 20b and the first 8 drawn from the 120b, decoded in *v1.0*, return the GGUF as it is (40 out of 40); and for each, the nibbles that change between the v1.0 decoded and the v1.2 decoded are exactly the −0 of the GGUF (L19 E109 of the 120b: 1 746 601 changed, 1 746 601 −0) [meas.].
- **The breaks** all fall: a word of the CODES_CTX stream changed (lanes outside L, bytes, values and canonical fingerprint false); the canonicalisation removed from the GGUF reader (bytes and canonical fingerprint on the GGUF side false; the fingerprint of the decoded, which does not read the GGUF, holds: the judge had announced it as falling, and says so); one bit of the first bias of matrix 2 (bias, fingerprint of the part and canonical fingerprint false) [meas.].
- The share of the −0 in the sample, 6.42 ± 0.11 % (20b) and 6.33 ± 0.10 % (120b), against 6.393 % and 6.279 % over the files: at 0.3 and 0.5 standard deviation.

The figure of these decodings (Figure 9.7) is in the chapter on the codon, with which it forms the verdict on lot A.

11.2.4 What the v1.0 → v1.2 control proves

Let $D_0(x)$ and $D_2(x)$ be the decoded nibbles of expert x in v1.0 and v1.2, $G(x)$ those of the GGUF, and $\kappa(g) = g$ if $g \neq 8$, 0 otherwise (code 8 is the −0 of E2M1). The control measures $D_0(x) = G(x)$, and check (b) measures $D_2(x) = \kappa(G(x))$. Then

$$\{i : D_0(x)_i \neq D_2(x)_i\} = \{i : G(x)_i \neq \kappa(G(x)_i)\} = \{i : G(x)_i = 8\} :$$

the set of changed nibbles is *exactly* that of the −0, which the log checks expert by expert. This is the measured form of the sentence "v1.2 changes only the sign of the zeros", and, with (d), of "v1.2 is lossless in value".

Verdict J-T: holds. The `.trc` v1.2 is exactly what the format describes, and each drawn expert is the expert of the GGUF to the bit, up to the sign of the zeros.

11.3 The new windows, and the experts they call

The draw. The lots evaluated on R1, the first ten windows of 512 tokens of wiki.test (tokens 0 to 5 119) [125], and calibrated the synonymous genome on windows 10 to 21 (tokens 5 120 to 11 263). The judge sets aside the first 120 000 bytes of `wiki.test.raw` (SHA-256 `173c87a5…dd08`), which count 27 680 tokens: they cover R1, the calibration, and twice as much margin [meas.]. Beyond them it draws ten paragraph line starts (seed 20 261 022+1), more than 12 000 bytes apart from one another: at bytes 767 283, 511 976, 615 155, 854 814, 1 148 495, 1 228 294, 1 098 980, 1 240 966, 421 030 and 358 205. Each extract (9.0–10.4 KB) goes through `llama-perplexity` (processor only, options of the base, one window), and the ten files are merged into a single `base-juge-20b.kld`. Its size is checked by computation: $20 + 4 \times 5\,120 + 2\,550 \times 402\,184 = 1\,025\,589\,700$ bytes, the size of R1; each window is equal to its source byte for byte, and a shifted window does not equal it (break) [meas.] [calc.]. The format of `llama-perplexity` merges by simple concatenation of the tokens then of the entries: all the tools of the project read the merged file without change.

The reference. The f64 reference of these windows (`juge reference --yarn openai`, the call of R1) took 2 168 s and routed 755 distinct experts (R1: 734). KL(llama.cpp processor ‖ f64 reference) is 0.033 9 there (R1: 0.025 5): slightly harder windows, nothing abnormal [meas.].

The experts seen. Figure 11.1 draws the map: each cell of the 24 × 32 grid of the 20b, coloured according to the windows that routed tokens to it under the exact reference. 732 experts are routed by both sets, 2 by R1 alone, 23 by the new windows alone (465 calls out of 490 560), and 11 by neither; $732 + 2 + 23 + 11 = 768$ [meas.] [calc.]. Lot B counted 36 to 39 unrouted on R1 under substitution; the new windows judge part of them (the substituted f64 pass $\lambda = 0$ loads 756 experts there).

What the map says, and what it does not. The 11 struck-out cells are not "unverified" experts: their decoding is verified in value, like that of the 757 others (criterion C1 of lot A, 768 experts out of 768, Chapter 9). They lack a KL, not a measurement: no KL measurement of the construction called them, under the routing of the reference. A change confined to these 11 experts would move no KL of Chapters 9 to 11 measured with this routing; but a variant whose routing flips could call them. The first caption of the figure said more than that; the second judge of the figures had it rewritten.

11.4 J-C1: the logits to the bit on the new windows

`troncons evaluer --flux --empreintes`, ten new windows, engine built by the judge; comparison by its script, not by that of lot A. Threshold taken from lot A (proposition A1, criterion A-C1): 0 differing logit fingerprints out of 5 120 steps per model, and the output `.kld` equal byte for byte (SHA-256).

Verdict J-C1: holds , on two models and on a text that no one had seen. Proposition A1 of lot A (Chapter 9) is true of the real engine. The break was chosen to touch every position (each position routes four experts of layer 0 out of 32, and all have their bias flipped); the broken copy is erased afterwards.

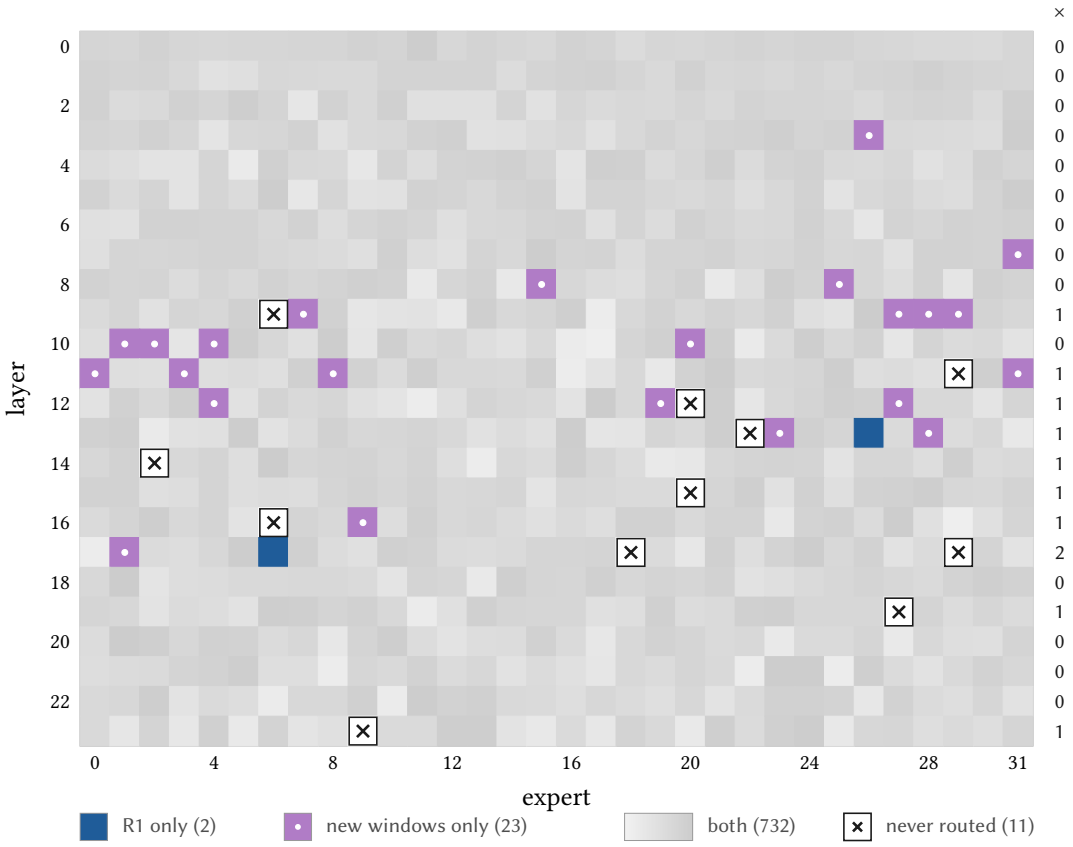


Figure 11.1: 11 experts of gpt-oss-20b were never routed, on either set of windows: no KL measurement of the construction called them (their decoding, for its part, is checked by value: C1, 768 experts of 768). [meas.] The 24 × 32 experts of gpt-oss-20b, each cell coloured by the windows that routed tokens to it: the ten windows of R1 only (blue), the ten new windows of the judge only (purple, white dot), both (grey, darker with the number of calls on a logarithmic scale), or none (framed and crossed; their count per layer in the margin). A call is one token routed to an expert; a window routes its 511 tokens, of which 256 are context not scored (490 560 calls in all on each set of windows; at most 4 764 for one expert on one set, 9 331 on both together). It is the map of the space the router visits on real text, under the routing of the reference: a change confined to the 11 crossed experts would move no KL of Chapters 9 to 11 measured with that routing, but a variant whose routing flips could call them.

Data: `mesures/genome/construction/juge/figures/routage-grille.csv`, `J-T0-fichiers.json` (*k*); `mesures/genome/construction/synonyme/kl/kl-frontiere-768-positions.csv` (scored positions); `mesures/genome/construction/codon/tailles.json` (C1); script `memoire/outils/tome3_juge.py`.

Table 11.3: J-C1: v1.2 against v1.0, logits on ten new windows [meas.].

	20b	120b
v1.2 against v1.0: differing logit fingerprints	**0 out of** 5 120	**0 out of** 5 120
output `.kld`	equal (`1aff1a6a…29ef`)	equal (`709367e7…8a73`)
control: v1.2 evaluated twice	identical CSVs (`fc427e9f…469b`)	—
break: the sign of the bias of row 0 of gate flipped for the 32 experts of layer 0, `--sans-garde`, window 0	**512 out of 512 differing**, from position 0; 73 first choices changed	—

Data: `mesures/genome/construction/juge/logits/comparer-*.json`, `logits/evaluer-*.\{csv,json,log\}`.

11.5 J-K: the KL of the synonymous genome outside R1

11.5.1 The judge's KL, and its control

For each scored position, the judge computes

$$\mathrm{KL}(P\|Q) = \sum_{i\,:\,\log p_i > -16} p_i\,(\log p_i - \log q_i),$$

P and Q decoded from their 16-bit coding, without renormalisation: this is the convention of lot R (Volume II, the reference and the judge), where a file against itself returns 0 [11]. **Control**: on a pair published by lot B (R1, $\lambda = 0$), its KL is 0.131 752 831 against 0.131 752 832 for `juge kl` ($1.1 \cdot 10^{-8}$ relative, threshold 10^{-6}); **break**: Q shifted by one position, 1.634 [meas.]. R1 recomputed by its code from the files of lot B: frontier 0.263 99, $\lambda = 1$ 0.152 16, $\lambda = 0$ 0.131 75; paired $-0.111\,83 \pm 0.029\,62$, $-0.020\,40 \pm 0.009\,74$ ($\bar{d} + 2\sigma = -0.000\,92$), $-0.132\,24 \pm 0.023\,43$: the figures of lot B are given again to four digits and more [meas.].

On the new windows, two paths: the *engine* (`troncons evaluer --flux --rapide` on the companions, the product as it is served) and the *substituted f64 reference* (`genome-capture kl --sub`, the lots' path). The truth is the f64 reference of the new windows.

11.5.2 The numbers

Table 11.4: J-K: the KL against the f64 reference on ten new windows (2 550 positions), the 768 experts substituted [meas.]. Perplexity of the reference: 392.5.

New windows	KL	σ naive	σ windows	median	99.9th pc.	first choice	perplexity
exact engine, v1.2	0.004 37	0.000 8	0.002 1	0.000 03	0.557	98.0 %	394.0
$\lambda = 0$, fast engine	0.144 8	0.006 0	0.016 4	0.060 8	3.61	71.7 %	402.1
$\lambda = 0$, substituted f64	0.144 6	0.005 9	0.016 5	0.061 7	3.51	71.3 %	399.5
$\lambda = 1$, fast engine	0.175 1	0.006 2	0.018 6	0.075 5	3.40	67.6 %	453.7
frontier, substituted f64	0.301 1	0.007 4	0.041 5	0.191 7	4.30	51.5 %	239.8

Data: `mesures/genome/construction/juge/kl/kl-neuf-*.json`; $\lambda = 1$ through the substituted reference: not run (Section 11.11).

The declared verdicts.

- **J-K2, B2 redone outside R1** (the rule of lot B, taken as is): through the engine, $\bar{d} = \mathrm{KL}_{\lambda=0} - \mathrm{KL}_{\lambda=1} = -0.030\,3$, σ by windows $0.008\,4$, $\bar{d} + 2\sigma = -0.013\,6 < 0$ ($3.62\,\sigma$, against 2.09 on R1); nine windows out of ten (window 0 against, +0.016); $\lambda = 0$ has less KL on 1 858 positions out of 2 550; first choice $+4.1 \pm 1.1$ points. The judge's prediction ("equal") is refuted.
- **J-K1, $\lambda = 0$ against the frontier**, through the substituted reference: $-0.156\,5 \pm 0.033\,4$ ($4.7\,\sigma$), all ten windows, 2 297 positions out of 2 550, first choice $+19.8 \pm 3.2$ points; ratio $\lambda = 0$ over frontier 0.480 (R1: 0.499). Holds, as predicted.
- **J-K3, the fast engine against the substituted f64** (thresholds S3-K of lot B): $0.003\,7 \leq 0.011$ and $97.8\,\% \geq 95\,\%$, holds; the two paths return the same KL against the truth, $0.144\,8$ and $0.144\,6$, paired difference $+0.000\,2 \pm 0.000\,6$.
- **J-K4, the exact mode**: KL $0.004\,37$, first choice 98.0 % (R1: $0.003\,68$ and 97.2 %); llama.cpp on the processor gives $0.033\,9$ there. The form of C4 holds against llama.cpp on the processor; against llama.cpp on the card, the exact comparator of C4, not measured here.

A single path for $\lambda = 1$. The $\lambda = 1$ substituted reference was not run on the new windows (time: 47 to 52 min per pass under the machine's load, and the disk placement stream was waiting for calm for its session). B2 is therefore judged there by the engine alone. The judge justifies this by two measured agreements: the two paths return the same KL for $\lambda = 0$ to within 0.000 2 on these windows, and for $\lambda = 1$ on R1 (0.152 2 by both, lot B). Mixing the paths in a paired difference ($\lambda = 1$ engine against frontier f64: $-0.126\,0 \pm 0.036\,6$) is published for information only; it is the panel with hollow diamonds of Figure 10.4.

What else the new windows say. *The level*: $\lambda = 0$ gives $0.144\,8$ on the new windows and $0.131\,8$ on R1; the exact engine $0.004\,4$ and $0.003\,7$; the llama.cpp reference $0.033\,9$ and $0.025\,5$. These windows are harder for everyone, by 10 to 33 %; the synonymous genome does not degrade there more than the rest [calc.]. *The tail* and *the perplexity*: two sentences of lot B fall, struck through in Chapter 10 (findings M2 and M3). *The experts*: the new windows bring into play 23 experts that R1 had never routed; their share of the calls is small (0.1 %), and the KL of $\lambda = 0$ includes them.

11.5.3 The uncertainty of a mean KL: by positions or by windows

The 255 scored positions of a window share their past, hence their routing flips: they are not independent. Two standard deviations of the mean stand against each other:

$$\sigma_f = \frac{s(\bar{K}_1, \dots, \bar{K}_{10})}{\sqrt{10}}, \qquad \sigma_n = \frac{s(K_1, \dots, K_{2\,550})}{\sqrt{2\,550}},$$

the first on the ten window means, the second on the positions as if they were independent. Their ratio defines the *design effect* and the number of effective positions:

$$\mathrm{deff} = \left(\frac{\sigma_f}{\sigma_n}\right)^2, \qquad n_{\mathrm{eff}} = \frac{n}{\mathrm{deff}}.$$

Proposition 11.4 (The design effect of a cluster sample)**.** *Let there be n clusters (the windows) of m positions, each position of variance s^2, two positions of the same cluster with correlation ρ, and two positions of different clusters independent. Then the variance of the overall mean is $\frac{s^2}{nm}\big(1 + (m-1)\rho\big)$, and* $\mathrm{deff} = 1 + (m-1)\rho$.

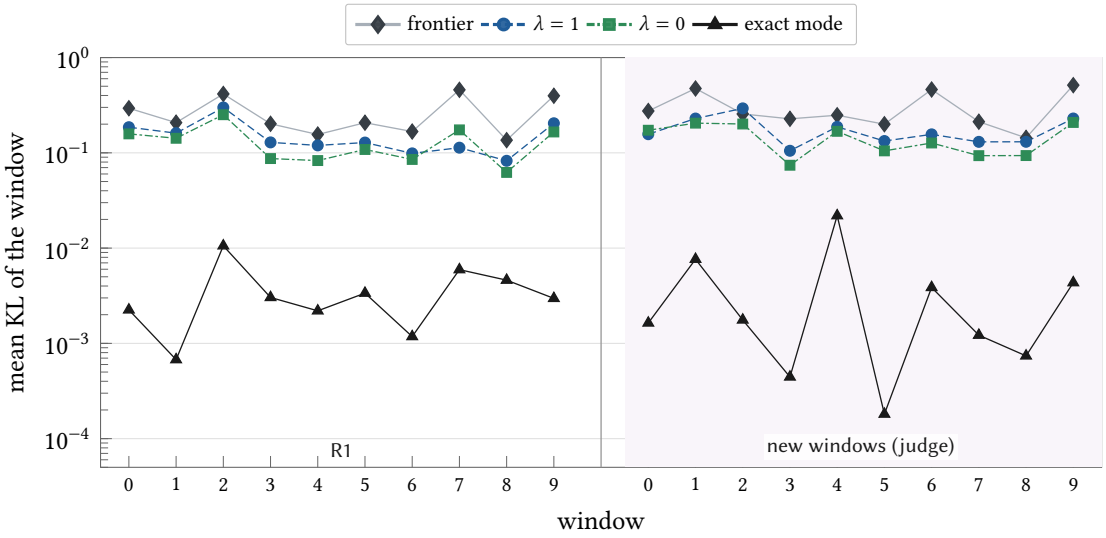


Figure 11.2: Some windows are harder than others for every variant alike; the exact mode stays far below them all. [meas.] Mean KL of each window (255 scored positions) for the frontier, the two synonyms and the exact mode, on the ten windows of R1 and the ten new windows of the judge (shaded). The thin lines join windows that do not follow one another: they only show that the variants rise and fall together. Paths: on R1, the variants through the substituted f64 reference and the exact mode from the per-position file of lot O; on the new windows, $\lambda = 0$, $\lambda = 1$ and the exact mode through the engine, the frontier through the substituted reference. One path is drawn per series and per set. On R1 the three variants also went through the engine: engine minus substituted reference, −0.0014 on average for the frontier, 0.0001 for $\lambda = 1$, −0.0004 for $\lambda = 0$, from −0.0076 to 0.0029 by window over the three; on the new windows, $\lambda = 0$ went through both, 0.0002 on average. The frontier ranges from 0.136 to 0.459 on R1: this spread from window to window, shared by all variants, makes the standard deviation by windows ($\sigma_f = 0.0376$ for the frontier) much larger than the naive one by positions, and pairing removes it (Figure 10.4). The new windows are drawn from wiki.test, the corpus of R1.

Data: `mesures/genome/construction/juge/figures/kl-par-fenetre.csv`; `mesures/moteur/O/journaux/o3/c4-20b-comparer.csv`; `mesures/genome/construction/synonyme/moteur/kl-moteur-frontiere-positions.csv`; script `memoire/outils/tome3_juge.py`.

Proof. The mean of a cluster, $\bar{K}_g = \frac{1}{m}\sum_i K_{gi}$, has variance $\frac{1}{m^2}\left(ms^2 + m(m-1)\rho s^2\right) = \frac{s^2}{m}\left(1 + (m-1)\rho\right)$: m variances and $m(m-1)$ covariances. The clusters being independent, the mean of the n means has n times less variance. The naive variance, which ignores the covariances, is $s^2/(nm)$; the quotient is $1 + (m-1)\rho$. ∎

On R1, frontier. $\sigma_f = 0.037\,6$, $\sigma_n = 0.006\,6$: deff ≈ 32 and $n_{\text{eff}} \approx 79$ [calc.]. A window of 255 positions is therefore worth about eight independent positions (255/32), and by the proposition, the mean correlation of two positions of the same window is $\rho \approx (32-1)/254 \approx 0.12$ [calc.]. Figure 11.2 shows where it comes from: some windows are harder than others for all the variants at once (the frontier ranges from 0.136 to 0.459 from one window of R1 to another), and the exact mode stays far below all of them (from 0.000 2 to 0.021 9).

For a paired difference , the design effect is much smaller, because pairing removes the difficulty specific to each window, common to both variants: for $\lambda = 0$ against $\lambda = 1$ on R1, $\sigma_f = 0.009\,74$ and $\sigma_n = 0.003\,85$, deff ≈ 6.4 [calc.] (`mesures/genome/construction/synonyme/kl/B2-synonym`

`e-11-contre-10-768.json`). This is why the lots are right to judge by paired differences, and why an absolute KL published with σ_n looks three to six times more precise than it is (finding M1). The judge's recipe: publish an absolute KL with its σ by windows.

11.5.4 Redoing a result at two standard deviations

Why did the judge predict "equal"? Take a paired difference measured at $z = \bar{d}/\sigma$ standard deviations, and suppose that the true difference is exactly $\bar{d}$, and that the second series of ten windows has the same σ. The new estimator roughly follows $\mathcal{N}(\bar{d}, \sigma^2)$; with $\bar{d} = -z\sigma$, the probability that it passes the threshold $\bar{d}' + 2\sigma < 0$ again is

$$P\big(\bar{d}' < -2\sigma\big) = \Phi\Big(\frac{-2\sigma + z\sigma}{\sigma}\Big) = \Phi(z - 2).$$

For $z = 2.09$, $\Phi(0.09) \approx 0.54$ [calc.]: barely more than one chance in two. And if the true difference is smaller than $\bar{d}$, which is likely for a result kept *because it* passed the threshold, the probability falls. The prediction followed this computation; the measurement refuted it, $z = 3.62$ on the new windows. The judge did not change its prediction afterwards: it was declared, and the measurement decided.

11.6 J-V: speed, and what it separates

11.6.1 The trap

Each of the two lots compared a new file with an old file: lot A the v1.2, written on 17 October, with the original v1.0, written on the 15th; lot B its companions, written on the 17th and the 22nd, with the same original v1.0. Lot A saw the trap and said so: two v1.0 files identical to the bit were read at 1.74 and 1.34 GB/s. Lot B did not see it. If two files of the same model are read at different speeds for a reason that is not their mechanism, a speed ratio between them does not measure the mechanism.

11.6.2 The three-file session

The judge puts the three files of each model into a single session: A the original v1.0, B the v1.2 (same writer as the companions, written like them after the 15th), R the fast companion, in the order

A B R R B A A B R R B A.

Proposition 11.5 (The plan balances a linear drift)**.** *If the speed of a pass of rank i of file X is $v_X\,(1 + a + b\,i)$, the ratio of the means of the four passes of two files is v_X/v_Y, without a drift term. A quadratic drift, for its part, is not balanced.*

Proof. The ranks are A $\{1, 6, 7, 12\}$, B $\{2, 5, 8, 11\}$, R $\{3, 4, 9, 10\}$; their sums are all 26, their means $\bar{\imath} = 6.5$. The mean of the four passes of X is $v_X\,(1 + a + b\,\bar{\imath}) = v_X\,(1 + a + 6.5\,b)$, the same factor for the three files, which disappears from the ratio. The sums of the squares of the ranks are $1+36+49+144 = 230$, $4 + 25 + 64 + 121 = 214$ and $9 + 16 + 81 + 100 = 206$: a term in $c\,i^2$ would leave a bias. ∎

The rule reads *medians* of four passes, that is the mean of the two in the middle. Under a monotone drift that dominates the noise, the middle passes are those of ranks 6 and 7 for A, 5 and 8 for B, 4 and 9 for R, whose mean is again 6.5: the median is balanced in this case too [calc.].

The decomposition. With three files from the same session,

$$r(R/A) = r(R/B) \cdot r(B/A).$$

$r(B/A)$ measures what lot A measured, $r(R/A)$ what lot B measured, and $r(R/B)$ what the fast mode is worth against an exact file that the disk stage reads at the same speed. The ceiling that a smaller format can return, with the disk limiting, is the ratio of the bytes read per step ($o_A/o_B = 1/0.967\,1 = 1.034$ for v1.2); any $r(B/A)$ beyond it comes from something other than the bytes. The disk stage alone (`troncons disque --n 1500`, the instrument of lot A) runs first, A B R R B A, and says which of the factors is not the mechanism.

The threshold. With $\sigma = \max(\sigma_A, \sigma_B, \sigma_R)$, relative standard deviations of the four passes, a published gain g is a gain of the fast mode if $r(R/B) \geq g\,(1 - 2\sigma)$; $(1 - 2\sigma)$ is the lower bound of a ratio of medians at two standard deviations of its noise. The conditions: mains power at each pass; reading of the other streams before and after each pass (more than 5 % of a core or more than 8 MB read in 3 s: "under load", the pass does not conclude); start on two empty readings one minute apart; lock held. **None of the 36 passes was under load** [meas.].

11.6.3 The 120b

Settings of J1 for the three files: `mesurer --flux --emplacements 326`, wiki-3000 prompt, 256 tokens.

Table 11.5: J-V, the 120b: the three-file session [meas.].

	A, original v1.0	B, v1.2	R, fast frontier
generation, tokens/s	1.995 · 1.785 · 2.013 · 2.019	2.892 · 2.886 · 2.900 · 2.868	3.712 · 3.709 · 3.698 · 3.719
median; relative σ	2.004; 5.8 %	2.889; 0.5 %	3.711; 0.2 %
chunk bytes per step	710.1 MB	686.8 MB (× 0.967 1)	468.5 MB
per-byte rate, generation	1.42 GB/s	1.98 GB/s	1.74 GB/s
VRAM hits	0.590	0.590	0.607
text (256 tokens)	`e4c6dbac`…	`e4c6dbac`… (the same)	`2cf1d474`… (that of lot B)
peak memory	388.7 MB	382.3 MB	342.4 MB
disk stage alone (2 passes)	1.772 · 1.744 GB/s	2.394 · 2.422	2.324 · 2.279

Data: `mesures/genome/construction/juge/vitesse/seance-120b.\{json,log\}`, `vitesse/analyse-120b.json`, `figures/vitesse-passes.csv`, `figures/disque-passes.csv`.

- **J-V1 (A-C3 of lot A, taken up)**: $r(B/A) = 1.442$, $\sigma = 5.8$ %: no regression, gain announced; lot A published 1.454. J1 is reproduced; the bytes fall by only 3.3 % (ceiling +3.4 %), and the disk stage reads B 1.37 times faster than A.
- **J-V2**: $r(R/B) = 1.284$; threshold $2.00 \times (1 - 2\sigma) = 1.769$, with $\sigma = 0.057\,64$ (rounded to 5.8 % in the judgement; `mesures/genome/construction/juge/vitesse/analyse-120b.json`): **the ×2.00 is not a gain of the fast mode.** Against an exact file read at the same speed by the disk stage, the fast frontier mode returns ×1.28 in generation, for a ceiling of the bytes of 686.8/468.5 = ×1.47 [calc.]. The judge's prediction (×1.38) was too high: the throughput per byte of the companion (1.74 GB/s) stays below that of v1.2 (1.98), shorter reads (8.3 MB instead of 11.9 per chunk) [hyp. for the cause].

- **Where the rest of the** ×2.00 **comes from**: $r(R/A) = 1.851$ here (lot B measured 2.00 with other settings). Lot B split its ratio into ×1.524 of bytes and a ×1.31 of throughput per byte "that the counters do not separate" (Proposition 10.5). The disk stage alone returns R/A = 1.309: this ×1.31 is ~~the state of file A~~ the v1.0 read pattern of file A (corrected on 27 October, reserve R1), not the mode.

11.6.4 The 20b

Settings of session s2 of lot B: `mesurer`, default slots, 256 tokens.

Table 11.6: J-V, the 20b: the three-file session [meas.].

	A, original v1.0	B, v1.2	R, fast $\lambda = 0$
generation, median; σ	9.395; 0.4 %	12.175; 0.2 %	11.466; 0.6 %
prefill (708 tokens)	10.9 tok/s	13.7 tok/s	16.9 tok/s
chunk bytes per step	116.4 MB	112.5 MB	115.1 MB
VRAM hits	0.899	0.899	0.856
per-byte rate, generation	1.09 GB/s	1.37 GB/s	1.32 GB/s
text	`1b14e44a`…	`1b14e44a`… (the same)	`18be41e4`… (that of lot B)
disk stage alone	1.786 · 1.775 GB/s	2.461 · 2.443	2.475 · 2.472

Data: `mesures/genome/construction/juge/vitesse/seance-20b.\{json,log\}`, `vitesse/analyse-20b.json`.

- $r(R/A) = 1.220$: the measurement of lot B (1.241) is reproduced to within 1.7 %.
- **J-V3**: $r(R/B) = 0.942$; threshold $1.241 \times (1 - 2 \times 0.0056) = 1.227$: **the** ×1.24 **is not a gain of the fast mode.** Against v1.2, the fast mode $\lambda = 0$ generates 6 % more slowly: it reads as many bytes per step (ceiling ×0.98), because its routing, lossy, misses the VRAM more often. It wins only in prefill (×1.23), where the text is imposed.
- **The cause proposed by lot B is impossible**: the engine reads each chunk without buffering (`trc-es`, `FILE_FLAG_NO_BUFFERING | FILE_FLAG_OVERLAPPED`), and the placement stream measured that the disk reads 1.000 0–1.005 7 times the bytes requested. The judge's declared prediction already held it false "by construction"; the session judged it. The disk stage reads R 1.39 times faster than A, B 1.38 times.
- $r(B/A) = 1.296$ on the 20b: v1.2 also runs 30 % faster than its v1.0, for 3.4 % fewer bytes; same cause, the disk stage.

11.6.5 What makes the disk stage: reserve R1

The judge of the construction had first attributed the gap of the disk stage to the physical state of the files, after a reading of the disk placement stream: ~~it is the state of the files, not the format (+3.4 % at most)~~. The placement judge overturned this reading on 27 October (`genome/JUGEMENT-placement.md`, §§ 0, 5.3, 5.6, 5.7), and the judgement of the construction carries the correction at every place, strike-throughs included. On this disk (NVMe behind Intel RST VMD, under BitLocker), the gap between v1.0 and v1.2 follows *the pattern of read requests that the format imposes*: one and the same file read with the boundaries of the other format changes speed, and the exact variable of the pattern is not isolated (Section 9.7.1).

What this fixes, for the whole volume:

- **The** ×1.44 **of J1 is a gain of the v1.2 format**, through the pattern of its read requests, *specific to this disk*; it comes neither from its 3.3 % fewer bytes, nor from "the state of the file". It does not

carry over to another disk without being remeasured there.

- **The verdicts on the fast mode keep their meaning**: B and R both have a fast pattern (about 11.6 and 8.3 MB per request), A the slow pattern of v1.0 (about 12.0 MB); $r(R/B)$ compares two fast patterns. Wherever the judgement wrote "with a file of comparable age" and "the state of file A", read "with a comparable read pattern" and "the v1.0 pattern of file A".
- **The lever that works today** is to recode the v1.0 files as v1.2 (×1.30 to ×1.44 between two files of the same model). The chapter on the codon develops it (Chapter 9); the disk placement is the subject of a chapter of Volume II, "The disk stage", by decision of the placement judge.

11.7 The reading: what is false, too good, or wrongly attributed

The judge read the two deliverables sentence by sentence against their logs, and looked, for each figure of their "In brief" sections, for the committed file that carries it. Two findings are *serious*: they change what the project would do next. The others are minor: they overturn no conclusion, but they make a figure look better than it is.

G1, the "120b ×2.00" of the fast mode. Well measured, false as a gain of the mode: the exact file of the comparison is the v1.0 of 15 October, which the disk stage reads 1.31 times more slowly than the companion at equal bytes per read; against v1.2, the fast mode returns ×1.28. Lot B had isolated this ×1.31 and left it "not measured"; it is measured. *What this changes*: the specification built the mixed miss policy on this gain; ×1.28 for a KL of 0.14 is not ×2.00 for the same KL.

G2, the "20b ×1.24". Same correct measurement, same false attribution, and here the fast mode gains nothing in generation. Lot B had the two facts needed to see it: the engine read almost the same bytes per step, and the first B pass, just after the companion was written and after an A pass, was twice as slow as the following ones (5.46 against 11.8 tokens/s), which no read cache explains. *What this changes*: on the 20b, the fast mode is a prefill mode, not a generation mode.

The minor findings. M1: the ± of the absolute KLs of lot B are naive, three to six times too narrow (Section 11.5.3); the paired verdicts, for their part, are right. M2: "the tail falls too" with $\lambda = 0$ is not reproduced against $\lambda = 1$, and is reproduced against the frontier. M3: the direction of the perplexity of $\lambda = 0$ reverses outside R1. M4: the speed rule of lot B looked only at the noise of the A passes; with $\sigma = \max(\sigma_A, \sigma_B)$ ($\sigma_B = 31\,\%$), the gain would not have been announced. M5: "the gain does not melt, it grows" compared two variants; at equal variant, the sentence stays true.

What else the judge checked, and which holds. The per-tensor gain of lot A equals twice the share of the −0 to within −0.000 167 … 0.000 161 bit over the 180 tensors (shares 6.393 % and 6.279 %, recomputed); the eight passes of session s2 of lot A give again 1.988 and 2.890 tokens/s and $r = 1.454$; the sixteen passes of s2 of lot B give again 9.54, 11.84, 1.98 and 3.97 tokens/s and their bytes per step; the struck-through correction of the synthesis (the KL 0.026 2 is that of $\lambda = 0$) is right; no attribution line nor e-mail address in the documents of the two lots.

11.8 What holds according to the judge, and what is struck through

What is struck through , each time where it was published (the judge carried its strike-throughs into deliverable 07 and into its note for the thesis, without removing anything: *answer by delegation of 2026-10-22, reversible*):

Table 11.7: What holds according to the judge of the construction, and by which path it redid it [meas.].

Figure	Lot	Judge's path
`.trc` v1.2: 10 851 844 096 and 55 930 175 488 bytes, −1 800 486 912 bytes; 0.127 91 and 0.125 62 bit per weight	A	its reader of the format, whole files
each v1.2 expert is the expert of the GGUF up to the sign of the zeros; no 8; three fingerprints	A	its decoder, 96 drawn experts; v1.0 control 40 out of 40
no v1.2 table can produce an 8 (0 out of 540)	A	its reader of the tables
logits identical to the bit, v1.2 against v1.0, on both models	A	ten new windows; break 512 out of 512
J1 of the 120b: v1.2 1.44 times faster than the original v1.0, ~~for the files, not for the format~~ through the format, by the pattern of its requests, on this disk (reserve R1)	A	three-file session: 1.442
$\lambda = 0$ beats $\lambda = 1$ (B2)	B	new windows, engine: $-0.030\,3 \pm 0.008\,4$ ($3.6\,\sigma$)
the engine serves the fast mode as faithfully as the substituted f64	B	new windows: 0.003 7; 97.8 %
the exact mode is unchanged, and close to the f64	A, B	new windows: 0.004 4; 98.0 %
R1: 0.264 0, 0.152 2, 0.131 8; paired −0.111 8, −0.020 4, −0.132 2	B	its KL on their files
$\lambda = 0$ better than the frontier, and its tail lower	B	new windows, substituted f64: $-0.156\,5 \pm 0.033\,4$ ($4.7\,\sigma$); 99th −26 %, 99.9th −18 %

Data: `genome/JUGEMENT-construction.md`, § 8, with the correction of 27 October (§ 6.1).

- ~~120b ×2.00 (gain of the fast mode)~~ → ×1.28 against an exact file read at the same speed by the disk stage; the ×2.00 remains true as a comparison of two files.
- ~~20b ×1.24 (gain of the fast mode)~~ → ×0.94 in generation against v1.2; ×1.23 in prefill.
- ~~cause: the system's file cache~~ → impossible (unbuffered reads); measured cause: the disk stage, ~~the physical state of the exact file~~ the v1.0 read pattern (R1).
- ~~with $\lambda = 0$, the tail also falls at the 99.9th percentile, against $\lambda = 1$~~ → not reproduced outside R1; not a measurable quantity at 2 550 positions.
- ~~$\lambda = 0$ distorts the distribution in a privileged direction~~ → the direction of the perplexity reverses on other windows.
- The ± of the absolute KLs are not struck through: they are *requalified*, as naive.

11.9 What the judge's method teaches

- **A three-file session separates the format, the mode and the file**: $r(R/A) = r(R/B) \cdot r(B/A)$, and the disk stage measured in the same session says which of the factors is not the mechanism. One hour per model; it brought down two figures.
- **Before attributing a speed gain to a mechanism, measure the disk stage of each file in the same session**: one minute.
- **A judge's decoder written from the description alone, in another language, vectorised by prefix sum**: two seconds per expert, right to the bit at the first attempt on the decoding.
- **New windows drawn far from the region already seen, merged into a single file** that all the tools read without change.
- **Do not compare a 99.9th percentile of** 2 550 **positions from one variant to another**: it rests on two or three positions.
- **The text of a lock counts**: the judge's first lock said "pas une séance de vitesse" (*not a speed*

session), and the leads stream, which looked for the word “vitesse” (*speed*), paused for nothing.
- **Stopping a bash loop that launches a native program under Windows does not kill the program**: the f64 pass in progress finished on its own, and its complete output was kept.

11.10 What remains

1. **The mixed miss policy** is to be taken up with $r(R/B)$: on the 120b, ×1.28 in generation for a KL of 0.14 (proxy); on the 20b, nothing in generation. Simulate it on the traces before building it.
2. **The speed lever that the construction found is not a mechanism of the genome**: it is the pattern of read requests of the format on this disk. The exact variable remains to be isolated, at the disk stage, by patterns declared beforehand (placement judge, § 12).
3. **Another corpus** (code, dialogue) for the KL of the synonymous genome, and the synonymous mutation of the 120b.
4. The judge’s heavy files (about 13 GB of `.kld` outside the repository) can be regenerated by the committed tools; they can be erased whenever Stone wishes.

11.11 What could be wrong

What could be wrong

- **One session per model, a single morning** (22 October, from 6:17 to 7:11 for the engine). The ratios $r(R/B)$ rest on the fact, measured in the same session, that B and R are read at the same speed at the disk stage (R/B = 0.956 and 1.009, below the separation threshold of 11.8 %); if this were no longer true on another day, the session would have to be redone, not the figure reused.
- **The settings are not all those of lot B**: on the 120b, those of J1 for the three files; the judge’s $r(R/A)$ (1.85) differs from that of the lot (2.00) for this reason at least. What judges the attribution is $r(R/B)$, measured with the same settings for R and B.
- **The new windows are from wiki.test**, like R1 and the calibration: another text, not another kind of text.
- **The judge’s decoder reads the same description as the authors of the coder.** A shared misreading would not be seen by comparing the decoded bytes… except that it would have been: the GGUF is read by a reader with nothing of the `.trc` format, and the fingerprints are SHA-256 computed on the judge’s side [121]. What is not covered: the parts of the format that neither v1.0 nor v1.2 exercise (levels, raw, columns).
- **96 experts out of** 5 376 **(**1.8 %**)** by the judge’s path. The probability that a defect touching at least 5 % of the experts of a model escapes 32 draws is $0.95^{32} \approx 19\,\%$ (20b), and at 64 draws $0.95^{64} \approx 3.75\,\%$ (120b) [calc.] (the source writes 3.7 %, a rounding too low). The verifier of lot A, for its part, covers 100 % of the experts; the draw judges this verifier, it does not replace it.
- **The judge’s KL shares the convention of lot R** (cut at $\log p > -16$, without renormalisation): independent of the code, not of the convention. The paired differences hardly depend on it.
- **The judge’s predictions were wrong twice** (B2 “equal”, measured 3.6 σ; $r(R/B)$ of the 120b ≈ 1.38, measured 1.28); they were declared beforehand, and the measurement decided.
- $\lambda = 1$ **through the substituted reference was not run** on the new windows; B2 is judged there only by the engine, the frontier only by the substituted reference.

- **The engine and `genome-capture`** that the judge ran are built from the same code as those of the lots: it judges the files and the claims, not a second engine. The second path for the logits remains the f64 reference of lot R.
- **Reserve R1 itself**: the read pattern is established on this disk, by the placement judge, at the disk stage; the exact variable is not isolated, and another reading could replace it again. The verdicts of this chapter depend only on what the disk stage measures in the session, not on its explanation.

Part III

The leads and the assessment

Chapter 12

The tropical geometry of the router

Where we start, what this chapter establishes, what remains

At the start, lot R of Volume II had shown that routing flips carry most of the KL between two close computations: imposing the routing of the reference removes 88 % of it, and 96.5 % of the KL lies on the positions that flip; the router margin, the gap between its 4th and 5th logit, had a median of 0.083 over all cells and 0.0095 over those that flip. The synthesis of the round (`genome/05-synthese.md` § 3.1) had drawn an idea of Stone's from it: a *guard* that would read this margin and redo only the fragile cells in exact mode. It remained to be judged. **This chapter establishes** that the guard falls at the threshold declared before the reading: for 10 % of the cells flagged, it catches 57.1 % of the flips (2 905 out of 5 087), far from the 80 % required, and 80 % would cost 26.6 %. It poses the geometry that makes the margin a distance (the router is a tropical polynomial, its regions polyhedra), measures the laws of the margin (almost exponential), of the flip ($\Phi(-m/\sigma)$, plateau $\frac{1}{2}$) and of the propagation (26.6 % against 3.4 %), proves a closed form of the guard's curve, which depends only on a dimensionless number κ, and says, with its judge, what this theorem does not explain. Two explorations, after the verdict and stated as such: the guard catches 85.2 % of the cascade roots; a guard that remembers the conflict gains +1.61 points, below the 5 declared. **What remains** is a routing log of the fast mode, to know its κ and judge the guard of the roots, and the measurement of the repair: does redoing a layer in exact mode extinguish the root and its consequences? Sources: `genome/pistes/tropicale/garde-tropicale.md`, `pour-le-memoire.md`, `seuil-declare.md`, `seuil-declare-indices.md`, `JUGEMENT.md`; `mesures/pistes/tropicale/` (logs, figures, `indices/`, `juge/`); tools `outils/pistes/tropicale/`.

In brief

The router of a layer gives a score to each of the 32 experts and keeps the four best. The space of its inputs is therefore cut into regions, one per quartet, and these regions are polyhedra: pieces with flat faces. An input close to a face is fragile: a slight change moves it to another quartet, and this change, a "flip", costs dearly in quality. The margin measures this closeness. The idea of the guard is that of a controller that watches only the borderline cases; the image is borrowed from the cingulate cortex, which monitors the conflict between two competing responses, and it is only an image. The measurement says that it watches the first layers well and the last ones badly, because one flip brings others, far from the faces, that the margin cannot see.

12.1 The question and the declared threshold

The router of layer ℓ computes $z = W_r h + b_r$ (32 logits) and keeps the four largest. The fast mode (Chapter 10) replaces exact experts by compressed experts; the error it introduces moves the state h

and, near a face, makes the choice of the next layer flip. *The guard*: in fast mode, after the fast experts of layer ℓ, the engine reads the margin of the router of $\ell + 1$; below a threshold τ, it redoes layer ℓ in exact mode for that position.

Its cost is the share of (layer, position) cells flagged, $c(\tau)$; its gain the share of flips flagged, $b(\tau)$. **Declared threshold** (`genome/pistes/tropicale/seuil-declare.md`, commit `69260f1`, before any reading): the guard is worth building if it catches at least 80 % of the flips while flagging at most 10 % of the cells. The 10 % derives from the cost: an exact-expert miss on 10 % of the cells costs at most 10 % of the bytes of the exact mode. The guard runs in the fast computation and sees only its own margin; the margin that decides is that of B, the computation that departs from the reference A. The prediction, written in the same declaration: 55 to 65 %, hence a failure.

The judge checked the order by `git log`: the threshold (`69260f1`, 2026-10-21 23:22:18, the only commit of this file) precedes the first reading tools (`4b2a8eb`, 23:23:32) and the verdict (`77d8110`, 23:25:17); git does not prove that no exploratory reading took place before, and the clock of the lead's notes is wrong by a few minutes (`JUGEMENT.md` § 1).

12.2 The router as a tropical polynomial

Let $z(h) = W_r h + b_r \in \mathbb{R}^{32}$, $W_r = [w_1 \dots w_{32}]^\top$. For a set S of $k = 4$ indices, the region

$$P_S = \{\, h \,:\, z_i(h) \geq z_j(h),\ \forall i \in S,\ \forall j \notin S \,\} \tag{12.1}$$

is an intersection of $k(32-k) = 112$ half-spaces: a convex polyhedron, possibly empty. The P_S cover the space and overlap only on their boundaries.

Proposition 12.1 (The top-k is a tropical polynomial)**.** *The function $F_k(h) = \max_{|S|=k} \sum_{i \in S} z_i(h)$ is a polynomial of the* $(\max, +)$ *semiring in the affine forms z_i; it is convex and piecewise affine, its pieces are the P_S, and its locus of non-differentiability, its tropical hypersurface, is the union of the faces $\{z_i = z_j\}$ where i is at rank 4 and j at rank 5.*

Proof. In the tropical semiring, the "sum" is the max and the "product" is +: F_k is the tropical sum of the monomials $\bigodot_{i \in S} z_i$, hence a tropical polynomial; as a maximum of affine functions, it is convex and piecewise affine. On the interior of P_S, the max is attained by S alone (any other set replaces an $i \in S$ by a $j \notin S$ with $z_j < z_i$), and F_k is affine there with gradient $\sum_{i \in S} w_i$. Two sets attain the max at once exactly when an element of rank 4 can be exchanged for an element of rank 5 without changing the sum, that is on $\{z_{(4)} = z_{(5)}\}$. ∎

This is the frame of Zhang, Naitzat and Lim for ReLU networks [115]; the top-k falls under it through the same algebra.

Definition 12.2 (Margin)**.** The margin of a cell is $m(h) = z_{(4)}(h) - z_{(5)}(h) \geq 0$, the gap between the 4th and the 5th logit. It is zero exactly on the tropical hypersurface.

Proposition 12.3 (The margin as a distance)**.** *Let h be interior to P_S. (a) In the space of logits, the distance from z to the boundary of its region is $m/\sqrt{2}$. (b) In the space of states, the distance from h to the boundary of P_S is*

$$d(h, \partial P_S) = \min_{i \in S,\, j \notin S} \frac{z_i(h) - z_j(h)}{\|w_i - w_j\|} \ \geq\ \frac{m(h)}{\max_{i \in S,\, j \notin S} \|w_i - w_j\|}, \tag{12.2}$$

and the distance to the face of the pair (i, j) of ranks 4 and 5 is exactly $m/\|w_i - w_j\|$.

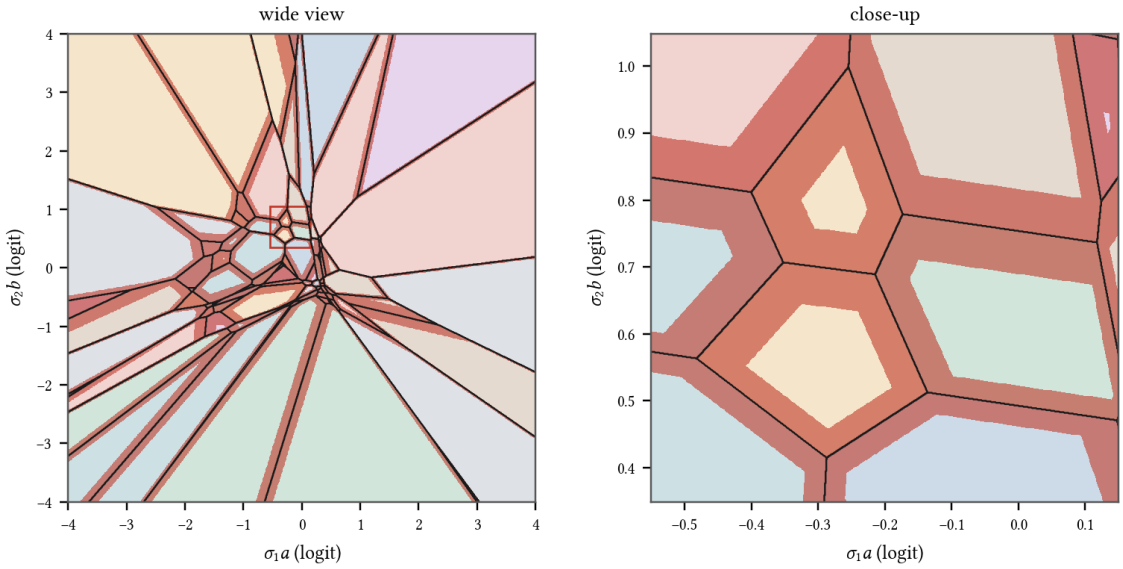


Figure 12.1: The real router cuts the space of states into polyhedra; the guard band follows their faces, wider where two neighbouring experts resemble each other. [calc.] The top-4 regions of the router of layer 12 of gpt-oss-20b, in the plane of the first two right singular directions of W_r ($h = au + bv$, through the origin of the space of normalised states); in red, the band $\{m < \tau_{10}\}$. Left: 92 regions over 8×8 logits; right: the enlargement of a cluster (0.7×0.7 logit). A two-dimensional cut of a space of 2 880: the geometry is true in it, the areas are not probabilities, and the framing of the enlargement is a choice. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/pistes/tropicale/figures/coupe-routeur-c12-en.png`; script `outils/pistes/tropicale/figures.py`.

Proof. The distance from an interior point to the boundary of a convex polyhedron is the minimum of its distances to the hyperplanes that define it: the point nearest to the boundary lies on one of them, and a redundant hyperplane, which carries no face, lies beyond the boundary. The distance from h to $\{z_i = z_j\}$ is $(z_i - z_j)/\|w_i - w_j\|$, hence the equality of (b); each numerator is at least m, hence the bound. (a) is the same computation in the coordinates z, where $w_i - w_j$ becomes $e_i - e_j$, of norm $\sqrt{2}$, and the smallest numerator is m. ∎

The lead's note writes that $m/\|w_i - w_j\|$, for the pair of ranks 4 and 5, bounds from below the distance to any boundary of the region. This is true of that face, and in the space of logits; in the space of states, another pair whose router vectors differ more may be closer, and the general bound is that of (12.2). The guard band $\{m < \tau\}$ is therefore, along the face of the pair (i, j), a slab of thickness $\tau/\|w_i - w_j\|$ on each side, and of half-thickness $\tau/\|P(w_i - w_j)\|$ in a cut by a plane of projector P (Figure 12.1): wide where two neighbouring experts resemble each other in the plane.

12.3 The data and the controls

The data. Three routing logs of lot R (commit `21453da`, SHA-256 fingerprints in the declaration), gpt-oss-20b, 24 layers × 5 110 positions (10 windows of 511), that is 122 640 cells: A (`ref-openai-bis`), B (`ref-llamacpp-bis`), C (the position ramp of llama.cpp with the routing of A imposed). A flip is a cell where the *sets* of four experts of A and B differ: 5 087. (Comparing the ordered lists would count 11 028; the top-4 is a set, the internal order does not change the experts called.) These

Table 12.1: The tropical guard: the controls of the instrument, each with its break [meas.].

Control	Known answer	Obtained	Break
T1, the reading	5 087 flips and the 8 margin quantiles of the summary of lot R	to the last digit of f32	one expert byte changed: 5 088; step of 7 bytes: refused
T2, the answer in advance	A against C (imposed routing): 0 flips	0	one expert changed: 1
T3, the instrument discriminates	permuted margins: $b(\tau_{10}) = 10 \pm 2$ %; oracle margins: $b = 100$ % for $c \leq 4.2$ %	10.05 %; 100 % at 4.15 %	(its own break)

Data: `mesures/pistes/tropicale/temoins.json`; `genome/pistes/tropicale/garde-tropicale.md` § 2.

Table 12.2: The curve of the guard: cells and flips below the threshold τ of the margin of B [meas.]. The judge redoes it by an explicit ranking of the 122 640 cells, without an interpolated threshold, to the unit.

τ	cells below τ	share of cells c	flips below τ	share of flips b
10^{-4}	106	0.09 %	58	1.1 %
$3.16 \cdot 10^{-4}$	336	0.27 %	168	3.3 %
10^{-3}	995	0.81 %	477	9.4 %
$3.16 \cdot 10^{-3}$	3 238	2.64 %	1 265	24.9 %
10^{-2}	10 089	8.23 %	2 654	52.2 %
$\tau_{10} = 0.012410$	**12 264**	**10.00 %**	**2 905**	**57.1 %**
$1.58 \cdot 10^{-2}$	15 406	12.56 %	3 167	62.3 %
$3.16 \cdot 10^{-2}$	28 733	23.43 %	3 932	77.3 %
0.0366 (80 % of the flips)	32 623	26.60 %	4 070	80.0 %
10^{-1}	69 090	56.34 %	4 829	94.9 %

Data: `mesures/pistes/tropicale/courbe-garde.csv`, `garde-resume.json`, `second-chemin.json`; judge: `mesures/pistes/tropicale/juge/juge.json`.

are the only logs of the repository that carry a margin. **No routing log of the fast mode exists**: the perturbation measured here is that of two exact computations, the position ramp of OpenAI and that of llama.cpp.

T3 is its own break: the same instrument returns 10 % when the margin knows nothing and 100 % when it knows everything (Table 12.1). The judge redid T2 and T3 with its code, and added its own: one byte changed on a cell without a flip gives 5 088; its permuted margins give 10.4 %; and a control J5, which puts 600 uncaught flips at zero margin, expects $2\,905 + 600 - 64 = 3\,441$ caught and obtains them. Its first writing expected ~~2 905 + 600 = 3 505~~: it forgot that the 600 cells entering at the top push out 600, of which 64 are flips (`JUGEMENT.md` § 3).

12.4 The verdict: the curve of the guard

At the declared threshold, the guard falls: at the decile $\tau_{10} = 0.012410$, it flags 12 264 cells and catches 2 905 flips out of 5 087, 57.1 % (Table 12.2 and Figure 12.2). For 80 % it would need 32 623 cells, 26.6 %. The prediction written beforehand (55 to 65 %) holds. The second path (a reading in pure Python, through the reading function of lot R, counting by loops) returns the same 2 905 flips on the

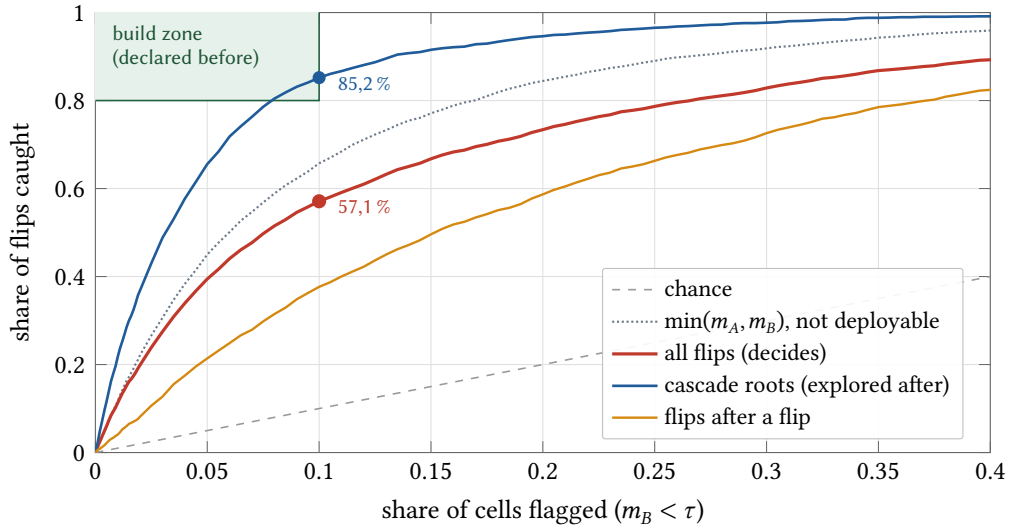


Figure 12.2: The guard catches 57.1 % of the flips for 10 % of the cells, far from the declared build zone; it catches 85.2 % of the cascade roots. [meas.] Share of flips caught against share of cells flagged ($m_B < \tau$): all flips (the verdict), the cascade roots and the flips that follow a flip (explored after the verdict), the non-deployable oracle $\min(m_A, m_B)$, and chance. Green zone: the build zone declared before the reading ($c \leq 10$ %, $b \geq 80$ %).

Data: `mesures/pistes/tropicale/figures/fig-courbe.csv`; script `outils/pistes/tropicale/figures.py`.

same 12 264 cells; so does the judge, with its own code and an explicit ranking (a single tie at the boundary, without ambiguity). The threshold τ_{10} is the decile of the margins measured on the logs themselves: this favours the guard, which falls nonetheless.

The variants, which do not decide. The margin of A instead of B: 56.1 % (area under the ROC curve 0.862 for both). The non-deployable oracle $\min(m_A, m_B)$: 65.7 %, and 80 % for 16.9 % of the cells; even it misses the threshold. Only the positions scored by the judge of lot R: 53.4 %. A cross-check under another perturbation (the card's engine against the reference, 22 flips) catches 9 flips out of 22; it judges nothing (95 % interval from 21 to 64 %), but does not contradict [meas.].

Depth. The guard sees the first layers well and the last ones badly (Table 12.3 and Figure 12.3). The robust scale of the perturbation of the gap, $\delta = m_A - m_B$ on the cells without a flip with a large margin ($m_B > 0.1$), rises from 0.0022 at layer 0 to 0.0059–0.0086 at layers 9 to 23. A guard tuned layer by layer (τ_ℓ learnt on five windows, judged on the other five, then the reverse) gains nothing: 55.4 and 56.0 % against 55.9 and 58.7 % for the global τ on the same halves [meas.].

12.5 Why it falls

12.5.1 The law of the margin: almost exponential

Lemma 12.4 (Rényi representation). *If $X_1, \ldots, X_n$ are independent exponentials of rate λ, sorted in decreasing order $X_{(1)} \geq \ldots \geq X_{(n)}$, the spacings $D_k = X_{(k)} - X_{(k+1)}$, $k = 1, \ldots, n$, with $X_{(n+1)} = 0$, are independent and D_k follows an exponential law of rate $k\lambda$.*

Proof. Sort the same variables in increasing order, $Y_1 \leq \ldots \leq Y_n$ (that is $Y_j = X_{(n+1-j)}$), and set $Y_0 = 0$,

Table 12.3: The guard by layer, at the global τ_{10} [meas.]. The ranges are redone by the judge.

Layers	flip rate	$b(\tau_{10})$	median margin at the flips	ROC area
0	0.72 %	100 %	0.0013	0.991
1 to 4	1.3 to 1.9 %	86.0 to 91.0 %	0.0029 to 0.0045	0.956 to 0.961
5 to 7	2.1 to 3.1 %	73.9 to 79.6 %	0.0042 to 0.0060	0.930 to 0.957
8 to 14	3.3 to 5.0 %	51.2 to 58.6 %	0.0092 to 0.0113	0.851 to 0.896
15 to 23	4.7 to 8.1 %	37.8 to 60.5 %	0.0084 to 0.0199	0.771 to 0.864

Data: `mesures/pistes/tropicale/decile-par-couche.csv`.

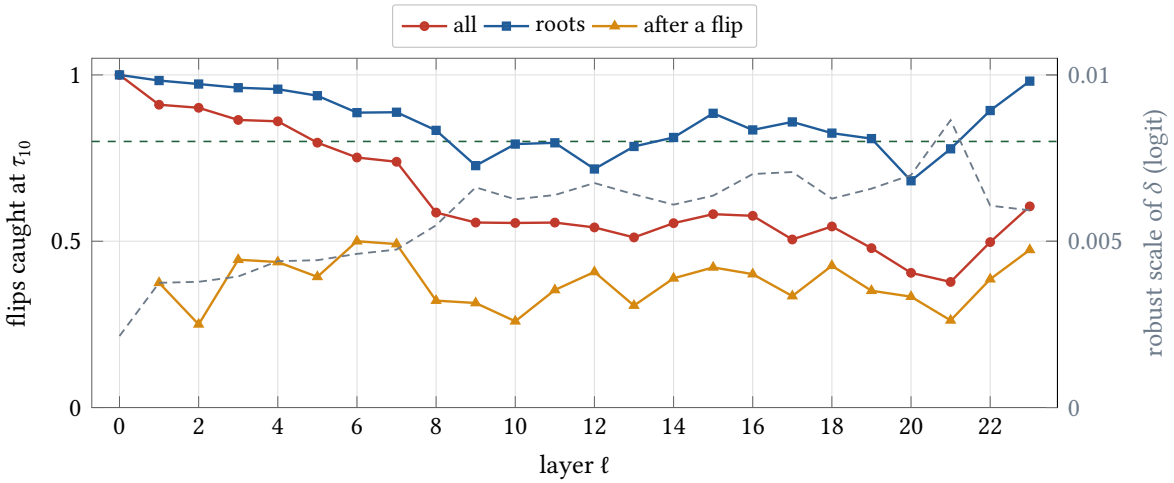


Figure 12.3: The guard sees the first layers and loses the last ones, as the perturbation grows. [meas.] Share of flips caught at the global τ_{10}, by layer: all, roots, flips that follow a flip; dashes (right axis): the robust scale of the perturbation δ. Horizontal line: 80 %.

Data: `mesures/pistes/tropicale/figures/fig-par-couche.csv`; script `outils/pistes/tropicale/figures.py`.

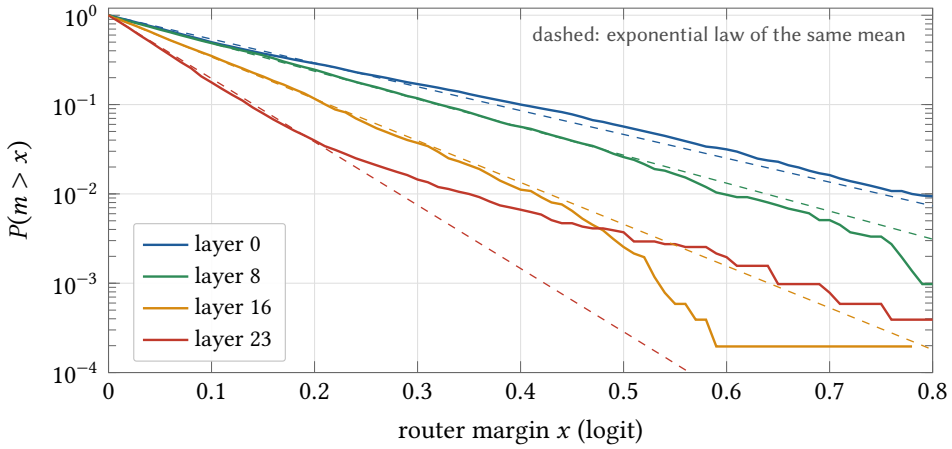


Figure 12.4: The survival of the margin is a straight line on a logarithmic scale: the margin is almost exponential, as the spacing of order statistics predicts. [meas.] $P(m > x)$ for four layers (0, 8, 16, 23); dashes: the exponential law of the same mean.

Data: `mesures/pistes/tropicale/figures/fig-survie.csv`; script `outils/pistes/tropicale/figures.py`.

that is $X_{(n+1)} = 0$. The minimum of n independent exponentials of rate λ follows the exponential law of rate $n\lambda$, since $P(Y_1 > y) = \prod_i P(X_i > y) = e^{-n\lambda y}$. Given $Y_1 = y$ and the index of the variable that attains it, the $n-1$ others are independent and conditioned by $X_i > y$; by the memorylessness of the exponential law, their excesses $X_i - y$ are exponential of rate λ, independent of one another, and their law depends neither on y nor on the index. The spacing $Y_2 - Y_1$, the minimum of these $n-1$ excesses, therefore follows the exponential law of rate $(n-1)\lambda$ and is independent of Y_1. Starting again from Y_2, then from Y_3, and so on, one obtains that the $Y_{j+1} - Y_j$ ($j = 0, \dots, n-1$) are independent, of rate $(n-j)\lambda$. Now $Y_{j+1} - Y_j = X_{(n-j)} - X_{(n-j+1)} = D_{n-j}$: with $k = n - j$, D_k follows the exponential law of rate $k\lambda$, and the D_k are independent. ■

This is Rényi's theorem [126] (record checked by the lead on 2026-10-22, article not reread); the proof above, by memorylessness, does not depend on the article. For a parent law of density f and hazard rate $\eta(x) = f(x)/(1 - F(x))$, the transformation $x \mapsto -\ln(1 - F(x))$ makes the variables exponential of rate 1 and, near the top of the sample, it is locally linear with slope $\eta(x_{(k)})$; hence $D_k \approx \mathrm{Exp}\big(k\,\eta(x_{(k)})\big)$. The margin $m = D_4$ is therefore, to first order, exponential: finite non-zero density at zero, $f_m(0) \approx 1/\mathbb{E}[m]$, and $P(m < \tau) \approx \tau/\mathbb{E}[m]$ for small τ [calc.]. The 32 logits are neither independent nor identically distributed; the measurement says that the conclusion holds nonetheless.

Measured at each layer [meas.] (`mesures/pistes/tropicale/loi-marge-par-couche.csv`): the gamma fit gives a shape of 0.91 to 1.09 (1 would be the exponential); the Kolmogorov-Smirnov distance to the exponential of the same mean is 0.006 to 0.042, against 0.09 to 0.18 to the half-normal and 0.06 to 0.08 to the log-normal; the slope of $\log P(m < \tau)$ against $\log \tau$ on $[10^{-4}; 10^{-2}]$ is 0.90 to 1.20; the density below 10^{-3} (5.5 to 17.6) follows $1/\mathbb{E}[m]$ (5.2 to 16.3). The deep layers have tighter margins (Figure 12.4).

From one layer to the next. For the same position, the Spearman correlation between m_ℓ and $m_{\ell+1}$ ranges from 0.002 to 0.125; pooled over the 23 pairs, 0.058, above the 99.9th percentile of 999 permutations (0.023): real and tiny [meas.]. A tight margin at one layer says almost nothing about the next; each layer draws its margin afresh. Direct consequence: as a prefetch priority for the next layer, the margin gives a ROC area of 0.567, below the declared bar of 0.60; for the next token, 0.627: useful

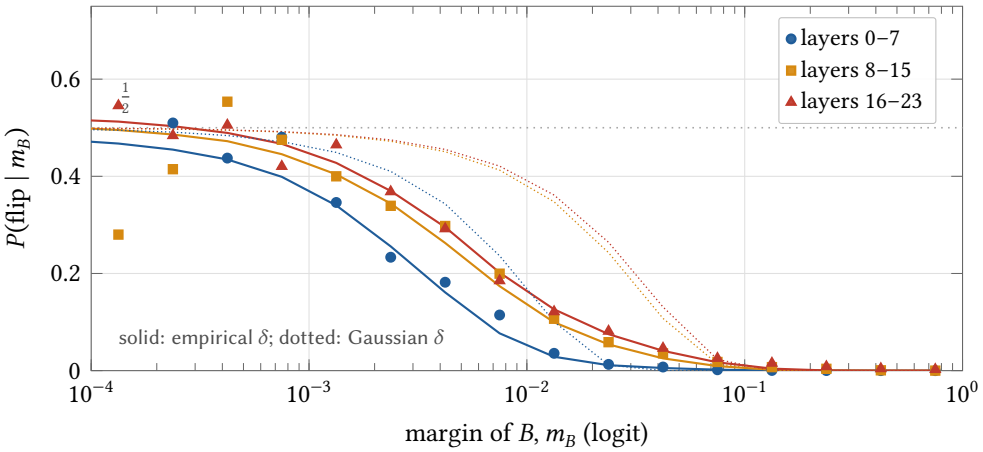


Figure 12.5: The flip probability tends to $\frac{1}{2}$ as the margin tends to zero: the law is $\Phi(-m/\sigma)$, not 2Φ. [meas.] [calc.] $P(\text{flip} \mid m_B)$ measured by margin classes, three groups of layers; solid line: the model with empirical δ, $P(\delta < -m)$; dotted: the Gaussian $\Phi(-m/\hat{\sigma})$.

Data: `mesures/pistes/tropicale/figures/fig-loi-bascule.csv`; script `outils/pistes/tropicale/figures.py`.

by the bar, but weakly (the next token keeps on average 2.0 experts out of 4, 1.89 in the lowest margin decile, 2.17 in the highest, against 0.5 by chance).

12.5.2 The flip law

Proposition 12.5 (Flip law). *Let m_B be the margin of the computation that guards and δ the perturbation, between A and B, of the gap between the experts at ranks 4 and 5 of B. The cell flips as soon as the gap of A goes below zero, $m_B + \delta < 0$. If δ is independent of m_B, with symmetric law G,*

$$P(\mathit{flip} \mid m_B = m) = G(-m), \qquad \lim_{m \to 0} P = G(0) = \frac{1}{2}. \tag{12.3}$$

Proof. $P(m + \delta < 0) = P(\delta < -m) = G(-m)$, and $G(0) = \frac{1}{2}$ by symmetry. ∎

For a Gaussian perturbation, $\Phi(-m/\sigma)$. The declaration had written ~~$2\Phi(-m/\sigma)$~~, which would be 1 at zero, as if the flip were certain on the face; the derivation gives Φ, and the data decide: below $m = 10^{-4}$, 58 flips out of 106 cells (0.55 ± 0.05), and the likelihood of Φ beats that of 2Φ at all 24 layers [meas.] (`loi-bascule-sigma.csv`). This is a lower bound on the real probability: pairs other than (4, 5) can cross, but they need a perturbation larger than m. The judge notes it: the form changed after the reading; the theorem that follows and the extrapolation rest on it, and remain models. The Gaussian form is wrong in the tail: δ has an excess kurtosis of 3.5 (layer 0) to 134 (layer 7). The Gaussian model predicts 7.8 % of flips (measured 4.15 %); the empirical form of δ, rescaled, 3.6 % [calc.] (Figure 12.5).

12.5.3 Propagation: one flip calls for others

Given a flip at (ℓ, p), the same position flips at $\ell + 1$ in **26.6 %** of cases (4 672 pairs), against **3.4 %** without a flip at ℓ [meas.] (`propagation.csv`; the judge: 26.65 and 3.37 %). The gap grows with the margin of the next layer (Table 12.4).

After a flip, the hidden state has changed experts: the perturbation is no longer a rounding, it is the difference between two experts (33 to 84 % of the output energy, `05-synthese.md` § 2.2). The local κ becomes large, and the next flip falls at a wide margin that the guard does not flag. This is the

Table 12.4: Propagation: probability of a flip at $\ell+1$, according to the margin at $\ell+1$, after a flip at ℓ or without [meas.]. In parentheses, the number of pairs.

Margin at $\ell+1$	after a flip	no flip before	ratio
$< 10^{-3}$	61.8 % (34)	47.5 % (930)	1.3
10^{-3} to 10^{-2}	57.1 % (380)	23.1 % (8 403)	2.5
10^{-2} to $3\cdot10^{-2}$	47.2 % (755)	5.4 % (15 896)	8.7
$3\cdot10^{-2}$ to 0.1	27.0 % (1 716)	1.3 % (38 415)	21
> 0.1	10.5 % (1 787)	0.14 % (49 214)	75

Data: `mesures/pistes/tropicale/propagation.csv`.

main cause of the fall with depth: the secondary flips are 59 % of the flips (3 006 out of 5 087), with a median margin of 0.0194 against 0.0037 for the roots. The ratio ×7.9 (and ×75 at wide margin) is an association; the intervention of lot R (imposed routing, 88 % of the KL removed) points the same way without proving it for the cascades.

12.5.4 The guard number: a closed form

Assumption 12.6 (The homogeneous model)**.** The margin M follows an exponential law of rate λ; the perturbation δ follows $\mathcal{N}(0,\sigma^2)$, independent of M; the cell flips if $\delta < -M$; the guard flags $M < \tau$. Write $\kappa = \lambda\sigma = \sigma/\mathbb{E}[M]$, the *guard number*, $t = \tau/\sigma$, and Φ and φ for the normal distribution function and density.

Lemma 12.7. *For $\kappa > 0$ and $u \geq 0$: $e^{-\kappa u}\varphi(u) = e^{\kappa^2/2}\varphi(u+\kappa)$.*

Proof. $-\kappa u - u^2/2 = -(u+\kappa)^2/2 + \kappa^2/2$. ∎

Lemma 12.8. $\displaystyle\int_0^T \lambda e^{-\lambda m}\,\Phi(-m/\sigma)\,\mathrm{d}m = \tfrac{1}{2} - e^{-\lambda T}\Phi(-T/\sigma) - e^{\kappa^2/2}\big[\Phi(T/\sigma+\kappa) - \Phi(\kappa)\big]$.

Proof. By parts, with $U = \Phi(-m/\sigma)$, $\mathrm{d}V = \lambda e^{-\lambda m}\,\mathrm{d}m$, $V = -e^{-\lambda m}$, $\mathrm{d}U = -\varphi(m/\sigma)/\sigma\,\mathrm{d}m$:

$$\int_0^T = \big[-e^{-\lambda m}\Phi(-m/\sigma)\big]_0^T - \int_0^T e^{-\lambda m}\,\frac{\varphi(m/\sigma)}{\sigma}\,\mathrm{d}m = \tfrac{1}{2} - e^{-\lambda T}\Phi(-T/\sigma) - \int_0^{T/\sigma} e^{-\kappa u}\varphi(u)\,\mathrm{d}u,$$

and by lemma 12.7 the last integral is $e^{\kappa^2/2}\int_0^{T/\sigma}\varphi(u+\kappa)\,\mathrm{d}u = e^{\kappa^2/2}\big[\Phi(T/\sigma+\kappa) - \Phi(\kappa)\big]$. ∎

Theorem 12.9 (The guard number)**.** *Under assumption 12.6, the share of flagged cells is $c = 1 - e^{-\lambda\tau}$, the flip rate is*

$$P(B) = \tfrac{1}{2} - e^{\kappa^2/2}\,\Phi(-\kappa), \tag{12.4}$$

and the share of flagged flips is

$$b = \frac{\tfrac{1}{2} - e^{-\lambda\tau}\Phi(-t) - e^{\kappa^2/2}\big(\Phi(t+\kappa) - \Phi(\kappa)\big)}{\tfrac{1}{2} - e^{\kappa^2/2}\,\Phi(-\kappa)}, \qquad \lambda\tau = -\ln(1-c), \quad t = \lambda\tau/\kappa. \tag{12.5}$$

The curve (c, b) depends only on κ.

Proof. $c = P(M < \tau) = 1 - e^{-\lambda\tau}$. $P(B)$ is lemma 12.8 as $T \to \infty$: the terms in $e^{-\lambda T}$ vanish, $\Phi(T/\sigma+\kappa) \to 1$, hence $\tfrac{1}{2} - e^{\kappa^2/2}(1-\Phi(\kappa)) = \tfrac{1}{2} - e^{\kappa^2/2}\Phi(-\kappa)$. $b = P(M < \tau, B)/P(B)$ is lemma 12.8 at $T = \tau$ divided by $P(B)$. Finally $\lambda\tau = -\ln(1-c)$ and $t = \tau/\sigma = \lambda\tau/\kappa$ depend only on (c,κ). ∎

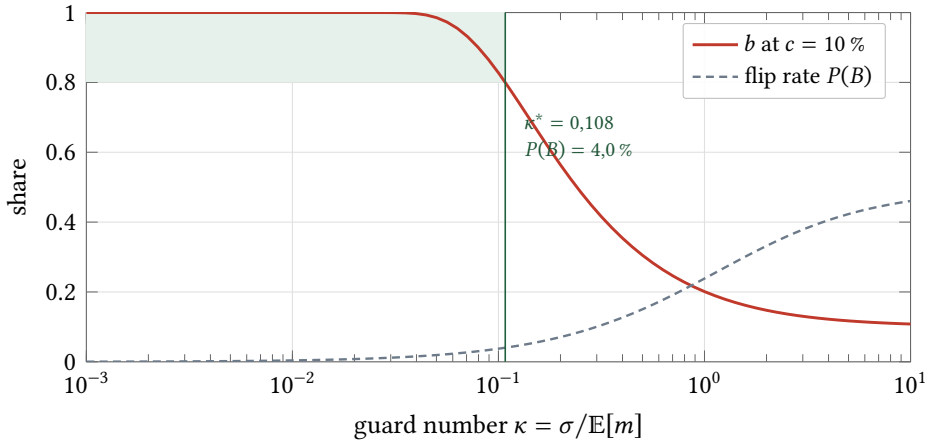


Figure 12.6: In the homogeneous model, a single quantity, $\kappa = \sigma/\mathbb{E}[m]$, decides every margin guard: the guard holds 80 / 10 if and only if $\kappa \leq 0.108$. [calc.] b at $c = 10\,\%$ and flip rate $P(B)$ against κ, by the closed form (12.5); green zone: the build zone. The model is homogeneous (one margin law, one σ); the real guard mixes 24 layers with κ_ℓ from 0.016 to 0.48, and this theorem explains only about one point of the 23 that are missing (text).

Data: `mesures/pistes/tropicale/theoreme-kappa.csv`; script `outils/pistes/tropicale/theoreme.py`.

The limits can be read on (12.5): $\kappa \to 0$ gives $b \to 1$ (the flips concentrate at $m = 0$); $\kappa \to \infty$ gives $b \to c$ (the margin no longer knows anything), control T3 seen by the theory. Verification [calc.] (`mesures/pistes/tropicale/theoreme.json`): a numerical quadrature (tolerances 10^{-13}) against the closed form on 18 pairs $\kappa \in \{0.01; 0.05; 0.1; 0.3; 1; 3\}$, $c \in \{1; 10; 30\,\%\}$: maximal gap ~~$1.3 \cdot 10^{-14}$~~ $3.7 \cdot 10^{-15}$; the first figure came from the naive writing $e^{\kappa^2/2}\Phi(-\kappa)$, which loses digits as κ grows, since rewritten through the logarithm of the normal tail. An example: $P(B)$ at $\kappa = 0.2$ is $\frac{1}{2} - e^{0.02} \times 0.42074 = 0.07076$, 0.0707601917665 by the closed form against 0.0707601917664 by the quadrature.

The 80 / 10 threshold resolves to $\kappa^* = 0.1079$, where $P(B) = 4.03\,\%$ [calc.] (Figure 12.6); the judge, by a Simpson quadrature without the closed form, gives again $b = 79.993\,\%$ and $P(B) = 4.0294\,\%$ at $\kappa = 0.1079$.

The corollary, and what the judge made of it. Since b decreases and $P(B)$ increases with κ, the lead had drawn from it: ~~"a margin guard can hold 80 / 10 only if the fast computation flips fewer than about 4 % of the cells"~~. The judge (`JUGEMENT.md` § 4.1) makes it a property of the homogeneous model alone, by a constructed counter-example: 2 layers out of 24 with tight margins (mean 10^{-4}, $\sigma = 10^{-3}$) and 22 with wide margins (mean 0.1, $\sigma = 0.004$), guard with a global threshold: $P(B) = 5.26\,\%$ and yet $b = 85.9\,\%$ for 10 % of the cells [calc.] (`mesures/pistes/tropicale/juge/juge-modele.json`). The sentence ~~"a heavier tail of δ only lowers this ceiling"~~ is true only in the single case computed: the order "heavier" is not defined there.

The theorem does not explain the failure. The deliverable set side by side "at most 4.0 % of flips are needed" and "the two ramps make 4.15 %", which reads as an explanation. At the measured rate ($5\,087/122\,640 = 4.148\,\%$), the homogeneous model gives $\kappa = 0.1113$ and $b = 78.8\,\%$; the measurement says 57.1 %. The guard number accounts for about 1 point of the 23 that are missing. The rest comes from the heterogeneity of the layers (the real guard applies a global threshold to 24 layers whose mean margin ranges from 0.061 to 0.193 and whose κ_ℓ ranges from 0.016 to 0.48) and from

Table 12.5: The extrapolation: $b(\tau_{10})$ predicted when the perturbation of the gap is multiplied by s [calc.] (model).

s	all, Gaussian δ	all, empirical δ	roots, empirical δ
0.5	71.4 %	74.5 %	93.8 %
1 (measured: 57.1 %; roots 85.2 %)	49.0 %	62.1 %	85.3 %
1.5	38.4 %	54.9 %	77.2 %
2	32.4 %	49.8 %	69.9 %
3	25.9 %	42.5 %	58.7 %
5	20.2 %	34.2 %	45.1 %
10	15.5 %	25.3 %	30.8 %

Data: `mesures/pistes/tropicale/extrapolation-garde.csv`, `cascades.json`.

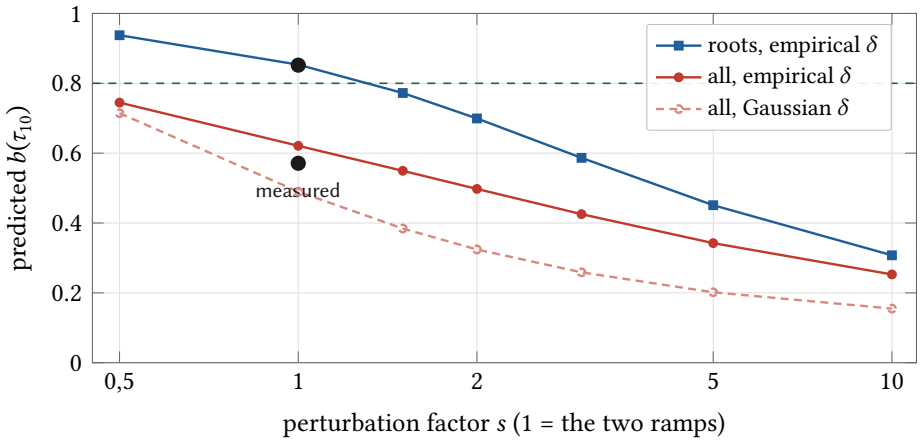


Figure 12.7: If the fast mode perturbs more than the two ramps, the guard catches even less, and the guard of the roots goes below 80 % around $s \approx 1.3$. [calc.] Model, not measurement: $b(\tau_{10})$ predicted against the perturbation factor s (roots and all, empirical δ; all, Gaussian δ); black points: the two values measured at $s = 1$.

Data: `mesures/pistes/tropicale/figures/fig-extrapolation.csv`; script `outils/pistes/tropicale/figures.py`.

the cascades. Layer by layer, the closed form follows the measurement of b at layers 0 to 9 to within 0.1 and underestimates it by 0.02 to 0.17 deeper, where the fitted σ absorbs the secondary flips [meas.] (`theoreme-par-couche.csv`). The guard number is a fine closed form; it is not the reason for the failure.

12.5.5 The extrapolation to the fast mode (model)

The fast mode probably perturbs more than the two ramps: with 32 experts out of 768 replaced, its KL is already 0.036, against 0.0145 for the two ramps, and it rises with the number of experts replaced (0.0636 with 128). The perturbation is multiplied by s while keeping the margins, $P(\text{flip} \mid m) = G(-m/s)$, with two G: the Gaussian with fitted σ_ℓ, and the empirical law of δ per layer (Table 12.5 and Figure 12.7). At $s = 1$, the two models bracket the measurement (49.0 and 62.1 % against 57.1 %); for the roots, the empirical model is right on (1 956 roots predicted against 2 081 counted, b 85.3 against 85.2 %). The roots go below 80 % from $s \approx 1.3$ (linear interpolation between $s = 1$ and 1.5; a commit message said ~~1.4~~). These are extrapolations, to be read as such.

12.6 After the verdict: two explorations

12.6.1 The guard of the roots

This reading was chosen *after* seeing the propagation; it does not change the verdict. It nevertheless has no parameter fitted on the flips: τ_{10} is a quantile of the margins alone, and the root (the first flip of the position, all layers together) depends on no threshold. 2 081 positions out of 5 110 (40.7 %) have at least one flip; the guard flags their root in **85.2 %** of cases (1 773), the secondary flips in 37.7 % (1 132 out of 3 006); second path and judge equal to the unit [meas.] (`cascades.json`, `second-chemin-racines.json`). By layer, the roots are caught from 68 % (layer 20) to 100 % (layer 0), above 80 % at 17 layers out of 24.

If repairing a root extinguished its consequences, it is the root that would have to be caught. Two things are missing, which no log provides. *The repair*: in these logs the perturbation comes from the position ramp, in the attention, and redoing a layer in exact mode would change nothing there; in fast mode, redoing layer ℓ removes the error of the experts of ℓ, not that of the earlier layers nor that of the KV cache of past positions. *The κ of the fast mode.* The definition "first flip of the position" also ignores propagation through the KV cache: a root may be the consequence of a flip at an earlier position.

The measurement that would judge (G2, then G3). Two logs in the format of lot R, the exact engine and the engine in fast mode, each with its own margin (8 bytes per cell, 981 128 bytes per pass; the `routeur_topk` kernel already sorts, the margin is one more subtraction). Threshold to be declared before the pass: the guard of the roots is worth building if, at the τ_{10} of the margins of the fast mode, it flags at least 80 % of the roots for at most 10 % of the cells, and $P(B)$ of the fast mode is measured to locate its κ. The lead's prediction, "it falls if $P(B)$ exceeds about 5 %", is that of the homogeneous model, which the judge's counter-example can refute. Only if G2 passes, G3, the repair: at each flagged root, redo the previous layer in exact mode, and judge the paired KL against the fast mode without a guard.

12.6.2 The memory of conflict: a multi-cue guard

Where the idea comes from. The anterior cingulate cortex monitors the conflict between competing responses and accumulates its trace from one trial to the next [127]; under high conflict, the mediofrontal cortex raises the decision threshold [128] (records checked by the lead on 2026-10-26, articles not reread). The guard does half of this mechanism: it reads the instantaneous conflict, the margin. The measurement asks whether the other half, the memory of conflict, sorts the flips better. It is an analogy that proposes a measurement; it proves nothing.

The protocol (`genome/pistes/tropicale/seuil-declare-indices.md`, commit `50bf472`, written before any log and unchanged since). Five cues drawn from the margin of B alone: $f_1 = \ln m$; f_2 = ln of the smallest margin of the lower layers, same position (the conflict already seen in depth); f_3 = ln margin of the previous position, same layer; f_4 = share of cells (layers $\leq \ell$, last eight positions) below 0.01241 (the conflict accumulated over time); f_5 = the depth. A logistic regression, ten folds (one test window each), and a guard that flags in each test window the cells with the highest score. Threshold: useful if the gain over the margin alone is at least 5 points, with a 95 % interval that excludes 0 and a positive gain in 8 windows out of 10. Written prediction: +1 to +6 points, and 80 % out of reach.

Table 12.6: The multi-cue guard: flips caught for 10 % of the cells of each test window [meas.]. CI: 10 000 draws of the ten windows (seed 20261026). Redone to the unit by the judge.

Model	caught	gain (points)	95 % CI	positive windows
M0, the margin alone	56.99 % (2 899)	—	—	—
M1, f_1 alone	56.99 %	0	—	—
M2, depth (f_1, f_5, $f_1 f_5$)	57.26 %	+0.28	−0.74 to +1.17	6/10
M3, all the cues	**58.60 %** (2 981)	**+1.61**	+0.99 to +2.34	9/10

Data: `mesures/pistes/tropicale/indices/indices-resume.json`; judge: `mesures/pistes/tropicale/juge/juge-indices.json`.

Two derivations. *The fit.* For standardised cues z_i (with a leading 1) and labels $y_i \in \{0, 1\}$, one minimises

$$J(w) = -\sum_i \big[y_i \ln p_i + (1 - y_i) \ln(1 - p_i)\big] + \frac{\lambda}{2}\|w_{1:}\|^2, \qquad p_i = \sigma(z_i^\top w), \tag{12.6}$$

with $\lambda = 1$ and the intercept not penalised. The gradient is $g = Z^\top(p - y) + \Lambda w$, the Hessian $H = Z^\top \operatorname{diag}(p(1 - p))\, Z + \Lambda$, positive definite for $\lambda > 0$ outside the intercept; the Newton step is $w \leftarrow w - H^{-1} g$, stopped when $\max|\Delta w| < 10^{-9}$. The guard's score is the logit $z_i^\top w$, monotone in p_i: the ranking does not depend on the function σ. *The fixed cost.* In each test window ($24 \times 511 = 12\,264$ cells), the guard flags exactly $K = \lfloor 0.10 \times 12\,264 \rfloor = 1\,226$ cells; the share caught is $b = \sum_w (\text{flips flagged in } w)/5\,087$. Each model costs the same and differs only by its sorting.

Verdict at the declared threshold: not useful (Table 12.6). The gain is real (the interval excludes 0, 9 windows out of 10) but below 5 points, and the guard remains very far from 80 %. The second path recounts the written scores in pure Python, equal to the unit in all ten folds, and another class of model (probabilities by classes, without regression) returns 59.05 % (+2.06). The judge redoes the fit itself from the declaration alone, and finds 2 981 and the interval to the thousandth; it notes that the lead's second path tested the counting, not the fit. Robustness: with $K = 1\,227$, +1.612; without penalty, +1.59 [meas.].

Where the gain comes from. The ablations, exploratory and read after the verdict, put the whole gain on f_2, the conflict already seen lower down at the same position: f_1, $f_5 + f_2$ returns +1.65 (CI +0.77 to +2.60); with f_3 or f_4 alone, +0.33 and +0.45, with intervals that contain 0; M3 without f_2, +0.43 [meas.] (`indices/complements.json`). The analogy therefore half holds: the memory of conflict along the *depth* helps a little; that along *time* brings ~~nothing~~ no detectable gain: the judge corrects the wording, because the intervals of f_3 and f_4 rise to +1.25 and +1.43 points, and a contribution of that order is not excluded.

The controls, and three wrong expectations. The identity M1 = M0 holds fold by fold; f_2, f_3, f_4 permuted within each layer bring the gain back to +0.33 (the judge: +0.35). Three declared expectations were wrong, and the lead stated two of them. (1) The break of M1 by shuffled labels was to return 10 ± 2 %; it returns ~~50.6 %~~: a model with a single input sorts the cells by $\pm f_1$ whatever the fit, only the sign of the coefficient changes; the right break permutes the test inputs (9.97 %). (2) The oracle "flip verified at $(\ell - 1, p)$" was to gain more than 15 points; it gains 10.99, because it concerns only 3.8 % of the cells and 24.5 % of the flips. (3) The judge adds the third: M3 on shuffled labels, declared at 10 ± 2 % and published at 11.6 %, returns 15.9 % on average over twenty seeds (standard

deviation 4.8, from 5.3 to 24.5 %), and only 3 seeds out of 20 fall within the expectation: the 11.6 % was a favourable draw. The control falls (58.6 → 16 ± 5 %), but not to the expectation. None of these expectations touches the verdict.

A sentence to strike through. The lead wrote that even an oracle brings only about 11 points to a guard that judges cell by cell: ~~"a ceiling of about 11 points even with an oracle"~~. The judge corrects it: the cell-by-cell oracle of the first verdict (zero margin at the flips) catches 100 % for 4.2 % of the cells; what the oracle of 10.99 points measures is that the strict vertical cascade explains only a quarter of the flips.

12.7 What could be wrong

What could be wrong

- **The perturbation is not that of the fast mode.** Everything is measured on the gap between two exact computations (two position ramps, in the attention). The verdict carries over to the fast mode through the theorem and through the extrapolation, which are models; a fast mode whose error were concentrated in the layers where the guard sees well (0 to 4) would do better, and the judge's counter-example shows that a favourable heterogeneity suffices.
- **The theorem rests on a form chosen after the reading** (Φ instead of the declared 2Φ) and on a Gaussian perturbation, wrong in the tail (kurtosis up to 134). It is labelled a model and must remain one; it explains only about one point of the failure.
- **The roots are a reading chosen afterwards**: 85.2 % is not a verdict, and the definition ignores propagation through the KV cache.
- **The causality of the cascades** is not proved by intervention: the ratio ×7.9 is an association.
- **A single text**: ten windows of wikitext; the intervals give the variation between windows of one text, not between texts. τ_{10} is the decile of the same sample, which favours the guard.
- **Rényi's law and the two cortex records** are cited after their checked records, without rereading the articles; Rényi's law is proved here without the article (lemma 12.4), but only for independent exponential variables of the same rate, which the 32 logits are not; the measurement stands on its own, the analogy proves nothing.
- **The cut of the router** passes through the origin of the space of normalised states, not through real states; its areas are not frequencies, and it is a PNG figure outside the common style.
- **Proposition 12.3** corrects a sentence of the lead's note by an argument of the writing (the distance to the boundary of a convex polyhedron is the minimum of the distances to its hyperplanes); no figure depends on it.
- **What the judge did not redo**: the laws of the margin, the correlation between layers, the prefetch, the extrapolation, the figures, the cross-check K3 d1, and the ablations of the cues. They decide nothing, but are not tested by a second look.

Chapter 13

The spectra of the matrices and the fractal

Where we start, what this chapter establishes, what remains

At the start, no slope measurement. Stone had proposed the image of a fractal: weights that would resemble one another from one scale to the next, and that could be described by few parameters. The committed data gave only, for expert 0 of each tensor, the share of energy at seven ranks: a modest excess over chance for gate and up, a strong one for down (`genome/05-synthese.md` § 3.3). The literature (Martin and Mahoney) gave a hypothesis, that of "heavy-tailed" spectra in trained networks, not the answer for gpt-oss; the prognosis of the synthesis for compression was unfavourable. **This chapter establishes**, by the method of Clauset, Shalizi and Newman on 48 matrices of gpt-oss-20b drawn before looking, that these matrices *do not follow* a power law: 0 out of 48 under the declared criterion, 9 out of 48 under the criterion amended before the reading, when 32 were needed. It poses why the declared criterion could never say yes (a theorem of the judge: the power law is the boundary of the truncated log-normal), and what is true nonetheless: all the matrices have more tail than their permuted version (48 out of 48), and this tail, heaviest in down and attn_output of layer 0, has the shape of a log-normal. One sentence of the lead is struck through ("the only place": eleven matrices, not four). **What remains**, not judged: the compression lever (none in the weights, on the sample), the spectral ceiling and its share beyond the row scales (pass unfinished), the test of the rule "$\hat{\alpha} < 2$" (it falls); and, not measured: the 21 layers not drawn, the 120b, the original weights before quantisation. Sources: `genome/pistes/spectres/spectres.md`, `pour-le-memoire.md`, `seuil.md`, `juge-seuil.md`, `JUGEMENT.md`; `mesures/pistes/spectres/` (`ajustements.out`, `second-chemin-v2.out`, `temoins.out`, `puissance-critere.out`, `rapport.json`, `fig-*.csv`, `juge/`).

In brief

A weight matrix can be described by its singular values: how much energy it puts into each of its directions. For a matrix drawn at random, this distribution is known and bounded, without a tail. A "fractal", in the sense Stone meant, would leave a precise signature: a power law, the same shape at every scale. The fractal is an image here, and the question it raises can be measured. The answer is no: training did put energy into a few directions, everywhere, but the shape of this tail is not a power law; where it is heaviest, it copies the law of a few giant rows, the outlier channels, which a simple per-row scale already captures.

13.1 The question and the quantity

Three questions: do the spectra of gpt-oss follow a power law; with which exponents, by role and by layer; and does this spectrum open a measurable compression lever, in bits per weight? This chapter answers the first two, judged, and reports the third as not judged.

Definition 13.1 (Empirical spectral density)**.** For $W \in \mathbb{R}^{m\times n}$, $m \le n$ (otherwise transpose), the correlation matrix $X = WW^\top/n$ has eigenvalues $\lambda_i = \sigma_i^2/n$, where the σ_i are the singular values of W. The empirical spectral density is $\rho_W(\lambda) = \frac{1}{m}\sum_i \delta(\lambda - \lambda_i)$. A *power-law tail* is $\rho(\lambda) \propto \lambda^{-\alpha}$ beyond a threshold $\lambda_{\min}$.

The exponent depends on the quantity fitted: if $\rho(\lambda) \propto \lambda^{-\alpha}$, then with $\lambda = \sigma^2$, $\rho(\sigma) = \rho(\lambda)\,|\mathrm{d}\lambda/\mathrm{d}\sigma| \propto \sigma^{-2\alpha}\,\sigma = \sigma^{-(2\alpha-1)}$. The fit is on λ, like Martin and Mahoney [118, 129], who describe in state-of-the-art networks a "heavy-tailed" self-regularisation and show that metrics drawn from these laws judge the quality of a model better than norms do (abstracts read by the lead on 2026-10-21; their numerical thresholds for α, in the body of the articles, are not cited here).

The sample, drawn before looking (seed 20261021, `seuil.md` § 1; the judge redoes the draw and finds the same experts). Layers 0, 12, 23; experts [0, 6, 22, 24] (layer 0), [1, 3, 19, 31] (layer 12), [7, 21, 26, 27] (layer 23); gate, up and down of each (36 matrices $2\,880 \times 2\,880$, MXFP4 dequantised in exact f64), plus attn_q, attn_k, attn_v, attn_output of the three layers (12 Q8_0 matrices). In all 48 matrices and 124 032 singular values, $36 \times 2\,880 + 3 \times (2\,880 + 512 + 512 + 2\,880)$, saved together with those of the 48 permuted matrices and of the controls; 617 s of computation on the processor [meas.].

13.2 The control of chance: Marchenko-Pastur

For W with independent entries of variance s^2, when $m, n \to \infty$ with $q = m/n \le 1$ fixed, the spectral density of $X = WW^\top/n$ tends to the law of Marchenko and Pastur [130]

$$\rho_{\mathrm{MP}}(\lambda) = \frac{\sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)}}{2\pi s^2 q\,\lambda}, \qquad \lambda \in [\lambda_-, \lambda_+], \quad \lambda_\pm = s^2(1 \pm \sqrt{q})^2. \tag{13.1}$$

For a square matrix ($q = 1$), the support is $[0, 4s^2]$, the density diverges as $\lambda^{-1/2}$ at 0 and vanishes as a square root at the right edge. *The support is bounded*: there is no tail, and the largest eigenvalue exceeds λ_+ only by fluctuations of order $n^{-2/3}$. For $q = 512/2\,880 = 0.178$ (attn_k and attn_v seen from their short side), $\lambda_\pm = s^2(1 \pm 0.4216)^2$, that is $[0.3345\,s^2;\ 2.021\,s^2]$ [calc.]. A power-law fit on such a spectrum must find nothing: this is control T-MP, which must fall.

The second control, *T-perm*, permutes the entries of each matrix: it keeps the marginal law of the entries (and their kurtosis, up to 2 000 on down) and destroys all rank structure. The gap between W and its permuted version is what training put into the directions.

13.3 The instrument: Clauset, Shalizi and Newman, step by step

The method [131] is written from scratch by the lead (`outils/pistes/spectres/csn.py`). The journal references of the articles of CSN, of Marchenko and Pastur and of Vuong, given from memory by the lead, were checked by the writers on 2026-10-29 in their Crossref records (DOI `10.1137/070710111`, `10.1070/SM1967v001n04ABEH001994`, `10.2307/1912557`): journal, volume, issue and first page agree; the articles are not reread.

Table 13.1: The power of the two criteria, on synthetic draws of $n = 2\,880$, before the reading of the real spectra [meas.].

20 draws	declared criterion	amended criterion "compatible"	median $\hat{\alpha}$
Pareto $\alpha = 2.5$	0 / 20	**18 / 20**	2.496
Pareto $\alpha = 3.5$	0 / 20	**20 / 20**	3.510
exponential	0 / 20	0 / 20	—
log-normal $\sigma = 1$	0 / 20	4 / 20	—

Data: `mesures/pistes/spectres/puissance-critere.out` (commit `fdf5929`); `genome/pistes/spectres/spectres.md` § 2.

The exponent by maximum likelihood. For the continuous law $p(x) = \frac{\alpha-1}{x_{\min}}\left(\frac{x}{x_{\min}}\right)^{-\alpha}$, $x \geq x_{\min}$, the log-likelihood of n points of the tail is $\ell(\alpha) = n\ln(\alpha-1) - n\ln x_{\min} - \alpha\sum_i \ln(x_i/x_{\min})$. Setting $\mathrm{d}\ell/\mathrm{d}\alpha = n/(\alpha-1) - \sum_i \ln(x_i/x_{\min})$ to zero,

$$\hat{\alpha} = 1 + \frac{n}{\sum_i \ln(x_i/x_{\min})}, \qquad \sigma_{\hat{\alpha}} = \frac{\hat{\alpha}-1}{\sqrt{n}}, \tag{13.2}$$

the standard error coming from the Fisher information, $\mathrm{d}^2\ell/\mathrm{d}\alpha^2 = -n/(\alpha-1)^2$.

The threshold by Kolmogorov-Smirnov. For each candidate $x_{\min}$ (observed values, tail of at least 50 points), $\hat{F}(x) = 1 - (x/x_{\min})^{1-\hat{\alpha}}$ and $D = \max_x |S(x) - \hat{F}(x)|$, S the empirical distribution function of the tail; at the sorted points $t_{(j)}$, $D = \max\big(\max_j((j+1)/n - \hat{F}(t_{(j)})),\ \max_j(\hat{F}(t_{(j)}) - j/n)\big)$. The $x_{\min}$ with minimal D is kept.

The competitors, truncated at $x_{\min}$. The exponential, $p(x) = \lambda e^{-\lambda(x-x_{\min})}$, with maximum likelihood $\hat{\lambda} = 1/(\bar{x} - x_{\min})$; the log-normal, $p(x) = \varphi\big((\ln x - \mu)/\sigma\big)/\big(x\sigma\,(1 - \Phi((\ln x_{\min} - \mu)/\sigma))\big)$, without a closed form, maximised numerically.

Vuong's ratio [132]. With $\ell_i = \ln p_{\mathrm{PL}}(x_i) - \ln p_{\mathrm{alt}}(x_i)$ and $R = \sum_i \ell_i$, under the hypothesis "the two laws are equally close to the true one", $R/(\hat{\sigma}\sqrt{n})$ follows $\mathcal{N}(0,1)$ asymptotically; $p = \mathrm{erfc}\big(|R|/(\hat{\sigma}\sqrt{2n})\big)$. $R > 0$ with small p: the power law is preferred.

Plausibility. Replicas draw each point, with probability n_{tail}/N, from the fitted power law ($x = x_{\min}u^{-1/(\hat{\alpha}-1)}$), otherwise from among the points below $x_{\min}$; $x_{\min}$ and α are re-estimated, and p_{gof} is the share of replicas whose D is at least that of the data (100 replicas).

13.3.1 The declared threshold, and why it was amended before the reading

Declared threshold (`genome/pistes/spectres/seuil.md`, commit `a171622`, 2026-10-21 23:22:25): call a matrix "power law" only if the method prefers it to the exponential *and* to the log-normal with $p < 0.05$, and call the matrices of gpt-oss so only if two thirds of the sample (32 out of 48) are. A trial of the instrument on synthetic draws, *before* any fit of gpt-oss, showed that this criterion could not say yes (Table 13.1).

Hence the **amendment** (`seuil.md` § 4 bis, commit `bfc1c8c`, 23:25:57, eight minutes before the real spectra existed in the repository): the declared verdict is reported as is, and CSN's "*compatible with a power law*" verdict is added (their "moderate or better"): $p_{\mathrm{gof}} \geq 0.1$, the exponential ruled out

($R > 0$, $p < 0.05$) and the log-normal not preferred. It always reads "power law, or log-normal of very large σ, indistinguishable here". It was to be kept only if it said "compatible" on at least 18 Paretos out of 20 and at most 2 exponentials out of 20: it passes, at the edge, with 18 out of 20 at $\alpha = 2.5$. The judge notes two reservations of form (`JUGEMENT.md` § 1): the amendment quotes median p of 0.59 and 0.63 against the log-normal, which come from a trial without a log; the only committed log gives 0.61 and 0.69, which the deliverable quotes.

13.3.2 The declared criterion can never say yes: the judge's theorem

The lead explained the lack of power by the size: "over an observed range of a few units of y, the curvature that separates them is below the noise". The judge posed it, before computation, as a stronger theorem (`juge-seuil.md`, J1; `JUGEMENT.md` § 2).

Theorem 13.2 (The power law is the boundary of the truncated log-normal)**.** *On a tail $\lambda \geq \lambda_{\min}$, set $y = \ln(\lambda/\lambda_{\min}) \geq 0$. For every sample, the likelihood ratio $R = \ell_{\mathrm{PL}}(\hat{\alpha}) - \sup \ell_{\mathrm{LN}}$ of the power law against the truncated log-normal fitted at its maximum is ≤ 0, at any size n. The criterion "$R > 0$ and $p < 0.05$ against the log-normal" is therefore impossible.*

Proof. In y, the truncated log-normal with parameters (μ, σ) has density (the Jacobian $1/\lambda$ of the law in λ cancels with $\mathrm{d}\lambda = \lambda\,\mathrm{d}y$)

$$g(y) \propto \exp\!\Big(-\frac{(y + \ln\lambda_{\min} - \mu)^2}{2\sigma^2}\Big) \propto \exp(-c_1 y^2 + b\,y), \qquad c_1 = \frac{1}{2\sigma^2} > 0, \quad b = \frac{\mu - \ln\lambda_{\min}}{\sigma^2},$$

a truncated Gaussian. The power law $\rho(\lambda) \propto \lambda^{-\alpha}$ has density in y $(\alpha - 1)\,e^{-(\alpha-1)y}$: it is the same model at the boundary $c_1 = 0, b = -(\alpha - 1)$. The log-likelihood $\ell(c_1, b) = \sum_i (-c_1 y_i^2 + b\,y_i) - n \ln Z(c_1, b)$, with $Z = \int_0^\infty e^{-c_1 y^2 + by}\,\mathrm{d}y = \frac{1}{2}\sqrt{\pi/c_1}\ \mathrm{erfcx}\big(-b/(2\sqrt{c_1})\big)$, is continuous up to the boundary ($Z \to 1/|b|$ as $c_1 \to 0^+$, $b < 0$) and concave (exponential family with statistics $-y^2$, y). Therefore $\sup_{c_1 > 0} \ell \geq \ell(0, -(\hat{\alpha} - 1)) = \ell_{\mathrm{PL}}(\hat{\alpha})$, and $R \leq 0$. The Jacobians, common to both laws, cancel in R. ∎

Checked on the judge's 141 spectra (48 real, 48 permuted, 45 controls): $R_{\text{log-normal}} \leq 2.7 \times 10^{-13}$, machine zero [meas.]. The log-normal likelihood reaches the boundary ($\sigma = \infty$) on 11 of the 48 real matrices and on most of the true Paretos. Corollary: the *thirteen* values $R_{\text{log-normal}} > 0$ of the lead's table (up to +0.096 on L12.down.E19) are unfinished numerical optima of the (μ, σ) parametrisation, which heads towards $\mu \to -\infty$ without reaching it; no verdict depends on them ($p \geq 0.34$ everywhere there), and this is also the true reason for the gap in p against the log-normal between the lead's two paths. The declared verdict is therefore not "without power at $n = 2\,880$": it is empty by construction.

13.3.3 The controls, the break, the second path

The **break** swaps the inputs of T-Pareto and of square T-MP: under the amended criterion, both fall (T-Pareto not compatible, $\hat{\alpha} = 16.9$; T-MP compatible, $\hat{\alpha} = 2.49$): the instrument can say yes and no, and it is the data that decide (Table 13.2). Under the declared criterion, the break brings down only T-Pareto: a verdict that never says yes cannot make a negative control fail, a second proof, through the instrument, of its lack of power. One remark helps to read what follows: pure chance returns $\hat{\alpha} = 9$ to 17, because the drop of a spectrum with bounded support, seen from above, looks like a very steep slope. *A small $\hat{\alpha}$ means a heavy tail; an $\hat{\alpha}$ of 10 or more means "no tail".*

Table 13.2: The controls of the instrument, under the two criteria [meas.].

Control	expected	declared criterion	amended criterion	$\hat{\alpha}$
T-Pareto ($\alpha = 2.5$ planted, passed through an SVD)	recognised, $\alpha \in [2.3; 2.7]$	failure (foreseen)	**holds**	2.492 ± 0.032
T-MP $2\,880 \times 2\,880$	not a power law	holds	**holds**	16.9
T-MP, three other shapes	not a power law	holds	**holds**	14.9; 15.8; 8.8
T-exponential	not a power law	holds	**holds**	5.75
T-log-normal ($\sigma = 1$)	reported	not settled	not compatible	3.03

Data: `mesures/pistes/spectres/temoins.out`, `temoins-casse.out`; `genome/pistes/spectres/spectres.md` § 2.

Table 13.3: The verdict by family, 48 matrices of gpt-oss-20b [meas.]. Redone identically by the judge, with the same nine compatible matrices (agreement 48 / 48).

Family	n	declared criterion	**amended criterion**	expon. ruled out	log-norm. preferred	$p_{\text{gof}} \geq 0.1$	permuted compatible
gate	12	0	**4**	4	0	12	0
up	12	0	**0**	0	3	6	0
down	12	0	**3**	7	4	5	0
attention	12	0	**2**	5	2	9	1
all	**48**	**0**	**9**	16	9	32	1

Data: `mesures/pistes/spectres/ajustements.out`, `rapport.json`; judge: `mesures/pistes/spectres/juge/juge-csn.csv`, `comparaison-juge-csn.txt`.

The second path (`second-chemin-v2.out`) redoes the dequantisation from the specification (E2M1 decoded bit by bit, E8M0 scale $= 2^{e-127}$), the eigenvalues through the Gram matrix, the sweep of $x_{\min}$ and α differently: 48 equal SHA-256 fingerprints out of 48, eigenvalues of the tail to 3.8×10^{-14}, equal $\lambda_{\min}$, α to 2.2×10^{-7}, equal verdicts [meas.]. Its first version was wrong: ~~$\hat{\alpha}$ from 1.1 to 2.5 instead of 1.6 to 6.9~~; its likelihood carried $-\alpha \sum \ln t_i$ instead of $-\alpha \sum \ln(t_i/\lambda_{\min})$, and its fingerprints differed on the sign of the zeros (the bit-by-bit decoder returns -0.0 for code 8, the table $+0.0$: the project's synonymous codon, found again here by accident).

13.4 The verdict: not a power law

At the declared threshold: 0 matrices out of 48 under the declared criterion, 9 out of 48 under the amended criterion; 32 were needed. *One cannot write that the matrices of gpt-oss-20b follow a power law*: neither under the declared criterion, which can say nothing, nor under the amended criterion, which could say yes and says no (Table 13.3). No family reaches its two thirds; gate is the closest (4 out of 12). The judge redoes everything by a path of its own (exact two-sided KS, log-normal in natural parameters starting from the moments, replicas from another seed): 0 / 48, 9 / 48, the same nine matrices, $\lambda_{\min}$ and $\hat{\alpha}$ equal on 48 / 48. The number 9 depends a little on its edges (four compatible matrices rule out the exponential narrowly, three non-compatible ones miss it by little); even tipping all these cases in the favourable direction, one reaches 12, far from 32. With a looser criterion still (the exponential ruled out, nothing else), 16 out of 48: the verdict does not depend on the choice of criterion.

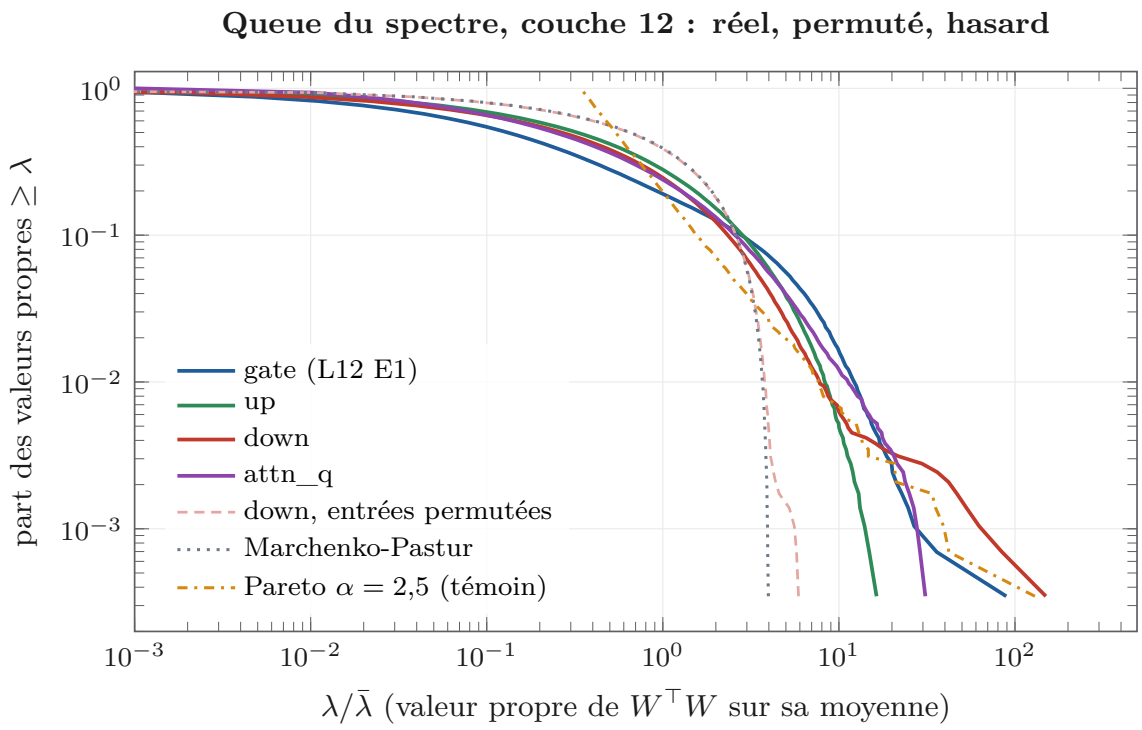


Figure 13.1: All the families have more tail than chance, but none traces the straight line of a power law. [meas.] Share of eigenvalues $\geq \lambda$ against $\lambda/\bar{\lambda}$, in log-log, for one matrix of each family at layer 12 (gate, up, down of expert 1; attn_q), the same down with permuted entries, the Marchenko-Pastur law (vertical drop at the edge of its support) and the Pareto control $\alpha = 2.5$ (a straight line of slope $-(\alpha - 1)$). Page 1 of the lead's figures, judged. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/pistes/spectres/fig-ccdf-*.csv`; `genome/pistes/spectres/figures/spectres-figures.tex`, page 1.

Table 13.4: The exponents $\hat{\alpha}$ (on $\lambda = \sigma^2$), **medians by layer**; in brackets, the four matrices in the order of the draw, the compatible ones in bold; last row, the medians of the permuted matrices [meas.]. The individual $\hat{\alpha}$ range from 2.70 to 5.95 for gate and from 3.50 to 8.32 for up.

	layer 0	layer 12	layer 23
gate	3.75 [4.15; 3.35; 5.95; **2.98**]	3.96 [3.67; 3.78; 4.54; 4.13]	3.02 [**3.04**; 3.00; **2.70**; **3.21**]
up	5.84 [6.24; 5.43; 6.94; 3.96]	4.33 [3.83; 4.83; 3.50; 8.32]	3.70 [3.75; 3.64; 3.57; 4.12]
down	**1.63** [1.65; 1.61; 1.71; 1.61]	3.09 [**2.72**; 3.18; **2.99**; 3.60]	6.44 [6.06; 6.83; **2.74**; 7.25]
attention (q, k, v, o)	6.20; 4.23; 6.04; 1.72	2.87; 2.95; 5.12; **2.89**	**1.90**; 1.50; 1.71; 4.05
permuted (gate; up; down; att.)	15.4; 17.2; 7.9; 8.9	12.5; 17.3; 5.4; 7.9	14.9; 13.5; 16.3; 8.7

Data: `mesures/pistes/spectres/rapport.json` (field `alpha`); `genome/pistes/spectres/spectres.md` § 4.1; judge: $\hat{\alpha}$ equal on the 48 matrices and the 48 permuted ones.

What the 39 non-compatible matrices are (Figure 13.1). Most often a *tail too short to decide*: the tail kept by KS retains only 50 to 120 eigenvalues for 18 matrices, a few spiking directions above an almost random mass, and the exponential is ruled out for only 16 matrices out of 48. For up, a *lighter tail*: the exponential or a log-normal of small σ (0.17 to 0.85 for 10 out of 12) is preferred or equivalent. For down and attn_output of layer 0, a *heavy but curved tail*: $\hat{\alpha} \approx 1.6$ to 1.7 over 565 to 973 eigenvalues, but the log-normal is preferred for all five ($p \leq 0.0044$; $\sigma = 1.8$ to 3.0), and the power law is not even plausible for the four down matrices ($p_{\text{gof}} = 0$).

13.4.1 The exponents, by role and by layer

The standard errors range from 0.02 (down, layer 0) to 0.99 (up, layer 12, E31, on 55 points) [meas.] (Table 13.4 and Figure 13.2). The lead gave "gate 3.0 to 4.0; up 3.7 to 5.8": these are the medians by layer, not ranges, and the judge asks for it to be written that way.

What they say. (1) *All the matrices have more tail than chance*: the real $\hat{\alpha}$ is below that of the same permuted matrix in all 48 cases (ratio permuted over real from 1.26 to 8.81, median 3.49, redone by the judge); training put energy into a few directions (the share of the first 5 % of directions is 1.25 to 5.3 times that of chance). This is the only "positive" result of the lead, and it is broad. (2) *up is the closest to chance* ($\hat{\alpha}$ 3.5 to 8.3, none compatible); gate is intermediate and hardens with depth (3 compatible out of 4 at layer 23). A reading, not measured: gate and up receive the same input, but only the output of gate goes through the non-linearity that decides which channel opens. (3) *down and attn_output of layer 0, and q, k, v of layer 23, have very heavy tails* ($\hat{\alpha} = 1.5$ to 1.9: below 2, the fitted law does not even have a finite mean); these are also the matrices whose entries have an enormous kurtosis (down of layer 0: 60 to 1 988). The matrices that write into the residual stream at layer 0 are those that manufacture its outlier channels. (4) *This heavy tail is not a power law*: for down of layer 0, the log-normal is preferred and the power law rejected. (5) The lead wrote: ~~"Layer 12 of down is the only place where the power law and the log-normal truly merge"~~. This is false (`JUGEMENT.md` § 4.1): the fitted log-normal *is* the power law, at the boundary $c_1 = 0$ of theorem 13.2, on eleven matrices: the four down matrices of layer 12, but also L00.gate.E22, L00.attn_k, L12.gate.E01, L12.attn_output, L23.down.E07, E21 and E27; the published table already showed it ($\mu = -69$ to -118 for those seven).

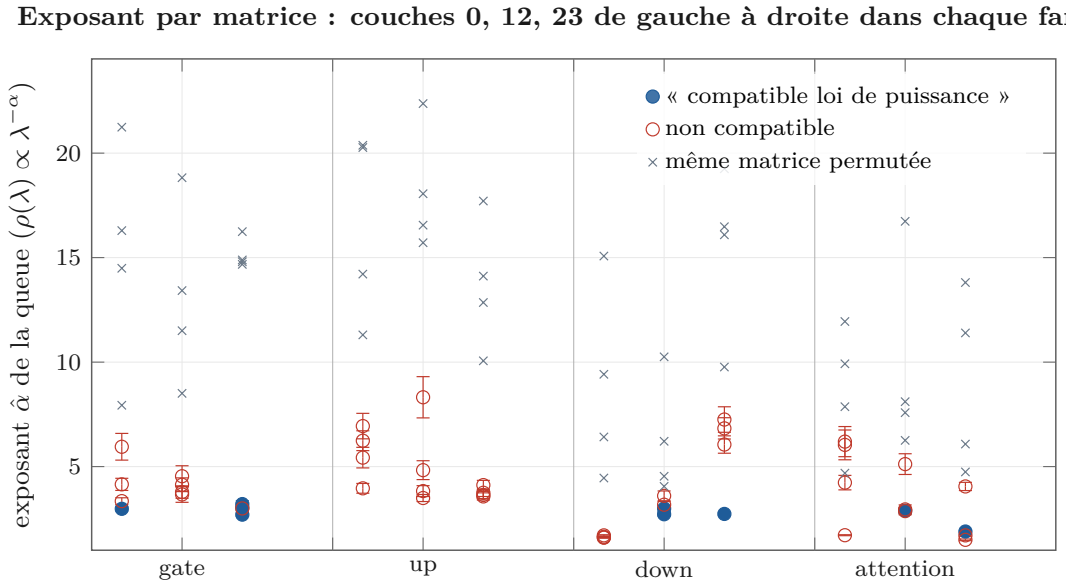


Figure 13.2: The real exponent is below that of the same permuted matrix in all 48 cases; only nine matrices are compatible with a power law. [meas.] $\hat{\alpha}$ by matrix and its standard error $(\hat{\alpha}-1)/\sqrt{n_q}$, by family, layers 0, 12, 23 from left to right within each family; filled discs: compatible; circles: not compatible; crosses: the same matrix with permuted entries. Page 2 of the lead's figures, judged. (figure outside the common style of the thesis, included as its lead drew it)

Data: `mesures/pistes/spectres/fig-alpha.csv`; `genome/pistes/spectres/figures/spectres-figures.tex`, page 2.

Relation to Martin and Mahoney. What is measured here agrees with the first half of their description (tails heavier than chance in 48 matrices out of 48, an excess that varies strongly by role and by layer) and does not confirm the second in the strict sense: under the CSN test, the tail is a plausible power law in only 9 matrices out of 48, and the heaviest of all is a log-normal. A difference of method (they often fit the density of $W^\top W/N$ with the `powerlaw` library) can move α by a fraction of a unit; it does not change the shape.

13.5 The lever and the ceiling: what is not judged

The lead's judge judged neither the lever, nor the spectral ceiling, nor the per-rank ceiling, nor the prediction test, nor the exploratory normalisation measurement, nor the pass of the per-row ceiling, which was still running (`JUGEMENT.md` § 5). This section poses their derivations, which depend on no measurement, and reports their numbers with the label [not judged]; their figures (pages 3 to 5 of the lead's file) are not placed. All these gains are *in the domain of the weights*, and the synthesis of the round showed that the weight error does not predict the KL: a lever here would be a necessary condition, never a sufficient one.

The spectral ceiling. W is read as $\max(m,n)$ Gaussian vectors of dimension $p = \min(m,n)$, with a covariance of eigenvalues λ_j. At relative error D, reverse water-filling in the eigenbasis (the Karhunen-Loève transform) gives $D_j = \min(\theta, \lambda_j)$, $\sum_j D_j = D\sum_j \lambda_j$, and the rate $R_{\mathrm{KLT}} = \frac{1}{p}\sum_j \max\bigl(0, \frac{1}{2}\log_2(\lambda_j/\theta)\bigr)$ bits per weight [lit.] (Cover and Thomas [2]; the passage is cited after

the lead's note, not reread here). The flat spectrum of the same energy costs $R_{\text{white}} = \frac{1}{2}\log_2(1/D)$, that is 2.2535 bits at $D = 0.044$. The ceiling $G = R_{\text{white}} - R_{\text{KLT}}$ bounds from above the gain of any method that codes in a basis, under this model: coding coordinate by coordinate with its own variances is allowed, and does no better than the transform (at high rate, this is Hadamard's inequality, $\det C \le \prod_i c_{ii}$), which itself does no worse than the white basis (the inequality of means). At high rate, if $\theta < \lambda_{\min}$,

$$G = \tfrac{1}{2}\log_2 \frac{\bar{\lambda}}{\lambda_g}, \tag{13.3}$$

the logarithm of the ratio of the arithmetic and geometric means of the spectrum, a flatness measure in which the small values weigh as much as the large ones. Two closed forms follow from it [calc.]: for Marchenko-Pastur with ratio 1, $\mathbb{E}\ln\lambda = -1$ and $G = \frac{1}{2}\log_2 e = 0.7213$ bit; for a Pareto ($\lambda \ge 1$, $\alpha > 2$), $\bar{\lambda} = (\alpha-1)/(\alpha-2)$, $\mathbb{E}\ln\lambda = 1/(\alpha-1)$, and

$$G_{\text{Pareto}}(\alpha) = \tfrac{1}{2}\log_2 \frac{\alpha-1}{\alpha-2} - \frac{1}{2(\alpha-1)\ln 2}, \tag{13.4}$$

which is $0.7925 - 0.4809 = 0.3116$ at $\alpha = 2.5$, *less* than square chance (0.5899 at $D = 0.044$). A heavy tail at the top does not, by itself, make a large ceiling; this is why the lead's first positive control, ~~"Pareto above MP"~~, fell, a wrong expectation replaced before any reading of gpt-oss by a planted rank 8 (predicted ≈ 0.24, measured 0.216), and why the excess is read against the permuted version of the same matrix. On the sample, the weighted excess is 0.424 bit per weight [not judged], largest in down and attn_output of layer 0 and q, k, v of layer 23: exactly the matrices with $\hat{\alpha} < 2$.

What remains of it once the basis is paid for. Offering the decoder the first r directions costs b bits per factor coefficient, $b\,r\,(m+n)/(mn)$ bits per weight, that is $0.0111\,r$ at $b = 16$ for a square matrix [calc.]; a complete basis costs at least b bits per weight, for a ceiling that does not exceed 1.8 bits over the 48 matrices: it never pays for itself. Two thirds of the ceiling live beyond the 64th direction, and at $b = 16$ no family has a lever at the threshold of 0.05 bit (down: 0.042) [not judged]. The lever measured on the matrices themselves, lossless on the MXFP4 codes through the conditional mutual information $I(C; B_r \mid E)$ and lossy through a per-row Gaussian bound and a real quantiser, is worth on the sample only 0.0009, 0.0036 and 0.0007 bit per weight [not judged]. The real quantiser requires subtractive dithering: on an already discrete source, a uniform quantiser without dithering measures the grid and not the structure (the permuted control returned the same gain), whereas with a dither $u \sim U(-\frac{1}{2}, \frac{1}{2})$ known to the decoder, the error becomes uniform and independent of the signal whatever its law (results of Schuchman and of Zamir and Feder, cited after the lead's note, not reread), with energy exactly $N\Delta^2/12$.

Where the structure went. The ceiling mixes two things: the inequality of the per-row variances, which any coder with a per-row scale captures, and the structure between rows, which only a basis captures. The chain $R_{\text{KLT}} \le R_{\text{row}} \le R_{\text{white}}$ separates them [calc.]. For L00.down.E24, $R_{\text{row}} = 0.755$ bit instead of 2.254: the scales alone take 1.50 of the 1.74 bits of the ceiling. MXFP4 has better still, one scale per block of 32 weights. An exploratory measurement, not declared and not judged, confirms it for the tail: normalising the rows of down and attn_output of layer 0 takes their $\hat{\alpha}$ from 1.6–1.7 to 3.4–7.0, normalising the columns does not change it; their largest singular value is only 1.1 to 1.4 times their largest row norm. The heaviest tail is the law of a few giant rows, copied. Finally, the rule "$\hat{\alpha} < 2$ designates the matrices where a low rank pays for itself", seen before being declared, was tested on 36 new matrices (layers 3, 9 and 21): 4 out of 6 below 2, but 5 out of 30 above for 10 % allowed: it falls [not judged]. The idea of the fractal gives here neither a law nor a selector.

13.6 What could be wrong

What could be wrong

- **The sample**: 48 matrices out of the 2 400 of the model (2 %), 3 layers out of 24, 4 experts out of 32. The verdict holds for this randomly drawn sample; a layer not drawn could differ, and the slopes with depth are read on three points.
- **The quantity**: the spectra are those of the dequantised weights (MXFP4, Q8_0), not of the original weights in BF16, not downloaded. Quantisation adds a roughly white noise that raises the bottom of the spectrum and can shorten the apparent tail; its effect on α is not measured.
- **The guard of 50 points** on the tail is a declared choice, not a derivation; five matrices have their $\lambda_{\min}$ at the guard or very close to it, and for them nothing is decidable.
- **The amended criterion** is set after seeing that the declared criterion had no power, but before any reading of the real spectra; it passes its power test at the edge (18 / 20). The verdict does not depend on this choice (16 / 48 at the loosest).
- **The permuted control is a single permutation per matrix**; at the measured gap (ratio ≥ 1.26), another seed would not overturn "48 / 48", but this is not measured.
- **The references**: Martin and Mahoney are known only from their abstracts; the records of CSN, Marchenko-Pastur and Vuong are checked, not the articles; Cover and Thomas, Schuchman, Zamir and Feder, Gray and Markel are cited after the lead's note, without verification.
- **Everything labelled "not judged"** (lever, ceilings, test of the rule, normalisation) has had no second look; the lead is not closed (pass of the per-row ceiling unfinished at the judgement, summary of the deliverable not written).
- **The figures** are those of the lead, outside the common style; only the two judged pages are placed.

Chapter 14

Discussion

Where we start, what this chapter establishes, what remains

At the start, twelve chapters, each closed on its own "What could be wrong" section, and each judged by a judge that had not done the work. **This chapter establishes** what the volume says as a whole: the verdicts, each with its judge and the number that decides it; the struck-through figures, gathered together; the lesson of the quest; what could be wrong at the scale of the volume; and the work to come, in order of expected gain, costed where it is. **What remains** is to do it.

14.1 What the volume establishes, together

Table 14.1 gathers the verdicts. A verdict is that of the criterion declared before the measurement, as the judge of the step redid it; "open" means that neither of the two thresholds, success or death, is crossed, and the chapter says why.

Table 14.1: The verdicts of Volume III, each with the number that decides it and the chapter that carries it.

Mechanism	Verdict	The number that says it	Where
They win (the synonyms)			
zero-synonym codon	wins, lossless	the sign of a zero has an entropy of 0.9999999967 bit and the computation never reads it; 0.127 91 and 0.125 62 bit per expert weight; logits identical to the bit (judge: 0 out of 5 120, two models)	Chapters 3, 9 and 11
column context	wins, lossless, at layer 0	−0.142 and −0.123 bit at layer 0 (gate and up, against the byte stream); below 0.01 bit elsewhere	Chapter 8
synonymous mutation ($\lambda = 0$, 768 experts)	wins in KL	0.131 8 against 0.264 0 (R1); −0.156 5 ± 0.033 4 against the frontier outside R1 (judge, 4.7 σ)	Chapters 10 and 11
$\lambda = 0$ against $\lambda = 1$	$\lambda = 0$ kept	2.09 σ on R1, 3.6 σ outside R1	Chapter 10
Equal			
partitions by κ	equal in KL, fewer bits	paired +0.006 6 ± 0.005 3; 0.911 times the bits	Chapter 8
S* (the strand and its exceptions)	wins at expert output, equal in KL	error ratio 0.747; paired KL +0.000 4 ± 0.001 9 (ten windows; R4)	Chapters 4 and 8
They fall			
seed and coefficients	falls	3.06 bits at the base's error	Chapter 4
shared motifs	falls	2.90 bits	Chapter 4
recipe and exact complement	falls	4.05–4.37 bits against 3.84	Chapter 4
duplication (the parent)	falls	0.003 4 bit, of which 0.001 6 by chance	Chapter 5

Mechanism	Verdict	The number that says it	Where
Hebbian, pruning, partitions by e	fall	Hebbian: KL ×3.06 the base for an expert error ×1.02; pruning by magnitude: ×1.012 the bits of the base at equal error	Chapters 6 and 8
fast mode, generation gain	does not hold	×1.28 (120b) and ×0.94 (20b) against v1.2 (judge)	Chapters 10 and 11
tropical guard	falls	57.1 % of the flips for 10 % of the cells; 80 % would cost 26.6 % (judge)	Chapter 12
power law of the spectra	cannot be written	0 matrices out of 48 under the main criterion, 9 "compatible" under the amended criterion, 32 were needed (judge)	Chapter 13
Open			
P1, two-scale cache memory	open	the protocol's "falls" is decided by tokens produced after the end of the message (judge)	Chapter 7
P2, exclusive cache	open	an inclusive RAM tier smaller than the VRAM is useless	Chapter 7
P3, cache-conditioned routing	at the limit	about 10 % of the misses removed for a KL of 0.031 4 at the online router ($\beta_1 = 0.041$), 0.020 9 and 0.023 9 at stage B ($\beta = 0.06$ and 0.07) (what is judged; reserve R6)	Chapter 7
P4, prefetch with calibrated confidence	open, rather falls	simulator not validated	Chapter 7
P5, annealing and replica exchange	open	median 5.09 % of gain held out; the announced mechanism does not hold at the threshold (judge)	Chapter 7
P6, fidelity decided per call	open; B holds at step 0	a "holds" that could hardly fall (judge)	Chapter 7

What this does, for the machine. Of the three synonyms, two are built: the zero-synonym codon, in the format, and the synonymous mutation, in the fast mode; the column context, which returns something only at layer 0, stays outside the v1.2 format (Section 9.4.1). Losslessly, the v1.2 format makes the 120b read 3.3 % fewer bytes per token, and J1 gains ×1.44 there on this disk, because the pattern of its read requests is fast where that of v1.0 is not (reserve R1). Lossily, the fast mode serves the 20b with a KL of 0.13 for 2.67 bits per expert weight instead of 3.86, but this gain in bytes does not become a generation gain: the routing of a lossy model misses the VRAM more often, and the bytes per step stay the same. The speed gain that the quest found is not a mechanism of the genome: it is the format, read on this disk.

14.2 The struck-through figures

When a measurement contradicted a published figure, the sentence was struck through where it stood. Table 14.2 gathers them; it is the trace that the method worked.

14.3 The lesson of the quest

The synthesis of the round said it in one sentence per level, and the construction confirmed it.

- **In the domain of the weights, there is nothing to generate.** The gain is taken on what the computation does not read (the sign of a zero) or on what the coder already knows (the scale, the column). The recipe, the seed, the kinship assume a dependence between weights that these weights, once the scale is known, do not have.

Table 14.2: The figures and sentences struck through during the quest, and what replaced them.

Struck through	Replaced by, and why	Where
~~S*: 33 % more KL~~ (three windows)	equal to the base over ten windows, +0.000 4 ± 0.001 9 (R4)	Chapter 8
~~$\lambda = 1$: KL 0.0262~~	the KL 0.0262 was that of $\lambda = 0$; no variant was yet below the frontier in bits and in KL at once	Chapter 8
~~768 substituted experts~~ (control)	substituted = read (732 out of 732): 36 experts are routed by no position	Chapter 10
~~20b ×1.24, 120b ×2.00~~ (gains of the fast mode)	×0.94 and ×1.28 against v1.2, which the disk stage reads at the same speed (G1, G2)	Chapters 10 and 11
~~cause: the system's file cache~~	impossible: the engine reads without buffering	Chapter 11
~~it is the state of the files, not the format~~	the pattern of read requests of the format, specific to this disk (reserve R1)	Chapter 11
~~with $\lambda = 0$ the tail falls too, against $\lambda = 1$~~	not reproduced outside R1; a 99.9th percentile of 2 550 positions rests on three positions (M2)	Chapter 10
~~$\lambda = 0$ distorts in a privileged direction~~	the direction of the perplexity reverses outside R1 (M3)	Chapter 10
~~at most 4.0 % of cells that flip~~ (guard)	true only for a homogeneous population; a counter-example of the judge holds 80/10 with 5.26 %	Chapter 12
~~only place~~ (spectra)	false: 11 matrices out of 48	Chapter 13
~~the misses are not correlated with the small weights~~ (P6)	they are: 35 % of misses at rank 0, 56 % at rank 3	Chapter 7

- **In the domain of the outputs, the mean of the error is not enough.** What counts is where the error falls: the synonymous mutation places it where the inputs do not look, the Hebbian where the energy is low; both have a low expert error, only the first wins in KL.
- **In the domain of the text, what costs is the flip.** An error is worth only its probability of making a later layer cross a boundary of the router; and one flip brings others, far from the boundaries, that the local margin does not see (Chapter 12).

For Stone's conviction, the answer is twofold, and both halves are measured. DNA as a *recipe* does not work on these weights: theory predicted it before the measurement, and the measurement confirmed it. DNA as a *redundant code*, where a synonym changes the sequence without changing the phenotype, is exactly what wins, three times. Living things had the right idea; it was not the one we expected of them.

14.4 What could be wrong, at the scale of the volume

What could be wrong

- **The samples.** The round measured on 12 reference tensors (layers 0, 8, 16, 23 of gpt-oss-20b) and often on 16 or 32 experts out of 768; the spectra on 48 matrices (2 %) of three layers; P5 on synthetic weights. A property true of these samples may not be true of another.
- **The text.** All the KLs are taken on wiki.test: R1, the calibration and the judge's new windows come from it. Ten windows of one corpus, whose positions are correlated (a design effect of 32 for an absolute KL); no code or dialogue text. Mimicry judges generation on two traces of 256 tokens, which the judges showed to be degenerate.
- **The protocols at 32 and at 768 experts** measure two different things (R3); a variant judged on 32 experts is not thereby judged: $\lambda = 1$ equals the frontier there, and beats it by 42 % at 768.

- **A single disk, a single machine** (reserve R1). Every speed figure of the volume holds for this NVMe, behind Intel RST VMD, under BitLocker, on a given day; the gain of the v1.2 format rests on the pattern of its requests on this disk, and the exact variable of the pattern is not isolated.
- **A single model, born in MXFP4.** gpt-oss has expert weights published in four bits; the volume says nothing of a model published in BF16, where the frontier and the noise are different.
- **The analogies.** The genome, the ribosome, the cingulate cortex, the spin glass oriented the research; none of them proved anything. A chapter that read an analogy as an argument would be mistaken, and the volume states them as such.
- **The figures not judged.** Pages 3 to 5 of the figures of the spectra (the lever, the ceiling) are not judged and are not placed; the explorations of the tropical lead after its verdict (the cascade roots, the memory of conflict) are presented as such; the second draw of P3 does not enter the volume (R6); the quality proxy of the 120b is validated on the 20b, not on the 120b.
- **The judges share conventions.** The judge of the construction has its own code, but the same KL convention (cut at $\log p > -16$) and the same engine, built from the same repository; a defect common to this convention or to this engine would not be seen. The second path for the logits remains the f64 reference of Volume II.
- **A judge can be wrong, and a judge was**: the "state of the file" attribution of the judge of the construction fell before the placement judge. Nothing guarantees that another reading will not come.

14.5 Future work

Ranked by expected gain, each with the measurement that will judge it.

1. Make v1.2 the default format, and isolate the pattern. Both files are already recoded and checked (Section 9.6); the move from v1.0 to v1.2 is the only speed lever measured end to end: ×1.30 (20b) to ×1.44 (120b) between two files of the same model on this disk, losslessly. The choice of the v1.2 files as default files, and the erasure of the v1.0 ones (57.7 GB and 11.2 GB), belong to Stone. The exact variable of the slow v1.0 pattern is to be sought at the disk stage, by patterns declared beforehand (the placement judge proposes four); a read lever can be measured end to end only once it is found.

2. The tail of the KL, or the flips. It carries most of the mean, and no mechanism of the volume targets it: the synonymous mutation lowers the smooth part (median divided by nearly three) much more than the tail (99.9th percentile against the frontier on R1: $-3.4\,\%$ for $\lambda = 1$, $-17.2\,\%$ for $\lambda = 0$); the tropical guard falls because the secondary flips arrive at wide margins. The next step that the measurements point to is the cascade: the guard catches 85.2 % of the cascade roots, an exploratory reading made after the verdict, to be declared as a criterion before any new measurement.

3. The synonymous mutation of the 120b. It needs the capture of the f64 reference on the 120b (36 layers, 128 experts), a coder on the card (the Python coder would require about 23 h), then the end-to-end KL against an f64 reference of the 120b on the ten windows, which does not exist yet. Predicted, not measured: around 0.07 of KL if the ratio of the 20b carries over.

4. The uses of the fast mode, redone with the right ratios. The mixed miss policy (serving a cold expert in fast mode, rereading a promoted expert in exact mode) is first to be simulated on the traces by the same code as the real cache, with $r(R/B)$ and not $r(R/A)$: ×1.28 in generation for a KL of 0.14 on the 120b, nothing in generation on the 20b. The speculative draft, an idea of Stone's ("un genre de puissance de raisonnement différent", *a different kind of reasoning power*), returns the text exact to the bit; but by the lemma of the synthesis, a draft read from the disk pays only if $\alpha > 1 - \rho + b_d/b_e$, and with independent routing the reuse ρ of experts between neighbouring positions is of the order of 0.03 on the 120b: the only resident draft of a 120b on this machine would be the 20b in fast mode, whose agreement with the 120b is not measured (Chapter 8).

5. Another corpus. Windows of code and dialogue, kept apart, for the KL of the synonymous genome and for B2; and, for mimicry, generations of the 120b that stop at the end token, the measurement that the judge of P1 asks for to decide.

6. What the round leaves. S^* combined with feedback through the full H (the prediction that would say why S^* does not win in KL); the bridge expert, whose best paralogue leaves 33 to 84 % of the output energy in error; the second draw of P3. None has a gain figure today.

Conclusion

Volume I had measured the machine and proved that, on it, everything comes down to bits read per weight. Volume II had built a program that serves a model four times larger than the RAM, exactly. This volume looked in living things for a way to read fewer bits, and it found three, all of the same family: a synonym changes the code without changing what counts. It also showed, with the same demands, what does not work and why: the weights of gpt-oss, once the scale is known, are noise; what costs the text is the flip of the router; and the speed gain found along the way belongs to the format and to this disk, not to the genome. Each verdict has its threshold declared beforehand, its control that can fail, and a judge that did not do the work. What remains is written above, in the order in which it would pay.

Appendix A

Protocols: declared thresholds and controls

Where we start, what this chapter establishes, what remains

At the start, the method of Volumes I and II: each measurement has its control, and every new control must be able to fail; a threshold derives from a named quantity and is declared before the measurement, by a dated commit; a cost is judged in ABBA series; every figure one depends on is redone by a second path; a refuted sentence is struck through. **This appendix establishes**, for Volume III, the list of declared thresholds with the quantity from which each derives and its verdict; the table, generated from git, of each threshold file and of the time of the commit that created it; the controls and their breaks; the amendments made before reading, and those made after, stated as such; the protocols of the judges. **What remains** is what git cannot prove: that no exploratory reading preceded a threshold (Section A.6). Sources: the threshold files of Table A.2, the deliverables and the judgements of the chapters, `memoire/tome3/JUGEMENT-figures3.md` (§ 4.4), `memoire/outils/tome3_cassures.py`.

A.1 What is inherited

The rules do not change from one volume to the next; they are set out in Volume I (appendix on the protocols) and applied to the program in Volume II (its appendix A). Volume III practises them more densely, because the quest for the genome declared, from 16 to 27 October 2026, thirty-five files of thresholds, criteria and amendments (Table A.2), often by agents working at the same time: the rule "the threshold is declared before the pass, and the time of the commit is authoritative" is the only one there that allows the order of events to be read. Three forms recur, which the volume names the same way everywhere:

- **success and death**: a mechanism has a success threshold and a death threshold; between the two, it is "inconclusive" or "open", and says so (the triage of mimicry declares them this way for its six leads);
- **the rule of paired differences**: a variant b is better than a variant a if $\bar{d} + 2\sigma < 0$, $d = \mathrm{KL}_b - \mathrm{KL}_a$ position by position, σ the standard deviation by windows; equal if $|\bar{d}| \leq 2\sigma$; worse otherwise (rule of lot R, Volume II; Section 11.5.3 for the reason for σ by windows);
- **the announced gain**: a median speed ratio is announced only if it exceeds $1 + 2\sigma$, σ the relative noise of a file against itself in the same session; the judge of the construction tightened the rule to σ = max over the files of the session (Section 11.6.2).

A.2 The declared thresholds, step by step

Table A.1 gives the criteria that decide a verdict of the volume, each with the quantity from which it derives; Table A.2 gives, for each file of thresholds, criteria or amendment, the commit that created it. The verdict is that of the judge of the step when there is one.

Table A.1: The criteria declared before their measurement that decide a verdict of Volume III, the quantity from which each derives, and the verdict (that of the judge when there is one).

Step	Declared criterion	Where it derives from	Verdict
The round (ch. 4 to 6, 8)			
genome, W	the strand reads fewer bits per weight than the base of B at equal weight error, on each of the 12 tensors	the base of B: 2.644 1 bits at 4.29 % relative error	holds (12 out of 12, −0.319 bit); Chapter 4
genome, O and F	the *output* error of a layer of experts below that of the base	the output is what the model reads	fall; Chapter 4
genome, S*	declared before its pass: output below the base on the 8 whole experts	idem	holds (8 out of 8); KL equal to the base on K10
duplication, D1	bits of the child conditioned on the parent below A_ctx on each of the matrices and overall	the lossless frontier, 3.843 1 bits	does not work (net gain 0.001 9 bit, 21 matrices out of 48); Chapter 5
Mimicry (ch. 7): success and death, `mesures/mimetisme/tri/00-seuils-declares.md`			
P1	half-life LFU − LRU ≥ +0.02 of hits on both generations; death below +0.005	0.02 out of 0.2621 of misses in LRU, that is ×1.083 if the disk bounds everything	open (judge P1)
P2	disk share −15 % at $R = 336$ and copies ≤ 25 % of the token; death below 5 %	half of the simulated −29.6 %, judged too optimistic	open
P3	misses −10 % for an excess KL ≤ 0.038; death beyond 0.07	the slope of the fast mode, 0.13 of KL for −34 % of bytes	at the limit (what is judged; R6)
P4	simulated gain ≥ +5 % for ≤ 5 % more bytes; death below +2 %	three quarters of the ×1.07 computed under the ceiling ×1.21	open, rather falls
P5	held-out error −10 % against Babai on 6 experts out of 8; death below 3 %	half the gain of the synonymous mutation (−0.0098)	open
P6	$r(\theta) \leq 0.7$ at $\theta = 0.20$ for ≥ 8 % of bytes; death beyond 0.9	half of the proportional KL of all-fast	open; B holds at step 0
The construction (ch. 9 to 11)			
lot A, A-C1	0 differing logit fingerprints out of 5 120 steps	proposition A1 (exactness by construction)	holds (lot A, judge)
lot A, A-C2	gain ≥ 0.12 bit per expert weight	twice the measured share of the −0	holds (0.127 91, 0.125 62)
lot A, A-C3	no regression: $r \geq 1 - 2\sigma$	noise of the session	holds; gain ×1.44 (R1)
lot B, B2 (i)	synonym better than the frontier: $\bar{d} + 2\sigma < 0$	rule of lot R	holds (3.8 σ)
lot B, B2 of the λ	$\lambda = 0$ kept if $\bar{d} + 2\sigma < 0$ against $\lambda = 1$, otherwise $\lambda = 1$	declared after B0, before the $\lambda = 0$ coding	holds (2.09 σ; judge 3.6 σ)
lot B, S3-K	KL(fast f64 ‖ fast engine) ≤ 0.011, first choice ≥ 95 %	3× the C4 gap of the exact engine (0.003 68)	holds (three variants; judge)
lot B, S3-V	gain announced if $B/A > 1 + 2\sigma_A$	noise of the A passes	announced, then struck through as a gain of the mode (G1, G2)

Step	Declared criterion	Where it derives from	Verdict
judge, J-V2 and J-V3	gain of the mode if $r(R/B) \geq g\,(1-2\sigma)$, $\sigma = \max$	lower bound of a ratio of medians	fall (1.284 against 1.769; 0.942 against 1.227)
judge, J-K2	B2 redone outside R1, same rule; prediction "equal"	$\Phi(z-2) \approx 0.54$	holds, strengthened; prediction refuted
The leads (ch. 12 and 13)			
tropical	the guard is worth building if it catches $\geq 80\,\%$ of the flips for $\leq 10\,\%$ of the cells	specification of the synthesis; written prediction: 55 to 65 %	falls (57.1 %; judge)
spectra	power law if the method of Clauset, Shalizi and Newman prefers it to the log-normal, on at least 2/3 of the sample	32 matrices out of 48	amended before reading (without power); 0 out of 48, 9 compatible (judge)

The dated table. Table A.2 is generated by `memoire/outils/tome3_annexes.py` from `git log`: for each file, the commit that created it and its time, then the number of commits that touched it (a dated amendment raises this number; it can be read in the file, with its time and the reason). It is sorted by creation time. The second draw of P3 (files `07*`), in progress in another session, is not in it (reserve R6).

Table A.2: The files of thresholds, criteria and amendments declared before a measurement, with the commit that created them (local time) and the number of commits that touched them (`memoire/outils/tome3_annexes.py`).

Declared before	Created by commit	Commits
`mesures/genome/genome/00-seuils-declares.md`	`e777d6d`, 2026-10-16 21:55	11
`mesures/genome/duplication/00-seuils-declares.md`	`40d4b5c`, 2026-10-16 21:58	3
`mesures/genome/construction/synonyme/00-seuils-declares.md`	`78ce4ed`, 2026-10-17 08:19	7
`genome/pistes/tropicale/seuil-declare.md`	`69260f1`, 2026-10-21 23:22	1
`genome/pistes/spectres/seuil.md`	`a171622`, 2026-10-21 23:22	13
`mesures/genome/construction/juge/00-seuils-declares.md`	`1f5fe0f`, 2026-10-22 03:53	1
`mesures/disque/placement/juge/00-seuils-declares.md`	`c34d836`, 2026-10-22 12:44	6
`mesures/pistes/spectres/puissance-critere.json`	`b9e9ea4`, 2026-10-22 13:49	1
`genome/pistes/tropicale/seuil-declare-indices.md`	`50bf472`, 2026-10-26 23:41	1
`genome/mimetisme/terre_cosmos/seuil-declare.md`	`841e914`, 2026-10-26 23:52	3
`mesures/mimetisme/humain/00-seuils-declares.md`	`9055b23`, 2026-10-26 23:53	3
`mesures/mimetisme/physique/00-seuils-declares.md`	`59eb5a2`, 2026-10-26 23:54	4
`mesures/mimetisme/mathematiques/00-seuils-declares.md`	`2017fcf`, 2026-10-26 23:54	3

Declared before	Created by commit	Commits
genome/mimetisme/vivant/seuil-declare.md	b7ace2d, 2026-10-26 23:54	9
genome/mimetisme/ingenierie/seuils-declares.md	2a4c60e, 2026-10-26 23:56	3
mesures/mimetisme/tri/00-seuils-declares.md	ec124f5, 2026-10-27 00:43	1
genome/pistes/spectres/juge-seuil.md	a52beeb, 2026-10-27 00:47	1
genome/mimetisme/P2/seuil-declare.md	6d325d8, 2026-10-27 00:47	1
mesures/mimetisme/P5/00-seuil-declare.md	ab92f4e, 2026-10-27 00:49	4
mesures/mimetisme/P4/00-seuils-declares.md	47339c2, 2026-10-27 00:50	3
mesures/mimetisme/P6/00-seuils-declares.md	4d2b915, 2026-10-27 00:50	2
mesures/mimetisme/P3/00-seuils-declares.md	1606f9f, 2026-10-27 00:51	1
genome/mimetisme/P2/seuil-declare-selectif.md	7146816, 2026-10-27 01:08	1
genome/mimetisme/P2/juge-seuil.md	122b81d, 2026-10-27 01:16	1
mesures/mimetisme/P3/01-affinage-declare.md	174b4e2, 2026-10-27 01:20	1
genome/mimetisme/P1/posthoc-declare.md	00093d8, 2026-10-27 01:25	2
mesures/mimetisme/P6/juge/00-seuils-du-juge.md	aa1f5be, 2026-10-27 01:26	1
mesures/mimetisme/P4/juge/00-seuil-du-juge.md	82c49a6, 2026-10-27 01:29	1
mesures/mimetisme/P3/juge/00-criteres-du-juge.md	c23e3db, 2026-10-27 01:32	1
genome/mimetisme/P1/juge/criteres.md	238965b, 2026-10-27 01:34	1
mesures/mimetisme/P5/juge/00-seuil-juge.md	c5d8c30, 2026-10-27 01:55	2
mesures/mimetisme/P3/02-etage-B-declare.md	c4a037a, 2026-10-27 05:48	1
mesures/mimetisme/P3/03-critere-marge-declare.md	e8bf954, 2026-10-27 08:06	1
mesures/mimetisme/P3/05-temoin-b0.3-declare.md	3a3713e, 2026-10-27 10:06	1
mesures/mimetisme/P3/06-routeur-ligne-declare.md	0f38124, 2026-10-27 10:31	1

A.3 The controls and their breaks

Table A.3 gathers the controls that carry a verdict of the volume. Each has an answer known in advance, and a break that made it fall: without one, the control proves nothing.

Table A.3: The controls that carry a verdict of Volume III: the answer known in advance, the break, and the result [meas.].

Control	Answer known in advance	Break	Result
scout, T1	with $H = I$, the synonym returns the frontier (0 differing symbols)	the real H	1 789 132 differing symbols

Control	Answer known in advance	Break	Result
lot B, W1	redoing L00-E00 returns the scout's file byte for byte	$\delta = 0.99$	117 501 differing symbols
lot B, W2	second path equal to the log to within 10^{-9} relative	one class bit on one row	$7.2 \cdot 10^{-3}$
lot B, substitutions	substituted = read	the scout's pass (32 out of 731)	holds; control first declared wrongly, struck through
lot B, `gemv_base`	equal to the f64 to the bit on an exact case	the exact mode on the same bytes	0 rows out of 2 880; falls
lot B, S3-T1	768 experts equal to the `.sym`, to the scales, to the fingerprints	`--casser C X`	the targeted expert, and it alone
lot B, S3-K	fast engine = fast f64	*frontier* f64 against synonym engine	0.190 and 0.149: falls
judge, J-T1	v1.0: decoded = GGUF; changed v1.0 → v1.2 = the −0	stream word, canonicalisation removed, bias bit	40 out of 40; three breaks fall
judge, J-C1	v1.2 evaluated twice: identical CSVs	bias flipped on layer 0	512 out of 512 differing
judge, KL	its KL = `juge kl` to within 10^{-6}	Q shifted by one position	$1.1 \cdot 10^{-8}$; 1.634
judge, windows	each merged window equal to its source	a shifted window	falls
tropical, T2	routing of A imposed on C: 0 flips	—	0; break 1
tropical, T3	permuted margins: the guard at chance	—	10.0 %; oracles 100 %
spectra, judge	the judge's controls	its break	falls on both targeted controls
appendices, `tome 3_annexes.py`	every file cited by a "Data" row exists	a non-existent file added	falls (code 1)

The guards of the figures. The data tools of the figures (`memoire/outils/tome3_*.py`) stop at their first guard; the judge of the third pass of the figures forced nine of them to false by its own means, in a copy, and all of them stopped their script (`memoire/tome3/JUGEMENT-figures3.md`, § 4.4): GN2 (III-8.5, a rate of the real codec changed), GN1 (III-10.1, 767 experts decoded instead of 768), G7 (III-9.6, the frame rule), J5 and J6 (III-8.6 bis), the companion (III-8.6 bis), $j = 11$ (the row of III-9.1), E1 (III-9.7), the second spike (III-7.4, where it is the compilation that stops) and DN7 (III-4.8). `memoire/outils/tome3_cassures.py` replays four of these breaks; it has no place in a build, since it forces the guards to false.

A.4 The amendments

A threshold may change: before the reading, with its reason and its time; after, never without saying so. The volume counts three kinds.

Before the reading, because the instrument could not answer. The spectra lead amended its main verdict on 21 October (§ 4 bis of `genome/pistes/spectres/seuil.md`, commit `bfc1c8c`), before any reading of the real spectra: a trial of the instrument on 40 draws of true Paretos ($n = 2\,880$, $\alpha = 2.5$ and 3.5) had returned all of them "not settled" against the log-normal. The criterion had no power; the lead's judge has since shown that it is impossible to satisfy at any size (Chapter 13). The main verdict is reported as declared, next to the amended verdict.

Before the measurement, because an earlier measurement called for it. Lot B declared the rule for settling the two λ at 12:15 on 17 October, after B0 and before the $\lambda = 0$ coding (Section 10.5.4), then the calm rules of session s2 on the 21st at 23:59, before any pass. P1 declared its post hoc analyses before computing them (`genome/mimetisme/P1/posthoc-declare.md`); P2 its selective demotion, a post hoc hypothesis stated as such, before opening the 20b (`seuil-declare-selectif.md`); P4 an amendment to the reading of the protocol, before any launch; P5 three addenda, each before its launch; P6 amendment A1, before any simulation (Chapter 7).

After a reading, stated as such. Two controls of lot B were corrected after the pass that had made them fall without cause: the substitution counter ("768" become "substituted = read") and the tolerance of the recomputed KL (10^{-12} absolute become 10^{-9} relative). Neither decides a verdict; both are struck through in place, with the time (Section 10.5.1). The margin-weighted criterion of P3 was declared after the reading of $N = 146$ and before any new pass, which its file says in its title.

A.5 The protocols of the judges

Each step of the volume had its judge, who had built nothing of what it judged. Their protocol is the same everywhere:

1. declare its thresholds before any measurement, with a prediction (the judge of the construction: "equal" for B2, ×1.38 for $r(R/B)$; both refuted, and published as such);
2. redo each published figure by a path of its own (another decoder, another KL, another text, another experimental design), without reading the judged code before having written its own;
3. by default, *a figure not reproduced does not hold*; a figure reproduced but wrongly attributed is struck through as an attribution and kept as a measurement;
4. read the deliverable sentence by sentence against its logs, and classify what does not hold into serious findings (which change what the project would do next) and minor ones;
5. carry the strike-throughs into the judged deliverable, without removing anything, and write its own "What could be wrong" section.

A judge's judgement can itself fall: that of the construction attributed the gap of the disk stage to "the physical state of the files"; the judge of the disk placement overturned it on 27 October, and the judgement carries the correction at every place (reserve R1, Section 11.6.5).

A.6 What could be wrong

What could be wrong

- **Git dates a commit, not a reading.** That the threshold is committed before the first log does not prove that no exploratory reading preceded it; the judge of the tropical lead says so of its own lead (the 222 lines of code committed 74 seconds after the threshold were written before it; writing a reader is not reading).
- **The times written in the notes** do not always agree with git, by a few minutes; the time of the commit is authoritative.
- **The table of criteria is a selection**: it keeps those that decide a verdict of the volume; the threshold files contain others (second-rank controls, reading criteria without a verdict), which can be read in the files of Table A.2.
- **The verdicts of mimicry** are those of the judges of each lead, reported by Chapter 7; the

second draw of P3, in progress elsewhere, may change its own.

- **The dated table** sees only the files whose name says “seuil”, “critère” or “déclaré” (*threshold*, *criterion*, *declared*); a threshold written in another file (a specification, a deliverable) escapes it.

Appendix B

Tools

Where we start, what this chapter establishes, what remains

At the start, the appendix on the tools of Volumes I and II: each tool sorted by study or by lot, with the first sentence of its own description, and the chain that builds the thesis. **This appendix establishes** the same for Volume III: the chain that builds it and what it checks; each committed tool of the quest, of mimicry, of the leads, of the judges of the figures and of the thesis, sorted by chapter; each figure in the style of the volume and the files it reads; the licences of the dependencies. **What remains** ~~is to put the tool of the appendices into the tome3 target of construire.sh~~: nothing, for the chain; the tool of the appendices has been there since 2026-10-29 (commit 57db4ce), and it reads the commit rather than the working tree since commit e5ce266 (Section B.1).

B.1 The chain of Volume III

Volume III is built like the first two, by the same script and the same `latexmk` configuration (`memoire/construire.sh`, `memoire/latexmkrc`), from `memoire/`:

- `./construire.sh tome3` (or `tome3-fr`, `tome3-en`) first runs the eight data tools of the figures, one per lot, under `memoire/outils/`: `tome3_` followed by `eclaireur`, `genome`, `duplication`, `synthese`, `codon`, `synonyme`, `juge` or `latents`, then `.py`. Each reads the committed logs, writes its CSVs and its numbers under `memoire/figures/tome3/donnees/`, and stops at its first guard; their regeneration gives the committed files again to the bit (`memoire/tome3/JUGEMENT-figures3.md`, § 4.3).
- Next, it generates the four tables of appendices A, B and C by `memoire/outils/tome3_annexes.py` (Tables A.2, B.1, B.2 and C.1), from the ~~commit; it refuses to run if a file it counts differs from the commit~~ publication commit of the volume, `4b071d0` (paragraph “The tool of the appendices”, below).
- It then compiles, separately, the figures of the spectra lead: `genome/pistes/spectres/figures/spectres-figures.tex` is a standalone document whose data paths are relative to its folder, which is read-only; it is therefore compiled in that folder, with its output under `memoire/tome3/pistes/` (ignored by git), where the chapter on the spectra takes its pages.
- It finally builds the two PDFs by `latexmk` (lualatex, biber) in `memoire/tome3/fr/` and `memoire/tome3/en/`, and stops with an error on any undefined reference or citation and on any missing data file.

Two tools have no place in a build, and are not in it: `memoire/outils/tome3_palette.py`, which rewrites its files with other line endings (equal in content only), and `memoire/outils/tome3_cassures.py`, which forces the guards to false (appendix A).

The tool of the appendices. `memoire/outils/tome3_annexes.py` generates the four tables of appendices A, B and C from the ~~HEAD commit~~ publication commit of the volume (`git ls-tree`, `git cat-file`, `git log`): the tools, the figures and their data, the log folders, the dated threshold files. Its control: every file cited by the "Data" row of a figure (full path, patterns `*` and `{a,b}` allowed) exists in the repository; its break (`--casser`, a non-existent file added to the first figure) makes it fall, code 1 [meas.]. ~~It is not yet in the tome3 target of construire.sh, which this writer was not allowed to modify~~; it has been there since 2026-10-29 (commit 57db4ce): the `tome3` target of `memoire/construire.sh` runs it before latexmk, and each build of the volume rewrites its tables from the repository. Corrected on 2026-10-29 (commit e5ce266, grave G3 of `memoire/tome3/JUGEMENT-final.md`): it counted the lines and the bytes in the working tree, so that an uncommitted change from another session gave there 124 lines to `outils/mimetisme/humain/e4b_defauts.py`, which has 122 in the commit, and the sizes of appendix C depended on the line endings of the copy (`mesures/genome/construction/codon/`: 2.47 MB in one copy, 2.43 MB in the commit). It now counts the lines on the content of the git objects and the sizes on their size, ~~and it refuses to run, code 1, if a file it counts differs from the commit in the working tree~~ [meas.]. *Pinned on 2026-10-29* to the publication commit of the volume, `4b071d0`, like the tools of Volumes I and II: read at the HEAD commit, it refused to run as soon as another session left a counted file uncommitted; read at this commit, it gives back the four tables bit for bit, and neither the working tree nor later commits enter them any more.

B.2 The committed tools

Every number [meas.] or [calc.] of this volume comes out of a committed tool. Table B.1 lists them, sorted by chapter, each with the first line of its own description, quoted as written (in French), and its number of lines. The tools of the quest, of mimicry and of the leads belong to their streams; the thesis cites them and does not modify them. Those of the second draw of P3, in progress in another session, are not in it (reserve R6).

Table B.1: The committed tools of Volume III, by chapter, each with the first line of its own description (`memoire/outils/tome3_annexes.py`).

Tool	Its own description (first line, quoted as written, in French)	Lines
Ch. 2 and 3: the question, the scout		
`outils/genome/eclaireur/bib_sources.py`	Écrit le BibTeX et le tableau des sources de l'éclaireur depuis les métadonnées primaires.	128
`outils/genome/eclaireur/borne_rd.py`	La borne distorsion-débit des poids MXFP4 de gpt-oss, dans le domaine des poids.	196
`outils/genome/eclaireur/capture/src/main.rs`	genome-capture : entrées réelles des experts de gpt-oss, et KL d'une base substituée.	237
`outils/genome/eclaireur/capture/src/modele.rs`	FOURCHE de moteur/crates/trc-reference/src/modele.rs (copie du 2026-10-16, commit 6dd7414) pour l'éclaireur du génome des poids (genome/01-eclaireur.md). …	770
`outils/genome/eclaireur/choix_delta.py`	Le choix de l'amortissement δ de la mutation synonyme, par validation croisée SUR LA CALIBRATION SEULE.	74
`outils/genome/eclaireur/diagnostic_calibration.py`	Diagnostic de la mutation synonyme : apprend-elle les entrées, ou seulement l'échantillon ?	67

Tool	Its own description	Lines
outils/genome/eclaireur/frontiere_csv.py	Place chaque mécanisme mesuré sur le plan débit × erreur (la figure de synthèse du mémoire).	65
outils/genome/eclaireur/passes.sh	Les passes de la mutation synonyme, dans l'ordre déclaré (genome/REPRISE-etat-de-l-art.md). …	23
outils/genome/eclaireur/synonyme.py	La mutation synonyme : une base de 2 bits choisie là où la sortie ne la voit pas.	575
outils/genome/eclaireur/zero-synonyme/src/main.rs	FOURCHE de outils/arbitre/decoupe (la mesure F1, copiée le 2026-10-16) pour l'éclaireur du génome des poids (genome/01-eclaireur.md § 4) …	844
outils/genome/eclaireur/zero_compte.py	Le codon synonyme du zéro, compté : combien de codes −0 et +0, et ce que coûte le signe d'un zéro.	86
Ch. 4: the genome of the weights (the recipe)		
outils/genome/recette-kl/src/main.rs	genome-recette-kl : les logits des fenêtres d'un .kld avec des experts substitués (base .sym ou recette .f32w) …	98
outils/genome/recette-kl/src/modele.rs	Copie (2026-10-16, sujet genome) de la fourche de l'éclaireur (outils/genome/eclaireur/capture/src/modele.rs), avec un seul ajout …	775
outils/genome/recette/analyse.py	Second chemin du verdict W : refait, depuis les CSV par rangée de genome codons, les courbes débit-distorsion par allocation lagrangienne et les bits à erreur …	142
outils/genome/recette/analyse_fidelite.py	Verdict F (fidélité de sortie) depuis le journal de genome fidelite.	67
outils/genome/recette/borne_par_tenseur.py	Ce qu'atteindrait un code gaussien idéal (débit-distorsion de Shannon) sur les rangées de l'échantillon, à l'erreur de la base de chaque tenseur …	33
outils/genome/recette/chaine.sh	Chaîne du sujet genome après les passes de poids : sortie des experts sur entrées réelles (et écriture des experts reconstruits), puis KL contre la référence …	30
outils/genome/recette/erreur_colonnes.py	Où tombe l'erreur de la recette : par colonne (canal d'entrée) de chaque matrice d'un expert.	53
outils/genome/recette/erreur_sortie_forme.py	La forme de l'erreur de sortie : quelle part est un simple changement de gain (parallèle à la sortie exacte), quelle part est orthogonale …	81
outils/genome/recette/figures_transformations.py	Données des figures « avant / après » du sujet genome (SOCLE, demande de Stone du 2026-10-17) :	94
outils/genome/recette/file.sh	File des passes du sujet genome, lancées l'une après l'autre (chacune prend tous les fils).	12
outils/genome/recette/graine_theorie.py	Ce que la recette graine + coefficients peut atteindre au mieux, calculé (sans quantifier les coefficients) …	39
outils/genome/recette/kl.sh	Passe K du sujet genome : la base et la recette S* (pondérée, exceptions 0,1 %, 2,64 bits) substituées aux experts 0 et 16 des couches 0, 8, 16, 23 dans la …	20
outils/genome/recette/kl10.sh	Passe K du sujet genome : la base et la recette S* (pondérée, exceptions 0,1 %, 2,64 bits) substituées aux experts 0 et 16 des couches 0, 8, 16, 23 dans la …	20
outils/genome/recette/src/base.rs	La frontière : la base de l'architecture B (signe + classe de magnitude parmi {0 ; 0,5} {1 ; 1,5 ; 2} {3 …	70

Tool	Its own description	Lines
outils/genome/recette/src/codons.rs	La lecture par codons glissants : le « ribosome » des poids.	406
outils/genome/recette/src/donnees.rs	Les poids réels : un expert d'un tenseur MXFP4 de gpt-oss, rangée par rangée, lu par le lecteur GGUF de trc-reference (chemin indépendant du .trc).	95
outils/genome/recette/src/fidelite.rs	La fidélité de sortie en fonction du débit : pour chaque expert, la courbe (bits par poids, erreur relative de la sortie de l'expert sur des entrées réelles) …	336
outils/genome/recette/src/graine.rs	La recette littérale : un bloc de C poids = U(graine) · t, U une matrice C × P tirée d'un générateur pseudo-aléatoire par sa graine de K bits, t P coefficients …	291
outils/genome/recette/src/hadamard.rs	Rotation de Hadamard aléatoire par morceaux, le long des colonnes d'une rangée.	74
outils/genome/recette/src/main.rs	genome : mesures du génome des poids sur gpt-oss (sujet genome). Processeur seul.	42
outils/genome/recette/src/mesure.rs	Les mesures. Chaque sous-commande écrit un journal JSON sur la sortie standard.	832
outils/genome/recette/src/motifs.rs	Les motifs partagés, combinés : un bloc de d poids = somme de M motifs, chacun tiré d'un dictionnaire de 2^b motifs partagé par tous les experts d'un tenseur …	74
outils/genome/recette/src/sortie.rs	L'erreur sur la sortie des experts, pour des entrées réelles.	433
outils/genome/recette/structure.py	Où est le bruit, où est la structure : trois mesures sur les experts MXFP4 réels de gpt-oss-20b.	135
outils/genome/recette/vers_csv.py	Les CSV des figures du mémoire (sujet genome), régénérés depuis les journaux commités.	76
Ch. 5: gene duplication		
outils/genome/duplication/ecran_parente.py	Écran de parenté : chaque neurone d'un expert a-t-il un « gène parent » ailleurs dans le modèle ?	146
outils/genome/duplication/experts.py	Accès aux experts MXFP4 de gpt-oss pour l'étude de la duplication (génome des poids).	101
outils/genome/duplication/figure_phylogenie.py	Données de la figure « l'arbre phylogénétique des experts » (SOCLE : espaces latents, 2026-10-17).	88
outils/genome/duplication/lancer_echantillon.sh	L'échantillon de D1, D2, D2b, D2c, T, O (seuils : mesures/genome/duplication/00-seuils-declares.md). …	11
outils/genome/duplication/mutation.py	Un expert codé comme mutation de son parent : choix du parent, coût sans perte (D1), coût avec perte contre le même quantificateur sans parent (D2), mutation …	341
outils/genome/duplication/paires_arbre.py	Parenté d'expert à expert à une permutation des neurones près, et l'arbre de parenté le plus économe.	183
outils/genome/duplication/paralogues.py	Les paralogues : un expert absent peut-il être remplacé par un autre expert de sa couche ?	140
outils/genome/duplication/spectre_famille.py	La famille de gènes : les 32 experts d'une couche partagent-ils un sous-espace (un « domaine » commun) ?	111
outils/genome/duplication/synthese.py	Synthèse de l'échantillon de la duplication : CSV pour le mémoire et verdicts contre les seuils déclarés.	143
Ch. 6: predictive coding and pruning		

Tool	Its own description	Lines
outils/genome/predictif-elagage/contextes.py	Codage prédictif sans perte des codes MXFP4 : ce que chaque prédicteur retire de l'entropie.	176
outils/genome/predictif-elagage/contextes_temoin.py	Recalcule le témoin « gauche permutée » de contextes.py avec la formule de biais corrigée.	22
outils/genome/predictif-elagage/debit_distorsion.py	Débit-distorsion des codes MXFP4 réels : élagage synaptique et quantification prédictive bornée.	380
outils/genome/predictif-elagage/debit_hebb.py	Débit-distorsion à la sensibilité hebbienne : remplissage inverse sur (e, classe de ligne, classe de colonne).	151
outils/genome/predictif-elagage/debit_sensibilite.py	Débit-distorsion à la sensibilité mesurée : remplissage inverse sur (échelle e, activité de colonne q).	168
outils/genome/predictif-elagage/histo_hebb.py	Sensibilité hebbienne de chaque poids d'un expert : activité présynaptique × influence postsynaptique.	117
outils/genome/predictif-elagage/histo_sensibilite.py	Loi conjointe pondérée par la sensibilité mesurée : l'élagage synaptique dépendant de l'activité.	88
outils/genome/predictif-elagage/histogramme.py	Loi conjointe exacte (échelle e, δ de ligne, code) de chaque tenseur d'experts MXFP4.	59
outils/genome/predictif-elagage/kld_identiques.py	Témoin de la chaîne à poids altérés : les entrées .kld d'un plan sans perte en valeur sont-elles identiques, octet pour octet, à celles de la référence exacte, …	42
outils/genome/predictif-elagage/kld_tranche.py	Tranche un fichier .kld (format de llama-perplexity) : garde les fenêtres [a, b).	45
outils/genome/predictif-elagage/mxfp4.py	Lecture des experts MXFP4 de gpt-oss, matrice par matrice, pour l'étude predictif-elagage.	109
outils/genome/predictif-elagage/plans.py	Plans de reconstruction : pour une famille et une erreur visée, la table T[ctx][code] de chaque tenseur.	164
outils/genome/predictif-elagage/resume_csv.py	Table de synthèse de l'étude (CSV pour le mémoire), tirée des journaux commités, jamais de mémoire.	60
outils/genome/predictif-elagage/sonde/src/main.rs	La sonde de l'étude genome/predictif-elagage, sur la référence f64 de troncons (trc-reference).	254
outils/genome/predictif-elagage/sortie_experts.py	Erreur à la SORTIE des experts, sur les entrées réelles capturées par la référence f64.	186
outils/genome/predictif-elagage/taux_reel/src/bin/taux_plan.rs	Taux réel d'un plan avec perte (élagage, remplissage inverse) : les cellules de chaque poids sont codées par le rANS de l'étude 03, une table par (contexte du …	327
outils/genome/predictif-elagage/taux_reel/src/main.rs	Taux réel des codes MXFP4 de gpt-oss, codés par le rANS de l'étude 03 et décodés exactement.	308
Ch. 7: mimicry, the judges		
outils/mimetisme/P1/juge/juge_p1.py	P1, juge : simulateur indépendant de cache à règle de groupe (LRU, LFU à demi-vie, décroissance inversée).	219
outils/mimetisme/P1/juge/sondes_p1.py	P1, juge : sondes S1 (tête avant la fin de message), S3 (fenêtres froides) et cassure C1ter (post hoc, dite).	92
outils/mimetisme/P2/juge/a_budget_egal.py	P2, juge, contrôle hors seuil (dit comme tel) : l'inclusif à R <= V est dégénéré par construction …	38
outils/mimetisme/P2/juge/depouiller.py	P2, juge : confronte juge-brut.json aux seuils de genome/mimetisme/P2/juge-seuil.md (rend verdict-juge.json). Usage …	80
outils/mimetisme/P2/juge/juge_p2.py	P2, juge : mon chemin pour refaire le chiffre principal (seuils : genome/mimetisme/P2/juge-seuil.md).	240

Tool	Its own description	Lines
outils/mimetisme/P3/juge/N238/juge_N238.py	Juge de P3 à N = 238 : mon propre chemin, sans importer p3_*.py, le juge kl ni le lecteur du juge de N = 146.	344
outils/mimetisme/P3/juge/etageB_juge.py	Juge de l'étage B de P3 : un second lecteur des .kld et du journal de routage, écrit sans reprendre p3_etageB*.py.	277
outils/mimetisme/P3/juge/juge_p3.py	P3, juge indépendant : second chemin de l'étage A (routage biaisé vers les résidents du cache).	254
outils/mimetisme/P3/juge/routeur-ligne/juge_routeur_ligne.py	Juge du routeur biaisé EN LIGNE de P3 : mon propre chemin, sans importer p3_*.py, le juge kl, rejouer, ni le juge de N = 238.	575
outils/mimetisme/P4/juge/juge_generation.py	P4, juge : la jambe « génération » du verdict. Les deux traces de génération du 120b bouclent-elles, et que donne le prédicteur sur leur moitié de test ? Usage …	44
outils/mimetisme/P4/juge/juge_p4.py	P4, juge indépendant : refait précision, couverture, octets et gain du bras P par un second chemin.	319
outils/mimetisme/P5/juge/juge_p5.py	P5, juge indépendant : le gain d'évaluation de la descente cyclique (CD) contre Babai, par un chemin écrit de zéro.	244
outils/mimetisme/P5/juge/juge_pt.py	P5 juge, phase optionnelle (descriptive) : échange de répliques écrit de zéro en numpy, sur les poids de J1.	118
outils/mimetisme/P5/juge/lot_j5_pt.sh	J5 (demi-calibration) puis PT numpy descriptif, sequentiels (2 fils au plus).	16
outils/mimetisme/P5/juge/lot_juge.sh	Lot du juge P5 : J1 (graine 1), J2 (graine 2026), J3 (regimes), temoins T1-T3, lambda 0,2 (graine 1).	16
outils/mimetisme/P5/juge/resume_juge.py	P5 juge : agrège les journaux de juge_p5.py et les compare aux chiffres de P5 (gains-d1.csv, table-regime.csv). …	135
outils/mimetisme/P6/juge/juge_controles.py	Juge P6 : contrôles exploratoires (hors seuils déclarés, étiquetés tels dans JUGEMENT.md).	85
outils/mimetisme/P6/juge/juge_p6.py	Juge P6 : second chemin indépendant du chiffre principal (variante A) et de la variante B.	257
outils/mimetisme/P6/juge/juge_rang.py	Juge P6 : le mélange par couche (T2 et « les défauts ne sont pas corrélés aux petits poids ») garde le rang.	57
Ch. 7: mimicry, the six leads		
outils/mimetisme/P1/cache_p1.py	P1 : mémoire du cache à deux échelles (LFU à demi-vie), jugée en génération.	526
outils/mimetisme/P1/fenetres_p1.py	P1, analyse post hoc A : fenêtres froides de 256 jetons sur des textes cohérents (lectures du 120b). Déclarée avant calcul …	91
outils/mimetisme/P1/figures_p1.py	P1 : figures du mémoire, tirées des données commitées (hits-*.npz, fenetres-*.csv) et des traces extraites. Usage …	131
outils/mimetisme/P1/noyau-p1/src/main.rs	noyau-p1 : copie de outils/mimetisme/physique/noyau-cache (rôle éclaireur physique, 2026-10-26), adaptée pour P1 (rôle expérimentateur, 2026-10-27). …	229
outils/mimetisme/P1/rust_fenetres_p1.py	P1, second chemin des fenêtres froides : noyau-p1 rejoue LRU et LFU(h) sur des fenêtres de 256 jetons écrites en fichiers, comparées à fenetres-120b.csv …	33
outils/mimetisme/P1/rust_p1.py	P1, second chemin : noyau-p1 (Rust, copie de noyau-cache du rôle physique avec deux changements dits dans son en-tête) rejoue les politiques de cache_p1.py sur …	119

Tool	Its own description	Lines
outils/mimetisme/P2/cache_exclusif.py	P2, cache exclusif VRAM vers RAM : simulateur et bancs (mon implémentation, indépendante de cache2.py).	192
outils/mimetisme/P2/diag_t2_20b.py	P2 : diagnostic du témoin T2 sur le 20b (écart de l'exclusif au LRU de V + R, avec et sans épinglage).	19
outils/mimetisme/P2/explorations.py	P2, explorations POST-HOC (hors seuil déclaré, à lire comme des pistes, pas comme le verdict).	118
outils/mimetisme/P2/figures.py	P2 : figures tirées des CSV commités (aucune valeur saisie à la main). fig1 distances de pile LRU …	80
outils/mimetisme/P2/holdout20b.py	P2, test de la démotion sélective sur le 20b (seuil : genome/mimetisme/P2/seuil-declare-selectif.md). Usage …	62
outils/mimetisme/P2/net_selectif.py	P2 : temps de service des défauts par jeton, 120b, R = 336 : LRU seul, exclusif simple (s=0), sélectif (s=1). …	23
outils/mimetisme/P2/second_chemin.rs	P2, second chemin : cache LRU à deux étages, inclusif et exclusif, en Rust. Indépendant de cache2.py et de cache_exclusif.py …	172
outils/mimetisme/P2/verdict.py	P2 : lit temoins.csv, grille.csv, chaine.csv (et rust.csv s'il existe), vérifie les témoins déclarés dans genome/mimetisme/P2/seuil-declare.md, calcule le …	160
outils/mimetisme/P2/verdict_selectif.py	P2 : verdict de la démotion sélective sur le 20b au seuil déclaré (seuil-declare-selectif.md).	37
outils/mimetisme/P3/p3_affinage.py	P3, affinage post hoc de l'étage A (déclaré dans mesures/mimetisme/P3/01-affinage-declare.md avant ce lancement).	155
outils/mimetisme/P3/p3_cassures_reelles.py	P3 : les cassures des témoins W3 et W5 sur les données réelles (elles n'avaient été passées que sur les logits synthétiques). W3-hasard …	33
outils/mimetisme/P3/p3_etageB.py	P3 étage B : une passe de la référence f64 à routage imposé, sa RAM de pointe, sa KL (chemin 2).	101
outils/mimetisme/P3/p3_etageB_N238.py	P3 étage B à N = 238 : deux passes de la référence f64 à routage imposé (β_1 puis β_2), l'une après l'autre.	163
outils/mimetisme/P3/p3_etageB_N238_analyse.py	P3 étage B à N = 238 : second chemin (numpy depuis le CSV par position de juge kl) et verdict aux deux lectures de 03-critere-marge-declare.md.	92
outils/mimetisme/P3/p3_etageB_N238_lecteur.py	P3 étage B à N = 238 : troisième chemin. Relit les .kld avec le lecteur du juge (etageB_juge.py, écrit sans le code de l'étage B), recalcule KL, premier choix …	61
outils/mimetisme/P3/p3_etageB_chemin2bis.py	P3 étage B, second chemin du chiffre principal : KL moyenne, premier choix identique, KL cumulée par fenêtre et arrêt d'O'Brien-Fleming, recalculés en numpy …	30
outils/mimetisme/P3/p3_fichiers_B.py	P3 : les fichiers d'entrée de l'étage B (ensembles imposés à la référence f64), autour du franchissement de -10 % de défauts.	41
outils/mimetisme/P3/p3_fichiers_N238.py	P3 étage B à N = 238 : choix de beta_1 (premier pas de 0,001 qui retire au moins 10 % des défauts) et fichiers imposés.	34
outils/mimetisme/P3/p3_routage.py	P3 : routage conditionné par le cache, borné par la marge. Étage A (boucle ouverte), outils et témoins.	451

Tool	Its own description	Lines
outils/mimetisme/P3/p3_routeur_ligne.py	P3, routeur biaisé en ligne : pilote des passes d'évaluation du moteur (gpt-oss-20b, 10 fenêtres de R1, 238 emplacements).	171
outils/mimetisme/P3/p3_routeur_ligne_analyse.py	P3, routeur biaisé en ligne : analyse des passes d'évaluation du moteur et verdicts aux seuils de 06-routeur-ligne-declare.md.	207
outils/mimetisme/P3/p3_routeur_ligne_vitesse.py	P3, routeur biaisé en ligne : génération (jetons par seconde), séries ABBA (06-routeur-ligne-declare.md § 7).	120
outils/mimetisme/P3/p3_series.py	P3 : découpe affinage.csv en séries pour pgfplots (une par variante et budget, bêta positifs jusqu'à 1, triés), défauts évités en % du compte à bêta = 0.	16
outils/mimetisme/P4/p4.py	P4 : préchargement et rétention à confiance calibrée (prédiction conforme), qui cède à la demande.	314
outils/mimetisme/P4/p4_figures.py	P4 : figures et CSV du mémoire, tirés de mesures/mimetisme/P4/mesure-*.json (aucune valeur saisie à la main). Usage …	95
outils/mimetisme/P4/p4_mesure.py	P4 : mesure par session (calibration fendue, bras P et R, simulateur, second chemin, cassure).	187
outils/mimetisme/P4/p4_synthese.py	P4 : synthèse au seuil déclaré (mesures/mimetisme/P4/00-seuils-declares.md § 2) à partir des JSON de p4_mesure.py. Usage …	88
outils/mimetisme/P4/p4_temoins.py	P4 : témoins du simulateur et de la calibration (déclarés dans mesures/mimetisme/P4/00-seuils-declares.md § 3).	165
outils/mimetisme/P5/agreger_etape1.py	P5 etape 1 : agrege les journaux mesures/mimetisme/P5/etape1/L??-E??-d<delta>.json, applique la regle declaree (mesures/mimetisme/P5/00-seuil-declare.md + …	89
outils/mimetisme/P5/etape1.py	P5 etape 1 : le H reel des captures, des poids SYNTHETIQUES gaussiens sur la grille MXFP4.	225
outils/mimetisme/P5/etape2.py	P5 etape 2 : poids REELS (gpt-oss-20b), un expert a la fois (13 Mo de lecture : gate, up, down).	151
outils/mimetisme/P5/figures.py	P5 : figures vraies, tirees des donnees commitees (CSV ecrits a cote). Sorties : mesures/mimetisme/P5/figures/.	194
outils/mimetisme/P5/gibbs.py	P5 : l'estimateur de Gibbs. La mecanique statistique de l'apprentissage (Seung, Sompolinsky, Tishby 1992 …	66
outils/mimetisme/P5/lot_diag.sh	Exploration (addendum 2) : cible diagonale de l'amortissement, Babai et CD seulement. usage : lot_diag.sh CIBLE DAMP	11
outils/mimetisme/P5/lot_etape1.sh	Les 8 instances (couches 0,8,16,23 x experts 0,16) pour un amortissement donne. 2 fils, priorite normale. usage : lot_etape1.sh DAMP	11
outils/mimetisme/P5/lot_regime.sh	Addendum 3 : variation de N. usage : lot_regime.sh REGIME	11
outils/mimetisme/P5/p5core/src/lib.rs	P5 : noyau du recuit et de l'echange de repliques sur le choix des symboles synonymes.	335
outils/mimetisme/P5/p5lib.py	P5 : bibliotheque du recuit et de l'echange de repliques sur les symboles synonymes.	177
outils/mimetisme/P5/resume.py	Affiche les lignes d'un journal etape1 (une ou plusieurs).	13

Tool	Its own description	Lines
outils/mimetisme/P5/table_delta.py	P5 : l'erreur d'evaluation de chaque methode a chaque amortissement, rapportee a Babai deploye (delta = 1, lam = 0). …	31
outils/mimetisme/P5/table_diag.py	Addendum 2 : cible diagonale de l'amortissement. Erreur d'evaluation (Babai, CD) rapportee a Babai iso delta = 1.	32
outils/mimetisme/P5/table_regime.py	Addendum 3 : variation de N. Gain de CD contre Babai du meme regime, sur le test, par instance.	42
outils/mimetisme/P5/temoins.py	P5 : temoins W1, W2, W4 (sans fichier modele, sans capture). Seuils : mesures/mimetisme/P5/00-seuil-declare.md.	164
outils/mimetisme/P5/temoins_reglage.py	P5 : reglage du PT sur les instances ou la premiere passe (200 balayages, 8 temperatures, 0,03 a 1,5) n'a pas retrouve l'optimum de la force brute. …	32
outils/mimetisme/P6/analyses.py	P6 étape 0 : analyses exploratoires, hors verdict (étiquetées telles dans resultat.md). 1. …	118
outils/mimetisme/P6/etape0.py	P6 étape 0 : la mesure, après les témoins. Seuils : mesures/mimetisme/P6/00-seuils-declares.md. Usage …	287
outils/mimetisme/P6/etape0_B.py	P6 étape 0, variante B : préremplissage par lots, décision rapide/exact par masse agrégée. Seuils : mesures/mimetisme/P6/00-seuils-declares.md § 3. …	112
outils/mimetisme/P6/figures.py	Figures de P6 (étape 0), tirées de mesures/mimetisme/P6/*.json. Usage : python -I -X utf8 figures.py <racine du dépôt>	71
outils/mimetisme/P6/niveaux-rust/src/main.rs	Second chemin de P6, étape 0 : cache d'experts à niveaux (exact / rapide), écrit à part de tiers.py.	126
outils/mimetisme/P6/second_chemin.py	Second chemin du chiffre principal de P6 (étape 0) : le automate Rust à étiquettes contre tiers.py. Usage …	25
outils/mimetisme/P6/second_chemin_B.py	Second chemin de la variante B : boucles en dictionnaires, tri de Python, sans numpy dans le calcul de r. Usage …	33
outils/mimetisme/P6/temoins.py	P6 étape 0 : témoins T0, T1, T4 (et sa cassure), avant toute mesure. Usage : python -I -X utf8 temoins.py <dossier t120> <exe noyau-cache> <dossier_sortie> …	105
outils/mimetisme/P6/tiers.py	P6, étape 0 : cache d'experts à niveaux (exact / rapide) et portillon de poids de mélange.	207
Ch. 7: mimicry, the domain scouts		
outils/mimetisme/humain/e1_memoire.py	E1 : politique de cache « mémoire humaine » (Anderson et Schooler 1991, ACT-R) contre LRU, LFU et Belady.	162
outils/mimetisme/humain/e1b_demivie.py	E1b (POST HOC, non déclaré avant) : le LFU du dépôt (demi-vie 8) est-il mal réglé ? Balayage de la demi-vie, pooled et par genre, et meilleur réglage par genre …	25
outils/mimetisme/humain/e2_triage.py	E2 : tri par poids de mélange (triage) des défauts de cache. Proxy de masse, pas de qualité.	132
outils/mimetisme/humain/e345.py	E3 (allocation marginale par couche), E4 (précédent inter-couches), E5 (classes ABC). Seuils et témoins déclarés avant l'écriture de cet outil …	289
outils/mimetisme/humain/e4b_defauts.py	E4b (POST HOC, non déclaré avant) : le prédicteur « précédent » de E4 (régime en ligne) couvre-t-il les DÉFAUTS de cache ?	122

Tool	Its own description	Lines
outils/mimetisme/humain/e6_sequentiel.py	E6 : arrêt séquentiel de groupe (forme O'Brien-Fleming) rejoué sur les séries de KL par fenêtre du juge. Seuils et témoins déclarés avant …	76
outils/mimetisme/humain/e7_marge.py	E7 (POST HOC, descriptif, sans critère déclaré) : borne haute des défauts qu'un routage conscient du cache pourrait éviter en échangeant le 4e expert contre le …	66
outils/mimetisme/ingenierie/cache2.py	Simulateur de cache d'experts à un ou deux étages, sur les traces de routage du 120b.	322
outils/mimetisme/mathematiques/m1_m2.py	Éclaireur « mathématiques et théorie de l'information » : mesures M1 et M2.	320
outils/mimetisme/mathematiques/maslov_jouet.py	Calcul jouet (pas une mesure sur le modèle) : lissage de Maslov d'une décision à seuil.	22
outils/mimetisme/mathematiques/plafond_speculatif.py	Calcul (pas une mesure) : gain d'octets du décodage spéculatif à brouillon d'acceptation alpha par jeton, long de gamma jetons, de coût relatif c (octets lus …	23
outils/mimetisme/mathematiques/t2bis.py	Témoin T2bis (amendement du 2026-10-26) : PB avec le prédicteur appris, relevé par l'indicateur vrai du prochain jeton. Seuil déclaré …	19
outils/mimetisme/physique/e1_e2.py	E1 (noyaux de mémoire du cache) et E2 (nombres d'occupation) : seuils dans mesures/mimetisme/physique/00-seuils-declares.md (commit 59eb5a2, avant toute …	188
outils/mimetisme/physique/e3.py	E3 : admission par seuil de score (BYPASS) contre LFU ordinaire à capacité égale de pointe. Seuil …	66
outils/mimetisme/physique/noyau-cache/src/main.rs	Cache global d'experts à score par noyau de mémoire (rôle éclaireur physique, 2026-10-26).	213
outils/mimetisme/terre_cosmos/alloc_lfu.py	Post hoc, hors seuil déclaré : M-A avec une LFU (demi-vie 32 jetons) par couche au lieu d'un LRU. …	65
outils/mimetisme/terre_cosmos/cache_terre.py	Éclaireur Terre et cosmos : deux politiques de cache d'experts inspirées de la Terre, jugées sur les traces de routage commitées (seuils …	376
outils/mimetisme/terre_cosmos/eau_usage.py	Éclaireur Terre et cosmos : eau-filling par l'usage (addendum 2 de seuil-declare.md, commité avant).	172
outils/mimetisme/terre_cosmos/lfu_demivie.py	Post hoc, hors seuil déclaré : la LFU de la base (demi-vie 8 jetons) est-elle réglée au bon endroit ? Taux de succès moyen sur les quatre genres (cache vide au …	31
outils/mimetisme/terre_cosmos/phase_120b_reduit.py	M-C sur le 120b, version réduite (la passe complète coûte 150 s par cas et a été arrêtée après 3 cas sur 16) …	42
outils/mimetisme/terre_cosmos/phase_terre.py	Éclaireur Terre et cosmos : M-C, éviction par temps estimé jusqu'à la prochaine utilisation.	160
outils/mimetisme/terre_cosmos/t2_melange.py	Témoin T2 (cassure) déclaré dans seuil-declare.md : ordre des jetons mélangé, l'avance de M-B sur la LFU doit tomber dans +-0,5 point …	29
outils/mimetisme/vivant/ablation_demivie.py	Contrôle de E-A (non déclaré avant, dit tel) : le gain d'ACT-R contre LFU8 vient-il seulement d'une mémoire plus longue ?	55
outils/mimetisme/vivant/bancs_traces.py	Bancs de l'éclaireur du vivant, sur les traces de routage de gpt-oss (aucun fichier de modèle ouvert).	766
outils/mimetisme/vivant/casser_temoins.py	Cassures des témoins de bancs_traces.py : chaque témoin doit pouvoir tomber.	87

Tool	Its own description	Lines
outils/mimetisme/vivant/eviction_episodique.py	E-L : éviction pilotée par la mémoire épisodique (cosinus exact, pas de deux jetons). Seuil : seuil-declare.md (E-L). Usage …	105
outils/mimetisme/vivant/eviction_synergies.py	E-J : éviction pilotée par le prédicteur de synergies (pas de deux jetons). Seuil : seuil-declare.md (E-J). Usage …	146
outils/mimetisme/vivant/memoire_episodique.py	E-K : mémoire épisodique clairsemée (la mouche), cosinus exact. Seuil : seuil-declare.md (E-K). Usage : python -I memoire_episodique.py DOSSIER_TRACES	105
outils/mimetisme/vivant/poids_stats.py	Statistiques complémentaires de E-C : forme de la loi des poids de mélange (descriptif). Usage : python -I poids_stats.py DOSSIER	18
outils/mimetisme/vivant/predicteur_synergies.py	E-I : un prédicteur de synergies, appris sur d'autres genres, devine-t-il les experts du jeton suivant ? Seuil et témoins …	201
outils/mimetisme/vivant/residence_partielle.py	E-G : résidence partielle (« polymérase en pause »). Seuil et prédiction : seuil-declare.md, ajout du 2026-10-27.	47
outils/mimetisme/vivant/surprise_mode.py	E-H : le signal de surprise prédit-il où le mode rapide perd de la KL ? Seuil : seuil-declare.md (ajout E-H).	91
Ch. 8: the synthesis of the round		
outils/genome/synthese/frontiere.py	Synthèse de la ronde 1 du génome des poids : la frontière bits par poids contre KL sur le texte.	340
Ch. 9: the codon (lot A)		
outils/genome/construction/codon/analyse_codon.py	Analyse du lot A (codon) : tailles v1.2 contre v1.0, et verdicts du J1 ABBA.	175
outils/genome/construction/codon/copie_temoin.sh	Témoin de la disposition sur le disque (lot A, codon) : copie neuve du v1.0 du 120b, SHA-256 des deux fichiers par Get-FileHash (chemin indépendant), puis les …	16
outils/genome/construction/codon/lancer.ps1	Lance trc-codeur (cible du lot A, codon), sorties redirigées ; note l'alimentation (Win32_Battery.BatteryStatus …	22
outils/genome/construction/codon/pilote_codon.py	Pilote du lot A (codon synonyme du zéro, format .trc v1.2) : critère A-C1 (logits au bit) et jalon J1 refait sur le 120b en v1.2 contre v1.0 (A-C3 vitesse, …	316
outils/genome/construction/codon/recoder.sh	Lot A (codon) : recodage v1.2 et vérification C1 des deux modèles, dans l'ordre. 1. témoin d'identité …	40
outils/genome/construction/codon/recoder_temoin_120b.sh	Séance s2 du lot A (codon) : le v1.0 du 120b recodé aujourd'hui par trc-codeur (témoin d'identité du codeur modifié sur le 120b, et témoin de disposition pour …	17
outils/genome/construction/codon/tables_octets.py	Les fréquences des tables CODES_CTX lues dans les fichiers .trc, v1.0 et v1.2 (figure F2 du mémoire : la carte des 256 octets avant et après le fondu du zéro).	91
outils/genome/construction/codon/verrou.py	Le verrou de la carte pour le lot A (codon) : prendre, rendre, état.	46
Ch. 10: the synonymous genome (lot B)		
outils/genome/synonyme/apparier.py	Différence appariée de deux passes KL (lot B, génome synonyme ; règle déclarée dans mesures/genome/construction/synonyme/00-seuils-declares.md).	117

Tool	Its own description	Lines
`outils/genome/synonyme/coder_768.sh`	Lot B, pas S1 : la mutation synonyme (λ = 1, δ* = 1) et la base frontière des 768 experts de gpt-oss-20b, par l'outil de l'éclaireur tel quel …	46
`outils/genome/synonyme/donnees_figures.py`	Données des figures du lot B (génome synonyme) pour le mémoire : la transformation du synonyme vue dans la base propre de H (avant : frontière ; après …	97
`outils/genome/synonyme/evaluer_moteur.sh`	Lot B, S3-K et S3-KL : la KL du moteur (carte, f32) sur les 10 fenêtres de R1, sous le verrou de la carte et sur secteur …	22
`outils/genome/synonyme/kl_passe.sh`	Lot B, pas S2 : la KL de bout en bout d'un dossier de .sym substitués (768 experts), sur les 10 fenêtres de R1, par la référence f64 (genome-capture kl), puis …	20
`outils/genome/synonyme/mode_rapide_l0.sh`	Lot B, après B2 (λ = 0 meilleure à 768 experts, 2026-10-21) : le compagnon rapide de la variante λ = 0, sa vérification S3-T1 (et sa cassure), la KL du moteur …	37
`outils/genome/synonyme/pilote_vitesse.py`	Lot B, pas S3-V : la vitesse du mode rapide contre le mode exact sur gpt-oss-20b, séance ABBA.	183
`outils/genome/synonyme/proxy_120b.sh`	Lot B, 120b (frontière seulement ; seuils : 00-seuils-declares.md § 120b, déclarés avant) : T2 …	51
`outils/genome/synonyme/rapide/src/main.rs`	trc-rapide : le fichier compagnon du mode rapide (lot B, génome synonyme ; genome/05-synthese.md § 4.8, genome/07-genome-synonyme.md).	498
`outils/genome/synonyme/repere_llama.sh`	Lot B, critère B2 (ii) : le repère de l'outil de référence. La requantification llama.cpp b11515 des seuls tenseurs d'experts en Q2_0 (2,25 bits par poids, …	18
`outils/genome/synonyme/survie_kl.py`	Données de la figure F-S2 du lot B : la fonction de survie de la KL par position, P(KL > t), sur une grille logarithmique de t, pour plusieurs passes juge kl …	43
`outils/genome/synonyme/tableau_variantes.py`	Lot B : la table des variantes à 768 experts (figure F-S8), tirée des journaux commités.	33
`outils/genome/synonyme/verifier_sym.py`	Second chemin du lot B (génome synonyme) : relire les fichiers .sym et recalculer, sans le code qui les a écrits, ce que l'éclaireur a publié …	225
Ch. 11: the judge of the construction		
`outils/genome/construction/juge/construire.bat`	Juge de la construction : construit depuis main (arbre du depot) le moteur, juge et genome-capture dans la cible du juge, par la recette du SOCLE …	14
`outils/genome/construction/juge/decodeur_juge.py`	Le décodeur indépendant du juge de la construction : le format .trc (v1.0 et v1.2), lu et décodé depuis la seule description de decisions/ARCHITECTURE.md §§ …	705
`outils/genome/construction/juge/donnees_juge.py`	Les CSV du juge pour le mémoire (Tome III), tirés des journaux commités du juge.	85
`outils/genome/construction/juge/evaluer_juge.sh`	Juge de la construction : les passes troncons evaluer –flux sur les fenêtres neuves (J-C1 et J-K, chemin du moteur), sous le verrou de la carte et sur …	27
`outils/genome/construction/juge/fenetres_juge.py`	Les fenêtres neuves du juge : dix fenêtres de 512 jetons de wiki.test que les lots n'ont pas vues.	170
`outils/genome/construction/juge/kl_juge.py`	La KL du juge, calculée par son propre code depuis les fichiers .kld (format de llama-perplexity).	179

Tool	Its own description	Lines
outils/genome/construction/juge/logits_juge.py	Les logits au bit (J-C1) : comparaison des empreintes de logits de deux passes troncons evaluer, et la copie à biais retournés de la cassure.	105
outils/genome/construction/juge/reference_juge.sh	Juge de la construction : la référence f64 des fenêtres neuves, puis la référence f64 substituée (le chemin des lots, genome-capture kl –sub) pour λ = 0, la …	38
outils/genome/construction/juge/routage_juge.py	Les experts routés par les fenêtres neuves que R1 n'a jamais routés (J-K, « ce que je publie de plus »).	52
outils/genome/construction/juge/seance_juge.py	La séance de vitesse du juge : trois fichiers par modèle, ordre équilibré, machine calme relevée.	269
outils/genome/construction/juge/verrou_juge.py	Le verrou de la carte pour le juge de la construction : prendre, rafraîchir, rendre, état.	59
Ch. 12: tropical geometry		
outils/pistes/tropicale/cascades.py	Exploration, après le verdict (ne décide rien, seuil-declare.md § 5) : les cascades de bascules.	114
outils/pistes/tropicale/figures.py	Données des figures de la garde tropicale (Tome III), tirées des journaux commités, et la coupe réelle du routeur (régions top-4 et bande de garde) rendue par …	228
outils/pistes/tropicale/garde.py	La courbe de la garde tropicale : (part des cellules sous τ, part des bascules sous τ).	123
outils/pistes/tropicale/indices.py	La garde à indices multiples (mémoire du conflit) : protocole de genome/pistes/tropicale/seuil-declare-indices.md.	236
outils/pistes/tropicale/indices_complements.py	Compléments de la garde à indices multiples, APRÈS le verdict (étiquetés exploratoires).	99
outils/pistes/tropicale/juge/juge.py	Juge indépendant de la garde tropicale : le chiffre principal refait par un autre chemin.	263
outils/pistes/tropicale/juge/juge_classes.py	Juge : l'autre classe de modèle (probabilités par classes) refaite, et sa sensibilité aux ex aequo.	94
outils/pistes/tropicale/juge/juge_indices.py	Juge indépendant de la garde à indices multiples (la mémoire du conflit).	274
outils/pistes/tropicale/juge/juge_modele.py	Juge de la garde tropicale : deux contrôles du théorème du nombre de garde, par quadrature seule.	104
outils/pistes/tropicale/juge/juge_w4.py	Juge : le témoin W4 (M3 entraîné sur étiquettes mélangées) sur vingt graines.	49
outils/pistes/tropicale/lois.py	Ce que 05 § 3.1 dit non mesuré, lu sur les mêmes journaux (seuil-declare.md § 5) :	273
outils/pistes/tropicale/routage.py	Lecture des journaux de routage du lot R, et la cohabitation avec le flux qui mesure des vitesses.	114
outils/pistes/tropicale/second_chemin.py	Second chemin du point principal de la garde tropicale, indépendant du chemin numpy.	61
outils/pistes/tropicale/second_chemin_indices.py	Second chemin de la garde à indices multiples (seuil-declare-indices.md § 6).	137
outils/pistes/tropicale/second_chemin_racines.py	Second chemin, en Python pur (lecture de bascules.py, lot R), du chiffre exploratoire des racines …	52
outils/pistes/tropicale/temoins.py	Témoins T1, T2, T3 de la garde tropicale (genome/pistes/tropicale/seuil-declare.md § 4).	114

Tool	Its own description	Lines
outils/pistes/tropicale/theoreme.py	Le nombre de garde κ = σ / E[m] : forme close de la courbe de la garde sous deux hypothèses (marge exponentielle de taux λ, bascule si la perturbation …	94
outils/pistes/tropicale/verifier_chiffres.py	Relit les chiffres principaux de garde-tropicale.md contre les fichiers commités qui les portent. Chaque ligne …	54
Ch. 13: the spectra		
outils/pistes/spectres/ajuster.py	Piste spectres, pas 3 : ajustement CSN des 48 spectres réels et de leurs témoins T-perm.	72
outils/pistes/spectres/alpha_levier.py	Piste spectres : l'exposant $\hat{\alpha}$ prédit-il le levier de rang payé ? (exploratoire, NON déclaré avant)	56
outils/pistes/spectres/apres-chaine.sh	Piste spectres : lance le test de prédiction (seuil.md § 7 octies) une fois la chaîne levier3 normalise finie (« fin normalise » dans chaine.log), pour ne …	14
outils/pistes/spectres/chaine.sh	Piste spectres : enchaîne les calculs, sans jamais démarrer pendant une séance de vitesse. …	58
outils/pistes/spectres/commun.py	Piste spectres : ce que partagent les outils (priorité, garde du verrou, lecture des matrices).	152
outils/pistes/spectres/csn.py	Loi de puissance par la méthode de Clauset, Shalizi et Newman (arXiv 0706.1062), écrite de zéro.	144
outils/pistes/spectres/fig_plafond.py	Piste spectres : CSV des figures du plafond (P1 : plafond par rang ; P2 : $\hat{\alpha}$ contre levier payé).	66
outils/pistes/spectres/fig_rangee.py	Piste spectres : CSV de la figure P3 (d'où vient le plafond : échelles de rangée, artefact MP, structure).	56
outils/pistes/spectres/juge/comparer.py	Juge de la piste « spectres » : comparaison du chemin du juge au tableau publié (ajustements.out).	103
outils/pistes/spectres/juge/juge_csn.py	Juge de la piste « spectres » : le chiffre principal refait par un chemin propre.	344
outils/pistes/spectres/juge/juge_npz.py	Juge de la piste « spectres » : les spectres du npz viennent-ils bien du modèle ?	91
outils/pistes/spectres/levier.py	Piste spectres, pas 4 : le spectre ouvre-t-il un levier de compression ? (seuil.md § 7)	287
outils/pistes/spectres/levier_l3.py	Piste spectres : L3 v3, le quantificateur scalaire à tramage soustractif (seuil.md § 7 ter).	94
outils/pistes/spectres/normalise.py	Piste spectres : d'où vient la queue ? (mesure exploratoire, NON déclarée avant ; dite comme telle)	62
outils/pistes/spectres/plafond.py	Piste spectres : le plafond spectral (seuil.md § 7 quinquies), sur les valeurs singulières sauvées.	256
outils/pistes/spectres/plafond_rang.py	Piste spectres : le plafond par rang, offert ou payé (seuil.md § 7 septies).	140
outils/pistes/spectres/plafond_rangee.py	Piste spectres : le plafond au-delà des échelles de rangée (seuil.md § 7 nonies et § 7 decies).	167
outils/pistes/spectres/prediction.py	Piste spectres : l'exposant $\hat{\alpha}$ prédit-il le levier de rang payé ? (seuil.md § 7 octies)	133
outils/pistes/spectres/puissance_critere.py	Piste spectres : la puissance du critère déclaré, mesurée sur des lois de puissance vraies.	56
outils/pistes/spectres/rangee_graine.py	Piste spectres : stabilité de la référence « permuté par rangée » d'une graine à l'autre (seuil.md § 7 undecies).	64

Tool	Its own description	Lines
outils/pistes/spectres/rapport.py	Piste spectres : les chiffres du livrable, lus dans les journaux commités (rien de mémoire).	253
outils/pistes/spectres/second_chemin.py	Piste spectres : le second chemin du chiffre principal (seuil.md § 6).	186
outils/pistes/spectres/spectres_svd.py	Piste spectres, pas 2 : les valeurs singulières complètes de l'échantillon et de ses témoins.	86
outils/pistes/spectres/temoins.py	Piste spectres : les témoins de l'instrument (seuil.md § 5), et leur cassure.	60
Figures: latent spaces and figure judges		
outils/genome/tome3/juge-construction/lectures.py	Juge des figures du lot « construction » : les lectures (ce qui serait faux ou trop beau).	198
outils/genome/tome3/juge-construction/points.py	Juge des figures du lot « construction » : un point tiré au hasard par série tracée, contrôlé à la source.	296
outils/genome/tome3/juge-construction/recalcul.py	Juge des figures du lot « construction » du Tome III (ch. 8 à 10) : les nombres des légendes.	447
outils/genome/tome3/juge-construction/regenerer.py	Juge des figures du lot « construction » : régénérer les données des figures et les comparer au bit.	85
outils/genome/tome3/juge-design/palette.py	Juge de design du Tome III : distances entre les couleurs du préambule et des lots, en vision normale, deutéranope, protanope (Machado, Oliveira, Fernandes …	71
outils/genome/tome3/juge-design/planche.py	Planche : rend des pages d'un PDF, découpe la zone utile (fractions), les colle en grille. Usage …	42
outils/genome/tome3/juge-design/zoom.py	Rend une page d'un PDF à une résolution donnée et en découpe une zone (fractions de page). Usage …	34
outils/genome/tome3/juge-final/listing.py	Juge final : le listing du fondu (chapitre 08) contre les sources Rust commitées, par mon propre extrait (git show main:..., lignes 67 à 77 et 317 à 327), …	31
outils/genome/tome3/juge-final/parite_ef.py	Juge final : dans un fichier bilingue (\EF{anglais}{français}), compare les nombres et les renvois de chaque paire. Usage …	34
outils/genome/tome3/juge-final/parite_nombres.py	Juge final du Tome III : compare, fichier par fichier, les nombres du français et de l'anglais. Chemin propre au juge (aucun outil des traducteurs repris) …	199
outils/genome/tome3/juge-final/refaire_neufs.py	Juge final du Tome III : refait, par mon propre chemin (lecture directe des JSON et CSV commités), les chiffres neufs ou déplacés depuis …	211
outils/genome/tome3/juge-final/resumes.py	Juge final : les deux blocs du résumé (Résumé, Abstract) sont-ils les mêmes, au caractère près, dans fr/resume.tex et en/resume.tex ? Usage …	25
outils/genome/tome3/juge-final/typo_en.py	Juge final : typographie anglaise du Tome III (sources anglaises), relevé mécanique, hors commentaires. Relève : espace avant « …	28
outils/genome/tome3/juge-latents/juge_commun.py	Juge du lot « espaces latents » du Tome III : lectures communes, écrites de zéro.	150
outils/genome/tome3/juge-latents/juge_etats.py	Juge du lot « espaces latents » : les nombres de III-7.6 (l'espace latent des 6 132 états de la couche 23), refaits de zéro depuis la capture h-L23.f32, le …	227

Tool	Its own description	Lines
outils/genome/tome3/juge-latents/juge_etats_marges.py	Juge du lot « espaces latents », III-7.6 : ce que dit la taille des points.	76
outils/genome/tome3/juge-latents/juge_rangee_nuage.py	Juge du lot « espaces latents » : les nombres de III-9.1 (c) (la rangée) et de III-9.7 (le nuage), refaits de zéro depuis le GGUF, les .sym du lot B, les …	226
outils/genome/tome3/juge-latents/juge_regen.py	Juge du lot « espaces latents » : compare au bit les fichiers régénérés par les scripts du lot (dans une racine de brouillon, jamais dans le dépôt) aux …	109
outils/genome/tome3/juge-redaction/assembler.py	Juge de la rédaction du Tome III (français) : assemble le tome à juger dans un dossier d'essai.	83
outils/genome/tome3/juge-redaction/barres.py	Juge de la rédaction du Tome III (français) : les chiffres barrés dans les sources reviennent-ils, non barrés, dans le texte ?	112
outils/genome/tome3/juge-redaction/calculs.py	Juge de la rédaction du Tome III (français) : refait les calculs neufs des rédacteurs (étiquetés \calc, qu'aucune source n'avait posés), par un second chemin, …	67
outils/genome/tome3/juge-redaction/construire.sh	Juge de la rédaction du Tome III (français) : construit le tome assemblé par assembler.py, comme memoire/construire.sh tome3-fr le construit (même latexmkrc, …	39
outils/genome/tome3/juge-redaction/enonces.py	Juge de la rédaction du Tome III (français) : chaque énoncé (théorème, proposition, lemme, corollaire) est-il suivi de sa démonstration ? Rend, par chapitre, …	35
outils/genome/tome3/juge-redaction/journal.py	Juge de la rédaction du Tome III (français) : lit le journal de la dernière passe de lualatex du tome assemblé et rend, par fichier source, les boîtes trop …	45
outils/genome/tome3/juge-redaction/paralogues.py	Juge de la rédaction du Tome III (français) : refait la correction neuve du chapitre 4 (rédacteur A), « substituer n'est jamais meilleur que retirer l'expert » …	36
outils/genome/tome3/juge-redaction/refaire.py	Juge de la rédaction du Tome III (français) : refait, par un chemin à moi, chaque nombre tiré par tirage.py (graine 20261029, journaux/tirage.csv), depuis le …	247
outils/genome/tome3/juge-redaction/statique.py	Juge de la rédaction du Tome III (français) : contrôles statiques du tome assemblé, sans compiler.	183
outils/genome/tome3/juge-redaction/tirage.py	Juge de la rédaction du Tome III (français) : tire au hasard les chiffres du texte à refaire.	98
outils/genome/tome3/juge-redaction/typographie.py	Juge de la rédaction du Tome III (français) : les décimaux à point qui s'imprimeraient tels quels.	62
outils/genome/tome3/juge-ronde/constats.py	Juge du lot « ronde » : les nombres des constats du jugement qui ne sont pas des clés du lot.	150
outils/genome/tome3/juge-ronde/echantillon.py	Juge du lot « ronde » : un point tiré au hasard dans chaque série tracée, contrôlé à la source.	251
outils/genome/tome3/juge-ronde/refaire.py	Juge du lot « ronde » (Tome III, figures des ch. 1 à 4 et 7) : chaque nombre refait par un autre chemin.	589
outils/genome/tome3/juge-ronde/regenerer.py	Juge du lot « ronde » : régénère les CSV du lot par ses propres scripts et les compare au bit.	83
outils/genome/tome3/juge-ronde/zeros.py	Juge du lot « ronde » : les nombres imprimés qui perdent un zéro final (fixed sans zerofill).	26
outils/genome/tome3/juge2-construction/cadres.py	Juge des chiffres du second flux, lot « construction » : chaque cadre de chaque figure contre ses données.	191

Tool	Its own description	Lines
outils/genome/tome3/juge2-construction/points.py	Juge des chiffres du second flux, lot « construction » : un point tiré au hasard dans chaque série NEUVE ou CHANGÉE des CSV du lot, recalculé à la source …	193
outils/genome/tome3/juge2-construction/refaire.py	Juge des chiffres du second flux, lot « construction » : refait chaque nombre NEUF ou DÉPLACÉ des trois fichiers *-nombres.tex (codon, synonyme, juge) depuis …	400
outils/genome/tome3/juge2-construction/regenerer.py	Juge des chiffres du second flux, lot « construction » : régénère au bit les fichiers de données du lot.	159
outils/genome/tome3/juge2-design/cadres.py	Juge de design du second flux (Tome III) : deux critères déclarés (JUGEMENT-figures2-design.md § 1.3), mesurés sur les fichiers commités, par un code à moi. …	67
outils/genome/tome3/juge2-design/couleurs.py	Juge de design du second flux (Tome III) : les couleurs RENDUES d'une page de pilote, et leurs distances.	180
outils/genome/tome3/juge2-design/lot.py	Juge de design du second flux (Tome III) : couleurs rendues de chaque page de figure des pilotes (français), par couleurs.py, et les paires d'entités NOMMÉES …	34
outils/genome/tome3/juge2-design/tetes.py	Juge de design du second flux (Tome III) : relevé des légendes dans les fichiers *-figures.tex. …	47
outils/genome/tome3/juge2-design/vues.py	Juge de design du second flux (Tome III) : une zone d'une page en trois vues côte à côte, couleur, gris (luminance relative) et deutéranope (Machado, Oliveira …	48
outils/genome/tome3/juge2-latents/juge2_barrees.py	Juge des chiffres du lot « espaces latents », second flux : une phrase barrée revient-elle dans une légende ?	91
outils/genome/tome3/juge2-latents/juge2_compiler.py	Juge des chiffres du lot « espaces latents », second flux : compiler le pilote du lot, en français et en anglais, sur les fichiers du commit jugé, et relever …	91
outils/genome/tome3/juge2-latents/juge2_etats.py	Juge des chiffres du lot « espaces latents », second flux : III-7.6 (petits multiples, marge z(1) − z(2), bords des groupes, noms des experts), refait à la …	275
outils/genome/tome3/juge2-latents/juge2_lecteurs.py	Juge des chiffres du lot « espaces latents », second flux : les lectures à la source, écrites de zéro.	141
outils/genome/tome3/juge2-latents/juge2_nuage_rangee.py	Juge des chiffres du lot « espaces latents », second flux : III-9.7 (fonction d'appui, rose, cône, barres) et III-9.1 (d) (la rangée, sa bande, sa phrase de …	350
outils/genome/tome3/juge2-latents/juge2_regen.py	Juge des chiffres du lot « espaces latents », second flux : la régénération au bit des données de figure.	144
outils/genome/tome3/juge2-ronde/barrees.py	Juge des chiffres du lot « ronde », second flux : une phrase barrée revient-elle dans une légende ?	70
outils/genome/tome3/juge2-ronde/cles.py	Juge des chiffres du lot « ronde », second flux : l'inventaire des nombres neufs ou déplacés.	151
outils/genome/tome3/juge2-ronde/langues.py	Juge des chiffres du lot « ronde », second flux : les nombres des deux langues, page par page.	61
outils/genome/tome3/juge2-ronde/refaire2.py	Juge des chiffres du lot « ronde », second flux : chaque nombre neuf ou déplacé, refait par un autre chemin.	426
outils/genome/tome3/juge2-ronde/regenerer2.py	Juge des chiffres du lot « ronde », second flux : les données du lot se régénèrent-elles au bit ?	114
outils/genome/tome3/juge2-ronde/series2.py	Juge des chiffres du lot « ronde », second flux : les séries neuves, point par point.	274

Tool	Its own description	Lines
outils/genome/tome3/juge3/comparer_pdf.py	Juge de la troisième passe : compare deux PDF page à page, par les mots et leurs boîtes (PyMuPDF) et par l'image (rendu en gris à 50 dpi par PyMuPDF, le même ...	70
outils/genome/tome3/juge3/compiler_pilotes.py	Juge de la troisième passe : copie les quatre pilotes du Tome III (memoire/essai/pilote-tome3-*.tex, non commités) dans memoire/essai/juge3/pilotes/ et les ...	59
outils/genome/tome3/juge3/construire_tomes.sh	Juge de la troisième passe : construit les Tomes I et II tels que commités (préambule réparé), sous MiKTeX tel qu'installé, sans toucher les PDF commités. ...	25
outils/genome/tome3/juge3/croisements.py	Juge de la troisième passe : sur une page de PDF, relève les traits dessinés (segments droits des chemins vectoriels, par PyMuPDF get_drawings) qui traversent ...	72
outils/genome/tome3/juge3/croisements_vite.py	Juge de la troisième passe : version rapide de croisements.py pour toutes les pages d'un PDF. ...	69
outils/genome/tome3/juge3/extraire_correctifs.py	Juge de la troisième passe : extrait du préambule commité (memoire/commun/preambule.tex, tel que dans git HEAD) les lignes des deux correctifs, mot pour mot, ...	36
outils/genome/tome3/juge3/figure3.py	Juge de la troisième passe : la zone de la figure d'une page de pilote (du haut du texte à la ligne « Figure n.m »), rendue par zoom3.py en couleur, gris et ...	21
outils/genome/tome3/juge3/gd1_bleu.py	Juge de la troisième passe : GD1, le bleu au rendu. Pour chaque page de figure donnée, rend la zone de la figure (du haut de la page au début de la légende, « ...	74
outils/genome/tome3/juge3/minimal_variantes.py	Juge de la troisième passe : compile les variantes du document minimal (deux passes chacune) et relève, pour chacune, les erreurs du journal (lignes « ! »), ...	31
outils/genome/tome3/juge3/paires3.py	Juge de la troisième passe : refait, par une colorimétrie écrite ici (sRGB ⟶ CIELAB D65 ; gris = luminance Rec. 709 affichée en gris ...	73
outils/genome/tome3/juge3/refaire3.py	Juge de la troisième passe : refait, par mon code, chaque nombre neuf ou déplacé des retouches (7a40c54, 6137230, 1482dbe), depuis les fichiers commités sous ...	290
outils/genome/tome3/juge3/regenerer3.py	Juge de la troisième passe : régénère les données des figures du Tome III par les scripts commités, dans une copie tirée par « git -c core.autocrlf=false ...	125
outils/genome/tome3/juge3/sources_latents.py	Juge de la troisième passe : refait à la source, par un troisième chemin, les nombres neufs de la tête de III-9.7 (rapport ‖ΔW_s x‖ / ‖ΔW_f x‖ sur les 1 329 ...	149
outils/genome/tome3/juge3/tetes3.py	Juge de la troisième passe : DN1 au rendu. Pour chaque légende des pilotes (bloc « Figure 1.n »), prend la tête (du début jusqu'à l'étiquette de provenance « ...	37
outils/genome/tome3/juge3/zoom3.py	Juge de la troisième passe : rend une zone d'une page (en points PDF) à fort agrandissement par PyMuPDF (clip), en couleur, et, sur demande, en gris et en ...	43
outils/genome/tome3/latents_commun.py	Lectures communes du lot « espaces latents » du Tome III (figures III-9.1 (c) et III-9.7).	98
outils/genome/tome3/latents_etats.py	L'espace latent des états de la couche 23 de gpt-oss-20b (Tome III, figure III-7.6).	365

Tool	Its own description	Lines
`outils/genome/tome3/latents_nuage.py`	Le nuage des entrées réelles d'un expert sur les trois premiers axes propres de H, et les ellipses de l'erreur de sortie de la frontière et du synonyme (Tome …	152
`outils/genome/tome3/latents_rangee.py`	Une rangée de 32 poids, avant et après la mutation synonyme (Tome III, figure III-9.1 (c)).	122
The thesis, Volume III		
`memoire/outils/tome3_annexes.py`	Produit les tables des annexes du Tome III depuis le commit HEAD du dépôt (git ls-tree, git cat-file, git log).	386
`memoire/outils/tome3_cassures.py`	Cassures des gardes neuves de la troisième passe des figures du Tome III (SOCLE : « tout témoin neuf doit pouvoir échouer »). …	101
`memoire/outils/tome3_codon.py`	Données des figures du chapitre 8 du Tome III (lot A, le codon synonyme du zéro), tirées des fichiers commités (aucun chiffre recopié à la main).	456
`memoire/outils/tome3_duplication.py`	Données des figures du chapitre 4 du Tome III (la duplication de gènes), tirées des fichiers commités de mesures/genome/duplication/ (aucun chiffre recopié à …	320
`memoire/outils/tome3_eclaireur.py`	Données des figures des chapitres 1 et 2 du Tome III (la question du génome, l'éclaireur), tirées des journaux commités de mesures/genome/etat-de-l-art/ (aucun …	141
`memoire/outils/tome3_genome.py`	Données des figures du chapitre 3 du Tome III (le génome des poids : le brin lu par codons), tirées des fichiers commités de mesures/genome/genome/ (aucun …	178
`memoire/outils/tome3_juge.py`	Données des figures du juge de la construction pour le Tome III (III-8.8, III-10.1, III-10.3), tirées des fichiers commités sous …	246
`memoire/outils/tome3_latents.py`	Données des figures du lot « espaces latents » du Tome III (III-7.6, III-9.1 (d), III-9.7), tirées des fichiers commités de mesures/genome/tome3/latents/ …	573
`memoire/outils/tome3_palette.py`	Le dictionnaire des couleurs du Tome III, chiffré : pour chaque paire que les figures demandent de distinguer, ΔE CIE76 (CIELAB, D65) en vision normale, …	321
`memoire/outils/tome3_synonyme.py`	Données des figures du chapitre 9 du Tome III (lot B, le génome synonyme), tirées des fichiers commités (aucun chiffre recopié à la main).	385
`memoire/outils/tome3_synthese.py`	Données des figures du chapitre 7 du Tome III (la synthèse de la ronde : frontière commune, erreur d'expert contre KL, deux parts de la KL, colonnes géantes), …	370
Total	332 tools	49 577

B.3 Each figure and the files it reads

A beautiful figure remains a true figure: it is drawn from the committed data, by a committed script. Table B.2 gives, for each figure in the style of Volume III, the files that its "Data" row cites; they are the ones read by the tool of the lot named at the end of the row. The figures outside the common style (mimicry, the two leads) cite their data in their caption, in the chapter that places them.

Table B.2: Each figure in the style of Volume III (its `\Source`) and the data files its "Data" line cites (`memoire/outils/tome3_annexes.py` ; un nom sans dossier se lit dans le dossier du précédent).

Figure (file, name)	Data
codon, CodonTheoreme	mesures/genome/construction/codon/tailles-par-tenseur.csv, memoire/outils/tome3_codon.py
codon, CodonOctets	mesures/genome/construction/codon/tables-octets.csv, memoire/outils/tome3_codon.py
codon, CodonRoue	mesures/genome/predictif-elagage/histo-toutes-gptoss20b.npz, mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json, mesures/genome/construction/codon/tailles.json, memoire/outils/tome3_codon.py
codon, CodonAccumulateur	mesures/genome/construction/codon/essais-espace-2026-10-17-a.log, genome/06-codon-v1-2.md, decisions/ARCHITECTURE.md
codon, CodonColonne	mesures/genome/construction/synthese/colonne-par-tenseur.csv, mesures/genome/construction/codon/tailles.json, mesures/genome/construction/synthese/k10/frontiere.csv, memoire/outils/tome3_codon.py
codon, CodonJalon	mesures/genome/construction/codon/j1-s2-passes.csv, j1-s2-verdicts.json, memoire/outils/tome3_codon.py
codon, CodonJalonBis	mesures/disque/placement/juge/seance/Jb3-*, J5-*, J6-*, J1-cartes.json, genome/JUGEMENT-placement.md, mesures/genome/construction/codon/disque-s2.json, mesures/genome/construction/juge/figures/vitesse-passes.csv, disque-passes.csv, genome/JUGEMENT-construction.md, memoire/outils/tome3_codon.py
codon, CodonFormat	decisions/ARCHITECTURE.md, genome/06-codon-v1-2.md
duplication, DupParente	mesures/genome/duplication/csv/scores-parente.csv, paires-temoin-planter-0.5.json, arbre-c23.json, memoire/outils/tome3_duplication.py
duplication, DupArbre	mesures/genome/duplication/csv/phylogenie-c23-noeuds.csv, phylogenie-c23-aretes.csv, phylogenie-c23.json, outils/genome/duplication/figure_phylogenie.py, arbre-c23.json, memoire/outils/tome3_duplication.py
eclaireur, EclBloc	—
eclaireur, EclBorne	mesures/genome/etat-de-l-art/journaux/borne-rd.json, zero-temoin-sans-drapeau.json, zero-canonique.json, mesures/genome/etat-de-l-art/passes/frontiere-lam0-damp0.01-e0_8_16_24.json, memoire/outils/tome3_eclaireur.py
eclaireur, EclZeros	mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json, zero-compte-120b.json, memoire/outils/tome3_eclaireur.py
genome, GenGauss	mesures/genome/genome/gauss-balayage.jsonl, gauss-T1.json, memoire/outils/tome3_genome.py
genome, GenCourbes	mesures/genome/genome/csv/courbes-tourne-l12.csv, courbes-brut-l12.csv, verdict-par-tenseur.csv, mesures/genome/genome/codons-tourne-l12.json, memoire/outils/tome3_genome.py

Figure (file, name)	Data
juge, JconsCodon	mesures/genome/construction/juge/J-T1-echantillon.csv, J-T0-fichiers.json, mesures/genome/construction/codon/tailles.json, memoire/outils/tome3_juge.py
juge, JconsExperts	mesures/genome/construction/juge/figures/routage-grille.csv, J-T0-fichiers.json, mesures/genome/construction/synonyme/kl/kl-frontiere-768-positions.csv, mesures/genome/construction/codon/tailles.json, memoire/outils/tome3_juge.py
juge, JconsFenetres	mesures/genome/construction/juge/figures/kl-par-fenetre.csv, mesures/moteur/O/journaux/o3/c4-20b-comparer.csv, mesures/genome/construction/synonyme/moteur/kl-moteur-frontiere-positions.csv, memoire/outils/tome3_juge.py
latents, LatentsEtats	mesures/genome/tome3/latents/etats-c23.csv, etats-c23.json, outils/genome/tome3/latents_etats.py, h-L23.f32, routage-L23.bin, outils/genome/tome3/juge-latents/juge-etats-marges.json, memoire/outils/tome3_latents.py, latents-temoins.json
latents, LatentsRangee	mesures/genome/tome3/latents/rangee-L08-E00-gate.csv, rangee-L08-E00-gate.json, outils/genome/tome3/latents_rangee.py, mesures/genome/construction/synonyme/figures/spectre-L08-E00-gate.csv, memoire/outils/tome3_latents.py
latents, LatentsNuage	mesures/genome/tome3/latents/nuage-L08-E00-gate-points.csv, nuage-L08-E00-gate-ellipses.csv, nuage-L08-E00-gate.json, outils/genome/tome3/latents_nuage.py, outils/genome/tome3/juge2-latents/journaux/juge2-nuage-rangee.json, memoire/outils/tome3_latents.py, latents-temoins.json
synonyme, SynoBasePropre	mesures/genome/construction/synonyme/figures/spectre-L08-E00-gate.csv, memoire/outils/tome3_synonyme.py
synonyme, SynoQueue	mesures/genome/construction/synonyme/kl/kl-*-768-positions.csv, repere/kl-q2_0-positions.csv, kl/kl-*-768-juge.json, mesures/genome/construction/juge/kl/kl-neuf-*-positions.csv, memoire/outils/tome3_synonyme.py
synonyme, SynoForet	mesures/genome/construction/synonyme/kl/apparie-*-par-fenetre.csv, mesures/genome/construction/juge/figures/apparie-par-fenetre.csv, mesures/genome/construction/synonyme/moteur/kl-moteur-*-positions.csv, memoire/outils/tome3_synonyme.py
synonyme, SynoVariantes	mesures/genome/construction/synonyme/figures/variantes-768.csv, kl/kl-*-768-positions.csv, repere/kl-q2_0-positions.csv, mesures/moteur/O/journaux/o3/c4-20b-comparer.json, mesures/genome/construction/juge/J-T0-fichiers.json, memoire/outils/tome3_synonyme.py

Figure (file, name)	Data
synthese, SynthFrontiere	mesures/genome/construction/synthese/k10/frontiere.csv, outils/genome/synthese/frontiere.py, mesures/genome/etat-de-l-art/passes/synonyme-lam1-damp1-e0_8_16_24.json, memoire/outils/tome3_synthese.py
synthese, SynthErreurKL	mesures/genome/construction/synthese/k10/erreur-contre-kl.csv, frontiere.py, mesures/genome/genome/kl/comparer10-base-contre-recette.json, memoire/outils/tome3_synthese.py
synthese, SynthDeuxParts	mesures/genome/etat-de-l-art/journaux/kl-frontiere-positions.csv, kl-synonyme-d1-positions.csv, kl-frontiere-juge.json, kl-synonyme-d1-juge.json, memoire/outils/tome3_synthese.py
synthese, SynthColonnes	mesures/genome/genome/csv/colonnes-energie-erreur.csv, outils/genome/recette/figures_transformations.py, memoire/outils/tome3_synthese.py

B.4 The licences

The SOCLE asks for the licence of each dependency and of each studied repository to be written down. No external code repository was read by the quest for the genome: the state of the art read only articles and their abstracts (genome/00-etat-de-l-art.md), and no code licence is at stake there. The dependencies are those of Table B.3, each read in the package installed on the machine.

Table B.3: The licences of the dependencies of the tools and of the thesis of Volume III, read in the packages installed on 29 October 2026.

Dependency	Licence	Where it is read
numpy 2.5.4	BSD-3-Clause (and 0BSD, MIT, Zlib, CC0-1.0 for parts)	License-Expression of numpy-2.5.4.dist-info/METADATA
scipy 1.18.1	three-clause BSD	scipy-1.18.1.dist-info/LICENSE.txt
rayon 1.12.0, half 2.7.1, anyhow 1.0.104 (Rust)	MIT or Apache-2.0	license field of their Cargo.toml (local Cargo registry)
llama.cpp b11515	MIT	Volume I, appendix on the tools (official sources of the same commit)
pgfplots	GNU GPL, version 3 or later	header of pgfplots.code.tex (MiKTeX)
PGF/TikZ	LPPL and/or GNU GPL	header of tikz.sty (MiKTeX)
biblatex	LPPL 1.3c	header of biblatex.sty (MiKTeX)

What could be wrong

What could be wrong

- **The first line of a description** is the one the tool gives of itself; it may have aged. The tool of the appendices quotes it, it does not judge it.
- **The control of the figures** checks that the cited files exist, not that they are those the tool really reads; it is the bit-exact regeneration of the data (judge of the third pass of the figures)

that checks this.
- **The licences** are read in the installed packages; another version may have another one.

Appendix C

Logs

Where we start, what this chapter establishes, what remains

At the start, the rule of the first two volumes: whatever produces a published number is committed, tools and logs. **This appendix establishes** where the logs of Volume III are, folder by folder; the sessions that carry a speed measurement, with their conditions; and the passes cut, discarded or published as inconclusive, with their reason, because none disappears silently. **What remains** are the heavy files outside the repository (f64 references, `.kld`), which can be regenerated by the committed tools.

C.1 The log folders

Table C.1 gives each folder of logs, CSVs and notes committed for this volume, its number of files and its size (the tools are in appendix B). It is generated by `memoire/outils/tome3_annexes.py` from the commit: the size of a folder is the sum of the sizes of the git objects of its files (`git ls-tree -l`), independent of the line endings of the copy (corrected on 2026-10-29, appendix B). The folders of the leads (`mesures/pistes/`) and of mimicry belong to their streams; those of the disk placement judge (`mesures/disque/placement/juge/`) are there for reserve R1. Those of the second draw of P3, in progress elsewhere, are not in it (R6).

Table C.1: The committed log folders of Volume III (logs, CSV, notes; tools excluded) (`memoire/outils/tome3_annexes.py`).

Folder	Files	Size (MB)
`mesures/disque/placement/juge/`	316	5.91
`mesures/genome/construction/codon/`	178	2.43
`mesures/genome/construction/juge/`	190	4.47
`mesures/genome/construction/synonyme/`	234	4.90
`mesures/genome/construction/synthese/`	10	0.02
`mesures/genome/duplication/`	35	0.95
`mesures/genome/duplication/csv/`	9	0.33
`mesures/genome/etat-de-l-art/`	6	0.03
`mesures/genome/etat-de-l-art/journaux/`	54	0.58
`mesures/genome/etat-de-l-art/passes/`	23	0.11
`mesures/genome/genome/`	39	0.15
`mesures/genome/genome/csv/`	12	10.36
`mesures/genome/genome/kl/`	21	0.26
`mesures/genome/predictif-elagage/`	41	6.15

Folder	Files	Size (MB)
mesures/genome/predictif-elagage/kl/	33	0.05
mesures/genome/predictif-elagage/plans/	37	7.67
mesures/genome/tome3/latents/	7	0.67
mesures/mimetisme/P1/	22	1.75
mesures/mimetisme/P1/juge/	7	0.04
mesures/mimetisme/P2/	13	0.11
mesures/mimetisme/P2/juge/	4	0.03
mesures/mimetisme/P3/	18	0.09
mesures/mimetisme/P3/etage-B/	56	2.47
mesures/mimetisme/P3/juge/	26	2.54
mesures/mimetisme/P3/routeur-ligne/	85	13.19
mesures/mimetisme/P3/series/	4	0.00
mesures/mimetisme/P4/	14	0.38
mesures/mimetisme/P4/figures/	3	0.16
mesures/mimetisme/P4/juge/	6	0.01
mesures/mimetisme/P5/	8	0.63
mesures/mimetisme/P5/etape1-diag/	65	0.05
mesures/mimetisme/P5/etape1-regime/	49	0.04
mesures/mimetisme/P5/etape1/	70	0.15
mesures/mimetisme/P5/figures/	10	0.32
mesures/mimetisme/P5/gibbs/	1	0.00
mesures/mimetisme/P5/juge/	162	0.09
mesures/mimetisme/P6/	11	0.20
mesures/mimetisme/P6/juge/	8	0.03
mesures/mimetisme/P6/temoins/	8	0.01
mesures/mimetisme/humain/	32	0.12
mesures/mimetisme/ingenierie/	9	0.01
mesures/mimetisme/mathematiques/	8	0.03
mesures/mimetisme/physique/	1	0.01
mesures/mimetisme/physique/e1/	17	0.22
mesures/mimetisme/physique/e2/	6	0.00
mesures/mimetisme/physique/e3/	24	0.07
mesures/mimetisme/terre_cosmos/	21	0.10
mesures/mimetisme/tri/	1	0.01
mesures/mimetisme/vivant/	48	0.18
mesures/pistes/spectres/	81	2.52
mesures/pistes/spectres/juge/	9	0.13
mesures/pistes/tropicale/	32	0.17
mesures/pistes/tropicale/figures/	9	0.35
mesures/pistes/tropicale/indices/	4	3.47
mesures/pistes/tropicale/juge/	10	0.03
outils/genome/tome3/juge-construction/	8	0.06
outils/genome/tome3/juge-design/	2	0.00
outils/genome/tome3/juge-final/	14	0.20
outils/genome/tome3/juge-latents/	5	0.02

Folder	Files	Size (MB)
`outils/genome/tome3/juge-redaction/`	18	0.18
`outils/genome/tome3/juge-ronde/`	18	0.11
`outils/genome/tome3/juge2-construction/`	12	0.05
`outils/genome/tome3/juge2-design/`	37	2.68
`outils/genome/tome3/juge2-latents/`	9	0.05
`outils/genome/tome3/juge2-ronde/`	23	0.09
`outils/genome/tome3/juge3/`	47	0.32
Total	2 400	78.56

What is not in the repository. The heavy files: the f64 references and the `.kld` of the KL passes (about 13 GB for the judge of the construction alone, under `~/dev/essais/troncons/`), the `.sym` files of the 768 experts and the fast companions. Each is regenerated by a committed tool, and its fingerprint is in a committed log when a figure depends on it (for example the output `.kld` of J-C1, Section 11.4).

C.2 The sessions that carry a speed figure

A speed figure is worth only as much as its session: the machine, the power supply, the load of the other streams, the card lock. Table C.2 gives those of the volume.

Table C.2: The sessions that carry a speed figure of Volume III, their conditions and what they decide [meas.].

Session	When	Conditions	What it decides
lot B, s1 (20b)	17 Oct., start at 11:45	A B B A A B B A; under the load of the KL pass B0, of the computation of the rate and of the writing of the companion of the 120b ($\sigma_A = 17.8\,\%$)	published, inconclusive
lot B, s2 (20b)	22 Oct., 1:11 – 1:25	A B B A A B B A; two empty readings at the start, no pass under load, mains power, lock ($\sigma_A = 2.3\,\%$)	×1.24 announced, struck through as a gain of the mode (G2)
lot B, 120b-V	22 Oct., 2:34 – 3:38	same rules ($\sigma_A = 6.6\,\%$)	×2.00 announced, struck through as a gain of the mode (G1)
judge, three-file session (120b then 20b)	22 Oct., engine from 6:17 to 7:11	A B R R B A A B R R B A, disk stage first; none of the 36 passes under load; mains power at each pass; lock held	$r(R/B) = 1.284$ and 0.942; J1 given again (1.442)

Data: `mesures/genome/construction/synonyme/vitesse/pilote-s1.json`, `pilote-s2.json`, `pilote-s2-120b.json`; `mesures/genome/construction/juge/vitesse/seance-\{120b,20b\}.\{json,log\}`.
The session of lot A is in the chapter on the codon; those of the disk placement, in the chapter "The disk stage" planned for Volume II.

C.3 The passes cut, discarded, or inconclusive

No pass disappears silently: those that decided nothing are published as such, with their reason.

- **Session s1 of lot B**: published, inconclusive, because it ran under the load of other passes; redone on a calm machine (s2), and this is what led to the recipe "a speed session starts only on a machine recorded as calm" (Section 10.7.1).
- **The first B pass of session s2** (5.46 tokens/s, just after the companion was written): kept in the median, as the declared rule required; the judge shows that with the noise of the B passes it would have prevented the announcement (finding M4).
- **The judge's substituted f64 pass** $\lambda = 0$: the bash loop that launched it was stopped, but the native program finished on its own; its redirected output was complete and was kept under the name `sub-10.json`. The substituted f64 pass $\lambda = 1$ was not launched (time, and the placement stream waiting for calm), and this is stated at each verdict that depends on it (Section 11.5.2).
- **The broken copies** (the flipped bias of J-C1, the breaks of the figures) are erased after use; their logs remain.
- **The second draw of P3**, in progress in another session, has attempts cut by external causes (mains power, the machine's memory, disk), without any usable measurement and without any threshold changed; its logs are committed by its session, and the volume draws no figure from them (reserve R6).

What could be wrong

> **What could be wrong**
>
> - **The size of a folder** is that of the committed files, with LF line endings (a CRLF copy is larger); it says nothing of the heavy regenerable files outside the repository.
> - **The sorting by folder** follows the depth of the paths (as in Volume II); a folder that mixes two steps appears in it as a single row.
> - **"No pass under load"** is what the declared reading says (more than 5 % of a core or more than 8 MB read in 3 s); a smaller load, or one between two readings, escapes it.

Bibliography

[1] C. E. Shannon. “A Mathematical Theory of Communication”. In: *Bell System Technical Journal* 27.3 (1948), pp. 379–423. DOI: 10.1002/j.1538-7305.1948.tb01338.x. (Visited on 2026-10-15).

[2] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. 2nd ed. Chapitre 10 : théorème 10.3.2 (source gaussienne) et théorème du débit-distorsion. Hoboken, NJ: Wiley-Interscience, 2006. ISBN: 978-0-471-24195-9. DOI: 10.1002/047174882X. (Visited on 2026-10-15).

[3] F. H. C. Crick, L. Barnett, S. Brenner, and R. J. Watts-Tobin. “General Nature of the Genetic Code for Proteins”. In: *Nature* 192.4809 (1961). Lecture du brin par triplets à partir d’un point fixe, sans virgule : le cadre de lecture, pp. 1227–1232. DOI: 10.1038/1921227a0. (Visited on 2026-10-16).

[4] F. H. C. Crick. “The Origin of the Genetic Code”. In: *Journal of Molecular Biology* 38.3 (1968). Le code génétique, table dégénérée (64 codons pour 20 acides aminés et l’arrêt), pp. 367–379. DOI: 10.1016/0022-2836(68)90392-6. (Visited on 2026-10-16).

[5] S. J. Freeland and L. D. Hurst. “The Genetic Code Is One in a Million”. In: *Journal of Molecular Evolution* 47.3 (1998-09). Métadonnées Crossref ; résumé de l’éditeur lu., pp. 238–248. DOI: 10.1007/pl00006381. URL: https://doi.org/10.1007/pl00006381 (visited on 2026-10-16).

[6] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025-08-08. arXiv: 2508.10925. URL: https://arxiv.org/abs/2508.10925 (visited on 2026-10-16).

[7] B. D. Rouhani, R. Zhao, A. More, et al. *Microscaling Data Formats for Deep Learning*. Version v3 du 2023-10-19. Résumé lu (API arXiv). 2023-10-16. arXiv: 2310.10537. URL: https://arxiv.org/abs/2310.10537 (visited on 2026-10-16).

[8] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0*. Définit MXFP4 : éléments E2M1, échelle partagée E8M0 par bloc de 32. 2023. URL: https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf (visited on 2026-10-15).

[9] C. E. Shannon. “Coding Theorems for a Discrete Source with a Fidelity Criterion”. In: *IRE National Convention Record*. Vol. 7. 4. Fonction distorsion-débit ; borne de la source gaussienne. 1959, pp. 142–163.

[10] R. E. Blahut. “Computation of channel capacity and rate-distortion functions”. In: *IEEE Transactions on Information Theory* 18.4 (1972-07). Métadonnées Crossref., pp. 460–473. DOI: 10.1109/TIT.1972.1054855. URL: https://doi.org/10.1109/TIT.1972.1054855 (visited on 2026-10-16).

[11] S. Kullback and R. A. Leibler. “On Information and Sufficiency”. In: *The Annals of Mathematical Statistics* 22.1 (1951-03). La divergence d’information entre deux lois, positive et nulle seulement pour deux lois égales ; c’est la mesure du juge., pp. 79–86. DOI: 10.1214/aoms/1177729694. URL: https://doi.org/10.1214/aoms/1177729694 (visited on 2026-10-16).

[12] H. Gish and J. N. Pierce. “Asymptotically efficient quantizing”. In: *IEEE Transactions on Information Theory* 14.5 (1968), pp. 676–683. DOI: 10.1109/TIT.1968.1054193.

[13] M. W. Marcellin and T. R. Fischer. “Trellis Coded Quantization of Memoryless and Gauss-Markov Sources”. In: *IEEE Transactions on Communications* 38.1 (1990), pp. 82–93. DOI: 10.1109/26.46532. (Visited on 2026-10-15).

[14] J. Lin, J. Tang, H. Tang, et al. *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration*. Version v6 du 2026-04-25. Résumé lu (API arXiv). 2023-06-01. arXiv: 2306.00978. URL: https://arxiv.org/abs/2306.00978 (visited on 2026-10-16).

[15] R. Shafipour, D. Harrison, M. Horton, et al. *SeedLM: Compressing LLM Weights into Seeds of Pseudo-Random Generators*. v1 du 2024-10-14, v2 du 2024-10-16 (la plus récente à la consultation). 2024-10-14. arXiv: 2410.10714. URL: https://arxiv.org/abs/2410.10714 (visited on 2026-10-16).

[16] P. Nooralinejad, A. Abbasi, S. A. Koohpayegani, et al. *PRANC: Pseudo RAndom Networks for Compacting deep models*. Version v2 du 2023-08-28. Résumé lu (API arXiv). 2022-06-16. arXiv: 2206.08464. URL: https://arxiv.org/abs/2206.08464 (visited on 2026-10-16).

[17] S. A. Koohpayegani, K. Navaneet, P. Nooralinejad, S. Kolouri, and H. Pirsiavash. *NOLA: Compressing LoRA using Linear Combination of Random Basis*. Version v2 du 2024-04-30. Résumé lu (API arXiv). 2023-10-04. arXiv: 2310.02556. URL: https://arxiv.org/abs/2310.02556 (visited on 2026-10-16).

[18] C. Li, H. Farkhoor, R. Liu, and J. Yosinski. *Measuring the Intrinsic Dimension of Objective Landscapes*. Version v1 du 2018-04-24. Résumé lu (API arXiv). 2018-04-24. arXiv: 1804.08838. URL: https://arxiv.org/abs/1804.08838 (visited on 2026-10-16).

[19] D. Ha, A. Dai, and Q. V. Le. *HyperNetworks*. v4 du 2016-12-01 (la plus récente à la consultation). 2016-09-27. arXiv: 1609.09106. URL: https://arxiv.org/abs/1609.09106 (visited on 2026-10-16).

[20] K. O. Stanley, D. B. D’Ambrosio, and J. Gauci. “A Hypercube-Based Encoding for Evolving Large-Scale Neural Networks”. In: *Artificial Life* 15.2 (2009-04). Métadonnées Crossref ; résumé de l’éditeur lu., pp. 185–212. DOI: 10.1162/artl.2009.15.2.15202. URL: https://doi.org/10.1162/artl.2009.15.2.15202 (visited on 2026-10-16).

[21] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. “Growing Neural Cellular Automata”. In: *Distill* 5.2 (2020-02-11). Métadonnées Crossref ; résumé de l’éditeur lu. DOI: 10.23915/distill.00023. URL: https://doi.org/10.23915/distill.00023 (visited on 2026-10-16).

[22] E. Najarro, S. Sudhakaran, C. Glanois, and S. Risi. *HyperNCA: Growing Developmental Networks with Neural Cellular Automata*. Version v1 du 2022-04-25. Résumé lu (API arXiv). 2022-04-25. arXiv: 2204.11674. URL: https://arxiv.org/abs/2204.11674 (visited on 2026-10-16).

[23] P. Li, Z. Zhang, P. Yadav, et al. *Merge, Then Compress: Demystify Efficient SMoE with Hints from Its Routing Policy*. Version v2 du 2024-03-14. Résumé lu (API arXiv). 2023-10-02. arXiv: 2310.01334. URL: https://arxiv.org/abs/2310.01334 (visited on 2026-10-16).

[24] S. He, R.-Z. Fan, L. Ding, L. Shen, T. Zhou, and D. Tao. *Merging Experts into One: Improving Computational Efficiency of Mixture of Experts*. Version v3 du 2023-11-21. Résumé lu (API arXiv). 2023-10-15. arXiv: 2310.09832. URL: https://arxiv.org/abs/2310.09832 (visited on 2026-10-16).

[25] I.-C. Chen, H.-S. Liu, W.-F. Sun, C.-H. Chao, Y.-C. Hsu, and C.-Y. Lee. *Retraining-Free Merging of Sparse MoE via Hierarchical Clustering*. Version v4 du 2025-10-26. Résumé lu (API arXiv). 2024-10-11. arXiv: 2410.08589. URL: https://arxiv.org/abs/2410.08589 (visited on 2026-10-16).

[26] L. Li, Z. Qiyuan, J. Wang, et al. *Sub-MoE: Efficient Mixture-of-Expert LLMs Compression via Subspace Expert Merging*. Version v1 du 2025-06-29. Résumé lu (API arXiv). 2025-06-29. arXiv: 2506.23266. URL: https://arxiv.org/abs/2506.23266 (visited on 2026-10-16).

[27] H. Gu, W. Li, L. Li, et al. *Delta Decompression for MoE-based LLMs Compression*. Version v1 du 2025-02-24. Résumé lu (API arXiv). 2025-02-24. arXiv: 2502.17298. URL: https://arxiv.org/abs/2502.17298 (visited on 2026-10-16).

[28] X. Chen, M. Ha, Z. Lan, J. Zhang, and J. Li. *MoBE: Mixture-of-Basis-Experts for Compressing MoE-based LLMs*. Version v1 du 2025-08-07. Résumé lu (API arXiv). 2025-08-07. arXiv: 2508.05257. URL: https://arxiv.org/abs/2508.05257 (visited on 2026-10-16).

[29] D. Li, L. Li, Q. Zhu, et al. *OMP-MoE: Efficient Expert Pruning for Mixture-of-Experts LLMs via Orthogonal Matching Pursuit*. Version v2 du 2026-09-29. Résumé lu (API arXiv). 2026-09-09. arXiv: 2609.31631. URL: https://arxiv.org/abs/2609.31631 (visited on 2026-10-16).

[30] J. Liu, G. Xiao, K. Li, et al. *BitDelta: Your Fine-Tune May Only Be Worth One Bit*. Version v3 du 2024-10-13. Résumé lu (API arXiv). 2024-02-15. arXiv: 2402.10193. URL: https://arxiv.org/abs/2402.10193 (visited on 2026-10-16).

[31] B. Ping, S. Wang, H. Wang, et al. *Delta-CoMe: Training-Free Delta-Compression with Mixed-Precision for Large Language Models*. Version v3 du 2024-11-26. Résumé lu (API arXiv). 2024-06-13. arXiv: 2406.08903. URL: https://arxiv.org/abs/2406.08903 (visited on 2026-10-16).

[32] R. P. N. Rao and D. H. Ballard. "Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects". In: *Nature Neuroscience* 2.1 (1999). Métadonnées relues par l'API Crossref, pp. 79–87. DOI: 10.1038/4580. (Visited on 2026-10-16).

[33] S. Wiedemann, H. Kirchhoffer, S. Matlage, et al. *DeepCABAC: Context-adaptive binary arithmetic coding for deep neural network compression*. Version v1 du 2019-05-15. Résumé lu (API arXiv). 2019-05-15. arXiv: 1905.08318. URL: https://arxiv.org/abs/1905.08318 (visited on 2026-10-16).

[34] M. Hershcovitch, A. Wood, L. Choshen, et al. *ZipNN: Lossless Compression for AI Models*. v2 du 2025-06-04. Code : github.com/zipnn/zipnn, MIT, commit 6009704 du 2026-04-11. 2024. arXiv: 2411.05239. URL: https://arxiv.org/abs/2411.05239 (visited on 2026-10-15).

[35] T. Zhang, M. Hariri, S. Zhong, et al. *70% Size, 100% Accuracy: Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float (DFloat11)*. NeurIPS 2025 ; v3 du 2026-01-01. Code : github.com/LeanModels/DFloat11, Apache-2.0, commit 4577338 du 2025-11-24. 2025-04-15. arXiv: 2504.11651. URL: https://arxiv.org/abs/2504.11651 (visited on 2026-10-15).

[36] P. Yubeaton, T. Mahmoud, S. Naga, et al. *Huff-LLM: End-to-End Lossless Compression for Efficient LLM Inference*. Aucun code publié trouvé. 2025-02-02. arXiv: 2502.00922. URL: https://arxiv.org/abs/2502.00922 (visited on 2026-10-15).

[37] R. Xie, A. Ul Haq, Y. Fang, et al. *TRACE: Unlocking Effective CXL Bandwidth via Lossless Compression and Precision Scaling*. v3 du 2026-01-30. Résumé seul lu : plans de bits par canal codés sans perte, lecture proportionnelle à la précision, dans un contrôleur CXL matériel ; BF16 réduit de 25,2 % sans perte (éditeur, non reproduit). 2025-09-03. arXiv: 2509.03377. (Visited on 2026-10-15).

[38] P. R. Huttenlocher. "Synaptic density in human frontal cortex: Developmental changes and effects of aging". In: *Brain Research* 163.2 (1979-03). Métadonnées relues par l'API Crossref, pp. 195–205. DOI: 10.1016/0006-8993(79)90349-4. (Visited on 2026-10-16).

[39] Y. LeCun, J. S. Denker, and S. A. Solla. "Optimal Brain Damage". In: *Advances in Neural Information Processing Systems 2 (NIPS 1989)*. 1989. URL: https://proceedings.neurips.cc/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html (visited on 2026-10-16).

[40] B. Hassibi and D. G. Stork. "Second order derivatives for network pruning: Optimal Brain Surgeon". In: *Advances in Neural Information Processing Systems 5 (NIPS 1992)*. Page des actes vérifiée (titre) le 2026-10-16. 1992. URL: https://proceedings.neurips.cc/paper/1992/hash/303ed4c69846ab36c2904d3ba8573050-Abstract.html (visited on 2026-10-16).

[41] S. Han, H. Mao, and W. J. Dally. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. v5 du 2016-02-15 (la plus récente à la consultation). 2015-10-01. arXiv: 1510.00149. URL: https://arxiv.org/abs/1510.00149 (visited on 2026-10-16).

[42] X. Lu, Q. Liu, Y. Xu, et al. *Not All Experts are Equal: Efficient Expert Pruning and Skipping for Mixture-of-Experts Large Language Models*. Version v2 du 2024-05-30. Résumé lu (API arXiv). 2024-02-22. arXiv: 2402.14800. URL: https://arxiv.org/abs/2402.14800 (visited on 2026-10-16).

[43] L. O. Martin, L. Bandarkar, and N. Peng. *ESTS at WMT26: Routing-Informed Expert Pruning for Model Compression*. Version v1 du 2026-09-11. Résumé lu (API arXiv). 2026-09-11. arXiv: 2609.12310. URL: https://arxiv.org/abs/2609.12310 (visited on 2026-10-16).

[44] B. A. Olshausen and D. J. Field. "Emergence of simple-cell receptive field properties by learning a sparse code for natural images". In: *Nature* 381.6583 (1996-06). Métadonnées Crossref ; résumé de l'éditeur lu., pp. 607–609. DOI: 10.1038/381607a0. URL: https://doi.org/10.1038/381607a0 (visited on 2026-10-16).

[45] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh. *Extreme Compression of Large Language Models via Additive Quantization*. Première version 2024-01-11, dernière 2024-09-11 (2401.06118v4). ICML, 2024. 2024-01-11. arXiv: 2401.06118. URL: https://arxiv.org/abs/2401.06118 (visited on 2026-10-15).

[46] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. D. Sa. *QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks*. Première version 2024-02-06, dernière 2024-06-04 (2402.04396v2). ICML 2024. 2024-02-06. arXiv: 2402.04396. URL: https://arxiv.org/abs/2402.04396 (visited on 2026-10-15).

[47] A. Tseng, Q. Sun, D. Hou, and C. De Sa. *QTIP: Quantization with Trellises and Incoherence Processing*. Version 4 du 18 juin 2025 (NeurIPS 2024), sections 2.3 à 3.2 et annexe A.1.1 lues ; erreur 0,069 d'un treillis de 256 dimensions à 2 bits sur une gaussienne, tableau 1 ; 0,0733 (tableau 2, treillis à queue mordue). 2024. arXiv: 2406.11235. URL: https://arxiv.org/abs/2406.11235v4 (visited on 2026-10-16).

[48] Y. Domb, H. Sackstein, and T. Solberg. *HyperQuant: A Rate-Distortion-Optimal Quantization Pipeline for Large Language and Diffusion Models*. Version v1 du 2026-06-22. Résumé lu (API arXiv). 2026-06-22. arXiv: 2606.23406. URL: https://arxiv.org/abs/2606.23406 (visited on 2026-10-16).

[49] M. Zhussip, D. Shopkhoev, A. Ali, and S. Lefkimmiatis. *Share Your Attention: Transformer Weight Sharing via Matrix-based Dictionary Learning*. Version v2 du 2026-02-19. Résumé lu (API arXiv). 2025-08-06. arXiv: 2508.04581. URL: https://arxiv.org/abs/2508.04581 (visited on 2026-10-16).

[50] E. Frantar, S. P. Singh, and D. Alistarh. *Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning*. Version v2 du 2023-01-08. Résumé lu (API arXiv). 2022-08-24. arXiv: 2208.11580. URL: https://arxiv.org/abs/2208.11580 (visited on 2026-10-16).

[51] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. Version v2 du 2023-03-22. Résumé lu (API arXiv). 2022-10-31. arXiv: 2210.17323. URL: https://arxiv.org/abs/2210.17323 (visited on 2026-10-16).

[52] E. Frantar and D. Alistarh. *QMoE: Practical Sub-1-Bit Compression of Trillion-Parameter Models*. Version v1 du 2023-10-25. Résumé lu (API arXiv). 2023-10-25. arXiv: 2310.16795. URL: https://arxiv.org/abs/2310.16795 (visited on 2026-10-16).

[53] X. Hu, Z. Chen, D. Yang, et al. *MoEQuant: Enhancing Quantization for Mixture-of-Experts Large Language Models via Expert-Balanced Sampling and Affinity Guidance*. Version v1 du 2025-05-02. Résumé lu (API arXiv). 2025-05-02. arXiv: 2505.03804. URL: https://arxiv.org/abs/2505.03804 (visited on 2026-10-16).

[54] Z. Fu, T. Zhao, N. Ding, et al. *EAQuant: Enhancing Post-Training Quantization for MoE Models via Expert-Aware Optimization*. Version v3 du 2026-02-01. Résumé lu (API arXiv). 2025-06-16. arXiv: 2506.13329. URL: https://arxiv.org/abs/2506.13329 (visited on 2026-10-16).

[55] Y. C. Yoon. "LIF and Simplified SRM Neurons Encode Signals Into Spikes via a Form of Asynchronous Pulse Sigma–Delta Modulation". In: *IEEE Transactions on Neural Networks and Learning Systems* 28.5 (2017-05). Métadonnées Crossref ; résumé de l'éditeur lu., pp. 1192–1205. DOI: 10.1109/tnnls.2016.2526029. URL: https://doi.org/10.1109/tnnls.2016.2526029 (visited on 2026-10-16).

[56] P. Nair, P. Datta, J. Dean, P. Jain, and A. Kusupati. *Matryoshka Quantization*. Version v3 du 2025-03-03. Résumé lu (API arXiv). 2025-02-10. arXiv: 2502.06786. URL: https://arxiv.org/abs/2502.06786 (visited on 2026-10-16).

[57] Y. Park, J. Hyun, S. Cho, B. Sim, and J. W. Lee. *Any-Precision LLM: Low-Cost Deployment of Multiple, Different-Sized LLMs*. Version v4 du 2024-06-21. Résumé lu (API arXiv). 2024-02-16. arXiv: 2402.10517. URL: https://arxiv.org/abs/2402.10517 (visited on 2026-10-16).

[58] M. Kleinegger, E. Crnčević, and D. Alistarh. *MatGPTQ: Efficient and Accurate Inference over Nested Quantized Models*. v2 du 2026-09-30. Résumé seul lu ; code annoncé IST-DASLab/MatGPTQ, licence non vérifiée. 2026-02-03. arXiv: 2602.03537. (Visited on 2026-10-15).

[59] A. Tseng, T. Yu, and Y. Park. *Training LLMs with MXFP4*. Version v3 du 2025-08-26. Résumé lu (API arXiv). 2025-02-27. arXiv: 2502.20586. URL: https://arxiv.org/abs/2502.20586 (visited on 2026-10-16).

[60] T.-L. Li, J. Huang, L.-C. Wang, and J. R. Gotei. *Scale-QLoRA: Code-Invariant Adapter Merging for Native 4-bit Microscaling LLMs*. Version v1 du 2026-09-03. Résumé lu (API arXiv). 2026-09-03. arXiv: 2609.04526. URL: https://arxiv.org/abs/2609.04526 (visited on 2026-10-16).

[61] M. Grella. *Tied Trit-Planes: Constraining PTQTP to a Uniform Nine-Level Quantizer, with a Persistent Folded Format for Disk-Streamed Mixture-of-Experts Serving*. Première version 2026-08-09, dernière 2026-08-09 (2608.08910v1). 2026-08-09. arXiv: 2608.08910. URL: https://arxiv.org/abs/2608.08910 (visited on 2026-10-15).

[62] S. Ma, H. Wang, L. Ma, et al. *The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits*. Première version 2024-02-27, dernière 2024-02-27 (2402.17764v1). Work in progress. 2024-02-27. arXiv: 2402.17764. URL: https://arxiv.org/abs/2402.17764 (visited on 2026-10-15).

[63] L. M. Adleman. "Molecular Computation of Solutions to Combinatorial Problems". In: *Science* 266.5187 (1994-11-11). Métadonnées Crossref ; résumé de l'éditeur lu., pp. 1021–1024. DOI: 10.1126/science.7973651. URL: https://doi.org/10.1126/science.7973651 (visited on 2026-10-16).

[64] G. M. Church, Y. Gao, and S. Kosuri. "Next-Generation Digital Information Storage in DNA". In: *Science* 337.6102 (2012-09-28). Métadonnées Crossref ; résumé de l'éditeur lu., pp. 1628–1628. DOI: 10.1126/science.1226355. URL: https://doi.org/10.1126/science.1226355 (visited on 2026-10-16).

[65] N. Goldman, P. Bertone, S. Chen, et al. "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA". In: *Nature* 494.7435 (2013-01-23). Métadonnées Crossref ; résumé de l'éditeur lu., pp. 77–80. DOI: 10.1038/nature11875. URL: https://doi.org/10.1038/nature11875 (visited on 2026-10-16).

[66] Y. Erlich and D. Zielinski. "DNA Fountain enables a robust and efficient storage architecture". In: *Science* 355.6328 (2017-03-03). Métadonnées Crossref ; résumé de l'éditeur lu., pp. 950–954. DOI: 10.1126/science.aaj2038. URL: https://doi.org/10.1126/science.aaj2038 (visited on 2026-10-16).

[67] P. M. Sharp and W.-H. Li. "The codon adaptation index-a measure of directional synonymous codon usage bias, and its potential applications". In: *Nucleic Acids Research* 15.3 (1987). Métadonnées Crossref ; résumé de l'éditeur lu., pp. 1281–1295. DOI: 10.1093/nar/15.3.1281. URL: https://doi.org/10.1093/nar/15.3.1281 (visited on 2026-10-16).

[68] O. Sieberling, D. Kuznedelev, E. Kurtic, and D. Alistarh. *EvoPress: Accurate Dynamic Model Compression via Evolutionary Search*. Version v2 du 2025-07-01. Résumé lu (API arXiv). 2024-10-18. arXiv: 2410.14649. URL: https://arxiv.org/abs/2410.14649 (visited on 2026-10-16).

[69] T. Plate. “Holographic reduced representations”. In: *IEEE Transactions on Neural Networks* 6.3 (1995-05). Métadonnées Crossref ; résumé de l’éditeur lu., pp. 623–641. DOI: 10.1109/72.377968. URL: https://doi.org/10.1109/72.377968 (visited on 2026-10-16).

[70] T. J. Teyler and P. DiScenna. “The hippocampal memory indexing theory.” In: *Behavioral Neuroscience* 100.2 (1986). Métadonnées Crossref ; résumé de l’éditeur lu., pp. 147–154. DOI: 10.1037/0735-7044.100.2.147. URL: https://doi.org/10.1037/0735-7044.100.2.147 (visited on 2026-10-16).

[71] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. “YaRN: Efficient Context Window Extension of Large Language Models”. In: *arXiv* (2023-08-31). L’interpolation « NTK par parties » et le facteur de température $\sqrt{1/t} = 0{,}1 \ln s + 1$; cité par l’implémentation d’OpenAI. arXiv: 2309.00071. URL: https://arxiv.org/abs/2309.00071 (visited on 2026-10-15).

[72] *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. IEEE. 2019. DOI: 10.1109/IEEESTD.2019.8766229. (Visited on 2026-10-15).

[73] E. B. Wilson. “Probable Inference, the Law of Succession, and Statistical Inference”. In: *Journal of the American Statistical Association* 22.158 (1927). Intervalle de score utilisé pour le taux de premier choix identique (C4)., pp. 209–212. DOI: 10.1080/01621459.1927.10502953.

[74] M. Z. Mao and R. M. Gray. “Stationary and Trellis Encoding for IID Sources and Simulation”. In: *2010 Data Compression Conference*. Le treillis « à décalage » et le codeur à permutation aléatoire (RPTC). 2010, pp. 3–12. DOI: 10.1109/DCC.2010.8. (Visited on 2026-10-16).

[75] T. R. Fischer, M. W. Marcellin, and M. Wang. “Trellis-Coded Vector Quantization”. In: *IEEE Transactions on Information Theory* 37.6 (1991), pp. 1551–1566. DOI: 10.1109/18.104316. (Visited on 2026-10-16).

[76] A. J. Viterbi. “Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm”. In: *IEEE Transactions on Information Theory* 13.2 (1967), pp. 260–269. DOI: 10.1109/TIT.1967.1054010. (Visited on 2026-10-15).

[77] J. Max. “Quantizing for Minimum Distortion”. In: *IRE Transactions on Information Theory* 6.1 (1960), pp. 7–12. DOI: 10.1109/TIT.1960.1057548. (Visited on 2026-10-15).

[78] S. Ohno. *Evolution by Gene Duplication*. Métadonnées relues par l’API Crossref. Berlin, Heidelberg: Springer, 1970. DOI: 10.1007/978-3-642-86659-3. (Visited on 2026-10-16).

[79] S. K. Ainsworth, J. Hayase, and S. Srinivasa. *Git Re-Basin: Merging Models modulo Permutation Symmetries*. v3 du 2023-03-01 ; ICLR 2023. 2022-09-11. arXiv: 2209.04836. URL: https://arxiv.org/abs/2209.04836 (visited on 2026-10-16).

[80] R. Entezari, H. Sedghi, O. Saukh, and B. Neyshabur. *The Role of Permutation Invariance in Linear Mode Connectivity of Neural Networks*. v2 du 2022-07-05 ; ICLR 2022. 2021-10-12. arXiv: 2110.06296. URL: https://arxiv.org/abs/2110.06296 (visited on 2026-10-16).

[81] H. W. Kuhn. “The Hungarian method for the assignment problem”. In: *Naval Research Logistics Quarterly* 2.1–2 (1955). mise en œuvre utilisée : scipy.optimize.linear_sum_assignment (Crouse 2016), scipy 1.18.1, licence BSD-3, pp. 83–97. DOI: 10.1002/nav.3800020109. URL: https://doi.org/10.1002/nav.3800020109 (visited on 2026-10-16).

[82] D. F. Crouse. “On implementing 2D rectangular assignment algorithms”. In: *IEEE Transactions on Aerospace and Electronic Systems* 52.4 (2016), pp. 1679–1696. DOI: 10.1109/TAES.2016.140952. URL: https://doi.org/10.1109/TAES.2016.140952 (visited on 2026-10-16).

[83] G. A. Miller. “Note on the bias of information estimates”. In: *Information Theory in Psychology: Problems and Methods* (1955). chapitre d’ouvrage collectif (éd. H. Quastler), pp. 95–100.

[84] J. B. Kruskal. “On the shortest spanning subtree of a graph and the traveling salesman problem”. In: *Proceedings of the American Mathematical Society* 7.1 (1956), pp. 48–50. DOI: 10.1090/S0002-9939-1956-0078686-7. URL: https://doi.org/10.1090/S0002-9939-1956-0078686-7 (visited on 2026-10-16).

[85] Z. Gu, L. M. Steinmetz, X. Gu, C. Scharfe, R. W. Davis, and W.-H. Li. “Role of duplicate genes in genetic robustness against null mutations”. In: *Nature* 421 (2003), pp. 63–66. DOI: 10.1038/nature01198. URL: https://doi.org/10.1038/nature01198 (visited on 2026-10-16).

[86] A. Skliar, T. van Rozendaal, R. Lepert, T. Boinovski, et al. *Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference*. v2 du 2025-06-24. 2024-11-27. arXiv: 2412.00099. URL: https://arxiv.org/abs/2412.00099 (visited on 2026-10-16).

[87] S. Laughlin. “A Simple Coding Procedure Enhances a Neuron’s Information Capacity”. In: *Zeitschrift für Naturforschung C* 36.9-10 (1981-10). Codage efficace dans l’œil de la mouche : la précision d’un neurone se répartit selon la loi de ce qu’il code., pp. 910–912. DOI: 10.1515/znc-1981-9-1040. URL: https://doi.org/10.1515/znc-1981-9-1040 (visited on 2026-10-16).

[88] F. H. C. Crick. “Codon–anticodon pairing: The wobble hypothesis”. In: *Journal of Molecular Biology* 19.2 (1966-08). Les codons synonymes : plusieurs mots du code pour un même sens ; l’analogue des deux codes ±0 du E2M1., pp. 548–555. DOI: 10.1016/S0022-2836(66)80022-0. URL: https://doi.org/10.1016/S0022-2836(66)80022-0 (visited on 2026-10-16).

[89] P. A. Chou, T. Lookabaugh, and R. M. Gray. “Entropy-constrained vector quantization”. In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 37.1 (1989-01). Le lagrangien D + λR et la condition d’optimalité des centres ; l’ECSQ en est le cas scalaire., pp. 31–42. DOI: 10.1109/29.17498. URL: https://doi.org/10.1109/29.17498 (visited on 2026-10-16).

[90] N. Farvardin and J. W. Modestino. “Optimum quantizer performance for a class of non-Gaussian memoryless sources”. In: *IEEE Transactions on Information Theory* 30.3 (1984-05). Quantification scalaire contrainte par l’entropie (ECSQ) : à débit élevé, des cellules uniformes et une zone morte., pp. 485–497. DOI: 10.1109/TIT.1984.1056920. URL: https://doi.org/10.1109/TIT.1984.1056920 (visited on 2026-10-16).

[91] D. Muresan and M. Effros. “Quantization as histogram segmentation: globally optimal scalar quantizer design in network systems”. In: *Proceedings DCC 2002, Data Compression Conference*. La quantification comme segmentation d’un histogramme en intervalles, résolue exactement par programmation dynamique : notre geste sur les 16 codes MXFP4. 2002, pp. 302–311. DOI: 10.1109/dcc.2002.999968. URL: https://doi.org/10.1109/dcc.2002.999968 (visited on 2026-10-16).

[92] X. Wu and K. Zhang. “Quantizer monotonicities and globally optimal scalar quantizer design”. In: *IEEE Transactions on Information Theory* 39.3 (1993-05). Optimum global d’un quantificateur scalaire par programmation dynamique sur une source discrète., pp. 1049–1053. DOI: 10.1109/18.256513. URL: https://doi.org/10.1109/18.256513 (visited on 2026-10-16).

[93] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter. *A Simple and Effective Pruning Approach for Large Language Models*. Version 3. v1 du 2023-06-20, v3 du 2024-05-06. Élagage par |W_ij| · ‖X_j‖ : la sensibilité diagonale par colonne d'entrée. 2023. arXiv: 2306.11695. URL: https://arxiv.org/abs/2306.11695v3 (visited on 2026-10-16).

[94] E. Frantar and D. Alistarh. *SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot*. Version 3. v1 du 2023-01-02, v3 du 2023-03-22. Élagage avec compensation par la hessienne des entrées (OBS en ligne). 2023. arXiv: 2301.00774. URL: https://arxiv.org/abs/2301.00774v3 (visited on 2026-10-16).

[95] P. C. O'Brien and T. R. Fleming. "A Multiple Testing Procedure for Clinical Trials". In: *Biometrics* 35.3 (1979-09). Notice Crossref relue le 2026-10-29 (elle ne donne que la première page) ; article non relu., p. 549. DOI: 10.2307/2530245. URL: https://doi.org/10.2307/2530245 (visited on 2026-10-29).

[96] J. R. Anderson and L. J. Schooler. "Reflections of the Environment in Memory". In: *Psychological Science* 2.6 (1991). [mém.] non relu pour ce travail, pp. 396–408.

[97] K. Hasselmann. "Stochastic climate models. Part I. Theory". In: *Tellus* 28.6 (1976). [mém.] non relu pour ce travail, pp. 473–485.

[98] D. Lee, S. H. Noh, S. L. Min, et al. "LRFU: A Spectrum of Policies that Subsumes the Least Recently Used and Least Frequently Used Policies". In: *IEEE Transactions on Computers* 50.12 (2001). [mém.] non relu pour ce travail, pp. 1352–1361.

[99] L. A. Belady. "A Study of Replacement Algorithms for a Virtual-Storage Computer". In: *IBM Systems Journal* 5.2 (1966), pp. 78–101. DOI: 10.1147/sj.52.0078. (Visited on 2026-10-15).

[100] T. M. Wong and J. Wilkes. "My Cache or Yours? Making Storage More Exclusive". In: *Proceedings of the 2002 USENIX Annual Technical Conference*. Page de l'éditeur relue le 2026-10-29 (titre, auteurs) ; lue par la piste P2 le 2026-10-27. 2002. URL: https://www.usenix.org/conference/2002-usenix-annual-technical-conference/my-cache-or-yours-making-storage-more-exclusive (visited on 2026-10-29).

[101] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. "Evaluation techniques for storage hierarchies". In: *IBM Systems Journal* 9.2 (1970). Les algorithmes à pile (le contenu à la capacité C est inclus dans celui à C' > C) : LRU et MIN en sont ; c'est la propriété qui rend l'étage RAM inclusif de trc-cache égal à un cache unique de sa taille (lemme 2 du lot O)., pp. 78–117. DOI: 10.1147/sj.92.0078. URL: https://doi.org/10.1147/sj.92.0078 (visited on 2026-10-15).

[102] A. Skliar, T. van Rozendaal, R. Lepert, et al. "Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference". In: *Transactions on Machine Learning Research* (2025). arXiv:2412.00099, version 2 du 24 juin 2025. URL: https://arxiv.org/abs/2412.00099 (visited on 2026-10-27).

[103] A. N. Angelopoulos and S. Bates. *A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification*. Version v6 du 2022-12-07 (API arXiv, 2026-10-29) ; résumé lu par la piste P4 le 2026-10-27. 2021-07-15. arXiv: 2107.07511. URL: https://arxiv.org/abs/2107.07511 (visited on 2026-10-29).

[104] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei. *Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control*. Version v5 du 2022-09-29 (API arXiv, 2026-10-29) ; résumé lu par la piste P4 le 2026-10-27. 2021-10-03. arXiv: 2110.01052. URL: https://arxiv.org/abs/2110.01052 (visited on 2026-10-29).

[105] S. Shalunov, G. Hazel, J. Iyengar, and M. Kuehlewind. *Low Extra Delay Background Transport (LEDBAT)*. RFC 6817. Page de la RFC atteinte le 2026-10-29 ; résumé lu par la piste P4 le 2026-10-27. Internet Engineering Task Force, 2012-12. URL: https://www.rfc-editor.org/rfc/rfc6817.html (visited on 2026-10-29).

[106] C. J. Clopper and E. S. Pearson. "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial". In: *Biometrika* 26.4 (1934). Notice Crossref relue le 2026-10-29 ; article non relu., pp. 404–413. DOI: 10.1093/biomet/26.4.404. URL: https://doi.org/10.1093/biomet/26.4.404 (visited on 2026-10-29).

[107] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. "Optimization by Simulated Annealing". In: *Science* 220.4598 (1983-05-13). Notice Crossref relue le 2026-10-29 ; article non relu., pp. 671–680. DOI: 10.1126/science.220.4598.671. URL: https://doi.org/10.1126/science.220.4598.671 (visited on 2026-10-29).

[108] K. Hukushima and K. Nemoto. "Exchange Monte Carlo Method and Application to Spin Glass Simulations". In: *Journal of the Physical Society of Japan* 65.6 (1996-06-15). Notice Crossref relue le 2026-10-29 ; article non relu., pp. 1604–1608. DOI: 10.1143/JPSJ.65.1604. URL: https://doi.org/10.1143/JPSJ.65.1604 (visited on 2026-10-29).

[109] L. Babai. "On Lovász' lattice reduction and the nearest lattice point problem". In: *Combinatorica* 6.1 (1986). Notice Crossref relue le 2026-10-29 ; article non relu., pp. 1–13. DOI: 10.1007/BF02579403. URL: https://doi.org/10.1007/BF02579403 (visited on 2026-10-29).

[110] J. Chen, Y. Shabanzadeh, E. Crnčević, T. Hoefler, and D. Alistarh. *The Geometry of LLM Quantization: GPTQ as Babai's Nearest Plane Algorithm*. Version v4 du 2026-05-13 (API arXiv, 2026-10-29), ICLR 2026 ; page lue par la piste P5 le 2026-10-27. 2025-07-24. arXiv: 2507.18553. URL: https://arxiv.org/abs/2507.18553 (visited on 2026-10-29).

[111] B. Aubin, W. Perkins, and L. Zdeborová. *Storage capacity in symmetric binary perceptrons*. Version v2 du 2019-03-31 (API arXiv, 2026-10-29) ; J. Phys. A 52, 294003 (2019), référence donnée par la piste P5, non relue ; résumé lu par la piste le 2026-10-27. 2019-01-02. arXiv: 1901.00314. URL: https://arxiv.org/abs/1901.00314 (visited on 2026-10-29).

[112] U. Fincke and M. Pohst. "Improved methods for calculating vectors of short length in a lattice, including a complexity analysis". In: *Mathematics of Computation* 44.170 (1985). Notice Crossref relue le 2026-10-29 ; article non relu., pp. 463–471. DOI: 10.1090/S0025-5718-1985-0777278-8. URL: https://doi.org/10.1090/S0025-5718-1985-0777278-8 (visited on 2026-10-29).

[113] P. Tang et al. *HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference*. dernière version 2024-11-06. 2024-11-03. arXiv: 2411.01433. URL: https://arxiv.org/abs/2411.01433 (visited on 2026-10-15).

[114] A. Haidar, S. Tomov, J. Dongarra, and N. J. Higham. "Harnessing GPU Tensor Cores for Fast FP16 Arithmetic to Speed up Mixed-Precision Iterative Refinement Solvers". In: *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. Notice Crossref relue le 2026-10-29 ; article non relu. 2018-11, pp. 603–613. DOI: 10.1109/SC.2018.00050. URL: https://doi.org/10.1109/SC.2018.00050 (visited on 2026-10-29).

[115] L. Zhang, G. Naitzat, and L.-H. Lim. *Tropical Geometry of Deep Neural Networks*. v1 du 2018-05-18 ; résumé lu le 2026-10-17 (05-synthese § 3.1). 2018. arXiv: 1805.07091. URL: https://arxiv.org/abs/1805.07091 (visited on 2026-10-17).

[116] ggml-org. *GGUF file format specification (docs/gguf.md)*. Spécification publique lue par l'étude 06. URL: https://github.com/ggml-org/ggml/blob/master/docs/gguf.md (visited on 2026-10-15).

[117] Y. Leviathan, M. Kalman, and Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML 2023 ; dernière version 2023-05-18. 2022-11-30. arXiv: 2211.17192. URL: https://arxiv.org/abs/2211.17192 (visited on 2026-10-15).

[118] C. H. Martin and M. W. Mahoney. *Implicit Self-Regularization in Deep Neural Networks: Evidence from Random Matrix Theory and Implications for Learning*. v1 du 2018-10-02, seule version. 2018. arXiv: 1810.01075 [cs.LG]. URL: https://arxiv.org/abs/1810.01075 (visited on 2026-10-21).

[119] J. Duda. *Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding*. 2013. arXiv: 1311.2540. URL: https://arxiv.org/abs/1311.2540 (visited on 2026-10-15).

[120] F. Giesen. *Interleaved entropy coders*. 2014. arXiv: 1402.3392. URL: https://arxiv.org/abs/1402.3392 (visited on 2026-10-15).

[121] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Federal Information Processing Standards Publication 180-4. SHA-256, dont le préfixe de 64 bits est l'empreinte64 du format ; la garde de production (M1). 2015-08. DOI: 10.6028/NIST.FIPS.180-4. URL: https://doi.org/10.6028/NIST.FIPS.180-4 (visited on 2026-10-16).

[122] Microsoft. *FSCTL_GET_RETRIEVAL_POINTERS IOCTL (winioctl.h)*. Microsoft Learn. 2018-12-05. URL: https://learn.microsoft.com/en-us/windows/win32/api/winioctl/ni-winioctl-fsctl_get_retrieval_pointers (visited on 2026-10-22).

[123] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0*. Tech. rep. MXFP4 : blocs de 32 éléments E2M1 et une échelle E8M0 ; non reconsulté par le lot F le 2026-10-15. 2023. URL: https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf.

[124] The ggml authors. *llama.cpp, version b11515*. Version b11515, commit 3d65c90d0. MIT. Lus : `tools/perplexity/perplexity.cpp` (format du fichier `--kl-divergence-base`, codage des log-probabilités sur 16 bits, formule de la KL, quantiles), `src/models/openai-moe.cpp`, `src/llama-graph.cpp` (routage `SOFTMAX_WEIGHT`), `ggml/src/ggml.c` (`ggml_rope_yarn_corr_dims` : bornes de la rampe arrondies par `floorf` et `ceilf`), `ggml/src/ggml-cpu/ops.cpp` (`rope_yarn`, `swiglu_oai`), `src/llama-context.cpp` (facteur d'attention de YaRN), `include/llama.h` (API publique). Relus le 2026-10-15 (reprise, par raw.githubusercontent.com à l'étiquette b11515) : `ggml/src/ggml-cpu/ggml-cpu.c` (`type_traits_cpu` : MXFP4 et Q8_0 ont `vec_dot_type` = Q8_0, l'entrée d'un produit quantifié est donc ramenée en Q8_0), `ggml/src/ggml-cpu/arch/x86/quants.c` (`quantize_row_q8_0` AVX2 : d = amax/127 en f16, id = 127/amax, arrondi au plus proche), `ggml/src/ggml-cpu/ops.cpp` (attention éclair du processeur : noyau pavé quand le lot de requêtes est grand, puits appliqués au premier morceau), `ggml/src/ggml.c` (`ggml_rope_yarn_corr_dims`, revérifié). Rien n'est copié. 2026. URL: https://github.com/ggml-org/llama.cpp (visited on 2026-10-15).

[125] S. Merity, C. Xiong, J. Bradbury, and R. Socher. *Pointer Sentinel Mixture Models*. Le corpus WikiText-2 (CC BY-SA 3.0) ; `wiki.test.raw`, empreinte dans les journaux de l'étude 06. 2016-09-26. arXiv: 1609.07843. URL: https://arxiv.org/abs/1609.07843 (visited on 2026-10-15).

[126] A. Rényi. “On the theory of order statistics”. In: *Acta Mathematica Academiae Scientiarum Hungaricae* 4 (1953). Notice bibliographique vérifiée ; article non relu dans ce rôle, pp. 191–231. DOI: 10.1007/BF02127580. URL: https://link.springer.com/doi/10.1007/BF02127580 (visited on 2026-10-22).

[127] M. M. Botvinick, T. S. Braver, D. M. Barch, C. S. Carter, and J. D. Cohen. “Conflict monitoring and cognitive control”. In: *Psychological Review* 108.3 (2001). Notice vérifiée ; l’ordre des auteurs varie selon les fiches (celle de Princeton : Botvinick, Carter, Braver, Barch, Cohen), à confirmer sur la page de l’éditeur ; article non relu, pp. 624–652. URL: https://collaborate.princeton.edu/en/publications/conflict-monitoring-and-cognitive-control-2/ (visited on 2026-10-26).

[128] J. F. Cavanagh, T. V. Wiecki, M. X. Cohen, et al. “Subthalamic nucleus stimulation reverses mediofrontal influence over decision threshold”. In: *Nature Neuroscience* 14.11 (2011). Notice vérifiée ; manuscrit d’auteur PMC3394226 ; article non relu dans ce rôle, pp. 1462–1467. DOI: 10.1038/nn.2925. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC3394226 (visited on 2026-10-26).

[129] C. H. Martin, T. (Peng, and M. W. Mahoney. *Predicting trends in the quality of state-of-the-art neural networks without access to training or testing data*. v2 du 2021-06-02 ; Nature Communications. 2021. DOI: 10.1038/s41467-021-24025-8. arXiv: 2002.06716 [cs.LG]. URL: https://arxiv.org/abs/2002.06716 (visited on 2026-10-21).

[130] V. A. Marchenko and L. A. Pastur. “Distribution of eigenvalues for some sets of random matrices”. In: *Mathematics of the USSR-Sbornik* 1.4 (1967), pp. 457–483. DOI: 10.1070/SM1967v001n04ABEH001994.

[131] A. Clauset, C. R. Shalizi, and M. E. J. Newman. “Power-Law Distributions in Empirical Data”. In: *SIAM Review* 51.4 (2009), pp. 661–703. DOI: 10.1137/070710111. arXiv: 0706.1062. URL: https://arxiv.org/abs/0706.1062.

[132] Q. H. Vuong. “Likelihood Ratio Tests for Model Selection and Non-Nested Hypotheses”. In: *Econometrica* 57.2 (1989), pp. 307–333. DOI: 10.2307/1912557.